Parallel Transport of Tangent Vectors on Triangle Meshes: from the Vector Heat Equation to Code, with Finite Elements

Mauricio Cele Lopez Belon

Abstract. We derive, step by step, a program that spreads a tangent vector given at one point of a triangle mesh to every other point, so that the result at each point is the parallel transport of the given vector. The method is the vector heat method: diffuse the vector by the heat equation for tangent vector fields, whose operator is the connection Laplacian, and keep only the direction of the result. We start from the partial differential equation and go all the way down to the linear system that the code solves: the covariant derivative and the Levi-Civita connection of a surface, Green's identity for tangent fields, the backward Euler step, the weak form, the finite element space of piecewise linear tangent fields, the discrete parallel transport that makes that space possible, the gradients of the shape functions on a triangle embedded in space, the element stiffness and mass matrices, their assembly, the boundary condition, and the solution. Every step is motivated, every formula is derived, and every formula that the code evaluates is matched with the function that evaluates it, so that the text can be read side by side with the implementation [9]. A second part derives the same method in geometric algebra, where the tangent plane is a bivector, coordinates are complex numbers of that plane, parallel transport is a rolling rotor and the connection Laplacian becomes a complex Hermitian matrix, and it clarifies which parts of the computation are intrinsic to the curved surface. A third part removes the last extrinsic ingredient, the tangent plane of a vertex, and builds the whole method from the edge lengths of the mesh alone: the metric of a face is its Gram matrix, the plane of a vertex is the cone of its angles rescaled to a full turn, and the connection matches the bisectors of the corners; the result is exact on bent and folded meshes.

Mathematics Subject Classification (2010). 65N30 Finite element methods; 53A05 Surfaces in Euclidean space; 65F05 Direct methods for linear systems.

Keywords. Finite Element Method, Connection Laplacian, Levi-Civita Connection, Parallel Transport, Vector Heat Method.

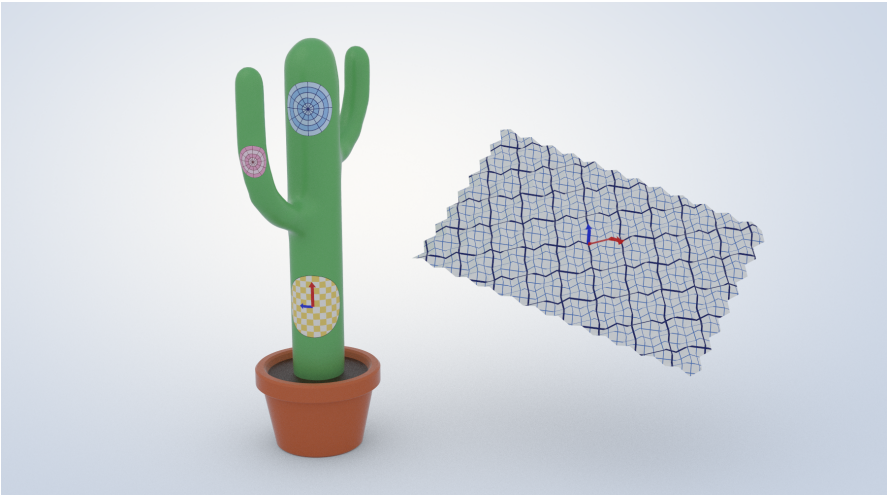


FIGURE 1. Parallel transport at work. Left: three decals pasted on a cactus with the logarithmic map, computed from one vector heat solve and one heat-method distance per decal (Section 36). Right: graph paper on a folded Miura sheet; with the intrinsic connection of Part III the lines run straight across every fold.

Contents

1. Introduction 5

Part I. The Derivation with Matrices 7
2. The Finite Element Method as a Recipe 8
3. Geometric Toolkit I: Tangent Vectors on a Surface 10
4. Geometric Toolkit II: Parallel Transport and the Levi-Civita Connection 16
5. Stage 1. The PDE: the Connection Laplacian and the Vector Heat Equation 32
6. Stage 2. The Function Spaces 42
7. Stage 3. The Variational Form 49
8. Stage 4. The Weak Form (Galerkin) 52
9. Stage 5. The Discretization: the Mesh and Piecewise Linear Tangent Fields 55
10. Stage 6. The Discrete Solution and the Test Functions 64
11. Stage 7. The Element Integrals 69
12. Stage 7, Continued: Shape Functions and Their Gradients on a Triangle in Space 73
13. Stage 7, Completed: The Element Matrices 84
14. Stage 8. Assembly 90
15. Stage 8, Continued: The Linear System and Its Solution 95
16. Summary: From the Equations to the Code 102

Part II. The Same Derivation in Geometric Algebra, and What Is Intrinsic 106
17. The Tools: the Geometric Algebra of Space 107
18. Stage 1: The PDE (Strong Form) in Geometric Algebra 109
19. Stage 2: The Function Spaces 125
20. Stage 3: The Variational Form 129
21. Stage 4: The Weak Form (Galerkin) 130
22. Stage 5: The Discretization: the Mesh and the Finite-Dimensional Space 137
23. Stage 6: The Discrete Solution and the Test Functions 155
24. Stage 7: The Element Integrals 159
25. Stage 8: Assembly and the Hermitian System 167
26. Dictionary 174

Part III. The Intrinsic Method: Everything from Edge Lengths 176
27. What It Means to Be Intrinsic 179
28. Stages 1 to 4 in Gauge Language 182
29. Stage 5, the Faces: Flat Charts Known Only by Their Edge Lengths 204
30. Stage 5, the Vertices: the Stretched Cone as a Flat Chart 214

31. Stage 5, the Gauge Transitions: from a Vertex Chart into a Face Chart 225
32. Stages 6 to 8: the Discrete Solution, the Element Integrals and the Assembly 236
33. Experiments 243
34. Part III in the Matrix Language of Part I 249
35. Summary: Two Ways to Give a Vertex a Plane 253

Applications 255
36. The Logarithmic Map: a Decal, a Folded Sheet and an Atlas 256
36.1. The logarithmic map 256
36.2. The key fact: the angle can be read at the far end 257
36.3. The distance by the heat method: the recipe once more, for scalars 258
36.4. The direction: the vector heat step 261
36.5. The angle at the vertices 262
36.6. A decal on the cactus 263
36.7. Graph paper on the folded sheet: extrinsic against intrinsic 266
References 268

1. Introduction

Suppose we are given one tangent vector at one point of a curved surface, and we want to know what that vector "is" at every other point: the vector we would obtain by carrying it along the surface without turning it. On a plane the answer is trivial, the same vector everywhere. On a curved surface it is not, for two reasons. First, the tangent planes at different points are different planes, so the vector cannot simply be copied; it has to be carried, and the rule that says how is the *parallel transport* of the surface, given by its *Levi-Civita connection*. Second, the result depends on the path: carrying a vector around a closed loop on a sphere brings it back rotated, by an angle equal to the curvature enclosed. A sensible global answer is to carry the vector along the shortest path from the source, the geodesic.

The *vector heat method* of Sharp, Soliman and Crane [11] computes that answer without computing a single geodesic. Its idea is simple. Heat spreading from a point source travels, for a short time, mostly along shortest paths. If what spreads is not a temperature but a tangent vector, which diffuses by the heat equation for tangent vector fields, then the direction of the diffused vector at each point is the direction of the source vector carried along the shortest path; only its length decays with distance. So: diffuse the vector for a short time, and normalize.

This text derives a finite element implementation of that idea [9], following the classical presentations of the finite element method [7, 10] and extending them from scalar functions on flat domains to tangent vector fields on surfaces. The derivation has a fixed structure, which recurs in every finite element method and which we follow strictly:

1. the continuous problem: the operator, the equation, the boundary condition (Sections 3–5);
2. the weak form, which needs only first derivatives (Section 8);
3. a finite dimensional space of candidate solutions, the *trial space*, and of *test functions* (Section 9);
4. the element matrices, computed triangle by triangle (Sections 12–13);
5. the assembly of the global matrices, the boundary condition, and the solution of the linear system (Sections 14–15).

The ingredient that is not in the scalar method is the connection: tangent vectors at different vertices live in different planes and cannot be added before they are brought into a common plane. Section 4 shows how the code does that, and Section 9 shows that this is exactly where the Levi-Civita connection enters the finite element method. Table 3 at the end of Part I lists every formula with the function of `FEM.cpp` that evaluates it. Part II repeats the derivation in the language of geometric algebra, where frames become a single vector, coordinates become complex numbers of the tangent plane, transports become rotors, and the gradients of the shape functions become a reciprocal frame; it also separates what is intrinsic to the curved surface from what depends on its embedding in space. Part III builds the same method

from the edge lengths alone, slowly and with every derivation, and compares the two connections on a folded sheet, a sphere and the cactus. The part *Applications* (Section 36) puts the method to work: geodesic polar coordinates, decals, the folded sheet as graph paper, and an atlas of charts over the whole cactus.

How to read the formulas. The text is meant to be read in pieces, on different days, without having to turn back. Two conventions help. Whenever an earlier equation is used, its number is followed by the equation itself in square brackets, for example "by (3.1) $[\,T^{T}T = I_2\,]$"; the bracket repeats the part that is needed, sometimes in a shortened form. And derivations are written out step by step, in the style of a school textbook: every substitution is shown, every identity that is used is named or proved on the spot, and when an expression is rewritten into a new form the intermediate lines are kept. A reader who already knows a step can skip it; a reader who does not will find it there.

Part I

The Derivation with Matrices

2. The Finite Element Method as a Recipe

The finite element method is a mechanical procedure. It does not depend on the equation: one starts from a partial differential equation (a PDE), applies the same stages in the same order, and at the end holds a linear system whose solution approximates the solution of the PDE. Once the stages are known, they can be applied to a new equation without new ideas, only with new computations. This part applies them to one equation, a single time step of the heat equation for tangent vector fields on a curved surface, and it names every stage as it goes, so that the reader always knows where in the procedure we are and why. Parts II and III run the same procedure again, with the same stage names, in other languages.

Here are the eight stages, each with the object it produces.

1. **The PDE (strong form).** The problem in its usual form: the unknown, the domain, the equation and the side conditions. For us the unknown is a tangent field U on a closed curved surface Ω, the equation is one backward Euler step of the vector heat equation, $U + \Delta t\, \nabla^* \nabla U = U^0$, and the side condition prescribes the vector at one source vertex. *Produces:* the equation, which involves second derivatives of U (Section 5).
2. **The function spaces.** Where the solution is looked for, and why there. *Produces:* the space H^1 of tangent fields with square-integrable covariant derivatives, the guarantee that the problem has exactly one solution there, the guarantee that the discrete solution is the best one available, and the requirement (conformity) that the discrete fields must meet (Section 6).
3. **The variational form.** An energy $J(U)$ whose minimizer is the solution. *Produces:* a second reading of the problem, as a minimization (Section 7).
4. **The weak form (Galerkin).** Multiply the PDE by a test field, integrate, and integrate by parts. *Produces:* the equation $a(U, V) = l(V)$ for every test field V, which contains only first derivatives (Section 8).
5. **The discretization.** The mesh, and a finite dimensional space of fields on it. *Produces:* the vertex and face frames, the discrete connection R_i^f, and $2n$ vector hat functions $\Psi_{i,k}$, two per vertex, which span the discrete space (Sections 3, 4 and 9).
6. **The discrete solution and the test functions.** Write the unknown as a combination of the basis functions, and test with every basis function. *Produces:* $2n$ linear equations in $2n$ unknowns, $(\mathrm{M} + \Delta t\, \mathrm{L})\, \mathrm{u} = \mathrm{M}\, \mathrm{u}^0$ (Section 10).
7. **The element integrals.** Compute the integrals of the matrices face by face, on a reference triangle, with the chain rule and an affine map. *Produces:* a 6×6 element stiffness matrix and a lumped element mass matrix per face (Sections 11, 12 and 13).

TABLE 1. The eight stages of the finite element method in this part, what each one produces, and where the code carries it out.

Stage	Produces	Code
1. PDE (strong form)	$U + \Delta t\, \nabla^* \nabla U = U^0$, source $U(s) = U_s$	the time step, `computeEdgeLengths`
2. Function spaces	H^1 of tangent fields; existence, uniqueness, best approximation, conformity	(none: it is the guarantee)
3. Variational form	the energy $J(U)$	(none)
4. Weak form	$a(U, V) = l(V)$ for all test fields V	(none)
5. Discretization	mesh, frames, R_i^f, basis $\Psi_{i,k}$	`computeNormals`, `computeVertexTangentSpace`, `computeFaceNormals`, `computeFaceTangentSpace`, `alignVectors`, `Rvf`
6. Discrete solution	$(\mathrm{M} + \Delta t\, \mathrm{L})\, \mathrm{u} = \mathrm{M}\, \mathrm{u}^0$	(the structure of the next two stages)
7. Element integrals	L^f, M^f per face	`AssembleStiffnessElement-Embedded2`
8. Assembly and solve	global matrices, source rows, LU, arrows	`AssembleMatrix2`, `AssembleDiagonalMassMatrix`, `IplusMinvTimesA`, `PreFactor`, `display`

8. **Assembly and the linear system.** Add the element matrices into the global ones, impose the source, solve, and read the solution back as vectors. *Produces:* the arrows of Figure 16 (Sections 14 and 15).

Table 1 lists the same stages with the functions of `FEM.cpp` that carry them out. Every stage opens by saying what it wants and why, and closes by saying what it produced and which stage uses it next.

Where the geometry comes in. On a flat plate, with a scalar unknown such as a temperature, this recipe is the classical finite element method of every textbook [7, 10]. Two things are different here, and both are geometric. The unknown is a *tangent vector* at every point, and a curved surface has a different tangent plane at every point, so vectors at different points cannot simply be added or subtracted. And the computer replaces the curved surface by flat triangles, whose planes meet at angles along the edges and at the vertices. The tools that handle both are the covariant derivative and parallel transport, the Levi-Civita connection of the surface. They are built in the two sections that come before Stage 1, a geometric toolkit. Their smooth halves (tangent planes, frames, the covariant derivative) are the language in which the PDE of Stage 1 is written: the operator $\nabla^*\nabla$ is made of covariant derivatives. Their discrete halves (the mesh, its normals and frames, and the 2×2 rotations R_i^f) are the ingredients of Stage 5, prepared early because the

program computes them first. Stage 5 collects them again and explains how they enter the finite element machinery.

Part I · The Derivation with Matrices

3. Geometric Toolkit I: Tangent Vectors on a Surface

Everything in this part is about one kind of object: a vector that lies flat on a surface, an arrow that a small creature living on the surface could draw on the ground. Before we can diffuse such vectors, differentiate them or transport them, we need two things. We need to say precisely what it means for a vector to lie flat on a surface, and we need a way to store such a vector with two numbers instead of three. This section provides both, first for a smooth surface and then for the triangle mesh that the code uses, and it shows the functions of `FEM.cpp` that build the corresponding data. In the recipe of Section 2 this is preparation: the smooth half is the language of Stage 1, and the discrete half, from Section 3.2 on, is the first ingredient of Stage 5.

3.1. Tangent planes and frames

Let $\Omega \subset \mathbb{R}^3$ be a smooth surface, and let $n(x)$ be its unit normal at the point x: the unit vector perpendicular to the surface there. Near x the surface looks, under a strong magnifying glass, like a plane through x perpendicular to $n(x)$. That plane, moved to the origin, is the *tangent plane* $T_x\Omega$:

$$T_x\Omega = \{\, v \in \mathbb{R}^3 : v \cdot n(x) = 0 \,\}.$$

A *tangent vector* at x is a vector of this plane, and a *tangent vector field* U is a rule that assigns to every point x of the surface a tangent vector $U(x) \in T_x\Omega$. The velocity of a curve drawn on the surface is a tangent vector, and so is the direction of the wind on the surface of the Earth; the vertical direction is not.

Note that the tangent plane changes from point to point. On a sphere, the tangent plane at the north pole is horizontal and the tangent plane at a point of the equator is vertical. This simple fact is the source of every difficulty in this text.

Frames. A vector of $\mathbb{R}^3$ needs three numbers, but a tangent vector needs only two, once we choose a basis of the tangent plane. We choose at each point an orthonormal, positively oriented *frame* $(t(x), b(x))$: two unit tangent vectors, orthogonal to each other, with $b = n \times t$. The three vectors (t, b, n) are then an orthonormal basis of $\mathbb{R}^3$, ordered like (e_x, e_y, e_z). We collect the first two as the columns of a 3×2 matrix,

$$T(x) = \big[\, t(x) \;\; b(x) \,\big], \qquad T^T T = I_2, \qquad TT^T = I_3 - nn^T. \tag{3.1}$$

Both identities are matrix products written out. The first multiplies the rows t^T, b^T of T^T by the columns t, b of T:

$$T^T T = \begin{pmatrix} t^T \\ b^T \end{pmatrix} \begin{bmatrix} t & b \end{bmatrix} = \begin{pmatrix} t \cdot t & t \cdot b \\ b \cdot t & b \cdot b \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I_2,$$

because t and b are unit vectors orthogonal to each other. The second multiplies in the other order, a column times a row for each of the two columns:

$$TT^T = \begin{bmatrix} t & b \end{bmatrix} \begin{pmatrix} t^T \\ b^T \end{pmatrix} = t\,t^T + b\,b^T.$$

For any orthonormal basis (t, b, n) of $\mathbb{R}^3$, every vector is the sum of its three components, $w = (t \cdot w)\,t + (b \cdot w)\,b + (n \cdot w)\,n = (t\,t^T + b\,b^T + n\,n^T)\,w$, so $t\,t^T + b\,b^T + n\,n^T = I_3$. Subtracting $n\,n^T$ from both sides gives $t\,t^T + b\,b^T = I_3 - n\,n^T$, which is the second identity.

The matrix $P = TT^T = I_3 - nn^T$ is the *orthogonal projection onto the tangent plane*: applied to any vector v it gives $Pv = v - n\,(n^T v) = v - (n \cdot v)\,n$, which is v with its component along n removed. It will be used constantly, because it is the tool that turns an arbitrary vector of $\mathbb{R}^3$ into a tangent vector by discarding the part that sticks out of the surface. It leaves tangent vectors unchanged: if $n \cdot v = 0$, then $Pv = v - 0 \cdot n = v$.

Coordinates. A tangent vector U at x has the two coordinates

$$u = T^T U = \begin{pmatrix} t \cdot U \\ b \cdot U \end{pmatrix} \in \mathbb{R}^2,$$

and is recovered from them as $U = Tu = u_1\,t + u_2\,b$. The two operations undo each other: $T^T(Tu) = (T^T T)u = I_2 u = u$, and $T(T^T U) = (TT^T)U = PU = U$ for tangent U. Because the frame is orthonormal, they preserve inner products: for two tangent vectors $U = Tu$ and $V = Tv$ at the same point,

$$U \cdot V = (Tu)^T (Tv) = u^T (T^T T)\,v = u^T I_2\,v = u \cdot v.$$

So all the geometry of tangent vectors *at one point* can be done with their coordinates. The difficulty, which the rest of this text addresses, is comparing vectors at *different* points, whose frames are different. The coordinates $(1, 0)$ at one point and $(1, 0)$ at another mean "the first frame vector here" and "the first frame vector there", which may point in completely different directions of space.

The choice of t is arbitrary: any rotation of the frame within the tangent plane is as good. The method does not depend on that choice, because every quantity that compares vectors at different points will carry the change of frames with it (Section 18.4 works this out).

3.2. The discrete surface (an ingredient of Stage 5)

The code does not have a smooth surface. It has a triangle mesh: vertices $x_1, \ldots, x_n$ and triangular faces, each face given by three vertices. For the cactus that `FEM.cpp` loads there are 502 vertices and 1000 faces, and the surface is closed: it has no boundary. To use the ideas above we must decide what the normal, the tangent plane and the frame are at the points of a

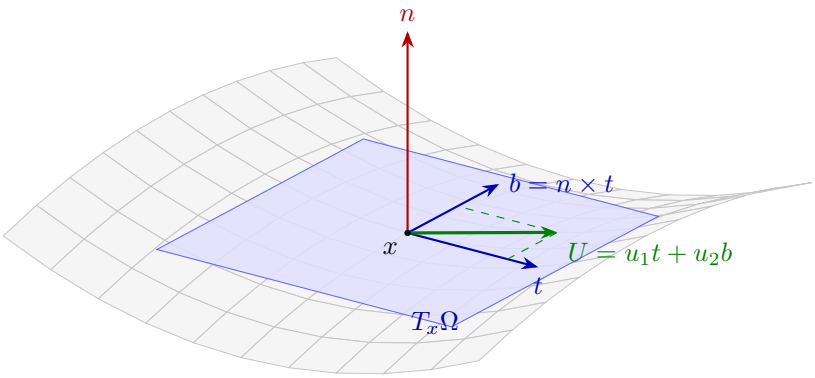


FIGURE 2. A point x of a surface, its normal n, its tangent plane $T_x\Omega$ (blue), an orthonormal frame (t, b) of the tangent plane, and a tangent vector U with its two coordinates u_1, u_2 in that frame.

mesh. There are two kinds of points that matter, the faces and the vertices, and they are treated differently.

Faces. A face is a flat triangle, so it has a well defined plane, and all its points share one tangent plane and one normal. For a face f with vertices $x_{f_1}, x_{f_2}, x_{f_3}$ in counterclockwise order, take the two edge vectors leaving the first vertex, $e_1 = x_{f_2} - x_{f_1}$ and $e_2 = x_{f_3} - x_{f_1}$. Their cross product $e_1 \times e_2$ is orthogonal to both edges, hence to the face, and points to the side from which the three vertices are seen counterclockwise. Its length is the area of the parallelogram spanned by the two edges, which is twice the area of the triangle. So the unit normal and the area of the face are

$$n_f = \frac{(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})}{|(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})|}, \qquad A_f = \tfrac{1}{2}|(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})|. \quad (3.2)$$

This is `computeFaceNormals`:

```
193 void computeFaceNormals(Mesh *mesh, vector<Eigen::Vector3d>& face_normals, vector<
        double>& face_areas) {
194     Vertex* v[3];
195     for(Face &face : mesh->getFaces()) {
196         v[0] = face.edge->vertex;
197         v[1] = face.edge->next->vertex;
198         v[2] = face.edge->next->next->vertex;
199         Eigen::Vector3d uu = v[1]->p - v[0]->p;
200         Eigen::Vector3d vv = v[2]->p - v[0]->p;
201         Eigen::Vector3d n = uu.cross(vv);
202         face_areas[face.ID] = 0.5 * n.norm();
203         face_normals[face.ID] = n.normalized();
204     }
205 }
```

The mesh is stored as a *half-edge structure*: every edge is split into two half-edges, one for each direction, and each half-edge knows the vertex where it starts (`vertex`), the next half-edge around its face (`next`), the half-edge

of opposite direction (`pair`) and its face (`face`). A face stores one of its half-edges, `face.edge`. Lines 196–198 walk once around the face: the start of `face.edge` is x_{f_1}, the start of the next half-edge is x_{f_2}, and the start of the one after is x_{f_3}. Lines 199–200 are the edge vectors e_1 (`uu`) and e_2 (`vv`); line 201 is the unnormalized normal $e_1 \times e_2$; line 202 stores A_f, half its length; and line 203 stores n_f, the same vector normalized. The arrays are indexed by `face.ID`, the number of the face, counted from zero.

Vertices. A vertex has no tangent plane of its own: it is a corner where several faces, with several different planes, meet. Yet the unknowns of the method will live at the vertices, so we must choose a plane there. The code takes the plane orthogonal to a *vertex normal*, an average of the normals of the faces around the vertex. The average is taken with the unnormalized face normals $e_1 \times e_2$, and since $e_1 \times e_2 = 2A_f\, n_f$ by (3.2) $\left[\, n_f = \frac{e_1 \times e_2}{|e_1 \times e_2|},\ A_f = \frac{1}{2}|e_1 \times e_2| \,\right]$, it is an average of the unit face normals weighted by the areas of the faces:

$$n_i = \frac{\sum_{f \ni i} 2A_f\, n_f}{\left|\sum_{f \ni i} 2A_f\, n_f\right|}. \tag{3.3}$$

The sum runs over the faces f that contain the vertex i, the *one-ring* of the vertex. Large faces pull the vertex normal more than small ones, which is reasonable: a large face represents a larger piece of the surface near the vertex. The function is `Mesh::computeNormals`, in `HalfEdge/Mesh.cpp`:

```
113 void Mesh::computeNormals()
114 {
115     map< int, Eigen::Vector3d >     normals;
116 
117     for (Face& face : faces) {
118         Eigen::Vector3d  u = face.edge->next->vertex->p - face.edge->vertex->p;
119         Eigen::Vector3d  v = face.edge->next->next->vertex->p - face.edge->vertex->p;
120         Eigen::Vector3d  n = u.cross(v);
121         normals.insert( pair<int, Eigen::Vector3d>(face.ID, n) );
122     }
123 
124     for (Vertex& vertex : vertices) {
125         int  valence = 0;
126         vertex.n = Eigen::Vector3d(0,0,0);
127 
128         for( Vertex::EdgeAroundIterator around = vertex.iterator(); !around.end();
                 around++ )
129         {
130             if(around.edge_out()->face)
131             {
132                 int  faceId = around.edge_out()->face->ID;
133                 vertex.n += normals[faceId];
134                 valence++;
135             }
136         }
137 
138         vertex.n /= (double)valence;
139         vertex.n.normalize();
140     }
141 }
```

The first loop (lines 117–122) computes $e_1 \times e_2 = 2A_f\, n_f$ for every face, exactly as lines 199–201 of `computeFaceNormals`, and keeps it unnormalized. The second loop visits every vertex. The iterator of line 128 turns around

the vertex, visiting its outgoing half-edges one after the other; each of them belongs to one face of the one-ring (line 132), whose vector is added to the sum (line 133). The test of line 130 skips the half-edges that have no face, which occur only on the boundary of an open mesh. Line 138 divides the sum by the number of faces and line 139 normalizes it; the division does not change the direction and is erased by the normalization, so the result is (3.3) $[\, n_i \propto \sum_{f \ni i} 2A_f\, n_f \,]$. The function `main` calls it right after loading the mesh.

Frames. Each vertex and each face gets an orthonormal frame.

- *Vertex* i (`computeVertexTangentSpace`). We need a unit vector t_i in the tangent plane of the vertex. A natural candidate is the direction of an edge from x_i to one of its neighbors x_j, but the edge vector $v = x_j - x_i$ is in general not orthogonal to n_i: the neighbor lies a little above or below the vertex plane. So we remove the normal component of v with the projection $P = I - n_i n_i^T$, which gives $Pv = v - (v \cdot n_i)\, n_i$, and normalize the result. The second frame vector is $b_i = n_i \times t_i$:

$$t_i = \frac{(x_j - x_i) - \big((x_j - x_i) \cdot n_i\big)\, n_i}{\big|(x_j - x_i) - \big((x_j - x_i) \cdot n_i\big)\, n_i\big|}, \qquad T_i = \big[\, t_i \;\; n_i \times t_i \,\big]. \tag{3.4}$$

- *Face* f (`computeFaceTangentSpace`). The first edge $x_{f_2} - x_{f_1}$ already lies in the plane of the face, so it only needs to be normalized: $t_f = (x_{f_2} - x_{f_1})/|x_{f_2} - x_{f_1}|$, and $T_f = [\, t_f \;\; n_f \times t_f \,]$.

Here is the code for the vertices, together with the helper `project` that it uses:

```
170 /// Project u on the orthgonal of n
171 /// \param u vector to project
172 /// \param n vector to build orthogonal space from
173 /// \return projected vector
174 static Eigen::Vector3d project(const Eigen::Vector3d & u, const Eigen::Vector3d & n)
175 {
176     return u - (u.dot(n) / n.squaredNorm()) * n;
177 }
178
179 void computeVertexTangentSpace(Mesh *mesh, vector<Matrix32d>& vertex_tangent_space) {
180     Matrix32d tangentSpace;
181     for(Vertex &vi : mesh->getVertices()) {
182         Eigen::Vector3d& ni = vi.n;
183         Eigen::Vector3d& pi = vi.p;
184         Eigen::Vector3d& pj = vi.edge->pair->vertex->p;
185         Eigen::Vector3d tangent = project(pj - pi, ni).normalized();
186         Eigen::Vector3d bitangent = ni.cross(tangent);
187         tangentSpace.col(0) = tangent;
188         tangentSpace.col(1) = bitangent;
189         vertex_tangent_space[vi.ID] = tangentSpace;
190     }
191 }
```

Line 176 is the projection $Pv = v - (v \cdot n)\, n$, written as $v - \frac{v \cdot n}{n \cdot n}\, n$ so that it also works for a normal that is not of unit length; for our unit n_i the two are the same. In `computeVertexTangentSpace`, `vi.edge` is a half-edge leaving the vertex i; its pair runs the other way and therefore starts at the neighbor, so `vi.edge->pair->vertex->p` (line 184) is x_j. Line 185 is the formula for t_i

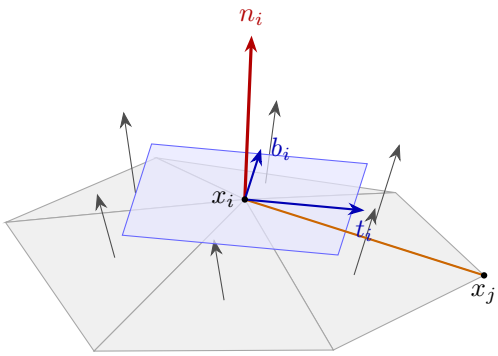


FIGURE 3. The one-ring of a vertex x_i. Each face has its own normal (gray arrows); the vertex normal n_i (red) is their area-weighted average (3.3). The vertex frame vector t_i is the edge to a neighbor x_j (orange), projected onto the vertex tangent plane (blue) and normalized, and $b_i = n_i \times t_i$.

in (3.4): project $x_j - x_i$ and normalize. Line 186 is $b_i = n_i \times t_i$, and lines 187–189 store the matrix T_i, column by column, at the index `vi.ID` of the vertex. The neighbor x_j is whichever one the half-edge structure happens to store; that is the arbitrary choice of frame mentioned above, and it does not matter. The faces are handled in the same way, without the projection:

```
207 void computeFaceTangentSpace(Mesh *mesh, const vector<Eigen::Vector3d>& face_normals,
        vector<Matrix32d>& face_tangent_space) {
208     Matrix32d tangentSpace;
209     Vertex* v[2];
210     for(Face &face : mesh->getFaces()) {
211         v[0] = face.edge->vertex;
212         v[1] = face.edge->next->vertex;
213         Eigen::Vector3d uu = (v[1]->p - v[0]->p).normalized();
214         tangentSpace.col(0) = uu;
215         tangentSpace.col(1) = face_normals[face.ID].cross(uu);
216         face_tangent_space[face.ID] = tangentSpace;
217     }
218 }
```

Lines 211–213 take the first edge of the face, from x_{f_1} to x_{f_2}, and normalize it to t_f; line 215 is $b_f = n_f \times t_f$, with the face normal computed before by `computeFaceNormals`. In `main` the three functions are called in the order vertex frames, face normals, face frames (lines 609–611 of `FEM.cpp`), since the face frames need the face normals. For a small example, let the vertex be $x_i = (0, 0, 0)$ with normal $n_i = (0, 0, 1)$, and let the stored neighbor be $x_j = (0, 2, 1)$. The edge vector $v = (0, 2, 1)$ has the normal component $v \cdot n_i = 1$, so the projection is $Pv = (0, 2, 1) - 1 \cdot (0, 0, 1) = (0, 2, 0)$, and normalizing it gives $t_i = (0, 1, 0)$. Then $b_i = n_i \times t_i = (0, 0, 1) \times (0, 1, 0) = (-1, 0, 0)$. These are the vertex frame vectors used in Example 4.4.

Unknowns. The unknown field is represented by one tangent vector per vertex, stored by its two coordinates in the vertex frame: $u_i = (u_{i,1}, u_{i,2}) \in \mathbb{R}^2$,

with the vector $T_i u_i = u_{i,1} t_i + u_{i,2} b_i \in \mathbb{R}^3$. All the unknowns form one vector of $2n$ numbers, the two coordinates of vertex 1, then the two of vertex 2, and so on:

$$\mathrm{u} = (u_{1,1}, u_{1,2}, u_{2,1}, u_{2,2}, \ldots, u_{n,1}, u_{n,2}). \tag{3.5}$$

These $2n$ numbers are the unknowns of the linear system of Stages 6 and 8. Vertex i is preceded by $i-1$ vertices, which use $2(i-1)$ entries, so its coordinate $k \in \{1, 2\}$ is the entry $2(i-1)+k$. The code counts vertices and coordinates from zero, $i, k \in \{0, 1, \ldots\}$, and the same count gives the entry $2i + k$. This is the expression `v->ID * 2 + row` that will appear in every function that touches the global vectors and matrices: `ID` is the zero-based number of the vertex and `row` is k.

The complex reading. It is often convenient to read u_i as the complex number $u_{i,1} + \mathrm{i}\, u_{i,2}$, so that a rotation of the tangent plane by an angle α is a multiplication by $e^{\mathrm{i}\alpha}$. The code uses this reading to set the source vector and to draw the solution (Section 15), and Part II builds the whole method on it.

Part I · The Derivation with Matrices

4. Geometric Toolkit II: Parallel Transport and the Levi-Civita Connection

The heat equation of Section 5 will need the derivative of a tangent field: how fast it changes as we move along the surface. For functions on the plane, derivatives are familiar from one-variable calculus once we know how to move in a direction. For tangent fields on a curved surface there is a new difficulty: the vectors at two nearby points live in two different planes, and part of their difference is caused by the surface and not by the field. This section takes these two steps in order. First we define the derivative of a function along a surface, and its gradient. Then we define the derivative of a tangent field, the *covariant derivative*, which removes the part caused by the surface, and from it the notion of a field that does not turn, which is *parallel transport*. Finally we build the discrete version that the code uses: a 2×2 rotation for every pair of a vertex and a face. In the recipe, the first three subsections give Stage 1 its operator, since the Laplacian of tangent fields is built from covariant derivatives; the last one, the discrete connection, is the second ingredient of Stage 5, the one that makes vector-valued basis functions possible on a curved mesh.

4.1. Derivatives of functions on a surface

The directional derivative in the plane. Start with a function $f(x_1, x_2)$ on the plane and a point x. To ask how fast f changes at x we must say in which direction we move, because f may grow to the east and decrease to the north. So take a vector X and move along the straight line $x + sX$, where

s is a number. Along this line f becomes a function of the single variable s, and we can differentiate it as in one-variable calculus:

$$X(f) = \frac{d}{ds} f(x + sX)\Big|_{s=0}. \tag{4.1}$$

This number is the *directional derivative* of f along X at x. If X has unit length, it is the slope of f in that direction: the change of f per unit of distance travelled. For example, for $f(x_1, x_2) = x_1^2 + 3x_2$, the point $x = (1, 2)$ and the direction $X = (1, 1)$, the function along the line is $f(1+s, 2+s) = (1+s)^2 + 6 + 3s$, whose derivative at $s = 0$ is $2 + 3 = 5$. The two *partial derivatives* $\partial f/\partial x_1$ and $\partial f/\partial x_2$ are the special cases $X = (1, 0)$ and $X = (0, 1)$: move along one axis and keep the other coordinate fixed.

The gradient. The directional derivative depends on X in a simple way: it is linear in X. The reason is that a differentiable function looks, under a magnifying glass, like a linear one: near x,

$$f(x + h) \approx f(x) + a_1 h_1 + a_2 h_2 = f(x) + a \cdot h$$

for some vector a, with an error that becomes negligible compared with $|h|$ as h shrinks. Then $f(x + sX) \approx f(x) + s\,(a \cdot X)$, a linear function of s with slope $a \cdot X$, so $X(f) = a \cdot X$. Taking $X = (1, 0)$ and $X = (0, 1)$ shows that the entries of a are the two partial derivatives. This vector is the *gradient* of f:

$$\nabla f = \Big(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}\Big), \qquad X(f) = \nabla f \cdot X \quad \text{for every direction } X. \tag{4.2}$$

In the example, $\nabla f = (2x_1, 3) = (2, 3)$ at $x = (1, 2)$, and $\nabla f \cdot X = (2, 3) \cdot (1, 1) = 5$, as computed directly. We will write $X \cdot \nabla f$ and $X(f)$ interchangeably.

The gradient has a clear geometric meaning, which we will use in Section 12. Since $X(f) = |\nabla f|\,|X| \cos \angle(\nabla f, X)$, among all unit directions the derivative is largest along ∇f, where it equals $|\nabla f|$, and it is zero along the directions orthogonal to ∇f. So *the gradient points in the direction of steepest increase, its length is the slope in that direction, and it is orthogonal to the level lines*, the curves along which f is constant.

On a surface. On a surface we cannot move along a straight line, because the line leaves the surface. Instead we move along a curve $x(s)$ drawn on the surface, with $x(0) = x$ and velocity $x'(0) = X$, which is a tangent vector. The directional derivative is defined as before, by differentiating along the curve:

$$X(f) = \frac{d}{ds} f(x(s))\Big|_{s=0}.$$

To first order the curve is $x + sX$, so the result depends only on the velocity X and not on the rest of the curve, and the same magnifying-glass argument applies: near x, $f(y) \approx f(x) + a \cdot (y - x)$ for the points y of the surface, with some vector $a \in \mathbb{R}^3$, and $X(f) = a \cdot X$. Only the tangent part of a matters here, because X is tangent: $a \cdot X = (Pa) \cdot X$ with the projection $P = I - nn^T$

of (3.1) [$TT^T = I_3 - nn^T$]. So we define the *gradient of f on the surface* as the tangent vector

$$\nabla f \in T_x\Omega, \qquad X(f) = \nabla f \cdot X \quad \text{for every tangent vector } X.$$

There is only one such tangent vector: two candidates g and g' would satisfy $(g - g') \cdot X = 0$ for every tangent X, in particular for $X = g - g'$, which forces $g = g'$. In a frame, the coordinates of the gradient are the derivatives along the two frame vectors, $T^T\nabla f = (t(f), b(f))$, because $t \cdot \nabla f = t(f)$ and $b \cdot \nabla f = b(f)$. The case that matters most for us is a function that is linear on a flat triangle, $f(y) = f(x) + g \cdot (y - x)$ with g in the plane of the triangle: then there is no approximation at all, and $\nabla f = g$ is constant on the triangle. The hat functions of Section 9 are of this kind.

Vectors and products. The same definition applies to a function with values in $\mathbb{R}^3$, such as a vector field U on the surface: $D_XU = \frac{d}{ds}U(x(s))\big|_{s=0}$ is the vector of the directional derivatives of the three components of U. Each component has its own gradient, and D_XU is the 3×3 matrix whose rows are those three gradients, applied to X; in particular it is linear in X. Since all these derivatives are ordinary derivatives in the single variable s, the product rule of one-variable calculus holds for them: for functions f, g and vector fields U, V,

$$X(fg) = X(f)\,g + f\,X(g), \qquad D_X(f\,V) = X(f)\,V + f\,D_XV,$$
$$X(U \cdot V) = D_XU \cdot V + U \cdot D_XV.$$

For a pair of functions $u = (u_1, u_2)$, such as the two coordinates of a field, we write $X \cdot \nabla u = (X(u_1), X(u_2))$.

4.2. The covariant derivative

To differentiate a tangent field we would like to compare $U(x + \epsilon X)$ with $U(x)$, but they lie in different tangent planes, and their difference is not a tangent vector. The derivative D_XU just defined is a perfectly good vector of $\mathbb{R}^3$, but it mixes two effects. An example shows them. On the unit sphere, walk along the equator, $x(s) = (\cos s, \sin s, 0)$, holding an arrow that always points straight ahead, $U(s) = x'(s) = (-\sin s, \cos s, 0)$. A walker on the sphere would say that the arrow never turns: it points forward all the time. Yet in space it does change,

$$D_{x'}U = \frac{d}{ds}(-\sin s, \cos s, 0) = (-\cos s, -\sin s, 0) = -x(s),$$

and this derivative points to the center of the sphere: it is along the normal $n = x$. It is caused entirely by the sphere curving away under the walker, not by the arrow turning on the surface. A derivative that measures only how a field turns *within* the surface must therefore discard the normal part.

That is easy to do with the projection $P = I - nn^T$ of (3.1) [$TT^T = I_3 - nn^T$]. Every vector of $\mathbb{R}^3$ splits into a tangent part and a normal part:

$$D_XU = \underbrace{(I - nn^T)\,D_XU}_{\text{tangent}} + \underbrace{(n \cdot D_XU)\,n}_{\text{normal}}.$$

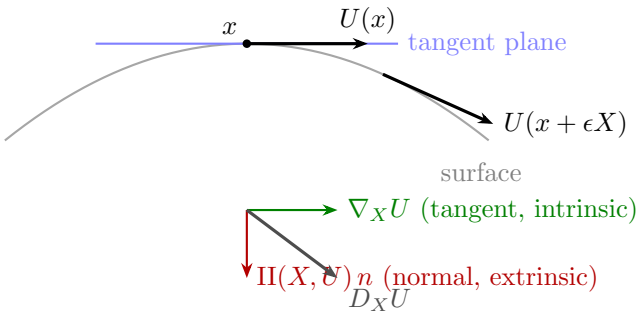


FIGURE 4. The covariant derivative. U at two nearby points lies in two different tangent planes; its ambient derivative $D_X U$ has a tangent part $\nabla_X U$, the covariant derivative, and a normal part, which depends only on how the surface bends (Section 22.5).

The tangent part measures how U turns within the surface; the normal part measures only how the surface itself bends under U (Section 22.5 makes this precise). The *covariant derivative* keeps the tangent part:

$$\nabla_X U = \left(I - nn^T\right) D_X U = TT^T D_X U, \tag{4.3}$$

the two forms being equal by $TT^T = I - nn^T$, (3.1) [$T = [\,t \;\; b\,]$, $T^T T = I_2$, $TT^T = I_3 - nn^T$]. For the walker on the equator, $\nabla_{x'} U = 0$: the forward arrow does not turn, as the walker says. Figure 4 shows the split in a section of the surface.

This is the *Levi-Civita connection* of a surface in $\mathbb{R}^3$ [1]. The name "connection" says what it does: it connects the tangent planes at nearby points, by telling us which change of a vector counts as a real change. Two of its properties will be used repeatedly.

It is compatible with the metric. We want a product rule for the inner product of two tangent fields that uses only covariant derivatives, because the energy of Section 5 is built from inner products. Let U, V be tangent fields and X a tangent direction. The ordinary product rule gives $X(U \cdot V) = D_X U \cdot V + U \cdot D_X V$. Now split $D_X U$ into $\nabla_X U$ plus its normal part $(n \cdot D_X U)\, n$. The normal part has no inner product with V, because V is tangent:

$$\big((n \cdot D_X U)\, n\big) \cdot V = (n \cdot D_X U)\,(n \cdot V) = 0.$$

So $D_X U \cdot V = \nabla_X U \cdot V$, and in the same way $U \cdot D_X V = U \cdot \nabla_X V$. Substituting both,

$$X(U \cdot V) = \nabla_X U \cdot V + U \cdot \nabla_X V. \tag{4.4}$$

In words: the change of an inner product is completely explained by the covariant derivatives of the two fields. Throwing away the normal parts loses nothing that an inner product of tangent vectors could see.

It is linear in X. $D_X U$ is a matrix applied to X, and $\nabla_X U$ is that, projected. So at each point the covariant derivative defines a linear map $\nabla U : X \mapsto \nabla_X U$ from the tangent plane to itself, the *total covariant derivative*. In a frame it is a 2×2 matrix whose column k holds the coordinates of $\nabla_{e_k} U$ for the frame vector e_k, exactly as the columns of any matrix are the images of the basis vectors.

In a frame. The code does not store vectors of $\mathbb{R}^3$; it stores their two coordinates in a frame. So we want the covariant derivative in coordinates: not the vector $\nabla_X U \in \mathbb{R}^3$, but its two coordinates $T^T \nabla_X U \in \mathbb{R}^2$. Write $U = Tu$ with a frame field T and the coordinates $u = (u_1, u_2)$, two functions on the surface. We expect two contributions: the change of the coordinates, and the turning of the frame in which they are measured. The computation confirms it in three steps.

Step 1: differentiate in space, one term at a time. Written out, the field is

$$U = Tu = \begin{bmatrix} t & b \end{bmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = u_1\, t + u_2\, b,$$

a sum of two products, each of a function (u_1 or u_2) and a vector field (t or b). We differentiate the sum term by term, apply the product rule $D_X(f\,V) = X(f)\,V + f\,D_X V$ to each product, and regroup:

$$\begin{aligned} D_X U &= D_X(u_1\, t) + D_X(u_2\, b) && \text{(derivative of a sum)} \\ &= \big(X(u_1)\, t + u_1\, D_X t\big) + \big(X(u_2)\, b + u_2\, D_X b\big) && \text{(product rule, twice)} \\ &= \big(X(u_1)\, t + X(u_2)\, b\big) + \big(u_1\, D_X t + u_2\, D_X b\big) && \text{(regroup).} \end{aligned}$$

Each group is now a 3×2 matrix applied to a vector of $\mathbb{R}^2$, read column by column: a matrix $[\, c_1 \;\; c_2 \,]$ applied to (α, β) is $\alpha\, c_1 + \beta\, c_2$. In the first group the columns are t, b and the numbers are $X(u_1), X(u_2)$; in the second the columns are $D_X t, D_X b$ and the numbers are u_1, u_2:

$$X(u_1)\, t + X(u_2)\, b = \begin{bmatrix} t & b \end{bmatrix} \begin{pmatrix} X(u_1) \\ X(u_2) \end{pmatrix} = T\,(X \cdot \nabla u),$$

$$u_1\, D_X t + u_2\, D_X b = \begin{bmatrix} D_X t & D_X b \end{bmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = (D_X T)\, u,$$

where $X \cdot \nabla u = (X(u_1), X(u_2))$ is the derivative of the two coordinate functions along X, and $D_X T = [\, D_X t \;\; D_X b \,]$ is the 3×2 matrix of the derivatives of the two frame vectors. Substituting the two groups,

$$D_X U = T\,(X \cdot \nabla u) + (D_X T)\, u.$$

The first term is tangent (it is a combination of t and b); the second is in general not, because the frame vectors turn out of the tangent plane as the surface bends.

Step 2: keep the tangent part. The covariant derivative is (4.3) [$\nabla_X U = TT^T D_X U$]. Into it we substitute Step 1 and use $T^T T = I_2$ from (3.1) [$T =$

$[\,t\ \ b\,],\ T^TT = I_2,\ TT^T = I_3 - nn^T\,]$:

$$\begin{aligned}
\nabla_X U &= TT^T\big(T\,(X\cdot\nabla u) + (D_X T)\,u\big) && \text{(substitute Step 1)}\\
&= TT^TT\,(X\cdot\nabla u) + TT^T(D_X T)\,u && \text{(distribute)}\\
&= T\,(T^TT)\,(X\cdot\nabla u) + T\,(T^TD_X T)\,u && \text{(regroup the products)}\\
&= T\,I_2\,(X\cdot\nabla u) + T\,(T^TD_X T)\,u && (T^TT = I_2)\\
&= T\,(X\cdot\nabla u) + T\,(T^TD_X T)\,u.
\end{aligned}$$

Step 3: read the coordinates. Multiply the result of Step 2 on the left by T^T and use $T^TT = I_2$ once more:

$$\begin{aligned}
T^T\nabla_X U &= T^T\big(T\,(X\cdot\nabla u) + T\,(T^TD_X T)\,u\big) && \text{(Step 2)}\\
&= (T^TT)\,(X\cdot\nabla u) + (T^TT)\,(T^TD_X T)\,u && \text{(distribute, regroup)}\\
&= X\cdot\nabla u + (T^TD_X T)\,u && (T^TT = I_2).
\end{aligned}$$

We give the 2×2 matrix in the last term a name, and obtain

$$T^T\nabla_X U = X\cdot\nabla u + \omega(X)\,u, \qquad \omega(X) = T^TD_X T. \tag{4.5}$$

Read column by column, the last term is $\omega(X)\,u = u_1\,T^TD_X t + u_2\,T^TD_X b$: the first column of $\omega(X)$ is the coordinate vector $T^TD_X t = (t\cdot D_X t,\ b\cdot D_X t)$ of the derivative of t, and the second is the coordinate vector $T^TD_X b = (t\cdot D_X b,\ b\cdot D_X b)$ of the derivative of b.

The two expected contributions are there: $X\cdot\nabla u$, the change of the coordinates, and $\omega(X)\,u$, the correction for the turning of the frame. There is a shortcut to the same result: $T^T\nabla_X U = T^T(TT^T)D_X U = (T^TT)\,T^TD_X U = T^TD_X U$, after which one substitutes Step 1 and multiplies by T^T directly. Geometrically, T^T takes inner products with t and b, and those already ignore any normal component, since $t\cdot n = b\cdot n = 0$. So *reading coordinates in the frame already performs the projection*, which is why the projection never appears explicitly in the code.

Why $T^TD_X T$ and not $TT^TD_X T$. Both matrices appear, with different roles. Keep track of the sizes: T is 3×2, $D_X T$ is 3×2, T^T is 2×3.

matrix	size	meaning
$D_X T$	3×2	derivatives of t and b in space: tangent and normal parts
$TT^TD_X T = \nabla_X T$	3×2	their tangent parts: how the frame turns *within* the surface
$T^TD_X T = \omega(X)$	2×2	the same turning, in the coordinates of the frame

The covariant derivative of the frame itself is $\nabla_X T = TT^TD_X T = T\,\omega(X)$: a pair of tangent vectors of $\mathbb{R}^3$, whose coordinates in the frame are the columns of $\omega(X)$. Equation (4.5) $[\,T^T\nabla_X U = X\cdot\nabla u + \omega(X)\,u,\ \omega(X) = T^TD_X T\,]$ is an equation between coordinates, vectors of $\mathbb{R}^2$; its term $\omega(X)\,u$ must be a 2×2 matrix applied to $u\in\mathbb{R}^2$, so that it can be added to $X\cdot\nabla u\in\mathbb{R}^2$. The 3×2 matrix $TT^TD_X T$ cannot be added to $X\cdot\nabla u$; it belongs to the vector form of the same equation, $\nabla_X U = T\,(X\cdot\nabla u) + (\nabla_X T)\,u$. What is discarded in passing from $D_X T$ to $\nabla_X T$ is its normal part $nn^TD_X T$, whose

entries $n \cdot D_X t$ and $n \cdot D_X b$ are the second fundamental form of Section 22.5: how the surface bends, which the covariant derivative must not see.

The entries of ω. Multiply the rows t^T, b^T of T^T by the columns $D_X t, D_X b$ of $D_X T$: the entry (k,l) is the inner product of frame vector k with the derivative of frame vector l,

$$\omega(X) = T^T D_X T = \begin{pmatrix} t^T \\ b^T \end{pmatrix} \begin{bmatrix} D_X t & D_X b \end{bmatrix} = \begin{pmatrix} t \cdot D_X t & t \cdot D_X b \\ b \cdot D_X t & b \cdot D_X b \end{pmatrix}.$$

These four numbers are not independent, because the frame stays orthonormal as it moves: $t \cdot t = 1$, $b \cdot b = 1$ and $t \cdot b = 0$ at every point. A constant function has derivative zero, so we differentiate each of these three relations along X with the product rule $X(U \cdot V) = D_X U \cdot V + U \cdot D_X V$.

- From $t \cdot t = 1$:

$$0 = X(1) = X(t \cdot t) = D_X t \cdot t + t \cdot D_X t = 2\, t \cdot D_X t, \qquad \text{so} \qquad t \cdot D_X t = 0.$$

- From $b \cdot b = 1$, in the same way: $0 = X(b \cdot b) = D_X b \cdot b + b \cdot D_X b = 2\, b \cdot D_X b$, so $b \cdot D_X b = 0$.
- From $t \cdot b = 0$:

$$0 = X(0) = X(t \cdot b) = D_X t \cdot b + t \cdot D_X b, \qquad \text{so} \qquad t \cdot D_X b = -\, b \cdot D_X t.$$

So the diagonal vanishes and the two off-diagonal entries are opposite. Call $\theta(X) = b \cdot D_X t$ the entry $(2,1)$; then the entry $(1,2)$ is $t \cdot D_X b = -\theta(X)$, and substituting the four entries,

$$\omega(X) = \begin{pmatrix} 0 & -\theta(X) \\ \theta(X) & 0 \end{pmatrix} = \theta(X)\, \mathcal{J}, \qquad \mathcal{J} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

The matrix $\mathcal{J}$ is the rotation of the plane by $90°$, $\mathcal{J}(u_1, u_2) = (-u_2, u_1)$, so $\omega(X)\, u = \theta(X)\, (-u_2, u_1)$. And $\omega(X)$ is an *infinitesimal rotation* by the angle $\theta(X)$: the rotation by a small angle α is

$$\begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \alpha \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} + O(\alpha^2) = I_2 + \alpha\, \mathcal{J} + O(\alpha^2),$$

using $\cos\alpha = 1 + O(\alpha^2)$ and $\sin\alpha = \alpha + O(\alpha^3)$, and its first-order part is $\alpha\mathcal{J}$. The matrix ω is the *connection form* of the frame [8]. The single number $\theta(X)$ is the rate at which t turns towards b as one moves along X; it is the $\omega(a)$ of Part II, (18.8) $[\, \nabla_a t = \omega(a)\, t \mathsf{T}\,]$.

Worked example 4.1. *The plane with the polar frame $t = e_r = (\cos\varphi, \sin\varphi, 0)$, $b = e_\varphi = (-\sin\varphi, \cos\varphi, 0)$, $n = e_z$, where φ is the polar angle of the point. Moving along X changes the polar angle at the rate*

$X \cdot \nabla\varphi$, and by the chain rule of one variable

$$\begin{aligned} D_X e_r &= (X \cdot \nabla\varphi)\, \frac{d}{d\varphi}(\cos\varphi, \sin\varphi, 0) \\ &= (X \cdot \nabla\varphi)\, (-\sin\varphi, \cos\varphi, 0) = (X \cdot \nabla\varphi)\, e_\varphi, \\ D_X e_\varphi &= (X \cdot \nabla\varphi)\, \frac{d}{d\varphi}(-\sin\varphi, \cos\varphi, 0) \\ &= (X \cdot \nabla\varphi)\, (-\cos\varphi, -\sin\varphi, 0) = -(X \cdot \nabla\varphi)\, e_r. \end{aligned}$$

So $\theta(X) = e_\varphi \cdot D_X e_r = (X \cdot \nabla\varphi)\,(e_\varphi \cdot e_\varphi) = X \cdot \nabla\varphi$: the frame turns exactly by the polar angle, $\omega(X) = (X \cdot \nabla\varphi)\,\mathcal{J}$. The constant field $U = e_x = (1, 0, 0)$ has the coordinates

$$u = (e_r \cdot e_x,\ e_\varphi \cdot e_x) = (\cos\varphi,\ -\sin\varphi),$$

which are not constant. Their derivative is, by the chain rule again, $X \cdot \nabla u = (X \cdot \nabla\varphi)\,(-\sin\varphi, -\cos\varphi)$, and the frame term is $\omega(X)\, u = \theta(X)\,(-u_2, u_1) = (X \cdot \nabla\varphi)\,(\sin\varphi, \cos\varphi)$. Substituting both into (4.5) [$T^T \nabla_X U = X \cdot \nabla u + \omega(X)\, u,\ \omega(X) = T^T D_X T$],

$$\begin{aligned} T^T \nabla_X U &= \big((X \cdot \nabla\varphi)(-\sin\varphi, -\cos\varphi)\big) + \big((X \cdot \nabla\varphi)(\sin\varphi, \cos\varphi)\big) \\ &= (X \cdot \nabla\varphi)\,\big(-\sin\varphi + \sin\varphi,\ -\cos\varphi + \cos\varphi\big) = (0, 0) : \end{aligned}$$

the change of the coordinates is exactly cancelled by the turning of the frame, as it must be for a field that does not change. Here the plane is flat and $D_X T$ has no normal part, so $TT^T D_X T = D_X T$; on a curved surface the two differ by the second fundamental form.

On a flat piece of surface with a constant frame, $D_X T = 0$, so $\omega = 0$, and the covariant derivative is the ordinary derivative of the coordinates. This is the situation inside one face of a mesh, and it is what will make the finite element computation simple.

4.3. Parallel transport

We can now say what it means to carry a vector along the surface without turning it. A field U is *parallel* along a curve $\gamma(s)$ if it does not change along it in the sense of the connection,

$$\nabla_{\gamma'} U = 0.$$

The forward arrow of the walker on the equator is an example. In a frame this reads, by (4.5) [$T^T \nabla_X U = X \cdot \nabla u + \omega(X)\, u$] with $X = \gamma'$,

$$\frac{du}{ds} + \omega(\gamma')\, u = 0,$$

two linear ordinary differential equations for the two coordinates, of the same kind as $y' = -a\, y$. Given U at the start of the curve, they determine it along the whole curve; that is the *parallel transport* of the vector along γ. We can write the solution. Substitute $\omega(\gamma') = \theta(\gamma')\,\mathcal{J}$ from the entries of ω, and let $\alpha(s) = \int_0^s \theta(\gamma'(\sigma))\, d\sigma$ be the angle that the frame has turned, so that

$\alpha' = \theta(\gamma')$. The equation is $du/ds = -\alpha'(s)\,\mathcal{J}\,u$. Let $\mathrm{Rot}(\beta)$ be the rotation of the plane by the angle β. Its derivative is

$$\frac{d}{d\beta}\begin{pmatrix}\cos\beta & -\sin\beta\\ \sin\beta & \cos\beta\end{pmatrix} = \begin{pmatrix}-\sin\beta & -\cos\beta\\ \cos\beta & -\sin\beta\end{pmatrix}$$
$$= \begin{pmatrix}0 & -1\\ 1 & 0\end{pmatrix}\begin{pmatrix}\cos\beta & -\sin\beta\\ \sin\beta & \cos\beta\end{pmatrix} = \mathcal{J}\,\mathrm{Rot}(\beta).$$

Try $u(s) = \mathrm{Rot}(-\alpha(s))\,u(0)$. By the chain rule with $\beta = -\alpha(s)$, $d\beta/ds = -\alpha'(s)$,

$$\frac{du}{ds} = -\alpha'(s)\,\mathcal{J}\,\mathrm{Rot}(-\alpha(s))\,u(0) = -\alpha'(s)\,\mathcal{J}\,u(s),$$

which is the equation, and at $s = 0$ it gives $\mathrm{Rot}(0)\,u(0) = u(0)$. So the solution rotates the coordinates by *minus* the angle $\alpha(s)$ that the frame has turned: the coordinates turn backwards exactly as fast as the frame turns forwards, so that the vector itself does not turn. This is what happened to the coordinates $(\cos\varphi, -\sin\varphi)$ in Example 4.1.

Transport is a rotation. Parallel transport preserves lengths and angles. To see it, take two parallel fields U and V along γ and differentiate their inner product with (4.4) [$X(U\cdot V) = \nabla_X U\cdot V + U\cdot\nabla_X V$], with $X = \gamma'$:

$$\frac{d}{ds}(U\cdot V) = \nabla_{\gamma'}U\cdot V + U\cdot\nabla_{\gamma'}V = 0\cdot V + U\cdot 0 = 0.$$

So $U\cdot V$ is constant along the curve; with $V = U$ the length $|U|$ is constant too, and then so is the angle between them. A map between two planes that preserves lengths and angles (and orientation, by continuity) is a rotation: parallel transport is a rotation from one tangent plane onto another. This is the property that the discrete transport must have, too.

The covariant derivative compares transported vectors. The two notions fit together in a way that explains the whole discrete method. Move along the curve γ from $\gamma(0)$ to $\gamma(s)$, and let V_s be the vector $U(\gamma(s))$ transported back along the curve to the starting point. Now V_s and $U(\gamma(0))$ lie in the same tangent plane and can be subtracted, and

$$\nabla_{\gamma'}U = \lim_{s\to 0}\frac{V_s - U(\gamma(0))}{s}.$$

To see it, use along the curve a frame that is itself parallel, so that $\omega(\gamma') = 0$ (start with any frame at $\gamma(0)$ and transport its two vectors; they stay orthonormal, since transport is a rotation). Transporting back then leaves the coordinates unchanged, so V_s has the coordinates $u(s)$ of $U(\gamma(s))$, and the difference quotient tends to du/ds, which is $T^T\nabla_{\gamma'}U$ by (4.5) [$T^T\nabla_X U = X\cdot\nabla u + \omega(X)\,u$] with $\omega = 0$. In words: *the covariant derivative is the ordinary derivative taken after every vector has been transported into one common plane.* The discrete method of Section 9 does exactly this: it transports the vertex vectors into the plane of a face, and only then differentiates.

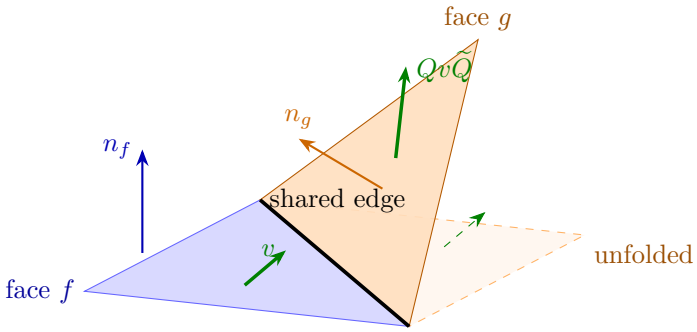


FIGURE 5. Transport across an edge. Two faces sharing an edge meet at an angle in space; unfolding face g into the plane of face f (dashed) is a rotation about the shared edge, and that rotation is exactly the smallest rotation taking n_f to n_g. The vector v of face f and its transport $Qv\widetilde{Q}$ into face g are the same vector of the unfolded plane.

4.4. The discrete connection (an ingredient of Stage 5)

The discrete transport. On a mesh we need transport only between a vertex and an incident face, two points whose tangent planes, with normals n_i and n_f, are close. Of all the rotations of $\mathbb{R}^3$ that take n_i to n_f, and therefore the tangent plane of i onto that of f, the natural one is the *smallest*: the rotation about the axis $n_i \times n_f$ by the angle between the normals. It turns the plane of the vertex onto the plane of the face as a hinge would, without any additional twist about the normal, and it is the discrete analogue of the transport along the short path from the vertex into the face. When the two planes are those of two faces sharing an edge, the axis $n_f \times n_g$ is the direction of the shared edge, and this rotation is exactly the *unfolding* of one face onto the plane of the other about their common edge, which is how a vector is transported across an edge of a polyhedral surface (Figure 5).

The smallest rotation as a matrix. We want a formula for the matrix Q of the smallest rotation taking a unit vector a to a unit vector c, one that the code can evaluate with a few products and no trigonometric functions. We start from the Rodrigues formula for the rotation about a unit axis k by an angle ϕ, and we derive it first. Let $[k]_\times$ be the matrix of the cross product, $[k]_\times w = k \times w$.

Step 0a: split w along the axis. Write $w = (k \cdot w)\,k + w_\perp$, with $w_\perp = w - (k \cdot w)\,k$ orthogonal to k. The rotation leaves the axial part $(k \cdot w)\,k$ alone, and turns $w_\perp$ by ϕ within the plane orthogonal to k. In that plane, $w_\perp$ and $k \times w$ are two orthogonal vectors of the same length ($k \times w = k \times w_\perp$, because $k \times k = 0$, and $|k \times w_\perp| = |w_\perp|$, because $k \perp w_\perp$ and $|k| = 1$), and $k \times w$ is $w_\perp$ turned by 90° about k. So the rotated vector is

$$R\,w = (k \cdot w)\,k + \cos\phi\; w_\perp + \sin\phi\; k \times w.$$

Step 0b: write $w_\perp$ with $[k]_\times$. The double cross product rule $a \times (b \times c) = b\,(a \cdot c) - c\,(a \cdot b)$, with $a = b = k$ and $c = w$, gives

$$[k]_\times^2\, w = k \times (k \times w) = k\,(k \cdot w) - w\,(k \cdot k) = (k \cdot w)\,k - w = -\,w_\perp .$$

So $w_\perp = -[k]_\times^2 w$, and the axial part is $(k \cdot w)\,k = w - w_\perp = w + [k]_\times^2 w$.

Step 0c: substitute. Put the two expressions of Step 0b into Step 0a:

$$\begin{aligned} R\,w &= \big(w + [k]_\times^2 w\big) + \cos\phi\,\big(-[k]_\times^2 w\big) + \sin\phi\,[k]_\times w \qquad \text{(substitute)} \\ &= w + \sin\phi\,[k]_\times w + (1 - \cos\phi)\,[k]_\times^2 w \qquad \text{(collect)}. \end{aligned}$$

Since this holds for every w,

$$R = I + \sin\phi\,[k]_\times + (1 - \cos\phi)\,[k]_\times^2 .$$

For our rotation the axis is $k = v/|v|$ with $v = a \times c$, and the angle ϕ is the angle between a and c. We now express $\sin\phi$, $\cos\phi$ and k through v and $a \cdot c$, which are cheap to compute.

Step 1: the sine and the cosine. By the definitions of the inner and cross products of unit vectors, $a \cdot c = |a|\,|c| \cos\phi = \cos\phi$ and $|v| = |a \times c| = |a|\,|c| \sin\phi = \sin\phi$ (with $0 \le \phi < \pi$, so that $\sin\phi \ge 0$).

Step 2: the first-order term. Since $k = v/|v|$, we have $v = |v|\,k = \sin\phi\,k$, and the cross product matrix is linear in its vector, $[v]_\times w = (\sin\phi\,k) \times w = \sin\phi\,(k \times w) = \sin\phi\,[k]_\times w$. So $[v]_\times = \sin\phi\,[k]_\times$, and the second term of the Rodrigues formula, $\sin\phi\,[k]_\times$, is simply $[v]_\times$.

Step 3: the second-order term. Squaring Step 2,

$$[v]_\times^2 = (\sin\phi\,[k]_\times)(\sin\phi\,[k]_\times) = \sin^2\phi\,[k]_\times^2 ,$$

so $[k]_\times^2 = [v]_\times^2 / \sin^2\phi$, and

$$\begin{aligned} (1 - \cos\phi)\,[k]_\times^2 &= \frac{1 - \cos\phi}{\sin^2\phi}\,[v]_\times^2 && \text{(substitute } [k]_\times^2\text{)} \\ &= \frac{1 - \cos\phi}{(1 - \cos\phi)(1 + \cos\phi)}\,[v]_\times^2 && (\sin^2\phi = 1 - \cos^2\phi) \\ &= \frac{1}{1 + a \cdot c}\,[v]_\times^2 && \text{(cancel, and } \cos\phi = a \cdot c\text{)}. \end{aligned}$$

Step 4: assemble. The Rodrigues formula again, $R = I + \sin\phi\,[k]_\times + (1 - \cos\phi)\,[k]_\times^2$. Substituting Step 2 into its second term and Step 3 into its third,

$$Q = I + \big([v]_\times\big) + \Big(\frac{1}{1 + a \cdot c}\,[v]_\times^2\Big),$$

that is,

$$Q = I + [v]_\times + \frac{1}{1 + a \cdot c}\,[v]_\times^2, \qquad v = a \times c. \tag{4.6}$$

The cancellation of Step 3 is allowed as long as $1 - \cos\phi \neq 0$; but the final formula makes sense even when $a = c$, where $v = 0$ and $Q = I$, which is the right answer. It fails only when $a \cdot c = -1$, for opposite vectors, where the axis is undetermined: every axis orthogonal to a works equally well.

Check: $Qa = c$. We apply (4.6) to a term by term, with the double cross product rule $x \times (y \times z) = y\,(x \cdot z) - z\,(x \cdot y)$ and $a \cdot a = c \cdot c = 1$. The second term:

$$[v]_\times a = (a \times c) \times a = -\,a \times (a \times c) = -\big(a\,(a \cdot c) - c\,(a \cdot a)\big) = c - (a \cdot c)\,a.$$

For the third term we also need $v \times c = (a \times c) \times c = -\,c \times (a \times c) = -\big(a\,(c \cdot c) - c\,(c \cdot a)\big) = -a + (a \cdot c)\,c$. Then

$$\begin{aligned}
[v]_\times^2 a &= v \times \big(c - (a \cdot c)\,a\big) && \text{(the second term)} \\
&= v \times c - (a \cdot c)\,v \times a && \text{(distribute)} \\
&= \big(-a + (a \cdot c)\,c\big) - (a \cdot c)\big(c - (a \cdot c)\,a\big) && \text{(substitute both)} \\
&= -a + (a \cdot c)^2\,a = -\big(1 - (a \cdot c)^2\big)\,a && \text{(the } c \text{ terms cancel).}
\end{aligned}$$

Dividing by $1 + a \cdot c$ and using $1 - (a \cdot c)^2 = (1 - a \cdot c)(1 + a \cdot c)$, the third term is $-(1 - a \cdot c)\,a$. Adding the three terms,

$$Qa = a + \big(c - (a \cdot c)\,a\big) + \big(-(1 - a \cdot c)\,a\big) = a + c - (a \cdot c)\,a - a + (a \cdot c)\,a = c.$$

Proposition 4.2. *For unit vectors a, c with $a \cdot c \neq -1$, the matrix (4.6) is the rotation about the axis $a \times c$ that takes a to c.*

The code. Formula (4.6) is `alignVectors`(n_i, n_f), and the cross product matrix $[v]_\times$ is `bracket`:

```
220 ///Return [n] as the 3x3 operator such that [n]q = n x q
221 ///@param n a vector
222 Eigen::Matrix3d bracket(const Eigen::Vector3d &n)
223 {
224     Eigen::Matrix3d brack;
225     brack << 0.0, -n(2), n(1),
226              n(2), 0.0 , -n(0),
227             -n(1), n(0), 0.0 ;
228     return brack;
229 }
```

The three rows of lines 225–227 are the three components of $v \times w$: for instance $(v \times w)_1 = v_2 w_3 - v_3 w_2$, which is the first row $(0, -v_3, v_2)$ applied to w. The code numbers components from zero, so `n(0)`, `n(1)`, `n(2)` are v_1, v_2, v_3.

```
267 /// https://math.stackexchange.com/questions/180418/calculate-rotation-matrix-to-align-
        vector-a-to-vector-b-in-3d
268 ///@return 3x3 Rotation matrix to align n_v to n_f
269 Eigen::Matrix3d alignVectors(const Eigen::Vector3d& nv, const Eigen::Vector3d& nf)
270 {
271     double c = nv.dot(nf);
272     //Special case for opposite nv and nf vectors.
273     if (std::abs(c + 1.0) < 0.00001)
274         return -Eigen::Matrix3d::Identity();
275 
276     auto vv = nv.cross(nf);
277     Eigen::Matrix3d skew = bracket(vv);
278     return Eigen::Matrix3d::Identity() + skew + 1.0 / (1.0 + c) * skew * skew;
279 }
```

Line 271 computes $a \cdot c$, here $n_i \cdot n_f$. Lines 273–274 catch the case of opposite normals, $a \cdot c = -1$, where the formula would divide by zero; the code then returns $-I$, which only occurs for degenerate meshes (a face folded over onto

its vertex) and is not a rotation (its determinant is -1). Line 276 is $v = a \times c$, line 277 is $[v]_\times$, and line 278 is (4.6) $[\, Q = I + [v]_\times + \frac{[v]_\times^2}{1+a\cdot c}\,]$ term by term. Figure 6 shows the rotation and the planes it connects.

The discrete connection. The rotation $Q_{i\to f}$ takes tangent vectors of vertex i to tangent vectors of face f. But the code works with coordinates, so we follow a vector through the three steps. A tangent vector of i with coordinates u is the vector $T_i u$ of $\mathbb{R}^3$; the rotation turns it into $Q_{i\to f} T_i u$, a tangent vector of the face; and its coordinates in the face frame are $T_f^T Q_{i\to f} T_i u$. So the transport from vertex i to face f, in coordinates, is the product of the three matrices,

$$R_i^f = T_f^T \, Q_{i\to f} \, T_i, \tag{4.7}$$

of sizes 2×3, 3×3 and 3×2, which is a 2×2 matrix. It is `Rvf`(v_i, f):

```
282 ///@return Levi-Civita connection from vertex v tangent space to face f
283 ///tangent space (2x2 rotation matrix)
284 Eigen::Matrix2d Rvf(const Vertex* v, const Face* f)
285 {
286     return face_tangent_space[f->ID].transpose() * alignVectors(v->n, face_normals[f->
            ID]) * vertex_tangent_space[v->ID];
287 }
```

Line 286 reads from right to left exactly as (4.7): the stored vertex frame `vertex_tangent_space[v->ID]` is T_i; the call of `alignVectors` with the vertex normal `v->n` and the face normal `face_normals[f->ID]` is $Q_{i\to f}$, the smallest rotation from n_i to n_f; and the transposed face frame, `face_tangent_space[f->ID].transpose()`, is T_f^T. The rotation is recomputed at every call; nothing is cached.

R_i^f is a rotation of the plane. Continuous parallel transport is a rotation, and we want the discrete one to be a rotation too: a 2×2 matrix with $(R_i^f)^T R_i^f = I_2$ and determinant 1. The idea is simple. The columns of QT_i are the rotated frame vectors Qt_i and Qb_i, an orthonormal basis of the plane of f; the columns of T_f are another orthonormal basis of the same plane; and the matrix that changes between two orthonormal bases of the same plane is orthogonal. In matrix algebra, write $Q = Q_{i\to f}$; then

$$\begin{aligned}
(R_i^f)^T R_i^f &= (T_f^T Q T_i)^T (T_f^T Q T_i) && \text{(definition)}\\
&= T_i^T Q^T \, (T_f T_f^T) \, Q \, T_i && ((ABC)^T = C^T B^T A^T)\\
&= T_i^T Q^T Q \, T_i && \text{(see below)}\\
&= T_i^T T_i = I_2 && (Q^T Q = I_3,\ T_i^T T_i = I_2).
\end{aligned}$$

The third line needs a word: $T_f T_f^T$ leaves the columns of QT_i unchanged. Indeed, $T_f T_f^T = I - n_f n_f^T$ is the projection onto the plane of f, by (3.1) $[\, TT^T = I_3 - nn^T\,]$, and the columns of QT_i already lie in that plane: t_i and b_i are orthogonal to n_i, and a rotation preserves inner products, so $Qt_i \cdot n_f = Qt_i \cdot Qn_i = t_i \cdot n_i = 0$, and the same for b_i. As for the determinant: both bases are positively oriented with respect to n_f, T_f by construction, and QT_i because the rotation Q preserves orientation and takes $b_i = n_i \times t_i$ to $Qn_i \times Qt_i = n_f \times Qt_i$. So the determinant is $+1$.

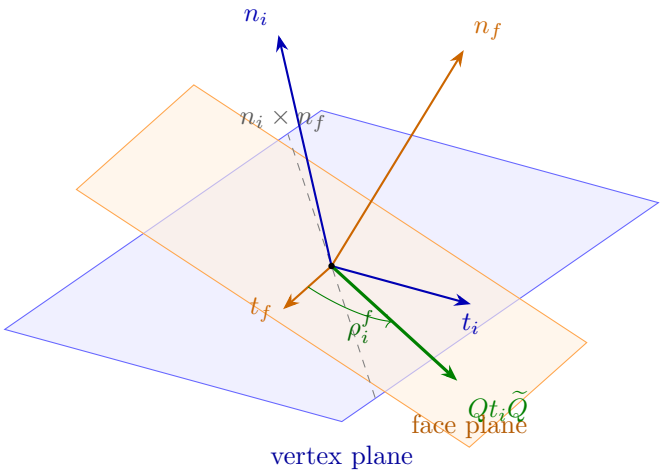


FIGURE 6. The discrete transport from vertex i into face f. The smallest rotation Q, about the axis $n_i \times n_f$ (dashed), takes the vertex plane (blue) onto the face plane (orange); the frame vector t_i becomes $Qt_i\widetilde{Q}$ (green, written as a rotor as in Part II), which makes the angle ρ_i^f with the face frame vector t_f. The matrix R_i^f is the rotation by ρ_i^f.

Proposition 4.3. *R_i^f is a rotation of $\mathbb{R}^2$: $(R_i^f)^T R_i^f = I_2$ and $\det R_i^f = 1$.*

Worked example 4.4. *Let the vertex have the normal $n_i = (0,0,1)$ and the frame vector $t_i = (0,1,0)$, so $b_i = n_i \times t_i = (-1,0,0)$, and let the face have the normal $n_f = (0,-1,1)/\sqrt{2}$ and the frame vector $t_f = (1,0,0)$, so $b_f = n_f \times t_f = (0,1,1)/\sqrt{2}$ (this is the face of Example 12.2). Then $n_i \cdot n_f = 1/\sqrt{2}$ and $v = n_i \times n_f = (1/\sqrt{2},0,0)$. Write $s = 1/\sqrt{2}$. The cross-product matrix of $v = (s,0,0)$ and its square are*

$$[v]_\times = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -s \\ 0 & s & 0 \end{pmatrix}, \qquad [v]_\times^2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -s^2 & 0 \\ 0 & 0 & -s^2 \end{pmatrix},$$

and the factor of (4.6) $[Q = I + [v]_\times + \frac{[v]_\times^2}{1+a\cdot c}]$ is $\frac{1}{1+a\cdot c} = \frac{1}{1+s}$. The diagonal entries of $Q = I + [v]_\times + \frac{1}{1+s}[v]_\times^2$ are therefore 1 and $1 - \frac{s^2}{1+s} = 1 - \frac{1/2}{1+1/\sqrt{2}} = 1 - \frac{1}{2+\sqrt{2}} = \frac{1+\sqrt{2}}{2+\sqrt{2}} = \frac{1}{\sqrt{2}} = s$ (twice), so

$$Q = \begin{pmatrix} 1 & 0 & 0 \\ 0 & s & -s \\ 0 & s & s \end{pmatrix},$$

the rotation by $45°$ about the x axis. As a check, Qn_i is the third column, $(0,-s,s) = n_f$. The rotation takes $t_i = (0,1,0)$ to its second column,

$(0, s, s) = (0, 1, 1)/\sqrt{2} = b_f$, *and* $b_i = (-1, 0, 0)$ *to itself. So the transported* t_i *is the second face frame vector, and the four entries are the inner products* $t_f \cdot Qt_i = (1, 0, 0) \cdot (0, s, s) = 0$, $t_f \cdot Qb_i = -1$, $b_f \cdot Qt_i = |b_f|^2 = 1$ *and* $b_f \cdot Qb_i = 0$*:*

$$R_i^f = T_f^T Q T_i = \begin{pmatrix} t_f \cdot Qt_i & t_f \cdot Qb_i \\ b_f \cdot Qt_i & b_f \cdot Qb_i \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix},$$

the rotation by $\rho_i^f = 90°$*. The program* **ga_fem** *prints these numbers (Section 25.2).*

A rotation of the plane is multiplication by a unit complex number, $R_i^f \leftrightarrow e^{\mathrm{i}\rho_i^f}$: the angle ρ_i^f is the discrete connection between the frame of vertex i and that of face f. The angle has two contributions, the turning of the tangent plane, from Q, and the arbitrary choice of the two frames. In the example the planes turn by only 45°, but $\rho_i^f = 90°$, because the two frame vectors t_i and t_f were chosen in unrelated directions. That is why only combinations in which the frame choices cancel have a geometric meaning.

Transport between two vertices. The stiffness matrix will couple pairs of vertices, so we need to bring a vector from vertex j to vertex i. Through a face f that contains both, the natural route is to go from j into the face, and from the face back to i:

$$R_{j\to i}^f = (R_i^f)^{-1} R_j^f = (R_i^f)^T R_j^f. \tag{4.8}$$

The inverse is the transpose because R_i^f is orthogonal (Proposition 4.3). In this product the choice of the face frame cancels, as it should, since the face frame is only an intermediate station. To see it, replace T_f by $T_f S$ for a rotation S of the plane ($S^T S = S S^T = I_2$). Then $R_i^f = T_f^T Q T_i$ becomes $(T_f S)^T Q T_i = S^T T_f^T Q T_i = S^T R_i^f$, likewise R_j^f becomes $S^T R_j^f$, and

$$(S^T R_i^f)^T (S^T R_j^f) = (R_i^f)^T \, S \, S^T \, R_j^f = (R_i^f)^T R_j^f :$$

unchanged. The choices of the vertex frames do not cancel, and should not: they define the coordinates of the unknowns, so the matrix that takes coordinates at j to coordinates at i must know them. This product is the matrix that will appear in the stiffness matrix.

The first version of the code, `AssembleStiffnessElementEmbedded`, uses instead the smallest rotation from n_j to n_i directly, without passing through the face:

```
289 ///@return Levi-Civita connection from vertex v tangent space to face f
290 ///tangent space (2x2 rotation matrix)
291 Eigen::Matrix2d Rvv(const Vertex* vi, const Vertex* vj)
292 {
293     return vertex_tangent_space[vj->ID].transpose() * alignVectors(vi->n, vj->n) *
          vertex_tangent_space[vi->ID];
294 }
```

Line 293 is $T_j^T Q_{i\to j} T_i$ for the arguments `(vi, vj)`: it transports from its first argument to its second, so the call `Rvv(v[j], v[i])` in that version is $T_i^T Q_{j\to i} T_j$. (Its comment, copied from `Rvf`, still says "to face f".) This

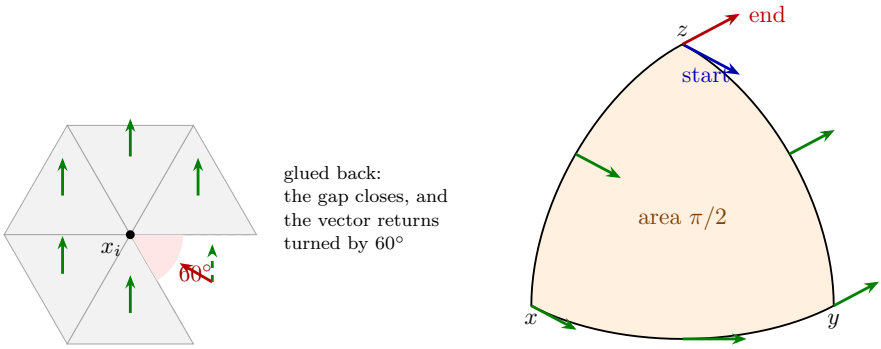


FIGURE 7. Holonomy. Left: the one-ring of a vertex of five equilateral triangles, unfolded into the plane; its angles add up to 300°, and a vector transported around the vertex (constant in the unfolded plane) comes back turned by the angle defect 60° when the gap is closed. Right: on the unit sphere, a vector carried around the octant bounded by three quarter great circles comes back turned by 90°, the area $\pi/2$ of the octant.

is the same as (4.8) to first order in the angles between the normals. The version used, `AssembleStiffnessElementEmbedded2`, goes through the face as in (4.8) [$R^f_{j\to i} = (R^f_i)^T R^f_j$], which is what the finite element derivation produces in Stage 7 (Section 11).

Remark 4.5 (Curvature). *Going around a vertex from face to face, the transports (4.8)* [$R^f_{j\to i} = (R^f_i)^T R^f_j$] *through consecutive faces compose to a rotation that is not the identity in general. For the intrinsic transport of the polyhedral surface (unfolding the faces one after another) its angle is the* angle defect $2\pi - \sum_f \alpha_f$ *of the vertex, the discrete Gaussian curvature; for the transport through vertex normals used here, it approximates the angle defect on fine meshes (Section 22.5). This is the path dependence of parallel transport mentioned in the introduction, and the reason why the transported field must be defined globally by an equation, rather than by composing transports along one particular path.*

Worked example 4.6. *Five equilateral triangles around a vertex (Figure 7, left) form a cone with total angle* $5 \cdot 60° = 300°$*, a pentagonal pyramid. Cutting it along an edge and laying it flat leaves a gap of* $60°$*. A vector carried around the vertex keeps its direction in the flat layout, and when the gap is glued shut it meets its starting copy turned by* $60° = 2\pi - 300°$*, the angle defect. On the unit sphere (right), start at the north pole with a vector, go down a meridian to the equator, along the equator by a quarter turn, and up another meridian back to the pole. Along each great circle the vector keeps its angle with the direction of motion, as the walker's arrow did, and it returns turned by* $90°$*,*

which is the area $4\pi/8 = \pi/2$ of the octant: on a sphere of curvature $K = 1$, holonomy is enclosed area.

Part I · The Derivation with Matrices

5. Stage 1. The PDE: the Connection Laplacian and the Vector Heat Equation

What we want. The first stage of the recipe states the problem in its usual, *strong* form: an equation that must hold at every point of the surface, together with its side conditions. Every later stage starts from it, so it must be written precisely: what the unknown is (a tangent field U), where it lives (a closed curved surface Ω), which operator acts on it, and what is prescribed. We now have a way to differentiate tangent fields. This section uses it to write the continuous problem that the code solves. We first measure how much a field turns, with an energy; differentiating that energy produces an operator, the connection Laplacian, and the tool needed for that differentiation is an integration by parts on the surface, Green's identity, which in turn rests on the divergence theorem. Then we let a field flow downhill in the energy, which is the heat equation, and take one step of it in time with the backward Euler method.

5.1. The Dirichlet energy

How much does a field turn? At a single point, the total covariant derivative ∇U tells us how fast U turns in every direction. A natural single number for its size is the sum of the squares of the rates in two orthogonal directions, and a natural measure for the whole surface is the integral of that number. This is the *Dirichlet energy*

$$E(U) = \frac{1}{2}\int_\Omega |\nabla U|^2 \, dA, \qquad |\nabla U|^2 = \nabla U : \nabla U = \sum_{k=1}^{2} |\nabla_{e_k} U|^2, \tag{5.1}$$

where e_1, e_2 is any orthonormal basis of the tangent plane and $A : B = \operatorname{tr}(A^T B) = \sum_{k,l} A_{kl} B_{kl}$ is the inner product of 2×2 matrices, the sum of the products of their entries. The factor $\frac{1}{2}$ only makes later formulas cleaner.

Integrals over a surface. The symbol $\int_\Omega g \, dA$ means what one expects: cut the surface into many small pieces, multiply the value of g on each piece by the area of the piece, add, and refine the pieces until the sum stops changing. On a mesh the pieces can be taken inside the faces, so the integral over the surface is the sum of the integrals over the faces. If g is constant on a face, its integral over the face is that constant times the area A_f; this simple fact will make all the integrals of Stage 7 (Section 11) exact.

The two expressions of $|\nabla U|^2$. In the frame e_1, e_2, column k of the matrix A of ∇U holds the coordinates of the vector $\nabla_{e_k} U$ (Section 4): $A_{lk} = e_l \cdot \nabla_{e_k} U$. The sum of the squares of all the entries is regrouped column by column,

$$\nabla U : \nabla U = \sum_{k=1}^{2} \sum_{l=1}^{2} A_{lk}^2 = \sum_{k=1}^{2} \Big(\sum_{l=1}^{2} (e_l \cdot \nabla_{e_k} U)^2 \Big) = \sum_{k=1}^{2} |\nabla_{e_k} U|^2,$$

the inner sum being the squared length of $\nabla_{e_k} U$ computed from its two coordinates. In the same way, for two fields with matrices A and B, $\nabla U : \nabla V = \sum_k \sum_l A_{lk} B_{lk} = \sum_k \nabla_{e_k} U \cdot \nabla_{e_k} V$, the inner product of the columns, summed over the columns. The result does not depend on the orthonormal basis. Another orthonormal frame is e'_k with $[\, e'_1 \quad e'_2 \,] = [\, e_1 \quad e_2 \,] S$ for a rotation S, and in it the matrix of ∇U is $A' = S^T A S$ (coordinates are read with S^T, and the columns are taken along the new directions, S). Then, with $\operatorname{tr}(XY) = \operatorname{tr}(YX)$ and $SS^T = S^T S = I_2$,

$$\begin{aligned} A' : A' &= \operatorname{tr}\big((S^T A S)^T (S^T A S)\big) && \text{(definition of :)} \\ &= \operatorname{tr}\big(S^T A^T S\, S^T A S\big) && ((XYZ)^T = Z^T Y^T X^T) \\ &= \operatorname{tr}\big(S^T A^T A\, S\big) && (SS^T = I_2) \\ &= \operatorname{tr}\big(A^T A\, S S^T\big) && (\operatorname{tr}(XY) = \operatorname{tr}(YX)) \\ &= \operatorname{tr}(A^T A) = A : A && (SS^T = I_2). \end{aligned}$$

$E(U) = 0$ exactly when $\nabla U = 0$ everywhere, that is, when U is parallel along every curve. On a curved surface such fields do not exist, because of holonomy (Example 4.6): transporting a vector around a vertex or around a loop brings it back turned. So the energy cannot be made zero, and the fields of smallest energy are the "most parallel" ones.

A simple field in the plane shows what the energy density $|\nabla U|^2$ measures. Let $U(x) = (\cos kx_1, \sin kx_1)$, a unit field that turns at the constant rate k (radians per unit length) as one moves along the x_1 axis, and does not change along x_2. In the plane, with the constant frame of the axes, the covariant derivative is the ordinary derivative, so $\nabla_{e_1} U = k\,(-\sin kx_1, \cos kx_1)$, of length $|k|$, and $\nabla_{e_2} U = 0$. Hence $|\nabla U|^2 = k^2$: the energy density is the square of the rate of turning. A field that turns twice as fast costs four times as much energy, so minimizing the energy spreads the unavoidable turning as evenly and as slowly as possible.

The derivative of the energy. The operator associated with this energy is found, as always, by differentiating it: we ask how E changes when U changes a little. Perturb U by ϵV, with V another tangent field and ϵ a small number.

Step 1: the derivative of the perturbed field. The covariant derivative is linear: by (4.3) $[\, \nabla_X U = (I - nn^T)\, D_X U \,]$ and the linearity of the ordinary derivative,

$$\begin{aligned} \nabla_X (U + \epsilon V) &= (I - nn^T)\, D_X (U + \epsilon V) = (I - nn^T) \big(D_X U + \epsilon\, D_X V \big) \\ &= (I - nn^T)\, D_X U + \epsilon\, (I - nn^T)\, D_X V = \nabla_X U + \epsilon\, \nabla_X V, \end{aligned}$$

for every X, so $\nabla(U + \epsilon V) = \nabla U + \epsilon\, \nabla V$.

Step 2: expand the square at one point. The product : is linear in each factor and symmetric, $A : B = B : A$. Expanding as for numbers, $(a+\epsilon b)^2 = a^2 + 2\epsilon ab + \epsilon^2 b^2$:

$$\begin{aligned} |\nabla(U+\epsilon V)|^2 &= (\nabla U + \epsilon\nabla V) : (\nabla U + \epsilon\nabla V) && \text{(Step 1)} \\ &= \nabla U : \nabla U + \epsilon\,\nabla U : \nabla V \\ &\qquad + \epsilon\,\nabla V : \nabla U + \epsilon^2\,\nabla V : \nabla V && \text{(distribute)} \\ &= |\nabla U|^2 + 2\epsilon\,\nabla U : \nabla V + \epsilon^2\,|\nabla V|^2 && \text{(symmetry of :).} \end{aligned}$$

Step 3: integrate. Substitute Step 2 into the definition (5.1) $[\,E(U) = \frac{1}{2}\int_\Omega |\nabla U|^2\,dA\,]$, and split the integral of the sum into three integrals, moving the constants out:

$$\begin{aligned} E(U+\epsilon V) &= \frac{1}{2}\int_\Omega \big(|\nabla U|^2 + 2\epsilon\,\nabla U : \nabla V + \epsilon^2\,|\nabla V|^2\big)\,dA \\ &= \frac{1}{2}\int_\Omega |\nabla U|^2\,dA + \epsilon\int_\Omega \nabla U : \nabla V\,dA + \frac{\epsilon^2}{2}\int_\Omega |\nabla V|^2\,dA \\ &= E(U) + \epsilon\int_\Omega \nabla U : \nabla V\,dA + \epsilon^2\,E(V). \end{aligned}$$

Step 4: differentiate in ϵ. This is a quadratic polynomial in ϵ, of the form $p + q\,\epsilon + r\,\epsilon^2$ with $p = E(U)$, $q = \int \nabla U : \nabla V\,dA$, $r = E(V)$. Its derivative is $q + 2r\,\epsilon$, and at $\epsilon = 0$ only the coefficient of ϵ is left:

$$\frac{d}{d\epsilon}E(U+\epsilon V)\Big|_{\epsilon=0} = \Big(\int_\Omega \nabla U : \nabla V\,dA + 2\epsilon\,E(V)\Big)\Big|_{\epsilon=0} = \int_\Omega \nabla U : \nabla V\,dA.$$

This is the rate of change of the energy when U moves in the direction V. For a function of finitely many variables, the rate of change in the direction v is $\nabla F \cdot v$, the gradient times the direction, and we would like the same form here: $\int(\cdots)\cdot V\,dA$, with V appearing by itself and not through its derivative. Then $(\cdots)$, the "gradient" of the energy, can be read off. Moving the derivative from V to U is the job of an integration by parts, and on a surface integration by parts rests on the divergence theorem.

5.2. The divergence theorem

In one variable. Recall the fundamental theorem of calculus, $\int_a^b w'(x)\,dx = w(b) - w(a)$: the integral of a derivative over an interval is determined by the values at its two ends. Applied to a product, $(fg)' = f'g + fg'$, it gives integration by parts,

$$\int_a^b f'(x)\,g(x)\,dx = \big[\,fg\,\big]_a^b - \int_a^b f(x)\,g'(x)\,dx,$$

which moves the derivative from one factor to the other at the price of a boundary term. The divergence theorem is the same statement for a region of the plane or of a surface, whose "ends" are a whole boundary curve.

Flux. Let W be a vector field in the plane, and think of it as the velocity of a thin layer of fluid. Let Ω be a region with boundary curve $\partial\Omega$, and let ν be the *outward unit normal* of the boundary: at each point of the curve, the unit vector perpendicular to it and pointing out of the region. Through a short piece of the boundary, of length ds, the fluid leaves the region at the rate $W \cdot \nu \, ds$: only the component of the velocity across the boundary carries fluid out. The total rate at which fluid leaves, the *flux* of W through the boundary, is the integral $\oint_{\partial\Omega} W \cdot \nu \, ds$ around the curve.

Divergence. The *divergence* of $W = (W_1, W_2)$ is the number

$$\operatorname{div} W = \frac{\partial W_1}{\partial x_1} + \frac{\partial W_2}{\partial x_2}.$$

It measures how much fluid is produced at a point, per unit area and unit time. To see why, take a small rectangle $[a_1, b_1] \times [a_2, b_2]$ and integrate the first term over it, first in x_1 and then in x_2. The inner integral is an integral of a derivative, so the fundamental theorem of calculus applies:

$$\int_{a_2}^{b_2} \int_{a_1}^{b_1} \frac{\partial W_1}{\partial x_1} \, dx_1 \, dx_2 = \int_{a_2}^{b_2} \big(W_1(b_1, x_2) - W_1(a_1, x_2) \big) \, dx_2.$$

On the right side of the rectangle, $x_1 = b_1$, the outward normal is $\nu = (1, 0)$ and $W \cdot \nu = W_1$; on the left side, $x_1 = a_1$, it is $\nu = (-1, 0)$ and $W \cdot \nu = -W_1$. Along both sides the arc length is $ds = dx_2$. So the right-hand side is

$$\int_{a_2}^{b_2} W_1(b_1, x_2) \, dx_2 + \int_{a_2}^{b_2} \big(-W_1(a_1, x_2) \big) \, dx_2 = \int_{\text{right}} W \cdot \nu \, ds + \int_{\text{left}} W \cdot \nu \, ds,$$

the flux of W through the left and right sides of the rectangle. The second term is the same computation with the roles of the coordinates exchanged: integrate first in x_2, by the fundamental theorem of calculus,

$$\int_{a_1}^{b_1} \int_{a_2}^{b_2} \frac{\partial W_2}{\partial x_2} \, dx_2 \, dx_1 = \int_{a_1}^{b_1} \big(W_2(x_1, b_2) - W_2(x_1, a_2) \big) \, dx_1,$$

and on the top side, $x_2 = b_2$, $\nu = (0, 1)$ and $W \cdot \nu = W_2$, while on the bottom side, $x_2 = a_2$, $\nu = (0, -1)$ and $W \cdot \nu = -W_2$, with $ds = dx_1$: this is the flux through the top and bottom sides. Adding the two integrals, $\operatorname{div} W = \partial W_1/\partial x_1 + \partial W_2/\partial x_2$ on the left, and the four sides on the right,

$$\int_{\text{rectangle}} \operatorname{div} W \, dA = \int_{\text{right}} + \int_{\text{left}} + \int_{\text{top}} + \int_{\text{bottom}} W \cdot \nu \, ds = \oint_{\partial \text{ rectangle}} W \cdot \nu \, ds.$$

Now cut any region into many small rectangles (or triangles, with the same argument) and add their equations. On the left, the integrals over the pieces add up to the integral over the region. On the right, every side shared by two neighboring pieces appears twice, with opposite outward normals, and the two fluxes cancel: what leaves one piece enters its neighbor. Only the sides on the boundary of the region survive. This is the *divergence theorem*: the total production inside equals the total outflow through the boundary.

A check. Take $W(x) = x = (x_1, x_2)$ on the unit disk. Its divergence is $\partial x_1/\partial x_1 + \partial x_2/\partial x_2 = 1 + 1 = 2$ everywhere, so the left side is 2 times the area π of the disk, 2π. On the unit circle the outward normal is $\nu = x$ itself, so $W \cdot \nu = x \cdot x = |x|^2 = 1$, and the right side is $\oint 1\, ds$, the length of the circle, 2π. The two agree.

On a surface. On a surface, W is a tangent field, the boundary curve lies on the surface, and ν is the *outward conormal*: the unit vector tangent to the surface, perpendicular to the boundary curve, pointing out of the region. The divergence must be written so that it does not depend on coordinates, and the covariant derivative does that. In an orthonormal frame field e_1, e_2,

$$\operatorname{div} W = \sum_{k=1}^{2} (\nabla_{e_k} W) \cdot e_k,$$

the sum of the diagonal entries of the 2×2 matrix of ∇W, its trace. In the plane with the constant frame $e_1 = (1, 0)$, $e_2 = (0, 1)$, the covariant derivative is the ordinary derivative, $(\nabla_{e_k} W) \cdot e_k = \partial W_k/\partial x_k$, and this is the divergence above. In general it does not depend on the frame: in another orthonormal frame, rotated by S, the matrix of ∇W is $S^T A S$ (as for $|\nabla U|^2$ above), and $\operatorname{tr}(S^T A S) = \operatorname{tr}(A S S^T) = \operatorname{tr}(A)$. A small piece of surface is nearly a piece of plane, so the rectangle argument applies to it, and the cancellation across shared sides works as before. So the theorem holds on surfaces in the same form:

$$\int_\Omega \operatorname{div} W \, dA = \oint_{\partial\Omega} W \cdot \nu \, ds. \tag{5.2}$$

On a closed surface, such as the cactus or the sphere, there is no boundary and the right-hand side is zero: what is produced in one place is absorbed in another.

5.3. Green's identity and the connection Laplacian

What we want. We want to move the derivative off V in $\int_\Omega \nabla U : \nabla V \, dA$. The one-variable model is $\int f'g' = [f'g] - \int f''g$, integration by parts with f' in the role of f; it puts both derivatives on f and leaves g alone, at the price of a second derivative and a boundary term. On a surface the result will be

$$\int_\Omega \nabla U : \nabla V \, dA = \int_\Omega (\nabla^* \nabla U) \cdot V \, dA + \oint_{\partial\Omega} (\nabla_\nu U) \cdot V \, ds, \tag{5.3}$$

where ν is the outward conormal and $\nabla^* \nabla U$ is a tangent field built from second covariant derivatives of U, which the derivation will produce. This is *Green's identity* for tangent fields.

The idea. In one variable, $\int f'g' = [f'g] - \int f''g$ comes from integrating the product rule $(f'g)' = f''g + f'g'$: we look for a quantity whose derivative contains $f'g'$, and integrate. Here we look for a tangent field W whose divergence contains $\nabla U : \nabla V$, and apply the divergence theorem. The analogue of $f'g$, "the derivative of U times V", is the field W defined by

$$W \cdot X = (\nabla_X U) \cdot V \qquad \text{for every tangent } X.$$

This defines a tangent vector at each point, because the right-hand side is linear in X (and a linear function of a tangent vector X is the inner product with one tangent vector, as for the gradient in Section 4).

In the frame e_1, e_2 its two coordinates are $W \cdot e_l = (\nabla_{e_l} U) \cdot V$, so $W = \sum_l \big((\nabla_{e_l} U) \cdot V\big)\, e_l$.

Step 1: the divergence of W, first rewriting. Use an orthonormal frame field e_1, e_2, so that $\operatorname{div} W = \sum_k (\nabla_{e_k} W) \cdot e_k$. Metric compatibility (4.4) again,

$$X(U \cdot V) = \nabla_X U \cdot V + U \cdot \nabla_X V, \tag{4.4}$$

applied with W in the place of U, e_k in the place of V, and $X = e_k$, gives

$$e_k(W \cdot e_k) = (\nabla_{e_k} W) \cdot e_k + W \cdot (\nabla_{e_k} e_k).$$

We solve it for the term we want, by subtracting $W \cdot (\nabla_{e_k} e_k)$ from both sides:

$$(\nabla_{e_k} W) \cdot e_k = e_k(W \cdot e_k) - W \cdot (\nabla_{e_k} e_k).$$

Now the definition $W \cdot X = (\nabla_X U) \cdot V$ is used twice: with $X = e_k$ it gives $W \cdot e_k = (\nabla_{e_k} U) \cdot V$, and with $X = \nabla_{e_k} e_k$ (a tangent vector, the turning of the frame vector e_k) it gives $W \cdot (\nabla_{e_k} e_k) = (\nabla_{\nabla_{e_k} e_k} U) \cdot V$. Substituting both,

$$(\nabla_{e_k} W) \cdot e_k = e_k\big((\nabla_{e_k} U) \cdot V\big) - (\nabla_{\nabla_{e_k} e_k} U) \cdot V.$$

Step 2: the product rule once more. The first term on the right is the derivative of an inner product of two tangent fields, $\nabla_{e_k} U$ and V. Metric compatibility (4.4), now with $\nabla_{e_k} U$ in the place of U, V in the place of V, and $X = e_k$:

$$e_k\big((\nabla_{e_k} U) \cdot V\big) = (\nabla_{e_k} \nabla_{e_k} U) \cdot V + (\nabla_{e_k} U) \cdot (\nabla_{e_k} V).$$

Step 3: substitute and sum over k. Substituting Step 2 into the result of Step 1, and then collecting the two terms that are inner products with V:

$$\begin{aligned}(\nabla_{e_k} W) \cdot e_k &= \Big((\nabla_{e_k} \nabla_{e_k} U) \cdot V + (\nabla_{e_k} U) \cdot (\nabla_{e_k} V)\Big) \\ &\qquad - (\nabla_{\nabla_{e_k} e_k} U) \cdot V \qquad \text{(substitute)} \\ &= \big(\nabla_{e_k} \nabla_{e_k} U - \nabla_{\nabla_{e_k} e_k} U\big) \cdot V \\ &\qquad + (\nabla_{e_k} U) \cdot (\nabla_{e_k} V) \qquad \text{(collect)}.\end{aligned}$$

Summing over $k = 1, 2$, and moving the sum inside the inner product with V in the first term,

$$\begin{aligned}\operatorname{div} W = \sum_{k=1}^{2} (\nabla_{e_k} W) \cdot e_k &= \Big(\sum_{k=1}^{2} \big(\nabla_{e_k} \nabla_{e_k} U - \nabla_{\nabla_{e_k} e_k} U\big)\Big) \cdot V \\ &\qquad + \sum_{k=1}^{2} (\nabla_{e_k} U) \cdot (\nabla_{e_k} V).\end{aligned}$$

The last sum is $\nabla U : \nabla V$, by the column argument used for $|\nabla U|^2$ above: the inner product of the columns of the two matrices, summed over the columns.

For the first sum we *define* the *connection Laplacian*

$$\nabla^*\nabla U = -\sum_{k=1}^{2} \big(\nabla_{e_k}\nabla_{e_k}U - \nabla_{\nabla_{e_k}e_k}U\big),$$

so that the first sum is $-\nabla^*\nabla U$. Substituting both,

$$\operatorname{div} W = -(\nabla^*\nabla U)\cdot V + \nabla U : \nabla V.$$

The second term of the definition of $\nabla^*\nabla$ corrects for the turning of the frame. In the plane, with the constant frame $e_1 = (1,0)$, $e_2 = (0,1)$ of the coordinate axes, $\nabla_{e_k}e_k = 0$ and $\nabla_{e_k} = \partial/\partial x_k$, so the second term vanishes and $\nabla^*\nabla U = -\big(\frac{\partial^2 U}{\partial x_1^2} + \frac{\partial^2 U}{\partial x_2^2}\big)$: minus the ordinary Laplacian, applied to each component of U.

Step 4: integrate. Integrate the last equation over Ω, the integral of the sum on the right being the sum of the integrals:

$$\int_\Omega \operatorname{div} W\, dA = -\int_\Omega (\nabla^*\nabla U)\cdot V\, dA + \int_\Omega \nabla U : \nabla V\, dA.$$

On the left we use the divergence theorem (5.2) [$\int_\Omega \operatorname{div} W\, dA = \oint_{\partial\Omega} W\cdot\nu\, ds$], and then the definition of W with $X = \nu$, which gives $W\cdot\nu = (\nabla_\nu U)\cdot V$:

$$\int_\Omega \operatorname{div} W\, dA = \oint_{\partial\Omega} W\cdot\nu\, ds = \oint_{\partial\Omega} (\nabla_\nu U)\cdot V\, ds.$$

Substituting this for the left side,

$$\oint_{\partial\Omega} (\nabla_\nu U)\cdot V\, ds = -\int_\Omega (\nabla^*\nabla U)\cdot V\, dA + \int_\Omega \nabla U : \nabla V\, dA.$$

Add $\int_\Omega (\nabla^*\nabla U)\cdot V\, dA$ to both sides and exchange the two sides:

$$\int_\Omega \nabla U : \nabla V\, dA = \int_\Omega (\nabla^*\nabla U)\cdot V\, dA + \oint_{\partial\Omega} (\nabla_\nu U)\cdot V\, ds,$$

which is (5.3).

Proposition 5.1 (Green's identity). *For tangent fields U and V on a surface Ω with boundary $\partial\Omega$ and outward unit conormal ν, the identity (5.3) holds, with the connection Laplacian $\nabla^*\nabla U = -\sum_k \big(\nabla_{e_k}\nabla_{e_k}U - \nabla_{\nabla_{e_k}e_k}U\big)$ in any orthonormal frame field.*

What it gives. With V vanishing on the boundary, or on a closed surface, which has no boundary, the boundary term of (5.3) is zero and Green's identity reads

$$\int_\Omega \nabla U : \nabla V\, dA = \int_\Omega (\nabla^*\nabla U)\cdot V\, dA :$$

moving ∇ off V turns it into $\nabla^*\nabla$ on U. In the language of linear algebra, $\nabla^*\nabla$ is ∇ transposed and then multiplied by ∇, like the matrix A^TA; ∇^* is the *adjoint* of ∇, hence the notation. The derivative of the energy computed

above is therefore $\int(\nabla^*\nabla U)\cdot V\,dA$: the gradient of E is $\nabla^*\nabla U$. Taking $V = U$ shows that the operator is *positive*, like A^TA:

$$\int(\nabla^*\nabla U)\cdot U\,dA = \int \nabla U : \nabla U\,dA = \int |\nabla U|^2\,dA \geq 0.$$

Its negative, $\Delta U = -\nabla^*\nabla U$, is the *rough* or *Bochner* Laplacian of tangent fields; on scalar functions the same construction gives the familiar Laplace–Beltrami operator. The sign matters: the energy decreases along $-\nabla^*\nabla U$, which is the direction in which the heat equation must move.

What the Laplacian measures. In one variable, Taylor's formula gives $u(x \pm h) = u(x) \pm h\,u'(x) + \frac{h^2}{2}u''(x) + \cdots$. Adding the two expansions, the terms $\pm h\,u'(x)$ cancel,

$$u(x+h) + u(x-h) = 2\,u(x) + h^2\,u''(x) + \cdots,$$

and dividing by 2 and subtracting $u(x)$,

$$\frac{u(x+h)+u(x-h)}{2} - u(x) = \frac{h^2}{2}\,u''(x) + \cdots.$$

So u'' measures how much the average of the neighbors exceeds the value at the point, and $-u''$, the one-variable $\nabla^*\nabla$, how much the value exceeds the average of its neighbors. For a tangent field the neighbors' vectors live in other tangent planes, and they must be transported to the point before they can be averaged with its vector. We will see in Section 11 that the discrete connection Laplacian does exactly that, edge by edge.

5.4. The vector heat equation

For a temperature u on a line, the heat equation is $\partial u/\partial t = u''$: by the previous paragraph, the temperature rises where it is below the average of its neighbors and falls where it is above. Heat flows from hot to cold, and the profile smooths out. The heat equation for tangent fields does the same with vectors: it lets a field flow downhill in the Dirichlet energy,

$$\frac{\partial U}{\partial t} = -\nabla^*\nabla U = \Delta U. \tag{5.4}$$

Here U depends on the point and on the time t, and $\partial U/\partial t$ is its derivative in time at a fixed point. Along the flow the field smooths out and decays. To see it, differentiate its total squared length in time, use the equation, and then Green's identity with $V = U$. Green's identity (5.3) again:

$$\int_\Omega \nabla U : \nabla V\,dA = \int_\Omega (\nabla^*\nabla U)\cdot V\,dA + \oint_{\partial\Omega}(\nabla_\nu U)\cdot V\,ds. \tag{5.3}$$

With $V = U$, and with no boundary (a closed surface) or with $U = 0$ or $\nabla_\nu U = 0$ on the boundary, the boundary term vanishes, and $\nabla U : \nabla U =$

$|\nabla U|^2$, so it reads $\int_\Omega (\nabla^* \nabla U) \cdot U \, dA = \int_\Omega |\nabla U|^2 \, dA$. Then

$$\begin{aligned} \frac{d}{dt} \frac{1}{2} \int_\Omega |U|^2 \, dA &= \int_\Omega U \cdot \frac{\partial U}{\partial t} \, dA \qquad \text{(product rule)} \\ &= -\int_\Omega U \cdot \nabla^* \nabla U \, dA \quad \text{(the heat equation (5.4))} \\ &= -\int_\Omega |\nabla U|^2 \, dA \le 0 \quad \text{(Green's identity with } V = U\text{)}. \end{aligned}$$

(The first step is $\frac{d}{dt} \frac{1}{2} |U|^2 = U \cdot \partial_t U$, the product rule.) Starting from a vector concentrated at a source point s, Sharp, Soliman and Crane [11] show that for short times the solution at a point x points in the direction of the source vector transported along the shortest geodesic from s to x, and that only its magnitude depends on the distance. Normalizing the solution at every point therefore gives the parallel transport of the source vector.

5.5. Backward Euler

The heat equation is continuous in time; the computer takes steps. The simplest step replaces the time derivative by a difference quotient over a step of length Δt. With U^0 the field at time 0 and U the field at time Δt, there are two simple choices for where to evaluate the right-hand side: at the old time (*forward* Euler) or at the new time (*backward* Euler). We take the backward one:

$$\frac{U - U^0}{\Delta t} = -\nabla^* \nabla U \qquad \Longleftrightarrow \qquad U + \Delta t \, \nabla^* \nabla U = U^0. \tag{5.5}$$

(Multiplying by Δt gives $U - U^0 = -\Delta t \, \nabla^* \nabla U$; moving the right side to the left and U^0 to the right gives the second form.) The price is that the new field is not given by a formula but by an equation, a linear system in the discrete setting. The reward is stability.

Why backward. The difference is clearest on the single equation $y' = -\lambda y$ with $\lambda > 0$, whose solution $y(t) = e^{-\lambda t} y(0)$ decays. Forward Euler gives $y^1 = (1 - \lambda \Delta t) \, y^0$, backward Euler gives $y^1 + \lambda \Delta t \, y^1 = y^0$, that is $y^1 = y^0 / (1 + \lambda \Delta t)$. For $\lambda \Delta t = 3$, say, forward Euler multiplies by $1 - 3 = -2$ at every step, a growing oscillation, while backward Euler multiplies by $1/(1 + 3) = 0.25$, a decay, as the true solution does. Backward Euler decays for every $\Delta t > 0$; forward Euler only for $\lambda \Delta t < 2$. On a mesh, the discrete Laplacian has fields that change sign from one vertex to the next, which behave like λ of the order of $1/h^2$, with h the spacing of the mesh. Forward Euler would need Δt below the order of h^2, and many small steps to get anywhere; backward Euler can take any step.

In operator form, stability rests on the fact that $I + \Delta t \, \nabla^* \nabla$ is positive definite: for every field $U \neq 0$, splitting the integral in two and replacing the second part by Green's identity with $V = U$ on the closed surface, $\int_\Omega (\nabla^* \nabla U) \cdot$

$U\,dA = \int_\Omega |\nabla U|^2\,dA$ (shown just above),

$$\int_\Omega \big(U + \Delta t\,\nabla^*\nabla U\big)\cdot U\,dA = \int_\Omega |U|^2\,dA + \Delta t \int_\Omega (\nabla^*\nabla U)\cdot U\,dA$$
$$= \int_\Omega |U|^2\,dA + \Delta t \int_\Omega |\nabla U|^2\,dA > 0.$$

(The first term alone is already positive for $U \neq 0$, and the second is not negative.) So the step is always solvable and stable for any Δt, and a single large step can be taken. After one step of length Δt the field is still nonzero at every vertex, only exponentially small far from the source; this is what allows the direction to be read everywhere, as the sphere of Example 15.1 will show. The vector heat method takes exactly one step, with Δt of the order of the square of the mesh spacing h, the time it takes heat to cross a few elements. The code uses $\Delta t = \bar{h}^2$, with $\bar{h}$ the mean edge length, computed by `computeEdgeLengths`:

```
461 double computeEdgeLengths(
462     Mesh* mesh
463 ) {
464     int count = 0;
465     double sumEdgeLengths = 0.0;
466     for(auto &eij : mesh->getEdges()) {
467         Eigen::Vector3d& pi = eij->vertex->p;
468         Eigen::Vector3d& pj = eij->pair->vertex->p;
469         sumEdgeLengths += (pj - pi).norm();
470         count++;
471     }
472     return sumEdgeLengths / (double)count;
473 }
```

The loop of line 466 visits the half-edges of the mesh; lines 467–468 take the two end points of each, and line 469 adds its length. Every edge is stored as two half-edges, one for each direction, of the same length, so the mean over half-edges (line 472) is the mean over edges, $\bar{h}$. In `main`, the result is stored in `avgEdgeLength` (line 613), and the time step is passed to the assembly as `avgEdgeLength*avgEdgeLength` (line 638).

The boundary condition used by the code. The source can be imposed in two ways. Sharp et al. put it in the initial condition, $U^0 = \delta_s U_s$, a vector concentrated at the source vertex. The code instead prescribes the value at the source vertex and starts from zero everywhere else: it solves

$$U + \Delta t\,\nabla^*\nabla U = 0 \quad \text{away from } s, \qquad U(s) = U_s, \tag{5.6}$$

a *Dirichlet* condition at one point (a condition that prescribes the value of the unknown itself). Both produce a field that decays away from the source and whose direction is transported along the paths of steepest heat flow; the direction, which is all we keep, is the same to the accuracy of the method. Equation (5.6) $[\,U + \Delta t\,\nabla^*\nabla U = 0$ off s, $U(s) = U_s\,]$ is a *screened* Poisson equation for the connection Laplacian: the term U makes the influence of the source decay over a distance of order $\sqrt{\Delta t} \approx \bar{h}$.

The one-dimensional version shows where that distance comes from. On a line, the equation reads $u - \Delta t\,u'' = 0$ away from a source at $x = 0$, with

$u(0) = 1$. Try an exponential $u(x) = e^{-x/L}$ for $x > 0$. Differentiating twice, $u' = -\frac{1}{L}e^{-x/L}$ and $u'' = \frac{1}{L^2}e^{-x/L}$. Substituting into the equation,

$$\left(e^{-x/L}\right) - \Delta t\left(\frac{1}{L^2}\,e^{-x/L}\right) = e^{-x/L}\left(1 - \frac{\Delta t}{L^2}\right) = 0,$$

and since $e^{-x/L} > 0$ this holds exactly when $1 - \Delta t/L^2 = 0$, that is, $L = \sqrt{\Delta t}$. By symmetry the solution is $u(x) = e^{-|x|/\sqrt{\Delta t}}$ on both sides (the growing exponential $e^{+|x|/L}$ also solves the equation, but it is not a field that dies away from the source). The solution drops by a factor $e \approx 2.718$ every $\sqrt{\Delta t}$, that is, with the choice $\Delta t = \bar{h}^2$, roughly every edge.

What Stage 1 produced. The strong form of the problem. The domain is a closed surface Ω (the cactus has no boundary); the unknown is a tangent field U on it; and U must satisfy, at every point, the backward Euler step (5.5) and the source condition of (5.6), which together read

$$\boxed{\begin{aligned} &U + \Delta t\,\nabla^*\nabla U = U^0 \ \text{ on } \Omega, \qquad U(s) = U_s, \\ &\text{with } \nabla^*\nabla U = -\sum_{k=1}^{2}\left(\nabla_{e_k}\nabla_{e_k}U - \nabla_{\nabla_{e_k}e_k}U\right), \end{aligned}} \tag{5.7}$$

with $U^0 = 0$ away from the source in the code. The word *strong* refers to the demand that the equation hold point by point, which needs the two covariant derivatives of $\nabla^*\nabla U$ to exist. That demand is the obstacle to computing with piecewise linear fields, and the next three stages remove it: Stage 2 says which fields we are willing to accept, Stage 3 rewrites the equation as a minimization, and Stage 4 rewrites it with only one derivative on U.

Part I · The Derivation with Matrices

6. Stage 2. The Function Spaces

What we want. Stage 1 asks for a field with two covariant derivatives at every point. The fields that the computer can build (Stage 5) are linear on each face and have kinks along the edges: they have no second derivatives at all. Before changing the equation we must decide which fields we are willing to accept as candidate solutions. That decision is the choice of a *function space*: a set of fields that can be added and multiplied by numbers, like the vectors of $\mathbb{R}^N$, together with a way to measure lengths and angles between them. Two requirements pull in opposite directions. The space must be large enough to contain the discrete fields, so that the computed field is a candidate at all. And it must be well behaved enough that the problem has exactly one solution in it, and that the computed field is close to that solution. This stage names the space, states the assumptions under which everything works, and states, without proofs, the three theorems on which the finite element method relies. No functional analysis is assumed: each notion is introduced through its model in $\mathbb{R}^N$, which the reader knows.

Assumptions. We state them once, plainly. The surface Ω is smooth and closed (bounded, without boundary), like the sphere, or like the smooth surface of which the cactus mesh is an approximation; for a surface with a boundary, the test fields are required to vanish on it. The unknown and the test fields are tangent fields. The time step Δt is a positive number. The data U^0 is square integrable, in the sense defined next. The source condition $U(s) = U_s$ at one point needs a separate comment, given at the end of this stage.

Fields as vectors with infinitely many entries. A vector of $\mathbb{R}^N$ is a list of N numbers, and the inner product of two of them is the sum of the products of their entries, $\mathrm{u} \cdot \mathrm{v} = \sum_m \mathrm{u}_m \mathrm{v}_m$. A tangent field assigns a vector to every point of the surface, so it is a list with one entry per point, infinitely many entries. The sum over the entries becomes an integral over the points, and we get the inner product and the length of tangent fields:

$$(U,V)_{L^2} = \int_\Omega U \cdot V \, dA, \qquad \|U\|_{L^2} = \Big(\int_\Omega |U|^2 \, dA \Big)^{1/2}. \tag{6.1}$$

It behaves like a dot product: it is symmetric, linear in each argument, and $(U,U)_{L^2} > 0$ for $U \neq 0$. The fields of finite length form the space L^2 of *square-integrable* tangent fields. Most fields one meets are in it. One that is not, on the unit disk of the plane: the field $U(x) = x/|x|^2$, of length $1/r$ at the distance r from the center. Here $|U|^2 = |x|^2/|x|^4 = 1/r^2$. Cutting the disk into thin rings of radius r, width dr and area $2\pi r \, dr$,

$$\int |U|^2 \, dA = \int_0^1 \frac{1}{r^2} \, 2\pi r \, dr = 2\pi \int_0^1 \frac{dr}{r} = \infty.$$

Since only the properties of a dot product are used in its proof, the Cauchy–Schwarz inequality of $\mathbb{R}^N$ holds as well: $|(U,V)_{L^2}| \le \|U\|_{L^2} \, \|V\|_{L^2}$.

Adding the derivative: the space H^1. The weak form of Stage 4 will contain $\nabla U : \nabla V$ besides $U \cdot V$. So we measure a field together with its covariant derivative, and define

$$\begin{aligned} (U,V)_{H^1} &= \int_\Omega U \cdot V \, dA + \int_\Omega \nabla U : \nabla V \, dA, \\ \|U\|^2_{H^1} &= \int_\Omega |U|^2 \, dA + \int_\Omega |\nabla U|^2 \, dA. \end{aligned} \tag{6.2}$$

The second term of the norm is twice the Dirichlet energy (5.1) [$E(U) = \frac{1}{2} \int_\Omega |\nabla U|^2 \, dA$]. The *Sobolev space* H^1 of tangent fields consists of the tangent fields for which both integrals are finite: the field is square integrable, and so is its covariant derivative. We write $\|\nabla U\|^2_{L^2} = \int_\Omega |\nabla U|^2 \, dA$ for the second part.

What does "covariant derivative" mean for a field with kinks? For a field that is smooth inside each face and continuous across the edges, ∇U is computed face by face, and the edges, curves of zero area, contribute nothing to the integrals. (In general, the derivative used in H^1 is the *weak derivative*: the field that makes integration by parts, Green's identity, hold against every

smooth test field. For the fields of this text it is the face-by-face derivative, and that is all we need.) The one-variable model shows exactly where the line is drawn.

- The tent $u(x) = 1 - |x|$ on $[-1, 1]$ has $u' = +1$ on the left and -1 on the right, so $\int_{-1}^{1} u'^2\, dx = 2$: finite. The tent belongs to H^1, kink and all.
- The step, $u = 0$ for $x < 0$ and $u = 1$ for $x > 0$, does not. Replace it by a ramp that rises from 0 to 1 over a small width ϵ. Its slope is $1/\epsilon$ on an interval of length ϵ, so $\int u'^2\, dx = \epsilon/\epsilon^2 = 1/\epsilon$, which grows without bound as the ramp becomes the step.

So a field of H^1 may have kinks, but not jumps. For fields that are smooth on every face of a mesh: if they are continuous across the edges, they are in H^1; if they jump across an edge, they are not. This is the origin of the conformity requirement below.

Why the strong form asks for more. The strong form (5.7) contains $\nabla^*\nabla U$, two derivatives, and asks for it at every point (or at least as a square-integrable field: the space H^2 of fields with two square-integrable derivatives). The tent fails: its second derivative is zero on both sides and, at the kink, the derivative of a jump, which is not even a function. The weak form of Stage 4 puts one derivative on U and one on the test field V, so H^1 is enough for both. Since $H^2 \subset H^1 \subset L^2$, every smooth solution remains a solution: the weak form does not change the answer, it only lets the question be asked for more fields.

Complete spaces: why H^1 and not the smooth fields. The space $\mathbb{R}^N$ has a property that we use without noticing: if the terms of a sequence of vectors come closer and closer to each other, the sequence converges to a vector of $\mathbb{R}^N$. The rational numbers do not have it: the terms of $1, 1.4, 1.41, 1.414, \ldots$ come as close to each other as we like, but their limit, $\sqrt{2}$, is not rational. A space with an inner product in which every such sequence has a limit in the space is called *complete*, and a complete inner product space is a *Hilbert space*. The spaces L^2 and H^1 are Hilbert spaces. The smooth fields, measured with the norm of H^1, are not: rounding off the kink of the tent over a smaller and smaller width gives smooth functions that approach the tent in that norm, and the limit has a kink. Completeness is what makes "find the field of smallest energy" a question with an answer: a sequence of fields with decreasing energy that come closer to each other has a limit, the minimizer, in the space; in an incomplete space it could be missing, as $\sqrt{2}$ is missing from the rationals.

The problem in two abbreviations. Stages 3 and 4 will lead to a problem that is best written with two abbreviations, a *bilinear form* a (a number that depends linearly on each of two fields) and a *linear form* l (a number that depends linearly on one field):

$$a(U, V) = \int_\Omega U \cdot V\, dA + \Delta t \int_\Omega \nabla U : \nabla V\, dA, \qquad l(V) = \int_\Omega U^0 \cdot V\, dA. \quad (6.3)$$

The problem will be:

$$\text{find } U \in H^1 \text{ such that } a(U,V) = l(V) \text{ for every } V \in H^1. \tag{6.4}$$

In $\mathbb{R}^N$ the same structure is a linear system. With a matrix A and a vector b, take $a(\mathrm{u},\mathrm{v}) = \mathrm{v}^T A\,\mathrm{u}$ and $l(\mathrm{v}) = \mathrm{v}^T\mathrm{b}$; then "$a(\mathrm{u},\mathrm{v}) = l(\mathrm{v})$ for every v" says $\mathrm{v}^T(A\mathrm{u} - \mathrm{b}) = 0$ for every v, which is $A\mathrm{u} = \mathrm{b}$ (take $\mathrm{v} = A\mathrm{u} - \mathrm{b}$, whose squared length is then zero). A linear system with a symmetric positive definite matrix has exactly one solution. The three properties of a that correspond to "symmetric", "finite entries" and "positive definite" are the following. In them we use the abbreviations

$$\|U\|_{L^2}^2 = \int_\Omega |U|^2\,dA, \qquad \|\nabla U\|_{L^2}^2 = \int_\Omega |\nabla U|^2\,dA,$$
$$\|U\|_{H^1}^2 = \|U\|_{L^2}^2 + \|\nabla U\|_{L^2}^2,$$

the last being (6.2) again.

Symmetric. $a(U,V) = a(V,U)$, because $U \cdot V = V \cdot U$ and $\nabla U : \nabla V = \nabla V : \nabla U$ at every point, so the two integrals of (6.3) [$a(U,V) = \int U\cdot V + \Delta t \int \nabla U : \nabla V$] do not change when U and V are exchanged.

Bounded. We want a constant C with $|a(U,V)| \le C\,\|U\|_{H^1}\|V\|_{H^1}$ for all U, V: small fields give small values. Three inequalities are used.

- The *triangle inequality* for numbers, $|x+y| \le |x| + |y|$.
- *Cauchy–Schwarz* for the two integral inner products, $\left|\int_\Omega U \cdot V\,dA\right| \le \|U\|_{L^2}\|V\|_{L^2}$, and $\left|\int_\Omega \nabla U : \nabla V\,dA\right| \le \|\nabla U\|_{L^2}\|\nabla V\|_{L^2}$. The second is the same inequality for the inner product $\int A : B\,dA$ of matrix fields, which has the properties of a dot product too.
- *Cauchy–Schwarz in* $\mathbb{R}^2$: for numbers $\alpha, \beta, \gamma, \delta$, $\alpha\beta + \gamma\delta \le (\alpha^2 + \gamma^2)^{1/2}(\beta^2 + \delta^2)^{1/2}$. To see it, expand the difference of the squares of the two sides:

$$\begin{aligned}
&(\alpha^2+\gamma^2)(\beta^2+\delta^2) - (\alpha\beta+\gamma\delta)^2 \\
&= \alpha^2\beta^2 + \alpha^2\delta^2 + \gamma^2\beta^2 + \gamma^2\delta^2 - \left(\alpha^2\beta^2 + 2\alpha\beta\gamma\delta + \gamma^2\delta^2\right) \\
&= \alpha^2\delta^2 - 2\alpha\delta\,\gamma\beta + \gamma^2\beta^2 = (\alpha\delta - \gamma\beta)^2 \;\ge\; 0.
\end{aligned}$$

Let $C = \max(1, \Delta t)$, so that $1 \le C$ and $\Delta t \le C$. Then, writing $\alpha = \|U\|_{L^2}$, $\gamma = \|\nabla U\|_{L^2}$, $\beta = \|V\|_{L^2}$, $\delta = \|\nabla V\|_{L^2}$, the chain below uses in turn the definition of a, the triangle inequality (with $\Delta t > 0$), Cauchy–Schwarz (C–S) for both integrals, the bounds $1 \le C$ and $\Delta t \le C$, the four names, and

Cauchy–Schwarz in $\mathbb{R}^2$:

$$\begin{aligned}
|a(U,V)| &= \Big| \int_\Omega U \cdot V \, dA + \Delta t \int_\Omega \nabla U : \nabla V \, dA \Big| && \text{(def.)} \\
&\le \Big| \int_\Omega U \cdot V \, dA \Big| + \Delta t \Big| \int_\Omega \nabla U : \nabla V \, dA \Big| && \text{(triangle)} \\
&\le \|U\|_{L^2} \|V\|_{L^2} + \Delta t \, \|\nabla U\|_{L^2} \|\nabla V\|_{L^2} && \text{(C–S)} \\
&\le C \, \|U\|_{L^2} \|V\|_{L^2} + C \, \|\nabla U\|_{L^2} \|\nabla V\|_{L^2} && (1, \Delta t \le C) \\
&= C \, (\alpha\beta + \gamma\delta) && \text{(names)} \\
&\le C \, (\alpha^2 + \gamma^2)^{1/2} (\beta^2 + \delta^2)^{1/2} && \text{(C–S in } \mathbb{R}^2) \\
&= C \left(\|U\|_{L^2}^2 + \|\nabla U\|_{L^2}^2 \right)^{1/2} \left(\|V\|_{L^2}^2 + \|\nabla V\|_{L^2}^2 \right)^{1/2} \\
&= C \, \|U\|_{H^1} \|V\|_{H^1} && \text{(def. of } \|\cdot\|_{H^1}).
\end{aligned}$$

Coercive. We want a constant $c > 0$ with $a(U,U) \ge c \|U\|_{H^1}^2$ for every U. Let $c = \min(1, \Delta t)$, so that $1 \ge c$ and $\Delta t \ge c$, and $c > 0$ because $\Delta t > 0$. Each of the two integrals below is at least zero, so multiplying it by a smaller nonnegative coefficient can only decrease it:

$$\begin{aligned}
a(U,U) &= \int_\Omega U \cdot U \, dA + \Delta t \int_\Omega \nabla U : \nabla U \, dA && \text{(def., } V = U) \\
&= \int_\Omega |U|^2 \, dA + \Delta t \int_\Omega |\nabla U|^2 \, dA && \text{(squares)} \\
&\ge c \int_\Omega |U|^2 \, dA + c \int_\Omega |\nabla U|^2 \, dA && (1 \ge c, \ \Delta t \ge c) \\
&= c \left(\|U\|_{L^2}^2 + \|\nabla U\|_{L^2}^2 \right) = c \, \|U\|_{H^1}^2 .
\end{aligned}$$

This is positive definiteness with a margin, like a symmetric matrix whose smallest eigenvalue is at least c: no field of size 1 makes $a(U,U)$ smaller than c. For the cactus, $\Delta t = \bar{h}^2$ is small, so $c = \Delta t$ and $C = 1$.

The linear form is bounded. We want a constant Λ with $|l(V)| \le \Lambda \|V\|_{H^1}$. By Cauchy–Schwarz, and because $\|V\|_{L^2}^2 \le \|V\|_{L^2}^2 + \|\nabla V\|_{L^2}^2 = \|V\|_{H^1}^2$,

$$|l(V)| = \Big| \int_\Omega U^0 \cdot V \, dA \Big| \le \|U^0\|_{L^2} \|V\|_{L^2} \le \|U^0\|_{L^2} \|V\|_{H^1},$$

so $\Lambda = \|U^0\|_{L^2}$, and this is where we need $U^0 \in L^2$.

Exactly one solution: the theorems of Lax–Milgram and Riesz. The theorem usually quoted in finite element texts is the following.

> *Lax–Milgram theorem.* Let H be a Hilbert space, a a bilinear form on H that is bounded with constant C and coercive with constant $c > 0$, and l a linear form bounded with constant Λ. Then there is exactly one $U \in H$ with $a(U,V) = l(V)$ for every $V \in H$, and $\|U\|_H \le \Lambda / c$.

Here $H = H^1$, and the three hypotheses are the three inequalities just written out. Two parts of the conclusion follow from coercivity in a few lines, and we show them.

- *At most one solution.* If U_1 and U_2 both solve the problem, subtract the two equations: $a(U_1, V) - a(U_2, V) = l(V) - l(V) = 0$, so $a(U_1 - U_2, V) = 0$ for every V, by the linearity of a in its first argument. Choose $V = U_1 - U_2$ and use coercivity:

$$c\,\|U_1 - U_2\|_{H^1}^2 \le a(U_1 - U_2, U_1 - U_2) = 0,$$

  and since $c > 0$, $\|U_1 - U_2\|_{H^1} = 0$, that is, $U_1 = U_2$.
- *The bound.* Test the equation with $V = U$ itself, and use coercivity on the left and the bound of l on the right:

$$c\,\|U\|_{H^1}^2 \le a(U, U) = l(U) \le \Lambda\,\|U\|_{H^1};$$

  dividing by $c\,\|U\|_{H^1}$ (if $U \neq 0$) gives $\|U\|_{H^1} \le \Lambda/c$.

Existence is where completeness enters, and for our symmetric a it is the *Riesz representation theorem*: in a Hilbert space every bounded linear form is the inner product with exactly one element of the space. A bilinear form that is symmetric, bounded and coercive is itself an inner product on H^1, the *energy inner product*, with the *energy norm* $\|U\|_a = a(U, U)^{1/2}$. Coercivity, and boundedness with $V = U$, give

$$c\,\|U\|_{H^1}^2 \;\le\; a(U, U) = \|U\|_a^2 \;\le\; C\,\|U\|_{H^1}^2 :$$

the two norms measure the same closeness, and H^1 is complete in either. Applied to l and the energy inner product, Riesz gives exactly one $U \in H^1$ with $l(V) = a(U, V)$ for every V, the solution of (6.4) [find $U \in H^1$ with $a(U, V) = l(V)$ for every $V \in H^1$]. In $\mathbb{R}^N$ this is the familiar fact: with the inner product $\langle \mathrm{u}, \mathrm{v} \rangle_A = \mathrm{v}^T A \mathrm{u}$ of a symmetric positive definite A, the linear form $\mathrm{v}^T \mathrm{b}$ equals $\langle \mathrm{u}, \mathrm{v} \rangle_A$ for every v exactly when $\mathrm{u} = A^{-1}\mathrm{b}$. Lax–Milgram reaches the same conclusion without symmetry.

Stability in the energy norm. A sharper bound than Λ/c comes from using the energy norm on both sides. Since $a(U, U) = \|U\|_{L^2}^2 + \Delta t\,\|\nabla U\|_{L^2}^2 \ge \|U\|_{L^2}^2$, we have $\|U\|_{L^2} \le \|U\|_a$. Then, testing with $V = U$,

$$\begin{aligned} \|U\|_a^2 = a(U, U) = l(U) \qquad & \text{(the equation with } V = U) \\ = \int_\Omega U^0 \cdot U \, dA \le \|U^0\|_{L^2}\|U\|_{L^2} \qquad & \text{(Cauchy–Schwarz)} \\ \le \|U^0\|_{L^2}\|U\|_a \qquad & (\|U\|_{L^2} \le \|U\|_a), \end{aligned}$$

and dividing by $\|U\|_a$, $\|U\|_a \le \|U^0\|_{L^2}$: the step never amplifies its data, whatever Δt, which is the stability of backward Euler found in Stage 1, now in the norm of the weak problem.

The best approximation: Galerkin and Céa. Stage 5 will build a finite dimensional space V_h of fields on the mesh, and Stage 6 will ask (6.4) only for the fields of V_h:

$$\text{find } U_h \in V_h \text{ such that } a(U_h, V_h) = l(V_h) \text{ for every } V_h \in V_h. \tag{6.5}$$

How good is U_h? Suppose that $V_h \subset H^1$. Then every V_h is also an allowed test field of (6.4), and we may subtract the two equations, tested with the

same V_h:

$$\begin{aligned} a(U, V_h) - a(U_h, V_h) &= l(V_h) - l(V_h) = 0 && \text{(subtract)} \\ a(U - U_h, V_h) &= 0 \qquad \text{for every } V_h \in V_h && \text{(linearity).} \end{aligned}$$

This is *Galerkin orthogonality*: the error $U - U_h$ is orthogonal, in the energy inner product, to the whole discrete space. So U_h is the orthogonal projection of U onto V_h, the foot of the perpendicular from U, and the foot of the perpendicular is the closest point. To see it, take any $W_h \in V_h$ and split $U - W_h = X + Y$ with $X = U - U_h$ and $Y = U_h - W_h$. The second part lies in V_h (a difference of two of its fields), so $a(X, Y) = 0$ by Galerkin orthogonality, and expanding the energy norm with the bilinearity and symmetry of a,

$$\begin{aligned} \|U - W_h\|_a^2 &= a(X + Y, X + Y) && \text{(definition)} \\ &= a(X, X) + a(X, Y) + a(Y, X) + a(Y, Y) && \text{(bilinearity)} \\ &= a(X, X) + 2\,a(X, Y) + a(Y, Y) && \text{(symmetry)} \\ &= \|U - U_h\|_a^2 + 0 + \|U_h - W_h\|_a^2 && \text{(orthogonality)} \\ &\geq \|U - U_h\|_a^2. \end{aligned}$$

This is Pythagoras' theorem and *Céa's lemma*: of all the fields of V_h, the Galerkin solution is the closest to the true solution in the energy norm. In the norm of H^1, with the two constants above, for every $W_h \in V_h$,

$$c\,\|U - U_h\|_{H^1}^2 \;\leq\; \|U - U_h\|_a^2 \;\leq\; \|U - W_h\|_a^2 \;\leq\; C\,\|U - W_h\|_{H^1}^2,$$

so $\|U - U_h\|_{H^1} \leq (C/c)^{1/2} \min_{W_h \in V_h} \|U - W_h\|_{H^1}$. The meaning for practice is that the finite element method throws nothing away: its error is as small as the best approximation that the chosen space permits. The accuracy is a question of approximation alone, of how well fields of V_h can imitate the solution; for piecewise linear functions on a mesh of spacing h, the error in the norm of H^1 is of the order of h [10].

Conformity. All of this assumed $V_h \subset H^1$; a discrete space with this property is called *conforming*. It was used at one precise point: to test the continuous equation with the discrete fields. By the tent and the step, a field that is smooth inside each face is in H^1 when it is continuous across every edge, and not when it jumps. For scalar hat functions this holds by construction (Stage 5). For tangent fields on a piecewise flat surface, "continuous across an edge" needs care, because the two faces of an edge lie in different planes: a vector of one face cannot equal a vector of the other unless it points along the edge. The right comparison is the parallel transport of the polyhedral surface across the edge, the unfolding of Figure 5: rotate the neighboring face about the shared edge into the plane of the first, and ask that, at every point of the edge, the unfolded value from one side equal the value from the other side. Stage 5 (Section 9.5) checks this for the space of this part. It holds exactly where the normals around a vertex lie on one great circle, as for faces folded along parallel lines, and approximately in general, with a mismatch of second order in the angles between neighboring normals. A space that is conforming only approximately is a mild *variational crime*, in the language of

the finite element literature: the guarantees above hold up to an additional error that shrinks as the mesh is refined. Part III returns to this question with a connection built from the edge lengths.

The source condition, honestly. One idealization remains. On a surface, a field of H^1 need not be continuous, and need not have a value at a single point. The scalar function $u = \log\log(1/r)$ near the center of a small disk of radius $r_0 < 1/e$ is unbounded at $r = 0$, yet its gradient is square integrable. By the chain rule, with $\log(1/r) = -\log r$,

$$\frac{du}{dr} = \frac{1}{\log(1/r)} \cdot \frac{d}{dr}\big(-\log r\big) = \frac{1}{\log(1/r)} \cdot \Big(-\frac{1}{r}\Big), \qquad |\nabla u| = \Big|\frac{du}{dr}\Big| = \frac{1}{r\log(1/r)}.$$

Integrating over rings of area $2\pi r\,dr$, and then substituting $\tau = \log(1/r)$, $d\tau = -dr/r$, which maps $r = 0$ to $\tau = \infty$ and $r = r_0$ to $\tau = \log(1/r_0)$,

$$\begin{aligned}\int |\nabla u|^2\,dA &= \int_0^{r_0} \frac{1}{r^2\log^2(1/r)}\,2\pi r\,dr = 2\pi \int_0^{r_0} \frac{1}{\log^2(1/r)}\,\frac{dr}{r} \\ &= 2\pi \int_{\log(1/r_0)}^{\infty} \frac{d\tau}{\tau^2} = 2\pi\Big[-\frac{1}{\tau}\Big]_{\log(1/r_0)}^{\infty} = \frac{2\pi}{\log(1/r_0)} < \infty.\end{aligned}$$

So the condition $U(s) = U_s$ at one point is not a condition that the space H^1 can express (and the alternative of Sharp et al., $U^0 = \delta_s U_s$ concentrated at a point, is not in L^2 either). Both are idealizations that become meaningful after discretization: the discrete fields are continuous and have values at the vertices, and the code prescribes the value at one vertex. This is also why the time step is tied to the mesh, $\Delta t = \bar{h}^2$: the problem being approximated is heat from a source of the size of an edge, spreading for the time it takes to cross a few edges, and both shrink together when the mesh is refined. For the discrete problem with the vertex prescribed, the theory above applies as it stands: the test fields vanish at s (Stage 4), and the equations of the free unknowns form a symmetric positive definite system (Section 15).

What Stage 2 produced. The space H^1 of tangent fields with square-integrable covariant derivatives, in which the solution is sought; the three properties of the form a (symmetric, bounded, coercive) and the boundedness of l; and three guarantees: the weak problem has exactly one solution (Riesz, Lax–Milgram), the Galerkin solution is the best approximation in the energy norm (Céa), and both hold provided the discrete space is conforming, that is, made of fields that are continuous across the edges. Stage 3 next reads the same problem as a minimization, Stage 4 derives the weak problem (6.4) from the PDE, and Stage 5 builds V_h and checks its conformity.

Part I · The Derivation with Matrices

7. Stage 3. The Variational Form

What we want. A PDE that comes from an energy can be read as a minimization: its solution is the field of smallest energy. This reading tells us why

the problem has exactly one solution, and, later, what the discrete solution is: the minimizer of the same energy over a smaller set of fields. Here we write the energy, compute the condition for its minimum, and show that this condition is equivalent to the PDE of Stage 1.

The energy. For a field U of H^1 consider

$$J(U) = \frac{1}{2}\int_\Omega |U|^2\,dA + \frac{\Delta t}{2}\int_\Omega |\nabla U|^2\,dA - \int_\Omega U^0\cdot U\,dA. \tag{7.1}$$

The three integrals are finite exactly for the fields of H^1 (with $U^0 \in L^2$), which is one reason for the choice of Stage 2. The middle term is Δt times the Dirichlet energy (5.1) [$E(U) = \frac{1}{2}\int_\Omega |\nabla U|^2\,dA$] of Stage 1.

The condition for a minimum. At a minimizer, the rate of change of J in every direction V must be zero, as for the minimum of a function of several variables: if U is a minimizer, the function of one variable $g(\epsilon) = J(U+\epsilon V)$ has its minimum at $\epsilon = 0$, so $g'(0) = 0$. To compute g, we expand $J(U+\epsilon V)$ term by term, as we expanded E in Section 5. The energy (7.1) with $U+\epsilon V$ in the place of U is

$$J(U+\epsilon V) = \frac{1}{2}\int_\Omega |U+\epsilon V|^2\,dA + \frac{\Delta t}{2}\int_\Omega |\nabla(U+\epsilon V)|^2\,dA - \int_\Omega U^0\cdot(U+\epsilon V)\,dA.$$

The first term. At each point, $|U+\epsilon V|^2 = (U+\epsilon V)\cdot(U+\epsilon V) = U\cdot U + \epsilon\,U\cdot V + \epsilon\,V\cdot U + \epsilon^2\,V\cdot V = |U|^2 + 2\epsilon\,U\cdot V + \epsilon^2|V|^2$. Integrating and moving the constants out,

$$\frac{1}{2}\int_\Omega |U+\epsilon V|^2\,dA = \frac{1}{2}\int_\Omega |U|^2\,dA + \epsilon\int_\Omega U\cdot V\,dA + \frac{\epsilon^2}{2}\int_\Omega |V|^2\,dA.$$

The second term. Step 2 of the derivative of E gave $|\nabla(U+\epsilon V)|^2 = |\nabla U|^2 + 2\epsilon\,\nabla U : \nabla V + \epsilon^2\,|\nabla V|^2$. Integrating and multiplying by $\Delta t/2$,

$$\frac{\Delta t}{2}\int_\Omega |\nabla(U+\epsilon V)|^2\,dA = \frac{\Delta t}{2}\int_\Omega |\nabla U|^2\,dA + \epsilon\,\Delta t\int_\Omega \nabla U : \nabla V\,dA + \frac{\epsilon^2\,\Delta t}{2}\int_\Omega |\nabla V|^2\,dA.$$

The third term. It is linear, so there is no ϵ^2:

$$-\int_\Omega U^0\cdot(U+\epsilon V)\,dA = -\int_\Omega U^0\cdot U\,dA - \epsilon\int_\Omega U^0\cdot V\,dA.$$

Collect by powers of ϵ. The three terms without ϵ add up to $J(U)$. The three terms with ϵ, and the two with ϵ^2, are grouped:

$$J(U+\epsilon V) = J(U) + \epsilon\Big(\int_\Omega U\cdot V\,dA + \Delta t\int_\Omega \nabla U : \nabla V\,dA - \int_\Omega U^0\cdot V\,dA\Big) + \frac{\epsilon^2}{2}\Big(\int_\Omega |V|^2\,dA + \Delta t\int_\Omega |\nabla V|^2\,dA\Big).$$

With the abbreviations (6.3) [$a(U,V) = \int U \cdot V + \Delta t \int \nabla U : \nabla V$, $l(V) = \int U^0 \cdot V$], the first bracket is $a(U,V) - l(V)$, and the second is $a(V,V)$ (since $V \cdot V = |V|^2$ and $\nabla V : \nabla V = |\nabla V|^2$):

$$J(U + \epsilon V) = J(U) + \epsilon \big(a(U,V) - l(V)\big) + \frac{\epsilon^2}{2}\, a(V,V).$$

Differentiate. The derivative in ϵ is $\big(a(U,V) - l(V)\big) + \epsilon\, a(V,V)$, and at $\epsilon = 0$

$$g'(0) = \frac{d}{d\epsilon} J(U + \epsilon V)\Big|_{\epsilon=0} = a(U,V) - l(V).$$

It vanishes for every V exactly when

$$\int_\Omega U \cdot V\, dA + \Delta t \int_\Omega \nabla U : \nabla V\, dA = \int_\Omega U^0 \cdot V\, dA \qquad \text{for every } V \in H^1, \tag{7.2}$$

which is $a(U,V) = l(V)$: the problem (6.4) [find $U \in H^1$ with $a(U,V) = l(V)$ for every $V \in H^1$] of Stage 2.

The condition is also sufficient. Conversely, let U satisfy (7.2), so that the bracket $a(U,V) - l(V)$ is zero for every V. Any other field W of H^1 can be written $W = U + V$ with $V = W - U$, and the expansion with $\epsilon = 1$ gives

$$J(W) = J(U+V) = J(U) + \big(0\big) + \frac{1}{2}\, a(V,V) \;\geq\; J(U) + \frac{c}{2}\, \|V\|^2_{H^1},$$

by coercivity, with $c = \min(1, \Delta t) > 0$. So $J(W) > J(U)$ whenever $W \neq U$: U is the minimizer, and the only one. J is strictly convex, curved upwards in every direction like a bowl, and its bottom is the solution.

What the energy says. The first and last terms of J together measure the distance to U^0. Expanding a square at one point, $\frac{1}{2}|U - U^0|^2 = \frac{1}{2}(U - U^0) \cdot (U - U^0) = \frac{1}{2}|U|^2 - U^0 \cdot U + \frac{1}{2}|U^0|^2$, so

$$\frac{1}{2}|U|^2 - U^0 \cdot U = \frac{1}{2}|U - U^0|^2 - \frac{1}{2}|U^0|^2,$$

and, integrating, $J(U) = \frac{1}{2}\int_\Omega |U - U^0|^2\, dA + \frac{\Delta t}{2}\int_\Omega |\nabla U|^2\, dA - \frac{1}{2}\int_\Omega |U^0|^2\, dA$. The last integral does not depend on U. So the backward Euler step looks for the field that is closest to U^0 and also as parallel as possible, with Δt trading one against the other.

Back to the strong form. Why is the minimizer the solution of the PDE? Suppose the minimizer is smooth, with at least two covariant derivatives, and move the derivative off V in the middle term of (7.2) with Green's identity. Green's identity (5.3) again:

$$\int_\Omega \nabla U : \nabla V\, dA = \int_\Omega (\nabla^* \nabla U) \cdot V\, dA + \oint_{\partial\Omega} (\nabla_\nu U) \cdot V\, ds. \tag{5.3}$$

Substituting its right side, in parentheses, for $\int_\Omega \nabla U : \nabla V\, dA$ in (7.2):

$$\int_\Omega U \cdot V\, dA + \Delta t \Big(\int_\Omega (\nabla^* \nabla U) \cdot V\, dA + \oint_{\partial\Omega} (\nabla_\nu U) \cdot V\, ds \Big) = \int_\Omega U^0 \cdot V\, dA.$$

We drop the boundary integral, move Δt inside the second integral and the right side to the left, and collect the three integrals into one:

$$\int_\Omega U \cdot V\, dA + \Delta t \int_\Omega (\nabla^* \nabla U) \cdot V\, dA - \int_\Omega U^0 \cdot V\, dA = 0 \qquad (\partial\Omega = \emptyset)$$

$$\int_\Omega U \cdot V\, dA + \int_\Omega (\Delta t\, \nabla^* \nabla U) \cdot V\, dA - \int_\Omega U^0 \cdot V\, dA = 0 \qquad (\Delta t \text{ in})$$

$$\int_\Omega \big(U + \Delta t\, \nabla^* \nabla U - U^0\big) \cdot V\, dA = 0 \qquad (\text{collect}).$$

The first of these lines uses that on a closed surface there is no boundary, and the boundary integral, an integral over nothing, is zero. The last line uses that the integral of a sum is the sum of the integrals and that $(A + B - C) \cdot V = A \cdot V + B \cdot V - C \cdot V$. This holds for every V.

From "for every V" to "at every point". Call the field in brackets $R = U + \Delta t\, \nabla^* \nabla U - U^0$, the *residual*; for smooth U and U^0 it is a smooth field, and therefore an allowed choice of V. With $V = R$,

$$\int_\Omega R \cdot R\, dA = \int_\Omega |R|^2\, dA = 0.$$

The integrand $|R|^2$ is continuous and never negative. If it were positive at some point x_0, say $|R(x_0)|^2 = m > 0$, then by continuity $|R|^2 > m/2$ on a small disk around x_0, of some area $\delta > 0$, and the integral would be at least $\frac{m}{2}\,\delta > 0$, not zero. So $|R|^2 = 0$, that is $R = 0$, at every point:

$$U + \Delta t\, \nabla^* \nabla U - U^0 = 0 \qquad \Longleftrightarrow \qquad U + \Delta t\, \nabla^* \nabla U = U^0,$$

which is the strong form (5.7) [$U + \Delta t\, \nabla^* \nabla U = U^0$ on Ω, $U(s) = U_s$]. Conversely, a smooth solution of the PDE satisfies (7.2): that is the derivation of Stage 4, which runs the same steps backwards. So for smooth fields, minimizing J and solving the PDE are the same thing; for fields that are only in H^1, such as the piecewise linear ones, the minimization still makes sense and the PDE does not. With the source condition, the minimization is over the fields with $U(s) = U_s$, and the directions V are those that vanish at s, so that $U + \epsilon V$ keeps the prescribed value.

What Stage 3 produced. The energy J of (7.1), whose unique minimizer is the solution, and the condition for its minimum, (7.2), which is the weak problem $a(U, V) = l(V)$. Stage 4 reaches the same equation from the PDE alone, by the Galerkin route of test fields and integration by parts, which also works for equations that do not come from an energy. Stage 6 uses the minimization to explain why the matrix of the linear system is symmetric.

8. Stage 4. The Weak Form (Galerkin)

What we want. An equation with only first derivatives, which makes sense for every field of H^1 (Stage 2), obtained from the PDE of Stage 1 by a procedure

that works for any PDE. The procedure has three moves: multiply by a test field, integrate, integrate by parts. It is the *Galerkin* route to the weak form; Stage 3 reached the same equation from an energy, but not every PDE has one.

The obstacle. The equation (5.5) $[\,U + \Delta t\,\nabla^*\nabla U = U^0\,]$ involves second derivatives of U, through $\nabla^*\nabla$. The fields that the computer can represent on a mesh are piecewise linear, with a kink across every edge like the tent of Stage 2: their first derivatives exist on each face but jump from face to face, and they have no second derivatives. So the equation, as written, cannot even be evaluated for the fields we can compute with.

The way out has two steps. First we replace the equation by a family of averaged versions of it, one for each *test field*. Then we move one of the two derivatives from U onto the test field, by Green's identity. The result, the *weak form*, contains only first derivatives, and it makes sense for piecewise linear fields.

Test fields. The first step rests on a fact from linear algebra. A vector $r \in \mathbb{R}^N$ is zero if and only if $r \cdot v = 0$ for every $v \in \mathbb{R}^N$ (take $v = r$ to get $|r|^2 = 0$). The same holds for fields, with the integral of the inner product in the role of the inner product of vectors: a tangent field R is zero if and only if

$$\int_\Omega R \cdot V \, dA = 0 \qquad \text{for every tangent field } V,$$

because the choice $V = R$ gives $\int |R|^2 \, dA = 0$, which forces $R = 0$ at every point (for continuous R, by the argument written out in Stage 3: a positive value of $|R|^2$ at one point would make the integral positive). Apply this to the *residual* of the backward Euler step, $R = U + \Delta t\,\nabla^*\nabla U - U^0$, which is zero exactly when U solves the step. So the step is equivalent to: multiply (5.5) $[\,U + \Delta t\,\nabla^*\nabla U = U^0\,]$ by an arbitrary tangent field V, the test field, take the inner product, integrate, and ask that the result hold for every V:

$$\int_\Omega U \cdot V \, dA + \Delta t \int_\Omega (\nabla^*\nabla U) \cdot V \, dA = \int_\Omega U^0 \cdot V \, dA. \tag{8.1}$$

(The inner product of a sum is the sum of the inner products, and the integral of a sum is the sum of the integrals, which splits the left side in two.) Nothing has been lost; the equation has only been rewritten. But it has been rewritten in a form that suggests what to do when we cannot ask for every V: ask for a finite number of them, as many as there are unknowns. In linear algebra this is the familiar statement that a vector is zero when its inner products with the vectors of a basis are zero. The Galerkin method of Stages 5 and 6 (Sections 9 and 10) will do exactly that, with a finite family of test fields.

Integration by parts. The second step removes the second derivatives. Green's identity (5.3) again:

$$\int_\Omega \nabla U : \nabla V \, dA = \int_\Omega (\nabla^*\nabla U) \cdot V \, dA + \oint_{\partial\Omega} (\nabla_\nu U) \cdot V \, ds. \tag{5.3}$$

Solved for the term with $\nabla^*\nabla$ (subtract the boundary integral from both sides and exchange the two sides), it reads

$$\int_\Omega (\nabla^*\nabla U)\cdot V\,dA = \int_\Omega \nabla U : \nabla V\,dA - \oint_{\partial\Omega} (\nabla_\nu U)\cdot V\,ds.$$

Substituting this into the second term of the tested equation (8.1),

$$\int_\Omega U\cdot V\,dA + \Delta t\int_\Omega (\nabla^*\nabla U)\cdot V\,dA = \int_\Omega U^0\cdot V\,dA, \tag{8.1}$$

with the substituted expression in parentheses, and then distributing Δt:

$$\int_\Omega U\cdot V\,dA + \Delta t\Big(\int_\Omega \nabla U : \nabla V\,dA - \oint_{\partial\Omega} (\nabla_\nu U)\cdot V\,ds\Big) = \int_\Omega U^0\cdot V\,dA,$$

$$\int_\Omega U\cdot V\,dA + \Delta t\int_\Omega \nabla U : \nabla V\,dA - \Delta t\oint_{\partial\Omega} (\nabla_\nu U)\cdot V\,ds = \int_\Omega U^0\cdot V\,dA.$$

Why the boundary term vanishes. The cactus and the sphere are closed surfaces: they have no boundary curve, and an integral over an empty curve is zero. On a surface with a boundary, we would choose test fields that vanish on the boundary, where U is prescribed, and then the integrand $(\nabla_\nu U)\cdot V$ is zero there. Either way the boundary term drops out, and the *weak form* of the backward Euler step is: find U such that

$$\boxed{\int_\Omega U\cdot V\,dA + \Delta t\int_\Omega \nabla U : \nabla V\,dA = \int_\Omega U^0\cdot V\,dA} \tag{8.2}$$

for every test field V.

Which test fields. On the cactus the surface is closed, so there is no boundary term at all. What remains is the prescribed value at the source vertex, $U(s) = U_s$, of (5.6) [$U + \Delta t\,\nabla^*\nabla U = 0$ off s, $U(s) = U_s$]. It is handled as in every finite element method: where the value of U is prescribed we do not need an equation to determine it, so we do not test there, and the test fields are those that vanish at s. In the discrete method this will mean that there is no equation for the two coordinates of s: the two equations tested with the basis functions of s are discarded and replaced by the prescribed values. That is exactly the row replacement that the code performs in Section 15.

The same steps in one variable. On an interval $[a, b]$ the backward Euler step reads $u - \Delta t\,u'' = u^0$. Multiply by a test function v that vanishes at both ends, and integrate:

$$\int_a^b u\,v\,dx - \Delta t\int_a^b u''\,v\,dx = \int_a^b u^0\,v\,dx.$$

For the middle term, integrate the product rule $(u'v)' = u''v + u'v'$ from a to b: the left side gives $[\,u'v\,]_a^b$ by the fundamental theorem of calculus, so

$$\begin{aligned}\int_a^b u''\,v\,dx &= [\,u'v\,]_a^b - \int_a^b u'v'\,dx \\ &= \big(u'(b)\cdot 0 - u'(a)\cdot 0\big) - \int_a^b u'v'\,dx = -\int_a^b u'v'\,dx,\end{aligned}$$

the boundary term vanishing with v. Substituting, $\int uv - \Delta t\,\big(-\int u'v'\big) = \int u^0 v$, so the weak form is

$$\int_a^b u\,v\,dx + \Delta t \int_a^b u'\,v'\,dx = \int_a^b u^0\,v\,dx \qquad \text{for every such } v.$$

For the tent $u(x) = 1 - |x|$ the original equation is meaningless at the kink, where u'' does not exist; but $\int u'v'\,dx$ is a perfectly good number, since u' is simply $+1$ on one side and -1 on the other. The surface version (8.2) is the same statement, with $\nabla U : \nabla V$ in place of $u'v'$.

What has been gained. Only first derivatives appear, of U and of V, and they appear symmetrically. A piecewise linear field has first derivatives on every face, constant on each, and $\nabla U : \nabla V$ can be integrated face by face, even though the derivatives jump across edges: a jump on a curve of zero area does not change an integral. So the equation makes sense for every field of H^1, which is the meaning of "weak".

The weak form and the energy. Compare (8.2) with the condition for the minimum of the energy of Stage 3, (7.2) [$\int U \cdot V + \Delta t \int \nabla U : \nabla V = \int U^0 \cdot V$ for every V]: they are the same equation. The two routes meet, as they must for a PDE that comes from an energy. So the weak solution is the minimizer of J, and the discrete solution of Stage 6 will be its minimizer over the discrete space.

What Stage 4 produced. With the abbreviations of Stage 2, (6.3) again,

$$a(U,V) = \int_\Omega U \cdot V\,dA + \Delta t \int_\Omega \nabla U : \nabla V\,dA, \qquad l(V) = \int_\Omega U^0 \cdot V\,dA, \quad (6.3)$$

the left side of the weak form (8.2) is $a(U,V)$ and its right side is $l(V)$. So the weak form is the problem (6.4) announced in Stage 2,

$$\text{find } U \in H^1 \text{ such that } a(U,V) = l(V) \text{ for every } V \in H^1, \qquad (6.4)$$

with $U(s) = U_s$ and test fields that vanish at s. It contains only first derivatives, it holds for every field of H^1, and by Stage 2 it has exactly one solution. The bilinear form a is symmetric, because the PDE came from an energy; it is the object that Stages 6 and 7 turn into a matrix, entry by entry, and l becomes the right-hand side. Stage 5 now builds the finite dimensional space on which this is done.

9. Stage 5. The Discretization: the Mesh and Piecewise Linear Tangent Fields

What we want. A finite dimensional space V_h of tangent fields on the mesh, in which Stage 6 will look for the discrete solution. Stage 2 told us what it must satisfy. Its fields must have square-integrable covariant derivatives; for fields that are smooth on each face this means that they must be continuous across the edges (conformity). And it should approximate smooth fields well,

so that Céa's lemma gives a small error. For Stage 7 we also want its basis functions to be simple on each face, so that the integrals can be computed exactly, and to vanish on most of the mesh, so that the matrices are sparse.

The weak form asks for one equation per test field, and there are infinitely many test fields. The computer can handle only finitely many numbers. The *Galerkin method* makes the problem finite in the most economical way. Choose a finite dimensional space of tangent fields, spanned by N basis fields. Look for the unknown U in that space, so that it is given by N coefficients. And ask the weak form only for the test fields of the same space, which gives N equations. So there are exactly as many equations as unknowns. This stage builds the space; Stage 6 writes the equations.

For scalar functions the standard choice is the piecewise linear functions, built from *hat functions*. This section builds their analogue for tangent fields, and shows that the connection is what makes it possible.

9.1. The mesh and what it provides

The first act of the discretization is to replace the smooth surface Ω by the triangle mesh Ω_h: vertices $x_1, \ldots, x_n$ and flat triangular faces (Section 3.2). From here on every integral over the surface is an integral over the mesh, and an integral over the mesh is the sum of the integrals over its faces,

$$\int_{\Omega_h} g\, dA = \sum_f \int_f g\, dA.$$

The replacement is itself an approximation, of the geometry rather than of the field. It is the reason why the geometric toolkit was needed: the mesh has a tangent plane on every face but none at the vertices, and the planes of neighboring faces meet at an angle. The toolkit has already computed everything that the finite element space needs. We collect it here, each piece with its equation.

- *Per face f:* the unit normal and the area, (3.2) $[\, n_f = \frac{e_1 \times e_2}{|e_1 \times e_2|},\ A_f = \frac{1}{2}|e_1 \times e_2| \,]$, and the frame $T_f = [\, t_f \ \ n_f \times t_f \,]$, with t_f along the first edge. Inside a face the tangent plane is the plane of the face, the same at every point.
- *Per vertex i:* the vertex normal, the area-weighted average (3.3) $[\, n_i \propto \sum_{f \ni i} 2A_f\, n_f \,]$, which defines a plane that the vertex does not have by itself, and the frame $T_i = [\, t_i \ \ n_i \times t_i \,]$ of (3.4). The unknowns are the two coordinates $u_i = T_i^T U(x_i)$ of the field in this frame, (3.5) $[\, \mathrm{u} = (u_{1,1}, u_{1,2}, \ldots, u_{n,2}) \,]$.
- *Per vertex i and incident face f:* the discrete connection, (4.7) with (4.6) again,

$$R_i^f = T_f^T\, Q_{i\to f}\, T_i,$$

$$Q_{i\to f} = I + [v]_\times + \frac{1}{1 + n_i \cdot n_f}\, [v]_\times^2, \qquad v = n_i \times n_f,$$

a 2×2 rotation (Proposition 4.3). It takes the coordinates of a vertex vector to the coordinates, in the face frame, of the same vector transported into the face by the smallest rotation from n_i to n_f.

9.2. The scalar hat functions

In one variable, on a line divided into intervals by nodes $x_1 < x_2 < \cdots$, the hat function of node i is the tent that equals 1 at x_i, falls linearly to 0 at the two neighboring nodes, and is 0 beyond them. Every continuous function that is linear between the nodes is a combination of the hats, $u(x) = \sum_i c_i\, \phi_i(x)$. The coefficient c_i is simply the value $u(x_i)$: at x_i only the hat ϕ_i is nonzero, and it equals 1 there, so $u(x_i) = c_i \cdot 1$.

On a mesh the construction is the same, with triangles in place of intervals. For each vertex i, the *hat function* ϕ_i is the continuous function on the mesh that is linear on every face, equals 1 at x_i and 0 at every other vertex. It is a tent over the one-ring of i (Figure 8). It is zero on every face that does not contain i (outside its *support*), because there it is linear and vanishes at the three vertices.

It is well defined because a linear function on a triangle is determined by its values at the three vertices. To see it, describe the points of the triangle by two coordinates ξ_1, ξ_2 in which the vertices are $(0,0)$, $(1,0)$, $(0,1)$ (Section 12 builds such coordinates), and write the linear function as $c_0 + c_1\xi_1 + c_2\xi_2$. Prescribing the values w_1, w_2, w_3 at the three vertices gives three equations,

$$c_0 = w_1, \qquad c_0 + c_1 = w_2, \qquad c_0 + c_2 = w_3,$$

whose only solution is $c_0 = w_1$, $c_1 = w_2 - w_1$, $c_2 = w_3 - w_1$. Two faces that share an edge have the same values at the two ends of the edge. Along the edge each of the two linear functions is a linear function of one variable, fixed by those two values, so they agree along the whole edge, and the hat is continuous.

On each face f the three hat functions of its vertices add up to 1. The sum $\phi_{f_1} + \phi_{f_2} + \phi_{f_3}$ is linear on the face and equals $1 + 0 + 0 = 1$ at each of its three vertices. The constant function 1 is also linear and has the same three values. Being determined by the same three values, they are the same function:

$$\phi_{f_1} + \phi_{f_2} + \phi_{f_3} = 1 \qquad \text{on the face } f.$$

This *partition of unity* will be used several times: for the gradients (which therefore add up to zero) and for the mass matrix. The scalar piecewise linear functions are the combinations $\sum_i c_i\phi_i$, and, as on the line, c_i is the value at vertex i.

9.3. Why the vector case needs a connection

On a flat domain a vector unknown causes no difficulty. All the vertex vectors live in the same plane $\mathbb{R}^2$ and have coordinates in one common frame. The basis functions are the scalar hats times the two constant vectors of that frame, $\phi_i\, \varepsilon_k$, and a field is $\sum_i \phi_i\, u_i$, a combination of vectors that can be

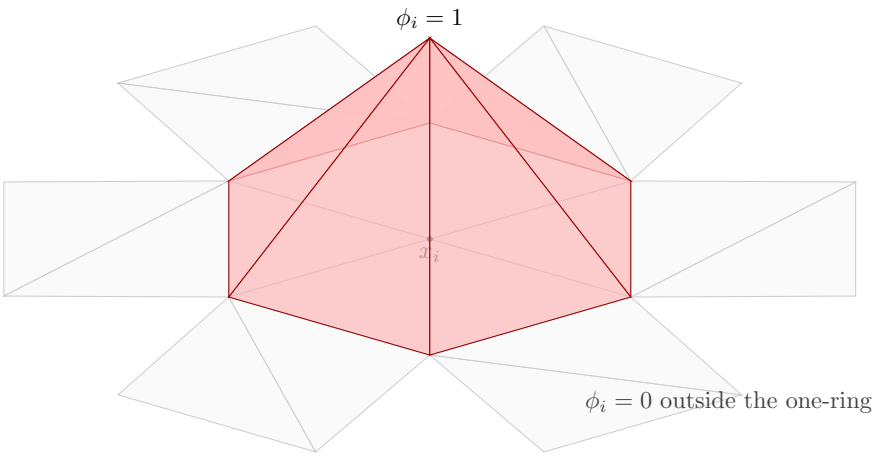


FIGURE 8. The hat function ϕ_i of a vertex: linear on every face, 1 at x_i, 0 at all other vertices, and therefore zero outside the one-ring of x_i.

added because they lie in one plane. On a curved surface replaced by flat faces this fails, for a reason that is purely geometric.

The obvious vector analogue would be $U = \sum_i \phi_i\,(T_i u_i)$: interpolate the three vertex vectors of each face with the hat functions. But $T_i u_i$ lies in the tangent plane of vertex i, and the three vertices of a face have three different tangent planes, none of which is the plane of the face. The interpolated vector would not be tangent to the face, and its derivative would contain the bending of the surface rather than the turning of the field, which is precisely what the covariant derivative was designed to exclude. Before interpolating, the three vectors must be brought into one plane, the plane of the face. The rule that brings a vector from one tangent plane into another is parallel transport. So, on each face f, we interpolate the *transported* vertex vectors. The vertex vector with coordinates u_i in the vertex frame has, after transport into the face, the coordinates $R_i^f u_i$ in the face frame, by (4.7) [$R_i^f = T_f^T Q_{i\to f} T_i$]; as a vector of $\mathbb{R}^3$ it is $T_f R_i^f u_i$. Interpolating these:

$$U\big|_f = \sum_{i\in f} \phi_i\, T_f R_i^f u_i, \qquad \text{in face coordinates:} \quad T_f^T U\big|_f = \sum_{i\in f} \phi_i\, R_i^f u_i. \tag{9.1}$$

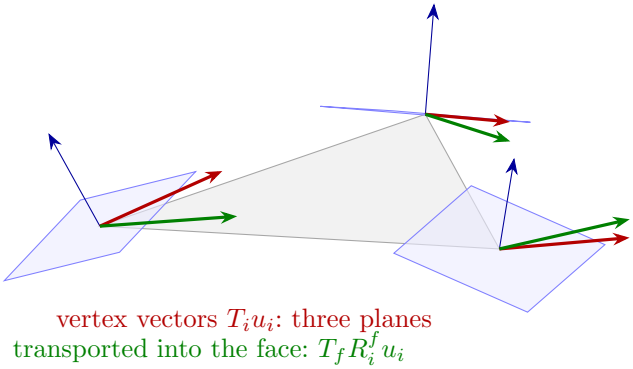


FIGURE 9. Why vectors must be transported before they are interpolated. The vertex vectors $T_i u_i$ (red) lie in three different vertex planes (blue), none of which is the plane of the face; transported into the face, $T_f R_i^f u_i$ (green), they lie in one plane and can be combined.

The second form follows from the first. Multiply the first by T_f^T, with the sum in parentheses, and take the constant matrix inside:

$$
\begin{aligned}
T_f^T U\big|_f &= T_f^T \Big(\sum_{i \in f} \phi_i \, T_f R_i^f u_i \Big) \\
&= \sum_{i \in f} \phi_i \, (T_f^T T_f) \, R_i^f u_i \quad \text{(matrix product distributes; } \phi_i \text{ is a number)} \\
&= \sum_{i \in f} \phi_i \, R_i^f u_i \quad (T_f^T T_f = I_2, \text{ by (3.1)}).
\end{aligned}
$$

Figure 9 shows the difference. Every term is a vector of the plane of the face, so the field is tangent to every face, and it is linear in the unknowns u_i.

The basis. The space of such fields is spanned by $2n$ *vector hat functions*, two per vertex:

$$
\Psi_{i,k}\big|_f = \phi_i \, T_f R_i^f \varepsilon_k, \qquad k = 1, 2, \tag{9.2}
$$

where $\varepsilon_1 = (1, 0)$ and $\varepsilon_2 = (0, 1)$. In words: the frame vector k of vertex i, transported into each incident face, and faded out by the scalar hat. Then $U = \sum_{i,k} u_{i,k} \Psi_{i,k}$, with the unknowns (3.5) [$\mathrm{u} = (u_{1,1}, u_{1,2}, \ldots, u_{n,2})$] as coefficients. To check it on one face, take the two basis functions of vertex i, substitute (9.2), and take the common factors out:

$$
\begin{aligned}
\sum_k u_{i,k} \Psi_{i,k}\big|_f &= \sum_k u_{i,k} \left(\phi_i \, T_f R_i^f \varepsilon_k \right) \\
&= \phi_i \, T_f R_i^f \Big(\sum_k u_{i,k} \varepsilon_k \Big) \quad (\phi_i T_f R_i^f \text{ does not depend on } k) \\
&= \phi_i \, T_f R_i^f u_i \quad (u_{i,1}(1,0) + u_{i,2}(0,1) = u_i).
\end{aligned}
$$

Summing over the vertices i of f gives (9.1) $[\,U|_f = \sum_{i\in f} \phi_i\, T_f R_i^f u_i\,]$. At vertex i itself, $\phi_i = 1$ and the other hats vanish, so the field takes, in each incident face, the transported value $T_f R_i^f u_i$: the vertex vector seen from that face. As a vector of $\mathbb{R}^3$ the field is not continuous across the edges, since two neighboring faces have different planes. Continuity must be understood after unfolding one face onto the other, and Section 9.5 shows that in this sense it holds, approximately. Inside each face it is a smooth tangent field, whose covariant derivative we can compute.

9.4. How the differential geometry fits into the finite element machinery

It helps to list exactly where the geometry of Sections 3 and 4 enters the machinery, and where it does not. Go through the mesh piece by piece.

1. *Inside a face* nothing is curved. The face is a flat triangle with one tangent plane and a constant frame T_f, so $D_X T_f = 0$ and the connection form $\omega(X) = T_f^T D_X T_f$ of (4.5) $[\,T^T \nabla_X U = X \cdot \nabla u + \omega(X)\, u\,]$ vanishes: the covariant derivative is the ordinary derivative of the face coordinates. Inside each face the method is the planar finite element method for a field with two components.
2. *At a vertex* there is no tangent plane. Several faces with several planes meet there, and the method needs one plane, because the unknown vector lives at the vertex. The vertex normal supplies it; this is a modelling choice, made with the positions of the vertices in space.
3. *Between a vertex and a face* the two planes differ, and a vertex vector cannot be used in the face until it has been brought into the plane of the face. This is the one place where the curvature of the surface enters the finite element space: the discrete connection R_i^f, applied to the vertex coordinates *before* the hat function is differentiated. In the continuous theory, the covariant derivative is the ordinary derivative taken after all the vectors have been transported into one common plane (Section 4). On a face, that common plane is the plane of the face, and R_i^f is the transport. The matrices R_i^f take over the role that the connection form ω plays in the smooth formula, as Section 9.6 shows.
4. *Across an edge* two faces meet at an angle. A vector of one face and a vector of the other can be compared only after one face has been unfolded onto the plane of the other, rotating it about the shared edge (Figure 5); that unfolding is the parallel transport of the polyhedral surface. Whether the discrete fields are continuous in this sense is the conformity question of Stage 2, answered in the next subsection.
5. *Around a vertex* the transports from face to face compose to a rotation that is not the identity: the holonomy of Example 4.6, the discrete Gaussian curvature. So no discrete field can be parallel everywhere, the energy $\int |\nabla U|^2$ cannot be made zero, and the curvature reaches the result of the method through the rotations.

TABLE 2. The ingredients of the finite element space of this part: which are extrinsic (they need the position of the surface in space) and which are intrinsic (they could be measured on the surface).

ingredient	extrinsic or intrinsic
face normal n_f, projection $I - nn^T$	extrinsic: they describe the position of the plane in space
vertex normal n_i, vertex plane	extrinsic, and a modelling choice: the polyhedral surface has no plane at a vertex
smallest rotation $Q_{i\to f}$, hence R_i^f	extrinsic: defined by the two normals in space
covariant derivative inside a face	intrinsic: the normal part has been discarded
area A_f, products $g_a^f \cdot g_b^f$	intrinsic: by the cotangent formula (Proposition 12.3), they depend only on the angles and the area of the triangle
transports $(R_a^f)^T R_b^f$ in the stiffness	extrinsic as computed; they approximate the intrinsic transport on fine meshes (Remark on curvature, Section 4)

So the finite element machinery itself is unchanged, stage by stage. What the geometry changes is the basis functions of Stage 5, and through them, in Stage 7, the element matrices.

What is extrinsic in this part. A quantity is *intrinsic* if a creature living on the surface could compute it from measurements on the surface (lengths, angles, areas), and *extrinsic* if it needs the position of the surface in space. Part I works extrinsically throughout: it stores vectors of $\mathbb{R}^3$ and uses the normals. Table 2 says what each ingredient is; Part II examines this more closely, and Part III rebuilds the method from edge lengths alone. The scalar part of the method is therefore intrinsic, even though the code computes it from positions in space through the 3×2 matrix B of Section 12; only the transports depend on the embedding.

9.5. Is the space conforming?

Stage 2 asked for continuity across the edges, understood after unfolding. Take an edge ij shared by the faces f and g, and look at the vertex i first. Seen from face f, the field at x_i is $T_f R_i^f u_i$; seen from g, it is $T_g R_i^g u_i$. The first can be written without the face frame. Substitute $R_i^f = T_f^T Q_{i\to f} T_i$ in parentheses and regroup:

$$\begin{aligned} T_f R_i^f u_i &= T_f \big(T_f^T Q_{i\to f} T_i\big) u_i \\ &= \big(T_f T_f^T\big)\,\big(Q_{i\to f} T_i u_i\big) \quad \text{(associativity)} \\ &= \big(I - n_f n_f^T\big)\,\big(Q_{i\to f} T_i u_i\big) \quad (TT^T = I - nn^T, \text{ by (3.1)}) \\ &= Q_{i\to f} T_i u_i \quad (Q_{i\to f} T_i u_i \text{ is orthogonal to } n_f). \end{aligned}$$

The last step holds because the columns of $Q_{i\to f}T_i$ lie in the plane of f (Proposition 4.3), so the projection does not change them. Likewise $T_g R_i^g u_i = Q_{i\to g}T_i u_i$. Unfold g onto f with the rotation about the shared edge, which is the smallest rotation $Q_{g\to f}$ from n_g to n_f (Section 4). The two values agree, for every u_i, when

$$Q_{g\to f}\, Q_{i\to g}\, T_i = Q_{i\to f}\, T_i :$$

going from the vertex plane into g and then across the edge into f must give the same as going directly from the vertex plane into f. Both sides take n_i to n_f, so they differ at most by a rotation about n_i, a rotation of the vertex plane by some angle. That angle is the holonomy of the closed path $n_i \to n_g \to n_f \to n_i$ traced by the normals on the unit sphere. The reason is that a smallest rotation turns a tangent plane of the unit sphere exactly as parallel transport along the great circle arc between the two normals does. By Example 4.6 (the sphere has curvature 1, and holonomy is enclosed area) the angle is the area of the small spherical triangle with vertices n_i, n_g, n_f. Two consequences follow.

- It is zero when the three normals lie on one great circle. This happens at every vertex whose faces are folded along parallel lines (the folds of a prism), where all the face normals, and therefore their average n_i, are orthogonal to the fold direction. There the space is exactly conforming.
- In general it is of second order in the angles between the normals. For instance, with $n_i = (0, 0, 1)$ and the two face normals tilted by $\alpha = 0.1$ radians ($5.7°$) about the x and the y axis, the mismatch is 0.0050 radians ($0.29°$), close to the area $\alpha^2/2$ of the triangle; with $\alpha = 0.05$ it is a quarter of that, $0.07°$. On a smooth surface sampled with spacing h the angles between neighboring normals are of order h, so the mismatch is of order h^2.

Along the edge, each side of the field is the linear interpolation, with the hats of i and j, of its two end values, and the unfolding is a linear map. So the mismatch along the edge is the linear interpolation of the mismatches at its two ends, and the edge matches wherever both of its ends do. The space of this part is therefore conforming exactly on meshes of parallel folds and approximately on curved meshes, with a mismatch that shrinks quickly under refinement. The method ignores the mismatch: its element integrals are computed face by face, as for a conforming space. Part III shows what happens when the mismatch is not small, on a sheet folded in four directions at every vertex, where the rotated faces no longer fit edge to edge.

9.6. The covariant derivative of a basis function

The weak form needs ∇U for U in the space just built; this is the last property of the space, and the one that Stage 7 will integrate. We compute it on one face, where everything is simple. Inside a face, the surface is a plane and the face frame T_f is constant, so $D_X T_f = 0$ and the connection form $\omega(X) = T_f^T D_X T_f$ vanishes. Then (4.5), with $\omega = 0$, says that the covariant

derivative is the ordinary derivative of the face coordinates:

$$T^T \nabla_X U = X \cdot \nabla u + \omega(X)\, u \quad \Longrightarrow \quad T_f^T \nabla_X U = X \cdot \nabla (T_f^T U),$$

where $u = T_f^T U$ are the face coordinates of the field and $X \cdot \nabla$ is the directional derivative along X. Now substitute the face coordinates of the discrete field, (9.1) $[\, T_f^T U|_f = \sum_{i \in f} \phi_i \, R_i^f u_i \,]$, in parentheses, and take one operation per line:

$$\begin{aligned} T_f^T \nabla_X U\big|_f &= X \cdot \nabla \Big(\sum_{i \in f} \phi_i \, R_i^f u_i \Big) \\ &= \sum_{i \in f} X \cdot \nabla \big(\phi_i \, R_i^f u_i \big) \quad \text{(derivative of a sum)} \\ &= \sum_{i \in f} (X \cdot \nabla \phi_i) \, R_i^f u_i \quad (R_i^f u_i \text{ is constant on the face}). \end{aligned}$$

The last line uses that the derivative of a function times a constant vector is the derivative of the function times the vector, $X(\phi\, c) = X(\phi)\, c$. Write $g_i^f = T_f^T \nabla \phi_i \in \mathbb{R}^2$ for the gradient of the hat function in the coordinates of the face. It is a constant vector on the face, since ϕ_i is linear there (Section 4), and Section 12 computes it. If $x = T_f^T X$ are the face coordinates of the direction X, then

$$X \cdot \nabla \phi_i = (T_f x) \cdot \nabla \phi_i = x \cdot (T_f^T \nabla \phi_i) = x \cdot g_i^f = (g_i^f)^T x.$$

Substituting this into the last line above, in parentheses, and taking the common vector x out to the right:

$$\begin{aligned} &\sum_{i \in f} (X \cdot \nabla \phi_i) \, R_i^f u_i \\ &\quad = \sum_{i \in f} \big((g_i^f)^T x \big) \, R_i^f u_i \\ &\quad = \sum_{i \in f} (R_i^f u_i) \big((g_i^f)^T x \big) \quad \text{(a number commutes with a vector)} \\ &\quad = \sum_{i \in f} \big((R_i^f u_i)(g_i^f)^T \big) \, x \quad \text{(associativity: column, row, column)} \\ &\quad = \Big(\sum_{i \in f} (R_i^f u_i)(g_i^f)^T \Big) \, x \quad \text{(distributivity)}. \end{aligned}$$

The 2×2 matrix in brackets, applied to the coordinates of the direction, gives the coordinates of the derivative, which is what the matrix of ∇U is. We have found both forms of the covariant derivative of the discrete field:

$$T_f^T \nabla_X U\big|_f = \sum_{i \in f} (X \cdot \nabla \phi_i) \, R_i^f u_i, \qquad T_f^T \nabla U\big|_f = \sum_{i \in f} (R_i^f u_i) \, (g_i^f)^T. \quad (9.3)$$

The total covariant derivative is a sum of rank-one 2×2 matrices, each a column times a row: the transported vertex vector times the gradient of its hat. For a single basis function, with u_i replaced by ε_k and the other

vertices set to zero, $\nabla\Psi_{i,k}$ has the matrix $(R_i^f\varepsilon_k)(g_i^f)^T$; in tensor notation, $\nabla\Psi_{i,k} = (R_i^f\varepsilon_k)\otimes g_i^f$.

Where the connection went. Compare (9.3) with the continuous formula (4.5) $[\,T^T\nabla_X U = X\cdot\nabla u + \omega(X)\,u\,]$. In the continuous setting, the turning of the frame appears as the term $\omega(X)\,u$ inside the derivative. In the discrete field there is no such term: inside every face the frame is constant and $\omega = 0$. The turning has not disappeared; it has been moved out of the derivative and into the matrices R_i^f, which are applied *before* differentiating. Each vertex vector is first expressed in the frame of the face, and only then are the hats differentiated. The rotation R_i^f plays the role of the connection form accumulated along the short way from the vertex into the face, which, by Section 4, is what parallel transport does to coordinates. This is why the code never differentiates a frame: the only trace of the connection is the rotations computed by `Rvf`.

What Stage 5 produced. The discrete space

$$V_h = \Big\{ \sum_{i,k} u_{i,k}\,\Psi_{i,k} \Big\}, \qquad \Psi_{i,k}\big|_f = \phi_i\; T_f R_i^f \varepsilon_k,$$

of dimension $2n$, spanned by the vector hat functions (9.2). Each basis function is tangent to every face, vanishes outside the one-ring of its vertex, and has on each face the constant covariant derivative $(R_i^f\varepsilon_k)(g_i^f)^T$ of (9.3). The geometry of the surface enters only through the rotations R_i^f, and the space is conforming exactly on meshes of parallel folds and approximately in general.

Part I · The Derivation with Matrices

10. Stage 6. The Discrete Solution and the Test Functions

What we want. Finitely many equations for finitely many numbers. Stage 2 wrote the target, the Galerkin problem (6.5) again:

$$\text{find } U_h \in V_h \text{ such that } a(U_h, V_h) = l(V_h) \text{ for every } V_h \in V_h, \qquad (6.5)$$

with the forms a and l of (6.3) again,

$$a(U,V) = \int_\Omega U\cdot V\,dA + \Delta t\int_\Omega \nabla U : \nabla V\,dA, \qquad l(V) = \int_\Omega U^0\cdot V\,dA. \quad (6.3)$$

Two moves turn it into a linear system: write the unknown with the basis, and test with the basis.

The discrete solution. Every field of V_h is a combination of the basis functions, so the unknown is

$$U_h = \sum_{j,l} u_{j,l}\,\Psi_{j,l}, \qquad (10.1)$$

and the $2n$ coefficients $u_{j,l}$, the coordinates (3.5) of the vertex vectors, are the unknowns. We use the letters j, l for the unknown and keep i, k for the

test function. On a face this is the interpolation of the transported vertex vectors, (9.1) again:

$$U\big|_f = \sum_{i\in f} \phi_i \; T_f R_i^f \, u_i. \tag{9.1}$$

From now on we write U for U_h, as the code does.

The test functions. Asking the equation for every $V_h \in V_h$ is the same as asking it for the $2n$ basis functions. One direction is clear, since the basis functions belong to V_h. Conversely, suppose that $a(U, \Psi_{i,k}) = l(\Psi_{i,k})$ for every (i,k), and take any $V_h = \sum_{i,k} v_{i,k}\Psi_{i,k}$. Multiply the equation (i,k) by the number $v_{i,k}$ and add all of them. Both forms are linear in the test field, so the numbers and the sum go inside:

$$\begin{aligned}
\sum_{i,k} v_{i,k}\, a(U, \Psi_{i,k}) &= \sum_{i,k} v_{i,k}\, l(\Psi_{i,k}) \\
a\Big(U, \sum_{i,k} v_{i,k}\Psi_{i,k}\Big) &= l\Big(\sum_{i,k} v_{i,k}\Psi_{i,k}\Big) \quad \text{(linearity of } a \text{ and } l \text{ in } V\text{)} \\
a(U, V_h) &= l(V_h).
\end{aligned}$$

So the test fields are $V = \Psi_{i,k}$, one per unknown: the *Galerkin* choice, in which test and trial spaces are the same.

Substituting. Recall the weak form (8.2):

$$\int_\Omega U \cdot V \, dA + \Delta t \int_\Omega \nabla U : \nabla V \, dA = \int_\Omega U^0 \cdot V \, dA. \tag{8.2}$$

And recall the discrete solution (10.1) [$U = \sum_{j,l} u_{j,l}\Psi_{j,l}$]. Substitute the sum for U and the basis function $\Psi_{i,k}$ for V, each in parentheses; the data U^0 stays as it is for the moment:

$$\begin{aligned}
\int_\Omega \Big(\sum_{j,l} u_{j,l}\Psi_{j,l}\Big) \cdot \Psi_{i,k} \, dA + \Delta t \int_\Omega \nabla\Big(\sum_{j,l} u_{j,l}\Psi_{j,l}\Big) : \nabla\Psi_{i,k} \, dA \\
= \int_\Omega U^0 \cdot \Psi_{i,k} \, dA.
\end{aligned}$$

Now we move the sums and the numbers $u_{j,l}$ out, one operation per line. The first term:

$$\begin{aligned}
&\int_\Omega \Big(\sum_{j,l} u_{j,l}\Psi_{j,l}\Big)\cdot\Psi_{i,k}\,dA\\
&\quad=\int_\Omega \sum_{j,l}\big(u_{j,l}\Psi_{j,l}\big)\cdot\Psi_{i,k}\,dA \quad (\cdot \text{ distributes over the sum})\\
&\quad=\int_\Omega \sum_{j,l} u_{j,l}\big(\Psi_{j,l}\cdot\Psi_{i,k}\big)\,dA \quad (\text{a number comes out of } \cdot)\\
&\quad=\sum_{j,l}\int_\Omega u_{j,l}\,\Psi_{j,l}\cdot\Psi_{i,k}\,dA \quad (\text{integral of a finite sum})\\
&\quad=\sum_{j,l} u_{j,l}\int_\Omega \Psi_{j,l}\cdot\Psi_{i,k}\,dA \quad (\text{a constant comes out of } \textstyle\int).
\end{aligned}$$

The second term needs one more step at the start: the covariant derivative is linear, $\nabla(V+W)=\nabla V+\nabla W$ and $\nabla(cV)=c\,\nabla V$ for a number c, because it is the tangent part of a directional derivative and both taking a derivative and taking the tangent part are linear. Then:

$$\begin{aligned}
&\int_\Omega \nabla\Big(\sum_{j,l} u_{j,l}\Psi_{j,l}\Big):\nabla\Psi_{i,k}\,dA\\
&\quad=\int_\Omega \Big(\sum_{j,l} u_{j,l}\,\nabla\Psi_{j,l}\Big):\nabla\Psi_{i,k}\,dA \quad (\nabla \text{ is linear})\\
&\quad=\int_\Omega \sum_{j,l} u_{j,l}\big(\nabla\Psi_{j,l}:\nabla\Psi_{i,k}\big)\,dA \quad (: \text{ is linear in its first factor})\\
&\quad=\sum_{j,l} u_{j,l}\int_\Omega \nabla\Psi_{j,l}:\nabla\Psi_{i,k}\,dA \quad (\text{sum and constant out of } \textstyle\int).
\end{aligned}$$

Put the two results back into the substituted equation, the second multiplied by Δt:

$$\sum_{j,l} u_{j,l}\int_\Omega \Psi_{j,l}\cdot\Psi_{i,k}\,dA+\Delta t\sum_{j,l} u_{j,l}\int_\Omega \nabla\Psi_{j,l}:\nabla\Psi_{i,k}\,dA=\int_\Omega U^0\cdot\Psi_{i,k}\,dA.$$

The two sums on the left run over the same indices (j,l), so they merge into one sum, and in each of its terms the common factor $u_{j,l}$ is taken out to the right. We have one equation per test function, $2n$ linear equations in the $2n$ unknowns:

$$\sum_{j,l}\Big(\int_\Omega \Psi_{j,l}\cdot\Psi_{i,k}\,dA+\Delta t\int_\Omega \nabla\Psi_{j,l}:\nabla\Psi_{i,k}\,dA\Big)\,u_{j,l}=\int_\Omega U^0\cdot\Psi_{i,k}\,dA, \quad (10.2)$$

that is, $\sum_{j,l} a(\Psi_{j,l},\Psi_{i,k})\,u_{j,l}=l(\Psi_{i,k})$. The bracket on the left is the number $a(\Psi_{j,l},\Psi_{i,k})$: it is the coefficient of the unknown $u_{j,l}$ in the equation of the test function $\Psi_{i,k}$.

The right side with the interpolated data. The right side of (10.2) is an integral of the data. To write it with numbers at the vertices, like the left side, we replace U^0 by its interpolant in the same space,

$$U^0 = \sum_{j,l} u^0_{j,l}\, \Psi_{j,l},$$

with the coordinates $u^0_{j,l}$ of the data at the vertices, $\mathrm{u}^0 = (u^0_{1,1}, u^0_{1,2}, \ldots, u^0_{n,2})$. Substituting it, in parentheses, into the right side of (10.2) and taking the same four steps as for the first term of the left side:

$$\begin{aligned}
&\int_\Omega U^0 \cdot \Psi_{i,k}\, dA \\
&\quad = \int_\Omega \Big(\sum_{j,l} u^0_{j,l} \Psi_{j,l} \Big) \cdot \Psi_{i,k}\, dA \\
&\quad = \int_\Omega \sum_{j,l} u^0_{j,l} \left(\Psi_{j,l} \cdot \Psi_{i,k} \right) dA \quad (\cdot \text{ distributes; the number comes out}) \\
&\quad = \sum_{j,l} u^0_{j,l} \int_\Omega \Psi_{j,l} \cdot \Psi_{i,k}\, dA \quad (\text{sum and constant out of } \textstyle\int).
\end{aligned}$$

So, with the interpolated data, equation (i,k) of (10.2) becomes

$$\begin{aligned}
\sum_{j,l} \Big(\int_\Omega \Psi_{j,l} \cdot \Psi_{i,k}\, dA + \Delta t \int_\Omega \nabla \Psi_{j,l} : \nabla \Psi_{i,k}\, dA \Big)\, u_{j,l} \\
= \sum_{j,l} \Big(\int_\Omega \Psi_{j,l} \cdot \Psi_{i,k}\, dA \Big)\, u^0_{j,l}.
\end{aligned}$$

(In the code the data is zero away from the source vertex, and the two equations of the source vertex are replaced by its prescribed value, Section 15.)

The matrix form. Give names to the two integrals. For each pair of indices, a row index (i,k) and a column index (j,l), let

$$\mathrm{M}_{(i,k),(j,l)} = \int_\Omega \Psi_{j,l} \cdot \Psi_{i,k}\, dA, \qquad \mathrm{L}_{(i,k),(j,l)} = \int_\Omega \nabla \Psi_{j,l} : \nabla \Psi_{i,k}\, dA.$$

Substituting these names into the last equation, equation (i,k) reads

$$\sum_{j,l} \left(\mathrm{M}_{(i,k),(j,l)} + \Delta t\, \mathrm{L}_{(i,k),(j,l)} \right) u_{j,l} = \sum_{j,l} \mathrm{M}_{(i,k),(j,l)}\, u^0_{j,l}.$$

A sum over the column index of a matrix entry times the entry of a vector is one entry of the product of the matrix with the vector: the left side is entry (i,k) of $(\mathrm{M} + \Delta t\, \mathrm{L})\, \mathrm{u}$, and the right side is entry (i,k) of $\mathrm{M}\, \mathrm{u}^0$. All the $2n$ equations together are

$$(\mathrm{M} + \Delta t\, \mathrm{L})\, \mathrm{u} = \mathrm{M}\, \mathrm{u}^0, \tag{10.3}$$

with the *mass matrix* M and the *stiffness matrix* L, both $2n \times 2n$. Grouping the two rows of vertex i and the two columns of vertex j, both are made of

2×2 blocks, one per pair of vertices (i, j):

$$\mathrm{M}_{ij} = \Big[\int_\Omega \Psi_{j,l} \cdot \Psi_{i,k} \, dA \Big]_{k,l}, \qquad \mathrm{L}_{ij} = \Big[\int_\Omega \nabla\Psi_{j,l} : \nabla\Psi_{i,k} \, dA \Big]_{k,l}. \tag{10.4}$$

So the row index belongs to the test function and the column index to the unknown.

The Galerkin equations as a minimization. The weak form was the condition that the energy of Stage 3 be stationary, (7.1) again:

$$J(U) = \frac{1}{2} \int_\Omega |U|^2 \, dA + \frac{\Delta t}{2} \int_\Omega |\nabla U|^2 \, dA - \int_\Omega U^0 \cdot U \, dA. \tag{7.1}$$

The Galerkin equations have the same reading in the finite space. Substitute $U = \sum_{i,k} u_{i,k} \Psi_{i,k}$ in the first term, writing $|U|^2 = U \cdot U$ and using the letters (i, k) for one factor and (j, l) for the other:

$$\begin{aligned}
&\int_\Omega |U|^2 \, dA \\
&\quad = \int_\Omega \Big(\sum_{i,k} u_{i,k} \Psi_{i,k} \Big) \cdot \Big(\sum_{j,l} u_{j,l} \Psi_{j,l} \Big) \, dA \\
&\quad = \int_\Omega \sum_{i,k} \sum_{j,l} u_{i,k} \, u_{j,l} \, \Psi_{i,k} \cdot \Psi_{j,l} \, dA \quad (\cdot \text{ distributes over both sums}) \\
&\quad = \sum_{i,k} \sum_{j,l} u_{i,k} \Big(\int_\Omega \Psi_{j,l} \cdot \Psi_{i,k} \, dA \Big) u_{j,l} \quad (\text{sums and numbers out of } \textstyle\int) \\
&\quad = \sum_{i,k} \sum_{j,l} u_{i,k} \, \mathrm{M}_{(i,k),(j,l)} \, u_{j,l} = \mathrm{u}^T \mathrm{M} \, \mathrm{u} \quad (\text{definition of M}).
\end{aligned}$$

The same four lines with ∇ (which is linear) and : in place of $\cdot$ give $\int_\Omega |\nabla U|^2 \, dA = \mathrm{u}^T \mathrm{L} \, \mathrm{u}$, and with the sum for U^0 in the first factor they give $\int_\Omega U^0 \cdot U \, dA = \mathrm{u}^T \mathrm{M} \, \mathrm{u}^0$ (here we use that M is symmetric, which holds because $\Psi_{j,l} \cdot \Psi_{i,k} = \Psi_{i,k} \cdot \Psi_{j,l}$). Substituting these three expressions into the three terms of (7.1):

$$\begin{aligned}
J(\mathrm{u}) &= \tfrac{1}{2} \left(\mathrm{u}^T \mathrm{M} \, \mathrm{u} \right) + \tfrac{\Delta t}{2} \left(\mathrm{u}^T \mathrm{L} \, \mathrm{u} \right) - \left(\mathrm{u}^T \mathrm{M} \, \mathrm{u}^0 \right) \\
&= \tfrac{1}{2} \, \mathrm{u}^T (\mathrm{M} + \Delta t \, \mathrm{L}) \, \mathrm{u} - \mathrm{u}^T \mathrm{M} \, \mathrm{u}^0 \quad (\text{the first two share } \tfrac{1}{2} \mathrm{u}^T \cdots \mathrm{u}).
\end{aligned}$$

On the finite space J is a function of the $2n$ numbers u. Its minimum is where its partial derivatives vanish. For a symmetric matrix A and a vector c, write the two kinds of terms entry by entry and differentiate with respect to one

entry u_p; the product rule hits each of the two factors u_q and u_r once:

$$\frac{\partial}{\partial u_p}\Big(\frac{1}{2}\sum_q\sum_r u_q\,\mathrm{A}_{qr}\,u_r\Big) = \frac{1}{2}\sum_r \mathrm{A}_{pr}\,u_r + \frac{1}{2}\sum_q u_q\,\mathrm{A}_{qp}$$
$$= \frac{1}{2}(\mathrm{Au})_p + \frac{1}{2}(\mathrm{Au})_p = (\mathrm{Au})_p \quad (\mathrm{A}_{qp} = \mathrm{A}_{pq}),$$
$$\frac{\partial}{\partial u_p}\Big(\sum_q u_q\,\mathrm{c}_q\Big) = \mathrm{c}_p.$$

With $\mathrm{A} = \mathrm{M}+\Delta t\,\mathrm{L}$ and $\mathrm{c} = \mathrm{M}\,\mathrm{u}^0$, the gradient of J is $(\mathrm{M}+\Delta t\,\mathrm{L})\,\mathrm{u}-\mathrm{M}\,\mathrm{u}^0$, and setting it to zero gives (10.3) $[\,(\mathrm{M}+\Delta t\,\mathrm{L})\,\mathrm{u} = \mathrm{M}\,\mathrm{u}^0\,]$. The Galerkin solution is the minimizer of the same energy over the finite space: the best piecewise linear tangent field in the sense of J. Testing with the same functions as the unknowns is what makes the matrix of the system symmetric, as the matrix of second derivatives of an energy must be.

Sparsity. Each integral is a sum over faces, and a face contributes to the block (i, j) only if it contains both i and j, because the hats vanish elsewhere. So the matrices are sparse, with a nonzero block for each vertex and for each pair of vertices joined by an edge: about seven blocks per row on a typical mesh, out of n. And they can be computed face by face, which is the next stage.

What Stage 6 produced. The linear system (10.3) $[\,(\mathrm{M}+\Delta t\,\mathrm{L})\,\mathrm{u} = \mathrm{M}\,\mathrm{u}^0\,]$, symmetric, sparse and positive definite, whose matrices are made of the integrals (10.4) of products of basis functions and of their covariant derivatives. By Céa's lemma of Stage 2 its solution is the best field of V_h in the energy norm, and by Stage 3 it is the minimizer of J over V_h. What is still missing is the value of every integral; that is Stage 7.

11. Stage 7. The Element Integrals

What we want. The numbers in the matrices of Stage 6, (10.4) again:

$$\mathrm{M}_{ij} = \Big[\int_\Omega \Psi_{j,l}\cdot\Psi_{i,k}\,dA\Big]_{k,l}, \qquad \mathrm{L}_{ij} = \Big[\int_\Omega \nabla\Psi_{j,l} : \nabla\Psi_{i,k}\,dA\Big]_{k,l}. \quad (10.4)$$

Every integral over the mesh is the sum of the integrals over its faces, and on one face only the basis functions of its three vertices are nonzero. So the work splits into small pieces, one per face: the *element integrals*. This stage computes them in three steps. Here we substitute the covariant derivatives of the basis functions into the integrand of L_{ij} and find the stiffness block of a face in terms of the hat gradients g_i^f. Section 12 computes those gradients on a reference triangle, with the chain rule and the affine map to the face. Section 13 collects the blocks of a face into its element matrices, stiffness and mass, with the code that computes them.

The stiffness block of a face. Fix a face f that contains the two vertices i and j. Its contribution to the block L_{ij} is the 2×2 matrix of the integrals over f alone,

$$\mathrm{L}_{ij}^f = \Big[\int_f \nabla\Psi_{j,l} : \nabla\Psi_{i,k}\, dA \Big]_{k,l}.$$

On the face, the covariant derivative of a basis function is (9.3) [$T_f^T \nabla U|_f = \sum_{i \in f} (R_i^f u_i)\,(g_i^f)^T$] with u_i replaced by ε_k and the coordinates of the other two vertices set to zero (Section 9.6):

$$T_f^T \nabla\Psi_{i,k}\big|_f = (R_i^f \varepsilon_k)(g_i^f)^T, \qquad T_f^T \nabla\Psi_{j,l}\big|_f = (R_j^f \varepsilon_l)(g_j^f)^T,$$

two rank-one 2×2 matrices, each a column times a row. The inner product : of two covariant derivatives can be computed with their matrices in the face frame, because the frame is orthonormal (Section 5), and it is the sum of the products of the entries, (5.1) [$A : B = \sum_{p,q} A_{pq} B_{pq}$].

The inner product of two rank-one matrices. Take four vectors a, b, c, d of $\mathbb{R}^2$. The entry (p, q) of ab^T is $a_p b_q$, and that of cd^T is $c_p d_q$. Then

$$\begin{aligned}
(ab^T) : (cd^T) &= \sum_p \sum_q (a_p b_q)(c_p d_q) \quad \text{(definition of :)} \\
&= \sum_p \sum_q (a_p c_p)(b_q d_q) \quad \text{(reorder the four numbers)} \\
&= \Big(\sum_p a_p c_p \Big) \Big(\sum_q b_q d_q \Big) \quad \text{(the } p \text{ factor does not depend on } q\text{)} \\
&= (a \cdot c)\,(b \cdot d).
\end{aligned}$$

The columns are paired with the columns and the rows with the rows.

Substituting. Put $a = R_j^f \varepsilon_l$, $b = g_j^f$, $c = R_i^f \varepsilon_k$, $d = g_i^f$:

$$\begin{aligned}
\nabla\Psi_{j,l} : \nabla\Psi_{i,k} &= \big((R_j^f \varepsilon_l)(g_j^f)^T\big) : \big((R_i^f \varepsilon_k)(g_i^f)^T\big) \\
&= \big((R_j^f \varepsilon_l) \cdot (R_i^f \varepsilon_k)\big)\,\big(g_j^f \cdot g_i^f\big) \quad \text{(the identity above)} \\
&= (g_i^f \cdot g_j^f)\,(R_i^f \varepsilon_k) \cdot (R_j^f \varepsilon_l) \quad (x \cdot y = y \cdot x).
\end{aligned}$$

The first factor is the inner product of the two hat gradients; the second is the inner product of the two transported frame vectors. Write the second as a matrix product, one step per line:

$$\begin{aligned}
(R_i^f \varepsilon_k) \cdot (R_j^f \varepsilon_l) &= (R_i^f \varepsilon_k)^T (R_j^f \varepsilon_l) \quad (x \cdot y = x^T y) \\
&= \varepsilon_k^T (R_i^f)^T R_j^f\, \varepsilon_l \quad ((XY)^T = Y^T X^T) \\
&= \big((R_i^f)^T R_j^f\big)_{kl} \quad (\varepsilon_k^T X \varepsilon_l = X_{kl}).
\end{aligned}$$

The last line holds for any 2×2 matrix X: $X\varepsilon_l$ is column l of X, and ε_k^T picks its entry k. So the integrand is

$$\nabla\Psi_{j,l} : \nabla\Psi_{i,k} = (g_i^f \cdot g_j^f)\,\big((R_i^f)^T R_j^f\big)_{kl} \qquad \text{on } f.$$

Integrating. Every factor of the integrand is constant on the face: g_i^f and g_j^f because the hats are linear there, and R_i^f, R_j^f because they are fixed matrices of the face. A constant comes out of the integral, and the integral of 1 over the face is its area:

$$\begin{aligned}\int_f \nabla\Psi_{j,l} : \nabla\Psi_{i,k}\, dA &= \int_f (g_i^f \cdot g_j^f)\left((R_i^f)^T R_j^f\right)_{kl} dA \\ &= (g_i^f \cdot g_j^f)\left((R_i^f)^T R_j^f\right)_{kl} \int_f dA \quad \text{(constant out)} \\ &= A_f\,(g_i^f \cdot g_j^f)\left((R_i^f)^T R_j^f\right)_{kl} \quad \Big(\textstyle\int_f dA = A_f\Big).\end{aligned}$$

Collecting the four entries $k, l \in \{1, 2\}$ into one 2×2 matrix, the stiffness block of the face is

$$\boxed{\mathrm{L}_{ij}^f = A_f\,(g_i^f \cdot g_j^f)\,(R_i^f)^T R_j^f} \tag{11.1}$$

The scalar factor $A_f\, g_i^f \cdot g_j^f$ is the entry (i, j) of the stiffness matrix of the ordinary scalar Laplacian. The 2×2 factor $(R_i^f)^T R_j^f$ is, by (4.8) $[\, R_{j\to i}^f = (R_i^f)^T R_j^f\,]$, the parallel transport from vertex j to vertex i through the face. So the connection Laplacian is the scalar Laplacian in which every coupling between two vertices first transports the vector of one into the frame of the other. For $i = j$ the transport is the identity, $(R_i^f)^T R_i^f = I$, and the diagonal block is the scalar diagonal entry times I. On a flat mesh with a common frame, every $R_i^f = I$, and the connection Laplacian is two copies of the scalar Laplacian, one per coordinate.

The discrete Laplacian compares a vector with its transported neighbors. Section 5 said that the Laplacian measures how much a value exceeds the average of its neighbors, and that for vectors the neighbors must first be transported. The face block (11.1),

$$\mathrm{L}_{ij}^f = A_f\,(g_i^f \cdot g_j^f)\,(R_i^f)^T R_j^f, \tag{11.1}$$

does exactly this. Write $w_{ij}^f = -A_f\, g_i^f \cdot g_j^f$ for $j \neq i$. The three hats of the face add up to 1, so their gradients add up to the gradient of a constant, which is zero, and $g_i^f = -\sum_{j\neq i} g_j^f$, the sum running over the other two vertices of the face. Substituting this, in parentheses, into one of the two factors of the diagonal factor:

$$\begin{aligned}A_f\,|g_i^f|^2 &= A_f\, g_i^f \cdot \Big(-\sum_{j\neq i} g_j^f\Big) \\ &= \sum_{j\neq i}\big(-A_f\, g_i^f \cdot g_j^f\big) \quad \text{(distribute)} \\ &= \sum_{j\neq i} w_{ij}^f \quad \text{(definition of } w_{ij}^f\text{)}.\end{aligned}$$

The contribution of the face to row i of $\mathrm{L}\,\mathrm{u}$ is $\sum_{j\in f} \mathrm{L}_{ij}^f u_j$. Separate the term $j=i$, substitute the block, and use $(R_i^f)^T R_i^f = I$ in that term:

$$\begin{aligned}\sum_{j\in f} \mathrm{L}_{ij}^f u_j &= A_f |g_i^f|^2\, u_i + \sum_{j\neq i} A_f\,(g_i^f\cdot g_j^f)\,(R_i^f)^T R_j^f\, u_j \\ &= \Big(\sum_{j\neq i} w_{ij}^f\Big) u_i + \sum_{j\neq i} \big(-w_{ij}^f\big)\,(R_i^f)^T R_j^f\, u_j \quad \text{(substitute)} \\ &= \sum_{j\neq i} w_{ij}^f \big(u_i - (R_i^f)^T R_j^f\, u_j\big) \quad \text{(collect each } j\text{)}:\end{aligned}$$

a weighted sum of the differences between the vector at i and the vectors of its neighbors *transported to* i. If every neighbor, transported, equals u_i, the row is zero, whatever the frames: the field is as parallel as it can be there.

In one variable. The same computation on a line makes the connection with the averaging of Section 5 explicit. Take hats on nodes with constant spacing h. The hat ϕ_i has slope $1/h$ on the interval to the left of x_i and $-1/h$ on the interval to the right; ϕ_{i+1} has slope $1/h$ on the interval between x_i and x_{i+1}, the only interval that the two share. So

$$\int \phi_i'^2\, dx = \Big(\frac{1}{h}\Big)^2 h + \Big(-\frac{1}{h}\Big)^2 h = \frac{2}{h},$$
$$\int \phi_i'\phi_{i+1}'\, dx = \Big(-\frac{1}{h}\Big)\Big(\frac{1}{h}\Big)\, h = -\frac{1}{h},$$

and likewise $\int \phi_i'\phi_{i-1}'\, dx = -\frac{1}{h}$. Row i of the stiffness matrix applied to u is therefore

$$-\frac{1}{h}u_{i-1} + \frac{2}{h}u_i - \frac{1}{h}u_{i+1} = \frac{2}{h}\Big(u_i - \frac{u_{i-1}+u_{i+1}}{2}\Big),$$

the excess of the value over the average of its two neighbors. The lumped mass of the node is h (half of each of its two intervals), and dividing by it gives

$$\frac{1}{h}\cdot\frac{1}{h}\big(2u_i - u_{i-1} - u_{i+1}\big) = -\frac{u_{i+1}-2u_i+u_{i-1}}{h^2},$$

which by the Taylor expansion of Section 5 is $-u''(x_i)$ up to terms of order h^2. On a surface the neighbors are the vertices of the one-ring, the weights are the cotangent weights of Section 12, and each neighbor is transported to i before it is compared.

Symmetry and positivity. The block for (j,i) is $A_f(g_j^f\cdot g_i^f)(R_j^f)^T R_i^f$. Transposing the block for (i,j) with $(XY)^T = Y^T X^T$ gives the same:

$$\big(A_f(g_i^f\cdot g_j^f)(R_i^f)^T R_j^f\big)^T = A_f(g_i^f\cdot g_j^f)(R_j^f)^T\big((R_i^f)^T\big)^T = A_f(g_j^f\cdot g_i^f)(R_j^f)^T R_i^f.$$

So the block (j,i) is the transpose of the block (i,j), and the matrix is symmetric. It is also positive semi-definite, as the continuous operator is. For a vector of coefficients u with field $U = \sum u_{i,k}\Psi_{i,k}$, the quadratic form is,

entry by entry,

$$\begin{aligned}
\mathrm{u}^T \mathrm{L}\, \mathrm{u} &= \sum_{i,k} \sum_{j,l} u_{i,k} \Big(\int_\Omega \nabla \Psi_{j,l} : \nabla \Psi_{i,k}\, dA \Big) u_{j,l} \\
&= \int_\Omega \sum_{i,k} \sum_{j,l} \big(u_{i,k} \nabla \Psi_{i,k} \big) : \big(u_{j,l} \nabla \Psi_{j,l} \big)\, dA \quad \text{(sums and numbers in)} \\
&= \int_\Omega \Big(\sum_{i,k} u_{i,k} \nabla \Psi_{i,k} \Big) : \Big(\sum_{j,l} u_{j,l} \nabla \Psi_{j,l} \Big)\, dA \quad (: \text{ is bilinear}) \\
&= \int_\Omega \nabla U : \nabla U\, dA = \int_\Omega |\nabla U|^2\, dA \geq 0 \quad (\nabla \text{ is linear}).
\end{aligned}$$

The quadratic form of the stiffness matrix is twice the Dirichlet energy (5.1) $[\, E(U) = \frac{1}{2} \int |\nabla U|^2\, dA \,]$ of the discrete field.

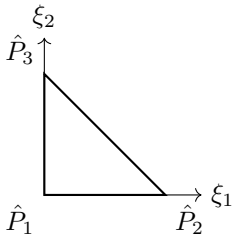
What remains. The block (11.1) is known once the hat gradients g_i^f of each face are known; the transports R_i^f came from Stage 5. The next section computes the gradients, the one piece of Stage 7 that is the same as in the planar finite element method, except that the triangle lies in space.

12. Stage 7, Continued: Shape Functions and Their Gradients on a Triangle in Space

What we want. The element matrices need the constant gradients g_i^f of the three hat functions on each face. A hat function on a face is linear and is known by its three vertex values, so its gradient is determined by the geometry of the triangle; the question is how to compute it conveniently. The standard way, which the code follows, is to do the work once, on a fixed *reference triangle*, and to carry the result to every face of the mesh with an affine map [10]. The only tool needed is the chain rule, which we will derive on the spot in the simple form it takes for an affine map. The new feature, compared with textbooks on the finite element method, is that the face is a triangle in space, not in the plane, so the map is not square and has no inverse.

12.1. The reference triangle

The reference triangle $\hat{K}$ has the vertices $\hat{P}_1 = (0,0)$, $\hat{P}_2 = (1,0)$, $\hat{P}_3 = (0,1)$ in the plane of the coordinates $\xi = (\xi_1, \xi_2)$:



It consists of the points with $\xi_1 \geq 0$, $\xi_2 \geq 0$ and $\xi_1 + \xi_2 \leq 1$, and its area is $\frac{1}{2}$. Its three linear *shape functions*, each equal to 1 at one vertex and 0 at the other two, are

$$\hat{\phi}_1(\xi) = 1 - \xi_1 - \xi_2, \qquad \hat{\phi}_2(\xi) = \xi_1, \qquad \hat{\phi}_3(\xi) = \xi_2, \tag{12.1}$$

as the nine values at the three vertices below show: $\hat{\phi}_a(\hat{P}_b) = \delta_{ab}$ (which is 1 if $a = b$ and 0 otherwise). The nine values are

$$\begin{aligned}
&\hat{\phi}_1(0,0) = 1 - 0 - 0 = 1, \quad \hat{\phi}_1(1,0) = 1 - 1 = 0, \quad \hat{\phi}_1(0,1) = 1 - 1 = 0,\\
&\hat{\phi}_2(0,0) = 0, \qquad\qquad \hat{\phi}_2(1,0) = 1, \qquad\qquad \hat{\phi}_2(0,1) = 0,\\
&\hat{\phi}_3(0,0) = 0, \qquad\qquad \hat{\phi}_3(1,0) = 0, \qquad\qquad \hat{\phi}_3(0,1) = 1.
\end{aligned}$$

They add up to 1, as hat functions must: $(1 - \xi_1 - \xi_2) + \xi_1 + \xi_2 = 1$. Their gradients are the vectors of their two partial derivatives (4.2) $[\,\nabla f = (\partial f/\partial x_1, \partial f/\partial x_2)\,]$. For $\hat{\phi}_1$, $\partial(1-\xi_1-\xi_2)/\partial\xi_1 = -1$ and $\partial(1-\xi_1-\xi_2)/\partial\xi_2 = -1$; for $\hat{\phi}_2 = \xi_1$ the two partial derivatives are 1 and 0; for $\hat{\phi}_3 = \xi_2$ they are 0 and 1. So the gradients are constant:

$$\hat{\nabla}\hat{\phi}_1 = (-1,-1), \qquad \hat{\nabla}\hat{\phi}_2 = (1,0), \qquad \hat{\nabla}\hat{\phi}_3 = (0,1). \tag{12.2}$$

The hat on $\hat{\nabla}$ is a reminder that these are derivatives with respect to the reference coordinates ξ.

12.2. The map to a face in space

The affine map that takes the reference triangle onto a face f with vertices x_1, x_2, x_3 (local numbering, in the order of the face) is

$$F(\xi) = x_1 + B\,\xi, \qquad B = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \end{bmatrix} \in \mathbb{R}^{3\times 2}. \tag{12.3}$$

Written out, $F(\xi) = x_1 + \xi_1 e_1 + \xi_2 e_2$, with the edges $e_1 = x_2 - x_1$ and $e_2 = x_3 - x_1$ of the face, the columns of B. It takes $\hat{P}_1, \hat{P}_2, \hat{P}_3$ to x_1, x_2, x_3: for instance $F(1,0) = x_1 + e_1 = x_2$. It takes the edges $\hat{e}_1, \hat{e}_2$ of the reference triangle to the edges e_1 and e_2 of the face (Figure 10), and the midpoint $(\frac{1}{2}, \frac{1}{2})$ of the third reference edge to $x_1 + \frac{1}{2}e_1 + \frac{1}{2}e_2 = \frac{1}{2}(x_2 + x_3)$, the midpoint of the third edge of the face. So the numbers ξ_1, ξ_2 are coordinates on the face, measured along its two edges: every point of the face is $F(\xi)$ for exactly one ξ of $\hat{K}$.

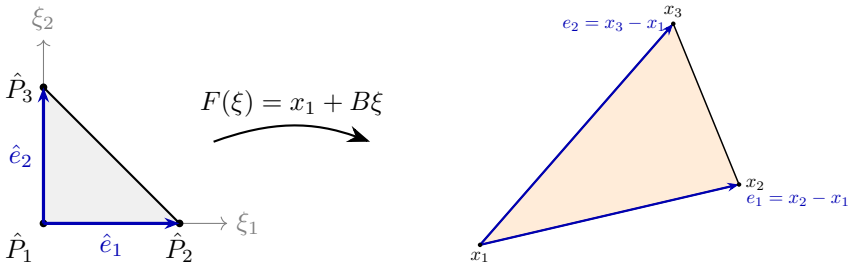


FIGURE 10. The affine map from the reference triangle to a face in space. The columns of B are the images of the reference edges.

The hats are the shape functions in other coordinates. The function $\phi_a(F(\xi))$, the hat of vertex a evaluated at the point with coordinates ξ, is linear in ξ (a linear function composed with an affine map), and it takes the values 1 at the vertex a and 0 at the other two, like $\hat{\phi}_a$. A linear function is determined by its three vertex values, so

$$\phi_a(F(\xi)) = \hat{\phi}_a(\xi) \qquad \text{for every } \xi \in \hat{K}.$$

In the plane we would write $\phi_a = \hat{\phi}_a \circ F^{-1}$ and differentiate, inverting the 2×2 matrix B. Here B is 3×2: it maps the plane of ξ onto the plane of the face, which is a plane in space, and has no inverse. We do not need one. What we need is the gradient of ϕ_a, a vector in the plane of the face, and the identity above, differentiated, determines it.

The chain rule for an affine map. We want the derivative of the left side, $\phi_a(F(\xi))$, with respect to ξ_m, $m \in \{1, 2\}$. A partial derivative is a directional derivative along an axis: increase ξ_m by s, keep the other coordinate fixed, and differentiate at $s = 0$,

$$\frac{\partial}{\partial \xi_m}\, \phi_a(F(\xi)) = \frac{d}{ds}\, \phi_a\big(F(\xi + s\,\varepsilon_m)\big)\Big|_{s=0}.$$

First see where the point goes. Substitute $\xi + s\,\varepsilon_m$ into (12.3) [$F(\xi) = x_1 + B\xi$]:

$$\begin{aligned} F(\xi + s\,\varepsilon_m) &= x_1 + B\,(\xi + s\,\varepsilon_m) \\ &= x_1 + B\xi + s\,B\varepsilon_m \quad \text{(distribute } B\text{)} \\ &= F(\xi) + s\,e_m \quad (x_1 + B\xi = F(\xi);\ B\varepsilon_m \text{ is column } m \text{ of } B). \end{aligned}$$

Because F is affine, the point moves along a straight line through $F(\xi)$ in the direction of the edge e_m, exactly. Substituting this into the derivative:

$$\begin{aligned} \frac{\partial}{\partial \xi_m}\, \phi_a(F(\xi)) &= \frac{d}{ds}\, \phi_a\big(F(\xi) + s\,e_m\big)\Big|_{s=0} \\ &= e_m(\phi_a) \quad \text{(definition (4.1) of the directional derivative)} \\ &= \nabla\phi_a \cdot e_m \quad \text{(property (4.2) of the gradient)}, \end{aligned}$$

where (4.1) is $[\, X(f) = \frac{d}{ds} f(x + sX)|_{s=0} \,]$ and (4.2) is $[\, X(f) = \nabla f \cdot X \,]$, here at the point $x = F(\xi)$ and in the direction $X = e_m$. This is the *chain rule* in the case we need: the derivative of a composition is the gradient of the outer function applied to the derivative $\partial F/\partial \xi_m = e_m$ of the inner one.

Two equations for the gradient. Write $g = \nabla \phi_a$ for the unknown gradient, a vector of $\mathbb{R}^3$ in the plane of the face. Differentiate both sides of the identity $\phi_a(F(\xi)) = \hat{\phi}_a(\xi)$ with respect to ξ_1, and then with respect to ξ_2. On the left the chain rule gives $g \cdot e_1$ and $g \cdot e_2$; on the right we get the two entries of $\hat{\nabla}\hat{\phi}_a$, which we write $(\hat{\nabla}\hat{\phi}_a)_1$ and $(\hat{\nabla}\hat{\phi}_a)_2$:

$$e_1 \cdot g = (\hat{\nabla}\hat{\phi}_a)_1, \qquad e_2 \cdot g = (\hat{\nabla}\hat{\phi}_a)_2.$$

The rows of B^T are e_1^T and e_2^T, so $B^T g$ is the column of the two inner products, and the two equations are one matrix equation:

$$B^T g = \begin{pmatrix} e_1^T g \\ e_2^T g \end{pmatrix} = \begin{pmatrix} e_1 \cdot g \\ e_2 \cdot g \end{pmatrix} = \hat{\nabla}\hat{\phi}_a.$$

In words: the slope of the hat along each edge of the face equals the slope of the shape function along the corresponding reference edge.

Solving them. These are two equations for three unknowns, the components of g. The missing condition is that g lies in the plane of the face, which is spanned by the columns of B. So $g = Bc$ for some $c \in \mathbb{R}^2$, the coordinates of g along the two edges. Substitute $g = Bc$, in parentheses, into $B^T g = \hat{\nabla}\hat{\phi}_a$:

$$\begin{aligned} B^T(Bc) &= \hat{\nabla}\hat{\phi}_a \\ (B^T B)\, c &= \hat{\nabla}\hat{\phi}_a \quad \text{(associativity: a } 2 \times 2 \text{ system for } c) \\ c &= (B^T B)^{-1} \hat{\nabla}\hat{\phi}_a \quad \text{(multiply by the inverse on the left)} \\ g &= Bc = B(B^T B)^{-1} \hat{\nabla}\hat{\phi}_a \quad \text{(back to } \mathbb{R}^3). \end{aligned}$$

The Gram matrix and its inverse. The entry (m, m') of $B^T B$ is row m of B^T times column m' of B, that is $e_m^T e_{m'} = e_m \cdot e_{m'}$. So $B^T B$ is the *Gram matrix* of the two edges:

$$B^T B = \begin{pmatrix} e_1 \cdot e_1 & e_1 \cdot e_2 \\ e_1 \cdot e_2 & e_2 \cdot e_2 \end{pmatrix}.$$

A symmetric 2×2 matrix with entries p, q, q, r has the inverse

$$\begin{pmatrix} p & q \\ q & r \end{pmatrix}^{-1} = \frac{1}{pr - q^2} \begin{pmatrix} r & -q \\ -q & p \end{pmatrix},$$

because the product of the matrix with the matrix on the right is $pr - q^2$ times the identity,

$$\begin{pmatrix} p & q \\ q & r \end{pmatrix} \begin{pmatrix} r & -q \\ -q & p \end{pmatrix} = \begin{pmatrix} pr - q^2 & -pq + qp \\ qr - rq & -q^2 + rp \end{pmatrix} = \begin{pmatrix} pr - q^2 & 0 \\ 0 & pr - q^2 \end{pmatrix}$$

With $p = e_1 \cdot e_1 = |e_1|^2$, $q = e_1 \cdot e_2$, $r = |e_2|^2$, the determinant is $|e_1|^2 |e_2|^2 - (e_1 \cdot e_2)^2$. Write θ for the angle between the two edges, so that $e_1 \cdot e_2 = |e_1||e_2| \cos\theta$

and $|e_1 \times e_2| = |e_1||e_2| \sin\theta$. Then

$$\begin{aligned}|e_1|^2|e_2|^2 - (e_1 \cdot e_2)^2 &= |e_1|^2|e_2|^2 - |e_1|^2|e_2|^2 \cos^2\theta \\ &= |e_1|^2|e_2|^2\,(1 - \cos^2\theta) \quad \text{(factor out)} \\ &= |e_1|^2|e_2|^2 \sin^2\theta = |e_1 \times e_2|^2 \quad (\sin^2 + \cos^2 = 1) \\ &= (2A_f)^2 = 4A_f^2 \quad \text{(by (3.2), } A_f = \tfrac{1}{2}|e_1 \times e_2|).\end{aligned}$$

This is Lagrange's identity. The determinant is positive for any triangle of positive area, so the inverse exists:

$$(B^TB)^{-1} = \frac{1}{4A_f^2}\begin{pmatrix} e_2 \cdot e_2 & -e_1 \cdot e_2 \\ -e_1 \cdot e_2 & e_1 \cdot e_1 \end{pmatrix}.$$

The three gradients written out. Now substitute this inverse and the three reference gradients (12.2) [$\hat{\nabla}\hat{\phi}_1 = (-1,-1)$, $\hat{\nabla}\hat{\phi}_2 = (1,0)$, $\hat{\nabla}\hat{\phi}_3 = (0,1)$] into $c = (B^TB)^{-1}\hat{\nabla}\hat{\phi}_a$ and $g = Bc = c_1e_1 + c_2e_2$. For $a = 2$ the inverse applied to $(1,0)$ is its first column, and for $a = 3$, applied to $(0,1)$, its second column:

$$c^{(2)} = \frac{1}{4A_f^2}\begin{pmatrix} e_2 \cdot e_2 \\ -e_1 \cdot e_2 \end{pmatrix}, \qquad \nabla\phi_2 = \frac{(e_2 \cdot e_2)\,e_1 - (e_1 \cdot e_2)\,e_2}{4A_f^2},$$

$$c^{(3)} = \frac{1}{4A_f^2}\begin{pmatrix} -e_1 \cdot e_2 \\ e_1 \cdot e_1 \end{pmatrix}, \qquad \nabla\phi_3 = \frac{(e_1 \cdot e_1)\,e_2 - (e_1 \cdot e_2)\,e_1}{4A_f^2}.$$

For $a = 1$, $(-1,-1) = -(1,0) - (0,1)$, and the map $\hat{\nabla}\hat{\phi}_a \mapsto g$ is linear, so $\nabla\phi_1 = -\nabla\phi_2 - \nabla\phi_3$. To check $\nabla\phi_2$ against its two equations, take its inner products with the edges:

$$\nabla\phi_2 \cdot e_1 = \frac{(e_2 \cdot e_2)(e_1 \cdot e_1) - (e_1 \cdot e_2)(e_2 \cdot e_1)}{4A_f^2} = \frac{4A_f^2}{4A_f^2} = 1,$$

$$\nabla\phi_2 \cdot e_2 = \frac{(e_2 \cdot e_2)(e_1 \cdot e_2) - (e_1 \cdot e_2)(e_2 \cdot e_2)}{4A_f^2} = 0,$$

which are $(\hat{\nabla}\hat{\phi}_2)_1 = 1$ and $(\hat{\nabla}\hat{\phi}_2)_2 = 0$, as required. The answer is a fixed 3×2 matrix per face, $B(B^TB)^{-1}$, applied to the three constant reference gradients.

Proposition 12.1. *On the face,* $\nabla\phi_a = B(B^TB)^{-1}\,\hat{\nabla}\hat{\phi}_a$, *and this vector lies in the plane of the face.*

The matrix $B(B^TB)^{-1}$ is the transpose of the Moore–Penrose pseudo-inverse $B^+ = (B^TB)^{-1}B^T$ of B (transpose it with $(XY)^T = Y^TX^T$ and use that B^TB is symmetric). For a triangle in the plane, where B is a square invertible 2×2 matrix, $(B^TB)^{-1} = B^{-1}B^{-T}$ and so $B(B^TB)^{-1} = BB^{-1}B^{-T} = B^{-T}$, the familiar formula of the planar finite element method. The formula for a triangle in space is the planar formula with the inverse replaced by the pseudo-inverse.

The code. The gradients are computed at the start of `AssembleStiffness-ElementEmbedded2`, which receives the three vertices of the face in the array `v`, in the order of the face (zero-based, `v[0]` is x_1):

```
347 Matrix66d AssembleStiffnessElementEmbedded2(Vertex* v[3], const Face& face) {
348
349     Eigen::MatrixXd B(3, 2);
350     Eigen::MatrixXd Binv(3, 2);
351     Eigen::Vector3d gradN[3];
352     Matrix66d elementMatrix;
353
354     B(0,0) = v[1]->p.x() - v[0]->p.x(); B(0,1) = v[2]->p.x() - v[0]->p.x();
355     B(1,0) = v[1]->p.y() - v[0]->p.y(); B(1,1) = v[2]->p.y() - v[0]->p.y();
356     B(2,0) = v[1]->p.z() - v[0]->p.z(); B(2,1) = v[2]->p.z() - v[0]->p.z();
357
358     Binv = ((B.transpose() * B).inverse() * B.transpose()).transpose();
359
360     double faceArea = 0.5 * ((v[1]->p - v[0]->p).cross(v[2]->p - v[0]->p)).norm();
361
362     //grad N^k_1(X) = B^-T (-1, -1)
363     //grad N^k_2(X) = B^-T (1, 0)
364     //grad N^k_3(X) = B^-T (0, 1)
365
366     gradN[0] = Eigen::Vector3d(-Binv(0,0) - Binv(0,1), -Binv(1,0) - Binv(1,1), -Binv
          (2,0) - Binv(2,1));
367     gradN[1] = Eigen::Vector3d( Binv(0,0), Binv(1,0), Binv(2,0));
368     gradN[2] = Eigen::Vector3d( Binv(0,1), Binv(1,1), Binv(2,1));
```

Lines 354–356 fill B row by row: row 0 holds the x components of the two edges, row 1 the y components, row 2 the z components, so that column 0 is $e_1 = x_2 - x_1$ and column 1 is $e_2 = x_3 - x_1$, as in (12.3) $[\,B = [\,x_2 - x_1 \;\; x_3 - x_1\,]\,]$. Line 358 computes `Binv` $= ((B^TB)^{-1}B^T)^T = B(B^TB)^{-1}$, the matrix of Proposition 12.1; the name and the comments of lines 362–364, which write B^{-T}, come from the planar method, of which this is the extension. Line 360 is the area $A_f = \frac{1}{2}|e_1 \times e_2|$, recomputed here rather than read from `face_areas`. Lines 366–368 apply $C =$ `Binv` to the three reference gradients (12.2) $[\,\hat{\nabla}\hat{\phi}_1 = (-1,-1),\ \hat{\nabla}\hat{\phi}_2 = (1,0),\ \hat{\nabla}\hat{\phi}_3 = (0,1)\,]$. Applying a matrix to $(1,0)$ or $(0,1)$ picks its first or second column, and applying it to $(-1,-1)$ gives minus the sum of its columns:

$$\nabla\phi_1 = -C_{:,1} - C_{:,2}, \qquad \nabla\phi_2 = C_{:,1}, \qquad \nabla\phi_3 = C_{:,2}, \qquad C = \texttt{Binv}, \quad (12.4)$$

which is `gradN[0]`, `gradN[1]`, `gradN[2]`, written component by component. (The function `Gradient`, lines 239–264, computes the same three vectors as the columns of a 3×3 matrix; it is not used by the solver.) The three gradients add up to zero, as they must, since the three hat functions add up to 1 and the gradient of a constant is zero.

The gradients as a reciprocal frame. The equations $B^Tg = \hat{\nabla}\hat{\phi}_a$ have a simple reading for $a = 2, 3$. With $\hat{\nabla}\hat{\phi}_2 = (1,0)$ and $\hat{\nabla}\hat{\phi}_3 = (0,1)$ they say

$$\nabla\phi_2 \cdot e_1 = 1, \qquad \nabla\phi_2 \cdot e_2 = 0, \qquad \nabla\phi_3 \cdot e_1 = 0, \qquad \nabla\phi_3 \cdot e_2 = 1.$$

So $\nabla\phi_2$ and $\nabla\phi_3$ are the two vectors of the plane of the face that are dual to the edges: each has inner product 1 with one edge and 0 with the other. Such a pair is called the *reciprocal frame* of (e_1, e_2), written e^1, e^2 (Figure 11); e^1 is orthogonal to e_2 and e^2 is orthogonal to e_1. The reciprocal frame reads off

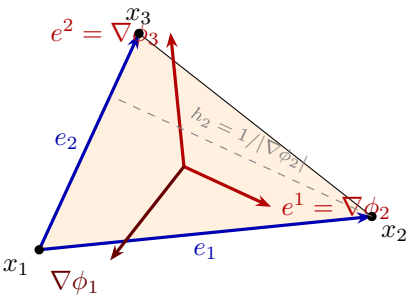


FIGURE 11. The gradients of the hat functions of a face. $\nabla\phi_2$ is orthogonal to the edge opposite to x_2, points towards x_2, and has length $1/h_2$, the reciprocal of the height; the gradients $\nabla\phi_2 = e^1$ and $\nabla\phi_3 = e^2$ form the *reciprocal frame* of the edges e_1, e_2 ($e^k \cdot e_m = \delta_{km}$), and the three gradients add up to zero.

the edge coordinates of a point: a point of the face is $y = x_1 + \xi_1 e_1 + \xi_2 e_2$, and $e^1 \cdot (y - x_1) = \xi_1$, $e^2 \cdot (y - x_1) = \xi_2$. Since $\phi_2 = \xi_1$ and $\phi_3 = \xi_2$ on the face, $\phi_2(y) = e^1 \cdot (y - x_1)$ and $\phi_3(y) = e^2 \cdot (y - x_1)$, linear functions with the gradients e^1 and e^2, which is the same result seen from another side. Part II builds the gradients in this way.

Face coordinates. The element matrix will combine the gradients with the transported vectors $R_i^f u_i$, which are stored by their coordinates in the face frame. So the gradients are expressed in the same coordinates:

$$g_a^f = T_f^T \nabla\phi_a \in \mathbb{R}^2. \tag{12.5}$$

In the code this is line 371, `Tf = face_tangent_space[face.ID]`, and then `grad_i = Tf.transpose() * gradN[i]` (lines 377–378, shown in Section 13). Changing to face coordinates does not change inner products of gradients:

$$g_a^f \cdot g_b^f = (T_f^T \nabla\phi_a)^T (T_f^T \nabla\phi_b) = \nabla\phi_a^T \, (T_f T_f^T) \, \nabla\phi_b = \nabla\phi_a \cdot \nabla\phi_b,$$

because $T_f T_f^T$ is the projection onto the plane of the face, (3.1) [$T = [\,t \;\; b\,]$, $T^T T = I_2$, $TT^T = I_3 - nn^T$], and $\nabla\phi_b$ already lies in that plane. So the projection does not change the stiffness; it only expresses the gradients in the same coordinates as the transported vectors.

Worked example 12.2. *Take the triangle* $x_1 = (0,0,0)$, $x_2 = (2,0,0)$, $x_3 = (0,1,1)$ *of Figure 12. Its edges are* $e_1 = x_2 - x_1 = (2,0,0)$ *and* $e_2 = x_3 - x_1 = (0,1,1)$. *The entries of the Gram matrix are*

$$e_1 \cdot e_1 = 4, \qquad e_1 \cdot e_2 = 2 \cdot 0 + 0 \cdot 1 + 0 \cdot 1 = 0, \qquad e_2 \cdot e_2 = 0 + 1 + 1 = 2,$$

and its determinant is $4 \cdot 2 - 0^2 = 8$. *The cross product is* $e_1 \times e_2 = (0 \cdot 1 - 0 \cdot 1,\ 0 \cdot 0 - 2 \cdot 1,\ 2 \cdot 1 - 0 \cdot 0) = (0, -2, 2)$, *of length* $\sqrt{8}$, *so* $A_f = \frac{1}{2}\sqrt{8} = \sqrt{2}$

and $4A_f^2 = 8$*, the determinant, as Lagrange's identity says. So*

$$B = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}, \quad B^T B = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}, \quad (B^T B)^{-1} = \frac{1}{8}\begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix} = \begin{pmatrix} 1/4 & 0 \\ 0 & 1/2 \end{pmatrix},$$

and

$$B(B^T B)^{-1} = \begin{pmatrix} 2\cdot\frac{1}{4} & 0 \\ 0 & 1\cdot\frac{1}{2} \\ 0 & 1\cdot\frac{1}{2} \end{pmatrix} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \\ 0 & 1/2 \end{pmatrix}.$$

The same comes from the formulas for $\nabla\phi_2$ *and* $\nabla\phi_3$ *written out above, with* $e_1 \cdot e_2 = 0$*:* $\nabla\phi_2 = \frac{2\,e_1 - 0\,e_2}{8} = \frac{e_1}{4} = (\frac{1}{2}, 0, 0)$ *and* $\nabla\phi_3 = \frac{4\,e_2 - 0\,e_1}{8} = \frac{e_2}{2} = (0, \frac{1}{2}, \frac{1}{2})$*. The edges are orthogonal, so the reciprocal frame is just* $e_k/|e_k|^2$*. By (12.4)* $[\nabla\phi_1 = -C_{:,1} - C_{:,2},\ \nabla\phi_2 = C_{:,1},\ \nabla\phi_3 = C_{:,2}\,]$*,*

$$\nabla\phi_1 = \left(-\tfrac{1}{2}, -\tfrac{1}{2}, -\tfrac{1}{2}\right), \qquad \nabla\phi_2 = \left(\tfrac{1}{2}, 0, 0\right), \qquad \nabla\phi_3 = \left(0, \tfrac{1}{2}, \tfrac{1}{2}\right),$$

which add up to zero. Check $\nabla\phi_2$*: along* e_1*,* ϕ_2 *goes from* 0 *to* 1*, and* $\nabla\phi_2 \cdot e_1 = \frac{1}{2}\cdot 2 = 1$*; along* e_2 *it stays* 0*, and* $\nabla\phi_2 \cdot e_2 = 0$*. The normal is* $n_f = (0, -2, 2)/\sqrt{8} = (0, -1, 1)/\sqrt{2}$*. The face frame of* `computeFaceTangentSpace` *is* $t_f = e_1/|e_1| = (1, 0, 0)$ *and*

$$b_f = n_f \times t_f = \tfrac{1}{\sqrt{2}}\big((-1)\cdot 0 - 1\cdot 0,\ 1\cdot 1 - 0\cdot 0,\ 0\cdot 0 - (-1)\cdot 1\big) = \tfrac{1}{\sqrt{2}}(0, 1, 1).$$

The face coordinates (12.5) $[g_a^f = T_f^T \nabla\phi_a\,]$ *are the two inner products* $(t_f \cdot \nabla\phi_a,\ b_f \cdot \nabla\phi_a)$*; for instance* $b_f \cdot \nabla\phi_1 = \frac{1}{\sqrt{2}}(0 - \frac{1}{2} - \frac{1}{2}) = -\frac{1}{\sqrt{2}}$*. The three are*

$$g_1^f = \left(-\tfrac{1}{2}, -\tfrac{1}{\sqrt{2}}\right), \qquad g_2^f = \left(\tfrac{1}{2}, 0\right), \qquad g_3^f = \left(0, \tfrac{1}{\sqrt{2}}\right),$$

with $1/\sqrt{2} \approx 0.7071$*. The inner products of the gradients are the same in either form:*

$$\begin{aligned} &\nabla\phi_1 \cdot \nabla\phi_1 = \tfrac{1}{4} + \tfrac{1}{4} + \tfrac{1}{4} = \tfrac{3}{4}, && |g_1^f|^2 = \tfrac{1}{4} + \tfrac{1}{2} = \tfrac{3}{4}, \\ &\nabla\phi_1 \cdot \nabla\phi_2 = -\tfrac{1}{4} + 0 + 0 = -\tfrac{1}{4}, && g_1^f \cdot g_2^f = -\tfrac{1}{4} + 0 = -\tfrac{1}{4}, \\ &\nabla\phi_1 \cdot \nabla\phi_3 = 0 - \tfrac{1}{4} - \tfrac{1}{4} = -\tfrac{1}{2}, && g_1^f \cdot g_3^f = 0 - \tfrac{1}{2} = -\tfrac{1}{2}, \\ &\nabla\phi_2 \cdot \nabla\phi_2 = \tfrac{1}{4}, \quad \nabla\phi_3 \cdot \nabla\phi_3 = \tfrac{1}{2}, && \nabla\phi_2 \cdot \nabla\phi_3 = 0. \end{aligned}$$

Multiplying by $A_f = \sqrt{2}$*, the scalar stiffness* $A_f\,\nabla\phi_a \cdot \nabla\phi_b$ *of the face is*

$$\sqrt{2}\begin{pmatrix} 3/4 & -1/4 & -1/2 \\ -1/4 & 1/4 & 0 \\ -1/2 & 0 & 1/2 \end{pmatrix} = \begin{pmatrix} 1.0607 & -0.3536 & -0.7071 \\ -0.3536 & 0.3536 & 0 \\ -0.7071 & 0 & 0.7071 \end{pmatrix},$$

whose rows add up to zero, since constants have zero gradient: $\frac{3}{4} - \frac{1}{4} - \frac{1}{2} = 0$*,* $-\frac{1}{4} + \frac{1}{4} + 0 = 0$*,* $-\frac{1}{2} + 0 + \frac{1}{2} = 0$*. The entry* $(2, 3)$ *vanishes because the angle at* x_1 *is a right angle* $(e_1 \cdot e_2 = 0)$*, in agreement with the cotangent formula of the next subsection. The entry* $(1, 2)$ *is* $-\frac{1}{2}\cot\theta_3$*, with the angle* θ_3 *at* x_3*, between* $x_1 - x_3 = (0, -1, -1)$ *and* $x_2 - x_3 = (2, -1, -1)$*:*

$$\cos\theta_3 = \frac{(x_1 - x_3)\cdot(x_2 - x_3)}{|x_1 - x_3|\,|x_2 - x_3|} = \frac{0 + 1 + 1}{\sqrt{2}\,\sqrt{6}} = \frac{1}{\sqrt{3}},$$

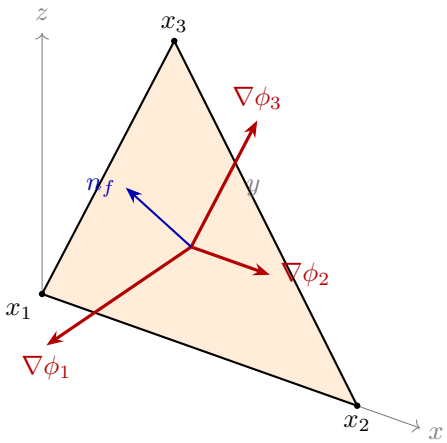


FIGURE 12. The triangle of Example 12.2, its normal $n_f = (0, -1, 1)/\sqrt{2}$, and the gradients of its three hat functions, drawn from the centroid. All three lie in the plane of the face.

$\sin\theta_3 = \sqrt{1 - \frac{1}{3}} = \frac{\sqrt{2}}{\sqrt{3}}$, *so* $\cot\theta_3 = \cos\theta_3 / \sin\theta_3 = \frac{1}{\sqrt{2}}$ *and* $-\frac{1}{2}\cot\theta_3 = -\frac{1}{2\sqrt{2}} = -\frac{\sqrt{2}}{4} = -0.3536$, *which is* $\sqrt{2} \cdot (-\frac{1}{4})$.

12.3. A check: the cotangent formula

The scalar factor of (11.1) $[\,\mathrm{L}_{ij}^f = A_f\,(g_i^f \cdot g_j^f)\,(R_i^f)^T R_j^f\,]$ has a classical closed form, which connects the finite element computation with the cotangent Laplacian of geometry processing (`CalcCotangentWeights` in `Laplacian.cpp`). It also shows that the scalar factor is intrinsic: it depends only on the angles of the triangle. We derive it from the geometric meaning of the gradient (Section 4): it is orthogonal to the level lines, and its length is the slope in the steepest direction. Name the three vertices of the face a, b, c, the angle of the triangle at each vertex $\theta_a, \theta_b, \theta_c$, and the length of the edge opposite to each vertex ℓ_a, ℓ_b, ℓ_c.

Step 1: the direction of $\nabla\phi_a$. Along the edge opposite to a, ϕ_a is constant (zero at both ends, and linear). So for a direction X along that edge, $X(\phi_a) = 0$, and by (4.2) $[\,X(f) = \nabla f \cdot X\,]$, $\nabla\phi_a \cdot X = 0$: the gradient is orthogonal to the opposite edge. It lies in the plane of the face (Proposition 12.1), so it is a multiple of the unit normal m_a of the opposite edge in the plane of the face, which we choose pointing into the triangle:

$$\nabla\phi_a = \lambda\, m_a \qquad \text{for some number } \lambda.$$

Step 2: the length of $\nabla\phi_a$. Walk from the opposite edge to a along m_a. The distance is the height h_a of the triangle from a, and ϕ_a grows linearly from 0 to 1 on the way, so its rate of growth along m_a is $1/h_a$. That rate is

the directional derivative along the unit vector m_a:

$$\begin{aligned}\frac{1}{h_a} &= m_a(\phi_a) = \nabla\phi_a \cdot m_a \quad \text{(by (4.2))} \\ &= (\lambda\, m_a)\cdot m_a = \lambda\,|m_a|^2 = \lambda \quad \text{(Step 1, and } |m_a| = 1).\end{aligned}$$

The area is half the base times the height, $A_f = \frac{1}{2}\ell_a h_a$, so $\frac{1}{h_a} = \frac{\ell_a}{2A_f}$. Substituting $\lambda = \frac{\ell_a}{2A_f}$ into Step 1:

$$\nabla\phi_a = \frac{\ell_a}{2A_f}\, m_a, \qquad \text{and likewise} \qquad \nabla\phi_b = \frac{\ell_b}{2A_f}\, m_b.$$

Step 3: the angle between m_a and m_b. The edge opposite to a is cb and the edge opposite to b is ca; they meet at c, with the angle θ_c. Measure angles in the plane of the face from the direction of $c \to b$, and orient the plane so that $c \to a$ is at the angle θ_c. The inward normal m_a of the edge cb is the direction $c \to b$ turned by $90°$ towards a, so it is at the angle $90°$. The inward normal m_b of the edge ca is the direction $c \to a$ turned by $90°$ towards b, that is, backwards, so it is at the angle $\theta_c - 90°$. The angle between them is

$$90° - (\theta_c - 90°) = 180° - \theta_c, \qquad \text{so} \qquad m_a \cdot m_b = \cos(\pi - \theta_c) = -\cos\theta_c.$$

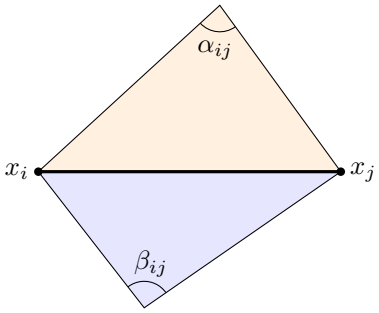
Step 4: two different vertices. Substitute Step 2, in parentheses, into the inner product, and then Step 3:

$$\begin{aligned}A_f\,\nabla\phi_a \cdot \nabla\phi_b &= A_f\left(\frac{\ell_a}{2A_f}\, m_a\right)\cdot\left(\frac{\ell_b}{2A_f}\, m_b\right) \\ &= A_f\,\frac{\ell_a\ell_b}{4A_f^2}\, m_a \cdot m_b \quad \text{(numbers out of the inner product)} \\ &= \frac{\ell_a\ell_b}{4A_f}\,(-\cos\theta_c) \quad \text{(cancel one } A_f\text{; Step 3)}.\end{aligned}$$

The two edges at c have the lengths ℓ_a and ℓ_b, and the area of a triangle is half the product of two sides times the sine of the angle between them: $A_f = \frac{1}{2}\ell_a\ell_b\sin\theta_c$, that is $\ell_a\ell_b = 2A_f/\sin\theta_c$. Substituting it, in parentheses:

$$\begin{aligned}A_f\,\nabla\phi_a \cdot \nabla\phi_b &= -\left(\frac{2A_f}{\sin\theta_c}\right)\frac{\cos\theta_c}{4A_f} \\ &= -\frac{2}{4}\,\frac{\cos\theta_c}{\sin\theta_c} \quad \text{(cancel } A_f\text{)} \\ &= -\frac{1}{2}\cot\theta_c.\end{aligned}$$

Step 5: the diagonal. The three hat functions add up to 1 on the face, so their gradients add up to zero, $\nabla\phi_a = -\nabla\phi_b - \nabla\phi_c$. Substitute it, in



$\mathrm{L}_{ij} = -\frac{1}{2}(\cot\alpha_{ij} + \cot\beta_{ij})\,(R_i^f)^T R_j^f$
(scalar part summed over the two faces)

FIGURE 13. The two angles opposite to an edge ij in its two faces. The scalar part of the stiffness block of the edge is the cotangent weight; the connection Laplacian multiplies it by the transport from j to i.

parentheses, into one factor of $|\nabla\phi_a|^2 = \nabla\phi_a \cdot \nabla\phi_a$:

$$\begin{aligned} A_f\,|\nabla\phi_a|^2 &= A_f\,\nabla\phi_a \cdot (-\nabla\phi_b - \nabla\phi_c) \\ &= -A_f\,\nabla\phi_a \cdot \nabla\phi_b - A_f\,\nabla\phi_a \cdot \nabla\phi_c \quad \text{(distribute)} \\ &= -\big(-\tfrac{1}{2}\cot\theta_c\big) - \big(-\tfrac{1}{2}\cot\theta_b\big) \quad \text{(Step 4 twice)} \\ &= \tfrac{1}{2}\cot\theta_b + \tfrac{1}{2}\cot\theta_c, \end{aligned}$$

where Step 4 is used for the pair a, b, whose third vertex is c, and for the pair a, c, whose third vertex is b.

Proposition 12.3. *For two distinct vertices a, b of a face with area A_f, $A_f\,\nabla\phi_a \cdot \nabla\phi_b = -\frac{1}{2}\cot\theta_c$, where θ_c is the angle at the third vertex c; and $A_f\,|\nabla\phi_a|^2 = \frac{1}{2}(\cot\theta_b + \cot\theta_c)$.*

In Example 12.2, the entry $(2,3)$ is $-\frac{1}{2}\cot\theta_1 = -\frac{1}{2}\cot 90^\circ = 0$. The entry $(1,3)$ is $-\frac{1}{2}\cot\theta_2$, with the angle at x_2 between $x_1 - x_2 = (-2,0,0)$ and $x_3 - x_2 = (-2,1,1)$: $\cos\theta_2 = \frac{4}{2\sqrt{6}} = \frac{2}{\sqrt{6}}$, $\sin\theta_2 = \sqrt{1 - \frac{4}{6}} = \frac{1}{\sqrt{3}}$, $\cot\theta_2 = \frac{2}{\sqrt{6}} \cdot \sqrt{3} = \sqrt{2}$, and $-\frac{1}{2}\cot\theta_2 = -0.7071$, which is $\sqrt{2}\cdot(-\frac{1}{2})$, as in the matrix. The diagonal entry $(1,1)$ is $\frac{1}{2}(\cot\theta_2 + \cot\theta_3) = \frac{1}{2}(\sqrt{2} + \frac{1}{\sqrt{2}}) = \frac{3}{2\sqrt{2}} = 1.0607$, which is $\sqrt{2}\cdot\frac{3}{4}$.

Summing over the faces of an edge. An interior edge ij belongs to two faces. In each of them the coupling of i and j is $-\frac{1}{2}\cot$ of the angle at the third vertex, the angle opposite to the edge. Call these two angles α_{ij} and β_{ij} (Figure 13). Adding the two contributions, the off-diagonal scalar stiffness of the edge is $-\frac{1}{2}(\cot\alpha_{ij} + \cot\beta_{ij})$, the *cotangent weight* of the edge. The connection Laplacian (11.1) is therefore the cotangent Laplacian with a 2×2

transport on every edge, as in the discrete connection Laplacians of geometry processing [6, 11]; the finite element derivation explains where both factors come from.

13. Stage 7, Completed: The Element Matrices

What we want. The global matrices (10.4) are integrals over the whole surface, and every integral over the surface is a sum of integrals over the faces. On one face only three hat functions are nonzero, those of its three vertices, so a face contributes only to the blocks that couple its vertices with each other. This suggests the organization of every finite element code: compute, face by face, a small *element matrix* that holds the contribution of the face, and then add the element matrices into the global one. This section computes the element matrices; Section 14 adds them up.

13.1. The element stiffness matrix

On one face, the hat functions of its three vertices are the only nonzero ones, so the face contributes to the 3×3 blocks of L indexed by its vertices, a 6×6 *element matrix* in the local numbering $a, b \in \{1, 2, 3\}$:

$$\mathrm{L}^f = \begin{pmatrix} \mathrm{L}^f_{11} & \mathrm{L}^f_{12} & \mathrm{L}^f_{13} \\ \mathrm{L}^f_{21} & \mathrm{L}^f_{22} & \mathrm{L}^f_{23} \\ \mathrm{L}^f_{31} & \mathrm{L}^f_{32} & \mathrm{L}^f_{33} \end{pmatrix}, \qquad \mathrm{L}^f_{ab} = A_f \, (g^f_a \cdot g^f_b) \, (R^f_a)^T R^f_b \in \mathbb{R}^{2\times 2}. \quad (13.1)$$

Each block is (11.1) for one pair of vertices of the face. This is `AssembleStiffnessElementEmbedded2`. Its first part, which computes B, `Binv`, A_f (`faceArea`) and the three gradients, was shown in Section 12; here is the rest, which fills the blocks:

```
369     elementMatrix.setZero();
370
371     Matrix32d Tf = face_tangent_space[face.ID];
372
373     for( int i = 0 ; i < 3 ; ++i ) { // for each test function
374         for (int j = 0 ; j < 3 ; ++j ) { // for each shape function
375             if (i < j) continue; // since stifness matrix is symmetric
376             //w_ij = area K <grad N^k_i(X), grad N^k_j(X)>
377             Eigen::Vector2d grad_i = Tf.transpose() * gradN[i];
378             Eigen::Vector2d grad_j = Tf.transpose() * gradN[j];
379             Eigen::Matrix2d R_i = Rvf(v[i], &face);
380             Eigen::Matrix2d R_j = Rvf(v[j], &face);
381
382             if(i == j) {
383                 elementMatrix.block<2,2>(i * 2, j * 2) = faceArea * grad_i.dot(grad_j) *
                        Eigen::Matrix2d::Identity();
384
385             }
386             else {
387                 elementMatrix.block<2,2>(i * 2, j * 2) = faceArea * grad_i.dot(grad_j) *
                        R_i.transpose() * R_j;
388                 elementMatrix.block<2,2>(j * 2, i * 2) = faceArea * grad_j.dot(grad_i) *
                        R_j.transpose() * R_i;
389             }
```

```
390         }
391     }
392     return elementMatrix;
393 }
```

Line 369 sets the 6×6 matrix to zero and line 371 fetches the face frame T_f. The two loops run over the local vertices, `i` for the test function (the block row a) and `j` for the unknown (the block column b), both counted from zero. Line 375 skips the pairs with $a < b$, because the matrix is symmetric and those blocks are written together with their transposes. For each remaining pair:

- lines 377–378 are the face coordinates (12.5) [$g_a^f = T_f^T \nabla \phi_a$] of the two gradients, g_a^f and g_b^f;
- lines 379–380 are the two transports R_a^f and R_b^f of (4.7) [$R_i^f = T_f^T Q_{i \to f} T_i$], computed by `Rvf`. (They are recomputed for every pair, which repeats work but does not change the result.)
- On the diagonal, $a = b$, line 383 writes $A_f \, |g_a^f|^2 \, I_2$. This is (13.1) with the product $(R_a^f)^T R_a^f$ replaced by I_2, which it equals exactly, by Proposition 4.3; the code takes the shortcut.
- Off the diagonal, line 387 writes the block (a, b), $A_f \, (g_a^f \cdot g_b^f) \, (R_a^f)^T R_b^f$, and line 388 the block (b, a), $A_f \, (g_b^f \cdot g_a^f) \, (R_b^f)^T R_a^f$, which is its transpose.

The block (a, b) starts at row $2a$ and column $2b$ of the element matrix (`block<2,2>(i * 2, j * 2)`), because each vertex has two coordinates, in the same zero-based count as the global index $2i + k$ of Section 3.

To picture the 6×6 matrix, consider first a flat mesh in which all frames agree, so that every $R_a^f = I_2$. Then each block is a number times I_2: the scalar element matrix of the face, with every entry $s_{ab} = A_f \, g_a^f \cdot g_b^f$ replaced by the block $s_{ab} I_2$, and the two coordinates do not interact. For the triangle of Example 12.2 with such frames, the first block row would be $(1.0607 \, I_2, \; -0.3536 \, I_2, \; -0.7071 \, I_2)$. On a curved mesh, or with frames that do not agree, the diagonal blocks stay the same and each off-diagonal block becomes its number times the rotation $(R_a^f)^T R_b^f$, which mixes the two coordinates.

Exactness of the integration. The integrand of (11.1) [$\mathrm{L}_{ij}^f = A_f \, (g_i^f \cdot g_j^f) \, (R_i^f)^T R_j^f$] is constant on the face, so "area times integrand" is the exact integral, with no quadrature error. For higher order elements the gradients would vary and a quadrature rule would be needed; with linear elements it is not.

13.2. The element mass matrix and its lumping

The mass matrix needs the integrals $\int_f \Psi_{b,l} \cdot \Psi_{a,k} \, dA$ over a face f, for two of its vertices a, b. Substitute the basis functions (9.2) [$\Psi_{a,k}|_f = \phi_a \, T_f R_a^f \varepsilon_k$],

in parentheses, into the integrand, and simplify one step per line:

$$\begin{aligned}
\Psi_{b,l}\cdot\Psi_{a,k} &= \big(\phi_b\, T_f R_b^f \varepsilon_l\big)\cdot\big(\phi_a\, T_f R_a^f \varepsilon_k\big)\\
&= \phi_a\phi_b\ (T_f R_a^f \varepsilon_k)\cdot(T_f R_b^f \varepsilon_l) \quad \text{(the hats are numbers; reorder)}\\
&= \phi_a\phi_b\ (T_f R_a^f \varepsilon_k)^T (T_f R_b^f \varepsilon_l) \quad (x\cdot y = x^T y)\\
&= \phi_a\phi_b\ \varepsilon_k^T (R_a^f)^T (T_f^T T_f) R_b^f \varepsilon_l \quad ((XYZ)^T = Z^T Y^T X^T)\\
&= \phi_a\phi_b\ \varepsilon_k^T (R_a^f)^T R_b^f \varepsilon_l \quad (T_f^T T_f = I_2)\\
&= \phi_a\phi_b\ \big((R_a^f)^T R_b^f\big)_{kl} \quad (\varepsilon_k^T X \varepsilon_l = X_{kl}).
\end{aligned}$$

Only the product $\phi_a\phi_b$ varies over the face; the transports are constant, so they come out of the integral:

$$\int_f \Psi_{b,l}\cdot\Psi_{a,k}\, dA = \Big(\int_f \phi_a\phi_b\, dA\Big)\,\big((R_a^f)^T R_b^f\big)_{kl}.$$

Everything reduces to the integrals $\int_f \phi_a\phi_b\, dA$ of products of two hats over the face: nine of them, of which six are different, because $\phi_a\phi_b = \phi_b\phi_a$.

From the face to the reference triangle. These integrals are easiest on the reference triangle, where the hats are the simple shape functions (12.1) [$\hat\phi_1 = 1-\xi_1-\xi_2$, $\hat\phi_2 = \xi_1$, $\hat\phi_3 = \xi_2$]. To move an integral from the face to the reference triangle we need to know how the map (12.3) [$F(\xi) = x_1 + B\xi$] changes areas. A small rectangle of the reference plane, with corner ξ and sides $d\xi_1$ and $d\xi_2$ along the axes, is mapped, by the computation of the chain rule above [$F(\xi+s\varepsilon_m) = F(\xi)+s\,e_m$], to the parallelogram with corner $F(\xi)$ and sides $d\xi_1\, e_1$ and $d\xi_2\, e_2$. The area of a parallelogram is the length of the cross product of its sides:

$$\begin{aligned}
|(d\xi_1\, e_1)\times(d\xi_2\, e_2)| &= |e_1\times e_2|\, d\xi_1\, d\xi_2 \quad \text{(numbers out of the cross product)}\\
&= 2A_f\, d\xi_1\, d\xi_2 \quad \text{(by (3.2), } A_f = \tfrac{1}{2}|e_1\times e_2|).
\end{aligned}$$

So every small piece of the reference triangle has its area multiplied by the same constant $2A_f$ (as a check, the reference triangle has area $\frac{1}{2}$, and $2A_f\cdot\frac{1}{2} = A_f$). Cutting both triangles into corresponding small pieces, and using $\phi_a(F(\xi)) = \hat\phi_a(\xi)$ on each piece,

$$\int_f \phi_a\phi_b\, dA = 2A_f \int_{\hat K} \hat\phi_a\hat\phi_b\, d\xi.$$

The six integrals over the reference triangle. An integral over $\hat K$ is computed as two nested integrals of one variable: for each ξ_1 between 0 and 1, the points of the triangle have ξ_2 between 0 and $1-\xi_1$, so we integrate first in ξ_2 over that range and then in ξ_1. Write $L = 1-\xi_1$ for the length of the inner range; it does not depend on ξ_2. Two integrals of one variable will be used several

times:

$$\int_0^1 (1-\xi_1)^3\,d\xi_1 = \Big[-\frac{(1-\xi_1)^4}{4}\Big]_0^1 = 0-\Big(-\frac{1}{4}\Big) = \frac{1}{4},$$

$$\int_0^1 \xi_1(1-\xi_1)^2\,d\xi_1 = \int_0^1 (\xi_1 - 2\xi_1^2+\xi_1^3)\,d\xi_1 \quad \text{(expand the square)}$$

$$= \frac{1}{2}-\frac{2}{3}+\frac{1}{4} = \frac{6-8+3}{12} = \frac{1}{12}.$$

The square of $\hat{\phi}_2 = \xi_1$. The integrand does not depend on ξ_2, so the inner integral is the integrand times the length L:

$$\int_{\hat{K}} \xi_1^2\,d\xi = \int_0^1 \Big(\int_0^{1-\xi_1} \xi_1^2\,d\xi_2\Big)d\xi_1 = \int_0^1 \xi_1^2\,(1-\xi_1)\,d\xi_1$$

$$= \int_0^1 (\xi_1^2-\xi_1^3)\,d\xi_1 = \frac{1}{3}-\frac{1}{4} = \frac{1}{12}.$$

The square of $\hat{\phi}_3 = \xi_2$. The inner integral is $\int_0^L \xi_2^2\,d\xi_2 = \frac{L^3}{3}$:

$$\int_{\hat{K}} \xi_2^2\,d\xi = \int_0^1 \frac{(1-\xi_1)^3}{3}\,d\xi_1 = \frac{1}{3}\cdot\frac{1}{4} = \frac{1}{12}.$$

The square of $\hat{\phi}_1 = 1-\xi_1-\xi_2 = L-\xi_2$. Substitute $t = L-\xi_2$ in the inner integral; t runs from L down to 0 as ξ_2 runs from 0 to L, so the inner integral is $\int_0^L t^2\,dt = \frac{L^3}{3}$, as for $\hat{\phi}_3$:

$$\int_{\hat{K}} (1-\xi_1-\xi_2)^2\,d\xi = \int_0^1 \frac{(1-\xi_1)^3}{3}\,d\xi_1 = \frac{1}{12}.$$

The product $\hat{\phi}_2\hat{\phi}_3 = \xi_1\xi_2$. The factor ξ_1 comes out of the inner integral, which leaves $\int_0^L \xi_2\,d\xi_2 = \frac{L^2}{2}$:

$$\int_{\hat{K}} \xi_1\xi_2\,d\xi = \int_0^1 \xi_1\,\frac{(1-\xi_1)^2}{2}\,d\xi_1 = \frac{1}{2}\cdot\frac{1}{12} = \frac{1}{24}.$$

The product $\hat{\phi}_1\hat{\phi}_2 = \xi_1(L-\xi_2)$. The factor ξ_1 comes out of the inner integral, which leaves $\int_0^L (L-\xi_2)\,d\xi_2 = L^2-\frac{L^2}{2} = \frac{L^2}{2}$:

$$\int_{\hat{K}} \xi_1(1-\xi_1-\xi_2)\,d\xi = \int_0^1 \xi_1\,\frac{(1-\xi_1)^2}{2}\,d\xi_1 = \frac{1}{24}.$$

The product $\hat{\phi}_1\hat{\phi}_3 = (L-\xi_2)\,\xi_2$. The inner integral is $\int_0^L (L\xi_2-\xi_2^2)\,d\xi_2 = L\,\frac{L^2}{2}-\frac{L^3}{3} = \frac{L^3}{6}$:

$$\int_{\hat{K}} (1-\xi_1-\xi_2)\,\xi_2\,d\xi = \int_0^1 \frac{(1-\xi_1)^3}{6}\,d\xi_1 = \frac{1}{6}\cdot\frac{1}{4} = \frac{1}{24}.$$

So the three squares have the integral $\frac{1}{12}$ and the three products of two different shape functions have the integral $\frac{1}{24}$. Multiplying by $2A_f$,

$$\int_f \phi_a^2\,dA = 2A_f\cdot\frac{1}{12} = \frac{A_f}{6}, \qquad \int_f \phi_a\phi_b\,dA = 2A_f\cdot\frac{1}{24} = \frac{A_f}{12} \quad (a\neq b),$$

that is, $\int_f \phi_a\phi_b\,dA = \frac{A_f}{12}(1+\delta_{ab})$. As a 3×3 matrix,

$$\Big[\int_f \phi_a\phi_b\,dA\Big]_{a,b} = \frac{A_f}{12}\begin{pmatrix}2&1&1\\1&2&1\\1&1&2\end{pmatrix}.$$

So the *consistent* element mass matrix, the exact Galerkin one, has the blocks

$$\mathrm{M}^f_{ab} = \frac{A_f}{12}(1+\delta_{ab})\,(R^f_a)^T R^f_b. \tag{13.2}$$

Lumping. The consistent mass matrix couples each vertex with its neighbors, like the stiffness matrix. The code uses instead the simpler *lumped* mass matrix, which moves all the mass of each row of the scalar factor to the diagonal. The row sum of row a can be computed in two ways. With the numbers just found, one square and two products:

$$\sum_b \int_f \phi_a\phi_b\,dA = \frac{A_f}{6}+\frac{A_f}{12}+\frac{A_f}{12} = \frac{2A_f+A_f+A_f}{12} = \frac{A_f}{3}.$$

Or with the partition of unity, without computing any product:

$$\begin{aligned}\sum_b \int_f \phi_a\phi_b\,dA &= \int_f \phi_a\Big(\sum_b \phi_b\Big)\,dA \quad \text{(sum into the integral; factor out } \phi_a)\\ &= \int_f \phi_a\,dA \quad \Big(\textstyle\sum_b \phi_b = 1\Big)\\ &= 2A_f\int_{\hat K} \xi_1\,d\xi \quad \text{(to the reference triangle, } a=2)\\ &= 2A_f\int_0^1 \xi_1(1-\xi_1)\,d\xi_1 \quad \text{(inner integral: } \xi_1 \text{ times } L)\\ &= 2A_f\Big(\frac12-\frac13\Big) = 2A_f\cdot\frac16 = \frac{A_f}{3},\end{aligned}$$

and the same for $a=1$ and $a=3$. The lumped matrix keeps these row sums on the diagonal and puts zeros elsewhere:

$$\frac{A_f}{12}\begin{pmatrix}2&1&1\\1&2&1\\1&1&2\end{pmatrix} \quad\longrightarrow\quad \frac{A_f}{3}\begin{pmatrix}1&0&0\\0&1&0\\0&0&1\end{pmatrix}.$$

In blocks, the lumped element mass matrix is

$$\mathrm{M}^f_{aa} = \frac{A_f}{3}\,I_2, \qquad \mathrm{M}^f_{ab} = 0 \quad (a\neq b), \tag{13.3}$$

where the transport has disappeared because $(R^f_a)^T R^f_a = I$. For the triangle of Example 12.2, with $A_f = \sqrt{2}$, the consistent scalar entries are $A_f/6 = 0.2357$ on the diagonal and $A_f/12 = 0.1179$ off it, and the lumped entry is their row sum, $0.2357 + 2\cdot 0.1179 = A_f/3 = 0.4714$.

Lumping keeps two things exactly. The total mass of each row, by construction; and therefore the total mass of the whole matrix, $\sum_a \frac{A_f}{3} = A_f$ per face, so that the masses of all the vertices add up to the area of the

surface, as $\int_\Omega 1\, dA$ should. Why is this allowed? The mass matrix multiplies the vector of unknowns, and for a smooth field the values at the three vertices of a small face are nearly equal (after transport), so the row $\sum_b \mathrm{M}^f_{ab} u_b$ is nearly the row sum times u_a: lumping makes an error of the size of the variation of the field across one face, which is of the order of the discretization error that the method makes anyway. It keeps M diagonal and positive, and makes M^{-1} trivial, which the code uses in Section 15. Assembled, the mass of vertex i is one third of the total area of its incident faces, the same for both coordinates: the area of the surface that the vertex "owns". This is `AssembleDiagonalMassMatrix`:

```
475 std::shared_ptr<SparseMatrix<double>> AssembleDiagonalMassMatrix(Mesh *mesh) {
476     std::shared_ptr<SparseMatrix<double>> A(new SparseMatrix<double>(mesh->numVertices
            () * 2, mesh->numVertices() * 2));
477     double wij;
478     Vertex* v[3];
479     for (Face& face : mesh->getFaces()) {
480         v[0] = face.edge->vertex;
481         v[1] = face.edge->next->vertex;
482         v[2] = face.edge->next->next->vertex;
483         double faceArea = face_areas[face.ID];
484         wij = faceArea / 3.0;
485         (*A)(v[0]->ID * 2 , v[0]->ID * 2 ) += wij;
486         (*A)(v[0]->ID * 2 + 1, v[0]->ID * 2 + 1) += wij;
487         (*A)(v[1]->ID * 2 , v[1]->ID * 2 ) += wij;
488         (*A)(v[1]->ID * 2 + 1, v[1]->ID * 2 + 1) += wij;
489         (*A)(v[2]->ID * 2 , v[2]->ID * 2 ) += wij;
490         (*A)(v[2]->ID * 2 + 1, v[2]->ID * 2 + 1) += wij;
491     }
492     return A;
493 }
```

Line 476 creates an empty $2n \times 2n$ sparse matrix. For every face, lines 480–482 take its three vertices, line 483 reads the area A_f computed by `computeFaceNormals`, and line 484 is the lumped mass $A_f/3$ of (13.3). Lines 485–490 add it to the two diagonal entries $2\,\mathrm{ID}+0$ and $2\,\mathrm{ID}+1$ of each of the three vertices: the block $\frac{A_f}{3} I_2$, written as its two nonzero entries. This function does the assembly of the next section directly, without forming an element matrix, because the element matrix is diagonal.

What Stage 7 produced. For every face, the 6×6 element stiffness matrix (13.1) $[\mathrm{L}^f_{ab} = A_f\,(g^f_a \cdot g^f_b)\,(R^f_a)^T R^f_b\,]$ and the lumped element mass (13.3) $[\mathrm{M}^f_{aa} = \frac{A_f}{3} I_2\,]$, both computed exactly (the stiffness) or with a controlled simplification (the mass), from the reference triangle, the affine map and the chain rule. The geometry of the face enters through A_f and the gradients; the curvature of the surface through the transports. Stage 8 adds them up.

14. Stage 8. Assembly

What we want. The global matrices of Stage 6 from the element matrices of Stage 7, then the source condition, then the solution, read back as vectors. This section does the first step; Section 15 the rest.

From local to global indices. The global matrices are sums over the faces, and each face contributes its element matrix at the rows and columns of its own three vertices. To write this as a formula we need a way to pick, out of the global vector u of $2n$ numbers, the six numbers that belong to one face. That is a matrix P_f of size $6 \times 2n$, made of zeros and ones: its row (a,k), the row $2(a-1)+k$, has a single 1, in the column (f_a, k) of coordinate k of the vertex f_a:

$$(P_f)_{(a,k),\,p} = \begin{cases} 1 & \text{if } p = (f_a, k), \\ 0 & \text{otherwise.} \end{cases}$$

Then $(P_f \mathrm{u})_{(a,k)} = \sum_p (P_f)_{(a,k),p}\, \mathrm{u}_p = u_{f_a,k}$: $P_f \mathrm{u}$ is the local vector of the face, the coordinates of its three vertices in local order. The global matrices are

$$\mathrm{L} = \sum_f P_f^T\, \mathrm{L}^f\, P_f, \qquad \mathrm{M} = \sum_f P_f^T\, \mathrm{M}^f\, P_f. \tag{14.1}$$

What $P_f^T \mathrm{L}^f P_f$ is, entry by entry. Write out the entry (p,q) of the triple product, with the sums over the local rows (a,k) and the local columns (b,l):

$$\begin{aligned} \left(P_f^T \mathrm{L}^f P_f\right)_{p,q} &= \sum_{(a,k)} \sum_{(b,l)} (P_f^T)_{p,(a,k)}\, \mathrm{L}^f_{(a,k),(b,l)}\, (P_f)_{(b,l),q} \\ &= \sum_{(a,k)} \sum_{(b,l)} (P_f)_{(a,k),p}\, \mathrm{L}^f_{(a,k),(b,l)}\, (P_f)_{(b,l),q} \quad \text{(transpose)}. \end{aligned}$$

The first factor is 1 only for the local row with $(f_a, k) = p$, and the last only for the local column with $(f_b, l) = q$; every other term is zero. So

$$\left(P_f^T \mathrm{L}^f P_f\right)_{p,q} = \begin{cases} \mathrm{L}^f_{(a,k),(b,l)} & \text{if } p = (f_a,k) \text{ and } q = (f_b,l), \\ 0 & \text{if } p \text{ or } q \text{ is not a coordinate of a vertex of } f. \end{cases}$$

The product is a $2n \times 2n$ matrix that is zero except at the rows and columns of the three vertices of f, where it holds the entries of L^f, moved to the global positions.

Why the sum is the global matrix. The reason is the energy. Split the integral over the mesh into the integrals over the faces. On a face only its three vertices have nonzero hats, so the energy of the face is the quadratic form of L^f in the local vector $P_f \mathrm{u}$, by the same computation as for $\mathrm{u}^T \mathrm{L}\, \mathrm{u}$ in Section 11.

Then, one step per line:

$$\begin{aligned}
\mathrm{u}^T \mathrm{L}\,\mathrm{u} &= \int_{\Omega_h} |\nabla U|^2 \, dA = \sum_f \int_f |\nabla U|^2 \, dA \quad \text{(sum over the faces)} \\
&= \sum_f (P_f \mathrm{u})^T \, \mathrm{L}^f \, (P_f \mathrm{u}) \quad \text{(the energy of each face)} \\
&= \sum_f \mathrm{u}^T \, P_f^T \mathrm{L}^f P_f \, \mathrm{u} \quad ((XY)^T = Y^T X^T) \\
&= \mathrm{u}^T \Big(\sum_f P_f^T \mathrm{L}^f P_f \Big) \mathrm{u} \quad \text{(distributivity)}.
\end{aligned}$$

Two symmetric matrices with the same quadratic form for every u are equal, because the entries of a symmetric matrix A can be read off its quadratic form $Q(\mathrm{u}) = \mathrm{u}^T \mathrm{A}\,\mathrm{u}$: with the unit vectors ϵ_p of $\mathbb{R}^{2n}$, $Q(\epsilon_p) = \mathrm{A}_{pp}$ and $Q(\epsilon_p + \epsilon_q) = \mathrm{A}_{pp} + 2\mathrm{A}_{pq} + \mathrm{A}_{qq}$, so $\mathrm{A}_{pq} = \frac{1}{2}\big(Q(\epsilon_p + \epsilon_q) - Q(\epsilon_p) - Q(\epsilon_q)\big)$. Hence $\mathrm{L} = \sum_f P_f^T \mathrm{L}^f P_f$, which is (14.1); the same computation with $\int |U|^2$ gives the sum for M.

In code. Nobody forms P_f: multiplying by it only moves entries around. The entry (k, l) of the local block (a, b) of L^f is added to the global entry

$$\big(2\,\mathrm{ID}(f_a) + k,\ 2\,\mathrm{ID}(f_b) + l\big), \quad k, l \in \{0, 1\}, \tag{14.2}$$

with the zero-based vertex indices ID of the mesh. For example, a face whose vertices have the indices $\mathrm{ID} = 0, 3, 5$ (the vertices 1, 4, 6 of Figure 14, counted from one) sends its block $(2, 3)$, the coupling of its second and third vertices, to the rows $2 \cdot 3 + k = 6, 7$ and the columns $2 \cdot 5 + l = 10, 11$. The function is `AssembleMatrix2`:

```
std::shared_ptr<SparseMatrix<double>> AssembleMatrix2(Mesh *mesh, double delta_t) {
    std::shared_ptr<SparseMatrix<double>> A(new SparseMatrix<double>(mesh->numVertices
        () * 2, mesh->numVertices() * 2));
    Matrix66d stiffnessMatrix;
    Eigen::Matrix2d wij;
    Vertex* v[3];
    for (Face& face : mesh->getFaces()) {
        v[0] = face.edge->vertex;
        v[1] = face.edge->next->vertex;
        v[2] = face.edge->next->next->vertex;
        //stiffnessMatrix = connectionLaplacian(face);
        stiffnessMatrix = AssembleStiffnessElementEmbedded2(v, face);
        for( int i = 0 ; i < 3 ; ++i ) {
            for (int j = 0 ; j < 3 ; ++j ) {
                //wij = massMatrix(i, j) + delta_t * stiffnessMatrix(i, j);
                wij = delta_t * stiffnessMatrix.block<2,2>(i * 2, j * 2);
                for(int row = 0 ; row < 2 ; ++row) {
                    for(int col = 0 ; col < 2 ; ++col) {
                        //if(wij(row, col) != 0.0)
                        {
                            (*A)(v[i]->ID * 2 + row, v[j]->ID * 2 + col) += wij(row, col)
                                ;
                        }
                    }
                }
            }
        }
    }
```

```
456     return A;
457 }
```

Line 431 creates an empty sparse matrix of size $2n \times 2n$ (the class `SparseMatrix` of `numerics.h` stores each row as a map from column indices to values, and creates an entry, equal to zero, the first time it is accessed). The loop of line 435 visits every face; lines 436–438 take its three vertices in order, and line 440 computes its element matrix L^f with `AssembleStiffnessElementEmbedded2`. The loops of lines 441–442 run over the local block indices a, b (`i`, `j`), and line 444 takes the block (a, b) and multiplies it by Δt (`delta_t`, the argument of the function). The loops of lines 445–446 run over the entries (k, l) (`row`, `col`) of the block, and line 449 is the index map (14.2): `(*A)(v[i]->ID * 2 + row, v[j]->ID * 2 + col) += wij(row, col)`. So the function returns $\Delta t\,\mathrm{L}$, not L; the factor Δt is applied while assembling. The commented line 443 is a reminder of the other possible organization, adding $\mathrm{M}^f + \Delta t\,\mathrm{L}^f$ face by face; the code assembles the mass separately, with `AssembleDiagonalMassMatrix` and (13.3) $[\,\mathrm{M}^f_{aa} = \frac{A_f}{3} I_2\,]$.

Each interior edge gets contributions from its two faces, and each vertex from all its incident faces. The same result can be read directly from the integrals. Start from the block L_{ij} of (10.4) $[\,\mathrm{L}_{ij} = [\int_\Omega \nabla\Psi_{j,l} : \nabla\Psi_{i,k}\,dA]_{k,l}\,]$ and take one step per line:

$$
\begin{aligned}
\mathrm{L}_{ij} &= \Big[\int_{\Omega_h} \nabla\Psi_{j,l} : \nabla\Psi_{i,k}\,dA\Big]_{k,l} \\
&= \sum_f \Big[\int_f \nabla\Psi_{j,l} : \nabla\Psi_{i,k}\,dA\Big]_{k,l} \quad \text{(split the mesh into faces)} \\
&= \sum_{f \ni i,j} \Big[\int_f \nabla\Psi_{j,l} : \nabla\Psi_{i,k}\,dA\Big]_{k,l} \quad \text{(elsewhere a hat vanishes)} \\
&= \sum_{f \ni i,j} A_f\,(g_i^f \cdot g_j^f)\,(R_i^f)^T R_j^f \quad \text{(by (11.1))} \\
&= \sum_{f \ni i,j} \mathrm{L}^f_{ab} \quad (a, b\text{: the local numbers of } i, j \text{ in } f).
\end{aligned}
$$

The third line drops the faces that do not contain both i and j: on such a face ϕ_i or ϕ_j is zero, so $\nabla\Psi_{i,k}$ or $\nabla\Psi_{j,l}$ is zero. This is what the matrices P_f of (14.1) do: they put the block L^f_{ab} at the block position (i, j), and the sum adds the contributions of all the faces. Figure 14 shows one face being added.

Accumulation. The operator `+=` of line 449 is essential. A global entry that couples two vertices joined by an interior edge is touched by the two faces of that edge, and a diagonal entry by every face around the vertex; each face adds its share, and the sum is the integral over all the faces that the two hats share. For the scalar factor this sum is, by Proposition 12.3, the cotangent weight $-\frac{1}{2}(\cot\alpha_{ij} + \cot\beta_{ij})$ of the edge. For the connection Laplacian the two

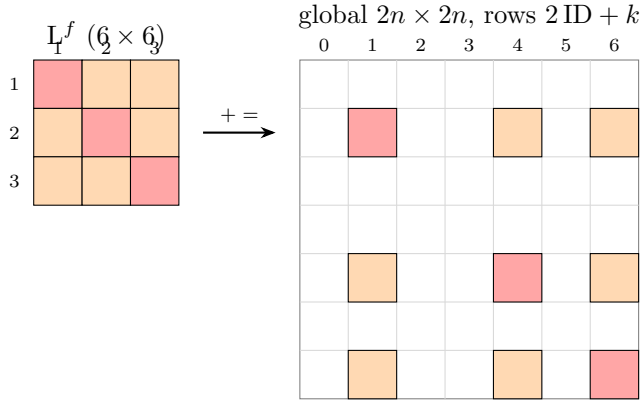


FIGURE 14. Assembly. The 6×6 element matrix of a face, made of 3×3 blocks of size 2×2 (diagonal blocks red), is added into the global matrix at the rows and columns of its vertices, here 1, 4 and 6.

faces of an edge contribute the same kind of number times two transports, $(R_i^f)^T R_j^f$ and $(R_i^g)^T R_j^g$, one through each face; on a smooth surface they are nearly equal, since both approximate the transport along the short edge from j to i.

Structure. The assembled L has a 2×2 block for each vertex (the diagonal) and for each edge, in both orders, with $\mathrm{L}_{ji} = \mathrm{L}_{ij}^T$. It is symmetric and positive semi-definite, because each term $P_f^T \mathrm{L}^f P_f$ is. It is singular only if some nonzero u has $\mathrm{u}^T \mathrm{L}\,\mathrm{u} = \int |\nabla U|^2 = 0$, that is, if a discrete field is parallel everywhere, which on a curved closed surface is never the case. In any case $\mathrm{M} + \Delta t\,\mathrm{L}$ is positive definite, since $\mathrm{u}^T(\mathrm{M} + \Delta t\,\mathrm{L})\mathrm{u} = \mathrm{u}^T\mathrm{M}\mathrm{u} + \Delta t\,\mathrm{u}^T\mathrm{L}\mathrm{u}$, the first term is positive for $\mathrm{u} \neq 0$ (the lumped masses are positive) and the second is not negative. This is the discrete counterpart of the positive definiteness of $I + \Delta t\,\nabla^*\nabla$ in Section 5.

Worked example 14.1 (Assembly on the unit square). *Take the unit square in the plane $z = 0$, with the vertices $x_1 = (0,0)$, $x_2 = (1,0)$, $x_3 = (1,1)$, $x_4 = (0,1)$ (we drop the third coordinate, which is zero), cut by the diagonal x_1x_3 into the two faces $f = (1,2,3)$ and $g = (1,3,4)$. The mesh is flat, so all the normals are $(0,0,1)$, every smallest rotation is $Q = I$, and we choose one common frame for all the vertices and faces, $T_i = T_f = T_g$. Then every transport is $R_a^f = T_f^T I\, T_a = T_f^T T_f = I_2$, and every block (13.1) [$\mathrm{L}_{ab}^f = A_f\,(g_a^f \cdot g_b^f)\,(R_a^f)^T R_b^f$] is the scalar entry $s_{ab} = A_f\,\nabla\phi_a \cdot \nabla\phi_b$ times I_2. So we can assemble the scalar 3×3 element matrices first and put I_2 in later.*

Face $f = (1,2,3)$. *The edges are $e_1 = x_2 - x_1 = (1,0)$ and $e_2 = x_3 - x_1 = (1,1)$, with $e_1 \cdot e_1 = 1$, $e_1 \cdot e_2 = 1$, $e_2 \cdot e_2 = 2$; the determinant of the Gram matrix is $1 \cdot 2 - 1^2 = 1 = 4A_f^2$, with $A_f = \frac{1}{2}$. The formulas for the gradients*

give

$$\nabla\phi_2 = \frac{(e_2\cdot e_2)\,e_1 - (e_1\cdot e_2)\,e_2}{1} = 2(1,0)-(1,1) = (1,-1),$$
$$\nabla\phi_3 = \frac{(e_1\cdot e_1)\,e_2 - (e_1\cdot e_2)\,e_1}{1} = (1,1)-(1,0) = (0,1),$$
$$\nabla\phi_1 = -\nabla\phi_2 - \nabla\phi_3 = (-1,1)-(0,1) = (-1,0).$$

Multiplying the inner products by $A_f = \frac{1}{2}$*:* $s_{11} = \frac{1}{2}\cdot 1$*,* $s_{12} = \frac{1}{2}\cdot(-1)$*,* $s_{13} = \frac{1}{2}\cdot 0$*,* $s_{22} = \frac{1}{2}\cdot 2$*,* $s_{23} = \frac{1}{2}\cdot(-1)$*,* $s_{33} = \frac{1}{2}\cdot 1$*. By the cotangent formula (Proposition 12.3) the same numbers come from the angles* $45°$*,* $90°$*,* $45°$ *at* x_1, x_2, x_3*: for instance* $s_{13} = -\frac{1}{2}\cot 90° = 0$ *and* $s_{22} = \frac{1}{2}(\cot 45° + \cot 45°) = 1$*.*

Face $g = (1,3,4)$*. The edges are* $e_1 = x_3 - x_1 = (1,1)$ *and* $e_2 = x_4 - x_1 = (0,1)$*, with* $e_1\cdot e_1 = 2$*,* $e_1\cdot e_2 = 1$*,* $e_2\cdot e_2 = 1$*, determinant* $2-1=1$*,* $A_g = \frac{1}{2}$*:*

$$\nabla\phi_3 = 1\cdot(1,1) - 1\cdot(0,1) = (1,0), \qquad \nabla\phi_4 = 2\,(0,1) - 1\cdot(1,1) = (-1,1),$$
$$\nabla\phi_1 = -(1,0) - (-1,1) = (0,-1).$$

The two scalar element matrices, with their rows and columns labelled by the global vertex numbers, are

$$S^f = \begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\!\!\begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} & 0 \\ -\frac{1}{2} & 1 & -\frac{1}{2} \\ 0 & -\frac{1}{2} & \frac{1}{2} \end{pmatrix} \end{array}, \qquad S^g = \begin{array}{c} \\ 1 \\ 3 \\ 4 \end{array}\!\!\begin{array}{c} \begin{array}{ccc} 1 & 3 & 4 \end{array} \\ \begin{pmatrix} \frac{1}{2} & 0 & -\frac{1}{2} \\ 0 & \frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & -\frac{1}{2} & 1 \end{pmatrix} \end{array}.$$

The matrices P_f and P_g*. In the scalar case each vertex has one coordinate, and* P_f *is* 3×4*: its row* a *has its* 1 *in column* f_a*. With* $f = (1,2,3)$ *and* $g = (1,3,4)$*,*

$$P_f = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \qquad P_g = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

By the entry-by-entry rule above, $P_g^T S^g P_g$ *has at the position* $(p,q) = (g_a, g_b)$ *the entry* S^g_{ab}*, and zero in the row and the column of vertex* 2*, which is not in* g*. For instance, the entry* $(3,4)$*: vertex* 3 *is the local vertex* $a = 2$ *of* g *and vertex* 4 *the local vertex* $b = 3$*, so the entry is* $S^g_{23} = -\frac{1}{2}$*. All sixteen entries:*

$$P_f^T S^f P_f = \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} & 0 & 0 \\ -\frac{1}{2} & 1 & -\frac{1}{2} & 0 \\ 0 & -\frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad P_g^T S^g P_g = \begin{pmatrix} \frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & 0 & -\frac{1}{2} & 1 \end{pmatrix}.$$

Their sum, entry by entry, is the assembled scalar stiffness matrix:

$$L = \begin{pmatrix} \frac{1}{2}+\frac{1}{2} & -\frac{1}{2}+0 & 0+0 & 0-\frac{1}{2} \\ -\frac{1}{2}+0 & 1+0 & -\frac{1}{2}+0 & 0+0 \\ 0+0 & -\frac{1}{2}+0 & \frac{1}{2}+\frac{1}{2} & 0-\frac{1}{2} \\ 0-\frac{1}{2} & 0+0 & 0-\frac{1}{2} & 0+1 \end{pmatrix} = \begin{pmatrix} 1 & -\frac{1}{2} & 0 & -\frac{1}{2} \\ -\frac{1}{2} & 1 & -\frac{1}{2} & 0 \\ 0 & -\frac{1}{2} & 1 & -\frac{1}{2} \\ -\frac{1}{2} & 0 & -\frac{1}{2} & 1 \end{pmatrix}.$$

The diagonal entries of vertices 1 *and* 3 *received a contribution from both faces, and so did the coupling* $L_{13} = 0 + 0$ *of the diagonal edge that the faces share: this is the accumulation* `+=`*. That coupling is zero because both angles opposite to the diagonal are right angles,* $-\frac{1}{2}(\cot 90^\circ + \cot 90^\circ) = 0$*. Every row adds up to zero, because constants have zero gradient.*

The mass. *Each face has the area* $\frac{1}{2}$*, so it gives each of its vertices the lumped mass* $\frac{A_f}{3} = \frac{1}{6}$ *by (13.3)* $[\mathrm{M}^f_{aa} = \frac{A_f}{3} I_2\,]$*. Vertices* 1 *and* 3 *belong to both faces and vertices* 2 *and* 4 *to one:*

$$m_1 = \tfrac{1}{6} + \tfrac{1}{6} = \tfrac{1}{3}, \qquad m_2 = \tfrac{1}{6}, \qquad m_3 = \tfrac{1}{6} + \tfrac{1}{6} = \tfrac{1}{3}, \qquad m_4 = \tfrac{1}{6},$$

which add up to 1*, the area of the square.*

The vector matrices. *With two coordinates per vertex,* P_f *is* 6×8 *and every* 1 *of the scalar* P_f *becomes the block* I_2*. Since every element block is* $s_{ab} I_2$*, the global block* (i, j) *is* $\mathrm{L}_{ij} = L_{ij}\, I_2$*, and* M *has the diagonal blocks* $m_i I_2$*. In the zero-based count of the code, the block* $(a, b) = (2, 3)$ *of* g*, the coupling of the vertices with* $\mathrm{ID} = 2$ *and* $\mathrm{ID} = 3$*, goes by (14.2)* $[(2\,\mathrm{ID}(f_a) + k,\ 2\,\mathrm{ID}(f_b) + l)\,]$ *to the rows* $2{\cdot}2 + k = 4, 5$ *and the columns* $2{\cdot}3 + l = 6, 7$ *of the* 8×8 *matrix. The two coordinates never mix, so each coordinate of the vector problem is the scalar problem with the matrices* L *and* $\mathrm{diag}(m_1, m_2, m_3, m_4)$*. Section 15 continues the example to the solution.*

Part I · The Derivation with Matrices

15. Stage 8, Continued: The Linear System and Its Solution

All the pieces are now computed. This section follows `main` to the end: it scales the system by the inverse mass, imposes the source vector, factors and solves, and turns the solution into arrows.

15.1. The system the code solves

The discrete backward Euler step is (10.3) $[\,(\mathrm{M} + \Delta t\, \mathrm{L})\,\mathrm{u} = \mathrm{M}\,\mathrm{u}^0\,]$. The lumped M is diagonal with positive entries, so it has the inverse M^{-1}, diagonal with the reciprocal entries. Multiply both sides by it on the left, which does not change the solution because M^{-1} is invertible (multiplying back by M recovers the original equation). One step per line:

$$\begin{aligned} \mathrm{M}^{-1}\big((\mathrm{M} + \Delta t\, \mathrm{L})\,\mathrm{u}\big) &= \mathrm{M}^{-1}\big(\mathrm{M}\,\mathrm{u}^0\big) \\ (\mathrm{M}^{-1}\mathrm{M} + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} &= (\mathrm{M}^{-1}\mathrm{M})\,\mathrm{u}^0 \quad \text{(distribute; associativity)} \\ (I + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} &= \mathrm{u}^0 \quad (\mathrm{M}^{-1}\mathrm{M} = I), \end{aligned}$$

which we label

$$(I + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} = \mathrm{u}^0. \tag{15.1}$$

Multiplying by a diagonal matrix on the left multiplies each row by the corresponding diagonal entry: $(\mathrm{M}^{-1}\mathrm{L})_{pq} = \sum_r (\mathrm{M}^{-1})_{pr} \mathrm{L}_{rq} = \mathrm{L}_{pq}/\mathrm{M}_{pp}$, since

only $r = p$ survives. So row $2i + k$ of $\Delta t\,\mathrm{L}$ is divided by the mass of vertex i, one third of the area of its one-ring. This has a meaning: L came from integrals, which grow with the area of the faces, and dividing by the area that the vertex owns turns the integrated quantity back into a value per unit area, as in the continuous equation $U + \Delta t\,\nabla^*\nabla U = U^0$. The code builds this form in place, with `IplusMinvTimesA`:

```
495 void IplusMinvTimesA(std::shared_ptr<SparseMatrix<double>> M, std::shared_ptr<
        SparseMatrix<double>> A)
496 {
497     auto numRows = A->numRows();
498     for (int i = 0; i < numRows; ++i)
499     {
500         SparseMatrix<double>::RowIterator aIter = A->iterator(i);
501         double oneOverVertexOneRingArea = 1.0 / (*M)(i, i);
502         for (; !aIter.end(); ++aIter)
503         {
504             auto j = aIter.columnIndex();
505             (*A)(i, j) *= oneOverVertexOneRingArea;
506             if (i == j) {
507                 (*A)(i, j) += 1.0; // this completes the (I + M^-1 L)
508             }
509         }
510     }
511 }
```

The argument `A` holds $\Delta t\,\mathrm{L}$, as returned by `AssembleMatrix2`, and `M` the lumped mass matrix. The loop of line 498 visits every row i. Line 501 is the reciprocal of the diagonal mass of that row, $1/\mathrm{M}_{ii}$ (the name recalls that it is one over a third of the one-ring area). The inner loop (lines 502–509) visits the nonzero entries of the row; line 505 divides each by the mass, and lines 506–508 add 1 to the diagonal entry, the I of (15.1). Every row has a diagonal entry, since every vertex belongs to at least one face. After the call, `A` holds $I + \Delta t\,\mathrm{M}^{-1}\mathrm{L}$.

The unit square, continued. In Example 14.1 take $\Delta t = 1$. The masses are $m_1 = \frac{1}{3}$, $m_2 = \frac{1}{6}$, $m_3 = \frac{1}{3}$, $m_4 = \frac{1}{6}$, so M^{-1} multiplies the rows of vertices 1 and 3 by 3 and the rows of vertices 2 and 4 by 6; then `IplusMinvTimesA` adds 1 on the diagonal. Row by row, for one coordinate:

$$\begin{aligned}
\text{row 1:}\quad & 3\left(1, -\tfrac{1}{2}, 0, -\tfrac{1}{2}\right) + (1,0,0,0) = \left(4, -\tfrac{3}{2}, 0, -\tfrac{3}{2}\right),\\
\text{row 2:}\quad & 6\left(-\tfrac{1}{2}, 1, -\tfrac{1}{2}, 0\right) + (0,1,0,0) = (-3, 7, -3, 0),\\
\text{row 3:}\quad & 3\left(0, -\tfrac{1}{2}, 1, -\tfrac{1}{2}\right) + (0,0,1,0) = \left(0, -\tfrac{3}{2}, 4, -\tfrac{3}{2}\right),\\
\text{row 4:}\quad & 6\left(-\tfrac{1}{2}, 0, -\tfrac{1}{2}, 1\right) + (0,0,0,1) = (-3, 0, -3, 7).
\end{aligned}$$

The matrix $I + \Delta t\,\mathrm{M}^{-1}\mathrm{L}$ is no longer symmetric: its entry $(1,2)$ is $-\frac{3}{2}$ and its entry $(2,1)$ is -3. In the 8×8 vector matrix each of these numbers is a multiple of I_2, as before.

The scaled matrix is no longer symmetric (its rows are scaled by different masses), which is one reason why the code factors it with a general sparse LU. The symmetric form (10.3) $[\,(\mathrm{M} + \Delta t\,\mathrm{L})\,\mathrm{u} = \mathrm{M}\,\mathrm{u}^0\,]$ could be factored with a Cholesky factorization instead, at half the cost; the two have the same solution.

15.2. The Dirichlet condition

The source. The source vector is prescribed at one vertex s: $u_s = \bar{u}_s$, given as the unit complex number $e^{2\pi i/3}$, that is, the unit tangent vector at 120° from the first frame vector t_s, with coordinates $(\cos 120°, \sin 120°) = (-\frac{1}{2}, \frac{\sqrt{3}}{2})$. The code chooses the vertex and fills the right-hand side in `main`:

```
615     right_hand_side = Eigen::VectorXd(mesh.numVertices() * 2);
616     right_hand_side.setZero(); // solve laplace's equation where RHS is zero
617
618     for( Vertex& vertex : mesh.getVertices())
619     {
620         if (vertex.p.norm() < 2.5e-2) {
621             std::complex<double> U_i = std::complex<double>(cos(2*M_PI/3), sin(2*M_PI
                      /3));
622             right_hand_side(vertex.ID * 2 ) = U_i.real();
623             right_hand_side(vertex.ID * 2 + 1) = U_i.imag();
624             allconstraints.insert(vertex.ID * 2 );
625             allconstraints.insert(vertex.ID * 2 + 1);
626             break;
627         }
628         if (vertex.p.z() > 0.83) {
629             std::complex<double> U_i = std::complex<double>(cos(2*M_PI/3), sin(2*M_PI
                      /3));
630             right_hand_side(vertex.ID * 2 ) = U_i.real();
631             right_hand_side(vertex.ID * 2 + 1) = U_i.imag();
632             allconstraints.insert(vertex.ID * 2 );
633             allconstraints.insert(vertex.ID * 2 + 1);
634             break;
635         }
636     }
```

Lines 615–616 create the right-hand side, a vector of $2n$ zeros. The loop of line 618 scans the vertices. The first vertex that is either near the center of the model (line 620, at distance less than 0.025 from the origin, after `CenterAndNormalize` has centered the mesh) or above the height 0.83 (line 628) becomes the source. Line 621 (or 629) forms $\bar{u}_s = \cos\frac{2\pi}{3} + \mathrm{i}\sin\frac{2\pi}{3}$, lines 622–623 write its real and imaginary parts, its two coordinates, at the entries $2s$ and $2s+1$, and lines 624–625 record these two entries (not the vertex) as constrained. The `break` of line 626 (or 634) ends the loop, so exactly one vertex is constrained.

Row replacement. The two coordinates of s are then not unknowns, and their two equations of (15.1) $[\,(I+\Delta t\,\mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} = \mathrm{u}^0\,]$ are replaced by the prescribed values: in the matrix, the rows $2s$ and $2s+1$ become rows of the identity, and in the right-hand side, the entries become $\bar{u}_s$. A row of the identity, a 1 on the diagonal and zeros elsewhere, multiplied by u, picks one entry: the equation of row $2s+k$ now reads $u_{s,k} = \bar{u}_{s,k}$, the prescribed value. The other rows are unchanged, so they still couple u_s to its neighbors: through them, the prescribed vector acts as a source for the rest of the mesh. Every other entry of the right-hand side is zero, $\mathrm{u}^0 = 0$ away from the source, which is the problem (5.6) $[\,U + \Delta t\,\nabla^*\nabla U = 0$ off $s,\ U(s) = U_s\,]$. The row replacement is done by `PreFactor`, which also factors the matrix:

```
513 bool is_constrained(std::set<int>& constraints, int vertex)
514 {
515     return constraints.find(vertex) != constraints.end();
516 }
```

```
517
518 void PreFactor(std::shared_ptr<SparseMatrix<double>> A, std::set<int>& constraints,
        Eigen::SparseLU<Eigen::SparseMatrix<double>>& solver)
519 {
520
521     Eigen::SparseMatrix<double> Lc = Eigen::SparseMatrix<double>(A->numRows(), A->
            numColumns());
522
523     auto numRows = A->numRows();
524     for (int i = 0; i < numRows; ++i)
525     {
526         if (!is_constrained(constraints, i))
527         {
528             SparseMatrix<double>::RowIterator aIter = A->iterator(i);
529             for (; !aIter.end(); ++aIter)
530             {
531                 auto j = aIter.columnIndex();
532                 Lc.insert(i, j) = (*A)(i, j);
533             }
534         }
535         else
536         {
537             Lc.insert(i, i) = 1.0;
538         }
539     }
540
541     Lc.makeCompressed();
542     solver.compute(Lc);
543     if (solver.info() != Eigen::Success) {
544         std::cerr << "Error:␣" << "Prefactor␣failed." << std::endl;
545         exit(1);
546     }
547 }
```

`is_constrained` (lines 513–516) tests whether a row index is in the set of constrained entries. `PreFactor` builds a new matrix `Lc` in the sparse format of the Eigen library (line 521), row by row (line 524). An unconstrained row is copied entry by entry (lines 528–533). A constrained row gets a single 1 on its diagonal (line 537), which makes it a row of the identity. Line 541 converts `Lc` to the compressed storage that the solver needs, and line 542, `solver.compute(Lc)`, computes its sparse LU factorization: a lower and an upper triangular matrix whose product is `Lc`, so that solving reduces to two triangular substitutions. Lines 543–546 stop the program if the factorization fails. The factorization is the expensive step. Once it is done, `solver.solve` only performs the two substitutions, so a different source vector at the same vertex would cost almost nothing; a different source vertex would change the replaced rows and require a new factorization.

The unit square, finished. Put the source at vertex 1 of Example 14.1, with the prescribed vector $\bar{u}$ (in the code, $\bar{u} = (-\frac{1}{2}, \frac{\sqrt{3}}{2})$). Each coordinate is a separate scalar problem with the same matrix, so write $\bar{u}$ for one coordinate of the source vector and $u_1, \ldots, u_4$ for the same coordinate at the four vertices. `PreFactor` replaces the row of vertex 1 by the row $(1, 0, 0, 0)$ of the identity, and the right-hand side is $\bar{u}$ in that row and 0 elsewhere. With the scaled

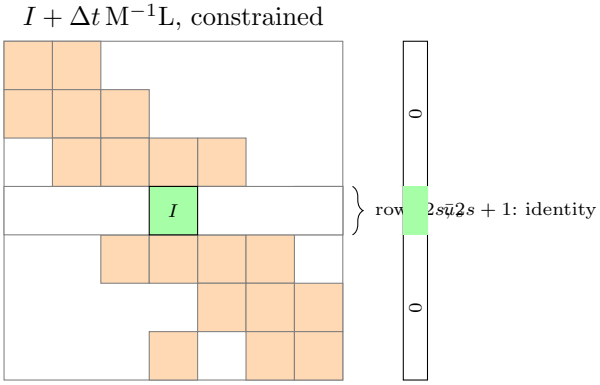


FIGURE 15. The Dirichlet condition by row replacement: the two rows of the source vertex s become rows of the identity, and the right-hand side holds the prescribed vector $\bar{u}_s$ there and zero elsewhere.

rows computed above, the system is

$$\begin{aligned} u_1 &= \bar{u}, \\ -3u_1 + 7u_2 - 3u_3 &= 0, \\ -\tfrac{3}{2}u_2 + 4u_3 - \tfrac{3}{2}u_4 &= 0, \\ -3u_1 - 3u_3 + 7u_4 &= 0. \end{aligned}$$

Substitute the first equation, $u_1 = \bar{u}$, into the second and the fourth, and move the known term to the right:

$$7u_2 - 3u_3 = 3\bar{u}, \qquad 7u_4 - 3u_3 = 3\bar{u}.$$

Subtracting the second of these from the first gives $7u_2 - 7u_4 = 0$, so $u_2 = u_4$, as the symmetry of the square about its diagonal requires. Substitute $u_4 = u_2$ into the third equation and collect:

$$\begin{aligned} -\tfrac{3}{2}u_2 + 4u_3 - \tfrac{3}{2}u_2 &= 0 \\ -3u_2 + 4u_3 &= 0 \quad \text{(collect the } u_2 \text{ terms)} \\ u_3 &= \tfrac{3}{4}\, u_2. \end{aligned}$$

Substitute $u_3 = \frac{3}{4}u_2$, in parentheses, into $7u_2 - 3u_3 = 3\bar{u}$:

$$\begin{aligned} 7u_2 - 3\left(\tfrac{3}{4}u_2\right) &= 3\bar{u} \\ \left(\tfrac{28}{4} - \tfrac{9}{4}\right) u_2 &= 3\bar{u} \quad \text{(collect)} \\ \tfrac{19}{4}\, u_2 &= 3\bar{u} \\ u_2 &= \tfrac{12}{19}\, \bar{u}. \end{aligned}$$

Then $u_4 = u_2 = \frac{12}{19}\bar{u}$ and $u_3 = \frac{3}{4}\cdot\frac{12}{19}\bar{u} = \frac{9}{19}\bar{u}$. As a check, the third equation gives $\frac{1}{19}\big(-\frac{3}{2}\cdot 12 + 4\cdot 9 - \frac{3}{2}\cdot 12\big)\bar{u} = \frac{-18+36-18}{19}\bar{u} = 0$. So

$$u_1 = \bar{u}, \qquad u_2 = u_4 = \tfrac{12}{19}\,\bar{u} \approx 0.632\,\bar{u}, \qquad u_3 = \tfrac{9}{19}\,\bar{u} \approx 0.474\,\bar{u},$$

for each of the two coordinates, so the vector at every vertex is a positive multiple of the source vector $\bar{u}$. That is the right answer: on a flat mesh with a common frame, parallel transport does not turn a vector, and the normalization described below would draw the same unit vector at all four vertices. The magnitudes decrease with the distance from the source, the screening of (5.6) [$U + \Delta t\,\nabla^*\nabla U = 0$ off s, $U(s) = U_s$].

15.3. The calls in `main`

The whole computation of Sections 13–15 is a handful of lines of `main`:

```
638     A = g_intrinsic ? AssembleMatrixIntrinsic(&mesh, avgEdgeLength*avgEdgeLength) :
            AssembleMatrix2(&mesh, avgEdgeLength*avgEdgeLength);
639     std::shared_ptr<SparseMatrix<double>> M;
640     M = AssembleDiagonalMassMatrix(&mesh);
641     IplusMinvTimesA(M, A);
642
643     PreFactor(A, allconstraints, solver);
644
645     solutionU = solver.solve(right_hand_side);
```

Line 638 assembles $\Delta t\,\mathrm{L}$ with $\Delta t = \bar{h}^2$ (with the option `--intrinsic` it calls instead the assembly of Part III written in this part's matrices, Section 34); lines 639–640 assemble the lumped M; line 641 turns `A` into $I + \Delta t\,\mathrm{M}^{-1}\mathrm{L}$; line 643 replaces the rows of the source and factors; and line 645 solves the system for the right-hand side built above, storing u in `solutionU`. The program then enters the drawing loop, and the drawing function reads `solutionU`.

15.4. From the solution to vectors

The solve returns u, two coordinates per vertex. Its magnitudes decay away from the source; its directions are the transported source vector. So the code keeps only the direction. It reads u_i as the complex number $u_{i,1} + \mathrm{i}u_{i,2}$, normalizes it, $\hat{u}_i = u_i/|u_i|$, which needs $u_i \neq 0$ (far from the source u_i is tiny, but not zero), takes its angle $\alpha_i = \arg\hat{u}_i$, and rotates the first frame vector t_i by α_i about the vertex normal n_i. A rotation by the angle α about a unit axis n takes a vector t to $\cos\alpha\,t + \sin\alpha\,n\times t + (1-\cos\alpha)(n\cdot t)\,n$ (Rodrigues' formula). With $n = n_i$ and $t = t_i$, which is orthogonal to n_i:

$$\begin{aligned}
&\cos\alpha_i\,t_i + \sin\alpha_i\,n_i\times t_i + (1-\cos\alpha_i)(n_i\cdot t_i)\,n_i\\
&= \cos\alpha_i\,t_i + \sin\alpha_i\,n_i\times t_i \quad (n_i\cdot t_i = 0)\\
&= \cos\alpha_i\,t_i + \sin\alpha_i\,b_i \quad (b_i = n_i\times t_i)\\
&= T_i\begin{pmatrix}\cos\alpha_i\\ \sin\alpha_i\end{pmatrix} = T_i\hat{u}_i \quad (T_i = [\,t_i\;\; b_i\,]),
\end{aligned}$$

the tangent vector whose coordinates are $\hat{u}_i = (\cos\alpha_i, \sin\alpha_i)$. That vector is drawn as an arrow at the vertex, in `display`:

```
769     for(int i = 0; i < mesh.numVertices(); ++i){
770         Vertex& vi = mesh.vertexAt(i);
771         std::complex<double> R_i = std::complex<double>(solutionU[vi.ID * 2], solutionU
                [vi.ID * 2 + 1]);
772         R_i = R_i / abs(R_i);
773         Eigen::Vector3d X_i = project(vi.edge->pair->vertex->p - vi.p, vi.n).normalized
                ();
774         Eigen::Quaterniond Q(Eigen::AngleAxisd(log(R_i).imag(), vi.n));
775         Eigen::Vector3d U_i = 0.06 * (g_intrinsic ? intrinsic::lift(mesh,
                g_intrinsicGeometry, i, R_i) : Q._transformVector(X_i));
776         DrawArrow(c3gaPoint(vi.p.x(), vi.p.y(), vi.p.z()), _vectorE3GA(U_i.x(), U_i.y()
                , U_i.z()));
777     }
```

Line 771 forms the complex number $u_{i,1} + \mathrm{i}\, u_{i,2}$ from the entries $2i$ and $2i+1$ of the solution, and line 772 normalizes it to $\hat{u}_i$. Line 773 recomputes the frame vector t_i exactly as line 185 of `computeVertexTangentSpace` did, from the same neighbor. Line 774 builds the rotation about n_i by the angle α_i: the complex logarithm of a unit complex number $e^{\mathrm{i}\alpha}$ is $\mathrm{i}\alpha$, so `log(R_i).imag()` is α_i. Line 775 applies the rotation to t_i, giving $T_i \hat{u}_i$ (or, with `--intrinsic`, calls `intrinsic::lift` of Part III), and scales it to the length 0.06 for display; line 776 draws the arrow at the vertex position x_i.

15.5. What the program computes

Figure 16 shows the result on the cactus that `FEM.cpp` loads: one vector prescribed at one vertex has been spread over the whole surface, following its arms and trunk. To see that the directions are right we need a surface where parallel transport is known exactly.

Worked example 15.1. *On the unit sphere, the geodesics from a point s are the great circles through it, and transporting a vector along the great circle from s to x is the smallest rotation taking s to x, applied to the vector: the rolling of Section 22.5, with the normal turning from s to x in their common plane. The program* ***ga_fem*** *solves the problem of this text on a geodesic sphere of* 2,562 *vertices (an icosahedron subdivided four times, mean edge length* 0.076*), with the source at its top vertex, and compares the direction of the result at every vertex with that exact transport (Figure 17). Out to three quarters of the way to the antipode, the angle between the two is* 0.07° *on average and* 0.30° *at most. Near the antipode the error grows without bound, and it must: every great circle from s reaches the antipode, each with a different transported vector, so there is no shortest path and no correct answer there. The magnitude $|u_i|$ decays by twenty orders of magnitude from the source to the antipode, the screening of (5.6) [$U + \Delta t\, \nabla^* \nabla U = 0$ off s, $U(s) = U_s$] over a distance of order $\sqrt{\Delta t} = h$, and yet the direction, which is all we keep, is accurate wherever it is defined.*

What Stage 8 produced. The global sparse matrices, assembled from the element matrices; the source condition, imposed by replacing the two rows of the source vertex, which is the discrete form of testing only with fields that vanish at s (Stage 4); the solution u of the linear system by one sparse LU

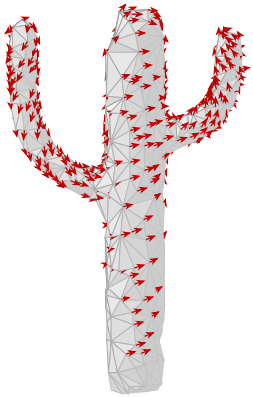


FIGURE 16. The output of the method on the cactus of `FEM.cpp` (502 vertices): the normalized solution at every visible vertex, drawn as a tangent vector.

Algorithm 1 Parallel transport of a tangent vector by one step of the vector heat equation (`main` of `FEM.cpp`).

1: Vertex normals (3.3), vertex frames T_i (3.4), face normals, areas and frames T_f (3.2)
2: $\Delta t \leftarrow \bar{h}^2$, the square of the mean edge length
3: choose the source vertex s and its vector $\bar{u}_s$; $\mathrm{b} \leftarrow 0$, $\mathrm{b}_s \leftarrow \bar{u}_s$
4: **for** every face f **do**
5: $\quad B \leftarrow [x_2 - x_1,\ x_3 - x_1]$, $\quad \nabla\phi_a \leftarrow B(B^TB)^{-1}\hat{\nabla}\hat{\phi}_a$, $\quad g_a^f \leftarrow T_f^T \nabla\phi_a$
6: $\quad R_a^f \leftarrow T_f^T\, Q_{a\to f}\, T_a$ (Rodrigues rotation $n_a \to n_f$)
7: $\quad$ add $\Delta t\, A_f\, (g_a^f \cdot g_b^f)\, (R_a^f)^T R_b^f$ to the block (a,b) of $\Delta t\,\mathrm{L}$, $\quad \frac{A_f}{3} I$ to the block (a,a) of M
8: **end for**
9: $\mathrm{A} \leftarrow I + \mathrm{M}^{-1}(\Delta t\,\mathrm{L})$; replace the rows of s by identity rows
10: solve $\mathrm{A\,u} = \mathrm{b}$ by sparse LU
11: for every vertex i: draw $T_i\, u_i/|u_i|$

factorization; and, read back with the vertex frames of Stage 5, a unit tangent vector at every vertex: the source vector transported over the surface.

PART I · THE DERIVATION WITH MATRICES

16. Summary: From the Equations to the Code

Algorithm 1 lists the steps of the program, and Table 3 the formula behind every function. Read from top to bottom, the table is the derivation of this text in the order the code executes it: the geometry first, then the connection,

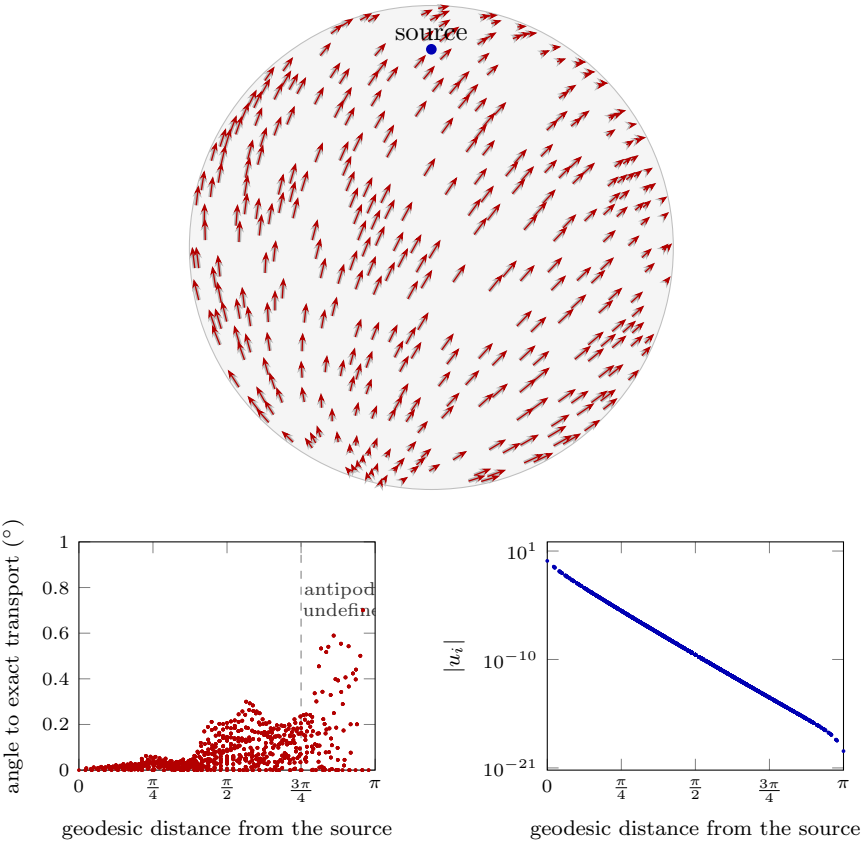


FIGURE 17. Validation on the sphere (Example 15.1). Top: the computed directions (thin red) over the exact parallel transport along great circles from the source (thick gray), on the visible half of the sphere. Bottom: the angle between the two at every vertex against the geodesic distance from the source (beyond the dashed line, three quarters of the way, the errors grow towards the antipode, where transport is not defined, and are not shown), and the magnitude of the solution, on a logarithmic scale.

then the element matrices, their assembly, the boundary condition, and the solve. The last column gives the stage of the recipe of Section 2 to which each line belongs. The code begins with Stage 5, because the frames and the transports are needed by everything that follows, and it contains no trace of Stages 1 to 4 at all: those stages are the derivation that tells the code what to compute, and Stage 2 is the guarantee that what it computes is the best available approximation.

TABLE 3. Every formula and the function of `FEM.cpp` that evaluates it.

Quantity	Equation	Function	Stage
vertex normal n_i	(3.3)	`Mesh::computeNormals`	5
vertex frame T_i	(3.4)	`computeVertexTangentSpace`, `project`	5
face normal n_f, area A_f	(3.2)	`computeFaceNormals`	5
face frame T_f	Section 3	`computeFaceTangentSpace`	5
smallest rotation $Q_{i\to f}$	(4.6)	`alignVectors`, `bracket`	5
discrete connection R_i^f	(4.7)	`Rvf`	5
vertex-to-vertex transport (variant)	(4.8)	`Rvv`	5
edge matrix B, $B(B^TB)^{-1}$	(12.3), Prop. 12.1	`B`, `Binv`	7
gradients $\nabla\phi_a$, g_a^f	(12.4), (12.5)	`gradN`, `grad_i`	7
element stiffness L^f	(13.1)	`AssembleStiffnessElement-Embedded2`	7
lumped mass M	(13.3)	`AssembleDiagonalMassMatrix`	7, 8
assembly of $\Delta t\,\mathrm{L}$	(14.1), (14.2)	`AssembleMatrix2`	8
time step $\Delta t = \bar{h}^2$	Section 5	`computeEdgeLengths`	1
$I + \Delta t\,\mathrm{M}^{-1}\mathrm{L}$	(15.1)	`IplusMinvTimesA`	8
Dirichlet rows, LU factorization	Section 15	`PreFactor`	8
normalization and drawing	Section 15	`display`	8

The recipe once more. Here is the whole chain, each stage with the equation it produced.

1. *Stage 1, the strong form* (5.7): one backward Euler step of the vector heat equation, with the source at s,

$$U + \Delta t\,\nabla^*\nabla U = U^0 \quad \text{on } \Omega, \qquad U(s) = U_s.$$

2. *Stage 2, the function space:* H^1, the tangent fields with square-integrable covariant derivatives, in which the form a of (6.3) is symmetric, bounded and coercive, $a(U,U) \geq \min(1,\Delta t)\,\|U\|_{H^1}^2$; so the weak problem has exactly one solution, and the Galerkin solution in a conforming subspace is the best approximation in the energy norm (Céa).
3. *Stage 3, the energy* (7.1):

$$J(U) = \frac{1}{2}\int_\Omega |U|^2\,dA + \frac{\Delta t}{2}\int_\Omega |\nabla U|^2\,dA - \int_\Omega U^0\cdot U\,dA.$$

4. *Stage 4, the weak form* (8.2), the condition that J be stationary, for every test field V:

$$\int_\Omega U\cdot V\,dA + \Delta t\int_\Omega \nabla U : \nabla V\,dA = \int_\Omega U^0\cdot V\,dA.$$

5. *Stage 5, the discrete space:* the vector hat functions (9.2), and their covariant derivatives (9.3),

$$\Psi_{i,k}\big|_f = \phi_i \, T_f R_i^f \varepsilon_k, \qquad T_f^T \nabla \Psi_{i,k}\big|_f = (R_i^f \varepsilon_k)(g_i^f)^T.$$

6. *Stage 6, the discrete solution and the test functions:* $U = \sum_{j,l} u_{j,l} \Psi_{j,l}$ and $V = \Psi_{i,k}$ in the weak form give the Galerkin equations (10.2) and, with the interpolated data, the system (10.3),

$$(\mathrm{M} + \Delta t \, \mathrm{L})\, \mathrm{u} = \mathrm{M}\, \mathrm{u}^0.$$

7. *Stage 7, the element integrals:* the stiffness block (11.1), with the gradients of Proposition 12.1, and the lumped mass (13.3),

$$\mathrm{L}_{ab}^f = A_f \left(g_a^f \cdot g_b^f\right) (R_a^f)^T R_b^f, \qquad \nabla \phi_a = B(B^T B)^{-1} \hat{\nabla} \hat{\phi}_a, \qquad \mathrm{M}_{aa}^f = \frac{A_f}{3} I_2.$$

8. *Stage 8, assembly and solution:* the sums (14.1), the scaled system (15.1) with the rows of the source replaced by rows of the identity, and one sparse LU solve,

$$\mathrm{L} = \sum_f P_f^T \, \mathrm{L}^f \, P_f, \qquad (I + \Delta t \, \mathrm{M}^{-1} \mathrm{L})\, \mathrm{u} = \mathrm{u}^0.$$

To apply the method to another PDE, replace the equation of Stage 1, check the three properties of Stage 2 for the new form a, and repeat the computations of Stages 4, 6 and 7 with the new integrand; Stages 5 and 8 do not change.

Three ideas carry the whole construction. The weak form trades the second derivatives of the heat equation for first derivatives of a trial field and a test field, so that piecewise linear fields can be used. Piecewise linear *tangent* fields exist only once the vectors of the three vertices of a face are brought into the plane of the face, and that is done by the discrete Levi-Civita connection, a 2×2 rotation per vertex and face. And because a face is flat, the covariant derivative inside it is an ordinary derivative, so that the stiffness of the connection Laplacian is the stiffness of the scalar Laplacian, multiplied on every edge by the transport between its two vertices.

Part II

The Same Derivation in Geometric Algebra, and What Is Intrinsic

Part I worked with matrices: frames as 3×2 arrays, transports as 2×2 rotations, coordinates as pairs of numbers. Each of those objects is a geometric object in disguise: a plane, a rotation, a direction in a plane. This part derives the finite element method again, from the beginning, with the objects themselves, in the geometric algebra $\mathbb{G}_3$ of space [2,5]. Nothing is taken over from Part I except the problem and the mesh. Each result is compared with its matrix counterpart only at the end of its derivation.

The recipe. We follow the same eight stages as the other two parts:

1. *The PDE (strong form):* the unknown, the domain, the equation and its side condition, written with geometric algebra.
2. *The function spaces:* where the solution lives, with which inner product, and why.
3. *The variational form:* an energy whose minimizer solves the equation.
4. *The weak form (Galerkin):* multiply by a test field, integrate, and integrate by parts with the divergence theorem of geometric calculus.
5. *The discretization:* the mesh, the tangent planes of its faces and vertices, the rotors that connect them, and the spinor hat functions.
6. *The discrete solution and the test functions:* the unknown as a combination of hat functions, tested against each of them.
7. *The element integrals:* one complex number per pair of vertices of each face, computed with the reciprocal frame.
8. *Assembly and the linear system:* a complex Hermitian system, which turns out to be the real system of Part I in disguise.

Curved surfaces. On a curved surface the unknowns are vectors that live in different planes, and the mesh is made of flat pieces that only approximate the surface. Differential geometry says what the equation means there. We use it where a stage needs it: the covariant derivative and the second fundamental form in Stage 1, to say what the equation means and which of its parts are *intrinsic*, that is, depend only on lengths and angles measured within the surface; parallel transport and holonomy in Stage 5, to say why the basis functions must carry a connection.

Each formula that the program `ga_fem.cpp` evaluates is shown next to the lines of code that evaluate it, at the stage it implements.

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

17. The Tools: the Geometric Algebra of Space

We recall the rules of $\mathbb{G}_3$ that are used below, and derive each one from the previous ones, so that every later step can point to a rule.

The product of two vectors. The geometric product of two vectors a, b is a scalar plus a bivector,

$$ab = a \cdot b + a \wedge b.$$

The inner product is symmetric, $b \cdot a = a \cdot b$, and the outer product is anti-symmetric, $b \wedge a = -\,a \wedge b$. So the product in the other order is

$$ba = b \cdot a + b \wedge a = a \cdot b - a \wedge b.$$

Three rules follow at once.

- *A vector squared.* With $b = a$: $a \wedge a = -\,a \wedge a$, so $a \wedge a = 0$, and $aa = a \cdot a = |a|^2$.
- *Orthogonal vectors anticommute.* If $a \cdot b = 0$, then $ab = 0 + a \wedge b$ and $ba = 0 - a \wedge b$, so $ab = -\,ba$.
- *Parallel vectors commute.* If $b = \lambda a$, then $a \wedge b = \lambda\, a \wedge a = 0$, so $ab = a \cdot b = ba$.

The pseudoscalar. Let e_1, e_2, e_3 be the orthonormal basis of space and $I = e_1e_2e_3$. Its square, moving one factor at a time and using that distinct basis vectors anticommute:

$$\begin{aligned}
I^2 &= e_1e_2e_3\, e_1e_2e_3 \\
&= e_1e_2\,(e_3e_1)\,e_2e_3 = -\,e_1e_2\,e_1e_3\,e_2e_3 && e_3e_1 = -e_1e_3 \\
&= -\,e_1\,(e_2e_1)\,e_3e_2e_3 = +\,e_1e_1\,e_2e_3e_2e_3 && e_2e_1 = -e_1e_2 \\
&= e_2\,(e_3e_2)\,e_3 = -\,e_2e_2\,e_3e_3 && e_1e_1 = 1,\ e_3e_2 = -e_2e_3 \\
&= -1 && e_2e_2 = e_3e_3 = 1.
\end{aligned}$$

I commutes with every vector. For e_1: $Ie_1 = e_1e_2e_3e_1$, and moving the last e_1 to the front passes e_3 and e_2, two sign changes, so $Ie_1 = e_1e_1e_2e_3 = e_1I$. The same two swaps work for e_2 and e_3, and by linearity $Ia = aI$ for every vector a, hence for every product of vectors. Finally, I turns a vector into the bivector of the orthogonal plane:

$$a \wedge b = I\,(a \times b).$$

On the basis: $Ie_3 = e_1e_2e_3e_3 = e_1e_2 = e_1 \wedge e_2$ and $e_1 \times e_2 = e_3$; in the same way $Ie_1 = e_2e_3$ and $Ie_2 = e_3e_1$; both sides are bilinear, so the identity holds for all a, b. In particular every bivector of space is $B = Ib$ for one vector b, its *dual vector*.

A vector times a bivector. Let a be a vector and $B = Ib$ a bivector. Then

$$\begin{aligned}
a\,(Ib) &= I\,(ab) && I \text{ commutes with } a \\
&= I\,(a \cdot b) + I\,(a \wedge b) && ab = a \cdot b + a \wedge b \\
&= I\,(a \cdot b) + I\,I\,(a \times b) && a \wedge b = I(a \times b) \\
&= I\,(a \cdot b) - a \times b && I^2 = -1 \\
&= \underbrace{I\,(a \cdot b)}_{\text{grade 3}} + \underbrace{b \times a}_{\text{grade 1}} && a \times b = -\,b \times a.
\end{aligned}$$

The grade-3 part (a multiple of I, a *pseudoscalar*) is the outer product, and the grade-1 part (a vector) is the inner product:

$$a \wedge (Ib) = I\,(a \cdot b), \qquad a \cdot (Ib) = b \times a.$$

We call this the *splitting rule*. The product in the other order is

$$\begin{aligned}(Ib)\,a = I\,(ba) &= I\,(a\cdot b) - I\,(a\wedge b) && ba = a\cdot b - a\wedge b\\ &= I\,(a\cdot b) - I\,I\,(a\times b) = I\,(a\cdot b) + a\times b && I^2 = -1\\ &= a\wedge (Ib) - a\cdot (Ib) && a\times b = -\,b\times a.\end{aligned}$$

Subtracting the two products, $aB - Ba = 2\,a\cdot B$, so $a\cdot B = \frac{1}{2}(aB - Ba)$, and we write $B\cdot a = -\,a\cdot B$.

Squares and inverses of bivectors. For $B = Ib$,

$$B^2 = I\,b\,I\,b = I\,I\,b\,b = -\,|b|^2,$$

moving the second I past b and then using $I^2 = -1$ and $bb = |b|^2$. So a unit bivector squares to -1, and $B\,(-B/|B|^2) = -B^2/|B|^2 = |B|^2/|B|^2 = 1$, that is, $B^{-1} = -B/|B|^2$.

The reverse. The reverse $\widetilde{X}$ of a product of vectors writes the factors in the opposite order, $\widetilde{ab} = ba$, and it is extended linearly; for any two elements, $\widetilde{XY} = \widetilde{Y}\,\widetilde{X}$. Scalars and vectors are their own reverses. A bivector changes sign: for orthogonal a, b, $\widetilde{ab} = ba = -\,ab$, and every bivector is a sum of such products.

Rotors. A *rotor* is an even element R (a scalar plus a bivector) with $R\widetilde{R} = 1$. It rotates by the sandwich $X \mapsto RX\widetilde{R}$. The rotor that turns by the angle θ in the plane of the unit bivector B, in the sense of B, is $R = e^{-B\theta/2} = \cos\frac{\theta}{2} - B\sin\frac{\theta}{2}$. Rotors are used from Stage 5 on.

The scalar part. The scalar part $\langle X\rangle$ of an element X is its grade-0 component. Two rules move factors inside it.

- *Reversion does not change it*, $\langle\widetilde{X}\rangle = \langle X\rangle$: reversion leaves scalars alone and only changes signs of bivector and trivector parts.
- *It is cyclic*, $\langle XY\rangle = \langle YX\rangle$ for any two elements. Write X and Y in the basis of products e_A of distinct basis vectors $(1,\, e_1,\, e_1e_2,\, \dots)$. The product e_Ae_B is, up to sign, the product of the basis vectors that appear in exactly one of A and B, so it has a scalar part only when $A = B$, and then $e_Ae_B = e_Be_A$. So $\langle e_Ae_B\rangle = \langle e_Be_A\rangle$ for every pair, and by linearity $\langle XY\rangle = \langle YX\rangle$. For two vectors this is $\langle ab\rangle = a\cdot b = \langle ba\rangle$.

The scalar part will be our inner product, and the cyclic rule is what lets us move factors around inside it.

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

18. Stage 1: The PDE (Strong Form) in Geometric Algebra

What we want. We want to write one backward Euler step of the vector heat equation on a curved surface, with a source condition, using only geometric algebra. The unknown is a field of tangent vectors. So we need, in order:

the tangent planes; the derivative of a tangent field that stays tangent (the covariant derivative); a way of storing a tangent vector as a number (the spinor coordinate); and the equation itself.

18.1. The tangent plane as a bivector

The tangent bivector. In Part I a tangent plane was described by its unit normal n. What the method uses is the plane and its orientation, the sense in which "counterclockwise" turns in it. Geometric algebra has an object that is exactly an oriented plane: a unit bivector. The bivector of the plane orthogonal to n is In, by the duality $a \wedge b = I(a \times b)$ of Section 17. So at each point of the surface we replace the normal by

$$\mathsf{T} = In, \tag{18.1}$$

the *tangent bivector*: the tangent plane itself, as an oriented plane element of unit area.

The same plane, written with a frame. Take an orthonormal, positively oriented frame (t, b) of the tangent plane, $b = n \times t$, as in Part I. Then

$$\begin{aligned} t\,b &= t \cdot b + t \wedge b && \text{product of two vectors} \\ &= t \wedge b && t \cdot b = 0 \\ &= I\,(t \times b) && a \wedge b = I(a \times b) \\ &= I\,n = \mathsf{T} && t \times b = n. \end{aligned}$$

The step $t \times b = n$ holds because (t, b, n) is a right-handed orthonormal triple, ordered like (e_1, e_2, e_3). So T is the product of the two frame vectors, and it is the same for every positive frame of the plane: the frame vectors change when the frame is rotated, their product does not.

Its square. Using $\mathsf{T} = tb$ and the anticommutation of the orthogonal vectors t and b:

$$\begin{aligned} \mathsf{T}^2 = (tb)(tb) &= t\,(bt)\,b && \text{associativity} \\ &= t\,(-tb)\,b && bt = -tb \\ &= -(tt)(bb) && \text{regroup} \\ &= -1 && tt = bb = 1. \end{aligned}$$

The tangent bivector squares to -1, like the imaginary unit. Its inverse is therefore $\mathsf{T}^{-1} = -\mathsf{T}$, because $\mathsf{T}(-\mathsf{T}) = -\mathsf{T}^2 = 1$. Its reverse is $\widetilde{\mathsf{T}} = \widetilde{tb} = bt = -tb = -\mathsf{T}$.

Tangency. The splitting rule of Section 17, with b replaced by n [$a \wedge (Ib) = I(a \cdot b)$, $a \cdot (Ib) = b \times a$], gives for any vector a of space

$$a \wedge \mathsf{T} = a \wedge (In) = I\,(a \cdot n), \qquad a \cdot \mathsf{T} = a \cdot (In) = n \times a.$$

So $a \wedge \mathsf{T} = 0$ exactly when $a \cdot n = 0$: a vector is tangent exactly when its outer product with T vanishes.

The projection, step by step. We want to split a vector a of space into a tangent part and a normal part using T only. Insert $1 = \mathsf{T}\mathsf{T}^{-1}$ and split the product $a\mathsf{T}$ of a vector and a bivector into its two grades:

$$\begin{aligned} a &= a\,\mathsf{T}\,\mathsf{T}^{-1} && \mathsf{T}\mathsf{T}^{-1} = 1 \\ &= (a\,\mathsf{T})\,\mathsf{T}^{-1} && \text{associativity} \\ &= (a\cdot\mathsf{T} + a\wedge\mathsf{T})\,\mathsf{T}^{-1} && \text{splitting rule} \\ &= (a\cdot\mathsf{T})\,\mathsf{T}^{-1} + (a\wedge\mathsf{T})\,\mathsf{T}^{-1} && \text{distribute.} \end{aligned}$$

We compute each term. The second one, with $a\wedge\mathsf{T} = I(a\cdot n)$ and $\mathsf{T}^{-1} = -\mathsf{T} = -In$:

$$\begin{aligned} (a\wedge\mathsf{T})\,\mathsf{T}^{-1} &= \big(I\,(a\cdot n)\big)\,\big(-In\big) && \text{substitute both} \\ &= -\,I\,I\,(a\cdot n)\,n && a\cdot n \text{ is a scalar} \\ &= (a\cdot n)\,n && I^2 = -1, \end{aligned}$$

the normal component of a. The first one, with $a\cdot\mathsf{T} = n\times a$ and $\mathsf{T}^{-1} = -In$:

$$\begin{aligned} (a\cdot\mathsf{T})\,\mathsf{T}^{-1} &= -\,(a\cdot\mathsf{T})\,\mathsf{T} = -\,(n\times a)\,(In) && \text{substitute both} \\ &= -\,I\,\big((n\times a)\,n\big) && I \text{ commutes} \\ &= -\,I\,\big((n\times a)\cdot n + (n\times a)\wedge n\big) && \text{product of two vectors} \\ &= -\,I\,(n\times a)\wedge n && (n\times a)\cdot n = 0 \\ &= -\,I\,I\,\big((n\times a)\times n\big) && c\wedge d = I(c\times d) \\ &= (n\times a)\times n && I^2 = -1 \\ &= (n\cdot n)\,a - (a\cdot n)\,n && \text{triple product rule (below)} \\ &= a - (a\cdot n)\,n && n\cdot n = 1. \end{aligned}$$

The triple product rule is $(u\times v)\times w = (u\cdot w)\,v - (v\cdot w)\,u$, here with $u = w = n$ and $v = a$. This is a with its normal component removed: the tangent part. So

$$P(a) = (a\cdot\mathsf{T})\,\mathsf{T}^{-1} = -(a\cdot\mathsf{T})\,\mathsf{T}, \qquad a - P(a) = (a\wedge\mathsf{T})\,\mathsf{T}^{-1} = (a\cdot n)\,n, \quad (18.2)$$

and the two parts add up to a, as the first computation said they must. The tangent part $a - (a\cdot n)\,n = (I_3 - nn^T)\,a$ is the matrix $TT^T = I_3 - nn^T$ of (3.1) [$T = [\,t\;\;b\,]$, $T^TT = I_2$, $TT^T = I_3 - nn^T$] applied to a, now written without a choice of frame.

A check in coordinates. Take $n = e_3$, so $\mathsf{T} = Ie_3 = e_1e_2$, and $a = a_1e_1 + a_2e_2 + a_3e_3$. Then $a\cdot\mathsf{T} = n\times a = e_3\times(a_1e_1 + a_2e_2 + a_3e_3) = a_1e_2 - a_2e_1$, and

$$\begin{aligned} -(a\cdot\mathsf{T})\,\mathsf{T} &= -\,(a_1e_2 - a_2e_1)\,e_1e_2 && \text{substitute} \\ &= -\,a_1\,e_2e_1e_2 + a_2\,e_1e_1e_2 && \text{distribute} \\ &= -\,a_1\,(-\,e_1e_2e_2) + a_2\,e_2 && e_2e_1 = -e_1e_2,\;\; e_1e_1 = 1 \\ &= a_1e_1 + a_2e_2 && e_2e_2 = 1, \end{aligned}$$

the tangent part of a, with its e_3 component removed.

18.2. The tangent plane is the complex plane

We now look at what T does to a tangent vector when we multiply by it. This is the step that makes the rest of the part work.

Right multiplication is a quarter turn. Let a be a tangent vector, so $a \cdot n = 0$. Multiply it on the right by $\mathsf{T} = In$:

$$\begin{aligned} a\,\mathsf{T} &= a\,I\,n && \text{definition (18.1) of } \mathsf{T} \\ &= I\,(a\,n) && I \text{ commutes with } a \\ &= I\,(a \cdot n) + I\,(a \wedge n) && \text{product of two vectors} \\ &= I\,(a \wedge n) && a \cdot n = 0\text{: } a \text{ is tangent} \\ &= I\,I\,(a \times n) && a \wedge n = I(a \times n) \\ &= -\,a \times n && I^2 = -1 \\ &= n \times a && \text{antisymmetry of } \times\,. \end{aligned}$$

The fourth line is where tangency is used. For a vector that is not tangent, the pseudoscalar $I\,(a \cdot n)$ survives and $a\mathsf{T}$ is not a vector at all; for a tangent vector it vanishes, and what remains is

$$a\,\mathsf{T} = n \times a. \tag{18.3}$$

Read the right side geometrically. The vector $n \times a$ is orthogonal to n, so it is tangent; it is orthogonal to a; its length is $|n|\,|a| \sin 90^\circ = |a|$, because $n \perp a$; and $(a,\ n \times a,\ n)$ is right-handed, so seen from the tip of n the turn from a to $n \times a$ is counterclockwise. So right multiplication by T turns a tangent vector by a quarter turn, counterclockwise about n, within the tangent plane. (Check with $n = e_3$, $\mathsf{T} = e_1e_2$, $a = e_1$: $a\mathsf{T} = e_1e_1e_2 = e_2$, and $e_3 \times e_1 = e_2$.)

Multiplying on the left turns the other way:

$$\begin{aligned} \mathsf{T}\,a = I\,n\,a &= I\,(n \cdot a + n \wedge a) && \text{product of two vectors} \\ &= I\,(n \wedge a) = I\,I\,(n \times a) && n \cdot a = 0,\ \text{duality} \\ &= -\,n \times a = -\,a\,\mathsf{T} && I^2 = -1,\ (18.3). \end{aligned}$$

A tangent vector and T anticommute. We always multiply on the right.

Complex numbers of the plane. Consider the even elements $\alpha + \beta\,\mathsf{T}$ with real α, β. Their product, distributing and using $\mathsf{T}^2 = -1$ once:

$$\begin{aligned} (\alpha + \beta\mathsf{T})(\gamma + \delta\mathsf{T}) &= \alpha\gamma + \alpha\delta\,\mathsf{T} + \beta\gamma\,\mathsf{T} + \beta\delta\,\mathsf{T}^2 && \text{distribute} \\ &= (\alpha\gamma - \beta\delta) + (\alpha\delta + \beta\gamma)\,\mathsf{T} && \mathsf{T}^2 = -1. \end{aligned}$$

This is the rule for $(\alpha+\beta\mathrm{i})(\gamma+\delta\mathrm{i})$. The result is symmetric in the two factors, so these elements commute with each other. The reverse is $(\alpha + \beta\mathsf{T})^{\sim} = \alpha + \beta\,\widetilde{\mathsf{T}} = \alpha - \beta\mathsf{T}$, because $\widetilde{\mathsf{T}} = -\mathsf{T}$: *reversion is complex conjugation.*

The exponential. The power series $e^{\mathsf{T}\theta} = \sum_{m \geq 0} (\mathsf{T}\theta)^m / m!$ separates into even and odd powers. For even $m = 2j$, $\mathsf{T}^{2j} = (\mathsf{T}^2)^j = (-1)^j$; for odd

$m = 2j+1$, $\mathsf{T}^{2j+1} = (\mathsf{T}^2)^j\,\mathsf{T} = (-1)^j\,\mathsf{T}$. So

$$e^{\mathsf{T}\theta} = \sum_{j\ge 0} \frac{(-1)^j\theta^{2j}}{(2j)!} + \mathsf{T}\sum_{j\ge 0} \frac{(-1)^j\theta^{2j+1}}{(2j+1)!} = \cos\theta + \mathsf{T}\sin\theta.$$

Its reverse is $\cos\theta - \mathsf{T}\sin\theta = e^{-\mathsf{T}\theta}$, and

$$\begin{aligned} e^{\mathsf{T}\theta}\,e^{-\mathsf{T}\theta} &= (\cos\theta + \mathsf{T}\sin\theta)(\cos\theta - \mathsf{T}\sin\theta) \\ &= \cos^2\theta - \cos\theta\sin\theta\,\mathsf{T} + \sin\theta\cos\theta\,\mathsf{T} - \sin^2\theta\,\mathsf{T}^2 && \text{distribute} \\ &= \cos^2\theta + \sin^2\theta = 1 && \mathsf{T}^2 = -1. \end{aligned}$$

Multiplying a tangent vector a on the right by $e^{\mathsf{T}\theta}$:

$$\begin{aligned} a\,e^{\mathsf{T}\theta} &= a\,(\cos\theta + \mathsf{T}\sin\theta) = a\cos\theta + (a\,\mathsf{T})\sin\theta \\ &= a\cos\theta + (n\times a)\sin\theta, \end{aligned} \tag{18.4}$$

by (18.3) [$a\,\mathsf{T} = n\times a$]. This is a rotated by θ about n: $\cos\theta$ times a plus $\sin\theta$ times the quarter-turned a (Figure 18). This is why Part I could read the coordinates u_i as complex numbers: there the imaginary unit was a notational device; here it is the tangent plane itself.

Spinor coordinates. Fix the first frame vector t. A tangent vector U with coordinates (u_1, u_2), that is $U = u_1 t + u_2 b = Tu$, is

$$U = t\,\psi, \qquad \psi = u_1 + u_2\,\mathsf{T}. \tag{18.5}$$

Indeed:

$$\begin{aligned} t\psi = t\,(u_1 + u_2\,\mathsf{T}) &= u_1\,t + u_2\,(t\,\mathsf{T}) && \text{distribute} \\ &= u_1\,t + u_2\,(n\times t) && (18.3)\text{ with } a = t \\ &= u_1\,t + u_2\,b && b = n\times t. \end{aligned}$$

We call ψ the *spinor coordinate* of U in the frame t. Its length is that of U. Since U is a vector, $\widetilde{U} = U$ and $U^2 = U\widetilde{U} = |U|^2$, so

$$\begin{aligned} |U|^2 = U\,\widetilde{U} = (t\psi)\,(t\psi)^{\sim} = t\,\psi\,\widetilde{\psi}\,t && \widetilde{XY} = \widetilde{Y}\,\widetilde{X},\ \widetilde{t} = t \\ \psi\widetilde{\psi} = (u_1 + u_2\mathsf{T})(u_1 - u_2\mathsf{T}) && \\ = u_1^2 - u_1u_2\mathsf{T} + u_2u_1\mathsf{T} - u_2^2\,\mathsf{T}^2 && \text{distribute} \\ = u_1^2 + u_2^2 && \mathsf{T}^2 = -1 \\ |U|^2 = t\,(u_1^2 + u_2^2)\,t = (u_1^2 + u_2^2)\,tt && \text{scalars commute} \\ = u_1^2 + u_2^2 && tt = 1. \end{aligned}$$

So $|\psi|^2 = \psi\widetilde{\psi} = |U|^2$, and the same computation shows $|t\phi| = |\phi|$ for every complex number ϕ of the plane. The argument of ψ is the angle from t to U. The pair (u_1, u_2) that the code stores is ψ in the basis $(1, \mathsf{T})$. The frame is now a single unit vector t; the second vector $b = t\mathsf{T}$ is determined by the plane.

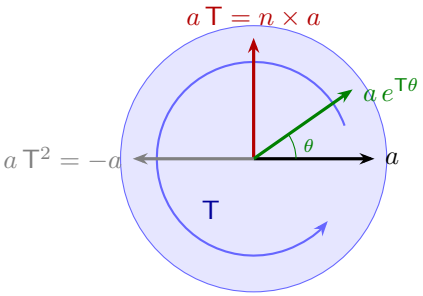


FIGURE 18. The tangent bivector $\mathsf{T} = In$ as an oriented disk in the tangent plane. Right multiplication by T turns a tangent vector a by a quarter turn in the sense of T, twice gives $-a$, and $e^{\mathsf{T}\theta} = \cos\theta + \mathsf{T}\sin\theta$ turns it by θ: the even elements of the plane are the complex numbers, with T as imaginary unit.

Changing the frame vector. If the frame is turned by θ, $t' = t\,e^{\mathsf{T}\theta}$, the coordinate becomes $\psi' = e^{-\mathsf{T}\theta}\psi$, because then the vector is unchanged:

$$t'\psi' = \left(t\,e^{\mathsf{T}\theta}\right)\left(e^{-\mathsf{T}\theta}\psi\right) = t\left(e^{\mathsf{T}\theta}e^{-\mathsf{T}\theta}\right)\psi = t\psi.$$

Section 18.4 builds on this.

The algebra in code. The header `GAConnection.h` implements only the grades the method needs. Here is its first part, with the rules of Section 17 and of this subsection:

```
using Vec3 = Eigen::Vector3d;
using Complex = std::complex<double>;

/// A bivector B = I b, stored by its dual vector b. Unit bivectors square to -1.
struct Bivector {
    Vec3 b;
    Bivector operator*(double s) const { return { b * s }; }
    double norm() const { return b.norm(); }
};

/// The outer product a ^ c = I (a x c): the oriented plane of a and c.
inline Bivector wedge(const Vec3& a, const Vec3& c) { return { a.cross(c) }; }

/// The inner product of a vector and a bivector, a . B, a vector of the plane of B,
     orthogonal to a.
/// For B = I b: a . (I b) = I (a ^ b) = I I (a x b) = b x a.
inline Vec3 dot(const Vec3& a, const Bivector& B) { return B.b.cross(a); }

/// B^-1 = -B / |B|^2, since B^2 = -|B|^2.
inline Bivector inverse(const Bivector& B) { return B * (-1.0 / B.b.squaredNorm()); }

/// A rotor R = w + I b (Section 12). R a ~R rotates the vector a.
struct Rotor {
    double w;
    Vec3 b;
    /// R a ~R. With R = w + I b, the rotation is the one of the unit quaternion (w, -b
        ):
    /// a' = a + 2 w (u x a) + 2 u x (u x a), u = -b.
    Vec3 rotate(const Vec3& a) const {
        const Vec3 u = -b;
```

```
47          const Vec3 t = 2.0 * u.cross(a);
48          return a + w * t + u.cross(t);
49      }
50  };
51
52  /// The geometric product of two vectors, c a = c . a + c ^ a, a rotor up to
        normalization.
53  inline Rotor product(const Vec3& c, const Vec3& a) { return { c.dot(a), c.cross(a) };
        }
54
55  /// The tangent bivector T = I n of a unit normal n (eq. ga_T): the tangent plane as an
        oriented plane element.
56  inline Bivector tangentBivector(const Vec3& n) { return { n }; }
57
58  /// a T for a tangent vector a: the quarter turn n x a (eq. ga_quarter_turn).
59  /// For a in the plane of T the geometric product equals the inner product, a T = a . T
        .
60  inline Vec3 quarterTurn(const Vec3& a, const Bivector& T) { return dot(a, T); }
```

- Lines 19–20: vectors are Eigen vectors, and the complex numbers $\alpha+\beta\,\mathsf{T}$ of a plane are `std::complex`.
- Lines 23–27: a bivector $B = Ib$ is stored by its dual vector b. Since $Ie_1 = e_2e_3$, $Ie_2 = e_3e_1$ and $Ie_3 = e_1e_2$, the components b_x, b_y, b_z are the coefficients of e_2e_3, e_3e_1, e_1e_2. The norm of B is that of b.
- Line 30: the outer product $a \wedge c = I\,(a \times c)$ is stored as the cross product.
- Line 34: the inner product of a vector with a bivector, $a \cdot (Ib) = b \times a$, the vector part of the splitting rule.
- Line 37: $B^{-1} = -B/|B|^2$. For $B = Ib$, $B^2 = I^2b^2 = -|b|^2$, so $B\,(-B/|b|^2) = 1$.
- Lines 40–50: a rotor $R = w + Ib$, used in Stage 5. The sandwich $Ra\widetilde{R}$ is evaluated with the classical formula for a unit quaternion (w, u), $a' = a + 2w\,(u \times a) + 2\,u \times (u \times a)$; lines 47–48 compute it with $t = 2\,u \times a$. The quaternion has $u = -b$ (line 46) because the quaternion units correspond to $-Ie_1, -Ie_2, -Ie_3$. For the rotor $e^{-Ik\theta/2} = \cos\frac{\theta}{2} - Ik\sin\frac{\theta}{2}$ of the rotation by θ about the unit axis k, this gives $u = k\sin\frac{\theta}{2}$, the familiar quaternion of that rotation.
- Line 53: the geometric product of two vectors, $c\,a = c \cdot a + I\,(c \times a)$, stored as a rotor; it is a rotor only after normalization (Section 22.2).
- Line 56: $\mathsf{T} = In$ is stored by its dual vector, which is n itself. The data does not change; its meaning does, and with it the operations allowed on it.
- Line 60: the quarter turn (18.3) $[\,a\,\mathsf{T} = n \times a\,]$, computed as the inner product $a \cdot \mathsf{T} = n \times a$. For a tangent a this is the whole product $a\mathsf{T}$, since the outer part $I(a \cdot n)$ vanishes; for a vector that is not tangent the function silently drops that part, so it must be called only with tangent vectors.

18.3. The vector derivative and the covariant derivative

What we want. The heat equation differentiates the unknown field. On a curved surface the derivative of a tangent field, taken in space, is in general

not tangent. We need the derivative that stays in the tangent plane, and we want to know what is thrown away in getting it.

The directional derivative. Let F be a field on the surface, with values that may be scalars, vectors or any elements of $\mathbb{G}_3$. For a tangent vector a at a point x, take a curve $\gamma(s)$ on the surface with $\gamma(0) = x$ and $\gamma'(0) = a$, and define

$$a \cdot \nabla F = \frac{d}{ds} F(\gamma(s))\Big|_{s=0},$$

the rate of change of each component of F in space as we move along a. (Part I wrote it $D_X U$; here the letter D is reserved for spinors.) It is linear in a, and it obeys the product rule, because it differentiates components: $a \cdot \nabla(FG) = (a \cdot \nabla F)\, G + F\, (a \cdot \nabla G)$, keeping the order of the factors.

The vector derivative. The vector derivative collects the directional derivatives along an orthonormal basis e_1, e_2 of the tangent plane into one object, with the geometric product:

$$\nabla F = \sum_{k=1}^{2} e_k\, (e_k \cdot \nabla F). \tag{18.6}$$

It does not depend on the basis. Write $L(a) = a \cdot \nabla F$, linear in a, and $L_k = L(e_k)$. Rotate the basis to $e_1' = c\, e_1 + s\, e_2$, $e_2' = -s\, e_1 + c\, e_2$, with $c = \cos\alpha$, $s = \sin\alpha$. By linearity $L(e_1') = cL_1 + sL_2$ and $L(e_2') = -sL_1 + cL_2$. Then

$$e_1' L(e_1') = (ce_1 + se_2)(cL_1 + sL_2) = c^2 e_1 L_1 + cs\, e_1 L_2 + sc\, e_2 L_1 + s^2 e_2 L_2,$$
$$e_2' L(e_2') = (-se_1 + ce_2)(-sL_1 + cL_2) = s^2 e_1 L_1 - sc\, e_1 L_2 - cs\, e_2 L_1 + c^2 e_2 L_2.$$

Adding, the cross terms cancel and $e_1' L(e_1') + e_2' L(e_2') = (c^2 + s^2)(e_1 L_1 + e_2 L_2) = e_1 L_1 + e_2 L_2$.

For a scalar function f the scalar $e_k \cdot \nabla f$ commutes with e_k, so $\nabla f = \sum_k (e_k \cdot \nabla f)\, e_k$, the gradient, a tangent vector. The notation is consistent: for a tangent $a = \sum_k (a \cdot e_k)\, e_k$,

$$a \cdot (\nabla f) = \sum_k (e_k \cdot \nabla f)\, (a \cdot e_k) = \Big(\sum_k (a \cdot e_k)\, e_k \Big) \cdot \nabla f = a \cdot \nabla f,$$

by linearity of the directional derivative in its direction. For a vector field W, write $w_k = e_k \cdot \nabla W$, a vector; each product $e_k w_k$ splits into a scalar and a bivector,

$$\nabla W = \sum_k e_k\, w_k = \sum_k e_k \cdot w_k + \sum_k e_k \wedge w_k,$$

and the scalar part $\langle \nabla W \rangle = \sum_k e_k \cdot (e_k \cdot \nabla W)$ is the *divergence*. In Stage 4 it is what the divergence theorem integrates. On a flat face with edge vectors that are not orthonormal, the vector derivative needs the reciprocal frame, which we build in Stage 7.

The Gauss formula. Let a be a tangent direction and b a tangent field. The derivative $a \cdot \nabla b$ is a vector of space. The projection (18.2) [$P(a) = -(a \cdot \mathsf{T})\,\mathsf{T}$, $a - P(a) = (a \cdot n)\,n$], with a replaced by $a \cdot \nabla b$, splits it into its tangent and normal parts:

$$\begin{aligned} a \cdot \nabla b &= P(a \cdot \nabla b) + \big(a \cdot \nabla b - P(a \cdot \nabla b)\big) \\ &= P(a \cdot \nabla b) + \big((a \cdot \nabla b) \cdot n\big)\, n. \end{aligned}$$

We name the two parts:

$$a \cdot \nabla b = \underbrace{P(a \cdot \nabla b)}_{\nabla_a b} + \underbrace{\big((a \cdot \nabla b) \cdot n\big)\, n}_{\mathrm{II}(a,b)\, n}. \tag{18.7}$$

This is the *Gauss formula*. Its tangent part is the *covariant derivative*; with P from (18.2),

$$\nabla_a b = P(a \cdot \nabla b) = -\big((a \cdot \nabla b) \cdot \mathsf{T}\big)\,\mathsf{T},$$

which is (4.3) [$\nabla_X U = (I - nn^T)\, D_X U$] of Part I, written with the plane instead of the normal. The coefficient of the normal part is the *second fundamental form*

$$\mathrm{II}(a,b) = (a \cdot \nabla b) \cdot n = -\,b \cdot (a \cdot \nabla n).$$

The second expression comes from the identity $b \cdot n = 0$, which holds at every point because b is tangent. Differentiate it along a with the product rule:

$$0 = a \cdot \nabla(b \cdot n) = (a \cdot \nabla b) \cdot n + b \cdot (a \cdot \nabla n) \quad \Longrightarrow \quad (a \cdot \nabla b) \cdot n = -\,b \cdot (a \cdot \nabla n).$$

In words: $\mathrm{II}(a,b)$ measures how fast the normal tilts in the direction b as we move along a. It is the bending of the surface, and it is zero on a plane, where n is constant. It sees nothing of how b turns within the surface.

The cylinder. Take the cylinder of radius r about the z axis, the points $(r\cos\varphi, r\sin\varphi, z)$, with the outward normal $n = (\cos\varphi, \sin\varphi, 0)$, and the horizontal unit field $b = e_\varphi = (-\sin\varphi, \cos\varphi, 0)$, which goes around the cylinder.

- Move around the cylinder at unit speed: $\gamma(s) = (r\cos\frac{s}{r}, r\sin\frac{s}{r}, z_0)$, so $a = \gamma'(0) = e_\varphi$. Along the curve $b(\gamma(s)) = (-\sin\frac{s}{r}, \cos\frac{s}{r}, 0)$, and

  $$a \cdot \nabla b = \frac{d}{ds}\Big(-\sin\frac{s}{r},\ \cos\frac{s}{r},\ 0\Big) = -\frac{1}{r}\Big(\cos\frac{s}{r},\ \sin\frac{s}{r},\ 0\Big) = -\frac{1}{r}\,n.$$

  The derivative is entirely normal. Its tangent part is $\nabla_a b = P(-\frac{1}{r}n) = -\frac{1}{r}\big(n - (n \cdot n)\,n\big) = 0$, and $\mathrm{II}(a,b) = (-\frac{1}{r}n) \cdot n = -\frac{1}{r}$. The second expression agrees: $n(\gamma(s)) = (\cos\frac{s}{r}, \sin\frac{s}{r}, 0)$ gives $a \cdot \nabla n = \frac{1}{r}\,e_\varphi$, and $-\,b \cdot (a \cdot \nabla n) = -\frac{1}{r}\,e_\varphi \cdot e_\varphi = -\frac{1}{r}$. The size $1/r$ is the curvature of the circle we walk along; the sign says that the surface bends away from the side to which n points.
- Move along the axis, $a = e_z$: b does not change, $a \cdot \nabla b = 0$, so $\nabla_a b = 0$ and $\mathrm{II}(e_z, b) = 0$. In that direction the cylinder does not bend.

In both directions $\nabla_a b = 0$. An inhabitant of the surface agrees: unroll the cylinder into a flat sheet, and the circles around it become parallel straight lines, and e_φ becomes a constant field. All the change that we see in space

is bending, recorded by II. On a sphere of radius r, with $n = x/r$, along any curve $n(\gamma(s)) = \gamma(s)/r$, so $a \cdot \nabla n = a/r$ and $\mathrm{II}(a,a) = -\,a \cdot (a/r) = -1/r$ for every unit tangent a: the sphere bends equally in every direction.

Metric compatibility. For tangent fields U, V, the covariant derivative explains every change of their inner product:

$$\begin{aligned} a \cdot \nabla(U \cdot V) &= (a \cdot \nabla U) \cdot V + U \cdot (a \cdot \nabla V) && \text{product rule} \\ &= \big(\nabla_a U + \mathrm{II}(a,U)\, n\big) \cdot V \\ &\quad + U \cdot \big(\nabla_a V + \mathrm{II}(a,V)\, n\big) && \text{Gauss (18.7) twice} \\ &= \nabla_a U \cdot V + \mathrm{II}(a,U)\,(n \cdot V) \\ &\quad + U \cdot \nabla_a V + \mathrm{II}(a,V)\,(U \cdot n) && \text{distribute} \\ &= \nabla_a U \cdot V + U \cdot \nabla_a V && n \cdot V = U \cdot n = 0. \end{aligned}$$

This is the metric compatibility (4.4) [$X(U \cdot V) = \nabla_X U \cdot V + U \cdot \nabla_X V$] of Part I. It will give the product rule for spinors and Green's identity in Stage 4.

Why the covariant derivative is intrinsic. Imagine a two-dimensional being living on the surface. It can measure lengths of curves drawn on the surface and angles between them, and nothing else. Quantities computed from lengths and angles alone are *intrinsic*; quantities that also depend on how the surface sits in space, like n, T as an element of $\mathbb{G}_3$, and II, are *extrinsic*. The fundamental theorem of Riemannian geometry [1] says that there is exactly one way of differentiating tangent fields that is metric compatible, as just shown, and torsion free, $\nabla_a b - \nabla_b a = [a,b]$ (the infinitesimal parallelogram built from two directions closes up), and that this connection is given by a formula (the Koszul formula) that involves only inner products of fields and their derivatives, that is, only lengths and angles. The projection $\nabla_a b = P(a \cdot \nabla b)$ has both properties, so it is that connection, and it is intrinsic, although we computed it with the extrinsic plane T. Hence the equation we are about to write is intrinsic. The discrete method computes it with extrinsic tools, and Section 22.5 examines what that costs.

18.4. Spinor coordinates and the connection form

What we want. The code never stores vectors of space; it stores coordinates. So we need the covariant derivative of a field written through its spinor coordinate, $U = t\psi$, in terms of the derivatives of ψ. Part I did this with matrices in (4.5) [$T^T \nabla_X U = X \cdot \nabla u + \omega(X)\, u,\ \omega(X) = T^T D_X T$]; here the 2×2 matrix $\omega(X)$ becomes a single number times T.

The frame field and its turning. Choose a frame field, a unit tangent vector $t(x)$ at each point, with $b = t\mathsf{T}$ the second frame vector. As we move along a tangent direction a, its covariant derivative $\nabla_a t$ says how the frame turns within the surface. Two facts about it:

- it is tangent, by definition of ∇_a;

- it is orthogonal to t. Metric compatibility with $U = V = t$, and $t \cdot t = 1$ constant, give

$$0 = a \cdot \nabla(t \cdot t) = \nabla_a t \cdot t + t \cdot \nabla_a t = 2\, t \cdot \nabla_a t.$$

Expand the tangent vector $\nabla_a t$ in the orthonormal basis (t, b) of the plane:

$$\begin{aligned} \nabla_a t &= (t \cdot \nabla_a t)\, t + (b \cdot \nabla_a t)\, b && \text{orthonormal expansion} \\ &= 0 + \omega(a)\, b && \text{second fact; } \omega(a) := b \cdot \nabla_a t \\ &= \omega(a)\, t\, \mathsf{T} && b = t\mathsf{T}. \end{aligned}$$

So

$$\nabla_a t = \omega(a)\, t\, \mathsf{T}, \tag{18.8}$$

where the number $\omega(a) = b \cdot \nabla_a t$, linear in a, is the rate at which the frame turns, measured against parallel transport. It is the *connection form* of the frame. In (4.5) [$\omega(X) = T^T D_X T$] it appeared as the entry $b \cdot D_X t$ of the 2×2 matrix $\omega(X) = \begin{pmatrix} 0 & -\theta \\ \theta & 0 \end{pmatrix}$, with $\theta = \omega(X)$.

The second frame vector. Expand $\nabla_a b$ in the same basis, $\nabla_a b = (t \cdot \nabla_a b)\, t + (b \cdot \nabla_a b)\, b$, and find the two coefficients by metric compatibility:

$$\begin{aligned} t \cdot b = 0 &\implies 0 = \nabla_a t \cdot b + t \cdot \nabla_a b = \omega(a) + t \cdot \nabla_a b \implies t \cdot \nabla_a b = -\,\omega(a), \\ b \cdot b = 1 &\implies 0 = 2\, b \cdot \nabla_a b \implies b \cdot \nabla_a b = 0. \end{aligned}$$

So $\nabla_a b = -\,\omega(a)\, t$. In geometric algebra this is $\omega(a)\, b\, \mathsf{T}$, because $b\mathsf{T} = (t\mathsf{T})\mathsf{T} = t\,\mathsf{T}^2 = -t$. Both frame vectors obey the same rule, $\nabla_a e = \omega(a)\, e\, \mathsf{T}$: the frame turns rigidly.

The covariant derivative of a field. Write $U = t\psi = u_1 t + u_2 b$ with coordinate functions u_1, u_2. First the product rule for one term, $u_1 t$: the directional derivative in space gives $a \cdot \nabla(u_1 t) = (a \cdot \nabla u_1)\, t + u_1\, (a \cdot \nabla t)$; the projection P is linear and leaves the tangent vector t unchanged, so

$$\nabla_a(u_1 t) = P\big((a \cdot \nabla u_1)\, t + u_1\, (a \cdot \nabla t)\big) = (a \cdot \nabla u_1)\, t + u_1\, \nabla_a t,$$

and the same holds for $u_2 b$. Now factor by factor:

$$\begin{aligned} \nabla_a U &= \nabla_a(u_1 t) + \nabla_a(u_2 b) && P \text{ is linear} \\ &= (a \cdot \nabla u_1)\, t + u_1\, \nabla_a t + (a \cdot \nabla u_2)\, b + u_2\, \nabla_a b && \text{product rule} \\ &= (a \cdot \nabla u_1)\, t + u_1\, \big(\omega(a)\, b\big) + (a \cdot \nabla u_2)\, b + u_2\, \big(-\,\omega(a)\, t\big) && \nabla_a t,\ \nabla_a b \\ &= \big(a \cdot \nabla u_1 - \omega(a)\, u_2\big)\, t + \big(a \cdot \nabla u_2 + \omega(a)\, u_1\big)\, b && \text{collect } t, b. \end{aligned}$$

Now write the result as t times a complex number, using $\alpha\, t + \beta\, b = \alpha\, t + \beta\, t\mathsf{T} = t\,(\alpha + \beta\, \mathsf{T})$:

$$\nabla_a U = t\,\Big(\big(a \cdot \nabla u_1 - \omega(a)\, u_2\big) + \big(a \cdot \nabla u_2 + \omega(a)\, u_1\big)\, \mathsf{T}\Big).$$

The complex number in the large parentheses, regrouped:

$$\begin{aligned}
&\big(a\cdot\nabla u_1-\omega(a)\,u_2\big)+\big(a\cdot\nabla u_2+\omega(a)\,u_1\big)\,\mathsf{T}\\
&\quad=\big(a\cdot\nabla u_1+(a\cdot\nabla u_2)\,\mathsf{T}\big)+\omega(a)\,\big(u_1\,\mathsf{T}-u_2\big) && \text{regroup}\\
&\quad=a\cdot\nabla\psi+\omega(a)\,\big(u_1\,\mathsf{T}+u_2\,\mathsf{T}^2\big) && -1=\mathsf{T}^2\\
&\quad=a\cdot\nabla\psi+\omega(a)\,\mathsf{T}\,\big(u_1+u_2\,\mathsf{T}\big) && \text{factor }\mathsf{T}\\
&\quad=a\cdot\nabla\psi+\omega(a)\,\mathsf{T}\,\psi && \psi=u_1+u_2\mathsf{T}.
\end{aligned}$$

Here $a\cdot\nabla\psi=(a\cdot\nabla u_1)+(a\cdot\nabla u_2)\,\mathsf{T}$ means: differentiate the two real coefficients and keep T as the imaginary unit. This convention is used for every derivative of a spinor below. So

$$\nabla_a(t\psi)=t\,\big(a\cdot\nabla\psi+\omega(a)\,\mathsf{T}\,\psi\big)=t\,D_a\psi,\qquad D_a=a\cdot\nabla+\omega(a)\,\mathsf{T}. \quad (18.9)$$

On spinor coordinates, the Levi-Civita connection is the ordinary derivative of the coordinates plus a multiplication by $\omega\,\mathsf{T}$, an infinitesimal rotation that corrects for the turning of the frame. It is (4.5) with the matrix $\begin{pmatrix}0&-\theta\\ \theta&0\end{pmatrix}$ replaced by the number $\theta\,\mathsf{T}$. The curvature of this connection is $d\omega$, which with the orientation $b=t\mathsf{T}=n\times t$ used here is minus the Gaussian curvature times the area form, $d\omega=-K\,dA$ [6] (Remark 18.1).

Frame independence. The frame t is arbitrary: the code takes the first edge of each vertex, projected, and any other choice would be as good. Nothing geometric may depend on it. We show that when the frame changes, $D\psi$ changes in exactly the same way as ψ, so $t\,D\psi$, the actual derivative vector, does not change. Rotate the frame by a field of angles $\theta(x)$, $t\to t'=t\,e^{\mathsf{T}\theta}$ (Figure 19). Three things change.

The coordinates. The same vector is $U=t\psi=t'\psi'$. Substitute $t'=t\,e^{\mathsf{T}\theta}$ and solve for ψ':

$$\begin{aligned}
t\,\psi&=\big(t\,e^{\mathsf{T}\theta}\big)\,\psi' && \text{substitute } t'\\
t\,t\,\psi&=t\,t\,e^{\mathsf{T}\theta}\,\psi' && \text{multiply on the left by } t\\
\psi&=e^{\mathsf{T}\theta}\,\psi' && tt=1\\
e^{-\mathsf{T}\theta}\,\psi&=e^{-\mathsf{T}\theta}e^{\mathsf{T}\theta}\,\psi'=\psi' && \text{multiply by } e^{-\mathsf{T}\theta}.
\end{aligned}$$

So $\psi'=e^{-\mathsf{T}\theta}\psi$: the frame turns forward by θ, so the coordinate, measured from the frame, turns back by θ.

The connection form. The new frame vector, multiplied out with $e^{\mathsf{T}\theta}=\cos\theta+\mathsf{T}\sin\theta$ and $t\mathsf{T}=b$, is

$$t'=t\,(\cos\theta+\mathsf{T}\sin\theta)=\cos\theta\;t+\sin\theta\;b.$$

Differentiate each term with the product rule. The derivative of $\cos\theta$ along a is $-\sin\theta\,(a\cdot\nabla\theta)$, that of $\sin\theta$ is $\cos\theta\,(a\cdot\nabla\theta)$, and $\nabla_a t=\omega\,b$, $\nabla_a b=-\omega\,t$ (we write ω for $\omega(a)$ and θ_a for $a\cdot\nabla\theta$):

$$\begin{aligned}
\nabla_a(\cos\theta\;t)&=-\sin\theta\;\theta_a\;t+\cos\theta\;\omega\,b,\\
\nabla_a(\sin\theta\;b)&=\cos\theta\;\theta_a\;b-\sin\theta\;\omega\,t.
\end{aligned}$$

Adding, and grouping the terms with θ_a and the terms with ω:

$$\begin{aligned}\nabla_a t' &= \theta_a\big(-\sin\theta\, t + \cos\theta\, b\big) + \omega\big(-\sin\theta\, t + \cos\theta\, b\big) && \text{group}\\ &= (\omega+\theta_a)\big(-\sin\theta\, t + \cos\theta\, b\big) && \text{factor.}\end{aligned}$$

The vector in parentheses is the quarter turn of t':

$$\begin{aligned}t'\,\mathsf{T} &= (\cos\theta\, t + \sin\theta\, b)\,\mathsf{T} = \cos\theta\, t\mathsf{T} + \sin\theta\, b\mathsf{T} && \text{distribute}\\ &= \cos\theta\, b + \sin\theta\,(-t) && t\mathsf{T}=b,\ b\mathsf{T}=-t\\ &= -\sin\theta\, t + \cos\theta\, b.\end{aligned}$$

So $\nabla_a t' = (\omega(a) + a\cdot\nabla\theta)\, t'\mathsf{T}$. Comparing with the definition (18.8) in the new frame, $\nabla_a t' = \omega'(a)\, t'\mathsf{T}$, the new connection form is

$$\omega' = \omega + d\theta, \qquad \text{that is,}\quad \omega'(a) = \omega(a) + a\cdot\nabla\theta :$$

the turning rate of the new frame is that of the old one plus the rate at which we added extra turning.

The covariant derivative. We need the derivative of the factor $e^{-\mathsf{T}\theta}$ along a. Differentiating its two real coefficients,

$$\begin{aligned}a\cdot\nabla\, e^{-\mathsf{T}\theta} &= a\cdot\nabla(\cos\theta - \mathsf{T}\sin\theta) = -\theta_a\,(\sin\theta + \mathsf{T}\cos\theta),\\ \mathsf{T}\, e^{-\mathsf{T}\theta} &= \mathsf{T}\cos\theta - \mathsf{T}^2\sin\theta = \sin\theta + \mathsf{T}\cos\theta && \mathsf{T}^2=-1,\end{aligned}$$

so $a\cdot\nabla\, e^{-\mathsf{T}\theta} = -\theta_a\,\mathsf{T}\, e^{-\mathsf{T}\theta}$. Now write $D'_a\psi'$ by (18.9) in the new frame, $D'_a\psi' = a\cdot\nabla\psi' + \omega'(a)\,\mathsf{T}\,\psi'$, and substitute $\psi' = e^{-\mathsf{T}\theta}\psi$ and $\omega' = \omega + \theta_a$:

$$\begin{aligned}D'_a\psi' &= a\cdot\nabla\big(e^{-\mathsf{T}\theta}\psi\big) + \big(\omega+\theta_a\big)\,\mathsf{T}\,\big(e^{-\mathsf{T}\theta}\psi\big) && \text{substitute}\\ &= \big(a\cdot\nabla e^{-\mathsf{T}\theta}\big)\,\psi + e^{-\mathsf{T}\theta}\,(a\cdot\nabla\psi)\\ &\quad + \omega\,\mathsf{T}e^{-\mathsf{T}\theta}\psi + \theta_a\,\mathsf{T}e^{-\mathsf{T}\theta}\psi && \text{product rule; distribute}\\ &= \big(-\theta_a\,\mathsf{T}e^{-\mathsf{T}\theta}\big)\,\psi + e^{-\mathsf{T}\theta}\,(a\cdot\nabla\psi)\\ &\quad + \omega\,\mathsf{T}e^{-\mathsf{T}\theta}\psi + \theta_a\,\mathsf{T}e^{-\mathsf{T}\theta}\psi && \text{derivative of } e^{-\mathsf{T}\theta}\\ &= e^{-\mathsf{T}\theta}\,(a\cdot\nabla\psi) + \omega\,\mathsf{T}e^{-\mathsf{T}\theta}\psi && \text{the } \theta_a \text{ terms cancel}\\ &= e^{-\mathsf{T}\theta}\,(a\cdot\nabla\psi) + e^{-\mathsf{T}\theta}\,\omega\,\mathsf{T}\psi && \mathsf{T},\ e^{-\mathsf{T}\theta} \text{ commute}\\ &= e^{-\mathsf{T}\theta}\,\big(a\cdot\nabla\psi + \omega(a)\,\mathsf{T}\psi\big) && \text{factor.}\end{aligned}$$

The extra turning appears twice, once from differentiating the rotated coordinate and once in the new connection form, with opposite signs. The last parenthesis is $D_a\psi$, so

$$D'_a\psi' = e^{-\mathsf{T}\theta}\, D_a\psi. \tag{18.10}$$

The covariant derivative transforms exactly like the coordinates themselves. So $\nabla_a U$ is the same tangent vector in both frames:

$$t'D'_a\psi' = \big(t\, e^{\mathsf{T}\theta}\big)\big(e^{-\mathsf{T}\theta}D_a\psi\big) = t\,\big(e^{\mathsf{T}\theta}e^{-\mathsf{T}\theta}\big)\,D_a\psi = t\,D_a\psi,$$

$|D'_a\psi'| = |e^{-\mathsf{T}\theta}|\,|D_a\psi| = |D_a\psi|$, and the energy, the connection Laplacian and the solution of the heat equation are frame independent too.

Frames with holes. On a closed surface other than a torus there is no continuous unit frame field at all: by the hairy ball theorem every continuous tangent field on a sphere vanishes somewhere. This is not a problem, because of (18.10) [$D'_a\psi' = e^{-\mathsf{T}\theta} D_a\psi$]. Cover the surface with patches, each with its own frame field; where two patches overlap, their coordinates differ by a rotation $e^{-\mathsf{T}\theta}$, and so do their covariant derivatives, so every quantity built from lengths and inner products of ψ and $D\psi$ agrees on the overlap. The discrete method needs even less: one frame vector per vertex and one per face, with no field and no continuity.

In the discrete setting. The same cancellation appears twice in the code. Changing the frame of a face rotates all its connection angles ρ_i^f by the same amount, and only differences $\rho_j^f - \rho_i^f$ enter the stiffness (Stage 5). Changing the frame t_i of a vertex rotates the computed coordinate ψ_i by the opposite angle and changes nothing else: the arrow $t_i\hat{\psi}_i$ drawn by the code (line 1136, Section 25.4) is independent of the choice of t_i in `computeVertexTangentSpace`.

Remark 18.1 (Cartan's structure equations). *Write θ^1, θ^2 for the* coframe *of the frame (t, b), the 1-forms $\theta^1(X) = t \cdot X$ and $\theta^2(X) = b \cdot X$, so that $X = \theta^1(X)\, t + \theta^2(X)\, b$ and $dA = \theta^1 \wedge \theta^2$. Then*

$$d\theta^1 = \omega \wedge \theta^2, \qquad d\theta^2 = -\,\omega \wedge \theta^1, \qquad d\omega = -K\,\theta^1 \wedge \theta^2.$$

The first two determine ω from the coframe, hence from the metric alone; the third says that the curvature is the exterior derivative of the connection form. On the unit sphere, with colatitude ϑ, longitude φ, the point $x = (\sin\vartheta\cos\varphi, \sin\vartheta\sin\varphi, \cos\vartheta)$, $n = x$ and $t = \partial_\vartheta x = (\cos\vartheta\cos\varphi, \cos\vartheta\sin\varphi, -\sin\vartheta)$, the second vector $b = n \times t = (-\sin\varphi, \cos\varphi, 0)$ points along increasing φ. The displacement is $dx = t\,d\vartheta + \sin\vartheta\, b\, d\varphi$, so $\theta^1 = d\vartheta$ and $\theta^2 = \sin\vartheta\, d\varphi$. Write $\omega = f\,d\vartheta + g\,d\varphi$ and use the equations one at a time:

- *$d\theta^1 = d(d\vartheta) = 0$ must equal $\omega \wedge \theta^2 = f \sin\vartheta\; d\vartheta \wedge d\varphi$ (the term $g\,d\varphi \wedge d\varphi$ vanishes), so $f = 0$;*
- *$d\theta^2 = \cos\vartheta\; d\vartheta \wedge d\varphi$ must equal $-\,\omega \wedge \theta^1 = -\,g\; d\varphi \wedge d\vartheta = g\; d\vartheta \wedge d\varphi$, so $g = \cos\vartheta$;*
- *$d\omega = d(\cos\vartheta\; d\varphi) = -\sin\vartheta\; d\vartheta \wedge d\varphi$ must equal $-K\,\theta^1 \wedge \theta^2 = -K \sin\vartheta\; d\vartheta \wedge d\varphi$, so $K = 1$.*

The definition (18.8) gives the same ω directly: along ∂_φ, $\partial_\varphi t = (-\cos\vartheta\sin\varphi, \cos\vartheta\cos\varphi, 0) = \cos\vartheta\; b$, so $\omega(\partial_\varphi) = b \cdot \partial_\varphi t = \cos\vartheta$; along ∂_ϑ, $\partial_\vartheta t = -x$ is normal, so $\omega(\partial_\vartheta) = 0$.

18.5. The vector heat equation and its backward Euler step

The connection Laplacian on spinors. Part I defined the connection Laplacian of a tangent field with an orthonormal frame field e_1, e_2,

$$\nabla^*\nabla U = -\sum_{k=1}^{2} \big(\nabla_{e_k}\nabla_{e_k} U - \nabla_{\nabla_{e_k} e_k} U\big).$$

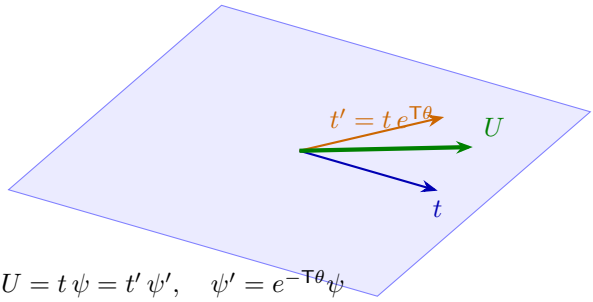


FIGURE 19. A change of frame is a change of gauge. The same tangent vector U has the spinor coordinate ψ in the frame t and $\psi' = e^{-\mathsf{T}\theta}\psi$ in the frame $t' = t\,e^{\mathsf{T}\theta}$; nothing geometric depends on the choice.

We write each of its terms for $U = t\psi$ with (18.9) [$\nabla_a(t\psi) = t\,D_a\psi$]:

- with $a = e_k$: $\nabla_{e_k} U = t\,D_{e_k}\psi$;
- with $a = e_k$, and ψ replaced by the spinor field $\phi = D_{e_k}\psi$: $\nabla_{e_k}\nabla_{e_k} U = \nabla_{e_k}\big(t\,\phi\big) = t\,D_{e_k}\phi = t\,D_{e_k}D_{e_k}\psi$;
- with $a = \nabla_{e_k} e_k$, a tangent vector: $\nabla_{\nabla_{e_k} e_k} U = t\,D_{\nabla_{e_k} e_k}\psi$.

Substituting the three into the definition:

$$\begin{aligned}\nabla^*\nabla(t\psi) &= -\sum_k \big((t\,D_{e_k}D_{e_k}\psi) - (t\,D_{\nabla_{e_k}e_k}\psi)\big) && \text{substitute}\\ &= t\Big(-\sum_k \big(D_{e_k}D_{e_k}\psi - D_{\nabla_{e_k}e_k}\psi\big)\Big) && \text{factor } t \text{ on the left.}\end{aligned}$$

So

$$\nabla^*\nabla(t\psi) = t\,D^*D\psi, \qquad D^*D\psi = -\sum_{k=1}^{2}\big(D_{e_k}D_{e_k}\psi - D_{\nabla_{e_k}e_k}\psi\big). \quad (18.11)$$

What D^*D does. In a plane with a constant frame, $\omega = 0$, so $D_a = a\cdot\nabla$; the basis is constant, so $\nabla_{e_k} e_k = 0$ and $D_0\psi = 0$. Then $D_{e_k}D_{e_k}\psi = \partial_k\partial_k\psi$, and

$$D^*D\psi = -\Big(\frac{\partial^2\psi}{\partial x_1^2} + \frac{\partial^2\psi}{\partial x_2^2}\Big) = -\,\Delta u_1 - (\Delta u_2)\,\mathsf{T}\,:$$

two uncoupled scalar Laplacians, one per coordinate. On a curved surface the frame turns. Expand one second derivative, writing $\omega_k = \omega(e_k)$:

$$\begin{aligned} D_{e_k} D_{e_k}\psi &= e_k \cdot \nabla\big(e_k \cdot \nabla\psi + \omega_k \mathsf{T}\psi\big) \\ &\quad + \omega_k \mathsf{T}\big(e_k \cdot \nabla\psi + \omega_k \mathsf{T}\psi\big) && D_{e_k} \text{ twice} \\ &= e_k \cdot \nabla(e_k \cdot \nabla\psi) + (e_k \cdot \nabla\omega_k)\,\mathsf{T}\psi \\ &\quad + \omega_k \mathsf{T}\,(e_k \cdot \nabla\psi) && \text{product rule} \\ &\quad + \omega_k \mathsf{T}\,(e_k \cdot \nabla\psi) + \omega_k^2\,\mathsf{T}^2\psi && \text{distribute} \\ &= e_k \cdot \nabla(e_k \cdot \nabla\psi) + (e_k \cdot \nabla\omega_k)\,\mathsf{T}\psi \\ &\quad + 2\,\omega_k \mathsf{T}\,(e_k \cdot \nabla\psi) - \omega_k^2\,\psi && \mathsf{T}^2 = -1. \end{aligned}$$

The terms with T couple the two coordinates: $\mathsf{T}\psi = -u_2 + u_1\mathsf{T}$ moves u_2 into the real part and u_1 into the T part.

Lengths, the energy and the heat equation. By $|t\phi| = |\phi|$ (shown after (18.5)), $|U| = |t\psi| = |\psi|$ and $|\nabla_a U| = |t\,D_a\psi| = |D_a\psi|$. The Dirichlet energy (5.1) $[\,E(U) = \frac{1}{2}\int_\Omega \sum_k |\nabla_{e_k} U|^2\,dA\,]$ becomes $\frac{1}{2}\int_\Omega \sum_k |D_{e_k}\psi|^2\,dA$. The vector heat equation (5.4) $[\,\partial U/\partial t = -\nabla^*\nabla U\,]$ with $U = t\psi$, and a frame t that does not depend on time (the time is also written t; the meaning is always clear from the context):

$$\begin{aligned} \frac{\partial(t\psi)}{\partial t} &= -\nabla^*\nabla(t\psi) && \text{substitute } U = t\psi \\ t\,\frac{\partial\psi}{\partial t} &= -\,t\,D^*D\psi && t \text{ constant in time, (18.11)} \\ \frac{\partial\psi}{\partial t} &= -\,D^*D\psi && \text{multiply on the left by } t,\ tt = 1. \end{aligned}$$

So the Dirichlet energy and the heat equation in spinor form are

$$E = \frac{1}{2}\int_\Omega \sum_k |D_{e_k}\psi|^2\,dA, \qquad \frac{\partial\psi}{\partial t} = -D^*D\,\psi, \tag{18.12}$$

where e_1, e_2 is an orthonormal basis of the tangent plane and Ω is the surface. Stage 4 shows that D^*D is the adjoint of D composed with D, as the notation says, and that $D^*D\psi$ is the direction in which E decreases fastest.

One backward Euler step. The backward Euler step of Part I, (5.5) again:

$$\frac{U - U^0}{\Delta t} = -\nabla^*\nabla U \qquad\Longleftrightarrow\qquad U + \Delta t\,\nabla^*\nabla U = U^0. \tag{5.5}$$

Substitute $U = t\psi$ and $U^0 = t\psi^0$ in the second form, and use (18.11), which says $\nabla^*\nabla(t\psi) = t\,D^*D\psi$:

$$\begin{aligned} (t\psi) + \Delta t\,\nabla^*\nabla(t\psi) &= (t\psi^0) && \text{substitute} \\ t\psi + \Delta t\,\big(t\,D^*D\psi\big) &= t\psi^0 && (18.11) \\ t\,\big(\psi + \Delta t\,D^*D\psi\big) &= t\psi^0 && \text{factor } t \\ t\,t\,\big(\psi + \Delta t\,D^*D\psi\big) &= t\,t\,\psi^0 && \text{multiply on the left by } t, \end{aligned}$$

and $tt = 1$ gives

$$\boxed{\psi + \Delta t\, D^* D\psi = \psi^0} \tag{18.13}$$

This is the strong form of our problem: a second-order linear equation for the complex-valued unknown ψ on the surface. The code uses it with the source condition of (5.6) [$U + \Delta t\, \nabla^* \nabla U = 0$ off s, $U(s) = U_s$], which by the same substitution is

$$\psi + \Delta t\, D^* D\psi = 0 \quad \text{away from } s, \qquad \psi(s) = \psi_s, \tag{18.14}$$

with the source $\psi_s = e^{2\pi \mathsf{T}/3} = \cos\frac{2\pi}{3} + \mathsf{T}\sin\frac{2\pi}{3} = -\frac{1}{2} + \frac{\sqrt{3}}{2}\,\mathsf{T}$ in the frame of the source vertex (`std::polar(1.0, 2.0 * M_PI / 3.0)` in `ga_fem.cpp`), and the time step $\Delta t = \bar{h}^2$, the square of the mean edge length.

What we have. On a smooth closed surface Ω, given $\Delta t > 0$ and ψ^0 (or the source value ψ_s), find the spinor field ψ that satisfies (18.13). Every ingredient is geometric algebra: the plane T, the projection P, the derivative $D = a \cdot \nabla + \omega \mathsf{T}$ of (18.9), and the frame vector t, a gauge choice that cannot affect the result. The equation has second derivatives; the next stages remove one of them.

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

19. Stage 2: The Function Spaces

What we want. Before transforming the equation we must say in which set of fields we look for its solution, and with which notion of distance. The weak form of Stage 4 will ask for less smoothness than the strong form (18.13), and the theorem that guarantees a unique solution needs the set to be a *Hilbert space*: a vector space with an inner product in which every sequence of fields that get closer and closer together converges to a field of the space. We assume: Ω is a smooth closed surface (bounded, without boundary; Stage 4 says what changes with a boundary), $\Delta t > 0$, and the fields are tangent fields $U = t\psi$, described by spinor coordinates as in Section 18.4. We do not prove the big theorems; we say what they state and why they apply.

The inner product of two spinors. Take two tangent vectors at a point, $U = t\psi$ and $V = t\chi$, with $\psi = u_1 + u_2\mathsf{T}$ and $\chi = v_1 + v_2\mathsf{T}$. The reverse of χ is $\widetilde{\chi} = v_1 - v_2\mathsf{T}$. Multiply out:

$$\begin{aligned} \psi\widetilde{\chi} &= (u_1 + u_2\mathsf{T})(v_1 - v_2\mathsf{T}) \\ &= u_1v_1 - u_1v_2\,\mathsf{T} + u_2v_1\,\mathsf{T} - u_2v_2\,\mathsf{T}^2 && \text{distribute} \\ &= u_1v_1 + u_2v_2 + (u_2v_1 - u_1v_2)\,\mathsf{T} && \mathsf{T}^2 = -1;\ \text{collect } \mathsf{T}. \end{aligned}$$

Its scalar part is $u_1v_1 + u_2v_2$. The inner product of the vectors themselves is the same number:

$$\begin{aligned} U \cdot V &= (u_1 t + u_2 b) \cdot (v_1 t + v_2 b) \\ &= u_1 v_1\, t\cdot t + u_1 v_2\, t \cdot b + u_2 v_1\, b \cdot t + u_2 v_2\, b \cdot b \quad \text{distribute} \\ &= u_1 v_1 + u_2 v_2 \qquad\qquad t\cdot t = b \cdot b = 1,\ t \cdot b = 0. \end{aligned}$$

So

$$\langle \psi \widetilde{\chi} \rangle = u_1 v_1 + u_2 v_2 = U \cdot V. \tag{19.1}$$

This is the real part of $\psi\bar{\chi}$, the familiar Hermitian inner product, with the reverse playing the role of complex conjugation. The bivector part is the oriented area spanned by V and U: $V \wedge U = (v_1 t + v_2 b) \wedge (u_1 t + u_2 b) = (v_1 u_2 - v_2 u_1)\, t \wedge b = (u_2 v_1 - u_1 v_2)\, \mathsf{T}$. Three properties are used below.

- *Symmetric*, $\langle \psi\widetilde{\chi}\rangle = \langle \chi\widetilde{\psi}\rangle$: by $\widetilde{XY} = \widetilde{Y}\widetilde{X}$ and $\widetilde{\widetilde{\chi}} = \chi$, $(\psi\widetilde{\chi})^{\sim} = \chi\widetilde{\psi}$, and the scalar part does not change under reversion, so $\langle \psi\widetilde{\chi}\rangle = \langle (\psi\widetilde{\chi})^{\sim}\rangle = \langle \chi\widetilde{\psi}\rangle$.
- *Positive*, $\langle \psi\widetilde{\psi}\rangle = u_1^2 + u_2^2 = |\psi|^2$, zero only for $\psi = 0$.
- *Frame independent.* Under $\psi \to e^{-\mathsf{T}\theta}\psi$ and $\chi \to e^{-\mathsf{T}\theta}\chi$, the reverse of the second is $(e^{-\mathsf{T}\theta}\chi)^{\sim} = \widetilde{\chi}\, e^{\mathsf{T}\theta}$, and

  $$\left(e^{-\mathsf{T}\theta}\psi\right)\left(\widetilde{\chi}\, e^{\mathsf{T}\theta}\right) = e^{-\mathsf{T}\theta} e^{\mathsf{T}\theta}\, \psi\widetilde{\chi} = \psi\widetilde{\chi},$$

  since complex numbers of one plane commute.

Square-integrable fields. The space L^2 is the set of spinor fields with finite $\int_\Omega |\psi|^2\, dA$, with the inner product and the norm

$$(\psi, \chi)_0 = \int_\Omega \langle \psi\widetilde{\chi}\rangle\, dA, \qquad \|\psi\|_0^2 = \int_\Omega |\psi|^2\, dA.$$

Its fields need not be continuous; they need only have a finite total squared length.

The Sobolev space H^1. The space H^1 is the set of spinor fields that are in L^2 and whose covariant derivatives $D_{e_k}\psi$ are in L^2 too, with

$$(\psi, \chi)_1 = \int_\Omega \langle \psi\widetilde{\chi}\rangle\, dA + \int_\Omega \sum_k \left\langle (D_{e_k}\psi)\, \widetilde{(D_{e_k}\chi)} \right\rangle dA, \qquad \|\psi\|_1^2 = (\psi,\psi)_1. \tag{19.2}$$

We write $|\psi|_1^2 = \int_\Omega \sum_k |D_{e_k}\psi|^2\, dA$ for the derivative part, so that $\|\psi\|_1^2 = \|\psi\|_0^2 + |\psi|_1^2$. Both integrands are frame independent, by (18.10) [$D'_a\psi' = e^{-\mathsf{T}\theta} D_a\psi$] and the frame independence of the scalar part, so H^1 is really a space of tangent fields. The derivative is meant in the *weak* sense: $D_{e_k}\psi$ need not exist at every point, only as a square-integrable field for which the integration by parts of Stage 4 holds. A field that is smooth on each triangle of a mesh and continuous across the edges is in H^1: its derivative jumps across the edges, but a jump on a curve of zero area does not affect an integral. A field that itself jumps across an edge is not in H^1, because the derivative of a jump is concentrated on the edge, like the derivative of a step function,

and is not square integrable. This is the property that the basis functions of Stage 5 must have.

Why a Hilbert space. The smooth fields with the norm $\|\cdot\|_1$ are not complete: a sequence of smooth fields can converge, in this norm, to a field with a kink, which is not smooth. The space H^1 is what we get by adding all such limits, and it is complete; with the inner product (19.2) it is a Hilbert space. Completeness is what the existence theorem below needs: it finds the solution as a limit. The smooth fields are *dense* in H^1: every field of H^1 is a limit of smooth ones. Stage 4 uses this to pass from smooth test fields to all of H^1.

The strong form needs more. The strong form (18.13) $[\,\psi + \Delta t\, D^*D\psi = \psi^0\,]$ contains $D^*D\psi$, second derivatives, so it makes sense for fields in H^2, the fields whose second covariant derivatives are square integrable as well. The fields we can compute with on a mesh, continuous and piecewise linear, are in H^1 but not in H^2: their first derivatives jump across edges, and the second derivative of the one-variable tent $1 - |x|$ is concentrated at the kink. The weak form will need only H^1.

The bilinear form and the linear form. The weak form of Stage 4 will read $a(\psi, \chi) = l(\chi)$, with

$$\begin{aligned} a(\psi,\chi) &= \int_\Omega \langle \psi\widetilde{\chi}\rangle \, dA + \Delta t \int_\Omega \sum_k \big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\big\rangle \, dA, \\ l(\chi) &= \int_\Omega \langle \psi^0\widetilde{\chi}\rangle \, dA. \end{aligned} \tag{19.3}$$

These are what H^1 is for. Write

$$A_0 = (\psi,\chi)_0, \qquad A_1 = \int_\Omega \sum_k \big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\big\rangle \, dA,$$

so that $a(\psi,\chi) = A_0 + \Delta t\, A_1$. The bilinear form is

- *symmetric*, $a(\psi,\chi) = a(\chi,\psi)$, by the symmetry of $\langle\psi\widetilde{\chi}\rangle$, applied to both integrands;
- *bounded*. The Cauchy–Schwarz inequality for each of the two inner products gives $|A_0| \le \|\psi\|_0\|\chi\|_0$ and $|A_1| \le |\psi|_1|\chi|_1$. Then

$$\begin{aligned} |a(\psi,\chi)| &\le |A_0| + \Delta t\,|A_1| \le \max(1,\Delta t)\,\big(|A_0| + |A_1|\big) \\ &\le \max(1,\Delta t)\,\big(\|\psi\|_0\|\chi\|_0 + |\psi|_1|\chi|_1\big) \\ &\le \max(1,\Delta t)\,\big(\|\psi\|_0^2 + |\psi|_1^2\big)^{1/2}\big(\|\chi\|_0^2 + |\chi|_1^2\big)^{1/2} \\ &= \max(1,\Delta t)\,\|\psi\|_1\,\|\chi\|_1, \end{aligned}$$

  where the third line is Cauchy–Schwarz in the plane, $x_1y_1 + x_2y_2 \le |x|\,|y|$, for the pairs $x = (\|\psi\|_0, |\psi|_1)$ and $y = (\|\chi\|_0, |\chi|_1)$;
- *coercive*: $a(\psi,\psi) = \|\psi\|_0^2 + \Delta t\,|\psi|_1^2 \ge \min(1,\Delta t)\,\big(\|\psi\|_0^2 + |\psi|_1^2\big) = \min(1,\Delta t)\,\|\psi\|_1^2$.

And l is bounded when $\psi^0 \in L^2$: $|l(\chi)| \le \|\psi^0\|_0\,\|\chi\|_0 \le \|\psi^0\|_0\,\|\chi\|_1$, the first step by Cauchy–Schwarz, the second because $\|\chi\|_0^2 \le \|\chi\|_0^2 + |\chi|_1^2$. Together: a is itself an inner product on H^1, whose norm $\|\psi\|_a = a(\psi,\psi)^{1/2}$, the *energy*

norm, satisfies $\min(1, \Delta t)\,\|\psi\|_1^2 \le \|\psi\|_a^2 \le \max(1, \Delta t)\,\|\psi\|_1^2$, so H^1 is a Hilbert space with it as well.

What the theorems give. The *Riesz representation theorem* says that in a Hilbert space every bounded linear functional is the inner product with exactly one element. Applied to l and the inner product a: there is exactly one $\psi \in H^1$ with $a(\psi, \chi) = l(\chi)$ for every $\chi \in H^1$, the *weak solution*. (The *Lax–Milgram theorem* gives the same conclusion for forms that are bounded and coercive but not symmetric.) For the discrete problem, if the finite-dimensional space V_h of Stage 5 lies inside H^1, a *conforming* space, the same theorem applied in V_h gives exactly one discrete solution ψ_h, and *Céa's lemma* says that it is the best approximation of ψ from V_h in the energy norm; Stage 6 derives it. Conformity, for tangent fields on a mesh, means: the discrete field has no jump across the edges. Since the two faces of an edge lie in different planes, "no jump" must be read after transporting the values of one face into the plane of the other; Stage 5 shows how close our basis comes to it.

A caveat about the source. In two dimensions a field of H^1 need not have a value at a point. The function $f = \log\log(1/r)$, with r the distance to a point, is unbounded as $r \to 0$, but it is in H^1 near that point. Its radial derivative is

$$f'(r) = \frac{1}{\log(1/r)} \cdot \frac{d}{dr} \log \frac{1}{r} = \frac{1}{\log(1/r)} \cdot \Big(-\frac{1}{r} \Big) = -\frac{1}{r \log(1/r)},$$

and, on a disk of radius $R < 1$, with $dA = 2\pi r\, dr$ and the substitution $v = \log(1/r)$, $dv = -dr/r$,

$$\begin{aligned} \int |\nabla f|^2\, dA &= 2\pi \int_0^R \frac{r\, dr}{r^2 \log^2(1/r)} = 2\pi \int_0^R \frac{dr}{r\, \log^2(1/r)} \\ &= 2\pi \int_{\log(1/R)}^{\infty} \frac{dv}{v^2} = \frac{2\pi}{\log(1/R)}, \end{aligned}$$

which is finite. So the condition $\psi(s) = \psi_s$ of (18.14) at a single point is not a condition that the continuous theory accepts, and neither is a source concentrated at a point in ψ^0. Both make sense for the discrete problem, where they are what the code does, and the continuous theory applies to a source spread over a small disk, which is what the discrete source approximates on a given mesh.

What we have. The unknown lives in H^1, the tangent fields with square-integrable covariant derivative; the forms a and l of (19.3) satisfy the hypotheses of the Riesz theorem, so the weak problem has exactly one solution; and the discrete space must be continuous across edges to be conforming. The next stage gives a and l a meaning as an energy.

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

20. Stage 3: The Variational Form

What we want. We want an energy whose minimizer is the solution. It explains why the problem has exactly one solution and what that solution is (a compromise between two wishes), and it gives the weak form directly, by differentiation.

The energy. Consider

$$J(\psi) = \frac{1}{2}\int_\Omega |\psi|^2\,dA + \frac{\Delta t}{2}\int_\Omega \sum_k |D_{e_k}\psi|^2\,dA - \int_\Omega \langle \psi^0 \widetilde{\psi}\rangle\,dA = \frac{1}{2}\,a(\psi,\psi) - l(\psi), \tag{20.1}$$

the spinor form of (7.1) [$J(U) = \frac{1}{2}\int |U|^2 + \frac{\Delta t}{2}\int |\nabla U|^2 - \int U^0 \cdot U$] of Part I; the two agree term by term by (19.1) and $|D_{e_k}\psi| = |\nabla_{e_k} U|$. The last equality in (20.1) is (19.3) with $\chi = \psi$. The first and last terms of J together measure the distance to ψ^0. Expand $|\psi - \psi^0|^2$:

$$\begin{aligned} |\psi-\psi^0|^2 &= \big\langle (\psi-\psi^0)(\widetilde{\psi}-\widetilde{\psi^0})\big\rangle \\ &= \langle \psi\widetilde{\psi}\rangle - \langle \psi\widetilde{\psi^0}\rangle - \langle \psi^0\widetilde{\psi}\rangle + \langle \psi^0\widetilde{\psi^0}\rangle \qquad \text{distribute} \\ &= |\psi|^2 - 2\,\langle \psi^0\widetilde{\psi}\rangle + |\psi^0|^2 \qquad \text{symmetry.} \end{aligned}$$

Halving and integrating, $\frac{1}{2}\int|\psi|^2 - \int\langle\psi^0\widetilde{\psi}\rangle = \frac{1}{2}\int|\psi-\psi^0|^2 - \frac{1}{2}\int|\psi^0|^2$. So J asks ψ to stay close to ψ^0 (the constant $\frac{1}{2}\int|\psi^0|^2$ does not depend on ψ); its middle term is Δt times the Dirichlet energy (18.12) [$E = \frac{1}{2}\int\sum_k |D_{e_k}\psi|^2$], so J also asks ψ to be as parallel as possible; Δt sets the balance.

Its derivative in a direction. Let χ be any field of H^1 and ϵ a real number. We expand $J(\psi+\epsilon\chi)$ term by term.

The squared length. The reverse is linear, $(\psi+\epsilon\chi)^\sim = \widetilde{\psi} + \epsilon\widetilde{\chi}$, so

$$\begin{aligned} |\psi+\epsilon\chi|^2 &= \big\langle (\psi+\epsilon\chi)\,(\widetilde{\psi}+\epsilon\widetilde{\chi})\big\rangle \\ &= \langle\psi\widetilde{\psi}\rangle + \epsilon\,\langle\psi\widetilde{\chi}\rangle + \epsilon\,\langle\chi\widetilde{\psi}\rangle + \epsilon^2\,\langle\chi\widetilde{\chi}\rangle \qquad \text{distribute} \\ &= |\psi|^2 + 2\epsilon\,\langle\psi\widetilde{\chi}\rangle + \epsilon^2|\chi|^2 \qquad \langle\chi\widetilde{\psi}\rangle = \langle\psi\widetilde{\chi}\rangle. \end{aligned}$$

The derivative term. D_{e_k} is linear, because both of its parts are: $e_k\cdot\nabla(\psi+\epsilon\chi) = e_k\cdot\nabla\psi + \epsilon\,e_k\cdot\nabla\chi$, and $\omega_k\mathsf{T}(\psi+\epsilon\chi) = \omega_k\mathsf{T}\psi + \epsilon\,\omega_k\mathsf{T}\chi$. So $D_{e_k}(\psi+\epsilon\chi) = D_{e_k}\psi + \epsilon\,D_{e_k}\chi$, and the same expansion, with $D_{e_k}\psi$ and $D_{e_k}\chi$ in the places of ψ and χ, gives

$$|D_{e_k}(\psi+\epsilon\chi)|^2 = |D_{e_k}\psi|^2 + 2\epsilon\,\big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\big\rangle + \epsilon^2|D_{e_k}\chi|^2.$$

The source term. $\langle\psi^0(\psi+\epsilon\chi)^\sim\rangle = \langle\psi^0\widetilde{\psi}\rangle + \epsilon\,\langle\psi^0\widetilde{\chi}\rangle$.

Collecting. Put the three expansions into (20.1) and group the powers of ϵ:

$$\begin{aligned} J(\psi+\epsilon\chi) &= \frac{1}{2}\int \big(|\psi|^2 + 2\epsilon\langle\psi\widetilde{\chi}\rangle + \epsilon^2|\chi|^2\big) \\ &\quad + \frac{\Delta t}{2}\int \sum_k \big(|D_{e_k}\psi|^2 + 2\epsilon\langle(D_{e_k}\psi)\widetilde{(D_{e_k}\chi)}\rangle + \epsilon^2|D_{e_k}\chi|^2\big) \\ &\quad - \int \big(\langle\psi^0\widetilde{\psi}\rangle + \epsilon\langle\psi^0\widetilde{\chi}\rangle\big) \\ &= J(\psi) + \epsilon\Big(\int\langle\psi\widetilde{\chi}\rangle + \Delta t\int\sum_k\langle(D_{e_k}\psi)\widetilde{(D_{e_k}\chi)}\rangle - \int\langle\psi^0\widetilde{\chi}\rangle\Big) \\ &\quad + \frac{\epsilon^2}{2}\Big(\int|\chi|^2 + \Delta t\int\sum_k|D_{e_k}\chi|^2\Big). \end{aligned}$$

With the forms (19.3), the parentheses are $a(\psi,\chi) - l(\chi)$ and $a(\chi,\chi)$:

$$J(\psi+\epsilon\chi) = J(\psi) + \epsilon\big(a(\psi,\chi) - l(\chi)\big) + \frac{\epsilon^2}{2}\,a(\chi,\chi).$$

Stationarity. The derivative of J in the direction χ is the derivative of this polynomial in ϵ at $\epsilon = 0$:

$$\frac{d}{d\epsilon}J(\psi+\epsilon\chi)\Big|_{\epsilon=0} = \big(a(\psi,\chi) - l(\chi) + \epsilon\,a(\chi,\chi)\big)\Big|_{\epsilon=0} = a(\psi,\chi) - l(\chi).$$

So ψ is a stationary point of J, a point where the derivative vanishes in every direction, exactly when

$$a(\psi,\chi) = l(\chi) \qquad \text{for every } \chi \in H^1,$$

which is the weak form announced in Stage 2. If ψ is stationary, the expansion becomes $J(\psi+\epsilon\chi) - J(\psi) = \frac{\epsilon^2}{2}\,a(\chi,\chi)$, which is positive for $\epsilon \neq 0$ and $\chi \neq 0$, by coercivity. Every other field $\psi + \epsilon\chi$ has a larger energy: the stationary point is the unique minimizer of J.

Minimizing J and the PDE.. Stage 4 derives the same equation $a(\psi,\chi) = l(\chi)$ from the strong form, by integration by parts, and shows that for a field smooth enough to have second derivatives the two are equivalent. So, for smooth fields: ψ solves (18.13) $\iff$ ψ is stationary for J $\iff$ ψ minimizes J. In the code, $\psi^0 = 0$ and the source value is fixed: the discrete solution is the field of the discrete space that minimizes $\frac{1}{2}a(\psi,\psi)$ among those with the value ψ_s at the source vertex.

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

21. Stage 4: The Weak Form (Galerkin)

What we want. We want to derive the weak form from the strong form (18.13), by the mechanical steps that work for any PDE: multiply by

a test field, integrate, and move one derivative from the unknown to the test field. The last step needs an integration by parts on a curved surface. We derive it from the one theorem of geometric calculus that contains all integration by parts, and from it Green's identity for the covariant derivative of spinors.

21.1. The divergence theorem of geometric calculus

The fundamental theorem. Let Ω be a region of the surface with boundary curve $\partial\Omega$, with outward unit conormal ν (the unit tangent vector of the surface, perpendicular to the boundary and pointing out of the region), and let F be a smooth field with values in $\mathbb{G}_3$. The *fundamental theorem of geometric calculus* [2, 5], written for a surface, is

$$\int_\Omega \mathsf{T}\,\nabla F\,dA = \oint_{\partial\Omega} \mathsf{T}\nu\,F\,ds. \tag{21.1}$$

On the left, $\mathsf{T}\,dA$ is the oriented area element, and the vector derivative (18.6) [$\nabla F = \sum_k e_k(e_k \cdot \nabla F)$] acts on F only, not on T. On the right, $\mathsf{T}\nu\,ds$ is the oriented line element of the boundary: since ν is tangent, $\mathsf{T}\nu = -\nu\mathsf{T} = -\,n \times \nu$ by (18.3), a unit vector along the boundary. The theorem holds on curved surfaces as written, with T varying from point to point.

A check in the plane. In a flat region T is constant. Multiply (21.1) on the left by T^{-1}, which moves inside both integrals because it is constant:

$$\int_\Omega \nabla F\,dA = \oint_{\partial\Omega} \nu F\,ds.$$

We check this one with the divergence theorem (5.2) [$\int_\Omega \operatorname{div} W\,dA = \oint_{\partial\Omega} W \cdot \nu\,ds$] of Part I. With the axes e_1, e_2, $\nabla F = e_1\,\partial_1 F + e_2\,\partial_2 F$ and $\nu = \nu_1 e_1 + \nu_2 e_2$. Let f be any component of F. For $W = (f, 0)$, $\operatorname{div} W = \partial_1 f$ and $W \cdot \nu = f\nu_1$; for $W = (0, f)$, $\operatorname{div} W = \partial_2 f$ and $W \cdot \nu = f\nu_2$. So

$$\int_\Omega \partial_1 f\,dA = \oint_{\partial\Omega} f\,\nu_1\,ds, \qquad \int_\Omega \partial_2 f\,dA = \oint_{\partial\Omega} f\,\nu_2\,ds,$$

for every component, hence for F itself. Multiply the first by e_1 and the second by e_2 on the left (constants, so they move inside the integrals) and add:

$$\begin{aligned}\int_\Omega \nabla F\,dA &= e_1 \int_\Omega \partial_1 F\,dA + e_2 \int_\Omega \partial_2 F\,dA \\ &= e_1 \oint_{\partial\Omega} \nu_1 F\,ds + e_2 \oint_{\partial\Omega} \nu_2 F\,ds \\ &= \oint_{\partial\Omega} (\nu_1 e_1 + \nu_2 e_2)\,F\,ds = \oint_{\partial\Omega} \nu F\,ds.\end{aligned}$$

On a curved surface T cannot be moved out of the integrals, and the theorem must keep it inside. For $F = 1$ the left side of (21.1) is zero, since $\nabla 1 = 0$, and so is the right side: $\mathsf{T}\nu$ is plus or minus the unit tangent γ' of the closed curve $\partial\Omega$, and $\oint \gamma'(s)\,ds = \gamma(\text{end}) - \gamma(\text{start}) = 0$. The flat version $\oint \nu\,ds$ would not

be zero on a curved surface. On the upper unit hemisphere the boundary is the equator, $n = (\cos\varphi, \sin\varphi, 0)$, and the conormal is $\nu = -e_z$ everywhere, so $\oint \nu\, ds = -2\pi e_z$. With T inside, $\mathsf{T}\nu = -\,n \times \nu = n \times e_z = (\sin\varphi, -\cos\varphi, 0)$, and $\int_0^{2\pi} (\sin\varphi, -\cos\varphi, 0)\, d\varphi = 0$.

The divergence theorem on a surface. We apply (21.1) to $F = W\mathsf{T}$, for a tangent field W, and keep the scalar parts. By the product rule, $e_k \cdot \nabla(W\mathsf{T}) = (e_k \cdot \nabla W)\,\mathsf{T} + W\,(e_k \cdot \nabla \mathsf{T})$. Put this into (18.6) and multiply by T on the left:

$$\begin{aligned}
\mathsf{T}\,\nabla(W\mathsf{T}) &= \sum_k \mathsf{T}\, e_k \left((e_k \cdot \nabla W)\,\mathsf{T} + W\,(e_k \cdot \nabla \mathsf{T}) \right) && (18.6)\\
&= \mathsf{T}\Big(\sum_k e_k\,(e_k \cdot \nabla W) \Big)\mathsf{T} + \sum_k \mathsf{T}\, e_k\, W\,(e_k \cdot \nabla \mathsf{T}) && \text{distribute}\\
&= \mathsf{T}\,(\nabla W)\,\mathsf{T} + \sum_k \mathsf{T}\, e_k\, W\,(e_k \cdot \nabla \mathsf{T}) && (18.6).
\end{aligned}$$

We take the scalar part of each term, and of the right side.

- *The first term.* For any element X, by the cyclic rule with the two factors T and $X\mathsf{T}$,

$$\langle \mathsf{T}\,(X\mathsf{T}) \rangle = \langle (X\mathsf{T})\,\mathsf{T} \rangle = \langle X\,\mathsf{T}^2 \rangle = -\langle X \rangle.$$

  With $X = \nabla W = \sum_k e_k w_k$, $w_k = e_k \cdot \nabla W$, the scalar part of each product $e_k w_k = e_k \cdot w_k + e_k \wedge w_k$ is $e_k \cdot w_k$. By the Gauss formula (18.7), $w_k = \nabla_{e_k} W + \mathrm{II}(e_k, W)\, n$, and $e_k \cdot n = 0$, so

$$\langle \mathsf{T}\,(\nabla W)\,\mathsf{T} \rangle = -\sum_k e_k \cdot \big(\nabla_{e_k} W + \mathrm{II}(e_k, W)\, n \big) = -\sum_k e_k \cdot \nabla_{e_k} W = -\operatorname{div} W,$$

  the divergence of Part I.

- *The second term.* Here the derivative falls on $\mathsf{T} = In$. Since I is constant, $e_k \cdot \nabla \mathsf{T} = I\, m$ with $m = e_k \cdot \nabla n$, and m is tangent: differentiating $n \cdot n = 1$ gives $2\, n \cdot m = 0$. Then

$$\begin{aligned}
\mathsf{T}\, e_k\, W\,(e_k \cdot \nabla \mathsf{T}) &= (In)\, e_k\, W\,(Im) && \text{substitute}\\
&= I\, I\, n\, e_k\, W\, m = -\,n\,(e_k W m) && I \text{ commutes; } I^2 = -1.
\end{aligned}$$

  The product of three tangent vectors is a tangent vector: $e_k W m = e_k(W \cdot m) + e_k(W \wedge m)$, and $e_k(W \wedge m) = e_k \cdot (W \wedge m) + e_k \wedge W \wedge m$, where $e_k \cdot (W \wedge m) = (e_k \cdot W)\, m - (e_k \cdot m)\, W$ is tangent, and the trivector $e_k \wedge W \wedge m$ vanishes because three vectors of one plane span no volume. So $v = e_k W m$ is tangent, and $nv = n \cdot v + n \wedge v = 0 + n \wedge v$ is a pure bivector, with no scalar part. This term contributes nothing: the bending of the surface drops out of the scalar part.

- *The right side.* With the cyclic rule, now with the factors T and $\nu W \mathsf{T}$:

$$\langle \mathsf{T}\,(\nu W \mathsf{T}) \rangle = \langle \nu W\, \mathsf{T}^2 \rangle = -\langle \nu W \rangle = -\,W \cdot \nu.$$

The scalar part of (21.1) with $F = W\mathsf{T}$ is therefore

$$\int_\Omega \big(-\operatorname{div} W + 0 \big)\, dA = \oint_{\partial\Omega} \big(-\,W \cdot \nu \big)\, ds,$$

and multiplying by -1,

$$\int_\Omega \operatorname{div} W \, dA = \oint_{\partial\Omega} W \cdot \nu \, ds, \qquad \operatorname{div} W = \sum_k e_k \cdot \nabla_{e_k} W, \tag{21.2}$$

the divergence theorem on a curved surface, derived from the fundamental theorem.

21.2. Green's identity for the covariant derivative of spinors

The product rule for spinors. Let $\psi = u_1 + u_2\mathsf{T}$ and $\chi = v_1 + v_2\mathsf{T}$ be two spinor fields in the same frame. The scalar part $\langle\psi\widetilde{\chi}\rangle = u_1v_1 + u_2v_2$, by (19.1), is a function, and the ordinary product rule gives

$$\begin{aligned} a \cdot \nabla\langle\psi\widetilde{\chi}\rangle &= a \cdot \nabla(u_1v_1 + u_2v_2) \\ &= \big((a \cdot \nabla u_1)\, v_1 + (a \cdot \nabla u_2)\, v_2\big) + \big(u_1\,(a \cdot \nabla v_1) + u_2\,(a \cdot \nabla v_2)\big) \\ &= \big\langle (a \cdot \nabla\psi)\,\widetilde{\chi}\big\rangle + \big\langle \psi\,\widetilde{(a \cdot \nabla\chi)}\big\rangle, \end{aligned}$$

where the last line reads each parenthesis as (19.1) again, with $a \cdot \nabla\psi = (a \cdot \nabla u_1) + (a \cdot \nabla u_2)\,\mathsf{T}$ in the place of ψ, and with $a \cdot \nabla\chi$ in the place of χ. We want covariant derivatives instead. The connection terms that D_a adds to each side add up to zero. The reverse of $\omega(a)\mathsf{T}\chi$ is $\omega(a)\,\widetilde{\chi}\,\widetilde{\mathsf{T}} = -\,\omega(a)\,\widetilde{\chi}\,\mathsf{T}$, so

$$\begin{aligned} &\big\langle \omega(a)\mathsf{T}\psi\,\widetilde{\chi}\big\rangle + \big\langle \psi\,\widetilde{(\omega(a)\mathsf{T}\chi)}\big\rangle \\ &\quad = \omega(a)\,\langle \mathsf{T}\psi\widetilde{\chi}\rangle - \omega(a)\,\langle \psi\widetilde{\chi}\,\mathsf{T}\rangle && \widetilde{XY} = \widetilde{Y}\widetilde{X},\ \widetilde{\mathsf{T}} = -\mathsf{T} \\ &\quad = \omega(a)\,\langle \mathsf{T}\psi\widetilde{\chi}\rangle - \omega(a)\,\langle \mathsf{T}\,\psi\widetilde{\chi}\rangle && \mathsf{T} \text{ commutes with } \psi\widetilde{\chi} \\ &\quad = 0. \end{aligned}$$

(The complex number $\psi\widetilde{\chi}$ commutes with T, as all complex numbers of one plane do.) Adding this zero to the right side, and grouping each derivative with its connection term, with $D_a = a \cdot \nabla + \omega(a)\mathsf{T}$ of (18.9):

$$\begin{aligned} a \cdot \nabla\langle\psi\widetilde{\chi}\rangle &= \big\langle (a \cdot \nabla\psi)\,\widetilde{\chi}\big\rangle + \big\langle \omega(a)\mathsf{T}\psi\,\widetilde{\chi}\big\rangle + \big\langle \psi\,\widetilde{(a \cdot \nabla\chi)}\big\rangle + \big\langle \psi\,\widetilde{(\omega(a)\mathsf{T}\chi)}\big\rangle \\ &= \big\langle (a \cdot \nabla\psi + \omega(a)\mathsf{T}\psi)\,\widetilde{\chi}\big\rangle + \big\langle \psi\,(a \cdot \nabla\chi + \omega(a)\mathsf{T}\chi)^{\sim}\big\rangle, \end{aligned}$$

that is,

$$a \cdot \nabla\langle\psi\widetilde{\chi}\rangle = \big\langle (D_a\psi)\,\widetilde{\chi}\big\rangle + \big\langle \psi\,\widetilde{(D_a\chi)}\big\rangle. \tag{21.3}$$

This is metric compatibility in spinor form: the connection is an infinitesimal rotation, and rotations do not change inner products.

The field to which we apply the divergence theorem. In one variable, $\int f'g' = [f'g] - \int f''g$ comes from integrating the product rule $(f'g)' = f''g + f'g'$. Here the analogue of $f'g$ is the tangent field

$$W = \sum_k \big\langle (D_{e_k}\psi)\,\widetilde{\chi}\big\rangle\, e_k.$$

For every tangent $X = \sum_k (e_k \cdot X)\, e_k$,

$$W \cdot X = \sum_k \big\langle (D_{e_k}\psi)\,\widetilde{\chi}\big\rangle\,(e_k \cdot X) = \Big\langle \Big(D_{\sum_k (e_k\cdot X)\, e_k}\psi\Big)\,\widetilde{\chi}\Big\rangle = \big\langle (D_X\psi)\,\widetilde{\chi}\big\rangle,$$

by linearity of D_X in X. Its divergence, with an orthonormal frame field e_1, e_2, is $\operatorname{div} W = \sum_j e_j \cdot \nabla_{e_j} W$, and we compute it in three steps.

Step 1. Metric compatibility for $W \cdot e_j$, with $U = W$ and $V = e_j$, gives

$$e_j \cdot \nabla(W \cdot e_j) = (\nabla_{e_j} W) \cdot e_j + W \cdot \nabla_{e_j} e_j.$$

Solve for the first term on the right, and replace $W \cdot e_j$ and $W \cdot \nabla_{e_j} e_j$ by $\langle (D_X\psi)\widetilde{\chi}\rangle$ with $X = e_j$ and $X = \nabla_{e_j} e_j$:

$$(\nabla_{e_j} W) \cdot e_j = e_j \cdot \nabla\big\langle (D_{e_j}\psi)\,\widetilde{\chi}\big\rangle - \big\langle (D_{\nabla_{e_j} e_j}\psi)\,\widetilde{\chi}\big\rangle.$$

Step 2. The spinor product rule (21.3) with $a = e_j$ and with $D_{e_j}\psi$ in the place of ψ:

$$e_j \cdot \nabla\big\langle (D_{e_j}\psi)\,\widetilde{\chi}\big\rangle = \big\langle \big(D_{e_j}(D_{e_j}\psi)\big)\,\widetilde{\chi}\big\rangle + \big\langle (D_{e_j}\psi)\,\widetilde{(D_{e_j}\chi)}\big\rangle.$$

Substituting it into Step 1:

$$(\nabla_{e_j} W) \cdot e_j = \Big(\big\langle (D_{e_j} D_{e_j}\psi)\,\widetilde{\chi}\big\rangle + \big\langle (D_{e_j}\psi)\,\widetilde{(D_{e_j}\chi)}\big\rangle\Big) - \big\langle (D_{\nabla_{e_j} e_j}\psi)\,\widetilde{\chi}\big\rangle.$$

Step 3. Sum over j, combine the two terms with $\widetilde{\chi}$ (the scalar part is linear), and recognize (18.11) [$D^*D\psi = -\sum_k (D_{e_k} D_{e_k}\psi - D_{\nabla_{e_k} e_k}\psi)$]:

$$\begin{aligned}
\operatorname{div} W &= \sum_j \big\langle (D_{e_j} D_{e_j}\psi - D_{\nabla_{e_j} e_j}\psi)\,\widetilde{\chi}\big\rangle \\
&\quad + \sum_j \big\langle (D_{e_j}\psi)\,\widetilde{(D_{e_j}\chi)}\big\rangle && \text{combine} \\
&= \Big\langle \Big(\sum_j (D_{e_j} D_{e_j}\psi - D_{\nabla_{e_j} e_j}\psi)\Big)\,\widetilde{\chi}\Big\rangle \\
&\quad + \sum_j \big\langle (D_{e_j}\psi)\,\widetilde{(D_{e_j}\chi)}\big\rangle && \text{sum inside} \\
&= -\big\langle (D^*D\psi)\,\widetilde{\chi}\big\rangle + \sum_j \big\langle (D_{e_j}\psi)\,\widetilde{(D_{e_j}\chi)}\big\rangle && (18.11).
\end{aligned}$$

Integrating. Integrate both sides over Ω. The divergence theorem (21.2) again,

$$\int_\Omega \operatorname{div} W\, dA = \oint_{\partial\Omega} W \cdot \nu\, ds, \tag{21.2}$$

turns the left side into a boundary integral, and on the boundary $W \cdot \nu = \langle (D_\nu\psi)\,\widetilde{\chi}\rangle$ (the formula for $W \cdot X$ with $X = \nu$):

$$\oint_{\partial\Omega} \big\langle (D_\nu\psi)\,\widetilde{\chi}\big\rangle\, ds = -\int_\Omega \big\langle (D^*D\psi)\,\widetilde{\chi}\big\rangle\, dA + \int_\Omega \sum_k \big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\big\rangle\, dA.$$

Moving the integral with D^*D to the left side:

$$\int_\Omega \sum_k \big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\big\rangle\,dA = \int_\Omega \big\langle (D^*D\psi)\,\widetilde{\chi}\big\rangle\,dA + \oint_{\partial\Omega} \big\langle (D_\nu\psi)\,\widetilde{\chi}\big\rangle\,ds. \tag{21.4}$$

This is *Green's identity* for spinors. It needs ψ with second derivatives ($\psi \in H^2$) and $\chi \in H^1$. With $U = t\psi$ and $V = t\chi$, by (19.1) and (18.11), it is Green's identity (5.3) [$\int \nabla U : \nabla V = \int (\nabla^*\nabla U)\cdot V + \oint (\nabla_\nu U)\cdot V$] of Part I, now obtained from the fundamental theorem of geometric calculus.

D^*D is positive, and the heat flow decreases E. On a closed surface there is no boundary term. With $\chi = \psi$, (21.4) says

$$\int_\Omega \big\langle (D^*D\psi)\,\widetilde{\psi}\big\rangle\,dA = \int_\Omega \sum_k \big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\psi)}\big\rangle\,dA = \int_\Omega \sum_k |D_{e_k}\psi|^2\,dA \geq 0 :$$

D^*D is positive, and it is the adjoint of D composed with D, like A^TA for a matrix A. The derivative of the energy E of (18.12) in a direction χ is, by the expansion of the derivative term in Stage 3 (halved, without Δt), $\int \sum_k \langle (D_{e_k}\psi)\widetilde{(D_{e_k}\chi)}\rangle\,dA$, and by (21.4) it equals $\int \langle (D^*D\psi)\widetilde{\chi}\rangle\,dA$. Along the heat flow, $\chi = \partial_t\psi = -D^*D\psi$, so $dE/dt = -\int |D^*D\psi|^2\,dA \leq 0$: the heat equation moves ψ in the direction in which E decreases fastest.

21.3. The weak form

Step 1: test the equation. A spinor field $r \in L^2$ is zero exactly when $\int_\Omega \langle r\widetilde{\chi}\rangle\,dA = 0$ for every χ: if $r = 0$ the integral is zero; conversely, the choice $\chi = r$ gives $\int |r|^2\,dA = 0$, so $r = 0$ (at every point where r is continuous, and almost everywhere in general). Apply this to the residual of the strong form, (18.13) again:

$$\psi + \Delta t\, D^*D\psi = \psi^0. \tag{18.13}$$

Multiply it on the right by the reverse $\widetilde{\chi}$ of a test spinor χ, distribute, take the scalar part (it is linear), and integrate (also linear):

$$\begin{aligned}
\big(\psi + \Delta t\, D^*D\psi\big)\,\widetilde{\chi} &= \psi^0\widetilde{\chi} && \text{multiply by } \widetilde{\chi}\\
\psi\widetilde{\chi} + \Delta t\,(D^*D\psi)\,\widetilde{\chi} &= \psi^0\widetilde{\chi} && \text{distribute}\\
\langle\psi\widetilde{\chi}\rangle + \Delta t\,\big\langle (D^*D\psi)\,\widetilde{\chi}\big\rangle &= \langle\psi^0\widetilde{\chi}\rangle && \text{scalar part}
\end{aligned}$$

$$\int_\Omega \langle\psi\widetilde{\chi}\rangle\,dA + \Delta t \int_\Omega \big\langle (D^*D\psi)\,\widetilde{\chi}\big\rangle\,dA = \int_\Omega \langle\psi^0\widetilde{\chi}\rangle\,dA \qquad \text{for every } \chi.$$

Nothing has been lost: by the first sentence, this for every χ is the same as (18.13).

Step 2: integrate by parts. Green's identity (21.4) again, solved for the term with D^*D by subtracting the boundary integral from both sides:

$$\int_\Omega \big\langle (D^*D\psi)\,\widetilde{\chi}\big\rangle\,dA = \int_\Omega \sum_k \big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\big\rangle\,dA - \oint_{\partial\Omega} \big\langle (D_\nu\psi)\,\widetilde{\chi}\big\rangle\,ds.$$

Substitute it, inside the parentheses, into the second term of the tested equation:

$$\int_\Omega \langle \psi \widetilde{\chi} \rangle \, dA + \Delta t \Big(\int_\Omega \sum_k \big\langle (D_{e_k}\psi)\, \widetilde{(D_{e_k}\chi)} \big\rangle \, dA - \oint_{\partial\Omega} \langle (D_\nu \psi)\, \widetilde{\chi} \rangle \, ds \Big) = \int_\Omega \langle \psi^0 \widetilde{\chi} \rangle \, dA,$$

and, distributing Δt,

$$\int_\Omega \langle \psi \widetilde{\chi} \rangle \, dA + \Delta t \int_\Omega \sum_k \big\langle (D_{e_k}\psi)\, \widetilde{(D_{e_k}\chi)} \big\rangle \, dA - \Delta t \oint_{\partial\Omega} \langle (D_\nu \psi)\, \widetilde{\chi} \rangle \, ds = \int_\Omega \langle \psi^0 \widetilde{\chi} \rangle \, dA.$$

One derivative has moved from ψ to χ.

Step 3: the boundary term. The surfaces of this text are closed, so $\partial\Omega$ is empty and the boundary integral is absent. (On a surface with boundary it vanishes in two common situations: where ψ is prescribed, because there the test fields are chosen to vanish, and where the natural condition $D_\nu \psi = 0$ holds. In the second case the boundary condition is built into the weak form without being imposed, which is why it is called natural.) The remaining equation contains only first derivatives, so it makes sense for every ψ and χ in H^1; both sides are bounded in χ (Stage 2), so, holding for the smooth χ, it holds for all $\chi \in H^1$ by density. The *weak form* of the backward Euler step is: find $\psi \in H^1$ such that

$$\int_\Omega \langle \psi\, \widetilde{\chi} \rangle \, dA + \Delta t \int_\Omega \sum_k \big\langle (D_{e_k}\psi)\, \widetilde{(D_{e_k}\chi)} \big\rangle \, dA = \int_\Omega \langle \psi^0\, \widetilde{\chi} \rangle \, dA \tag{21.5}$$

for every test spinor $\chi \in H^1$; in the notation of (19.3), $a(\psi,\chi) = l(\chi)$. It contains first derivatives of ψ and χ symmetrically, and it is the stationarity condition of the energy J of Stage 3. By (19.1), $\langle \psi\widetilde{\chi}\rangle = U \cdot V$ and $\sum_k \langle (D_{e_k}\psi)\widetilde{(D_{e_k}\chi)}\rangle = \sum_k \nabla_{e_k} U \cdot \nabla_{e_k} V = \nabla U : \nabla V$, so it is the weak form (8.2) $[\int U \cdot V + \Delta t \int \nabla U : \nabla V = \int U^0 \cdot V\,]$ of Part I, written for the coordinates.

Back to the strong form. Suppose the weak solution has second derivatives. Then Green's identity applies, and we run the steps backwards. Substitute (21.4), without boundary term, into the second integral of (21.5):

$$\int_\Omega \langle \psi \widetilde{\chi} \rangle \, dA + \Delta t \Big(\int_\Omega \big\langle (D^* D\psi)\, \widetilde{\chi} \big\rangle \, dA \Big) - \int_\Omega \langle \psi^0 \widetilde{\chi} \rangle \, dA = 0 \quad \text{substitute}$$

$$\int_\Omega \big\langle \big(\psi + \Delta t\, D^* D\psi - \psi^0\big)\, \widetilde{\chi} \big\rangle \, dA = 0 \quad \text{linearity.}$$

This holds for every χ. Call $r = \psi + \Delta t\, D^* D\psi - \psi^0$ the residual and choose $\chi = r$: $\int_\Omega |r|^2 \, dA = 0$, so $r = 0$, which is (18.13) at every point. So for smooth fields the weak and the strong forms are the same problem, and the weak form only enlarges the set of fields for which the question can be asked.

The source. In the code the value at the source vertex is prescribed, as in (18.14) [$\psi + \Delta t\, D^*D\psi = 0$ off s, $\psi(s) = \psi_s$]. Where the unknown is prescribed we need no equation to determine it, so we do not test there: the test fields are those that vanish at s. In the discrete method this means that the equation of the source vertex is discarded and replaced by the prescribed value, which is the row replacement of Stage 8.

What we have. The weak form (21.5), with only first derivatives, derived from the fundamental theorem of geometric calculus through the divergence theorem (21.2) and Green's identity (21.4). The remaining stages replace H^1 by a finite-dimensional space built on the mesh.

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

22. Stage 5: The Discretization: the Mesh and the Finite-Dimensional Space

What we want. We want a finite-dimensional space V_h of tangent fields on the mesh, with a basis, in which to look for an approximate solution of the weak form (21.5),

$$\int_\Omega \langle \psi\, \widetilde{\chi} \rangle\, dA + \Delta t \int_\Omega \sum_k \langle (D_{e_k}\psi)\, \widetilde{(D_{e_k}\chi)} \rangle\, dA = \int_\Omega \langle \psi^0\, \widetilde{\chi} \rangle\, dA. \tag{21.5}$$

In the plane this is easy: interpolate the values at the vertices with the hat functions. On a curved, piecewise flat surface it is not, and this is where differential geometry enters the finite element method. The unknown at a vertex is a vector of the vertex's tangent plane; inside a face, a tangent field must take values in the plane of the face; and the planes are all different. Before the vertex vectors can be interpolated in a face they must be carried into the plane of the face. So we need, in order: the tangent planes of the faces and the vertices, as bivectors; the rotor that carries one plane onto another, and the reason for choosing the smallest one; the resulting connection angle; what of all this is intrinsic; and finally the spinor hat functions.

22.1. Tangent bivectors of the faces and the vertices

A face. A face with vertices x_1, x_2, x_3, in counterclockwise order, has the edge vectors $e_1 = x_2 - x_1$ and $e_2 = x_3 - x_1$. Part I defined its normal and its area by (3.2),

$$n_f = \frac{e_1 \times e_2}{|e_1 \times e_2|}, \qquad A_f = \tfrac{1}{2}\,|e_1 \times e_2|, \tag{3.2}$$

so that $e_1 \times e_2 = |e_1 \times e_2|\, n_f = 2A_f\, n_f$. The outer product of the two edges is the *area bivector* of the face. We expand it one rule at a time:

$$\begin{aligned} E_f = e_1 \wedge e_2 &= I\,(e_1 \times e_2) && a \wedge b = I(a \times b) \\ &= I\,(2A_f\, n_f) && e_1 \times e_2 = 2A_f\, n_f \\ &= 2A_f\, I n_f && \text{the real number } 2A_f \text{ moves out} \\ &= 2A_f\, \mathsf{T}_f && \mathsf{T}_f = I n_f, \text{ by (18.1).} \end{aligned} \tag{22.1}$$

The face plane is the plane of its two edges, $\mathsf{T}_f = E_f/(2A_f)$. Its magnitude is $|E_f| = 2A_f\,|\mathsf{T}_f| = 2A_f$, since T_f has unit magnitude, so $A_f = \frac{1}{2}|E_f|$ and $\mathsf{T}_f = E_f/|E_f|$. The first frame vector of the face is its first edge, normalized, $t_f = e_1/|e_1|$; it lies in the plane already. The program computes these in `computeGeometry`:

```
58    const int nf = mesh.numFaces();
59    g.faceNormal.resize(nf); g.faceArea.resize(nf); g.faceT.resize(nf); g.faceFrame.
          resize(nf);
60    for (Face& f : mesh.getFaces()) { // computeFaceNormals, computeFaceTangentSpace
61        Vertex* v[3];
62        faceVertices(f, v);
63        const Vector3d n = (v[1]->p - v[0]->p).cross(v[2]->p - v[0]->p);
64        g.faceArea[f.ID] = 0.5 * n.norm();
65        g.faceNormal[f.ID] = n.normalized();
66        const Vector3d t = (v[1]->p - v[0]->p).normalized();
67        g.faceT[f.ID] = t;
68        g.faceFrame[f.ID].col(0) = t;
69        g.faceFrame[f.ID].col(1) = g.faceNormal[f.ID].cross(t);
70    }
```

Lines 58–62 size the arrays and visit the faces, each with its three vertices in counterclockwise order (`faceVertices`). Line 63 is the dual vector of E_f, the cross product $e_1 \times e_2$. Line 64 is $A_f = \frac{1}{2}|E_f|$ and line 65 the unit normal, the dual vector of T_f, which is how `tangentBivector` stores the plane. Line 66 is t_f, stored on line 67. Lines 68–69 build the 3×2 frame $[\,t_f \;\; n_f \times t_f\,]$ of Part I, used only by the matrix version; in geometric algebra the second column is $t_f \mathsf{T}_f$, determined by t_f and the plane.

A vertex. A vertex has no tangent plane of its own: it is the apex of a cone of faces. The code gives it one, orthogonal to the area-weighted normal (3.3),

$$n_i = \frac{\sum_{f \ni i} 2A_f\, n_f}{\left|\sum_{f \ni i} 2A_f\, n_f\right|}. \tag{3.3}$$

In geometric algebra the weighted sum of normals is a sum of planes. Multiply the numerator by I and use (22.1) $[\,E_f = 2A_f\, I n_f\,]$ term by term:

$$I \sum_{f \ni i} 2A_f\, n_f = \sum_{f \ni i} 2A_f\, I n_f = \sum_{f \ni i} E_f.$$

Multiplication by I does not change magnitudes ($|Iv|^2 = -(Iv)^2 = -I^2 v^2 = |v|^2$), so the denominator is $\left|\sum_f E_f\right|$ as well, and

$$\mathsf{T}_i = I n_i = \frac{I \sum_{f \ni i} 2A_f\, n_f}{\left|\sum_{f \ni i} 2A_f\, n_f\right|} = \frac{\sum_{f \ni i} E_f}{\left|\sum_{f \ni i} E_f\right|}\,:$$

the vertex plane is the normalized sum of the area bivectors of the faces around it. Its first frame vector t_i is the first edge from i, projected onto the plane with P of (18.2) [$P(a) = a - (a \cdot n)\, n$] and normalized, as in (3.4):

$$t_i = \frac{(x_j - x_i) - \big((x_j - x_i) \cdot n_i\big)\, n_i}{\big|(x_j - x_i) - \big((x_j - x_i) \cdot n_i\big)\, n_i\big|}. \tag{3.4}$$

```
51    for (Vertex& vi : mesh.getVertices()) { // computeVertexTangentSpace
52        const Vector3d e = vi.edge->pair->vertex->p - vi.p;
53        const Vector3d t = (e - e.dot(vi.n) * vi.n).normalized();
54        g.vertexT[vi.ID] = t;
55        g.vertexFrame[vi.ID].col(0) = t;
56        g.vertexFrame[vi.ID].col(1) = vi.n.cross(t);
57    }
```

Line 52 is the edge $x_j - x_i$ to the first neighbor. Line 53 is the projection P, written in its vector form $a - (a \cdot n_i)\, n_i$, followed by normalization: this is t_i. Line 56 stores the second frame vector $b_i = n_i \times t_i$, which by (18.3) [$a\,\mathsf{T} = n \times a$] is $t_i\,\mathsf{T}_i$. The geometric algebra version needs only line 54, the single vector t_i; the matrix version needs the whole 3×2 frame of lines 55–56.

22.2. The rotor between two planes

What we want, and why this rotor. To compare a vector of the vertex plane, with normal n_i, with vectors of a face plane, with normal n_f, we must carry the vertex plane onto the face plane. Any rotation that takes n_i to n_f does that, and there are infinitely many: after one such rotation we may spin the face plane about n_f by any angle and still have n_i taken to n_f. These rotations differ only by a rotation within the face plane, so they would give connection angles ρ_i^f that differ by an arbitrary constant. We must pick one, and the right choice is the one that does not spin: the rotation whose plane contains both normals, $n_i \wedge n_f$. It moves the normal n_i straight towards n_f, by the angle between them, and it leaves fixed the line in which the two tangent planes meet, the direction $n_i \times n_f$, which belongs to both planes. The next subsection shows that this "no spin" rule, applied to infinitely many infinitely small steps, is exactly parallel transport; here it is applied to one finite step, from a vertex into a face. It is the smallest rotation taking n_i to n_f, the rotation of the Rodrigues formula (4.6) of Part I,

$$Q = I + [v]_\times + \frac{1}{1 + a \cdot c}\,[v]_\times^2, \qquad v = a \times c. \tag{4.6}$$

In geometric algebra this rotation has a short formula, which we now derive. Let a and c be unit vectors (later $a = n_i$, $c = n_f$), with angle ϕ between them, and $a \cdot c = \cos\phi \neq -1$.

Step 1: the sandwich by one unit vector is a reflection. Two rules of Section 17 are all we need. Adding $xp = x \cdot p + x \wedge p$ and $px = x \cdot p - x \wedge p$ gives

$$xp + px = 2\,(x \cdot p), \qquad \text{so} \qquad xp = 2\,(x \cdot p) - px.$$

For a unit vector p ($pp = 1$) and any vector x, substitute this into $p\,x\,p$:

$$\begin{aligned} p\,x\,p &= p\,(xp) && \text{grouping} \\ &= p\left(2\,(x\cdot p) - px\right) && xp = 2(x\cdot p) - px \\ &= 2\,(x\cdot p)\,p - (pp)\,x && \text{distributing; the scalar } x\cdot p \text{ moves out} \\ &= 2\,(x\cdot p)\,p - x && pp = 1. \end{aligned}$$

Split $x = x_\parallel + x_\perp$ into its part along p, $x_\parallel = (x\cdot p)\,p$, and the rest. Then $2(x\cdot p)p - x = 2x_\parallel - x_\parallel - x_\perp = x_\parallel - x_\perp$: the component along p is kept and the orthogonal component is flipped. The map $x \mapsto p\,x\,p$ is the *reflection in the line of* p. The same result can be read off the commutation rules directly: $x_\parallel$ is parallel to p, so it commutes with p, and $p\,x_\parallel\,p = x_\parallel\,pp = x_\parallel$; $x_\perp$ is orthogonal to p, so it anticommutes, and $p\,x_\perp\,p = -x_\perp\,pp = -x_\perp$.

Step 2: a product of two unit vectors is a rotor. For unit vectors p, q, let $R = qp$. Its reverse is $\widetilde{R} = pq$ (reversion reverses the order of vector factors), and

$$R\widetilde{R} = (qp)(pq) = q\,(pp)\,q = q\,q = 1,$$

so R is a rotor. Its sandwich is two reflections, one after the other: $R\,x\,\widetilde{R} = q\,p\,x\,p\,q = q\,(p\,x\,p)\,q$, the reflection in the line of p followed by the reflection in the line of q.

Step 3: the right two vectors, and $m\,a\,m = c$. We want to take a to c. Take the half-way unit vector

$$m = \frac{a+c}{|a+c|}, \qquad |a+c|^2 = (a+c)\cdot(a+c) = a\cdot a + 2\,a\cdot c + c\cdot c = 2 + 2\,a\cdot c.$$

Reflect a in the line of m, with the formula of Step 1 [$p\,x\,p = 2(x\cdot p)\,p - x$] for $p = m$, $x = a$:

$$\begin{aligned} m\,a\,m &= 2\,(a\cdot m)\,m - a && \text{Step 1} \\ &= 2\,\frac{a\cdot(a+c)}{|a+c|}\,\frac{a+c}{|a+c|} - a && m = \tfrac{a+c}{|a+c|} \text{ substituted twice} \\ &= \frac{2\,(1+a\cdot c)}{|a+c|^2}\,(a+c) - a && a\cdot a = 1;\ \text{collecting the two denominators} \\ &= \frac{2\,(1+a\cdot c)}{2+2\,a\cdot c}\,(a+c) - a && |a+c|^2 = 2 + 2\,a\cdot c \\ &= (a+c) - a = c && \text{cancelling.} \end{aligned}$$

The line of m bisects the angle between a and c, so this is what a reflection must do. Reflecting c in its own line leaves it unchanged: $c\,c\,c = (cc)\,c = c$. So the rotor $Q = c\,m$ (first reflect in m, then in c), with $\widetilde{Q} = m\,c$, takes a to

c:

$$\begin{aligned} Q\,a\,\widetilde{Q} &= (c\,m)\,a\,(m\,c) && Q = cm,\ \widetilde{Q} = mc \\ &= c\,(m\,a\,m)\,c && \text{regrouping (the product is associative)} \\ &= c\,(c)\,c && m\,a\,m = c \\ &= (c\,c)\,c = c && c\,c = 1. \end{aligned}$$

Step 4: its formula. Substitute m into $Q = c\,m$:

$$\begin{aligned} Q = c\,m &= c\,\frac{a+c}{|a+c|} && m \text{ substituted} \\ &= \frac{c\,a + c\,c}{|a+c|} && \text{distributing the product} \\ &= \frac{1 + c\,a}{|a+c|} && c\,c = 1 \\ &= \frac{1 + c\,a}{\sqrt{2(1+a\cdot c)}} && |a+c|^2 = 2 + 2\,a\cdot c. \end{aligned}$$

The denominator is the magnitude of the numerator. To see it, multiply the numerator by its reverse, $\widetilde{(1+ca)} = 1 + ac$:

$$\begin{aligned} (1+ca)(1+ac) &= 1 + ac + ca + c\,(aa)\,c && \text{distributing} \\ &= 1 + (ac + ca) + c\,c && aa = 1 \\ &= 1 + 2\,a\cdot c + 1 && ac + ca = 2\,a\cdot c,\ cc = 1 \\ &= 2\,(1 + a\cdot c), \end{aligned}$$

a positive scalar, the squared magnitude $|1+ca|^2$. So

$$Q = \frac{1 + c\,a}{|1 + c\,a|} = \frac{1 + c\,a}{\sqrt{2(1+a\cdot c)}}. \tag{22.2}$$

Step 5: checks, factor by factor. We check directly that (22.2) does what Steps 1–3 promised. Write $N = |1+ca|$, so $N^2 = 2(1+a\cdot c)$, $Q = (1+ca)/N$ and $\widetilde{Q} = (1+ac)/N$.

- *It is a rotor.* $Q\widetilde{Q} = (1+ca)(1+ac)/N^2 = 2(1+a\cdot c)/N^2 = 1$, by the product just computed; and $\widetilde{Q}Q = (1+ac)(1+ca)/N^2 = \big(1+ca+ac+a(cc)a\big)/N^2 = \big(2+2\,a\cdot c\big)/N^2 = 1$ too.
- *It takes a to c.* The first product is $(1+ca)\,a = a + c\,(aa) = a + c$. The second:

$$\begin{aligned} (a+c)(1+ac) &= a + aac + c + cac && \text{distributing} \\ &= a + c + c + cac && aa = 1 \\ &= a + 2c + \big(2\,(a\cdot c)\,c - a\big) && c\,a\,c = 2(a\cdot c)\,c - a,\ \text{Step 1} \\ &= 2\,(1 + a\cdot c)\,c && \text{the two } a \text{ cancel.} \end{aligned}$$

Dividing by $N^2 = 2(1+a\cdot c)$: $Q\,a\,\widetilde{Q} = c$.

- *Its angle and plane.* The geometric product $ca = c\cdot a + c\wedge a = \cos\phi + c\wedge a$. Let B be the unit bivector of $a \wedge c$, so $a \wedge c = B\sin\phi$ and $c \wedge a = -a \wedge c = -B\sin\phi$. With the half-angle formulas $1 + \cos\phi = 2\cos^2\frac{\phi}{2}$ and $\sin\phi = 2\sin\frac{\phi}{2}\cos\frac{\phi}{2}$,

$$1 + ca = 1 + \cos\phi - B\sin\phi = 2\cos\tfrac{\phi}{2}\left(\cos\tfrac{\phi}{2} - B\sin\tfrac{\phi}{2}\right),$$
$$N = \sqrt{2(1+\cos\phi)} = \sqrt{4\cos^2\tfrac{\phi}{2}} = 2\cos\tfrac{\phi}{2},$$

  and dividing, $Q = \cos\frac{\phi}{2} - B\sin\frac{\phi}{2} = e^{-B\phi/2}$. By the rule of Section 17, this rotor turns by ϕ in the plane B of a and c, from a towards c.
- *It does not spin: the axis is fixed.* Let k be a vector orthogonal to both a and c, for instance $k = a \times c$. Then k anticommutes with a and with c, so it commutes with their product: $(ca)\,k = c\,(ak) = -c\,(ka) = -(ck)\,a = k\,c\,a$. Hence k commutes with $Q = (1 + ca)/N$, and $Q\,k\,\widetilde{Q} = k\,Q\widetilde{Q} = k$. The rotation fixes the axis $a \times c$, as the Rodrigues rotation (4.6) [axis $v = a \times c$] does; since both turn by ϕ about the same axis, they are the same rotation.
- *It carries the planes.* The pseudoscalar I commutes with every element, so $Q\,(Ia)\,\widetilde{Q} = I\,(Q\,a\,\widetilde{Q}) = Ic$: the plane orthogonal to a goes to the plane orthogonal to c.

Proposition 22.1. *For unit vectors a and c with $a \cdot c \neq -1$, the rotor (22.2) [$Q = \frac{1+ca}{\sqrt{2(1+a\cdot c)}}$] turns a to c in the plane $a \wedge c$, by their angle; it is the rotation of the Rodrigues formula (4.6), and it takes the bivector Ia to Ic.*

From a vertex to a face. With $a = n_i$ and $c = n_f$, the rotor

$$Q_{i\to f} = \frac{1 + n_f\,n_i}{|1 + n_f\,n_i|} \tag{22.3}$$

takes the normal of vertex i to that of face f and, by the last check, the tangent bivector $\mathsf{T}_i = In_i$ to $\mathsf{T}_f = In_f$:

$$Q_{i\to f}\,\mathsf{T}_i\,\widetilde{Q}_{i\to f} = Q_{i\to f}\,(In_i)\,\widetilde{Q}_{i\to f} = I\left(Q_{i\to f}\,n_i\,\widetilde{Q}_{i\to f}\right) = In_f = \mathsf{T}_f.$$

It carries the tangent plane of the vertex onto that of the face, and with it every tangent vector, $a \mapsto Q_{i\to f}\,a\,\widetilde{Q}_{i\to f}$. The failure of `alignVectors` for opposite normals is visible here: $1 + n_f n_i = 1 + n_f \cdot n_i + n_f \wedge n_i = 1 - 1 + 0 = 0$ when $n_f = -n_i$, and no plane of rotation is singled out.

22.3. Why the smallest rotor: rolling without twisting

What we want. Part I defined parallel transport by the equation $\nabla_{\gamma'} U = 0$, which is hard to picture. Here is a mechanical picture. Along a curve $\gamma(s)$ on the surface, carry a rigid plane, always tangent to the surface. The plane must follow the surface, so it must tilt as the normal $n(s)$ changes; but we may choose not to spin it about the normal. A vector drawn on the plane is then carried along. We show that this "rolling without twisting" is parallel transport, and that it is the limit of a chain of the smallest rotors (22.2).

This is the reason the discrete method uses the smallest rotor: it is one finite step of parallel transport.

The rotor of one small step. Over a small step ds the normal moves from n to $n + n'\,ds$, where $n' = dn/ds$. We keep terms of first order in ds and drop those in ds^2. Differentiating $n \cdot n = 1$ gives $2\,n \cdot n' = 0$. Hence the new normal has unit length to first order,

$$|n + n'ds|^2 = n \cdot n + 2\,(n \cdot n')\,ds + (n' \cdot n')\,ds^2 = 1 + 0 + O(ds^2),$$

and its inner product with the old one is $n \cdot (n + n'ds) = 1 + (n \cdot n')\,ds = 1$. Now apply (22.2) $\left[\,Q = \frac{1+ca}{\sqrt{2(1+a\cdot c)}}\,\right]$ with $a = n$ and $c = n + n'ds$. The numerator:

$$\begin{aligned} 1 + c\,a &= 1 + (n + n'\,ds)\,n && c = n + n'ds \\ &= 1 + n\,n + (n'\,n)\,ds && \text{distributing} \\ &= 2 + (n' \cdot n + n' \wedge n)\,ds && nn = 1;\ n'n = n' \cdot n + n' \wedge n \\ &= 2 + (n' \wedge n)\,ds && n' \cdot n = 0. \end{aligned}$$

The denominator: $\sqrt{2\,(1 + a \cdot c)} = \sqrt{2\,(1+1)} = 2$. Dividing, the rotor of the step is

$$Q_{ds} = \frac{2 + (n' \wedge n)\,ds}{2} = 1 + \tfrac{1}{2}\,\Omega\,ds, \qquad \Omega = n' \wedge n.$$

The rotor equation. Let $R(s)$ be the rotor that carries the plane from $\gamma(0)$ to $\gamma(s)$. The step from s to $s + ds$ acts after it, so a vector x goes first to $R\,x\,\widetilde{R}$ and then to $Q_{ds}\,(R\,x\,\widetilde{R})\,\widetilde{Q}_{ds} = (Q_{ds}R)\,x\,(\widetilde{R}\,\widetilde{Q}_{ds}) = (Q_{ds}R)\,x\,\widetilde{(Q_{ds}R)}$, using $\widetilde{XY} = \widetilde{Y}\,\widetilde{X}$. The rotor of the first $s + ds$ is therefore the product $Q_{ds}R$:

$$\begin{aligned} R(s + ds) = Q_{ds}\,R(s) &= \big(1 + \tfrac{1}{2}\,\Omega\,ds\big)\,R(s) \\ &= R(s) + \tfrac{1}{2}\,\Omega\,R(s)\,ds && \text{distributing.} \end{aligned}$$

Subtracting $R(s)$ and dividing by ds, $\big(R(s + ds) - R(s)\big)/ds = \frac{1}{2}\,\Omega\,R(s)$. In the limit $ds \to 0$,

$$R' = \tfrac{1}{2}\,\Omega\,R, \qquad \Omega = n' \wedge n, \qquad R(0) = 1.$$

This is the continuous version of the chain of smallest rotors, and Figure 20 draws it exactly that way. The bivector Ω is the plane of n and n': it contains the normal. A rotation in that plane tilts the normal towards n' and does not spin anything about the normal. A spin about the normal would be a rotation in the tangent plane, a multiple of $\mathsf{T} = In$, which is a different bivector; it is absent.

How a rotated vector changes. Let $v(s) = R\,v_0\,\widetilde{R}$ be a fixed vector v_0 carried by the rotor. We need its derivative, and for it the derivative of $\widetilde{R}$. Reversion is linear, so it commutes with the derivative, and

$$\widetilde{R}' = \widetilde{(R')} = \big(\widetilde{\tfrac{1}{2}\,\Omega\,R}\big) = \tfrac{1}{2}\,\widetilde{R}\,\widetilde{\Omega} = -\tfrac{1}{2}\,\widetilde{R}\,\Omega,$$

by $\widetilde{XY} = \widetilde{Y}\,\widetilde{X}$ and $\widetilde{\Omega} = -\Omega$ for a bivector. By the product rule,

$$\begin{aligned}
v' &= R'\,v_0\,\widetilde{R} + R\,v_0\,\widetilde{R}' && \text{product rule}\\
&= \left(\tfrac{1}{2}\,\Omega\,R\right) v_0\,\widetilde{R} + R\,v_0\left(-\tfrac{1}{2}\,\widetilde{R}\,\Omega\right) && \text{the rotor equation and its reverse}\\
&= \tfrac{1}{2}\,\Omega\,(Rv_0\widetilde{R}) - \tfrac{1}{2}\,(Rv_0\widetilde{R})\,\Omega && \text{regrouping}\\
&= \tfrac{1}{2}\,(\Omega\,v - v\,\Omega) && v = Rv_0\widetilde{R}\\
&= \Omega \cdot v && B\cdot v = -v\cdot B = \tfrac{1}{2}(Bv - vB).
\end{aligned}$$

The last rule is the one of Section 17, $v \cdot B = \frac{1}{2}(vB - Bv) = -B \cdot v$. For $\Omega = x \wedge y$ with $x = n'$ and $y = n$, we expand this inner product. Write $x \wedge y = xy - x \cdot y$; the scalar $x \cdot y$ commutes with v and drops out of $\Omega v - v\Omega$, so $\Omega v - v\Omega = xyv - vxy$. Move v to the front of xyv with the rule $pq = 2\,(p \cdot q) - qp$ of Step 1, twice:

$$\begin{aligned}
x\,y\,v &= x\left(2\,(y\cdot v) - v\,y\right) && yv = 2(y\cdot v) - vy\\
&= 2\,(y\cdot v)\,x - (x\,v)\,y && \text{distributing}\\
&= 2\,(y\cdot v)\,x - \left(2\,(x\cdot v) - v\,x\right) y && xv = 2(x\cdot v) - vx\\
&= 2\,(y\cdot v)\,x - 2\,(x\cdot v)\,y + v\,x\,y && \text{distributing.}
\end{aligned}$$

Subtracting vxy and halving,

$$(x \wedge y)\cdot v = \tfrac{1}{2}\,(xyv - vxy) = x\,(y\cdot v) - y\,(x\cdot v),$$

and with $x = n'$, $y = n$,

$$v' = \Omega \cdot v = n'\,(n\cdot v) - n\,(n'\cdot v).$$

The rotor follows the normal. Apply this to the starting normal, $m(s) = R\,n(0)\,\widetilde{R}$. It satisfies

$$m' = n'\,(n\cdot m) - n\,(n'\cdot m), \qquad m(0) = n(0),$$

a linear differential equation for m, with the coefficients $n(s), n'(s)$ given. Try $m = n$ on the right side: $n'\,(n\cdot n) - n\,(n'\cdot n) = n'\cdot 1 - n\cdot 0 = n'$, which is the derivative of n. So $n(s)$ solves the same equation with the same starting value, and a linear differential equation with a given starting value has only one solution. Hence $m = n$: $R\,n(0)\,\widetilde{R} = n(s)$. The rotor carries the normal at the start to the normal along the curve. It also preserves inner products: for vectors x, y, by the cyclic rule $\langle XY\rangle = \langle YX\rangle$,

$$(Rx\widetilde{R})\cdot(Ry\widetilde{R}) = \langle Rx\widetilde{R}\,Ry\widetilde{R}\rangle = \langle R\,x\,y\,\widetilde{R}\rangle = \langle x\,y\,\widetilde{R}R\rangle = \langle xy\rangle = x\cdot y,$$

using $\widetilde{R}R = 1$ twice. So a tangent vector a_0 at the start, with $a_0\cdot n(0) = 0$, stays tangent: $a(s) = R\,a_0\,\widetilde{R}$ has $a\cdot n = (Ra_0\widetilde{R})\cdot(R\,n(0)\,\widetilde{R}) = a_0\cdot n(0) = 0$.

The carried vector is parallel. For such a tangent vector, $n \cdot a = 0$, and the formula for the derivative loses its first term:

$$a' = n'\,(n \cdot a) - n\,(n' \cdot a) = n' \cdot 0 - (n' \cdot a)\,n = -(n' \cdot a)\,n.$$

The derivative of a in space is purely normal. Its tangent part is the covariant derivative, by the Gauss formula (18.7),

$$\gamma' \cdot \nabla a = \nabla_{\gamma'} a + \mathrm{II}(\gamma', a)\,n, \tag{18.7}$$

with $\gamma' \cdot \nabla a = a'$. Comparing, $\nabla_{\gamma'} a = P(a') = P\big(-(n' \cdot a)\,n\big) = 0$ and $\mathrm{II}(\gamma', a) = -n' \cdot a$. The vector carried by the rolling plane is parallel along the curve. All the change it undergoes in space is the tilting forced by the surface, recorded by II, exactly as for the field e_φ on the cylinder of Stage 1. Had we added a spin $\lambda\,\mathsf{T}$ to Ω, the derivative would have gained the term $\lambda\,\mathsf{T} \cdot a = \frac{\lambda}{2}(\mathsf{T}a - a\mathsf{T}) = \frac{\lambda}{2}(-a\mathsf{T} - a\mathsf{T}) = -\lambda\, a\mathsf{T}$ (a tangent a anticommutes with T), a tangent multiple of the quarter turn of a, and the vector would turn within the surface.

Proposition 22.2. *Let $R(s)$ solve $R' = \frac{1}{2}\Omega\,R$ with $R(0) = 1$, where $\Omega = n' \wedge n$ and $n(s)$ is the normal along the curve. Then $Rn(0)\widetilde{R} = n(s)$, and for every tangent vector a_0 at $\gamma(0)$, the field $a(s) = R\,a_0\widetilde{R}$ is tangent and parallel along the curve.*

Figure 20 shows the rolling planes along a curve on a sphere. The plane *rolls* along the curve, as a coin rolls on a table, without twisting about its normal. Parallel transport is this rolling. The discrete transport $Q_{i\to f}$ of (22.3) $\big[\,Q_{i\to f} = \frac{1+n_f n_i}{|1+n_f n_i|}\,\big]$ is the same idea over one finite step: the rotor in the plane $n_f \wedge n_i$, which contains both normals and therefore has no component of spin about either of them.

22.4. The connection angle

What we want. After the rotor, the first frame vector of the vertex has become $c = Qt_i\widetilde{Q}$, a unit vector of the face plane. The face has its own first frame vector t_f. The discrete connection is the angle ρ_i^f from t_f to c: it says how the frame of the vertex looks when seen from the face. In Part I it was hidden in the 2×2 matrix (4.7) $\big[\,R_i^f = T_f^T Q_{i\to f} T_i\,\big]$. Here it comes out of one product.

The product of two unit vectors of a plane. Let t_f and c be unit vectors of the face plane, and ρ the angle from t_f to c, measured counterclockwise about n_f. In the face frame (t_f, b_f), with $b_f = t_f\mathsf{T}_f$,

$$c = \cos\rho\; t_f + \sin\rho\; b_f = \cos\rho\; t_f + \sin\rho\; t_f\mathsf{T}_f.$$

Multiply on the left by t_f:

$$\begin{aligned}
t_f\,c &= t_f\left(\cos\rho\; t_f + \sin\rho\; t_f\mathsf{T}_f\right) && c \text{ substituted}\\
&= \cos\rho\; t_f t_f + \sin\rho\; t_f t_f\mathsf{T}_f && \text{distributing; real numbers move out}\\
&= \cos\rho + \mathsf{T}_f \sin\rho && t_f t_f = 1, \text{ twice}\\
&= e^{\mathsf{T}_f \rho} && e^{\mathsf{T}\theta} = \cos\theta + \mathsf{T}\sin\theta.
\end{aligned}$$

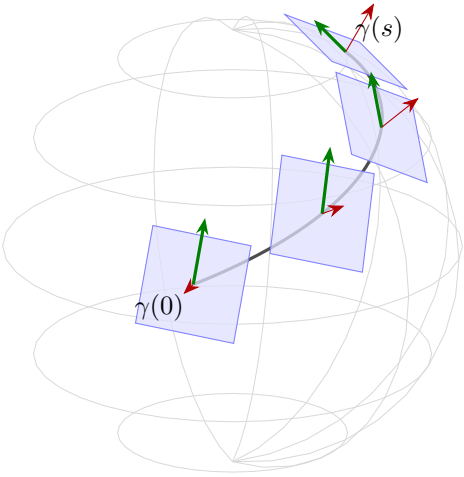


FIGURE 20. Parallel transport is rolling without twisting. Along a curve γ on a sphere, the tangent plane (blue) is rolled by the rotor of Proposition 22.2, here integrated as a sequence of smallest rotations between consecutive normals (red). The transported vector (green) changes only in the direction of the normal, so its covariant derivative vanishes.

The geometric product of two unit vectors of a plane is the unit complex number of that plane whose angle is the angle between them, from the first to the second. Its scalar part $t_f \cdot c$ is the cosine and its bivector part $t_f \wedge c$ is the sine times the plane. Applied to the transported frame vector, this gives the discrete connection as a single number:

$$e^{\mathsf{T}_f \rho_i^f} = t_f \left(Q_{i\to f}\, t_i\, \widetilde{Q}_{i\to f} \right). \tag{22.4}$$

Transport is multiplication. Now take any tangent vector of the vertex, $U = t_i\psi$ with $\psi = u_1 + u_2\mathsf{T}_i$ as in (18.5) [$U = t\psi$, $\psi = u_1 + u_2\mathsf{T}$], and carry it into the face. Write $Q = Q_{i\to f}$ and $\psi_f = u_1 + u_2\mathsf{T}_f$, the same pair of numbers read in the face plane. We want to show that

$$Q\, t_i \psi\, \widetilde{Q} = (Q t_i \widetilde{Q})(Q \psi \widetilde{Q}) = t_f\, e^{\mathsf{T}_f \rho_i^f}\, \psi_f. \tag{22.5}$$

Split the sandwich. Insert $1 = \widetilde{Q}Q$ between t_i and ψ:

$$\begin{aligned} Q\, t_i\, \psi\, \widetilde{Q} &= Q\, t_i\, (1)\, \psi\, \widetilde{Q} \\ &= Q\, t_i\, (\widetilde{Q}Q)\, \psi\, \widetilde{Q} && 1 = \widetilde{Q}Q \text{ (Step 5)} \\ &= (Q\, t_i\, \widetilde{Q})\, (Q\, \psi\, \widetilde{Q}) && \text{regrouping.} \end{aligned}$$

The rotation of a product is the product of the rotations.

Rotate the coordinate. Substitute $\psi = u_1 + u_2\mathsf{T}_i$:

$$\begin{aligned} Q\,\psi\,\widetilde{Q} &= Q\,(u_1 + u_2\mathsf{T}_i)\,\widetilde{Q} && \psi \text{ substituted} \\ &= u_1\,Q\widetilde{Q} + u_2\,Q\,\mathsf{T}_i\,\widetilde{Q} && \text{distributing; real numbers move out} \\ &= u_1 + u_2\,\mathsf{T}_f = \psi_f && Q\widetilde{Q} = 1,\ Q\mathsf{T}_i\widetilde{Q} = \mathsf{T}_f. \end{aligned}$$

The numbers u_1, u_2 do not change; only the plane they refer to does.

Rotate the frame vector. Write $c = Qt_i\widetilde{Q}$ and insert $1 = t_f t_f$ in front of it:

$$\begin{aligned} Q\,t_i\,\widetilde{Q} = c &= (t_f\,t_f)\,c && t_f t_f = 1 \\ &= t_f\,(t_f\,c) && \text{regrouping} \\ &= t_f\,e^{\mathsf{T}_f\rho_i^f} && t_f\,c = e^{\mathsf{T}_f\rho_i^f},\ \text{by (22.4)}. \end{aligned}$$

Put the two together. $Q\,t_i\,\psi\,\widetilde{Q} = \left(t_f\,e^{\mathsf{T}_f\rho_i^f}\right)\psi_f = t_f\,e^{\mathsf{T}_f\rho_i^f}\,\psi_f$, which is (22.5).

Read the result with (18.5) [$U = t\psi$]: the transported vector is t_f times the complex number $e^{\mathsf{T}_f\rho_i^f}\psi_f$, so that number is its spinor coordinate in the face frame. In spinor coordinates, transport from the vertex to the face is *multiplication by the unit complex number* $e^{\mathsf{T}_f\rho_i^f}$. The step "rotate the coordinate" also tells us how to compare complex numbers of different planes: the rotor maps $u_1 + u_2\mathsf{T}_i$ to $u_1 + u_2\mathsf{T}_f$, the same pair of coefficients. From now on we identify every $\alpha + \beta\mathsf{T}$, in whatever plane, with the complex number $\alpha + \beta\,\mathrm{i}$, and write ψ_i for the coordinate of vertex i read in any plane; this identification is exactly what the code does when it stores all of them as `std::complex`.

The matrix of the multiplication. Multiply $u_1 + u_2\mathsf{T}$ by $\cos\rho + \mathsf{T}\sin\rho$, using only $\mathsf{T}^2 = -1$:

$$\begin{aligned} &(\cos\rho + \mathsf{T}\sin\rho)(u_1 + u_2\mathsf{T}) \\ &\quad = u_1\cos\rho + u_2\cos\rho\,\mathsf{T} + u_1\sin\rho\,\mathsf{T} + u_2\sin\rho\,\mathsf{T}^2 \\ &\quad = (u_1\cos\rho - u_2\sin\rho) + (u_1\sin\rho + u_2\cos\rho)\,\mathsf{T}. \end{aligned}$$

In the basis $(1, \mathsf{T}_f)$ this takes the column (u_1, u_2) to the column $(u_1\cos\rho - u_2\sin\rho,\ u_1\sin\rho + u_2\cos\rho)$, so it is the matrix

$$R_i^f = \begin{pmatrix} \cos\rho_i^f & -\sin\rho_i^f \\ \sin\rho_i^f & \cos\rho_i^f \end{pmatrix}, \tag{22.6}$$

which is the matrix $R_i^f = T_f^T Q_{i\to f} T_i$ of (4.7), as the code shows below with numbers.

Transport between two vertices. The transport between two vertices through the face, (4.8) [$R_{j\to i}^f = (R_i^f)^T R_j^f$], becomes a product of two complex numbers. The transpose of (22.6) is

$$(R_i^f)^T = \begin{pmatrix} \cos\rho_i^f & \sin\rho_i^f \\ -\sin\rho_i^f & \cos\rho_i^f \end{pmatrix} = \begin{pmatrix} \cos(-\rho_i^f) & -\sin(-\rho_i^f) \\ \sin(-\rho_i^f) & \cos(-\rho_i^f) \end{pmatrix},$$

the matrix of multiplication by $e^{-\mathsf{T}_f \rho_i^f}$. A product of matrices of multiplications is the matrix of multiplication by the product of the numbers, and two exponentials of one plane multiply by adding the angles:

$$\begin{aligned} e^{-\mathsf{T}\alpha}\, e^{\mathsf{T}\beta} &= (\cos\alpha - \mathsf{T}\sin\alpha)(\cos\beta + \mathsf{T}\sin\beta) \\ &= \cos\alpha\cos\beta + \sin\alpha\sin\beta \\ &\quad + \mathsf{T}\,(\cos\alpha\sin\beta - \sin\alpha\cos\beta) && \mathsf{T}^2 = -1 \\ &= \cos(\beta-\alpha) + \mathsf{T}\sin(\beta-\alpha) && \text{addition formulas} \\ &= e^{\mathsf{T}(\beta-\alpha)}. \end{aligned}$$

With $\alpha = \rho_i^f$ and $\beta = \rho_j^f$:

$$(R_i^f)^T R_j^f \;\longleftrightarrow\; e^{-\mathsf{T}_f \rho_i^f}\, e^{\mathsf{T}_f \rho_j^f} = e^{\mathsf{T}_f (\rho_j^f - \rho_i^f)}.$$

The connection angles subtract, and the arbitrary face frame cancels. To see it, turn the face frame to $t'_f = t_f\, e^{\mathsf{T}_f \theta}$. A tangent vector anticommutes with T_f, so $t_f\, e^{\mathsf{T}_f\theta} = t_f \cos\theta + t_f \mathsf{T}_f \sin\theta = t_f\cos\theta - \mathsf{T}_f t_f \sin\theta = e^{-\mathsf{T}_f\theta}\, t_f$, and the new connection (22.4) is

$$t'_f\, c = e^{-\mathsf{T}_f\theta}\, t_f\, c = e^{-\mathsf{T}_f\theta}\, e^{\mathsf{T}_f\rho_i^f} = e^{\mathsf{T}_f(\rho_i^f - \theta)}.$$

Both ρ_i^f and ρ_j^f decrease by θ, and the difference $(\rho_j^f - \theta) - (\rho_i^f - \theta) = \rho_j^f - \rho_i^f$ does not change, as found in Section 4 and as the frame independence (18.10) [$D'_a\psi' = e^{-\mathsf{T}\theta} D_a \psi$] of Stage 1 demands.

In the code. The second part of `GAConnection.h` implements (22.3) [$Q_{i\to f} = \frac{1+n_f n_i}{|1+n_f n_i|}$] and (22.4) [$e^{\mathsf{T}_f \rho_i^f} = t_f\,(Q_{i\to f} t_i \widetilde{Q}_{i\to f})$]:

```
/// The smallest rotor taking the unit vector a to the unit vector c (Proposition ga_rotor):
/// Q = (1 + c a) / |1 + c a|. Undefined for c = -a, where 1 + c a = 0.
inline Rotor rotorBetween(const Vec3& a, const Vec3& c) {
    const Rotor ca = product(c, a);
    const double w = 1.0 + ca.w;
    const double norm = std::sqrt(w * w + ca.b.squaredNorm()); // |1 + c a| = sqrt(2 (1 + a.c))
    return { w / norm, ca.b / norm };
}

/// The complex number of the plane with normal n given by the product of two unit vectors of that plane:
/// t c = t . c + t ^ c = cos(rho) + T sin(rho), with t ^ c = (t x c . n) T (eq. ga_connection).
inline Complex planeProduct(const Vec3& t, const Vec3& c, const Vec3& n) {
    return { t.dot(c), t.cross(c).dot(n) };
}

/// The discrete connection from vertex i to face f, as a unit complex number (eq. ga_connection):
/// e^{T_f rho_i^f} = t_f (Q t_i ~Q), Q = rotorBetween(n_i, n_f).
/// Transporting the spinor coordinate psi of vertex i into the face is multiplication by it (eq. ga_transport).
inline Complex connection(const Vec3& t_i, const Vec3& n_i, const Vec3& t_f, const Vec3& n_f) {
    const Rotor Q = rotorBetween(n_i, n_f);
    return planeProduct(t_f, Q.rotate(t_i), n_f);
}
```

- Line 65 forms the product $c\,a = c\cdot a + I(c\times a)$, the scalar `ca.w` and the dual vector `ca.b`.
- Line 66 adds 1 to the scalar part: $1 + c\,a$ has the scalar part $1 + c\cdot a$ and the same bivector part.
- Line 67 computes the norm $|1+ca|$ from its two parts, as the square root of $(1 + c\cdot a)^2 + |c\times a|^2$. This is the second denominator of (22.2) [$Q = \frac{1+ca}{\sqrt{2(1+a\cdot c)}}$], because for unit vectors $|c\times a|^2 = 1 - (c\cdot a)^2$, and

$$(1 + c\cdot a)^2 + 1 - (c\cdot a)^2 = 1 + 2\,c\cdot a + (c\cdot a)^2 + 1 - (c\cdot a)^2 = 2\,(1 + c\cdot a).$$

- Line 68 divides both parts by it: the rotor Q.
- Lines 73–75: the product of two unit vectors of the plane with normal n, as a complex number. The real part is $t\cdot c = \cos\rho$. The bivector part is $t\wedge c = I(t\times c)$, and $t\times c$ is parallel to n (both vectors lie in the plane), so $t\times c = \big((t\times c)\cdot n\big)\,n$ and $t\wedge c = \big((t\times c)\cdot n\big)\,In = \big((t\times c)\cdot n\big)\,\mathsf{T}$. Its coefficient, line 74, is $\sin\rho$, with the sign of the turn from t to c about n.
- Lines 80–83: the connection (22.4), in the order of the formula: the rotor from n_i to n_f (line 81), the rotated frame vector $Qt_i\widetilde{Q}$ (`Q.rotate(t_i)`), and its product with t_f (line 82).

The matrix version of the same computation, in the `matrixform` namespace of `ga_fem.cpp`, is `alignVectors` and `Rvf` of `FEM.cpp`:

```
79  /// alignVectors: the Rodrigues matrix of the smallest rotation taking nv to nf (eq.
        rodrigues)
80  static Matrix3d alignVectors(const Vector3d& nv, const Vector3d& nf) {
81      const double c = nv.dot(nf);
82      const Vector3d v = nv.cross(nf);
83      Matrix3d skew;
84      skew << 0, -v(2), v(1), v(2), 0, -v(0), -v(1), v(0), 0;
85      return Matrix3d::Identity() + skew + skew * skew / (1.0 + c);
86  }
87
88  /// Rvf: R_i^f = T_f^T Q T_i (eq. connection)
89  static Matrix2d Rvf(const Geometry& g, const Vertex* v, const Face& f) {
90      return g.faceFrame[f.ID].transpose() * alignVectors(v->n, g.faceNormal[f.ID]) * g.
            vertexFrame[v->ID];
91  }
```

Line 85 is the Rodrigues matrix (4.6) and line 90 is $R_i^f = T_f^T Q T_i$. The two versions give the same numbers, not only the same mathematics. The first column of `Rvf` is $T_f^T Q t_i = (t_f\cdot c,\ b_f\cdot c)$. Its first entry is the real part of `connection`; its second entry is

$$b_f\cdot c = (n_f\times t_f)\cdot c = n_f\cdot(t_f\times c) = (t_f\times c)\cdot n_f,$$

by the cyclic symmetry of the triple product, which is the imaginary part. The second column of `Rvf` carries no new information: it is the first one turned by a quarter turn, as (22.6) shows.

Worked example 22.3. *For the vertex and the face of Example 4.4, $n_i = e_3$, $t_i = e_2$, $n_f = (0,-1,1)/\sqrt{2}$ and $t_f = e_1$. The product of the normals is*

$$n_f n_i = n_f \cdot n_i + n_f \wedge n_i = \tfrac{1}{\sqrt{2}} + I\,(n_f \times n_i),$$
$$n_f \times n_i = \tfrac{1}{\sqrt{2}}(0,-1,1) \times (0,0,1) = \big(-\tfrac{1}{\sqrt{2}}, 0, 0\big).$$

So $1 + n_f n_i = (1 + \frac{1}{\sqrt{2}}) - \frac{1}{\sqrt{2}}\, Ie_1$, whose norm is $\sqrt{(1+\frac{1}{\sqrt{2}})^2 + \frac{1}{2}} = \sqrt{2+\sqrt{2}} = \sqrt{2(1+1/\sqrt{2})} = 1.8478$. Dividing,

$$Q = 0.9239 - 0.3827\, Ie_1 = \cos 22.5^\circ - Ie_1 \sin 22.5^\circ = e^{-Ie_1\pi/8},$$

the rotor of the rotation by 45° in the plane $B = Ie_1 = e_2e_3$, about the x axis, as found with matrices. We rotate $t_i = e_2$ factor by factor, with $C = \cos\frac{\pi}{8}$, $S = \sin\frac{\pi}{8}$, $Q = C - BS$, $\widetilde{Q} = C + BS$, and the products $Be_2 = e_2e_3e_2 = -e_2e_2e_3 = -e_3$ and $e_2B = e_2e_2e_3 = e_3$, $e_3B = e_3e_2e_3 = -e_2e_3e_3 = -e_2$:

$$\begin{aligned} Q\,e_2 &= (C - BS)\,e_2 = C\,e_2 - S\,Be_2 = C\,e_2 + S\,e_3, \\ (Qe_2)\,\widetilde{Q} &= (C\,e_2 + S\,e_3)(C + BS) = C^2 e_2 + CS\,e_2B + SC\,e_3 + S^2\,e_3B \\ &= C^2 e_2 + CS\,e_3 + CS\,e_3 - S^2 e_2 = (C^2 - S^2)\,e_2 + 2CS\,e_3 \\ &= \cos\tfrac{\pi}{4}\,e_2 + \sin\tfrac{\pi}{4}\,e_3 = \tfrac{1}{\sqrt{2}}\,(e_2 + e_3). \end{aligned}$$

In the code's representation, $w = 0.9239$ and $b = (-0.3827, 0, 0)$; the quaternion vector is $u = -b = (0.3827, 0, 0)$, and `rotate` *gives $t = 2\,u \times e_2 = 0.7654\,e_3$ and $e_2 + 0.9239\,t + u \times t = e_2 + 0.7071\,e_3 - 0.2929\,e_2 = 0.7071\,(e_2 + e_3)$, the same vector. So $c = Qe_2\widetilde{Q} = (e_2 + e_3)/\sqrt{2}$, and the connection is the geometric product*

$$t_f c = e_1\,\tfrac{1}{\sqrt{2}}(e_2 + e_3) = \tfrac{1}{\sqrt{2}}\,(e_1e_2 + e_1e_3),$$

a unit bivector with no scalar part ($e_1 \cdot e_2 = e_1 \cdot e_3 = 0$). The face plane is, with $Ie_2 = e_3e_1$ and $Ie_3 = e_1e_2$,

$$\mathsf{T}_f = In_f = \tfrac{1}{\sqrt{2}}\,(-Ie_2 + Ie_3) = \tfrac{1}{\sqrt{2}}\,(-e_3e_1 + e_1e_2) = \tfrac{1}{\sqrt{2}}\,(e_1e_3 + e_1e_2),$$

using $-e_3e_1 = e_1e_3$. So $t_f c = \mathsf{T}_f = 0 + 1 \cdot \mathsf{T}_f = e^{\mathsf{T}_f \pi/2}$: $\rho_i^f = 90^\circ$, the same number as in Example 4.4, now read off a single product. The program computes this example in `workedExamples`*:*

```
    printf("== Worked example 2: transport from a vertex plane into the face plane\n");
    const Vector3d ni(0, 0, 1), ti(0, 1, 0), tf = (x2 - x1).normalized();
    const ga::Rotor Q = ga::rotorBetween(ni, n);
    const Vector3d c = Q.rotate(ti);
    const Complex rho = ga::connection(ti, ni, tf, n);
    printf("  n_f = (%g, %g, %g),  Q = %.6f + %.6f I (%.6f, %.6f, %.6f)\n", n.x(), n.y(), n.z
          (), Q.w, Q.b.x(), Q.b.y(), Q.b.z());
    printf("  Q t_i ~Q = (%.6f, %.6f, %.6f),  e^{T rho} = %.6f + %.6f T,  rho = %.1f
          degrees\n", c.x(), c.y(), c.z(),
           rho.real(), rho.imag(), std::arg(rho) * 180.0 / M_PI);
    Matrix32d Ti, Tf;
    Ti.col(0) = ti; Ti.col(1) = ni.cross(ti);
    Tf.col(0) = tf; Tf.col(1) = n.cross(tf);
    const Matrix2d R = Tf.transpose() * matrixform::alignVectors(ni, n) * Ti;
```

```
413     printf("␣␣R_i^f␣=␣T_f^T␣Q␣T_i␣=␣[%.6f␣%.6f;␣%.6f␣%.6f]\n", R(0, 0), R(0, 1), R(1,
            0), R(1, 1));
```

Line 402 sets n_i, t_i and t_f (the normal n of the face was computed earlier in the function, from the triangle of Example 12.2). Lines 403–405 evaluate (22.3), the rotated frame vector and (22.4); lines 409–412 build the two frames and the matrix $T_f^T Q T_i$ of Part I. The program prints

```
n_f = (0, -0.707107, 0.707107), Q = 0.923880 + I (-0.382683, 0.000000, 0.000000)
Q t_i ~Q = (0.000000, 0.707107, 0.707107), e^{T rho} = 0.000000 + 1.000000 T, rho =
     90.0 degrees
R_i^f = T_f^T Q T_i = [0.000000 -1.000000; 1.000000 0.000000]
```

and the first column of the matrix, $(0, 1)$, is the complex number $0 + 1\,\mathsf{T}$, as (22.6) with $\rho = 90°$ [first column $(\cos\rho, \sin\rho)$] says.

22.5. Intrinsic and extrinsic geometry of the discrete transport

What an inhabitant of the surface can measure. As in Stage 1, a quantity is *intrinsic* if a two-dimensional being living on the surface can compute it from lengths and angles measured on the surface, and *extrinsic* if it depends on how the surface sits in space (n, T as an element of $\mathbb{G}_3$, II). Intrinsic quantities do not change when the surface is bent without stretching, as a sheet of paper rolled into a cylinder. Gauss's *Theorema Egregium* says that the Gaussian curvature K, defined extrinsically as the product of the two principal curvatures, is intrinsic. The PDE of Stage 1 is intrinsic; the computation uses extrinsic tools (normals, rotors of $\mathbb{R}^3$, projections). On a triangle mesh the intrinsic geometry is that of the flat faces glued along their edges: it is given by the edge lengths. We ask which parts of the discrete transport a being on the mesh could reproduce.

Across an edge, the transport is intrinsic. Take two faces f and g sharing an edge with direction e. Both normals are orthogonal to the shared edge, so e anticommutes with n_f and with n_g, and it commutes with their product:

$$(n_f n_g)\, e = n_f\,(n_g\, e) = -n_f\,(e\, n_g) = -(n_f\, e)\, n_g = e\, n_f\, n_g.$$

So e commutes with $Q_{g\to f} = (1 + n_f n_g)/|1 + n_f n_g|$, and $Q_{g\to f}\, e\, \widetilde{Q}_{g\to f} = e\, Q_{g\to f} \widetilde{Q}_{g\to f} = e$: the rotor turns about the shared edge. It takes the plane of g onto the plane of f, keeping the shared edge in place, which is exactly the *unfolding* of one face onto the plane of the other (Figure 5). The unfolded pair of triangles is determined by their edge lengths alone, not by the angle at which they meet in space, so the transport across an edge is intrinsic. It is the parallel transport of the piecewise flat surface: a vector carried across the edge without turning, as seen by the being, which sees the two triangles as one flat piece.

Through a vertex, it is not. A vertex has no tangent plane: it is the apex of a cone whose total angle $\Theta_i = \sum_f \alpha_{i,f}$, the sum of the angles of the incident faces at the vertex, is in general not 2π. The code gives the vertex a plane anyway, the normalized sum of the face bivectors, and transports between that plane and the faces with the rolling rotor $Q_{i\to f}$. This is an *extrinsic* choice. Bend the mesh at the vertex without stretching any triangle, by changing the

angles between the faces: no length changes, but the sum of the face bivectors changes, and with it the rotors $Q_{i\to f}$ and the connection angles. The error it introduces vanishes as the mesh is refined on a smooth surface, because the vertex normal then converges to the normal of the smooth surface and the rolling rotors to its parallel transport. Part III replaces this choice by an intrinsic one, the angles around the vertex rescaled by $2\pi/\Theta_i$; in the code it replaces only the computation of the connection angle ρ_i^f (`Rvf`, or `connection` here), leaving the finite element machinery untouched.

Holonomy. Carry a vector parallel around a small closed loop and it comes back rotated, in the tangent plane, by an angle: the *holonomy* of the loop. On a plane the angle is zero; on the unit sphere, Example 4.6 found that the loop around an octant turns the vector by 90°, which is the area $\pi/2$ of the octant.

Rolling sees only the normals. The rolling picture explains where the angle comes from. The rotor equation $R' = \frac{1}{2}\,(n' \wedge n)\,R$ involves only the normal $n(s)$ and its derivative, not the curve γ itself. Draw the normal $n(s)$ as a point of the unit sphere. As we go around the loop on the surface, that point traces a closed curve on the unit sphere. The tangent plane of the unit sphere at the point $n(s)$ is the plane orthogonal to $n(s)$, the same plane as the tangent plane of the surface at $\gamma(s)$, and the normal of the unit sphere at the point $n(s)$ is $n(s)$ itself; so the rolling equation of the unit sphere along the curve $n(s)$ is the same equation, $R' = \frac{1}{2}\,(n' \wedge n)\,R$. Parallel transport around the loop on the surface is therefore parallel transport around the curve traced by the normal on the unit sphere, and the holonomy is the holonomy of that curve on the sphere: the area it encloses, as in the octant example (up to a sign that depends on the sense of the loop).

Curvature. The area enclosed by the normals on the unit sphere, divided by the area of the region they come from on the surface, is Gauss's original definition of the curvature K. So, for a small loop, the holonomy is $\int K\,dA$ over the region the loop encloses: holonomy measures curvature, and since parallel transport is intrinsic, so is the curvature, which is the Theorema Egregium seen from the side of transport. Along a small loop on a cylinder the normals move back and forth along the equator of the unit sphere and enclose no area, and indeed $K = 0$ there.

On a mesh. The intrinsic transport around a vertex, unfolding the faces one after another, closes with the rotation $2\pi - \Theta_i$, the *angle defect*, the discrete total curvature of the vertex (Example 4.6, the pyramid). The code's transport, which rolls through the vertex normals, closes with the area of the spherical polygon whose corners are the normals visited by the loop, those of the neighboring vertices and faces. That area approximates the angle defect on fine meshes, and the remark of Section 4 should be read in this sense. The same spherical areas decide, in the next subsection, how close the discrete space comes to being conforming.

22.6. Spinor hat functions

The scalar hats. On each face the hat function ϕ_i of a vertex i is the linear function that is 1 at x_i and 0 at the other two vertices; it is zero on the faces that do not contain i. The hats are continuous across edges, because on an edge both faces' hats are the linear function along the edge with the same end values, and they add up to 1 on every face.

The basis. The unknown at vertex i is a tangent vector $t_i\psi_i$ of the vertex plane. To spread it over the faces around i, carry it into each face with the rotor $Q_{i\to f}$ and fade it out with the hat. By (22.5),

$$Q_{i\to f}\, t_i\, \psi_i\, \widetilde{Q}_{i\to f} = t_f\, e^{\mathsf{T}_f \rho_i^f}\, \psi_i, \tag{22.5}$$

with ψ_i now read in the face plane. Multiplying by the real function ϕ_i, which commutes with everything, we define, for a complex coefficient α, the *spinor hat function* of vertex i,

$$\Phi_i[\alpha]\big|_f = t_f\ \phi_i\, e^{\mathsf{T}_f \rho_i^f}\, \alpha \quad (f \ni i), \qquad \Phi_i[\alpha]\big|_f = 0 \quad (f \not\ni i), \tag{22.7}$$

whose spinor coordinate in the face frame is $\phi_i\, e^{\mathsf{T}_f \rho_i^f}\alpha$. It is real-linear in α, and each vertex contributes two real basis fields, $\Phi_i[1]$ and $\Phi_i[\mathsf{T}]$: the frame vector t_i and its quarter turn $t_i\mathsf{T}_i$, carried into the faces and faded out. They are the basis fields of Part I, (9.2) [$\Psi_{i,k}|_f = \phi_i\, T_f R_i^f \varepsilon_k$]. To see it, write $\rho = \rho_i^f$, $b_f = t_f \mathsf{T}_f$, and multiply out:

$$\begin{aligned}
\Phi_i[1]\big|_f &= \phi_i\, t_f\, (\cos\rho + \mathsf{T}_f \sin\rho) = \phi_i\, \big(\cos\rho\, t_f + \sin\rho\, b_f\big) \\
&= \phi_i\, T_f \begin{pmatrix}\cos\rho\\ \sin\rho\end{pmatrix} = \phi_i\, T_f R_i^f \varepsilon_1, \\
\Phi_i[\mathsf{T}]\big|_f &= \phi_i\, t_f\, (\cos\rho\, \mathsf{T}_f + \sin\rho\, \mathsf{T}_f^2) = \phi_i\, \big(-\sin\rho\, t_f + \cos\rho\, b_f\big) \\
&= \phi_i\, T_f \begin{pmatrix}-\sin\rho\\ \cos\rho\end{pmatrix} = \phi_i\, T_f R_i^f \varepsilon_2,
\end{aligned}$$

using $T_f(\alpha,\beta)^T = \alpha\, t_f + \beta\, b_f$ and the columns of (22.6). The discrete space V_h is the set of all sums $\sum_i \Phi_i[\psi_i]$; it has n complex, or $2n$ real, dimensions, with n the number of vertices.

The trial field. On a face, only the three vertices of the face contribute, since $\Phi_i|_f = 0$ for $i \notin f$. Substituting (22.7) into the sum,

$$\begin{aligned}
U\big|_f = \sum_{i\in f} \Phi_i[\psi_i]\big|_f &= \sum_{i\in f} \Big(t_f\, \phi_i\, e^{\mathsf{T}_f \rho_i^f}\, \psi_i\Big) && \text{(22.7), } \alpha = \psi_i \\
&= t_f \Big(\sum_{i\in f} \phi_i\, e^{\mathsf{T}_f \rho_i^f}\, \psi_i\Big) && t_f \text{ factored out,}
\end{aligned}$$

so a field of V_h is

$$U\big|_f = t_f\, \psi_f, \qquad \psi_f = \sum_{i\in f} \phi_i\, e^{\mathsf{T}_f \rho_i^f}\, \psi_i : \tag{22.8}$$

each vertex coordinate is turned by its connection angle into the face, and the three are interpolated with the hats. It is the trial field of

Part I, (9.1) $[\,T_f^T U|_f = \sum_{i\in f} \phi_i\, R_i^f u_i\,]$, with the 2×2 rotation R_i^f replaced by the number $e^{\mathsf{T}_f \rho_i^f}$, by (22.6). Every value is a vector of the plane of its face, so the field is tangent to every face. The test spinors are built in the same way, with coordinates χ_i at the vertices.

Is the space conforming? Stage 2 asked the discrete fields to have no jump across edges, so that $V_h \subset H^1$. Take an edge between the faces f and g, with a vertex i at one of its ends. Along the edge both faces' hats agree, so the only possible jump comes from the transports. Seen from f, the contribution of vertex i is ϕ_i times the vector $Q_{i\to f}(t_i\psi_i)\widetilde{Q}_{i\to f}$; seen from g, it is ϕ_i times $Q_{i\to g}(t_i\psi_i)\widetilde{Q}_{i\to g}$, a vector of the other plane. To compare them, unfold g onto f with the intrinsic transport across the edge, the rotor $Q_{g\to f}$. Both $Q_{g\to f}Q_{i\to g}$ and $Q_{i\to f}$ take n_i to n_f:

$$\begin{aligned}(Q_{g\to f}Q_{i\to g})\, n_i\, \widetilde{(Q_{g\to f}Q_{i\to g})} &= Q_{g\to f}\left(Q_{i\to g}\, n_i\, \widetilde{Q}_{i\to g}\right)\widetilde{Q}_{g\to f}\\ &= Q_{g\to f}\, n_g\, \widetilde{Q}_{g\to f} = n_f,\end{aligned}$$

so they differ only by a rotation that fixes n_f, a rotation within the plane of f. That rotation is the holonomy of the three steps $n_i \to n_g \to n_f \to n_i$, and by the rolling picture its angle is the area of the spherical triangle with corners n_i, n_g, n_f. So the discrete field jumps across each edge by a small rotation. Neighboring normals differ by angles of the order of κh, with κ the curvature and h the edge length, so the area, and the jump, is of the order of $(\kappa h)^2$, and it vanishes on flat meshes and as the mesh is refined. The space is *nearly* conforming, not exactly. The method nevertheless computes all its integrals face by face, as if there were no jumps; in the language of finite elements this is a small *variational crime*, whose effect is controlled by Strang's lemma, Céa's lemma with one more term that measures the inconsistency.

The covariant derivative of the trial field. The face is flat and its frame vector t_f is constant on it, so the frame does not turn: $\nabla_a t_f = 0$, and by (18.8) $[\,\nabla_a t = \omega(a)\, t\mathsf{T}\,]\ \omega = 0$. Then (18.9),

$$D_a = a\cdot\nabla + \omega(a)\,\mathsf{T}, \tag{18.9}$$

reduces to $D_a = a\cdot\nabla$, the plain derivative of the coefficients. Apply it to ψ_f of (22.8):

$$\begin{aligned}D_a\psi_f &= a\cdot\nabla\Big(\sum_{i\in f}\phi_i\, e^{\mathsf{T}_f\rho_i^f}\,\psi_i\Big) && \omega=0,\ \psi_f \text{ substituted}\\ &= \sum_{i\in f} a\cdot\nabla\big(\phi_i\, e^{\mathsf{T}_f\rho_i^f}\,\psi_i\big) && \text{the derivative is linear}\\ &= \sum_{i\in f}(a\cdot\nabla\phi_i)\, e^{\mathsf{T}_f\rho_i^f}\,\psi_i && e^{\mathsf{T}_f\rho_i^f}\psi_i \text{ is constant on the face,}\end{aligned}$$

that is,

$$D_a\psi_f = \sum_{i\in f}(a\cdot\nabla\phi_i)\, e^{\mathsf{T}_f\rho_i^f}\,\psi_i. \tag{22.9}$$

Inside a face the covariant derivative of the discrete field is just the derivative of the hats, times constant complex numbers. All the geometry of the curved surface has been moved into the connection angles ρ_i^f, computed once per corner of each face.

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

23. Stage 6: The Discrete Solution and the Test Functions

What we want. We replace the infinite-dimensional problem of Stage 4 by a finite one: look for the solution in V_h, and ask the weak form to hold only for the test fields of V_h. This is the *Galerkin method*: the test space is the trial space. We want to see, by direct substitution, that this gives one complex linear equation per vertex.

The discrete solution and the tests. Write the discrete solution as a combination of the spinor hat functions (22.7),

$$\psi_h = \sum_{j=1}^{n} \Phi_j[\psi_j],$$

with the n unknown complex numbers ψ_j, the coordinates of the vertices. As test fields take the basis fields themselves, $\chi_h = \Phi_i[\chi_i]$, for every vertex $i \neq s$ and for the two real coefficients $\chi_i = 1$ and $\chi_i = \mathsf{T}$. That is $2(n-1)$ real tests for $2(n-1)$ real unknowns, the coordinates of the vertices other than the source.

The fields on one face. The weak form (21.5) again:

$$\int_\Omega \langle \psi\, \widetilde{\chi} \rangle\, dA + \Delta t \int_\Omega \sum_k \langle (D_{e_k}\psi)\, \widetilde{(D_{e_k}\chi)} \rangle\, dA = \int_\Omega \langle \psi^0\, \widetilde{\chi} \rangle\, dA. \tag{21.5}$$

The integrals over Ω are sums of integrals over the faces. On a face f that contains i, the trial field (22.8) and the test field have the spinor coordinates

$$\psi_f = \sum_{j \in f} \phi_j\, e^{\mathsf{T}_f \rho_j^f}\, \psi_j, \qquad \chi_f = \phi_i\, e^{\mathsf{T}_f \rho_i^f}\, \chi_i, \tag{22.8}$$

and on the faces that do not contain i the test field is zero, so they contribute nothing. By (22.9), their derivatives along a unit direction e_k of the face (here e_1, e_2 is an orthonormal basis of the face plane, not the edge frame) are

$$D_{e_k}\psi_f = \sum_{j \in f} (e_k \cdot \nabla\phi_j)\, e^{\mathsf{T}_f \rho_j^f}\, \psi_j, \qquad D_{e_k}\chi_f = (e_k \cdot \nabla\phi_i)\, e^{\mathsf{T}_f \rho_i^f}\, \chi_i. \tag{22.9}$$

The reverses of the test terms. The weak form needs $\widetilde{\chi_f}$ and $\widetilde{D_{e_k}\chi_f}$. Three rules give them: the reverse of a product reverses the order of its factors, $\widetilde{XY} = \widetilde{Y}\,\widetilde{X}$; a real number is its own reverse; and the reverse of $e^{\mathsf{T}\rho} = \cos\rho + \mathsf{T}\sin\rho$ is $\cos\rho - \mathsf{T}\sin\rho = e^{-\mathsf{T}\rho}$, because $\widetilde{\mathsf{T}} = -\mathsf{T}$. So

$$\widetilde{\chi_f} = \big(\phi_i\,\widetilde{e^{\mathsf{T}_f\rho_i^f}}\,\chi_i\big) = \widetilde{\chi_i}\,\big(\widetilde{e^{\mathsf{T}_f\rho_i^f}}\big)\,\phi_i = \phi_i\,\widetilde{\chi_i}\,e^{-\mathsf{T}_f\rho_i^f},$$
$$\widetilde{D_{e_k}\chi_f} = \big((e_k\cdot\nabla\phi_i)\,\widetilde{e^{\mathsf{T}_f\rho_i^f}}\,\chi_i\big) = (e_k\cdot\nabla\phi_i)\,\widetilde{\chi_i}\,e^{-\mathsf{T}_f\rho_i^f},$$

where in the last step of each line the real number was moved to the front.

Two abbreviations. The lines below are long, so we write, for the corners a of the face,

$$z_a = e^{\mathsf{T}_f\rho_a^f}, \qquad \widetilde{z_a} = e^{-\mathsf{T}_f\rho_a^f}, \qquad \partial_k\phi_a = e_k\cdot\nabla\phi_a.$$

Then $\psi_f = \sum_{j\in f}\phi_j\,z_j\,\psi_j$, $\widetilde{\chi_f} = \phi_i\,\widetilde{\chi_i}\,\widetilde{z_i}$, $D_{e_k}\psi_f = \sum_{j\in f}\partial_k\phi_j\;z_j\,\psi_j$ and $\widetilde{D_{e_k}\chi_f} = \partial_k\phi_i\;\widetilde{\chi_i}\,\widetilde{z_i}$. We also need the product of two exponentials of the plane, computed above: $\widetilde{z_i}\,z_j = e^{-\mathsf{T}_f\rho_i^f}e^{\mathsf{T}_f\rho_j^f} = e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}$.

The mass integrand. Substitute ψ_f and $\widetilde{\chi_f}$ into $\langle\psi_f\,\widetilde{\chi_f}\rangle$:

$$\begin{aligned}
\langle\psi_f\,\widetilde{\chi_f}\rangle &= \Big\langle\Big(\sum_{j\in f}\phi_j\,z_j\,\psi_j\Big)\Big(\phi_i\,\widetilde{\chi_i}\,\widetilde{z_i}\Big)\Big\rangle && \text{(a)}\\
&= \sum_{j\in f}\big\langle\phi_j\,z_j\,\psi_j\;\phi_i\,\widetilde{\chi_i}\,\widetilde{z_i}\big\rangle && \text{(b)}\\
&= \sum_{j\in f}\phi_i\,\phi_j\,\big\langle z_j\,\psi_j\,\widetilde{\chi_i}\,\widetilde{z_i}\big\rangle && \text{(c)}\\
&= \sum_{j\in f}\phi_i\,\phi_j\,\big\langle\widetilde{z_i}\,z_j\,\psi_j\,\widetilde{\chi_i}\big\rangle && \text{(d)}\\
&= \sum_{j\in f}\phi_i\,\phi_j\,\big\langle e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}\,\psi_j\,\widetilde{\chi_i}\big\rangle && \text{(e)}
\end{aligned}$$

(a) Both fields substituted. (b) The product distributes over the sum, and the scalar part is linear. (c) The real hats commute with everything and move out of the product and of the scalar part. (d) The cyclic rule $\langle XY\rangle = \langle YX\rangle$ of Section 17, with $X = z_j\psi_j\widetilde{\chi_i}$ and $Y = \widetilde{z_i}$; one could also use that all four factors are complex numbers of the plane T_f, which commute. (e) The product $\widetilde{z_i}\,z_j = e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}$.

The stiffness integrand. Substitute $D_{e_k}\psi_f$ and $\widetilde{D_{e_k}\chi_f}$, and sum over k:

$$\begin{aligned}
\sum_k \big\langle (D_{e_k}\psi_f)\,(\widetilde{D_{e_k}\chi_f})\big\rangle &= \sum_k \Big\langle \Big(\sum_{j\in f} \partial_k\phi_j\; z_j\,\psi_j\Big)\Big(\partial_k\phi_i\;\widetilde{\chi}_i\,\widetilde{z}_i\Big)\Big\rangle && \text{(a)}\\
&= \sum_k \sum_{j\in f} \partial_k\phi_i\;\partial_k\phi_j\,\big\langle z_j\,\psi_j\,\widetilde{\chi}_i\,\widetilde{z}_i\big\rangle && \text{(b)}\\
&= \sum_{j\in f} \Big(\sum_k \partial_k\phi_i\;\partial_k\phi_j\Big)\,\big\langle z_j\,\psi_j\,\widetilde{\chi}_i\,\widetilde{z}_i\big\rangle && \text{(c)}\\
&= \sum_{j\in f} (\nabla\phi_i\cdot\nabla\phi_j)\,\big\langle z_j\,\psi_j\,\widetilde{\chi}_i\,\widetilde{z}_i\big\rangle && \text{(d)}\\
&= \sum_{j\in f} (\nabla\phi_i\cdot\nabla\phi_j)\,\big\langle e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}\,\psi_j\,\widetilde{\chi}_i\big\rangle && \text{(e)}
\end{aligned}$$

(a) Both derivatives substituted. (b) Distributing, and moving the real numbers $\partial_k\phi_i$, $\partial_k\phi_j$ out of the product and of the scalar part. (c) Swapping the two finite sums; the scalar part does not depend on k. (d) The inner product in components: with $\nabla\phi_i = \sum_k (\partial_k\phi_i)\,e_k$, likewise for ϕ_j, and $e_k\cdot e_l = \delta_{kl}$,

$$\nabla\phi_i\cdot\nabla\phi_j = \sum_k\sum_l \partial_k\phi_i\;\partial_l\phi_j\;(e_k\cdot e_l) = \sum_k \partial_k\phi_i\;\partial_k\phi_j.$$

(e) As in steps (d) and (e) of the mass integrand: the cyclic rule moves $\widetilde{z}_i$ to the front, and $\widetilde{z}_i\,z_j = e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}$.

The right side. The datum ψ^0 is also written as a field of V_h, $\psi_h^0 = \sum_j \Phi_j[\psi_j^0]$, so that on the face $\psi_f^0 = \sum_{j\in f}\phi_j\,e^{\mathsf{T}_f\rho_j^f}\psi_j^0$. Its integrand is the mass integrand with ψ_j^0 in the place of ψ_j; the five lines (a)–(e) of the mass integrand, with that one replacement, give $\langle\psi_f^0\,\widetilde{\chi_f}\rangle = \sum_{j\in f}\phi_i\phi_j\,\langle e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}\psi_j^0\,\widetilde{\chi}_i\rangle$. In the code $\psi^0 = 0$, and the source enters only through the prescribed value.

Collecting. Put the three integrands into (21.5), integrated over the faces $f \ni i$. The left side is

$$\begin{aligned}
&\sum_{f\ni i}\int_f \sum_{j\in f}\phi_i\phi_j\,\big\langle e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}\psi_j\widetilde{\chi}_i\big\rangle\,dA\\
&\quad+\Delta t\sum_{f\ni i}\int_f\sum_{j\in f}(\nabla\phi_i\cdot\nabla\phi_j)\,\big\langle e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}\psi_j\widetilde{\chi}_i\big\rangle\,dA.
\end{aligned}$$

Three operations bring it to its final form. (1) Swap the sum over the faces and the integral with the sum over j: a pair (f,j) with $f\ni i$ and $j\in f$ is the same as a vertex j and a face $f\ni i,j$, so $\sum_{f\ni i}\sum_{j\in f} = \sum_j\sum_{f\ni i,j}$. (2) The number $e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}\psi_j\widetilde{\chi}_i$ is constant on the face, so the integral acts only on the hats, and it moves inside the scalar part: $\int_f g\,\langle z\rangle\,dA = \big\langle\big(\int_f g\,dA\big)\,z\big\rangle$ for a real function g and a constant z. (3) The scalar part is linear, so the sum over faces moves inside it, and the common right factor $\widetilde{\chi}_i$ moves out of the

sum. The left side becomes

$$\Big\langle \sum_j \Big(\sum_{f \ni i,j} \Big(\int_f \phi_i \phi_j \, dA + \Delta t \int_f \nabla\phi_i \cdot \nabla\phi_j \, dA \Big) e^{\mathsf{T}_f(\rho_j^f - \rho_i^f)} \Big) \psi_j \, \widetilde{\chi}_i \Big\rangle,$$

and the right side becomes $\big\langle \sum_j \big(\sum_{f \ni i,j} \int_f \phi_i \phi_j \, dA \;\; e^{\mathsf{T}_f(\rho_j^f - \rho_i^f)} \big) \psi_j^0 \;\; \widetilde{\chi}_i \big\rangle$. Name the face sums: one complex number for each pair of vertices,

$$\begin{aligned} \mathrm{M}_{ij} &= \sum_{f \ni i,j} \Big(\int_f \phi_i \, \phi_j \, dA \Big) e^{\mathsf{T}_f(\rho_j^f - \rho_i^f)}, \\ \mathrm{L}_{ij} &= \sum_{f \ni i,j} \Big(\int_f \nabla\phi_i \cdot \nabla\phi_j \, dA \Big) e^{\mathsf{T}_f(\rho_j^f - \rho_i^f)}, \end{aligned} \tag{23.1}$$

the sums running over the faces that contain both vertices (for $i = j$, all the faces around i, where the transport is $e^{\mathsf{T}_f \cdot 0} = 1$). Subtracting the right side from the left, the tested equation of vertex i is

$$\Big\langle \Big(\sum_j \big(\mathrm{M}_{ij} + \Delta t \, \mathrm{L}_{ij} \big) \psi_j - \sum_j \mathrm{M}_{ij} \, \psi_j^0 \Big) \widetilde{\chi}_i \Big\rangle = 0. \tag{23.2}$$

Two real tests make one complex equation. Let z be the complex number in the large parentheses of (23.2), $z = \alpha + \beta\mathsf{T}$. With $\chi_i = 1$, $\widetilde{\chi}_i = 1$ and the test gives $\langle z \rangle = \alpha = 0$. With $\chi_i = \mathsf{T}$, $\widetilde{\chi}_i = -\mathsf{T}$ and

$$z\,(-\mathsf{T}) = (\alpha + \beta\mathsf{T})(-\mathsf{T}) = -\alpha\mathsf{T} - \beta\mathsf{T}^2 = \beta - \alpha\mathsf{T},$$

so the test gives $\langle z\,(-\mathsf{T}) \rangle = \beta = 0$. The two real test functions of vertex i ask for the real and the imaginary parts of the same complex number to vanish, that is, $z = 0$:

$$\sum_{j=1}^{n} \big(\mathrm{M}_{ij} + \Delta t \, \mathrm{L}_{ij} \big) \psi_j = \sum_{j=1}^{n} \mathrm{M}_{ij} \, \psi_j^0 \quad (i \neq s), \qquad \psi_s \text{ prescribed.} \tag{23.3}$$

These are the *Galerkin equations*: one complex equation per vertex, a square linear system for the complex coordinates. Its coefficients (23.1) are sums of integrals over single faces, which Stage 7 computes.

Galerkin orthogonality and the best approximation. Pretend for a moment that V_h is exactly conforming, $V_h \subset H^1$. Write the weak form as $a(\psi, \chi) = l(\chi)$ with the forms (19.3),

$$\begin{aligned} a(\psi, \chi) &= \int_\Omega \langle \psi \widetilde{\chi} \rangle \, dA + \Delta t \int_\Omega \sum_k \big\langle (D_{e_k}\psi) \, \widetilde{(D_{e_k}\chi)} \big\rangle \, dA, \\ l(\chi) &= \int_\Omega \langle \psi^0 \widetilde{\chi} \rangle \, dA. \end{aligned} \tag{19.3}$$

The exact weak solution satisfies $a(\psi, \chi_h) = l(\chi_h)$ for every $\chi_h \in V_h$, since those are among all the tests of H^1; the discrete solution satisfies $a(\psi_h, \chi_h) = l(\chi_h)$ by construction. Subtracting, and using the linearity of a in its first argument,

$$a(\psi - \psi_h, \chi_h) = a(\psi, \chi_h) - a(\psi_h, \chi_h) = l(\chi_h) - l(\chi_h) = 0$$

for every $\chi_h \in V_h$: the error is orthogonal to the discrete space in the inner product a. Now take any $\chi_h \in V_h$ and split $\psi - \chi_h = x + y$ with $x = \psi - \psi_h$ and $y = \psi_h - \chi_h \in V_h$. With the energy norm $\|x\|_a^2 = a(x,x)$ and the symmetry of a,

$$\begin{aligned}
\|\psi - \chi_h\|_a^2 &= a(x+y, x+y) \\
&= a(x,x) + 2\,a(x,y) + a(y,y) && \text{bilinear, symmetric} \\
&= \|\psi - \psi_h\|_a^2 + 0 + \|\psi_h - \chi_h\|_a^2 && a(x,y) = 0,\ y \in V_h \\
&\geq \|\psi - \psi_h\|_a^2 && \|\psi_h - \chi_h\|_a^2 \geq 0.
\end{aligned}$$

This is Céa's lemma: the Galerkin solution is the best approximation of the exact solution from V_h in the energy norm. For piecewise linear hats on a mesh with edge length h the best approximation error in this norm is of order h, and so is the error of the method. Our space is conforming only up to the small rotations of Stage 5, which add a term of the order of their size to this bound (Strang's lemma).

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

24. Stage 7: The Element Integrals

What we want. On one face, with vertices x_1, x_2, x_3 in counterclockwise order, we need $\int_f \nabla\phi_a \cdot \nabla\phi_b \, dA$ and $\int_f \phi_a \phi_b \, dA$ for the nine pairs of local vertices. We compute them on the reference triangle, with the affine map and the chain rule, as Part I did; geometric algebra replaces the matrix inverse by the reciprocal frame of the edges.

24.1. The reference triangle, the chain rule and the reciprocal frame

The map and the chain rule. The reference triangle $\hat{K}$ has the vertices $(0,0)$, $(1,0)$, $(0,1)$ in the coordinates $\xi = (\xi_1, \xi_2)$, with the hats $\hat{\phi}_1 = 1 - \xi_1 - \xi_2$, $\hat{\phi}_2 = \xi_1$, $\hat{\phi}_3 = \xi_2$. The affine map (12.3) is

$$F(\xi) = x_1 + B\,\xi, \qquad B = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \end{bmatrix}, \tag{12.3}$$

which, written with the edge vectors, is

$$F(\xi) = x_1 + \xi_1\, e_1 + \xi_2\, e_2, \qquad e_1 = x_2 - x_1, \quad e_2 = x_3 - x_1,$$

and the hats of the face are $\phi_a(F(\xi)) = \hat{\phi}_a(\xi)$. Fix ξ_2 and move along ξ_1: the point $F(\xi)$ moves on the curve $s \mapsto F(\xi_1 + s, \xi_2) = F(\xi) + s\, e_1$, with velocity e_1. By the definition of the directional derivative (Stage 1), the derivative of ϕ_a along that curve is $e_1 \cdot \nabla\phi_a$; the same holds for ξ_2 and e_2. So the chain rule reads

$$\frac{\partial \hat{\phi}_a}{\partial \xi_m} = \frac{d}{ds}\,\phi_a\big(F(\xi) + s\, e_m\big)\Big|_{s=0} = e_m \cdot \nabla\phi_a, \qquad m = 1, 2.$$

These are two equations for the gradient $g = \nabla\phi_a$, a vector of the face plane: $g \cdot e_m = \partial\hat{\phi}_a/\partial\xi_m$. Part I solved them with the matrix $B(B^TB)^{-1}$. Here we solve them with a pair of vectors made for the purpose.

The reciprocal frame. The edges e_1, e_2 are in general neither orthogonal nor of unit length. The *reciprocal frame* of the edges is the pair of vectors e^1, e^2 of the face plane with

$$e^1 \cdot e_1 = 1, \quad e^1 \cdot e_2 = 0, \qquad e^2 \cdot e_1 = 0, \quad e^2 \cdot e_2 = 1,$$

that is, $e^k \cdot e_m = \delta_{km}$. Each reciprocal vector is orthogonal to the *other* edge, and its length is such that its inner product with its own edge is one. With it, every vector g of the plane is

$$g = \sum_k (g \cdot e_k)\, e^k.$$

To see it, take the inner product of the right side with e_m: $\sum_k (g{\cdot}e_k)\,(e^k{\cdot}e_m) = \sum_k (g \cdot e_k)\,\delta_{km} = g \cdot e_m$. Both sides are vectors of the plane with the same inner products with e_1 and e_2, and a vector of the plane is determined by those two numbers (their difference would be orthogonal to two independent vectors of the plane). Applied to $g = \nabla\phi_a$, with $g \cdot e_k = \partial\hat{\phi}_a/\partial\xi_k$ from the chain rule,

$$\nabla\phi_a = \sum_k \frac{\partial\hat{\phi}_a}{\partial\xi_k}\, e^k = e^1\,\frac{\partial\hat{\phi}_a}{\partial\xi_1} + e^2\,\frac{\partial\hat{\phi}_a}{\partial\xi_2},$$

so on the face the vector derivative is $\nabla = e^1\,\partial/\partial\xi_1 + e^2\,\partial/\partial\xi_2$. If the edges happened to be orthonormal, then $e^k = e_k$ and $\partial/\partial\xi_k = e_k \cdot \nabla$, and this is (18.6) [$\nabla F = \sum_k e_k\,(e_k{\cdot}\nabla F)$]. With the reference gradients (12.2) [$\hat{\nabla}\hat{\phi}_1 = (-1,-1)$, $\hat{\nabla}\hat{\phi}_2 = (1,0)$, $\hat{\nabla}\hat{\phi}_3 = (0,1)$],

$$\nabla\phi_2 = e^1 \cdot 1 + e^2 \cdot 0 = e^1, \qquad \nabla\phi_3 = e^1 \cdot 0 + e^2 \cdot 1 = e^2,$$
$$\nabla\phi_1 = e^1 \cdot (-1) + e^2 \cdot (-1) = -e^1 - e^2.$$

The matrix $C = B(B^TB)^{-1}$ of Part I, `Binv`, has the same defining property: $B^TC = (B^TB)(B^TB)^{-1} = I_2$, whose entry (m,k) says $e_m \cdot c_k = \delta_{mk}$ for the columns c_k of C. Its columns are combinations of the columns e_1, e_2 of B (the matrix B multiplies from the left), so they lie in the plane of the face. By the uniqueness just used, its columns are e^1 and e^2.

The reciprocal frame in geometric algebra. A vector orthogonal to e_2 in the face plane is e_2 turned by a quarter turn, $e_2\,\mathsf{T}_f$. So e^1 must be a multiple of $e_2\mathsf{T}_f$, and e^2 a multiple of $e_1\mathsf{T}_f$; only the factors are to be found. They come from the area bivector (22.1) [$E = e_1 \wedge e_2 = 2A_f\,\mathsf{T}_f$] of Stage 5, whose inverse is, with $\mathsf{T}_f^2 = -1$,

$$E^{-1} = -\frac{E}{|E|^2} = -\frac{2A_f\,\mathsf{T}_f}{4A_f^2} = -\frac{\mathsf{T}_f}{2A_f};$$

check: $E\,E^{-1} = 2A_f\,\mathsf{T}_f\big(-\mathsf{T}_f/(2A_f)\big) = -\mathsf{T}_f^2 = 1$. The geometric algebra formula for the reciprocal frame is

$$e^1 = e_2 \cdot E^{-1}, \qquad e^2 = -\,e_1 \cdot E^{-1}, \qquad E^{-1} = -\frac{E}{|E|^2}. \tag{24.1}$$

The products expanded. For a vector a of the face plane, the geometric product with T_f has no outer part: by the splitting rule, $a \wedge \mathsf{T}_f = I\,(a \cdot n_f) = 0$, so $a\,\mathsf{T}_f = a \cdot \mathsf{T}_f + a \wedge \mathsf{T}_f = a \cdot \mathsf{T}_f$, which by (18.3) is $n_f \times a$. Then

$$\begin{aligned} e^1 = e_2 \cdot E^{-1} &= e_2 \cdot \Big(-\frac{\mathsf{T}_f}{2A_f}\Big) && E^{-1} \text{ substituted} \\ &= -\frac{1}{2A_f}\; e_2 \cdot \mathsf{T}_f && \text{real factor out} \\ &= -\frac{e_2\,\mathsf{T}_f}{2A_f} = -\frac{n_f \times e_2}{2A_f} && e_2 \cdot \mathsf{T}_f = e_2\mathsf{T}_f, \end{aligned}$$

and in the same three steps, with $-e_1$ in the place of e_2,

$$\begin{aligned} e^2 = -\,e_1 \cdot E^{-1} &= -\,e_1 \cdot \Big(-\frac{\mathsf{T}_f}{2A_f}\Big) = \frac{1}{2A_f}\; e_1 \cdot \mathsf{T}_f \\ &= \frac{e_1\,\mathsf{T}_f}{2A_f} = \frac{n_f \times e_1}{2A_f}. \end{aligned}$$

The four conditions, in geometric algebra. For two vectors, $x \cdot y = \langle xy \rangle$. We need $\langle e_2 \mathsf{T}_f e_1 \rangle$ and its relatives. By the cyclic rule, $\langle e_2 \mathsf{T}_f e_1 \rangle = \langle e_1 e_2 \mathsf{T}_f \rangle$, and

$$\begin{aligned} e_1 e_2 &= e_1 \cdot e_2 + e_1 \wedge e_2 = e_1 \cdot e_2 + 2A_f\,\mathsf{T}_f, \\ e_2 e_1 &= e_1 \cdot e_2 - e_1 \wedge e_2 = e_1 \cdot e_2 - 2A_f\,\mathsf{T}_f. \end{aligned}$$

Each condition takes three lines: write the inner product as a scalar part, rotate the factors with the cyclic rule, and expand the product of the two

edges.

$$
\begin{aligned}
e^1 \cdot e_1 &= \Big\langle -\frac{e_2 \mathsf{T}_f}{2A_f}\, e_1 \Big\rangle = -\frac{1}{2A_f}\,\langle e_1 e_2\, \mathsf{T}_f\rangle && \text{cyclic rule}\\
&= -\frac{1}{2A_f}\,\big\langle (e_1 \cdot e_2)\, \mathsf{T}_f + 2A_f\, \mathsf{T}_f^2 \big\rangle && e_1 e_2 \text{ expanded}\\
&= -\frac{1}{2A_f}\,\big(0 - 2A_f\big) = 1 && \langle \mathsf{T}_f\rangle = 0,\ \mathsf{T}_f^2 = -1;\\
e^1 \cdot e_2 &= -\frac{1}{2A_f}\,\langle e_2\, \mathsf{T}_f\, e_2\rangle = -\frac{1}{2A_f}\,\langle e_2 e_2\, \mathsf{T}_f\rangle && \text{cyclic rule}\\
&= -\frac{|e_2|^2}{2A_f}\,\langle \mathsf{T}_f\rangle = 0 && e_2 e_2 = |e_2|^2;\\
e^2 \cdot e_2 &= \frac{1}{2A_f}\,\langle e_1\, \mathsf{T}_f\, e_2\rangle = \frac{1}{2A_f}\,\langle e_2 e_1\, \mathsf{T}_f\rangle && \text{cyclic rule}\\
&= \frac{1}{2A_f}\,\big\langle (e_1 \cdot e_2)\, \mathsf{T}_f - 2A_f\, \mathsf{T}_f^2 \big\rangle && e_2 e_1 \text{ expanded}\\
&= \frac{1}{2A_f}\,\big(0 + 2A_f\big) = 1 && \langle \mathsf{T}_f\rangle = 0,\ \mathsf{T}_f^2 = -1;\\
e^2 \cdot e_1 &= \frac{1}{2A_f}\,\langle e_1\, \mathsf{T}_f\, e_1\rangle = \frac{1}{2A_f}\,\langle e_1 e_1\, \mathsf{T}_f\rangle && \text{cyclic rule}\\
&= \frac{|e_1|^2}{2A_f}\,\langle \mathsf{T}_f\rangle = 0 && e_1 e_1 = |e_1|^2.
\end{aligned}
$$

The same, with cross products, as the code computes them: $e^1 \cdot e_2 = 0$ and $e^2 \cdot e_1 = 0$ because $n_f \times e_2$ is orthogonal to e_2 and $n_f \times e_1$ to e_1; and, with $(n \times u) \cdot v = n \cdot (u \times v)$ and $n_f \cdot (e_1 \times e_2) = n_f \cdot (2A_f\, n_f) = 2A_f$,

$$
e^1 \cdot e_1 = -\frac{n_f \cdot (e_2 \times e_1)}{2A_f} = \frac{n_f \cdot (e_1 \times e_2)}{2A_f} = 1, \qquad e^2 \cdot e_2 = \frac{n_f \cdot (e_1 \times e_2)}{2A_f} = 1.
$$

So no matrix needs to be inverted: each reciprocal vector is the other edge, turned by a quarter turn and divided by twice the area.

One vertex at a time. The three gradients have one common form. For a vertex a, let b, c be the next two vertices in the counterclockwise order a, b, c of the face, so that the edge opposite to a is $x_c - x_b$. Then

$$
\nabla \phi_a = \frac{(x_c - x_b)\, \mathsf{T}_f}{2A_f}\,: \tag{24.2}
$$

the opposite edge, turned a quarter turn, divided by twice the area. We check all three vertices, substituting the formulas just found, $e^1 = -\frac{e_2 \mathsf{T}_f}{2A_f}$ and $e^2 = \frac{e_1 \mathsf{T}_f}{2A_f}$:

- $a = 2$: the order is $2, 3, 1$, the opposite edge is $x_1 - x_3 = -(x_3 - x_1) = -e_2$, and
$$\frac{(x_1 - x_3)\,\mathsf{T}_f}{2A_f} = \frac{(-e_2)\,\mathsf{T}_f}{2A_f} = -\frac{e_2\mathsf{T}_f}{2A_f} = e^1 = \nabla\phi_2;$$
- $a = 3$: the order is $3, 1, 2$, the opposite edge is $x_2 - x_1 = e_1$, and $\frac{e_1\mathsf{T}_f}{2A_f} = e^2 = \nabla\phi_3$;
- $a = 1$: the order is $1, 2, 3$, the opposite edge is $x_3 - x_2 = (x_3 - x_1) - (x_2 - x_1) = e_2 - e_1$, and
$$\frac{(e_2 - e_1)\,\mathsf{T}_f}{2A_f} = \frac{e_2\mathsf{T}_f}{2A_f} - \frac{e_1\mathsf{T}_f}{2A_f} = (-e^1) - (e^2) = \nabla\phi_1.$$

The quarter turn is counterclockwise, and in a counterclockwise triangle the interior lies to the left of the edge from b to c, so the turned edge points into the triangle, towards a, where ϕ_a grows. Its length is $|(x_c - x_b)\mathsf{T}_f|/(2A_f) = \ell_a/(2A_f)$, with $\ell_a = |x_c - x_b|$ (the quarter turn keeps lengths). The area is half the base times the height, $2A_f = \ell_a h_a$, with h_a the height from a, so the length is $\ell_a/(\ell_a h_a) = 1/h_a$, the one found in the proof of Proposition 12.3. (On the triangle $x_1 = 0$, $x_2 = e_x$, $x_3 = e_y$ with $n = e_z$: $(x_1 - x_3)\mathsf{T} = n \times (-e_y) = e_x$, and $2A = 1$, which is indeed $\nabla\phi_2 = \nabla\xi_1 = e_x$.)

Proposition 24.1. *The gradients of the hat functions of a face are $\nabla\phi_2 = e^1$, $\nabla\phi_3 = e^2$ and $\nabla\phi_1 = -e^1 - e^2$, with the reciprocal frame (24.1) [$e^1 = e_2 \cdot E^{-1}$, $e^2 = -e_1 \cdot E^{-1}$] of the edges; the reciprocal frame is the pair of columns of $B(B^TB)^{-1}$, `Binv`; and for each vertex, $\nabla\phi_a$ is given by (24.2) [$\nabla\phi_a = \frac{(x_c - x_b)\mathsf{T}_f}{2A_f}$].*

The cotangent formula. The formula (24.2) gives the cotangent formula of Proposition 12.3. For the counterclockwise order a, b, c, the opposite edge of a is $x_c - x_b$ and that of b (in the order b, c, a) is $x_a - x_c$. First, the quarter turn preserves inner products: for vectors u, v of the plane, $u\mathsf{T}$ and $v\mathsf{T}$ are vectors, and

$$\begin{aligned}(u\mathsf{T}) \cdot (v\mathsf{T}) = \langle u\,\mathsf{T}\,v\,\mathsf{T}\rangle &= -\langle u\,v\,\mathsf{T}\,\mathsf{T}\rangle && \mathsf{T}v = -v\mathsf{T}\\ &= \langle u\,v\rangle = u \cdot v && \mathsf{T}^2 = -1.\end{aligned}$$

Then substitute (24.2) for a and for b:

$$\begin{aligned}A_f\,\nabla\phi_a \cdot \nabla\phi_b = A_f\,\frac{(x_c - x_b)\mathsf{T}_f}{2A_f} \cdot \frac{(x_a - x_c)\mathsf{T}_f}{2A_f} &&\text{(24.2) twice}\\ = \frac{A_f}{4A_f^2}\big((x_c - x_b)\mathsf{T}_f\big) \cdot \big((x_a - x_c)\mathsf{T}_f\big) &&\text{reals out}\\ = \frac{(x_c - x_b) \cdot (x_a - x_c)}{4A_f} &&\text{the turn keeps } \cdot\ .\end{aligned}$$

Write $u = x_b - x_c$ and $v = x_a - x_c$ for the two edges leaving c, with θ_c the angle between them. Then $(x_c - x_b) \cdot (x_a - x_c) = (-u) \cdot v = -|u||v|\cos\theta_c$,

and $2A_f = |u \wedge v| = |u||v| \sin\theta_c$, so

$$A_f \, \nabla\phi_a \cdot \nabla\phi_b = \frac{-|u||v| \cos\theta_c}{2\,(2A_f)} = \frac{-|u||v| \cos\theta_c}{2\,|u||v| \sin\theta_c} = -\tfrac{1}{2} \cot\theta_c.$$

In the code. The third part of `GAConnection.h` computes the gradients both ways:

```
85  /// The gradients of the three hat functions of the triangle x1, x2, x3 (Proposition
        ga_reciprocal):
86  /// the reciprocal frame e^1 = e2 . E^-1, e^2 = -e1 . E^-1 of the edges e1 = x2 - x1,
        e2 = x3 - x1, with
87  /// E = e1 ^ e2; then grad phi_2 = e^1, grad phi_3 = e^2, grad phi_1 = -e^1 - e^2.
88  inline void hatGradients(const Vec3& x1, const Vec3& x2, const Vec3& x3, Vec3 grad[3])
        {
89      const Vec3 e1 = x2 - x1, e2 = x3 - x1;
90      const Bivector Einv = inverse(wedge(e1, e2));
91      const Vec3 r1 = dot(e2, Einv);
92      const Vec3 r2 = -dot(e1, Einv);
93      grad[0] = -r1 - r2;
94      grad[1] = r1;
95      grad[2] = r2;
96  }
97
98  /// The same gradient, one vertex at a time (eq. ga_grad_edge): the opposite edge, from
        b to c in the
99  /// counterclockwise order a, b, c, turned a quarter turn in the plane of the face,
        divided by twice the area.
100 inline Vec3 hatGradient(const Vec3& xb, const Vec3& xc, const Bivector& Tf, double
        area) {
101     return quarterTurn(xc - xb, Tf) / (2.0 * area);
102 }
```

- Line 89: the edge frame $e_1 = x_2 - x_1$, $e_2 = x_3 - x_1$, with the local vertices numbered $0, 1, 2$ in the code for $1, 2, 3$ in the text.
- Line 90: E^{-1} of (24.1), the inverse of the outer product. In the stored dual form, E is $e_1 \times e_2 = 2A_f\, n_f$ and E^{-1} is $-(e_1 \times e_2)/|e_1 \times e_2|^2 = -2A_f\, n_f/(4A_f^2) = -n_f/(2A_f)$, the dual of $-\mathsf{T}_f/(2A_f)$.
- Lines 91–92: $e^1 = e_2 \cdot E^{-1}$ and $e^2 = -e_1 \cdot E^{-1}$. The function `dot` computes $b \times a$ for the dual vector b, so line 91 is $\left(-\frac{n_f}{2A_f}\right) \times e_2 = -\frac{n_f \times e_2}{2A_f}$, the formula derived above.
- Lines 93–95: the three gradients, $\nabla\phi_1 = -e^1 - e^2$, $\nabla\phi_2 = e^1$, $\nabla\phi_3 = e^2$, in the zero-based slots `grad[0]`, `grad[1]`, `grad[2]`.
- Lines 100–102: the one-vertex form (24.2): the edge from x_b to x_c, turned by `quarterTurn` and divided by $2A_f$. The caller must pass x_b, x_c in the counterclockwise order after a.

The `matrixform` namespace computes the same gradients as `FEM.cpp` does, with the matrix of Proposition 12.1:

```
93  /// gradN: B (B^T B)^-1 applied to the reference gradients (Proposition gradient)
94  static void gradients(Vertex* v[3], Vector3d gradN[3]) {
95      Matrix32d B;
96      B.col(0) = v[1]->p - v[0]->p;
97      B.col(1) = v[2]->p - v[0]->p;
98      const Matrix32d Binv = ((B.transpose() * B).inverse() * B.transpose()).transpose()
            ;
99      gradN[0] = -Binv.col(0) - Binv.col(1);
100     gradN[1] = Binv.col(0);
101     gradN[2] = Binv.col(1);
```

```
102 }
```

Lines 96–97 are the columns of B, line 98 is `Binv` $= \left((B^TB)^{-1}B^T\right)^T = B\left((B^TB)^{-1}\right)^T = B(B^TB)^{-1}$ (the matrix B^TB is symmetric, and so is its inverse), and lines 99–101 are (12.4) $[\,\nabla\phi_1 = -C_{:,1}-C_{:,2},\ \nabla\phi_2 = C_{:,1},\ \nabla\phi_3 = C_{:,2}\,]$. The two functions return the same vectors: the columns of `Binv` are the reciprocal frame.

Worked example 24.2. *For the triangle of Example 12.2, $x_1 = (0,0,0)$, $x_2 = (2,0,0)$, $x_3 = (0,1,1)$, the edges are $e_1 = (2,0,0)$ and $e_2 = (0,1,1)$, so $e_1 \times e_2 = (0\cdot1-0\cdot1,\ 0\cdot0-2\cdot1,\ 2\cdot1-0\cdot0) = (0,-2,2)$. The area bivector is $E = e_1 \wedge e_2 = I(e_1 \times e_2) = I(0,-2,2)$, with $|E| = \sqrt{0+4+4} = 2\sqrt{2} = 2A_f$, and $\mathsf{T}_f = E/|E| = In_f$ with $n_f = (0,-1,1)/\sqrt{2}$. The gradient of ϕ_2 by (24.2) $[\nabla\phi_a = \frac{(x_c-x_b)\mathsf{T}_f}{2A_f}\,]$ uses the opposite edge $x_1 - x_3 = (0,-1,-1)$, turned a quarter turn in the face:*

$$(x_1-x_3)\,\mathsf{T}_f = n_f \times (x_1-x_3) = \tfrac{1}{\sqrt{2}}(0,-1,1)\times(0,-1,-1)$$
$$= \tfrac{1}{\sqrt{2}}\big((-1)(-1)-(1)(-1),\ 0,\ 0\big) = \tfrac{1}{\sqrt{2}}(2,0,0)$$

(the second and third components are $1\cdot0-0\cdot(-1) = 0$ and $0\cdot(-1)-(-1)\cdot0 = 0$), divided by $2A_f = 2\sqrt{2}$: $\nabla\phi_2 = \frac{2}{\sqrt{2}\cdot2\sqrt{2}}(1,0,0) = (\frac{1}{2},0,0)$, as in Example 12.2, without inverting any matrix. The program checks the example in `workedExamples`:

```
380     const Vector3d x1(0, 0, 0), x2(2, 0, 0), x3(0, 1, 1);
381     Matrix32d B;
382     B.col(0) = x2 - x1;
383     B.col(1) = x3 - x1;
384     const Matrix32d Binv = B * (B.transpose() * B).inverse();
385     const double area = 0.5 * (x2 - x1).cross(x3 - x1).norm();
386     ga::Vec3 grad[3];
387     ga::hatGradients(x1, x2, x3, grad);
388     printf("  B^T B = [%g %g; %g %g], area = %.6f (sqrt 2 = %.6f)\n", (B.transpose() *
            B)(0, 0), (B.transpose() * B)(0, 1),
389           (B.transpose() * B)(1, 0), (B.transpose() * B)(1, 1), area, std::sqrt(2.0));
390     for (int a = 0; a < 3; ++a) printf("  grad phi_%d = (%g, %g, %g)\n", a + 1, grad[a
            ].x(), grad[a].y(), grad[a].z());
391     printf("  columns of Binv: (%g %g %g), (%g %g %g)\n", Binv(0, 0), Binv(1, 0), Binv
            (2, 0), Binv(0, 1), Binv(1, 1), Binv(2, 1));
392     const Vector3d n = (x2 - x1).cross(x3 - x1).normalized();
393     const ga::Vec3 g2 = ga::hatGradient(x3, x1, ga::tangentBivector(n), area);
394     printf("  grad phi_2 from the opposite edge turned a quarter turn: (%g, %g, %g)\n",
              g2.x(), g2.y(), g2.z());
395     printf("  stiffness A grad phi_a . grad phi_b:\n");
396     for (int a = 0; a < 3; ++a)
397         printf("    %9.6f %9.6f %9.6f\n", area * grad[a].dot(grad[0]), area * grad[a].
                dot(grad[1]), area * grad[a].dot(grad[2]));
398     const Vector3d u = x1 - x3, w = x2 - x3;
399     printf("  -cot(theta_3)/2 = %.6f\n", -0.5 * u.dot(w) / u.cross(w).norm());
```

Line 384 computes `Binv` as in Part I, line 387 the reciprocal frame, and line 393 the one-vertex form for $a = 2$, with $x_b = x_3$ and $x_c = x_1$, so that $x_c - x_b = x_1 - x_3$. Lines 396–399 print the scalar stiffness $A_f\,\nabla\phi_a\cdot\nabla\phi_b$ and $-\frac{1}{2}\cot\theta_3$, with θ_3 the angle at x_3 between $u = x_1 - x_3$ and $w = x_2 - x_3$, computed as $\cot\theta_3 = \frac{u\cdot w}{|u\times w|}$. The output is

```
grad phi_1 = (-0.5, -0.5, -0.5)
grad phi_2 = (0.5, 0, -0)
grad phi_3 = (-0, 0.5, 0.5)
columns of Binv: (0.5 0 0), (0 0.5 0.5)
grad phi_2 from the opposite edge turned a quarter turn: (0.5, 0, 0)
stiffness A grad phi_a . grad phi_b:
   1.060660 -0.353553 -0.707107
  -0.353553 0.353553 0.000000
  -0.707107 0.000000 0.707107
-cot(theta_3)/2 = -0.353553
```

the numbers of Example 12.2: the reciprocal frame agrees with the columns of `Binv`*, and the entry* $(1,2)$ *of the stiffness is* $-\frac{1}{2}\cot\theta_3$*. By hand:* $u = (0,-1,-1)$*,* $w = (2,-1,-1)$*,* $u\cdot w = 0+1+1 = 2$*,* $u\times w = \big((-1)(-1) - (-1)(-1),\ (-1)(2)-0\cdot(-1),\ 0\cdot(-1)-(-1)(2)\big) = (0,-2,2)$*,* $|u\times w| = 2\sqrt{2}$*, so* $-\frac{1}{2}\cot\theta_3 = -\frac{1}{2}\cdot\frac{2}{2\sqrt{2}} = -\frac{1}{2\sqrt{2}} = -0.353553$.

24.2. The element matrices

The area element. The map F takes a small rectangle $d\xi_1 \times d\xi_2$ of $\hat{K}$ to the parallelogram spanned by $e_1\,d\xi_1$ and $e_2\,d\xi_2$, whose area bivector is $(e_1\,d\xi_1)\wedge(e_2\,d\xi_2) = (e_1\wedge e_2)\,d\xi_1 d\xi_2 = E\,d\xi_1 d\xi_2$. Its area is the magnitude, $dA = |E|\,d\xi_1 d\xi_2 = 2A_f\,d\xi_1 d\xi_2$, and for any function g on the face,

$$\int_f g\,dA = 2A_f \int_{\hat{K}} g(F(\xi))\,d\xi_1\,d\xi_2.$$

(For $g = 1$: $2A_f\cdot\frac{1}{2} = A_f$, the area of $\hat{K}$ being $\frac{1}{2}$.)

The stiffness integrals. The gradients are constant on the face, so

$$\int_f \nabla\phi_a\cdot\nabla\phi_b\,dA = (\nabla\phi_a\cdot\nabla\phi_b)\int_f dA = A_f\,\nabla\phi_a\cdot\nabla\phi_b,$$

the area times the constant, which is $-\frac{1}{2}\cot\theta_c$ for $a\neq b$ by the cotangent formula.

The mass integrals. On $\hat{K}$ the inner variable runs from 0 to 1 minus the outer one. For the square of $\hat{\phi}_2 = \xi_1$,

$$\begin{aligned}\int_{\hat{K}} \xi_1^2\,d\xi &= \int_0^1 \Big(\int_0^{1-\xi_1} d\xi_2\Big)\,\xi_1^2\,d\xi_1 = \int_0^1 \xi_1^2(1-\xi_1)\,d\xi_1 \\ &= \Big[\frac{\xi_1^3}{3} - \frac{\xi_1^4}{4}\Big]_0^1 = \frac{1}{3}-\frac{1}{4} = \frac{1}{12},\end{aligned}$$

and for the product $\hat{\phi}_2\hat{\phi}_3 = \xi_1\xi_2$,

$$\begin{aligned}\int_{\hat{K}} \xi_1\xi_2\,d\xi &= \int_0^1 \xi_2\Big(\int_0^{1-\xi_2}\xi_1\,d\xi_1\Big)\,d\xi_2 = \int_0^1 \xi_2\,\frac{(1-\xi_2)^2}{2}\,d\xi_2 \\ &= \frac{1}{2}\int_0^1 \big(\xi_2 - 2\xi_2^2 + \xi_2^3\big)\,d\xi_2 \\ &= \frac{1}{2}\Big(\frac{1}{2}-\frac{2}{3}+\frac{1}{4}\Big) = \frac{1}{2}\cdot\frac{6-8+3}{12} = \frac{1}{24}.\end{aligned}$$

The other pairs give the same values: the map $(\xi_1, \xi_2) \mapsto (1 - \xi_1 - \xi_2,\ \xi_2)$ takes $\hat{K}$ onto itself, exchanges $\hat{\phi}_1$ and $\hat{\phi}_2$, and has Jacobian determinant -1, so it does not change integrals; and exchanging ξ_1 and ξ_2 exchanges $\hat{\phi}_2$ and $\hat{\phi}_3$. With the area element,

$$\int_f \phi_a^2\, dA = 2A_f \cdot \frac{1}{12} = \frac{A_f}{6}, \qquad \int_f \phi_a \phi_b\, dA = 2A_f \cdot \frac{1}{24} = \frac{A_f}{12} \quad (a \neq b).$$

With the transports of (23.1) this is the consistent mass $\frac{A_f}{12}(1 + \delta_{ab})\, e^{\mathsf{T}_f(\rho_b^f - \rho_a^f)}$, the complex form of (13.2) [$\mathrm{M}_{ab}^f = \frac{A_f}{12}(1 + \delta_{ab})\,(R_a^f)^T R_b^f$]. The code *lumps* it, as in (13.3) [$\mathrm{M}_{aa}^f = \frac{A_f}{3} I_2$]: it moves the row sum of the scalar factor to the diagonal, where the transport of a vertex to itself is 1. The row sum is

$$\sum_{b \in f} \int_f \phi_a \phi_b\, dA = \frac{A_f}{6} + 2 \cdot \frac{A_f}{12} = \frac{A_f}{3},$$

which is also $\int_f \phi_a \big(\sum_b \phi_b\big)\, dA = \int_f \phi_a\, dA = 2A_f \int_{\hat{K}} \xi_1\, d\xi = 2A_f \int_0^1 \xi_1(1 - \xi_1)\, d\xi_1 = 2A_f\big(\frac{1}{2} - \frac{1}{3}\big) = \frac{A_f}{3}$, because the hats add up to 1.

The element matrices. Putting the integrals into the face terms of (23.1) again,

$$\begin{aligned} \mathrm{M}_{ij} &= \sum_{f \ni i,j} \Big(\int_f \phi_i\, \phi_j\, dA \Big)\, e^{\mathsf{T}_f(\rho_j^f - \rho_i^f)}, \\ \mathrm{L}_{ij} &= \sum_{f \ni i,j} \Big(\int_f \nabla\phi_i \cdot \nabla\phi_j\, dA \Big)\, e^{\mathsf{T}_f(\rho_j^f - \rho_i^f)}, \end{aligned} \tag{23.1}$$

the stiffness integral $\big(A_f\, \nabla\phi_i \cdot \nabla\phi_j\big)$ in the place of $\int_f \nabla\phi_i \cdot \nabla\phi_j\, dA$, and the lumped mass $\frac{A_f}{3}$ on the diagonal, the contribution of one face f to the pair (i, j) is

$$\mathrm{L}_{ij}^f = A_f\, (\nabla\phi_i \cdot \nabla\phi_j)\, e^{\mathsf{T}(\rho_j^f - \rho_i^f)}, \qquad \mathrm{M}_{ii}^f = \frac{A_f}{3}\,: \tag{24.3}$$

one complex number per pair of vertices instead of a 2×2 block. The scalar factor is the cotangent weight of the edge, as in the scalar Laplacian; the complex factor is the transport from j to i through the face. The connection Laplacian is the scalar Laplacian in which every coupling first turns the neighbor's coordinate into the frame of the vertex.

Part II · The Same Derivation in Geometric Algebra, and What Is Intrinsic

25. Stage 8: Assembly and the Hermitian System

What we want. We want the global matrices, the linear system with the source condition, its solution by the program, and a check that it is the system of Part I.

25.1. Assembly and the system

Assembly. By (23.1), the global coefficients are sums over faces of the element contributions (24.3) again,

$$\mathrm{L}_{ij}^{f} = A_f\,(\nabla\phi_i\cdot\nabla\phi_j)\,e^{\mathsf{T}(\rho_j^f-\rho_i^f)}, \qquad \mathrm{M}_{ii}^{f} = \frac{A_f}{3}, \tag{24.3}$$

so that $\mathrm{L}_{ij} = \sum_{f\ni i,j}\mathrm{L}_{ij}^{f}$ and $\mathrm{M}_{ii} = \sum_{f\ni i}\frac{A_f}{3}$, one third of the area of the faces around i. This is the assembly (14.1) [$\mathrm{L} = \sum_f P_f^T \mathrm{L}^f P_f$] of Part I, with one complex entry per pair of vertices instead of a 2×2 block: every face adds its nine numbers to the rows and columns of its three vertices.

The matrix is Hermitian. Exchange i and j in (24.3). The real factor $A_f\,\nabla\phi_j\cdot\nabla\phi_i = A_f\,\nabla\phi_i\cdot\nabla\phi_j$ does not change, and the complex factor becomes

$$e^{\mathsf{T}(\rho_i^f-\rho_j^f)} = \cos(\rho_j^f-\rho_i^f) - \mathsf{T}\sin(\rho_j^f-\rho_i^f) = \overline{e^{\mathsf{T}(\rho_j^f-\rho_i^f)}},$$

using $\cos(-x)=\cos x$ and $\sin(-x)=-\sin x$. So $\mathrm{L}_{ji}^{f} = \overline{\mathrm{L}_{ij}^{f}}$ for every face, and, summing over the faces that contain both vertices, $\mathrm{L}_{ji} = \overline{\mathrm{L}_{ij}}$: the global stiffness L is an $n\times n$ complex *Hermitian* matrix.

The matrix is positive semidefinite. For a face and complex coordinates ψ_a of its vertices, write $w_a = e^{\mathrm{i}\rho_a^f}\psi_a$ (with T identified with i), so that $\overline{w_a} = e^{-\mathrm{i}\rho_a^f}\,\overline{\psi_a}$. Substitute (24.3) into the quadratic form of the face: With $\partial_k\phi_a = e_k\cdot\nabla\phi_a$ as in Stage 6,

$$\begin{aligned}
\sum_{a,b\in f}\overline{\psi_a}\,\mathrm{L}_{ab}^{f}\,\psi_b &= A_f\sum_{a,b}(\nabla\phi_a\cdot\nabla\phi_b)\,\overline{\psi_a}\,e^{\mathrm{i}(\rho_b^f-\rho_a^f)}\,\psi_b && \text{(a)}\\
&= A_f\sum_{a,b}(\nabla\phi_a\cdot\nabla\phi_b)\,\big(e^{-\mathrm{i}\rho_a^f}\,\overline{\psi_a}\big)\,\big(e^{\mathrm{i}\rho_b^f}\psi_b\big) && \text{(b)}\\
&= A_f\sum_{a,b}(\nabla\phi_a\cdot\nabla\phi_b)\,\overline{w_a}\,w_b && \text{(c)}\\
&= A_f\sum_{k}\sum_{a,b}\partial_k\phi_a\,\partial_k\phi_b\,\overline{w_a}\,w_b && \text{(d)}\\
&= A_f\sum_{k}\Big(\overline{\sum_a \partial_k\phi_a\,w_a}\Big)\Big(\sum_b\partial_k\phi_b\,w_b\Big) && \text{(e)}\\
&= A_f\sum_{k}\Big|\sum_b\partial_k\phi_b\,w_b\Big|^2 \;\ge 0 && \text{(f)}
\end{aligned}$$

(a) The element (24.3) substituted. (b) The exponential split, $e^{\mathrm{i}(\rho_b-\rho_a)} = e^{-\mathrm{i}\rho_a}e^{\mathrm{i}\rho_b}$, and the complex numbers regrouped (they commute). (c) The definition of w_a. (d) The inner product of the gradients in components, as in Stage 6. (e) The double sum is a product of two single sums; the real numbers $\partial_k\phi_a$ are their own conjugates. (f) $\bar z z = |z|^2$. By (22.9) [$D_{e_k}\psi_f = \sum_b (e_k\cdot\nabla\phi_b)\,e^{\mathsf{T}_f\rho_b^f}\psi_b$], the sum inside the last bars is $D_{e_k}\psi_f$, constant on the face, so the last line is $\int_f\sum_k |D_{e_k}\psi_f|^2\,dA$, the discrete energy of the face.

Summing over the faces, $\overline{\psi}^T \mathrm{L}\,\psi$ is the discrete Dirichlet energy of the field, twice E of (18.12) [$E = \frac{1}{2}\int_\Omega \sum_k |D_{e_k}\psi|^2\, dA$].

The system. The Galerkin equations (23.3) [$\sum_j (\mathrm{M}_{ij} + \Delta t\, \mathrm{L}_{ij})\,\psi_j = \sum_j \mathrm{M}_{ij}\,\psi_j^0$] are, in matrix form, the backward Euler step as the complex system

$$(\mathrm{M} + \Delta t\, \mathrm{L})\,\psi = \mathrm{M}\,\psi^0, \qquad \psi \in \mathbb{C}^n, \tag{25.1}$$

with one complex unknown per vertex. The matrix $\mathrm{M} + \Delta t\, \mathrm{L}$ is Hermitian and positive definite, since M is diagonal with positive entries:

$$\overline{\psi}^T (\mathrm{M} + \Delta t\, \mathrm{L})\,\psi = \sum_i \mathrm{M}_{ii}\,|\psi_i|^2 + \Delta t\, \overline{\psi}^T \mathrm{L}\,\psi \;\geq\; \Big(\min_i \mathrm{M}_{ii}\Big) \sum_i |\psi_i|^2,$$

which is positive for $\psi \neq 0$, the discrete counterpart of the coercivity of Stage 2. The code imposes the source and scales as Part I does. It multiplies by M^{-1}, which divides row i by M_{ii} and turns $\mathrm{M}^{-1}\mathrm{M}$ into I:

$$\mathrm{M}^{-1}(\mathrm{M} + \Delta t\, \mathrm{L})\,\psi = \mathrm{M}^{-1}\mathrm{M}\,\psi^0 \quad\Longrightarrow\quad (I + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\,\psi = \psi^0,$$

the complex form of (15.1) [$(I + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} = \mathrm{u}^0$]. It replaces the row of the source vertex s by the row of the identity, with the prescribed $\psi_s = e^{2\pi \mathsf{T}/3}$ on the right side; and it sets $\psi^0 = 0$ everywhere else, which is (18.14) [$\psi + \Delta t\, D^* D\psi = 0$ away from s, $\psi(s) = \psi_s$] on the mesh.

25.2. The program

The repository contains a second program, `ga_fem.cpp`, which needs no graphics and implements Part II with the small header `GAConnection.h`. It is built and run from `src/` with

```
c++ -std=c++17 -O2 -I<eigen> ga_fem.cpp HalfEdge/Mesh.cpp -o ga_fem
./ga_fem ../fem-basic-2d/data
```

It prints the worked examples of this text, solves the cactus of `FEM.cpp` twice, with the real 2×2 blocks of Part I and with the complex Hermitian system of Part II, solves the sphere of Example 15.1, and writes the data of Figures 16 and 17. The listings of `GAConnection.h` appear at the stages they implement: the algebra in Stage 1, the rotor and the connection in Stage 5, the gradients in Stage 7. This subsection shows the two solvers.

The complex system. The function `gaform::solve` assembles (24.3) [$\mathrm{L}_{ij}^f = A_f(\nabla\phi_i \cdot \nabla\phi_j)\, e^{\mathsf{T}(\rho_j^f - \rho_i^f)}$] and the lumped mass, scales by the mass, imposes the Dirichlet row, and solves, with `std::complex` entries. It does in one function what `AssembleStiffnessElementEmbedded2`, `AssembleMatrix2`, `IplusMinvTimesA` and `PreFactor` do in `FEM.cpp`, where each complex number is a 2×2 block:

```
151 /// Section 16: L_ij += A_f (grad phi_i . grad phi_j) e^{T (rho_j - rho_i)}, M_ii +=
        A_f / 3, then the same
152 /// scaling I + M^-1 (dt L) and the same Dirichlet row as the matrix version, now with
        complex numbers.
153 static Eigen::VectorXcd solve(Mesh& mesh, const Geometry& g, double dt, int source,
        Complex value) {
154     const int n = mesh.numVertices();
155     std::vector<double> mass(n, 0.0);
156     std::map<std::pair<int, int>, Complex> A;
```

```
157     for (Face& f : mesh.getFaces()) {
158         Vertex* v[3];
159         faceVertices(f, v);
160         ga::Vec3 grad[3];
161         ga::hatGradients(v[0]->p, v[1]->p, v[2]->p, grad); // the reciprocal frame
162         const double area = g.faceArea[f.ID];
163         Complex rho[3]; // e^{T_f rho_a^f}
164         for (int a = 0; a < 3; ++a)
165             rho[a] = ga::connection(g.vertexT[v[a]->ID], v[a]->n, g.faceT[f.ID], g.
                    faceNormal[f.ID]);
166         for (int a = 0; a < 3; ++a) {
167             mass[v[a]->ID] += area / 3.0;
168             for (int b = 0; b < 3; ++b) {
169                 // transport from b to a through the face: e^{T (rho_b - rho_a)} = e^{T
                        rho_b} conj(e^{T rho_a})
170                 const Complex transport = a == b ? Complex(1.0) : rho[b] * std::conj(rho
                        [a]);
171                 A[{ v[a]->ID, v[b]->ID }] += dt * area * grad[a].dot(grad[b]) *
                        transport;
172             }
173         }
174     }
175     std::vector<Eigen::Triplet<Complex>> triplets;
176     for (const auto& [ij, value_ij] : A) {
177         if (ij.first == source) continue;
178         triplets.emplace_back(ij.first, ij.second, value_ij / mass[ij.first] + (ij.
                first == ij.second ? 1.0 : 0.0));
179     }
180     triplets.emplace_back(source, source, 1.0);
181     Eigen::SparseMatrix<Complex> S(n, n);
182     S.setFromTriplets(triplets.begin(), triplets.end());
183     Eigen::VectorXcd rhs = Eigen::VectorXcd::Zero(n);
184     rhs(source) = value;
185     Eigen::SparseLU<Eigen::SparseMatrix<Complex>> lu(S);
186     return lu.solve(rhs);
187 }
```

- Lines 154–156: one mass per vertex and one complex entry per pair of vertices, stored in a map so that the contributions of different faces add up, which is the assembly without the P_f matrices of (14.1). The map holds $\Delta t\, \mathrm{L}$, as `AssembleMatrix2` does.
- Lines 157–161: the loop over faces, the three vertices in counterclockwise order, and the gradients of Proposition 24.1, Stage 7.
- Lines 163–165: the three connections $e^{\mathsf{T} f \rho_a^f}$ of (22.4), one per corner of the face, Stage 5. This is where the vertex normals, and so the extrinsic choice of Section 22.5, enter.
- Line 167: the lumped mass (24.3), $\frac{A_f}{3}$ to each vertex of the face.
- Lines 168–171: the element matrix (24.3). The local index `a` is the test vertex i, the row, and `b` the trial vertex j, the column. The transport from j to i is

$$e^{\mathsf{T}(\rho_j - \rho_i)} = e^{\mathsf{T}\rho_j}\, e^{-\mathsf{T}\rho_i} = e^{\mathsf{T}\rho_j}\, \overline{e^{\mathsf{T}\rho_i}},$$

  `rho[b] * std::conj(rho[a])`, since the conjugate of a unit complex number is its inverse. On the diagonal the code writes 1 directly, which is what the product $e^{\mathsf{T}\rho_i}\, \overline{e^{\mathsf{T}\rho_i}} = |e^{\mathsf{T}\rho_i}|^2 = 1$ gives up to rounding. Line 171 adds $\Delta t\, A_f\, (\nabla\phi_i \cdot \nabla\phi_j)$ times that number.

- Lines 175–179: the scaled system (15.1) $[(I + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} = \mathrm{u}^0]$: each row is divided by the mass of its vertex and 1 is added on the diagonal, `IplusMinvTimesA`. The row of the source is skipped (line 177).
- Lines 180–184: the Dirichlet condition by row replacement, `PreFactor`: the row of the source becomes a row of the identity, and the right-hand side holds the prescribed complex number there and zero elsewhere.
- Lines 185–186: the general sparse LU, because the scaled matrix with a replaced row is no longer Hermitian.

The real system. The function `matrixform::solve` is the same computation with the 2×2 blocks of Part I:

```
105 static Eigen::VectorXd solve(Mesh& mesh, const Geometry& g, double dt, int source,
        Vector2d value) {
106     const int n = mesh.numVertices();
107     std::vector<double> mass(n, 0.0);
108     std::map<std::pair<int, int>, double> A; // dt L, in the 2n x 2n real form
109     for (Face& f : mesh.getFaces()) {
110         Vertex* v[3];
111         faceVertices(f, v);
112         Vector3d gradN[3];
113         gradients(v, gradN);
114         const double area = g.faceArea[f.ID];
115         for (int a = 0; a < 3; ++a) {
116             mass[v[a]->ID] += area / 3.0;
117             for (int b = 0; b < 3; ++b) {
118                 const Vector2d ga = g.faceFrame[f.ID].transpose() * gradN[a];
119                 const Vector2d gb = g.faceFrame[f.ID].transpose() * gradN[b];
120                 const Matrix2d block = a == b ? Matrix2d(Matrix2d::Identity())
121                                               : Matrix2d(Rvf(g, v[a], f).transpose() * Rvf(
                                                     g, v[b], f));
122                 const Matrix2d w = dt * area * ga.dot(gb) * block;
123                 for (int r = 0; r < 2; ++r)
124                     for (int c = 0; c < 2; ++c) A[{ 2 * v[a]->ID + r, 2 * v[b]->ID + c
                            }] += w(r, c);
125             }
126         }
127     }
128     std::vector<Eigen::Triplet<double>> triplets;
129     for (const auto& [ij, value_ij] : A) {
130         const int i = ij.first, j = ij.second;
131         if (i / 2 == source) continue; // Dirichlet rows are replaced
132         triplets.emplace_back(i, j, value_ij / mass[i / 2] + (i == j ? 1.0 : 0.0)); //
                I + M^-1 (dt L)
133     }
134     triplets.emplace_back(2 * source, 2 * source, 1.0);
135     triplets.emplace_back(2 * source + 1, 2 * source + 1, 1.0);
136     Eigen::SparseMatrix<double> S(2 * n, 2 * n);
137     S.setFromTriplets(triplets.begin(), triplets.end());
138     Eigen::VectorXd rhs = Eigen::VectorXd::Zero(2 * n);
139     rhs.segment<2>(2 * source) = value;
140     Eigen::SparseLU<Eigen::SparseMatrix<double>> lu(S);
141     return lu.solve(rhs);
142 }
```

Lines 118–119 are the face coordinates (12.5) $[g_a^f = T_f^T \nabla\phi_a]$, lines 120–121 the transport (4.8) $[(R_a^f)^T R_b^f]$, with the identity on the diagonal, and line 122 the block (11.1) $[\mathrm{L}_{ij}^f = A_f\,(g_i^f \cdot g_j^f)\,(R_i^f)^T R_j^f]$ times Δt. Line 124 places the four entries of the block at the global indices $2\,\mathrm{ID} + r$, $2\,\mathrm{ID} + c$ of (14.2); in the complex version this line is the single entry `A[{v[a]->ID, v[b]->ID}]`. Lines 131–135 scale and replace the two rows of the source,

where the complex version replaces one row. Line by line, the two functions differ only where a complex number replaces a 2×2 block and an index i replaces the pair $2i, 2i+1$.

25.3. The same system as in Part I

A complex number is a 2×2 block. A complex number $z = \alpha + \beta\mathsf{T}$ acts on $\psi = u_1 + u_2\mathsf{T}$ by multiplication:

$$\begin{aligned} z\,\psi = (\alpha + \beta\mathsf{T})(u_1 + u_2\mathsf{T}) &= \alpha u_1 + \alpha u_2\mathsf{T} + \beta u_1\mathsf{T} + \beta u_2\mathsf{T}^2 \\ &= (\alpha u_1 - \beta u_2) + (\beta u_1 + \alpha u_2)\,\mathsf{T} \qquad \mathsf{T}^2 = -1. \end{aligned}$$

In the basis $(1, \mathsf{T})$ this takes the column (u_1, u_2) to $(\alpha u_1 - \beta u_2,\ \beta u_1 + \alpha u_2)$, so it is the real 2×2 matrix

$$[z] = \begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}, \qquad \alpha = \operatorname{Re} z,\ \beta = \operatorname{Im} z.$$

Two properties follow from the definition. A real number r is the block rI_2. The conjugate $\bar{z} = \alpha - \beta\mathsf{T}$ has the block $\begin{pmatrix} \alpha & \beta \\ -\beta & \alpha \end{pmatrix} = [z]^T$.

The complex system written in real numbers. Row i of (25.1) $[\,(\mathrm{M}+\Delta t\,\mathrm{L})\psi = \mathrm{M}\psi^0\,]$ is $\sum_j S_{ij}\psi_j = b_i$ with $S = \mathrm{M} + \Delta t\,\mathrm{L}$ and $b = \mathrm{M}\psi^0$. Write $S_{ij} = \alpha_{ij} + \beta_{ij}\mathsf{T}$, $\psi_j = u_{j,1} + u_{j,2}\mathsf{T}$, $b_i = b_{i,1} + b_{i,2}\mathsf{T}$. By the multiplication rule, the real and imaginary parts of row i are

$$\begin{aligned} \sum_j \left(\alpha_{ij}\,u_{j,1} - \beta_{ij}\,u_{j,2}\right) &= b_{i,1}, \\ \sum_j \left(\beta_{ij}\,u_{j,1} + \alpha_{ij}\,u_{j,2}\right) &= b_{i,2}\,: \end{aligned}$$

two real equations, the rows $2i$ and $2i+1$ of a $2n \times 2n$ real system whose block (i, j) is $[S_{ij}]$, for the $2n$ real unknowns $(u_{j,1}, u_{j,2})$. It has the same solution as the complex system, since it says the same thing.

The blocks, entry by entry. We compare $[\mathrm{L}^f_{ij}]$ with the block (11.1) of Part I,

$$\mathrm{L}^f_{ij} = A_f\left(g^f_i \cdot g^f_j\right)(R^f_i)^T R^f_j. \tag{11.1}$$

The scalar factor. With $g^f_a = T^T_f \nabla\phi_a$ of (12.5),

$$\begin{aligned} g^f_i \cdot g^f_j = (T^T_f\nabla\phi_i)^T(T^T_f\nabla\phi_j) &= \nabla\phi_i^T\,(T_f T_f^T)\,\nabla\phi_j \\ &= \nabla\phi_i^T\,(I - n_f n_f^T)\,\nabla\phi_j = \nabla\phi_i \cdot \nabla\phi_j, \end{aligned}$$

because the gradients lie in the face plane, $n_f^T\nabla\phi_j = 0$. *The rotation factor.* Write $c_a = \cos\rho^f_a$, $s_a = \sin\rho^f_a$, and multiply the transpose of (22.6) for i by (22.6) for j:

$$(R^f_i)^T R^f_j = \begin{pmatrix} c_i & s_i \\ -s_i & c_i \end{pmatrix}\begin{pmatrix} c_j & -s_j \\ s_j & c_j \end{pmatrix} = \begin{pmatrix} c_ic_j + s_is_j & -c_is_j + s_ic_j \\ -s_ic_j + c_is_j & s_is_j + c_ic_j \end{pmatrix}.$$

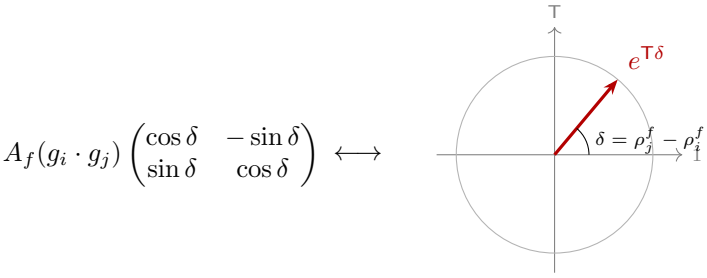


FIGURE 21. Every 2×2 block of the code is a real multiple of a rotation, the matrix of multiplication by a complex number $e^{\mathsf{T}\delta}$ of the tangent plane; the complex form stores only that number.

With $\delta = \rho_j^f - \rho_i^f$ and the addition formulas $\cos(\rho_j - \rho_i) = c_jc_i + s_js_i$ and $\sin(\rho_j - \rho_i) = s_jc_i - c_js_i$, the four entries are

$$(1,1):\quad c_ic_j + s_is_j = \cos\delta, \qquad (1,2):\quad -c_is_j + s_ic_j = -\sin\delta,$$
$$(2,1):\quad -s_ic_j + c_is_j = \sin\delta, \qquad (2,2):\quad s_is_j + c_ic_j = \cos\delta.$$

So $(R_i^f)^T R_j^f = \begin{pmatrix}\cos\delta & -\sin\delta\\ \sin\delta & \cos\delta\end{pmatrix} = [e^{\mathsf{T}\delta}]$. *The block.* Let $\lambda = A_f\,\nabla\phi_i \cdot \nabla\phi_j$. The complex element (24.3) is $\mathrm{L}_{ij}^f = \lambda\, e^{\mathsf{T}\delta} = \lambda\cos\delta + \lambda\sin\delta\ \mathsf{T}$, so $\alpha = \lambda\cos\delta$, $\beta = \lambda\sin\delta$, and its block is

$$\big[\lambda\, e^{\mathsf{T}\delta}\big] = \begin{pmatrix}\lambda\cos\delta & -\lambda\sin\delta\\ \lambda\sin\delta & \lambda\cos\delta\end{pmatrix} = \lambda\,(R_i^f)^T R_j^f = A_f\,(g_i^f \cdot g_j^f)\,(R_i^f)^T R_j^f,$$

entry by entry the block (11.1). *The mass.* The real number $\frac{A_f}{3}$ is the block $\frac{A_f}{3} I_2$, the lumped mass (13.3). *The symmetry.* The block (j,i) is $[\mathrm{L}_{ji}] = [\overline{\mathrm{L}_{ij}}] = [\mathrm{L}_{ij}]^T$: the Hermitian matrix becomes a symmetric one. *The source.* The scaling by M^{-1} divides rows by real numbers, the same in both forms; the identity row of the source is the identity block I_2 in two real rows; and the prescribed value $e^{2\pi\mathsf{T}/3}$ is the pair $(\cos\frac{2\pi}{3}, \sin\frac{2\pi}{3})$, which is `Vector2d(value.real(), value.imag())` below.

The two derivations, with matrices and with geometric algebra, arrive at the same linear system; the complex form halves its size and shows that every block of the code has the structure "scalar times rotation" (Figure 21).

The comparison. The program solves the cactus with both functions, the same mesh, the same source vector $e^{2\pi\mathrm{i}/3}$ and the same $\Delta t = \bar{h}^2$, and compares the solutions vertex by vertex:

```
1122        int source = 0;
1123        for (Vertex& v : mesh.getVertices())
1124            if (v.p.norm() < 2.5e-2 || v.p.z() > 0.83) { source = v.ID; break; }
1125        const Complex value = std::polar(1.0, 2.0 * M_PI / 3.0);
```

```
1126        const Eigen::VectorXd ur = matrixform::solve(mesh, g, h * h, source, Vector2d(
                value.real(), value.imag()));
1127        const Eigen::VectorXcd uc = gaform::solve(mesh, g, h * h, source, value);
1128        double diff = 0;
1129        for (int i = 0; i < mesh.numVertices(); ++i)
1130            diff = std::max(diff, std::abs(Complex(ur(2 * i), ur(2 * i + 1)) - uc(i)));
1131        printf("== Cactus (%d vertices, source %d): max |real form - complex form| = %.2e\n",
                mesh.numVertices(), source, diff);
```

Lines 1122–1124 choose the source vertex as `FEM.cpp` does, line 1125 is the source value $\psi_s = e^{2\pi \mathsf{T}/3}$, and lines 1126–1127 solve with $\Delta t = \bar{h}^2$, the square of the mean edge length `h`. Line 1130 reads the two real coordinates of vertex i as the complex number $u_{i,1} + \mathrm{i}\, u_{i,2}$ and measures its distance to the complex solution. The program prints

```
== Cactus (502 vertices, source 59): max |real form - complex form| = 5.76e-16
```

The two solutions agree to $6 \cdot 10^{-16}$, the rounding error of double precision: the two parts of this text compute the same thing.

25.4. From the spinors back to vectors

When the solution has been computed, the program turns the unit spinor coordinate $\hat{\psi}_i$ of every vertex into a vector of space, to draw it:

```
1134        for (Vertex& v : mesh.getVertices()) {
1135            const Complex z = uc(v.ID) / std::abs(uc(v.ID));
1136            const Vector3d u = g.vertexT[v.ID] * z.real() + ga::quarterTurn(g.vertexT[v
                    .ID], ga::tangentBivector(v.n)) * z.imag();
1137            out << v.p.x() << "," << v.p.y() << "," << v.p.z() << "," << u.x() << ","
                    << u.y() << "," << u.z() << "\n";
```

Line 1135 normalizes the computed complex number, $\hat{\psi}_i = \psi_i/|\psi_i|$: only the direction is kept, which is the transported source vector. Line 1136 is (18.5) $[\, U = t\psi,\ \psi = u_1 + u_2 \mathsf{T}\,]$ multiplied out:

$$t_i\, \hat{\psi}_i = t_i \left(\operatorname{Re} \hat{\psi}_i + \operatorname{Im} \hat{\psi}_i\ \mathsf{T}_i \right) = \operatorname{Re} \hat{\psi}_i\ t_i + \operatorname{Im} \hat{\psi}_i\ (t_i \mathsf{T}_i),$$

where the quarter turn $t_i \mathsf{T}_i$ is computed by `quarterTurn` from the vertex normal. The `std::complex` number holds $\alpha + \beta\,\mathsf{T}$ as the pair (α, β), so its imaginary part is the coefficient of T. Line 1137 writes the vertex and its vector, the data of Figure 16. By the frame independence of Stage 1, the drawn vector does not depend on the choice of t_i: turning the frame to $t_i' = t_i e^{\mathsf{T}\theta}$ turns the coordinate to $\hat{\psi}_i' = e^{-\mathsf{T}\theta} \hat{\psi}_i$, and $t_i' \hat{\psi}_i' = t_i\, e^{\mathsf{T}\theta} e^{-\mathsf{T}\theta}\, \hat{\psi}_i = t_i \hat{\psi}_i$.

26. Dictionary

Table 4 lists each object of the two parts side by side, with its nature, intrinsic or extrinsic.

The geometric algebra formulation removes the frames from the geometry: a tangent plane is a bivector, a direction in it is a complex number of that

TABLE 4. The objects of Part I and of Part II, and whether they depend only on the metric of the surface (intrinsic) or on its embedding in space (extrinsic).

Object	Matrices (Part I)	Geometric algebra	Nature
tangent plane	$TT^T = I - nn^T$	bivector $\mathsf{T} = In$	extrinsic
frame	$T = [t \;\; n \times t]$	one vector t, $b = t\mathsf{T}$	a gauge choice
coordinates	$u \in \mathbb{R}^2$	spinor $\psi = u_1 + u_2\mathsf{T}$	gauge dependent
inner product	$U \cdot V = u \cdot v$	$\langle \psi \widetilde{\chi} \rangle$	intrinsic
rotation in the plane	2×2 rotation	$e^{\mathsf{T}\theta}$	intrinsic
plane to plane	Rodrigues Q	rotor $(1 + n_f n_i)/\lvert 1 + n_f n_i \rvert$	extrinsic tool
discrete connection	$R_i^f = T_f^T Q T_i$	$e^{\mathsf{T}\rho_i^f} = t_f Q t_i \widetilde{Q}$	see Sec. 22.5
transport $j \to i$	$(R_i^f)^T R_j^f$	$e^{\mathsf{T}(\rho_j - \rho_i)}$	gauge covariant
covariant derivative	$\partial u + \omega u$	$D = \nabla + \omega\mathsf{T}$	intrinsic
hat gradients	$B(B^T B)^{-1}\hat{\nabla}\hat{\phi}$	reciprocal frame, $\frac{(x_c - x_b)\mathsf{T}}{2A}$	intrinsic
stiffness block	$A(g_i \cdot g_j)R_i^T R_j$	$A(\nabla\phi_i \cdot \nabla\phi_j)e^{\mathsf{T}(\rho_j - \rho_i)}$	intrinsic up to the vertex planes
global system	real $2n \times 2n$	complex Hermitian $n \times n$	—

plane, and transport is a rotor. Frames reappear only as the gauge in which coordinates are stored, and the gauge covariance (18.10) [$D'_a\psi' = e^{-\mathsf{T}\theta} D_a \psi$] guarantees that they do not affect the result. What remains of the embedding is the choice of a tangent plane at each vertex, the one extrinsic ingredient of the discrete connection, and the place where the intrinsic method of Part III differs.

Part III

The Intrinsic Method: Everything from Edge Lengths

Part II ended with one extrinsic ingredient left in the method: the tangent plane of a vertex, which the code takes orthogonal to the vertex normal. This part removes it. We build the whole method from the edge lengths of the mesh, and nothing else: no normal, no rotor of space, no position in $\mathbb{R}^3$. The program of this part reads from the mesh only two things: which triangles there are and how they are glued together, and the length of every edge. The positions of the vertices are read exactly once, to measure those lengths, and once more at the very end, to draw the result.

The same machine, fed in another way. Parts I and II ran the finite element method in eight stages, which we run here once more, under the same names:

1. the PDE (strong form): the equation, the unknown, the surface, the source condition;
2. the function spaces: where the solution lives, and why the problem has exactly one solution there;
3. the variational form: the energy whose minimizer solves the PDE;
4. the weak form (Galerkin): multiply by a test field, integrate, and integrate by parts;
5. the discretization: the mesh and the finite dimensional space;
6. the discrete solution and the test functions;
7. the element integrals, face by face;
8. assembly and the linear system.

What changes from part to part is the way differential geometry is fed into the machine. Part II fed it *extrinsically*: a tangent plane is a plane of space, written as a bivector T, and parallel transport is a rotor of space that rolls one plane onto another. This part feeds it *intrinsically*, in the language of a small *gauge theory*, the two dimensional version of the gauge theory of gravity of Doran and Lasenby [2]. All the computation happens in flat planes: the plane of each face, laid out from its lengths, and a plane for each vertex, made from the angles around it. Two fields say how those flat planes are related to the surface: the *displacement gauge* h, which says what a step in a flat plane is on the surface, and so how to measure lengths, and the *rotation gauge* Ω, which says how to compare vectors at nearby points, and so how to differentiate. We build both from the start in Section 28.

The element matrix (24.3) of Part II,

$$\mathrm{L}_{ij}^{f} = A_f \, (\nabla\phi_i \cdot \nabla\phi_j) \, e^{\mathsf{T}(\rho_j^f - \rho_i^f)},$$

will come out of the machine again, and it needs three things per face: an area A_f, the inner products $\nabla\phi_i \cdot \nabla\phi_j$ of the gradients of the three hat functions, and the connection angle ρ_i^f of each corner. In gauge language, the first two come from the metric, which is all that the displacement gauge leaves intrinsic; we compute them from the lengths of the face (Section 29). The third comes from the rotation gauge, and it is the real work. Inside a face the rotation gauge vanishes; the curvature of the mesh sits at its vertices, and it must be carried by the rotations that connect a plane for each vertex (Section 30) with the plane of each face (Section 31).

Three facts about $e^{\mathsf{T}\theta}$ used everywhere. From Part II we need the spinor coordinates and the unit complex numbers $e^{\mathsf{T}\theta}$ of a plane (the reader may read T as i). Since we shall use them in almost every line, we derive their three properties once, from $\mathsf{T}^2 = -1$ only. The powers of T repeat with period four: $\mathsf{T}^2 = -1$, $\mathsf{T}^3 = \mathsf{T}^2\,\mathsf{T} = -\mathsf{T}$, $\mathsf{T}^4 = (\mathsf{T}^2)^2 = 1$. In the exponential series we split the even and the odd powers:

$$\begin{aligned} e^{\mathsf{T}\theta} &= \sum_{n\ge 0} \frac{(\theta\mathsf{T})^n}{n!} && \text{the series} \\ &= \sum_{m\ge 0} \frac{\theta^{2m}\,(\mathsf{T}^2)^m}{(2m)!} + \sum_{m\ge 0} \frac{\theta^{2m+1}\,(\mathsf{T}^2)^m\,\mathsf{T}}{(2m+1)!} && \text{even, odd } n \\ &= \sum_{m\ge 0} \frac{(-1)^m\theta^{2m}}{(2m)!} + \mathsf{T}\sum_{m\ge 0} \frac{(-1)^m\theta^{2m+1}}{(2m+1)!} && \mathsf{T}^2 = -1 \\ &= \cos\theta + \mathsf{T}\sin\theta && \text{cos and sin.} \end{aligned}$$

(a) Its reverse. The reverse of a scalar is itself and the reverse of the bivector T is $\widetilde{\mathsf{T}} = -\mathsf{T}$, so

$$\widetilde{e^{\mathsf{T}\theta}} = \cos\theta + \widetilde{\mathsf{T}}\sin\theta = \cos\theta - \mathsf{T}\sin\theta = \cos(-\theta) + \mathsf{T}\sin(-\theta) = e^{-\mathsf{T}\theta}. \quad (26.1)$$

(b) Its derivative. If θ depends on a variable s, then, with the chain rule on cos and sin,

$$\begin{aligned} \frac{d}{ds}\,e^{\mathsf{T}\theta} &= -\theta'\sin\theta + \mathsf{T}\,\theta'\cos\theta = \theta'\left(\mathsf{T}\cos\theta - \sin\theta\right) && \text{chain rule} \\ &= \theta'\left(\mathsf{T}\cos\theta + \mathsf{T}^2\sin\theta\right) = \theta'\,\mathsf{T}\left(\cos\theta + \mathsf{T}\sin\theta\right) = \theta'\,\mathsf{T}\,e^{\mathsf{T}\theta} && -1 = \mathsf{T}^2. \end{aligned} \quad (26.2)$$

With $-\theta$ in place of θ: $\frac{d}{ds}e^{-\mathsf{T}\theta} = -\theta'\,\mathsf{T}\,e^{-\mathsf{T}\theta}$. For small θ, $\cos\theta \approx 1$ and $\sin\theta \approx \theta$, so $e^{\mathsf{T}\theta} \approx 1 + \theta\,\mathsf{T}$. *(c) Commutation.* All numbers $p + q\,\mathsf{T}$ commute with each other: $(p + q\mathsf{T})(r + s\mathsf{T}) = pr + ps\mathsf{T} + qr\mathsf{T} + qs\mathsf{T}^2$, and the same four terms come out of $(r + s\mathsf{T})(p + q\mathsf{T})$, because the real numbers p, q, r, s commute with everything and T commutes with itself. Spinor coordinates, T, $e^{\mathsf{T}\theta}$ and their products are all numbers of this kind, so we may reorder them freely. Finally, for the scalar part of a product of any two multivectors, $\langle AB\rangle = \langle BA\rangle$.

The plan. Section 27 says what it means to be intrinsic, with a folded sheet of paper. Section 28 builds the gauge language and runs Stages 1 to 4 in it, for a smooth surface. Stage 5, the discretization, takes three sections: the faces, flat charts in which the displacement gauge is constant and the rotation gauge vanishes (Section 29); the vertices, each with a flat chart made by stretching its cone of angles (Section 30); and the gauge transitions between the two, which carry the curvature (Section 31). Section 32 runs Stages 6 to 8. The program of this part is the header `IntrinsicConnection.h`, about one hundred and seventy lines that compute the whole geometry, with the function `intrinsicform::solve` of `ga_fem.cpp`, which assembles and solves the system. We show every function next to the formula it evaluates, at the

stage it serves. Section 33 compares the method with that of Part II on folded sheets, on spheres and on the cactus, with the code that does the comparison, and Section 35 compares the two ways of feeding the machine.

Part III · The Intrinsic Method: Everything from Edge Lengths

27. What It Means to Be Intrinsic

27.1. A folded sheet of paper

To test a method of parallel transport we need surfaces on which we know the answer, and on which a method can fail. The folded sheet of paper is such a test.

Take a flat sheet of paper and draw on it a field of parallel arrows, all pointing the same way. On the flat sheet these arrows are the parallel transport of each other: the vector heat method started from one of them should return all of them. Now fold the sheet, without stretching or tearing it, along a few straight lines. An ant walking on the paper cannot tell that the paper was folded: it measures the same lengths and the same angles as before, and to it the arrows are still parallel. Parallel transport is defined by what the ant can measure (Section 22.5), so the correct answer on the folded sheet is the same field of arrows, drawn on the folded paper.

A mesh carries two kinds of information. The first is what the ant measures: lengths, and the angles computed from them. The second is how the surface sits in space: the positions of the vertices, the normals, the angles at which neighboring faces meet. Folding changes the second kind and leaves the first kind exactly as it was. Since parallel transport depends only on the first kind, a method that gives a different answer after folding is using information it should not use. The exact answer is known at every resolution, so a correct method must reproduce it up to rounding errors, not merely approximately. On a curved surface such as the sphere no mesh is exact, and the test is instead how fast the error decreases under refinement; Section 33 uses both kinds of tests.

The method of Parts I and II gives the right answer for some foldings and not for others. If all the folds are parallel, as when the sheet is folded into a prism, it is exact. If the folds meet at angles, as in the Miura-ori pattern of Figure 22, in which four creases meet at every vertex, its vectors deviate from the drawn ones by 7.4° on average and by up to 21.9° (Section 33), although the paper is not stretched anywhere.

The culprit is the vertex normal (3.3),

$$n_i = \frac{\sum_{f \ni i} 2A_f\, n_f}{\left|\sum_{f \ni i} 2A_f\, n_f\right|},$$

the average of the normals of the faces around the vertex, weighted by their areas. Part II gives the vertex the plane orthogonal to that normal, and carries the vectors of each face into it by *rolling* the face about the axis $n_f \times n_i$, the

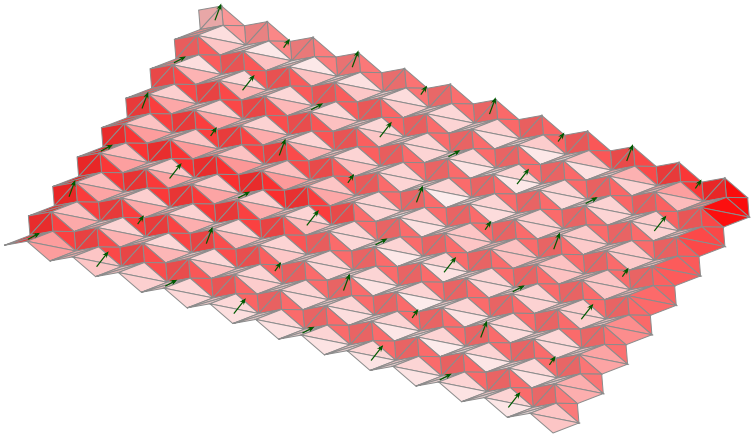


FIGURE 22. A Miura-ori fold: 24×16 parallelograms, each cut into two triangles, folded so that four creases meet at every vertex. The four angles at every vertex still add up to 2π, so the folded sheet is isometric to a flat one, and parallel transport on it is the transport of the flat sheet. Arrows: the exact field, started at the center, which the method of this part reproduces to $2 \cdot 10^{-13}$ degrees. Shading: the error of the method of Part II in each face, from 0° (white) to 21.9° (red).

line where the plane of the face meets the plane of the vertex. At a vertex of the Miura-ori the four faces are tilted in four different directions. Their averaged normal defines a plane that is none of them, and each face is rolled into it about a different axis; the rolled faces no longer fit together edge to edge as they do on the paper. When all the folds through a vertex are parallel, every face is rolled about the same line, the fold, and rolling a face about the fold is exactly unfolding the paper: that is why the prism comes out right.

27.2. Intrinsic quantities of a mesh

On a triangle mesh, what the ant can measure is the length of every edge, and from the lengths, the angles of every triangle, by the law of cosines. Inside a face it sees a flat triangle, and a flat triangle is determined, up to its position in space, by its three side lengths. So a mesh, as the ant knows it, is a list of triangles, the information of which triangles share which edges, and one length per edge.

Two meshes with the same triangles, glued in the same way, and the same edge lengths are called *isometric*: the second may be a folded copy of the first, as in Figure 22. A quantity is *intrinsic* if it can be computed from the edge lengths alone; it is then the same on all isometric meshes. A quantity that changes under folding, such as a normal, is *extrinsic*.

The program of Parts I and II reads from the mesh:

1. the positions x_a of the vertices of each face, through the matrix B, to compute the area and the gradients of the hat functions (Section 12);
2. the face normal n_f, for the face frame and the rotor $Q_{i\to f}$;
3. the vertex normal n_i, for the vertex frame and the rotor $Q_{i\to f}$.

Item 1 only looks extrinsic: the area and the gradients depend on the three edge lengths alone (Section 28.2 in general, Section 29 for a face). Items 2 and 3 enter only through the connection angle ρ_i^f; we replace them by angles measured on the faces (Sections 30 and 31). In the language of the next section: the positions enter through the displacement gauge h, the lengths through the metric that h produces, and the method will be allowed to use the metric only.

In the code. The program keeps the mesh in the half-edge structure of `src/HalfEdge`, which we describe in Section 30. For now we need only this: each face has a first half edge `f.edge`, and `next` goes to the following half edge of the same face, counterclockwise. The helper `faceCorners` collects the three vertices and the three half edges of a face, with `h[a]` going from `v[a]` to `v[a+1]`:

```
24  /// The three vertices of a face, counterclockwise, and the three half edges leaving
        them inside the face:
25  /// h[a] goes from v[a] to v[a+1].
26  inline void faceCorners(const Face& f, Vertex* v[3], Edge* h[3]) {
27      h[0] = f.edge; h[1] = f.edge->next; h[2] = f.edge->next->next;
28      for (int a = 0; a < 3; ++a) v[a] = h[a]->vertex;
29  }
```

In the notation of Part I, `v[0]`, `v[1]`, `v[2]` are x_1, x_2, x_3: the code counts from zero, the text from one. The first loop of `intrinsic::compute` is the only place where the positions `p` of the vertices are read. It stores the three lengths of each face and then calls `layOut`, which computes everything else about the face from those three numbers:

```
86  inline Geometry compute(Mesh& mesh) {
87      Geometry G;
88      G.faces.resize(mesh.numFaces());
89      for (Face& f : mesh.getFaces()) {
90          Vertex* v[3];
91          Edge* h[3];
92          faceCorners(f, v, h);
93          for (int a = 0; a < 3; ++a) G.faces[f.ID].length[a] = (v[(a + 1) % 3]->p - v[a
                ]->p).norm(); // the only use of p
94          layOut(G.faces[f.ID]);
95      }
```

Line 93 writes `length[a]` $= |x_{a+1} - x_a|$, the length of `h[a]`, with the index taken modulo three. So `length[0]` $= \ell_{12}$, `length[1]` $= \ell_{23}$ and `length[2]` $= \ell_{13}$, in the notation of Section 29. After line 95, the positions are not used again until the result is drawn.

28. Stages 1 to 4 in Gauge Language

What we want. Before we discretize, we write the continuous problem in a form that uses only what the ant can measure. Part II wrote it with vectors of space: a tangent field is a vector of $\mathbb{R}^3$ at each point, the tangent plane is a bivector T of space, and the covariant derivative removes from the derivative the part that leaves the plane. The ant has no $\mathbb{R}^3$. We now rewrite the same problem as a small gauge theory on flat planes, and then run Stages 1 to 4 in it. Nothing in this section is discrete: the surface is smooth, and the mesh returns in Section 29. The reward comes at Stage 5: the discretization will consist in choosing the two gauge fields piece by piece on the mesh.

28.1. The idea: compute in flat planes, keep the geometry in two fields

The finite element method computes in flat planes anyway: Part I integrated over the reference triangle, a triangle of the plane $\mathbb{R}^2$, and carried the results to the face with the affine map. How can a method that integrates over flat triangles solve an equation on a curved surface? The gauge language answers: the integrals are done in flat planes; the geometry of the surface enters only through the *metric*, which the displacement gauge produces in each plane, and through the *rotations* that relate the planes to each other, which the rotation gauge produces. Both fields contain arbitrary choices, and no measurable quantity may depend on them; a change of those choices is a *gauge transformation*, and a formula that does not change under it is *gauge invariant*.

In this part there are three kinds of flat planes: a *chart* of a smooth surface, that is, a patch P of $\mathbb{R}^2$ whose points label points of the surface (in this section); the plane of a face, which is the reference triangle or the layout of the face (Section 29); and the plane of a vertex, the stretched cone (Section 30).

A word on notation. Parts I and II use Ω alone for the surface, the domain of the integrals, and we keep that. The rotation gauge is written $\Omega(a)$, always with its argument, a flat direction a. In a chart we write $\partial_k = \partial/\partial\xi_k$, and for a flat vector $a = (a_1, a_2)$ the flat directional derivative is $a \cdot \hat{\nabla} = a_1\partial_1 + a_2\partial_2$.

28.2. The displacement gauge and the metric

The displacement gauge. Take a chart: a patch P of the plane with coordinates $\xi = (\xi_1, \xi_2)$, and a map $x(\xi)$ that places it on the surface. For the derivation we may picture the surface in space, but we shall keep only what survives without space. A small flat step ϵa, with $a = (a_1, a_2)$, at ξ goes to the step $x(\xi + \epsilon a) - x(\xi)$ on the surface. Taylor's formula in each of the two

variables gives

$$\begin{aligned} x(\xi+\epsilon a)-x(\xi) &= \epsilon a_1\,\partial_1 x+\epsilon a_2\,\partial_2 x+O(\epsilon^2) && \text{Taylor}\\ &= \epsilon\left(a_1\,\partial_1 x+a_2\,\partial_2 x\right)+O(\epsilon^2) && \epsilon \text{ out,}\end{aligned}$$

and the coefficient of ϵ is

$$h(a)=a_1\,\frac{\partial x}{\partial \xi_1}+a_2\,\frac{\partial x}{\partial \xi_2}, \tag{28.1}$$

the derivative of x in the direction a. The map h is linear, from the flat plane $\mathbb{R}^2$ to the tangent plane of the surface at $x(\xi)$; as a matrix it is the 3×2 matrix whose columns are $h(\varepsilon_1)=\partial_1 x$ and $h(\varepsilon_2)=\partial_2 x$, with $\varepsilon_1=(1,0)$ and $\varepsilon_2=(0,1)$. This is the *displacement gauge* of the chart: the rule that says what a flat vector "is" on the surface. (Doran and Lasenby write the map that acts on displacements as h^{-1}; we write h for it, because on a face it is B, as we now see.)

On a face. The affine map (12.3) is

$$F(\xi)=x_1+B\,\xi=x_1+\xi_1\,e_1+\xi_2\,e_2,\qquad B=\begin{bmatrix}e_1 & e_2\end{bmatrix}=\begin{bmatrix}x_2-x_1 & x_3-x_1\end{bmatrix}.$$

Its partial derivatives are $\partial_1 F=e_1$ and $\partial_2 F=e_2$, the same at every point, so by (28.1) $[\,h(a)=a_1\partial_1 x+a_2\partial_2 x\,]$ the displacement gauge of the face is $h(a)=a_1e_1+a_2e_2=Ba$: $h=B$, constant on the face.

The metric. The ant measures the lengths of steps and the angles between them, that is, inner products of surface vectors. For two flat steps a and b, writing h as a matrix, the inner product of their images is

$$\begin{aligned} h(a)\cdot h(b) &= (ha)^T(hb) && \text{inner product as } v^Tw\\ &= \left(a^Th^T\right)(h\,b) && (ha)^T=a^Th^T\\ &= a^T\left(h^Th\right)b && \text{regroup.}\end{aligned}$$

The entry (k,l) of h^Th is the product of column k and column l of h, that is $h(\varepsilon_k)\cdot h(\varepsilon_l)$. So

$$g(a,b)=h(a)\cdot h(b)=a^TG\,b,\qquad G=h^Th=\begin{pmatrix}h(\varepsilon_1)\cdot h(\varepsilon_1) & h(\varepsilon_1)\cdot h(\varepsilon_2)\\ h(\varepsilon_1)\cdot h(\varepsilon_2) & h(\varepsilon_2)\cdot h(\varepsilon_2)\end{pmatrix}. \tag{28.2}$$

The bilinear function g, or its symmetric matrix G, is the *metric* of the chart. We write G^{kl} for the entries of the inverse matrix G^{-1}; it is symmetric too, because $(G^{-1})^T=(G^T)^{-1}=G^{-1}$.

Only the metric is intrinsic. The gauge h says where a step goes in space; the metric says only how long it is and at what angle to the others. Rotate the surface in space by a rotation R, with $R^TR=I$. The gauge becomes Rh, and the metric becomes

$$(Rh)^T(Rh)=\left(h^TR^T\right)(R\,h)=h^T\left(R^TR\right)h=h^T\,I\,h=h^Th=G:$$

it does not change. Fold the surface without stretching it: each piece is moved rigidly, its gauge changes by its own rotation, and by the same lines its metric does not change. So everything the ant can measure is a function of g, and we

adopt the rule of this part: *the method may use the displacement gauge only through the metric.* The folded paper of Section 27 is the test of this rule.

What the finite element method needs from the metric. Three things: the area element, the inner product of gradients, and sums over an orthonormal basis.

(i) The area element and the change of variables. Cut the chart into small rectangles with sides $d\xi_1\,\varepsilon_1$ and $d\xi_2\,\varepsilon_2$. By (28.1), to first order, h carries such a rectangle to the parallelogram with sides $u\,d\xi_1$ and $v\,d\xi_2$, where $u = h(\varepsilon_1)$ and $v = h(\varepsilon_2)$. The oriented area of that parallelogram is the bivector $(u\,d\xi_1)\wedge(v\,d\xi_2) = (u\wedge v)\,d\xi_1 d\xi_2$, the scalars coming out of the outer product because it is bilinear; its area is $|u\wedge v|\,d\xi_1 d\xi_2$. We now compute $|u\wedge v|^2$ in geometric algebra. The geometric products of u and v in both orders are

$$uv = u\cdot v + u\wedge v, \qquad vu = v\cdot u + v\wedge u = u\cdot v - u\wedge v,$$

since the inner product is symmetric and the outer product antisymmetric. Multiply them in two ways. First by associativity, with $vv = |v|^2$ and $uu = |u|^2$:

$$(uv)(vu) = u\,(vv)\,u = |v|^2\,uu = |u|^2|v|^2.$$

Then by expanding the two sums, writing $c = u\cdot v$ (a scalar, which commutes with everything) and $B = u\wedge v$ for short:

$$\begin{aligned}(uv)(vu) &= (c+B)(c-B) && \text{the two products}\\ &= c^2 - cB + Bc - B^2 && \text{distribute}\\ &= c^2 - B^2 = (u\cdot v)^2 - (u\wedge v)^2 && cB = Bc.\end{aligned}$$

The bivector $u\wedge v$ lies in the tangent plane, so it is $|u\wedge v|$ times the unit bivector T of that plane (or minus it), and $(u\wedge v)^2 = |u\wedge v|^2\,\mathsf{T}^2 = -|u\wedge v|^2$. Equating the two results:

$$|u|^2|v|^2 = (u\cdot v)^2 + |u\wedge v|^2, \qquad \text{that is,} \qquad |u\wedge v|^2 = |u|^2|v|^2 - (u\cdot v)^2,$$

which is Lagrange's identity. By (28.2), $|u|^2 = G_{11}$, $|v|^2 = G_{22}$ and $u\cdot v = G_{12}$, so the right side is $G_{11}G_{22} - G_{12}^2 = \det G$. So the small rectangle $d\xi_1\,d\xi_2$ of the chart has, on the surface, the area

$$dA = \sqrt{\det G}\;d\xi_1\,d\xi_2, \tag{28.3}$$

and adding over all the small rectangles of the chart gives the change of variables of a double integral: for a function f on the part of the surface covered by the chart, written $\hat f(\xi) = f(x(\xi))$ in the chart,

$$\int f\,dA = \int_P \hat f(\xi)\,\sqrt{\det G(\xi)}\;d\xi_1\,d\xi_2. \tag{28.4}$$

For a face, $u = e_1$ and $v = e_2$, and $|e_1\wedge e_2| = |e_1\times e_2| = 2A_f$, so $\sqrt{\det G_f} = 2A_f$. For the triangle of Example 12.2, $e_1 = (2,0,0)$ and $e_2 = (0,1,1)$, so $G_f = \left(\begin{smallmatrix}4&0\\0&2\end{smallmatrix}\right)$, $\sqrt{\det G_f} = \sqrt{8} = 2\sqrt{2}$, and $A_f = \sqrt{2}$, the area of that triangle.

(ii) The gradients. A function f on the surface is a function $\hat f(\xi)$ in the chart. Its rate of change along the surface vector $h(a)$ is the derivative of

$\hat{f}(\xi + \epsilon a)$ at $\epsilon = 0$, because the curve $\epsilon \mapsto x(\xi + \epsilon a)$ leaves $x(\xi)$ with velocity $h(a)$; by the chain rule this is $a_1 \partial_1 \hat{f} + a_2 \partial_2 \hat{f} = a \cdot \hat{\nabla} \hat{f}$, with the flat gradient $\hat{\nabla} \hat{f} = (\partial_1 \hat{f}, \partial_2 \hat{f})$. The gradient ∇f is the tangent vector with

$$\nabla f \cdot h(a) = a \cdot \hat{\nabla} \hat{f} \qquad \text{for every flat } a.$$

Every tangent vector is the image of some flat vector, so we look for it as $\nabla f = h(c)$, and solve for c:

$$\begin{aligned} \nabla f \cdot h(a) &= h(c) \cdot h(a) && \nabla f = h(c) \\ &= c^T G\, a && \text{by (28.2)} \\ &= a^T G\, c && \text{a number is its own transpose, } G^T = G. \end{aligned}$$

So $a^T G c = a^T \hat{\nabla} \hat{f}$ for every a; with $a = \varepsilon_1$ and $a = \varepsilon_2$ this says that the two components of Gc and of $\hat{\nabla} \hat{f}$ agree, $Gc = \hat{\nabla} \hat{f}$, and so $c = G^{-1} \hat{\nabla} \hat{f}$. For two functions f, f', with $c' = G^{-1} \hat{\nabla} \hat{f}'$,

$$\begin{aligned} \nabla f \cdot \nabla f' = h(c) \cdot h(c') &= c^T G\, c' && \text{by (28.2)} \\ &= \left(G^{-1} \hat{\nabla} \hat{f}\right)^T G \left(G^{-1} \hat{\nabla} \hat{f}'\right) && \text{substitute } c,\ c' \\ &= (\hat{\nabla} \hat{f})^T \, (G^{-1})^T \, G \, G^{-1} \, \hat{\nabla} \hat{f}' && (AB)^T = B^T A^T \\ &= (\hat{\nabla} \hat{f})^T \, G^{-1} \, \hat{\nabla} \hat{f}' && (G^{-1})^T = G^{-1},\ GG^{-1} = I. \end{aligned} \tag{28.5}$$

The inner product of two gradients needs only the flat derivatives and the inverse metric. Section 29 does this computation again for a face, where $h = B$.

(iii) Sums over an orthonormal basis. The energy (5.1),

$$E(U) = \frac{1}{2} \int_\Omega |\nabla U|^2 \, dA, \qquad |\nabla U|^2 = \sum_{m=1}^{2} |\nabla_{e_m} U|^2,$$

sums over an orthonormal basis e_1, e_2 of the tangent plane. Write $e_m = h(c_m)$ and put the two flat vectors c_1, c_2 as the columns of a matrix C. Orthonormality says $e_m \cdot e_n = c_m^T G c_n = \delta_{mn}$, that is, $C^T G C = I$. Multiply on the left by $C^{-T} = (C^T)^{-1}$ and on the right by C^{-1}: $G = C^{-T} C^{-1}$. Invert, reversing the order of the factors: $G^{-1} = (C^{-1})^{-1} (C^{-T})^{-1} = C\, C^T$. The entry (k, l) of CC^T is row k of C times column l of C^T, so

$$G^{kl} = \sum_m (c_m)_k \, (c_m)_l.$$

Now let $X(e)$ and $Y(e)$ be two quantities that depend linearly on a direction e, such as two derivatives along e, and write $X_k = X(h(\varepsilon_k))$, $Y_l = Y(h(\varepsilon_l))$. Since $c_m = \sum_k (c_m)_k \varepsilon_k$ and h and X are linear,

$$X(e_m) = X\Big(h\Big(\sum_k (c_m)_k \, \varepsilon_k\Big)\Big) = \sum_k (c_m)_k \, X\big(h(\varepsilon_k)\big) = \sum_k (c_m)_k \, X_k,$$

and in the same way $Y(e_m) = \sum_l (c_m)_l\, Y_l$. Then

$$\begin{aligned}\sum_m X(e_m)\, Y(e_m) &= \sum_m \Big(\sum_k (c_m)_k X_k\Big)\Big(\sum_l (c_m)_l Y_l\Big) && \text{substitute}\\ &= \sum_{k,l} \Big(\sum_m (c_m)_k (c_m)_l\Big) X_k Y_l && \text{swap sums}\\ &= \sum_{k,l} G^{kl}\, X_k\, Y_l && \text{entry of } CC^T.\end{aligned} \tag{28.6}$$

An orthonormal basis of the surface is replaced, in the chart, by the inverse metric. With $X(e) = e \cdot \nabla f$ and $Y(e) = e \cdot \nabla f'$, $X_k = \partial_k \hat{f}$ and $Y_l = \partial_l \hat{f}'$, and (28.6) is (28.5) again: $\sum_m (e_m \cdot \nabla f)(e_m \cdot \nabla f') = \nabla f \cdot \nabla f' = \sum_{k,l} G^{kl}\, \partial_k \hat{f}\, \partial_l \hat{f}'$.

28.3. The rotation gauge and the covariant derivative

Spinor coordinates in a chart. A tangent field U is described by its spinor coordinate with respect to an orthonormal frame field t, as in (18.5):

$$U = t\,\psi, \qquad \psi = u_1 + u_2\,\mathsf{T}, \qquad b = t\,\mathsf{T} = n \times t.$$

In the chart we read $\psi(\xi)$ as a complex number attached to each point of the flat plane, with T as the imaginary unit; all planes have the same algebra of such numbers, so nothing of space is needed to compute with them. These numbers are what the finite element method will store at the vertices. To differentiate U we must compare the coordinates at two nearby points, and the coordinates change for two reasons: because the field turns, and because the frame turns. Only the first is geometric.

The rotation gauge. Part II measured the turning of the frame against parallel transport with the connection form (18.8) [$\nabla_a t = \omega(a)\, t\,\mathsf{T}$], where $\omega(a) = b \cdot \nabla_a t$. In the chart we attach it to flat directions: the *rotation gauge* of the chart is

$$\Omega(a) = \omega\big(h(a)\big)\,\mathsf{T}, \tag{28.7}$$

for each flat direction a, a bivector: the infinitesimal rotation of the frame, relative to parallel transport, per unit flat step along a. It is a bivector because rotations are generated by bivectors. Over the small flat step ϵa the frame turns by the angle $\epsilon\,\omega(h(a))$, that is, by the unit complex number $e^{\mathsf{T}\epsilon\,\omega(h(a))} \approx 1 + \epsilon\,\omega(h(a))\,\mathsf{T} = 1 + \epsilon\,\Omega(a)$, by the small-angle form $e^{\mathsf{T}\theta} \approx 1 + \theta\,\mathsf{T}$ of (26.2). In two dimensions every bivector of the plane is a multiple of T, so Ω amounts to one number per direction. Since ω and h are linear, with $a = a_1\varepsilon_1 + a_2\varepsilon_2$,

$$\omega_k = \omega\big(h(\varepsilon_k)\big), \qquad \omega\big(h(a)\big) = \omega\big(a_1 h(\varepsilon_1) + a_2 h(\varepsilon_2)\big) = a_1\,\omega_1 + a_2\,\omega_2 :$$

two numbers per point. ω is a *1-form*, a linear function of the direction.

The covariant derivative. We want the covariant derivative of $U = t\psi$ along the surface direction $h(a)$, written in the chart. Equation (18.9) of Part II is, for a surface direction X,

$$\nabla_X(t\psi) = t\,\big(X \cdot \nabla\psi + \omega(X)\,\mathsf{T}\,\psi\big).$$

Put $X = h(a)$. The term $h(a) \cdot \nabla\psi$ is the rate of change of the two real components of ψ along $h(a)$, which by the chain rule of (ii) above is the flat rate $a \cdot \hat{\nabla}\psi$; and $\omega(h(a))\,\mathsf{T} = \Omega(a)$ by (28.7). So

$$\begin{aligned}\nabla_{h(a)}(t\psi) &= t\,\big(h(a)\cdot\nabla\psi + \omega(h(a))\,\mathsf{T}\,\psi\big) && X = h(a)\\ &= t\,\big(a\cdot\hat{\nabla}\psi + \Omega(a)\,\psi\big) && \text{chain rule, (28.7)},\end{aligned}$$

and the covariant derivative of U along $h(a)$ is $t\,D_a\psi$ with

$$D_a\psi = a\cdot\hat{\nabla}\psi + \Omega(a)\,\psi, \qquad D_k\psi = \partial_k\psi + \omega_k\,\mathsf{T}\,\psi \quad (a = \varepsilon_k). \tag{28.8}$$

The first term is the change of the coordinates seen in the flat plane; the second is the correction for the turning of the frame. All the geometry of differentiation is in Ω. Both terms are linear in a, so $D_a\psi = a_1D_1\psi + a_2D_2\psi$.

The rotation gauge comes from the metric. The connection form is not extra data. Cartan's first two structure equations of Remark 18.1, $d\theta^1 = \omega\wedge\theta^2$ and $d\theta^2 = -\omega\wedge\theta^1$, determine ω from the coframe of t, and the coframe is determined by the metric and the chosen frame. So Ω is intrinsic, a function of g and of the choice of frame. One case matters above all. Suppose the chart is *isometric*, $G = I$, as for the layout of a face in the plane: $x(\xi) = x_0 + \xi_1E_1 + \xi_2E_2$ with two fixed orthonormal vectors E_1, E_2. Take as frame a constant direction of the chart, $t = \cos\gamma\,E_1 + \sin\gamma\,E_2$ with a fixed angle γ. Then t is the same vector at every point, its derivative along any direction is zero, and

$$\omega_k = b\cdot\nabla_{h(\varepsilon_k)}t = b\cdot 0 = 0, \qquad \Omega = 0, \qquad D_k\psi = \partial_k\psi + 0\cdot\mathsf{T}\psi = \partial_k\psi :$$

the covariant derivative is the plain derivative of the two components. A chart and frame with $\Omega = 0$ is called a *flat gauge*.

Parallel transport in a chart. A field is parallel along a curve $\xi(s)$ of the chart when its covariant derivative along the curve vanishes, $D_{\xi'(s)}\psi = 0$. By (28.8) with $a = \xi'(s)$, and since $\xi'\cdot\hat{\nabla}\psi = d\psi/ds$ by the chain rule,

$$0 = D_{\xi'}\psi = \frac{d\psi}{ds} + \omega\big(h(\xi')\big)\,\mathsf{T}\,\psi, \qquad \text{that is,} \qquad \frac{d\psi}{ds} = -\,\omega\big(h(\xi')\big)\,\mathsf{T}\,\psi.$$

Let $\varphi(s) = \int_0^s \omega(h(\xi'(\sigma)))\,d\sigma$, so that $\varphi' = \omega(h(\xi'))$, and try $\psi(s) = e^{-\mathsf{T}\varphi(s)}\psi(0)$. By (26.2) with $-\varphi$ in place of θ,

$$\frac{d\psi}{ds} = \Big(\frac{d}{ds}\,e^{-\mathsf{T}\varphi}\Big)\psi(0) = -\varphi'\,\mathsf{T}\,e^{-\mathsf{T}\varphi}\psi(0) = -\,\omega\big(h(\xi')\big)\,\mathsf{T}\,\psi(s),$$

which is the equation, and $\psi(s) = \psi(0)$ at $s = 0$. A linear differential equation has only one solution with a given initial value, so

$$\psi(s) = e^{-\mathsf{T}\int_0^s\omega}\,\psi(0), \qquad \int_0^s\omega = \int_0^s\big(\omega_1\,\xi_1'(\sigma) + \omega_2\,\xi_2'(\sigma)\big)\,d\sigma. \tag{28.9}$$

In coordinates, parallel transport is a rotation by minus the integral of the connection form along the path: one unit complex number per path. The frame turns forwards by $\int \omega$ relative to the parallel field, so the coordinates of the parallel field turn backwards by the same angle. The formula is this simple because rotations of a plane commute (fact (c)): the small rotations along the path can be multiplied in any order, and their angles simply add. The discrete method will replace each such integral, from a vertex into a face, by one angle ρ_i^f.

28.4. Gauge transformations and the covariance of the energy

The rotation gauge transformation. The frame t is an arbitrary choice. Turn it by an angle field $\theta(\xi)$: $t' = t\, e^{\mathsf{T}\theta}$. We compute the three things that change, in the chart.

The coordinates. The same vector is $U = t\psi = t'\psi'$. Substitute t':

$$t\,\psi = \left(t\, e^{\mathsf{T}\theta}\right)\psi'.$$

Multiply on the left by t, with $tt = 1$: $\psi = e^{\mathsf{T}\theta}\psi'$. Multiply on the left by $e^{-\mathsf{T}\theta}$, with $e^{-\mathsf{T}\theta}e^{\mathsf{T}\theta} = e^0 = 1$:

$$\psi' = e^{-\mathsf{T}\theta}\,\psi.$$

The connection form. By $e^{\mathsf{T}\theta} = \cos\theta + \mathsf{T}\sin\theta$ and $t\mathsf{T} = b$, the new frame vector is $t' = \cos\theta\, t + \sin\theta\, b$. Its derivative along $h(\varepsilon_k)$ follows from the product rule, the chain rule $\partial_k \cos\theta = -\sin\theta\,\partial_k\theta$, $\partial_k \sin\theta = \cos\theta\,\partial_k\theta$, and the turning of the old frame, $\nabla_{h(\varepsilon_k)} t = \omega_k\, b$ and $\nabla_{h(\varepsilon_k)} b = -\omega_k\, t$ (Section 18.4):

$$\begin{aligned}
\nabla_{h(\varepsilon_k)} t' &= -\sin\theta\,(\partial_k\theta)\,t + \cos\theta\,\omega_k\, b && \text{product rule}\\
&\quad + \cos\theta\,(\partial_k\theta)\,b - \sin\theta\,\omega_k\, t && \\
&= \left(\omega_k + \partial_k\theta\right)\left(-\sin\theta\, t + \cos\theta\, b\right) && \text{collect}\\
&= \left(\omega_k + \partial_k\theta\right) t'\,\mathsf{T} && \text{see below.}
\end{aligned}$$

The last line uses $t'\mathsf{T} = (\cos\theta\, t + \sin\theta\, b)\mathsf{T} = \cos\theta\; t\mathsf{T} + \sin\theta\; t\mathsf{T}\mathsf{T} = \cos\theta\; b - \sin\theta\; t$, by $t\mathsf{T} = b$ and $\mathsf{T}^2 = -1$. Comparing with the definition $\nabla_{h(\varepsilon_k)} t' = \omega_k'\, t'\mathsf{T}$, and with (28.7) for a general flat a,

$$\omega_k' = \omega_k + \partial_k\theta, \qquad \Omega'(a) = \Omega(a) + (a\cdot\hat{\nabla}\theta)\,\mathsf{T}.$$

The covariant derivative. We want $D_k'\psi'$, the covariant derivative (28.8) in the new frame, $D_k'\psi' = \partial_k\psi' + \omega_k'\,\mathsf{T}\,\psi'$. Substitute $\psi' = e^{-\mathsf{T}\theta}\psi$ and $\omega_k' =$

$\omega_k + \partial_k\theta$, each inside parentheses, and do one operation per line:

$$\begin{aligned}
D'_k\psi' &= \partial_k\big(e^{-\mathsf{T}\theta}\psi\big) + \big(\omega_k + \partial_k\theta\big)\,\mathsf{T}\,\big(e^{-\mathsf{T}\theta}\psi\big) && \text{substitute}\\
&= \big(\partial_k e^{-\mathsf{T}\theta}\big)\psi + e^{-\mathsf{T}\theta}\,\partial_k\psi && \text{product rule}\\
&\quad + \big(\omega_k + \partial_k\theta\big)\,\mathsf{T}\,e^{-\mathsf{T}\theta}\psi\\
&= -(\partial_k\theta)\,\mathsf{T}e^{-\mathsf{T}\theta}\psi + e^{-\mathsf{T}\theta}\,\partial_k\psi && \text{derivative}\\
&\quad + \big(\omega_k + \partial_k\theta\big)\,\mathsf{T}\,e^{-\mathsf{T}\theta}\psi\\
&= -(\partial_k\theta)\,\mathsf{T}e^{-\mathsf{T}\theta}\psi + e^{-\mathsf{T}\theta}\,\partial_k\psi && \text{distribute}\\
&\quad + \omega_k\,\mathsf{T}e^{-\mathsf{T}\theta}\psi + (\partial_k\theta)\,\mathsf{T}e^{-\mathsf{T}\theta}\psi\\
&= e^{-\mathsf{T}\theta}\,\partial_k\psi + \omega_k\,\mathsf{T}e^{-\mathsf{T}\theta}\psi && \text{1st, 4th cancel}\\
&= e^{-\mathsf{T}\theta}\,\partial_k\psi + e^{-\mathsf{T}\theta}\,\omega_k\,\mathsf{T}\psi && \text{fact (c)}\\
&= e^{-\mathsf{T}\theta}\big(\partial_k\psi + \omega_k\,\mathsf{T}\psi\big) && e^{-\mathsf{T}\theta}\text{ out}\\
&= e^{-\mathsf{T}\theta}\,D_k\psi && \text{by (28.8).}
\end{aligned}$$

In the line marked "derivative" we used (26.2), $\partial_k e^{-\mathsf{T}\theta} = -(\partial_k\theta)\,\mathsf{T}e^{-\mathsf{T}\theta}$, and in the line marked "fact (c)" that $\mathsf{T}e^{-\mathsf{T}\theta} = e^{-\mathsf{T}\theta}\mathsf{T}$. The extra turning appears twice, once from differentiating the rotated coordinate and once in the new connection form, with opposite signs. This is (18.10) $[\,D'_a\psi' = e^{-\mathsf{T}\theta}D_a\psi\,]$ in the chart: the covariant derivative transforms exactly as the coordinates do. The computation used nothing about ψ except that it is a spinor valued function, so for any such function F,

$$D'_k\big(e^{-\mathsf{T}\theta}F\big) = e^{-\mathsf{T}\theta}\,D_kF. \tag{28.10}$$

Covariance of the energy. The finite element method needs two densities: the product of two fields and the product of their derivatives. For spinors, the inner product is the scalar part of the product with the reverse, $U\cdot V = \langle\psi\widetilde{\chi}\rangle$ by (19.1). For the derivatives, apply (28.6) with $X(e) = \nabla_e U$ and $Y(e) = \nabla_e V$, whose products are taken as $X\cdot Y$; since $\nabla_{h(\varepsilon_k)}U = t\,D_k\psi$, the factors are $X_k = t\,D_k\psi$ and $Y_l = t\,D_l\chi$, and $X_k\cdot Y_l = \langle(D_k\psi)\,\widetilde{(D_l\chi)}\rangle$ by (19.1). So the energy density $\nabla U : \nabla V = \sum_m \nabla_{e_m}U\cdot\nabla_{e_m}V$ is, in the chart,

$$\nabla U : \nabla V = \sum_{k,l} G^{kl}\,\big\langle(D_k\psi)\,\widetilde{(D_l\chi)}\big\rangle. \tag{28.11}$$

Now change the frame. The reverse of a product is the product of the reverses in the opposite order, $\widetilde{AB} = \widetilde{B}\,\widetilde{A}$, so by (26.1)

$$\widetilde{\chi'} = \widetilde{e^{-\mathsf{T}\theta}\chi} = \widetilde{\chi}\,\widetilde{e^{-\mathsf{T}\theta}} = \widetilde{\chi}\,e^{\mathsf{T}\theta}.$$

Then the first density, line by line:

$$\begin{aligned}
\big\langle\psi'\widetilde{\chi'}\big\rangle &= \big\langle\big(e^{-\mathsf{T}\theta}\psi\big)\big(\widetilde{\chi}\,e^{\mathsf{T}\theta}\big)\big\rangle && \text{substitute}\\
&= \big\langle e^{\mathsf{T}\theta}\,e^{-\mathsf{T}\theta}\,\psi\,\widetilde{\chi}\big\rangle && \langle AB\rangle = \langle BA\rangle,\ A = e^{-\mathsf{T}\theta}\psi\widetilde{\chi},\ B = e^{\mathsf{T}\theta}\\
&= \langle\psi\widetilde{\chi}\rangle && e^{\mathsf{T}\theta}e^{-\mathsf{T}\theta} = 1.
\end{aligned}$$

By (28.10), $D'_k\psi' = e^{-\mathsf{T}\theta} D_k\psi$ and $D'_l\chi' = e^{-\mathsf{T}\theta} D_l\chi$, so the same three lines, with $D_k\psi$ in place of ψ and $D_l\chi$ in place of χ, give

$$\big\langle (D'_k\psi')\,\widetilde{(D'_l\chi')}\big\rangle = \big\langle \big(e^{-\mathsf{T}\theta} D_k\psi\big)\big(\widetilde{D_l\chi}\; e^{\mathsf{T}\theta}\big)\big\rangle = \big\langle (D_k\psi)\,\widetilde{(D_l\chi)}\big\rangle.$$

The metric G comes from h alone and does not depend on the frame at all. So both densities are gauge invariant, and so is every integral built from them: the energy, the weak form and the solution. The arbitrary frame cannot affect the result.

The displacement gauge transformation. The other arbitrary choice is the chart. Moving the surface rigidly or folding it changes h but not G, as we saw, and so changes nothing. Relabel the points of a chart, $\xi = \xi(\eta)$, with the Jacobian matrix $J = \partial\xi/\partial\eta$, whose entry (k,p) is $\partial\xi_k/\partial\eta_p$. A flat step a' in the η chart is the step Ja' in the ξ chart, so the new displacement gauge is $h'(a') = h(Ja')$, $h' = hJ$, and the three ingredients change as follows.

$$\begin{aligned} G' = h'^T h' = (hJ)^T(hJ) &= J^T\big(h^T h\big)\,J = J^T G\,J, \\ \det G' = \det J^T\,\det G\,\det J &= (\det J)^2\,\det G, \\ \sqrt{\det G'}\;d\eta_1 d\eta_2 &= \sqrt{\det G}\;|\det J|\;d\eta_1 d\eta_2, \\ G'^{-1} &= J^{-1}\,G^{-1}\,J^{-T}. \end{aligned}$$

The third line is the area element (28.3) of the new chart, and $|\det J|\,d\eta_1 d\eta_2 = d\xi_1 d\xi_2$ is the change of variables of a double integral, so dA is the same. For the derivatives, the chain rule gives $\partial/\partial\eta_p = \sum_k (\partial\xi_k/\partial\eta_p)\,\partial_k$, that is, the column of flat derivatives changes as $\hat\nabla_\eta = J^T\hat\nabla_\xi$; and $\omega'_p = \omega(h'(\varepsilon_p)) = \omega(h(J\varepsilon_p)) = \sum_k J_{kp}\,\omega_k$ changes in the same way, so the column $D_\eta\psi$ of covariant derivatives is $J^T D_\xi\psi$. Then the derivative density becomes

$$\begin{aligned} (D_\eta\psi)^T\,G'^{-1}\,(D_\eta\chi) &= \big(J^T D_\xi\psi\big)^T\big(J^{-1}G^{-1}J^{-T}\big)\big(J^T D_\xi\chi\big) && \text{substitute} \\ &= (D_\xi\psi)^T\,J\,J^{-1}\,G^{-1}\,J^{-T}J^T\,(D_\xi\chi) && (J^Tv)^T = v^TJ \\ &= (D_\xi\psi)^T\,G^{-1}\,(D_\xi\chi) && JJ^{-1} = I, \end{aligned}$$

where $(D\psi)^T G^{-1}(D\chi)$ is shorthand for $\sum_{k,l} G^{kl}\langle (D_k\psi)\,\widetilde{(D_l\chi)}\rangle$. So the densities, integrated with dA, give the same numbers in every chart.

Transitions. On the overlap of two charts A and B, with frames t_A and t_B, the two descriptions of a field agree after a rotation, the *transition* between their frames. If $t_B = t_A\,e^{\mathsf{T}\theta_{AB}}$ at a point, then $t_A\psi_A = t_B\psi_B = t_A\,e^{\mathsf{T}\theta_{AB}}\psi_B$; multiply on the left by t_A and then by $e^{-\mathsf{T}\theta_{AB}}$, as for the coordinates above:

$$\psi_B = e^{-\mathsf{T}\theta_{AB}}\,\psi_A. \tag{28.12}$$

This is all that is needed to cover a whole surface with charts, compute in each, and glue.

Remark 28.1. *Dictionary with the gauge theory of gravity [2]. Their displacement gauge is our h (28.1), seen by measurements only through g (28.2); on a face $h = B$ and $g = G_f = B^TB$. Their rotation gauge is our $\Omega(a) =$*

$\omega(h(a))\,\mathsf{T}$ (28.7); their covariant derivative is (28.8); their field strength is the $d\omega$ of Section 28.5. On a mesh, h is constant in each face, Ω is zero in each face, and the curvature sits in the transitions at the vertices.

28.5. Curvature: the field strength of the rotation gauge

The flat-gauge condition. The best chart for computing is a flat gauge, where $\Omega = 0$ and the covariant derivative is the plain derivative; the vector heat equation would then be two scalar heat equations. Can we always turn the frame so that Ω vanishes? By the transformation rule $\omega'_k = \omega_k + \partial_k\theta$, we need an angle field θ with

$$0 = \omega_1 + \partial_1\theta, \qquad 0 = \omega_2 + \partial_2\theta, \qquad \text{that is,} \qquad \omega_1 = -\partial_1\theta, \quad \omega_2 = -\partial_2\theta.$$

Differentiate the second equation by ξ_1 and the first by ξ_2, and subtract:

$$\begin{aligned} \partial_1\omega_2 - \partial_2\omega_1 &= \partial_1\big(-\partial_2\theta\big) - \partial_2\big(-\partial_1\theta\big) && \text{substitute} \\ &= -\partial_1\partial_2\theta + \partial_2\partial_1\theta && \text{distribute} \\ &= 0 && \text{mixed derivatives are equal.} \end{aligned}$$

So a flat gauge can exist only where the *curl* $\partial_1\omega_2 - \partial_2\omega_1$ vanishes. The curl is gauge invariant: in the new gauge,

$$\begin{aligned} \partial_1\omega'_2 - \partial_2\omega'_1 &= \partial_1\big(\omega_2 + \partial_2\theta\big) - \partial_2\big(\omega_1 + \partial_1\theta\big) && \text{substitute} \\ &= \partial_1\omega_2 - \partial_2\omega_1 + \big(\partial_1\partial_2\theta - \partial_2\partial_1\theta\big) && \text{regroup} \\ &= \partial_1\omega_2 - \partial_2\omega_1 && \text{mixed derivatives.} \end{aligned}$$

It is called the *field strength* of the rotation gauge, or the exterior derivative of ω, $d\omega = (\partial_1\omega_2 - \partial_2\omega_1)\,d\xi_1 \wedge d\xi_2$. In the geometric algebra form of Doran and Lasenby the field strength of Ω is the bivector

$$\mathsf{R}(a \wedge b) = a \cdot \hat{\nabla}\,\Omega(b) - b \cdot \hat{\nabla}\,\Omega(a) + \Omega(a) \times \Omega(b), \qquad A \times B = \tfrac{1}{2}(AB - BA).$$

For $a = \varepsilon_1$ and $b = \varepsilon_2$, with $\Omega(\varepsilon_k) = \omega_k\mathsf{T}$ and the constant T coming out of the derivatives,

$$\begin{aligned} \mathsf{R}(\varepsilon_1 \wedge \varepsilon_2) &= \partial_1(\omega_2\mathsf{T}) - \partial_2(\omega_1\mathsf{T}) + \tfrac{1}{2}\big(\omega_1\mathsf{T}\,\omega_2\mathsf{T} - \omega_2\mathsf{T}\,\omega_1\mathsf{T}\big) && \text{substitute} \\ &= (\partial_1\omega_2 - \partial_2\omega_1)\,\mathsf{T} + \tfrac{1}{2}\,\omega_1\omega_2\,\big(\mathsf{T}^2 - \mathsf{T}^2\big) && \text{scalars out} \\ &= (\partial_1\omega_2 - \partial_2\omega_1)\,\mathsf{T} && \mathsf{T}^2 - \mathsf{T}^2 = 0. \end{aligned}$$

The commutator term vanishes because rotations of a plane commute; this is the one fact that makes the two dimensional theory simple.

What the field strength measures. We want to see the curl as a circulation. Integrate ω counterclockwise around a small flat rectangle with corners (ξ_1, ξ_2) and $(\xi_1 + \delta_1, \xi_2 + \delta_2)$, using (28.9): along a side parallel to ε_1 only ω_1 contributes, and along a side parallel to ε_2 only ω_2. Up to terms of third

order in δ:

$$\begin{aligned} \text{bottom}, \ +\xi_1: &\quad \omega_1(\xi_1,\xi_2)\,\delta_1, \\ \text{right}, \ +\xi_2: &\quad \omega_2(\xi_1+\delta_1,\xi_2)\,\delta_2, \\ \text{top}, \ -\xi_1: &\quad -\omega_1(\xi_1,\xi_2+\delta_2)\,\delta_1, \\ \text{left}, \ -\xi_2: &\quad -\omega_2(\xi_1,\xi_2)\,\delta_2. \end{aligned}$$

(Each side would have a correction of second order, such as $\frac{1}{2}\partial_1\omega_1\,\delta_1^2$ on the bottom; the corrections of opposite sides differ by terms of third order.) Add the four lines, pairing the two sides of each kind, and apply Taylor's formula to each difference:

$$\begin{aligned} \oint \omega &= \big[\omega_2(\xi_1+\delta_1,\xi_2)-\omega_2(\xi_1,\xi_2)\big]\,\delta_2 && \text{pair} \\ &\quad -\big[\omega_1(\xi_1,\xi_2+\delta_2)-\omega_1(\xi_1,\xi_2)\big]\,\delta_1 \\ &= \big(\partial_1\omega_2\,\delta_1\big)\,\delta_2-\big(\partial_2\omega_1\,\delta_2\big)\,\delta_1 && \text{Taylor} \\ &= (\partial_1\omega_2-\partial_2\omega_1)\,\delta_1\delta_2 && \text{collect.} \end{aligned}$$

The field strength is the circulation of ω per unit flat area. Cut a region P' of the chart into small rectangles and add their circulations: a side shared by two rectangles is walked twice, in opposite directions, and its two contributions cancel, so only the boundary of P' remains. This is Green's theorem in the plane (the theorem of Stokes in two dimensions):

$$\oint_{\partial P'} \omega = \int_{P'} (\partial_1\omega_2-\partial_2\omega_1)\,d\xi_1\,d\xi_2. \tag{28.13}$$

It also gives the converse of the flat-gauge condition. If the curl vanishes on a region without holes, the right side of (28.13) vanishes for every loop in it, so the integral of ω from a fixed point ξ_0 to ξ does not depend on the path; then $\theta(\xi)=-\int_{\xi_0}^{\xi}\omega$ is well defined, $\partial_k\theta=-\omega_k$, and the frame $t\,e^{\mathsf{T}\theta}$ is a flat gauge.

Curvature. With the orientation $b=t\mathsf{T}=n\times t$ used throughout, Remark 18.1 found $d\omega=-K\,dA$. We use charts with the same orientation, $h(\varepsilon_1)\wedge h(\varepsilon_2)$ a positive multiple of T. Write both sides as multiples of $d\xi_1\wedge d\xi_2$, the left by the definition of $d\omega$ and the right by (28.3) [$dA=\sqrt{\det G}\,d\xi_1 d\xi_2$]:

$$(\partial_1\omega_2-\partial_2\omega_1)\,d\xi_1\wedge d\xi_2=-K\,\big(\sqrt{\det G}\,d\xi_1\wedge d\xi_2\big).$$

Comparing the coefficients,

$$\partial_1\omega_2-\partial_2\omega_1=-K\,\sqrt{\det G}. \tag{28.14}$$

The field strength of the rotation gauge is minus the Gaussian curvature, measured per unit of flat area.

Example: the unit sphere. Take the chart $(\xi_1,\xi_2)=(\vartheta,\varphi)$ of colatitude and longitude, $x=(\sin\vartheta\cos\varphi,\sin\vartheta\sin\varphi,\cos\vartheta)$. Then

$$\partial_1 x=(\cos\vartheta\cos\varphi,\ \cos\vartheta\sin\varphi,\ -\sin\vartheta),\quad \partial_2 x=(-\sin\vartheta\sin\varphi,\ \sin\vartheta\cos\varphi,\ 0),$$

so $G_{11} = \cos^2\vartheta(\cos^2\varphi + \sin^2\varphi) + \sin^2\vartheta = 1$, $G_{22} = \sin^2\vartheta(\sin^2\varphi + \cos^2\varphi) = \sin^2\vartheta$, $G_{12} = -\cos\vartheta\sin\vartheta\cos\varphi\sin\varphi + \cos\vartheta\sin\vartheta\sin\varphi\cos\varphi = 0$, and $G = \operatorname{diag}(1, \sin^2\vartheta)$, $\sqrt{\det G} = \sin\vartheta$ for $0 < \vartheta < \pi$. The frame of Remark 18.1 is $t = \partial_1 x$, with $b = (-\sin\varphi, \cos\varphi, 0)$, and there $\omega = \cos\vartheta\, d\varphi$, that is $\omega_1 = 0$ and $\omega_2 = \cos\vartheta$. Substituting into the left side of (28.14):

$$\partial_1\omega_2 - \partial_2\omega_1 = \partial_\vartheta(\cos\vartheta) - \partial_\varphi(0) = -\sin\vartheta = -K\left(\sin\vartheta\right) \quad \Longrightarrow \quad K = 1.$$

Holonomy. Carry a vector once around a closed loop $\partial P'$ of the chart, counterclockwise. By (28.9) $[\,\psi(s) = e^{-\mathsf{T}\int_0^s \omega}\psi(0)\,]$ its coordinate returns multiplied by $e^{-\mathsf{T}\oint\omega}$; the frame is the same at the start and at the end, so this compares the vector with itself. Substitute (28.13) and then (28.14) into the exponent:

$$\begin{aligned} -\oint_{\partial P'}\omega &= -\int_{P'}\left(\partial_1\omega_2 - \partial_2\omega_1\right)d\xi_1 d\xi_2 && \text{by (28.13)} \\ &= -\int_{P'}\left(-K\sqrt{\det G}\right)d\xi_1 d\xi_2 && \text{by (28.14)} \\ &= \int_{P'} K\left(\sqrt{\det G}\,d\xi_1 d\xi_2\right) = \int K\,dA && \text{by (28.3)}, \end{aligned}$$

so

$$\psi_{\text{end}} = e^{\mathsf{T}\int K\,dA}\,\psi_{\text{start}}. \tag{28.15}$$

The vector comes back turned counterclockwise by the total curvature enclosed, as found in Section 22.5, and now from the metric alone: ω is computed from g, the field strength from ω. This is the Theorema Egregium in gauge language.

Example: a cap of the unit sphere. Take $P' = [0, \vartheta_0]\times[0, 2\pi]$ in the chart above. Its boundary, walked counterclockwise, has four sides. The bottom ($\varphi = 0$) and the top ($\varphi = 2\pi$) are the same meridian on the sphere and carry $\omega_1 = 0$, so they give nothing. The right side ($\vartheta = \vartheta_0$, φ from 0 to 2π) is the circle of colatitude ϑ_0 and gives $\int_0^{2\pi}\cos\vartheta_0\,d\varphi = 2\pi\cos\vartheta_0$. The left side ($\vartheta = 0$, φ from 2π to 0) is the pole itself, where the frame $t = \partial_\vartheta x$ is singular, and gives $\int_{2\pi}^{0}\cos 0\,d\varphi = -2\pi$. The right side of (28.13) is $\int_0^{2\pi}\int_0^{\vartheta_0}(-\sin\vartheta)\,d\vartheta\,d\varphi = -2\pi(1-\cos\vartheta_0)$, and indeed $2\pi\cos\vartheta_0 - 2\pi = -2\pi(1-\cos\vartheta_0)$. A vector carried once around the circle alone returns, by (28.9), multiplied by

$$e^{-\mathsf{T}\,2\pi\cos\vartheta_0} = e^{-\mathsf{T}\,2\pi\cos\vartheta_0}\,e^{\mathsf{T}\,2\pi} = e^{\mathsf{T}\,2\pi(1-\cos\vartheta_0)},$$

using $e^{\mathsf{T}\,2\pi} = \cos 2\pi + \mathsf{T}\sin 2\pi = 1$: the rotation by $2\pi(1-\cos\vartheta_0) = \int_{\text{cap}} K\,dA$, the area of the cap. The extra full turn is the one the frame makes around its singular point at the pole, and a full turn is no rotation.

What this means for a mesh. Three consequences prepare Stage 5. First, on a region that encloses curvature there is no flat gauge: Ω cannot be made zero everywhere, because its field strength is gauge invariant. Second, a face of a mesh is flat, $K = 0$ inside it, and it has a flat gauge: the layout with a constant frame. Third, the curvature of a mesh sits at its vertices, where the flat faces meet (Section 30); since it cannot sit in the faces, it must be

carried by the transitions (28.12) [$\psi_B = e^{-\mathsf{T}\theta_{AB}}\psi_A$] between the charts that meet there.

28.6. Stage 1: the PDE (strong form)

What we want. The problem, the unknown, the domain and the source condition, written in a chart.

The domain is a surface Ω, closed (the sphere, the cactus) or with a boundary (the sheets), covered by charts. The unknown is a tangent field U; in each chart it is a field of spinor coordinates ψ, and on the overlaps the coordinates of two charts are related by (28.12) [$\psi_B = e^{-\mathsf{T}\theta_{AB}}\psi_A$]. The equation is one backward Euler step of the vector heat equation, (5.5) [$U + \Delta t\,\nabla^*\nabla U = U^0$]. In a chart, with $U = t\psi$, $U^0 = t\psi^0$ and $\nabla^*\nabla(t\psi) = t\,D^*D\psi$, it reads, after cancelling the common factor t on the left,

$$\psi + \Delta t\, D^*D\psi = \psi^0, \qquad D^*D\psi = -\frac{1}{\sqrt{\det G}}\sum_{k,l} D_k\Big(\sqrt{\det G}\,G^{kl}\,D_l\psi\Big), \tag{28.16}$$

where D_k acts on any spinor valued function F as in (28.8), $D_kF = \partial_kF + \omega_k\mathsf{T}F$. As in Part I, the operator is the one that Green's identity produces; Stage 4 derives this formula.

The flat case. In a flat gauge with $G = I$: $\sqrt{\det G} = 1$, $G^{kl} = \delta_{kl}$ and $D_k = \partial_k$. Substitute the three into (28.16):

$$D^*D\psi = -\frac{1}{(1)}\sum_{k,l}\partial_k\Big((1)\,(\delta_{kl})\,\partial_l\psi\Big) = -\sum_k \partial_k\partial_k\psi = -\big(\partial_1^2 + \partial_2^2\big)\psi,$$

the sum over l keeping only $l = k$. Since $\psi = u_1 + u_2\mathsf{T}$ and the derivatives act on the two real components, on a plane the vector heat equation is two scalar heat equations, one for each component. On a curved surface no gauge is flat everywhere, and the terms in ω couple the two components: $\omega_k\mathsf{T}\psi = \omega_k(u_1\mathsf{T} - u_2)$ mixes u_1 into the T part and u_2 into the scalar part. This coupling is how curvature acts on the field.

Covariance of the strong form. Under a change of frame, $\psi' = e^{-\mathsf{T}\theta}\psi$, and G does not change. Write $F_k = \sqrt{\det G}\sum_l G^{kl}D_l\psi$. Using (28.10) [$D'_k(e^{-\mathsf{T}\theta}F) = e^{-\mathsf{T}\theta}D_kF$] twice, first with $F = \psi$ and then with $F = F_k$, and in between moving the real scalars $\sqrt{\det G}\,G^{kl}$ past $e^{-\mathsf{T}\theta}$ and

summing over l:

$$\begin{aligned} D'^{*}D'\psi' &= -\frac{1}{\sqrt{\det G}}\sum_{k,l} D'_k\Big(\sqrt{\det G}\,G^{kl}\,\big(D'_l\psi'\big)\Big) && \text{by (28.16)}\\ &= -\frac{1}{\sqrt{\det G}}\sum_{k,l} D'_k\Big(\sqrt{\det G}\,G^{kl}\,\big(e^{-\mathsf{T}\theta}D_l\psi\big)\Big) && F=\psi\\ &= -\frac{1}{\sqrt{\det G}}\sum_{k} D'_k\big(e^{-\mathsf{T}\theta}F_k\big) && \text{reorder}\\ &= -\frac{1}{\sqrt{\det G}}\sum_{k} e^{-\mathsf{T}\theta}D_kF_k = e^{-\mathsf{T}\theta}\,D^*D\psi && F=F_k. \end{aligned}$$

Every term of (28.16) is multiplied by $e^{-\mathsf{T}\theta}$ under a change of frame, so the equation holds in every gauge if it holds in one.

The source condition is (5.6) [$U + \Delta t\,\nabla^*\nabla U = 0$ off s, $U(s) = U_s$]: $\psi^0 = 0$, and the value $\psi(s) = \psi_s$ is prescribed at the source.

28.7. Stage 2: the function spaces

What we want. To say where the solution lives, and why the problem has exactly one solution there. The finite element method will look for an approximate solution in a finite dimensional space, and it can only be good if that space sits inside the space of the true solution and the problem is well posed there. We state the facts plainly, without the proofs of the big theorems, and prove the small ones.

Square integrable fields. The space L^2 consists of the tangent fields of finite total squared length, $\int_\Omega |\psi|^2\,dA < \infty$, with the inner product and the norm

$$(\psi,\chi)_0 = \int_\Omega \langle \psi\widetilde{\chi}\rangle\,dA, \qquad \|\psi\|_0 = (\psi,\psi)_0^{1/2}.$$

The integrand is gauge invariant (Section 28.4), so the inner product is the same in every chart, and the space is defined on the whole surface. It is positive: for $\psi = u_1 + u_2\mathsf{T}$, $\widetilde{\psi} = u_1 - u_2\mathsf{T}$ and

$$\psi\widetilde{\psi} = u_1^2 - u_1u_2\mathsf{T} + u_2u_1\mathsf{T} - u_2^2\mathsf{T}^2 = u_1^2 + u_2^2 = |\psi|^2.$$

Fields with square integrable covariant derivatives. Write the derivative part of the energy as the form

$$b(\psi,\chi) = \int_\Omega \sum_{k,l} G^{kl}\big\langle (D_k\psi)\,\widetilde{(D_l\chi)}\big\rangle\,dA = \int_\Omega \sum_{m} \big\langle (D_{e_m}\psi)\,\widetilde{(D_{e_m}\chi)}\big\rangle\,dA,$$

the two expressions being equal by (28.6). The second shows that $b(\psi,\psi) = \int \sum_m |D_{e_m}\psi|^2\,dA \ge 0$. The Sobolev space H^1 consists of the fields of L^2 with $b(\psi,\psi) < \infty$, with the inner product and the norm

$$(\psi,\chi)_1 = (\psi,\chi)_0 + b(\psi,\chi), \qquad \|\psi\|_1 = (\psi,\psi)_1^{1/2}, \qquad \|\psi\|_1^2 = \|\psi\|_0^2 + b(\psi,\psi).$$

Here the derivatives are taken in the integrated ("weak") sense: a field that is smooth on each face of a mesh and has kinks along the edges belongs to H^1, as the tent function $1 - |x|$ does in one variable. H^1 is a *Hilbert space*:

an inner product space in which every sequence of fields that gets closer and closer to itself, in the norm $\|\cdot\|_1$, has a limit in the space. That completeness is what the theorems below need, and it holds because the derivatives are taken in the integrated sense; it is the reason for working in H^1 rather than with smooth fields.

The forms of the problem. Stages 3 and 4 will produce the *bilinear form* and the *linear form*

$$\begin{aligned} a(\psi,\chi) &= \int_\Omega \langle \psi\widetilde{\chi}\rangle \, dA + \Delta t \int_\Omega \sum_{k,l} G^{kl} \big\langle (D_k\psi)\,\widetilde{(D_l\chi)}\big\rangle \, dA \\ &= (\psi,\chi)_0 + \Delta t\, b(\psi,\chi), \\ l(\chi) &= \int_\Omega \langle \psi^0\widetilde{\chi}\rangle \, dA = (\psi^0,\chi)_0, \end{aligned} \tag{28.17}$$

and the problem: find ψ in H^1 with $a(\psi,\chi) = l(\chi)$ for every χ in H^1. We check three properties of a.

Symmetric. The scalar part does not change under reversion, $\langle A\rangle = \langle \widetilde{A}\rangle$, so with $\widetilde{AB} = \widetilde{B}\widetilde{A}$ and $\widetilde{\widetilde{\psi}} = \psi$,

$$\langle \psi\widetilde{\chi}\rangle = \langle \widetilde{\psi}\widetilde{\chi}\rangle = \langle \widetilde{\chi}\,\widetilde{\psi}\rangle = \langle \chi\widetilde{\psi}\rangle.$$

The same with $D_k\psi$ and $D_l\chi$ gives $\langle (D_k\psi)\widetilde{(D_l\chi)}\rangle = \langle (D_l\chi)\widetilde{(D_k\psi)}\rangle$; exchanging the names $k \leftrightarrow l$ of the summation indices and using $G^{lk} = G^{kl}$, $b(\psi,\chi) = b(\chi,\psi)$. So $a(\psi,\chi) = a(\chi,\psi)$.

Bounded. The Cauchy–Schwarz inequality holds for every symmetric bilinear form that is nonnegative on the diagonal, so for $(\cdot,\cdot)_0$ and for b. Then, one step per line:

$$\begin{aligned} |a(\psi,\chi)| &\le |(\psi,\chi)_0| + \Delta t\, |b(\psi,\chi)| && (1)\\ &\le \|\psi\|_0 \|\chi\|_0 + \Delta t\, b(\psi,\psi)^{1/2}\, b(\chi,\chi)^{1/2} && (2)\\ &\le M\left(\|\psi\|_0\|\chi\|_0 + b(\psi,\psi)^{1/2}\, b(\chi,\chi)^{1/2}\right) && (3)\\ &\le M\left(\|\psi\|_0^2 + b(\psi,\psi)\right)^{1/2}\left(\|\chi\|_0^2 + b(\chi,\chi)\right)^{1/2} && (4)\\ &= M\,\|\psi\|_1\,\|\chi\|_1, \qquad M = \max(1,\Delta t), \end{aligned}$$

by (1) the triangle inequality, (2) Cauchy–Schwarz for each form, (3) $1 \le M$ and $\Delta t \le M$, and (4) Cauchy–Schwarz in the plane, $pr + qs \le \sqrt{p^2+q^2}\sqrt{r^2+s^2}$, with $p = \|\psi\|_0$, $q = b(\psi,\psi)^{1/2}$, $r = \|\chi\|_0$, $s = b(\chi,\chi)^{1/2}$.

Coercive. With $\chi = \psi$ both terms are nonnegative:

$$a(\psi,\psi) = \|\psi\|_0^2 + \Delta t\, b(\psi,\psi) \ge \min(1,\Delta t)\left(\|\psi\|_0^2 + b(\psi,\psi)\right) = \min(1,\Delta t)\,\|\psi\|_1^2.$$

The three together say that a is itself an inner product on H^1, equivalent to $(\cdot,\cdot)_1$: the *energy inner product*, with the *energy norm* $\|\psi\|_a = a(\psi,\psi)^{1/2}$. And l is bounded in this norm whenever ψ^0 is in L^2:

$$|l(\chi)| = |(\psi^0,\chi)_0| \le \|\psi^0\|_0\,\|\chi\|_0 \le \|\psi^0\|_0\,\|\chi\|_1 \le \|\psi^0\|_0\,\min(1,\Delta t)^{-1/2}\,\|\chi\|_a,$$

by Cauchy–Schwarz, by $\|\chi\|_0^2 \le \|\chi\|_0^2 + b(\chi,\chi) = \|\chi\|_1^2$, and by coercivity.

What the theorems say, and why they apply. The *Riesz representation theorem* says that, in a Hilbert space, every bounded linear form is the inner product with exactly one element. Applied to l with the energy inner product, it gives exactly one ψ in H^1 with $a(\psi,\chi) = l(\chi)$ for every χ: the weak problem has a unique solution. (The Lax–Milgram theorem says the same for forms that are bounded and coercive but not symmetric; ours is symmetric, so Riesz suffices.)

Now take a subspace V_h of H^1, the finite element space of Stage 5; it is finite dimensional, hence closed. The *Galerkin solution* is the ψ_h in V_h with $a(\psi_h,\chi_h) = l(\chi_h)$ for every χ_h in V_h; it exists and is unique by Riesz in V_h. Since $V_h \subset H^1$, the continuous equation holds in particular for $\chi = \chi_h$. Subtract:

$$a(\psi,\chi_h) - a(\psi_h,\chi_h) = l(\chi_h) - l(\chi_h) = 0, \qquad \text{so} \qquad a(\psi-\psi_h,\chi_h) = 0$$

by linearity in the first argument: the error is orthogonal to V_h in the energy inner product. For any χ_h in V_h write $\psi - \chi_h = (\psi-\psi_h) + (\psi_h - \chi_h)$, where $\psi_h - \chi_h$ is in V_h. Then

$$\begin{aligned}
\|\psi-\chi_h\|_a^2 &= a\big((\psi-\psi_h)+(\psi_h-\chi_h),\ (\psi-\psi_h)+(\psi_h-\chi_h)\big) && \text{subst.}\\
&= \|\psi-\psi_h\|_a^2 + 2\,a(\psi-\psi_h,\psi_h-\chi_h) + \|\psi_h-\chi_h\|_a^2 && \text{expand}\\
&= \|\psi-\psi_h\|_a^2 + \|\psi_h-\chi_h\|_a^2 && \text{orthogonality}\\
&\ge \|\psi-\psi_h\|_a^2,
\end{aligned}$$

the expansion using that a is bilinear and symmetric, as in Stage 3, with equality for $\chi_h = \psi_h$. So

$$\|\psi-\psi_h\|_a = \min_{\chi_h\in V_h} \|\psi-\chi_h\|_a,$$

the best approximation in the energy norm. This is *Céa's lemma* in its symmetric form. It reduces the accuracy of the method to a question of approximation: how well can fields of V_h approximate the solution?

Why the weak form needs less. The strong form (28.16) contains second derivatives, and asks for ψ in H^2, the space with square integrable second covariant derivatives. A field with kinks along the edges is not in H^2: its first derivatives jump, and the derivative of a jump is not a function. The weak form contains only first derivatives and asks only for H^1. This is what allows piecewise linear fields.

Conforming spaces. The theorems above need $V_h \subset H^1$, a *conforming* space. For fields that are smooth on each face, this means one thing: continuity across every edge. A field that jumps along an edge has a derivative containing a "wall" concentrated on the edge, with infinite energy. In gauge language, continuity across the edge between two faces f and g means: the coordinate of the field in the chart of f, along the edge, carried into the chart of g by the transition (28.12) between the two face charts, equals the coordinate of the field in the chart of g along the same edge. Section 31 checks where the space of this part meets this requirement exactly (on every intrinsically flat

part of a mesh) and where it meets it only approximately (at curved vertices, with a mismatch proportional to the angle defect).

The assumptions, plainly. The surface is smooth and closed, or bounded with a boundary on which the test fields vanish; the fields are tangent; $\Delta t > 0$; ψ^0 is in L^2. One more point. In two dimensions a field of H^1 need not have a value at a point. Take, in a flat chart ($G = I$) with polar coordinates (r, ϑ), the radial function $f(r) = \log(-\log r)$ for $0 < r \le 1/e$; it is 0 at $r = 1/e$ and grows without bound as $r \to 0$. Its derivative is, by the chain rule,

$$f'(r) = \frac{1}{-\log r} \cdot \Big(-\frac{1}{r} \Big) = \frac{1}{r \log r},$$

and its energy, with $dA = r\, dr\, d\vartheta$ and the substitution $u = \log r$, $du = dr/r$, which takes $r \in (0, 1/e]$ to $u \in (-\infty, -1]$, is

$$\begin{aligned}
\int_0^{2\pi} \int_0^{1/e} \big(f'(r)\big)^2\, r\, dr\, d\vartheta &= 2\pi \int_0^{1/e} \frac{1}{r^2 \log^2 r}\, r\, dr && \text{integrate in } \vartheta \\
&= 2\pi \int_0^{1/e} \frac{1}{\log^2 r}\, \frac{dr}{r} && \text{cancel one } r \\
&= 2\pi \int_{-\infty}^{-1} \frac{du}{u^2} && u = \log r \\
&= 2\pi \Big[-\frac{1}{u} \Big]_{-\infty}^{-1} = 2\pi\,(1 - 0) = 2\pi,
\end{aligned}$$

finite, and yet f is unbounded. So the point condition $\psi(s) = \psi_s$ is not a constraint that the theory of H^1 can impose. It is a discrete device, used by Part I as well: after discretization a field does have values at the vertices, and fixing one of them is a well defined row replacement (Stage 8). The continuous counterpart is the source term of Section 5, and the directions, which are all we keep, agree to the accuracy of the method.

28.8. Stage 3: the variational form

What we want. The energy whose minimizer solves the PDE. It is the energy (7.1) of Part I, shown again:

$$J(U) = \frac{1}{2} \int_\Omega |U|^2\, dA + \frac{\Delta t}{2} \int_\Omega |\nabla U|^2\, dA - \int_\Omega U^0 \cdot U\, dA. \qquad (7.1)$$

Substitute $|U|^2 = \langle \psi \widetilde{\psi} \rangle$ and $U^0 \cdot U = \langle \psi^0 \widetilde{\psi} \rangle$, by (19.1), and $|\nabla U|^2 = \nabla U : \nabla U$ from (28.11) with $\chi = \psi$:

$$\begin{aligned}
J(\psi) = \frac{1}{2} \int_\Omega \big(\langle \psi \widetilde{\psi} \rangle \big)\, dA + \frac{\Delta t}{2} \int_\Omega \Big(\sum_{k,l} G^{kl} \big\langle (D_k \psi)\, \widetilde{(D_l \psi)} \big\rangle \Big)\, dA \\
- \int_\Omega \big(\langle \psi^0 \widetilde{\psi} \rangle \big)\, dA = \frac{1}{2}\, a(\psi, \psi) - l(\psi),
\end{aligned} \qquad (28.18)$$

the last form by (28.17). It is gauge invariant, because every density in it is (Section 28.4): the energy is a property of the field, not of the frames.

The first variation, term by term. Perturb ψ by $\epsilon\chi$, with a real number ϵ and a test field χ, and expand each of the three densities. The covariant derivative is linear, $D_k(\psi+\epsilon\chi) = D_k\psi + \epsilon\, D_k\chi$, because both ∂_k and the multiplication by $\omega_k\mathsf{T}$ are; and the reverse is linear, $\widetilde{\psi+\epsilon\chi} = \widetilde{\psi} + \epsilon\widetilde{\chi}$.

The mass density.

$$\begin{aligned}\big\langle(\psi+\epsilon\chi)\,\widetilde{(\psi+\epsilon\chi)}\big\rangle &= \big\langle(\psi+\epsilon\chi)(\widetilde{\psi}+\epsilon\widetilde{\chi})\big\rangle && \text{reverse is linear}\\ &= \langle\psi\widetilde{\psi}\rangle + \epsilon\,\langle\psi\widetilde{\chi}\rangle + \epsilon\,\langle\chi\widetilde{\psi}\rangle + \epsilon^2\langle\chi\widetilde{\chi}\rangle && \text{distribute}\\ &= \langle\psi\widetilde{\psi}\rangle + 2\epsilon\,\langle\psi\widetilde{\chi}\rangle + \epsilon^2\langle\chi\widetilde{\chi}\rangle && \langle\chi\widetilde{\psi}\rangle = \langle\psi\widetilde{\chi}\rangle.\end{aligned}$$

The derivative density. Write $\Psi_k = D_k\psi$ and $X_k = D_k\chi$ for short. Then

$$\begin{aligned}&\sum_{k,l} G^{kl}\big\langle(\Psi_k+\epsilon X_k)\,\widetilde{(\Psi_l+\epsilon X_l)}\big\rangle\\ &\quad= \sum_{k,l} G^{kl}\Big(\langle\Psi_k\widetilde{\Psi}_l\rangle + \epsilon\,\langle\Psi_k\widetilde{X}_l\rangle + \epsilon\,\langle X_k\widetilde{\Psi}_l\rangle + \epsilon^2\langle X_k\widetilde{X}_l\rangle\Big) && \text{distribute}\\ &\quad= \sum_{k,l} G^{kl}\langle\Psi_k\widetilde{\Psi}_l\rangle + 2\epsilon\sum_{k,l} G^{kl}\langle\Psi_k\widetilde{X}_l\rangle + \epsilon^2\sum_{k,l} G^{kl}\langle X_k\widetilde{X}_l\rangle,\end{aligned}$$

where the two middle sums are equal: $\langle X_k\widetilde{\Psi}_l\rangle = \langle\Psi_l\widetilde{X}_k\rangle$ by symmetry, and then exchanging the names $k \leftrightarrow l$ and using $G^{lk} = G^{kl}$ turns $\sum_{k,l} G^{kl}\langle\Psi_l\widetilde{X}_k\rangle$ into $\sum_{k,l} G^{kl}\langle\Psi_k\widetilde{X}_l\rangle$.

The source density. $\big\langle\psi^0\,\widetilde{(\psi+\epsilon\chi)}\big\rangle = \langle\psi^0\widetilde{\psi}\rangle + \epsilon\,\langle\psi^0\widetilde{\chi}\rangle$.

Collect. Put the three expansions into (28.18), with the factors $\frac{1}{2}$, $\frac{\Delta t}{2}$ and -1, integrate, and group by powers of ϵ:

$$\begin{aligned}J(\psi+\epsilon\chi) = \underbrace{\frac{1}{2}\int\langle\psi\widetilde{\psi}\rangle + \frac{\Delta t}{2}\int\sum_{k,l} G^{kl}\langle\Psi_k\widetilde{\Psi}_l\rangle - \int\langle\psi^0\widetilde{\psi}\rangle}_{J(\psi)}\\ + \epsilon\Big(\int\langle\psi\widetilde{\chi}\rangle + \Delta t\int\sum_{k,l} G^{kl}\langle\Psi_k\widetilde{X}_l\rangle - \int\langle\psi^0\widetilde{\chi}\rangle\Big)\\ + \frac{\epsilon^2}{2}\Big(\int\langle\chi\widetilde{\chi}\rangle + \Delta t\int\sum_{k,l} G^{kl}\langle X_k\widetilde{X}_l\rangle\Big),\end{aligned}$$

all integrals over Ω with dA. The factor 2 of the middle terms cancelled the $\frac{1}{2}$ in front. In the notation (28.17):

$$J(\psi+\epsilon\chi) = J(\psi) + \epsilon\big(a(\psi,\chi) - l(\chi)\big) + \frac{\epsilon^2}{2}\,a(\chi,\chi). \tag{28.19}$$

Minimizing is solving. The coefficient of ϵ is the rate of change of J in the direction χ, $\frac{d}{d\epsilon}J(\psi+\epsilon\chi)\big|_{\epsilon=0}$. At a minimizer it vanishes for every χ, which is $a(\psi,\chi) = l(\chi)$ for every χ: the weak problem of Stage 2. Conversely, if $a(\psi,\chi) = l(\chi)$ for every χ, then (28.19) reads $J(\psi+\epsilon\chi) = J(\psi) + \frac{\epsilon^2}{2}a(\chi,\chi)$, and $a(\chi,\chi) \geq \min(1,\Delta t)\|\chi\|_1^2 > 0$ for $\chi \neq 0$ by coercivity, so every other field

has a larger energy: ψ is the unique minimizer. Stage 4 shows, with Green's identity, that for a smooth ψ the coefficient of ϵ equals $\int_\Omega \langle (\psi + \Delta t\, D^*D\psi - \psi^0)\, \widetilde{\chi}\rangle\, dA$ plus a boundary term, and that its vanishing for every χ is the strong form (28.16). So the three are one problem: minimize J, solve the weak form, solve the PDE.

28.9. Stage 4: the weak form (Galerkin)

What we want. An equation with first derivatives only. As in Part I, multiply the strong form by a test field, integrate, and move one derivative onto the test field. The tool for the last step is Green's identity, which we now derive in the chart, with flat calculus only: this is the promise of the gauge picture.

Step 1: test the strong form. For a test field χ, multiply (28.16) $[\,\psi + \Delta t\, D^*D\psi = \psi^0\,]$ by $\widetilde{\chi}$ on the right, take the scalar part, which is linear, and integrate:

$$\int_\Omega \langle \psi\widetilde{\chi}\rangle\, dA + \Delta t \int_\Omega \big\langle (D^*D\psi)\, \widetilde{\chi}\big\rangle\, dA = \int_\Omega \langle \psi^0\widetilde{\chi}\rangle\, dA. \tag{28.20}$$

Step 2: the product rule for the scalar part. We want the derivative of the density $\langle\psi\widetilde{\chi}\rangle$ in terms of covariant derivatives. The ordinary product rule, with the reverse and the scalar part commuting with ∂_k (they act on the components, not on ξ), gives

$$\partial_k\langle\psi\widetilde{\chi}\rangle = \big\langle (\partial_k\psi)\, \widetilde{\chi}\big\rangle + \big\langle \psi\, \widetilde{(\partial_k\chi)}\big\rangle.$$

To turn ∂_k into D_k we must add the two connection terms, and we check that their sum is zero. The reverse of $\omega_k \mathsf{T}\chi$ is $\omega_k\widetilde{\chi}\,\widetilde{\mathsf{T}} = -\omega_k\,\widetilde{\chi}\,\mathsf{T}$, so

$$\begin{aligned}
&\big\langle (\omega_k\mathsf{T}\psi)\, \widetilde{\chi}\big\rangle + \big\langle \psi\, \widetilde{(\omega_k\mathsf{T}\chi)}\big\rangle \\
&\quad = \omega_k \big\langle \mathsf{T}\psi\widetilde{\chi}\big\rangle + \big\langle \psi\, \big(-\omega_k\,\widetilde{\chi}\,\mathsf{T}\big)\big\rangle && \text{reverse} \\
&\quad = \omega_k \big\langle \mathsf{T}\,(\psi\widetilde{\chi})\big\rangle - \omega_k \big\langle (\psi\widetilde{\chi})\,\mathsf{T}\big\rangle && \omega_k \text{ out} \\
&\quad = \omega_k \big\langle \mathsf{T}\,(\psi\widetilde{\chi})\big\rangle - \omega_k \big\langle \mathsf{T}\,(\psi\widetilde{\chi})\big\rangle = 0 && \langle AB\rangle = \langle BA\rangle.
\end{aligned}$$

Adding this zero to the product rule and grouping each connection term with its derivative:

$$\partial_k\langle\psi\widetilde{\chi}\rangle = \big\langle (D_k\psi)\, \widetilde{\chi}\big\rangle + \big\langle \psi\, \widetilde{(D_k\chi)}\big\rangle. \tag{28.21}$$

This is the metric compatibility (4.4) $[\,X(U\cdot V) = \nabla_X U\cdot V + U\cdot\nabla_X V\,]$ of Part I, in gauge form: the rotation gauge is a rotation, and rotations keep inner products.

Step 3: a flat vector field and its divergence. In one variable, $\int f'g' = [f'g] - \int f''g$ comes from integrating the product rule of $f'g$. Here the analogue of $f'g$ is the flat vector field $W = (W_1, W_2)$ with

$$W_k = \big\langle F_k\, \widetilde{\chi}\big\rangle, \qquad F_k = \sqrt{\det G}\, \sum_l G^{kl}\, D_l\psi.$$

Apply (28.21) with F_k in place of ψ:

$$\partial_k W_k = \partial_k\big\langle F_k\, \widetilde{\chi}\big\rangle = \big\langle (D_kF_k)\, \widetilde{\chi}\big\rangle + \big\langle F_k\, \widetilde{(D_k\chi)}\big\rangle.$$

Sum over k, substitute F_k in the second term, move the real scalars $\sqrt{\det G}$ and G^{kl} out of the scalar part, and finally exchange the names $k \leftrightarrow l$ of the summation indices, using $G^{lk} = G^{kl}$:

$$\begin{aligned}
&\partial_1 W_1 + \partial_2 W_2 \\
&\quad = \sum_k \big\langle (D_k F_k)\, \widetilde{\chi} \big\rangle + \sum_k \Big\langle \Big(\sqrt{\det G} \sum_l G^{kl} D_l \psi \Big)\, \widetilde{(D_k \chi)} \Big\rangle \qquad \text{subst.} \\
&\quad = \Big\langle \Big(\sum_k D_k F_k \Big)\, \widetilde{\chi} \Big\rangle + \sqrt{\det G} \sum_{k,l} G^{kl} \big\langle (D_l \psi)\, \widetilde{(D_k \chi)} \big\rangle \qquad \text{scalars out} \\
&\quad = \Big\langle \Big(\sum_k D_k F_k \Big)\, \widetilde{\chi} \Big\rangle + \sqrt{\det G} \sum_{k,l} G^{kl} \big\langle (D_k \psi)\, \widetilde{(D_l \chi)} \big\rangle \qquad k \leftrightarrow l,
\end{aligned}$$

By the definition (28.16) of D^*D, multiplied by $-\sqrt{\det G}$,

$$\sum_k D_k F_k = \sum_{k,l} D_k \Big(\sqrt{\det G}\; G^{kl}\, D_l \psi \Big) = -\sqrt{\det G}\; D^* D \psi.$$

Substitute it into the first term, and recognize the density (28.11) in the second:

$$\partial_1 W_1 + \partial_2 W_2 = -\sqrt{\det G}\, \big\langle (D^* D \psi)\, \widetilde{\chi} \big\rangle + \sqrt{\det G} \sum_{k,l} G^{kl} \big\langle (D_k \psi)\, \widetilde{(D_l \chi)} \big\rangle. \tag{28.22}$$

Step 4: the divergence theorem in the flat chart. The divergence theorem of the plane, (5.2) with the flat metric, says

$$\int_{P'} (\partial_1 W_1 + \partial_2 W_2)\, d\xi_1\, d\xi_2 = \oint_{\partial P'} W \cdot \hat{\nu}\, d\hat{s},$$

where $\hat{\nu}$ and $d\hat{s}$ are the outward normal and the length element of the flat boundary. Substitute (28.22) on the left:

$$\begin{aligned}
\int_{P'} \Big(-\sqrt{\det G}\, \big\langle (D^* D \psi)\, \widetilde{\chi} \big\rangle + \sqrt{\det G} \sum_{k,l} G^{kl} \big\langle (D_k \psi)\, \widetilde{(D_l \chi)} \big\rangle \Big)\, d\xi_1 d\xi_2 \\
= \oint_{\partial P'} W \cdot \hat{\nu}\, d\hat{s}.
\end{aligned}$$

Split the integral in two, move the term with D^*D to the right, and use (28.3), $\sqrt{\det G}\, d\xi_1 d\xi_2 = dA$:

$$\int \sum_{k,l} G^{kl} \big\langle (D_k \psi)\, \widetilde{(D_l \chi)} \big\rangle\, dA = \int \big\langle (D^* D \psi)\, \widetilde{\chi} \big\rangle\, dA + \oint_{\partial P'} W \cdot \hat{\nu}\, d\hat{s}. \tag{28.23}$$

This is Green's identity (5.3) of Part I,

$$\int_\Omega \nabla U : \nabla V\, dA = \int_\Omega (\nabla^* \nabla U) \cdot V\, dA + \oint_{\partial \Omega} (\nabla_\nu U) \cdot V\, ds,$$

derived in a flat plane. It also derives the formula of D^*D in (28.16): that is the operator for which the identity holds.

The boundary term is the flux of the surface. We check that $W \cdot \hat{\nu}\, d\hat{s} = \langle (D_\nu \psi)\, \widetilde{\chi} \rangle\, ds$, with ν the outward unit conormal of the surface and ds its length element. Walk the boundary counterclockwise with flat unit tangent $\hat{\tau} = (\tau_1, \tau_2)$; the outward flat normal is $\hat{\nu} = (\tau_2, -\tau_1) = R\hat{\tau}$ with $R = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$. The flat step $\hat{\tau}\, d\hat{s}$ is the surface step $h(\hat{\tau})\, d\hat{s}$, so $ds = |h(\hat{\tau})|\, d\hat{s} = (\hat{\tau}^T G \hat{\tau})^{1/2}\, d\hat{s}$. The flat vector $c = \lambda G^{-1}\hat{\nu}$, $\lambda > 0$, gives a surface vector $h(c)$ orthogonal to the boundary, since $g(c, \hat{\tau}) = \lambda\, \hat{\nu}^T G^{-1} G\, \hat{\tau} = \lambda\, \hat{\nu} \cdot \hat{\tau} = 0$, and pointing out, since $\hat{\nu} \cdot c = \lambda\, \hat{\nu}^T G^{-1} \hat{\nu} > 0$; it is a unit vector when $c^T G c = \lambda^2\, \hat{\nu}^T G^{-1} \hat{\nu} = 1$. So $\nu = h(c)$ with $\lambda = (\hat{\nu}^T G^{-1} \hat{\nu})^{-1/2}$, and by the linearity of D_a in a,

$$D_\nu \psi = \sum_l c_l\, D_l \psi = \lambda \sum_{k,l} \hat{\nu}_k\, G^{kl}\, D_l \psi.$$

For a symmetric 2×2 matrix $G = \left(\begin{smallmatrix} p & q \\ q & r \end{smallmatrix}\right)$, $G^{-1} = \frac{1}{\det G} \left(\begin{smallmatrix} r & -q \\ -q & p \end{smallmatrix}\right)$, and multiplying out, $R^T \left(\begin{smallmatrix} r & -q \\ -q & p \end{smallmatrix}\right) R = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right) \left(\begin{smallmatrix} q & r \\ -p & -q \end{smallmatrix}\right) = \left(\begin{smallmatrix} p & q \\ q & r \end{smallmatrix}\right) = G$. Hence $\hat{\nu}^T G^{-1} \hat{\nu} = \hat{\tau}^T R^T G^{-1} R\, \hat{\tau} = \hat{\tau}^T G \hat{\tau} / \det G$, and

$$\lambda\, ds = \left(\frac{\det G}{\hat{\tau}^T G \hat{\tau}} \right)^{1/2} \left(\hat{\tau}^T G \hat{\tau} \right)^{1/2} d\hat{s} = \sqrt{\det G}\, d\hat{s}.$$

Put the pieces together:

$$\left\langle (D_\nu \psi)\, \widetilde{\chi} \right\rangle ds = \sum_k \hat{\nu}_k \left\langle \left(\sqrt{\det G} \sum_l G^{kl} D_l \psi \right) \widetilde{\chi} \right\rangle d\hat{s} = \sum_k \hat{\nu}_k\, W_k\, d\hat{s} = W \cdot \hat{\nu}\, d\hat{s}.$$

So the boundary integral of (28.23) is $\oint \langle (D_\nu \psi) \widetilde{\chi} \rangle\, ds$, a gauge invariant quantity of the surface. To integrate over the whole surface, cut it into pieces each inside one chart and add the identities: on a cut between two pieces the integrand is the same and the two outward conormals are opposite, so the two boundary terms cancel, and only the true boundary $\partial\Omega$ of the surface remains.

Step 5: substitute. Solve (28.23), summed over the pieces, for the term with $D^* D$:

$$\int_\Omega \left\langle (D^* D \psi)\, \widetilde{\chi} \right\rangle dA = \int_\Omega \sum_{k,l} G^{kl} \left\langle (D_k \psi)\, \widetilde{(D_l \chi)} \right\rangle dA - \oint_{\partial\Omega} \left\langle (D_\nu \psi)\, \widetilde{\chi} \right\rangle ds.$$

Substitute it, inside parentheses, into (28.20):

$$\int_\Omega \langle \psi \widetilde{\chi} \rangle\, dA + \Delta t \left(\int_\Omega \sum_{k,l} G^{kl} \left\langle (D_k \psi)\, \widetilde{(D_l \chi)} \right\rangle dA - \oint_{\partial\Omega} \left\langle (D_\nu \psi)\, \widetilde{\chi} \right\rangle ds \right) = \int_\Omega \langle \psi^0 \widetilde{\chi} \rangle\, dA.$$

Distribute Δt:

$$\int_\Omega \langle \psi \widetilde{\chi} \rangle\, dA + \Delta t \int_\Omega \sum_{k,l} G^{kl} \left\langle (D_k \psi)\, \widetilde{(D_l \chi)} \right\rangle dA - \Delta t \oint_{\partial\Omega} \left\langle (D_\nu \psi)\, \widetilde{\chi} \right\rangle ds = \int_\Omega \langle \psi^0 \widetilde{\chi} \rangle\, dA.$$

The boundary term vanishes: the surfaces of our experiments are closed ($\partial\Omega$ is empty), or the test fields vanish on the boundary where needed and at the source, where the value is prescribed. What remains is the *weak form*: find ψ with

$$\boxed{\int_\Omega \langle \psi\widetilde{\chi}\rangle \, dA + \Delta t \int_\Omega \sum_{k,l} G^{kl} \big\langle (D_k\psi)\, \widetilde{(D_l\chi)} \big\rangle \, dA = \int_\Omega \langle \psi^0 \widetilde{\chi}\rangle \, dA} \qquad (28.24)$$

that is, $a(\psi,\chi) = l(\chi)$ in the notation (28.17), for every test field χ of H^1 that vanishes at the source. It is the weak form (21.5) of Part II, shown again,

$$\int_\Omega \langle \psi\, \widetilde{\chi}\rangle \, dA + \Delta t \int_\Omega \sum_m \langle (D_{e_m}\psi)\, \widetilde{(D_{e_m}\chi)}\rangle \, dA = \int_\Omega \langle \psi^0\, \widetilde{\chi}\rangle \, dA, \qquad (21.5)$$

with the sum over an orthonormal basis of the surface replaced by the inverse metric of the chart, by (28.6) with $X(e) = D_e\psi$, $Y(e) = D_e\chi$ and the product $\langle X\widetilde{Y}\rangle$: $\sum_m \langle (D_{e_m}\psi)\widetilde{(D_{e_m}\chi)}\rangle = \sum_{k,l} G^{kl}\langle (D_k\psi)\widetilde{(D_l\chi)}\rangle$. Galerkin's idea, used in Stage 6, is to ask (28.24) only for the test fields of a finite dimensional space.

Step 6: back to the PDE. The coefficient of ϵ in (28.19) is the left side minus the right side of the weak form. For a smooth ψ, the lines of Step 5 run backwards and give

$$a(\psi,\chi) - l(\chi) = \int_\Omega \langle r\, \widetilde{\chi}\rangle \, dA, \qquad r = \psi + \Delta t\, D^*D\psi - \psi^0,$$

for every χ that vanishes on $\partial\Omega$. If this is zero for every such χ, choose $\chi = \eta\, r$ with a smooth function $\eta \geq 0$ that vanishes on the boundary and near the source. Then $\langle r\, \widetilde{\chi}\rangle = \eta\, \langle r\widetilde{r}\rangle = \eta\, |r|^2$, by the positivity computed in Stage 2, and

$$0 = \int_\Omega \eta\, |r|^2 \, dA.$$

The integrand is continuous and nonnegative, so it is zero everywhere, and $r = 0$ wherever $\eta > 0$. Since η can be positive at any point away from the boundary and the source, the strong form (28.16) holds there. For smooth fields, minimizing J, solving the weak form and solving the PDE are the same problem.

What we got, and what Stage 5 must supply. The weak form (28.24) contains exactly three geometric ingredients: the metric, through G^{kl} and $\sqrt{\det G}$; the rotation gauge, through D_k; and, since a surface needs several charts, the transitions (28.12) between them. Nothing else of the surface enters. Stage 5 supplies the three on a mesh: on each face, a constant metric from three lengths and a flat gauge, $\Omega = 0$ (Section 29); at each vertex, a flat chart to hold the unknown (Section 30); and between each vertex chart and each face chart, a transition rotation, which carries the curvature (Section 31).

29. Stage 5, the Faces: Flat Charts Known Only by Their Edge Lengths

What we want. Stage 5 is the discretization: the mesh and the finite dimensional space. Stage 4 ended with three ingredients that Stage 5 must supply: the metric, the rotation gauge and the transitions between charts. This section supplies the first two on each face. The chart of a face is the reference triangle, or the layout of the face in the plane, and its displacement gauge is the same at every point of the face. We want to show two things. First, everything the finite element method needs from a face, other than the transitions, is a function of the three edge lengths, and we compute it from them. Second, in the chart of a face the rotation gauge vanishes.

29.1. The metric of a face is its Gram matrix

Recall from Section 12 the affine map from the reference triangle to the face,

$$F(\xi) = x_1 + B\,\xi, \qquad B = [\,x_2 - x_1 \;\; x_3 - x_1\,] = [\,e_1 \;\; e_2\,],$$

with B a 3×2 matrix. In the language of Section 28.2, the reference triangle is the chart of the face and B is its displacement gauge: the map (28.1) $[\,h(a) = a_1\,\partial x/\partial\xi_1 + a_2\,\partial x/\partial\xi_2\,]$ is $h(a) = Ba$, since $\partial F/\partial\xi_1 = e_1$ and $\partial F/\partial\xi_2 = e_2$. So $h = B$, constant on the face. The matrix B contains the full position of the face in space. We ask what part of B is needed to measure lengths on the face.

Lengths and angles. A small step $d\xi$ in the reference triangle is carried by F to the step $B\,d\xi$ on the face. Its squared length is the inner product of the vector with itself, $|v|^2 = v^T v$, and the transpose of a product reverses the order, $(B\,d\xi)^T = d\xi^T B^T$. So

$$\begin{aligned} |B\,d\xi|^2 &= (B\,d\xi)^T(B\,d\xi) && (|v|^2 = v^T v \text{ with } v = B\,d\xi)\\ &= (d\xi^T B^T)(B\,d\xi) && ((B\,d\xi)^T = d\xi^T B^T)\\ &= d\xi^T(B^T B)\,d\xi && \text{(matrix products are associative).} \end{aligned}$$

The same three lines for two steps $d\xi$ and $d\eta$ give their inner product on the face,

$$(B\,d\xi)\cdot(B\,d\eta) = (B\,d\xi)^T(B\,d\eta) = d\xi^T(B^T B)\,d\eta,$$

and with the lengths and the inner product we also have the angle between the two steps. So the 2×2 matrix $B^T B$ is all we need to measure lengths and angles on the face, in the coordinates ξ of the reference triangle.

Entry by entry. The entry (k,l) of $B^T B$ is row k of B^T times column l of B. Row k of B^T is column k of B, the edge e_k, written as a row. So the entry (k,l) is $e_k^T e_l = e_k \cdot e_l$:

$$G_f = B^T B = \begin{pmatrix} e_1^T \\ e_2^T \end{pmatrix}(e_1 \quad e_2) = \begin{pmatrix} e_1\cdot e_1 & e_1\cdot e_2 \\ e_2\cdot e_1 & e_2\cdot e_2 \end{pmatrix} = \begin{pmatrix} e_1\cdot e_1 & e_1\cdot e_2 \\ e_1\cdot e_2 & e_2\cdot e_2 \end{pmatrix}, \tag{29.1}$$

where the last step uses $e_2 \cdot e_1 = e_1 \cdot e_2$: the matrix is symmetric. In differential geometry such a rule is called a *metric*, or the *first fundamental form* of the surface in the coordinates ξ; in linear algebra G_f is the *Gram matrix* of the two edges, the matrix of their inner products. It is the metric (28.2) [$G = h^T h$] of the face chart, with $h = B$.

Rigid motions do not change it. The matrix B says more than G_f: where the face is in space and how it is turned. That extra information is exactly what the ant cannot see. Move the face rigidly, by a rotation R of space followed by a translation by a vector d. The vertices become $Rx_k + d$, so the edges become $(Rx_2 + d) - (Rx_1 + d) = R(x_2 - x_1) = Re_1$ and, in the same way, Re_2: the translation cancels in the differences, and B becomes RB. The Gram matrix becomes

$$\begin{aligned} (RB)^T(RB) &= (B^T R^T)(RB) && \text{(transpose of a product)} \\ &= B^T(R^T R)B && \text{(associativity)} \\ &= B^T I B = B^T B && (R^T R = I \text{ for a rotation}). \end{aligned}$$

So G_f is unchanged by every rigid motion: it keeps only what the ant can measure.

The metric from the three lengths. Write

$$\ell_{12} = |e_1| = |x_2 - x_1|, \qquad \ell_{13} = |e_2| = |x_3 - x_1|, \qquad \ell_{23} = |x_3 - x_2|.$$

The diagonal entries of (29.1) are squared lengths: $G_{11} = e_1 \cdot e_1 = \ell_{12}^2$ and $G_{22} = e_2 \cdot e_2 = \ell_{13}^2$. The off-diagonal entry $G_{12} = e_1 \cdot e_2$ is not a squared length, but the third edge gives it. That edge is

$$x_3 - x_2 = (x_3 - x_1) - (x_2 - x_1) = e_2 - e_1,$$

and its squared length, expanded with the distributive law of the inner product, is

$$\begin{aligned} \ell_{23}^2 &= (e_2 - e_1) \cdot (e_2 - e_1) && (|v|^2 = v \cdot v) \\ &= e_2 \cdot e_2 - e_2 \cdot e_1 - e_1 \cdot e_2 + e_1 \cdot e_1 && \text{(distribute)} \\ &= \ell_{13}^2 - 2\, e_1 \cdot e_2 + \ell_{12}^2 && (e_2 \cdot e_1 = e_1 \cdot e_2; \text{ the diagonal entries}). \end{aligned}$$

This is the law of cosines. We solve it for $e_1 \cdot e_2$, one operation per line:

$$\begin{aligned} 2\, e_1 \cdot e_2 &= \ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2 && \text{(add } 2\, e_1 \cdot e_2 - \ell_{23}^2 \text{ to both sides)} \\ e_1 \cdot e_2 &= \tfrac{1}{2}\big(\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2\big) && \text{(divide by 2).} \end{aligned}$$

Putting the three entries into (29.1),

$$G_f = \begin{pmatrix} \ell_{12}^2 & \tfrac{1}{2}(\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2) \\ \tfrac{1}{2}(\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2) & \ell_{13}^2 \end{pmatrix}. \tag{29.2}$$

The off-diagonal entry is also $e_1 \cdot e_2 = |e_1|\,|e_2| \cos\alpha_1 = \ell_{12}\ell_{13} \cos\alpha_1$, where α_1 is the angle of the face at x_1, between e_1 and e_2.

29.2. Why the Gram matrix carries everything a face needs

What does a face contribute to the finite element method, apart from the connection? Look at the weak form (28.24), shown again,

$$\int_\Omega \langle \psi \widetilde{\chi} \rangle \, dA + \Delta t \int_\Omega \sum_{k,l} G^{kl} \big\langle (D_k \psi) \, \widetilde{(D_l \chi)} \big\rangle \, dA = \int_\Omega \langle \psi^0 \widetilde{\chi} \rangle \, dA.$$

On a face it needs two things: the area element dA, which integrated over the face gives the area A_f (and the lumped mass $A_f/3$), and the sum with G^{kl}, which for the hat functions will become the inner products $\nabla\phi_a \cdot \nabla\phi_b$ of their gradients. We now show, one at a time, that both are functions of G_f alone. This is (28.3) and (28.5) for the chart of a face, done slowly.

The area. Twice the area of a triangle is the length of the cross product of two of its edges, $A_f = \frac{1}{2}|e_1 \times e_2|$. Lagrange's identity expresses that length with inner products only,

$$|e_1 \times e_2|^2 = |e_1|^2 \, |e_2|^2 - (e_1 \cdot e_2)^2.$$

The determinant of the matrix (29.1) [$G_f = \left(\begin{smallmatrix} e_1\cdot e_1 & e_1\cdot e_2 \\ e_1\cdot e_2 & e_2\cdot e_2 \end{smallmatrix}\right)$] is, by the rule $\det \left(\begin{smallmatrix} p & q \\ r & s \end{smallmatrix}\right) = ps - qr$,

$$\det G_f = (e_1 \cdot e_1)(e_2 \cdot e_2) - (e_1 \cdot e_2)(e_1 \cdot e_2) = |e_1|^2 \, |e_2|^2 - (e_1 \cdot e_2)^2,$$

the right-hand side of Lagrange's identity. So $|e_1 \times e_2|^2 = \det G_f$, $|e_1 \times e_2| = \sqrt{\det G_f}$, and

$$A_f = \tfrac{1}{2} \, |e_1 \times e_2| = \tfrac{1}{2} \sqrt{\det G_f}.$$

This is the area element (28.3) [$dA = \sqrt{\det G}\, d\xi_1 d\xi_2$] integrated over the reference triangle: $\sqrt{\det G_f}$ is constant on the face, so it comes out of the integral, and what remains is $\int_{\hat{K}} d\xi_1 d\xi_2 = \frac{1}{2}$, the flat area of the reference triangle.

The gradients. This takes a few more steps, because the reader may not have met the gradient on a surface. On a flat face, the gradient of a function ϕ is the vector g of the plane of the face with this property: if we move from a point of the face along a vector X of that plane, ϕ changes at the rate $g \cdot X$. A hat function is linear on the face, so moving along X from any point changes ϕ_a by exactly $g \cdot X$, not only approximately.

Step 1: what we know about g. The hat function on the face and the shape function on the reference triangle are the same function, written in two coordinates:

$$\phi_a(x_1 + B\xi) = \hat{\phi}_a(\xi) \qquad \text{for every } \xi.$$

Move one unit along the ξ_1 axis, from ξ to $\xi + (1, 0)$. On the right, $\hat{\phi}_a$ changes by $\partial\hat{\phi}_a/\partial\xi_1$, the first entry of the reference gradient $\hat{\nabla}\hat{\phi}_a$ (exactly, since $\hat{\phi}_a$ is linear). On the left, the point of the face moves from $x_1 + B\xi$ to $x_1 + B\xi + B(1,0)^T = x_1 + B\xi + e_1$, so ϕ_a changes by $g \cdot e_1$. The two changes are equal:

$$e_1 \cdot g = \frac{\partial \hat{\phi}_a}{\partial \xi_1}, \qquad \text{and in the same way along } \xi_2, \qquad e_2 \cdot g = \frac{\partial \hat{\phi}_a}{\partial \xi_2}.$$

(This is the chain rule of calculus, here in the simple case of a linear map.) The two inner products are the two entries of $B^T g$, since the rows of B^T are e_1^T and e_2^T. Writing the two equations as one,

$$B^T g = \begin{pmatrix} e_1 \cdot g \\ e_2 \cdot g \end{pmatrix} = \hat{\nabla}\hat{\phi}_a,$$

where the reference gradients are the fixed numbers (12.2) [$\hat{\nabla}\hat{\phi}_1 = (-1,-1)$, $\hat{\nabla}\hat{\phi}_2 = (1,0)$, $\hat{\nabla}\hat{\phi}_3 = (0,1)$].

Step 2: write g in the basis of the edges. The gradient lies in the plane of the face, which the two edges span, so

$$g = c_1\, e_1 + c_2\, e_2 = Bc, \qquad c = (c_1, c_2)^T,$$

the coordinates of g in the basis e_1, e_2.

Step 3: the coordinates come from G_f. Substitute $g = Bc$ of Step 2 into $B^T g = \hat{\nabla}\hat{\phi}_a$ of Step 1:

$$\begin{aligned} B^T(Bc) &= \hat{\nabla}\hat{\phi}_a && \text{(substitute } g = Bc\text{)} \\ (B^T B)\, c &= \hat{\nabla}\hat{\phi}_a && \text{(associativity)} \\ G_f\, c &= \hat{\nabla}\hat{\phi}_a && \text{(29.1) } [G_f = B^T B]. \end{aligned}$$

The matrix G_f is invertible for any triangle of positive area, since $\det G_f = (2A_f)^2 = 4A_f^2 > 0$ by the area paragraph. Multiplying both sides by G_f^{-1} on the left,

$$c_a = G_f^{-1}\, \hat{\nabla}\hat{\phi}_a.$$

The coordinates of the gradient are computed from G_f alone. Only the vector $g = Bc$ in space needs B.

Step 4: the inner products do not need B. The method never uses the gradient vector itself, only inner products of two gradients. With $g_a = Bc_a$ and $g_b = Bc_b$,

$$\begin{aligned} \nabla\phi_a \cdot \nabla\phi_b &= (Bc_a)^T (Bc_b) && \text{(inner product as a matrix product)} \\ &= c_a^T\, (B^T B)\, c_b && ((Bc_a)^T = c_a^T B^T\text{; associativity)} \\ &= c_a^T\, G_f\, c_b && (B^T B = G_f) \\ &= \left(G_f^{-1}\hat{\nabla}\hat{\phi}_a\right)^T G_f \left(G_f^{-1}\hat{\nabla}\hat{\phi}_b\right) && \text{(substitute } c_a, c_b \text{ of Step 3)} \\ &= \hat{\nabla}\hat{\phi}_a^T\, (G_f^{-1})^T\, G_f\, G_f^{-1}\, \hat{\nabla}\hat{\phi}_b && \text{(transpose of a product)} \\ &= \hat{\nabla}\hat{\phi}_a^T\, G_f^{-1}\, G_f\, G_f^{-1}\, \hat{\nabla}\hat{\phi}_b && (G_f^{-1} \text{ is symmetric, as } G_f \text{ is)} \\ &= \hat{\nabla}\hat{\phi}_a^T\, G_f^{-1}\, \hat{\nabla}\hat{\phi}_b && (G_f\, G_f^{-1} = I). \end{aligned}$$

The matrix B has disappeared. This is the gradient of Proposition 12.1, $\nabla\phi_a = B\, G_f^{-1}\, \hat{\nabla}\hat{\phi}_a$, seen from the inside: B only carries the coordinates c_a into space, and an inner product in space brings them back with $B^T B = G_f$.

Step 5: the explicit formulas. The inverse of a 2×2 matrix swaps the diagonal entries, changes the sign of the others and divides by the determinant:

$$G_f^{-1} = \frac{1}{\det G_f} \begin{pmatrix} G_{22} & -G_{12} \\ -G_{12} & G_{11} \end{pmatrix}, \qquad G_{11} = \ell_{12}^2, \quad G_{12} = e_1 \cdot e_2, \quad G_{22} = \ell_{13}^2.$$

With $\hat{\nabla}\hat{\phi}_2 = (1,0)$ and $\hat{\nabla}\hat{\phi}_3 = (0,1)$, the product $u^T G_f^{-1} w$ of Step 4 picks single entries of G_f^{-1}: $(1,0)\, G_f^{-1} (1,0)^T$ is the entry $(1,1)$, and so on. So

$$\nabla\phi_2 \cdot \nabla\phi_2 = \frac{G_{22}}{\det G_f}, \qquad \nabla\phi_3 \cdot \nabla\phi_3 = \frac{G_{11}}{\det G_f}, \qquad \nabla\phi_2 \cdot \nabla\phi_3 = \frac{-G_{12}}{\det G_f}.$$

For ϕ_1 we use $\hat{\nabla}\hat{\phi}_1 = (-1,-1) = -\hat{\nabla}\hat{\phi}_2 - \hat{\nabla}\hat{\phi}_3$, which by Step 3 (a linear map) gives $\nabla\phi_1 = -\nabla\phi_2 - \nabla\phi_3$. Then

$$\begin{aligned} \nabla\phi_1 \cdot \nabla\phi_1 &= (-\nabla\phi_2 - \nabla\phi_3) \cdot (-\nabla\phi_2 - \nabla\phi_3) && \text{(substitute)} \\ &= \nabla\phi_2 \cdot \nabla\phi_2 + 2\,\nabla\phi_2 \cdot \nabla\phi_3 + \nabla\phi_3 \cdot \nabla\phi_3 && \text{(distribute)} \\ &= \frac{G_{22} - 2G_{12} + G_{11}}{\det G_f} && \text{(above),} \end{aligned}$$

and in the same way $\nabla\phi_1 \cdot \nabla\phi_2 = -\nabla\phi_2 \cdot \nabla\phi_2 - \nabla\phi_3 \cdot \nabla\phi_2 = (-G_{22} + G_{12})/\det G_f$ and $\nabla\phi_1 \cdot \nabla\phi_3 = -\nabla\phi_2 \cdot \nabla\phi_3 - \nabla\phi_3 \cdot \nabla\phi_3 = (G_{12} - G_{11})/\det G_f$.

These have a simple geometric meaning. Since $\det G_f = 4A_f^2$ and $G_{22} = \ell_{13}^2$, the first is $|\nabla\phi_2|^2 = \ell_{13}^2/(2A_f)^2$, that is, $|\nabla\phi_2| = \ell_{13}/(2A_f)$. Twice the area is base times height, so $2A_f/\ell_{13}$ is the height of the triangle over the edge opposite to x_2, and $|\nabla\phi_2|$ is one over that height: the hat function rises from 0 to 1 over the height. For ϕ_1, the numerator is $G_{11} - 2G_{12} + G_{22} = e_1 \cdot e_1 - 2\, e_1 \cdot e_2 + e_2 \cdot e_2 = |e_2 - e_1|^2 = \ell_{23}^2$, so $|\nabla\phi_1| = \ell_{23}/(2A_f)$, one over the height at x_1.

We collect the result.

Proposition 29.1. *The area of a face, the gradients' inner products $\nabla\phi_a \cdot \nabla\phi_b$, and therefore the element stiffness and mass matrices of the scalar parts, $A_f\, \nabla\phi_a \cdot \nabla\phi_b$ and $A_f/3$, depend on B only through G_f, and so only on the three edge lengths: $A_f = \frac{1}{2}\sqrt{\det G_f}$ and $\nabla\phi_a \cdot \nabla\phi_b = \hat{\nabla}\hat{\phi}_a^T\, G_f^{-1}\, \hat{\nabla}\hat{\phi}_b$, with G_f given by (29.2).*

So the matrix B, which Part I needed in order to find the gradients as vectors of space, was carrying more than the method uses: the method uses only $B^T B$. This settles item 1 of the list of Section 27. For the triangle of Example 12.2, $G_f = \left(\begin{smallmatrix} 4 & 0 \\ 0 & 2 \end{smallmatrix}\right)$, so $G_{11} = 4$, $G_{12} = 0$, $G_{22} = 2$ and $\det G_f = 4 \cdot 2 - 0 = 8$. Step 5 gives $|\nabla\phi_2|^2 = \frac{2}{8} = \frac{1}{4}$, $|\nabla\phi_3|^2 = \frac{4}{8} = \frac{1}{2}$, $\nabla\phi_2 \cdot \nabla\phi_3 = \frac{0}{8} = 0$ and $|\nabla\phi_1|^2 = \frac{2-0+4}{8} = \frac{6}{8} = \frac{3}{4}$, the numbers found there with the vectors of space.

29.3. Heron's formula and the angles

The program does not invert G_f. It uses two classical formulas that give the area and the angles directly from the lengths, and then draws the triangle in

the plane (next subsection). With the half perimeter $\sigma = \frac{1}{2}(\ell_{12} + \ell_{13} + \ell_{23})$, *Heron's formula* is

$$A_f = \sqrt{\sigma(\sigma - \ell_{12})(\sigma - \ell_{13})(\sigma - \ell_{23})}. \tag{29.3}$$

It is $\frac{1}{2}\sqrt{\det G_f}$ with the entries (29.2) multiplied out and factored. To see it, write $a = \ell_{12}$, $b = \ell_{13}$, $c = \ell_{23}$, so that $G_{11} = a^2$, $G_{22} = b^2$, $G_{12} = \frac{1}{2}(a^2 + b^2 - c^2)$:

$$\begin{aligned}
4 \det G_f &= 4a^2b^2 - (a^2 + b^2 - c^2)^2 && \text{(det)} \\
&= \big(2ab - (a^2 + b^2 - c^2)\big) \\
&\qquad \cdot \big(2ab + (a^2 + b^2 - c^2)\big) && (p^2 - q^2) \\
&= \big(c^2 - (a - b)^2\big)\big((a + b)^2 - c^2\big) && \text{(regroup)} \\
&= (c - a + b)(c + a - b) \\
&\qquad \cdot (a + b - c)(a + b + c) && (p^2 - q^2) \\
&= (2\sigma - 2a)(2\sigma - 2b)(2\sigma - 2c)(2\sigma) && (2\sigma = a{+}b{+}c) \\
&= 16\,\sigma(\sigma - a)(\sigma - b)(\sigma - c).
\end{aligned}$$

Then $A_f^2 = \frac{1}{4}\det G_f = \frac{1}{16} \cdot 4 \det G_f = \sigma(\sigma - a)(\sigma - b)(\sigma - c)$, which is (29.3). The *law of cosines* gives each angle from the three lengths: we found above that the off-diagonal entry of (29.2) is both $\frac{1}{2}(\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2)$ and $\ell_{12}\ell_{13}\cos\alpha_1$; dividing by $\ell_{12}\ell_{13}$,

$$\cos\alpha_1 = \frac{\ell_{12}^2 + \ell_{13}^2 - \ell_{23}^2}{2\,\ell_{12}\,\ell_{13}}, \tag{29.4}$$

at the corner x_1, between the edges of lengths ℓ_{12} and ℓ_{13} and opposite to the edge of length ℓ_{23}. At the other two corners the formula is the same with the lengths relabeled: the two sides that meet at the corner in the denominator and first in the numerator, the opposite side with the minus sign. These two formulas are the only places where the lengths enter the program: everything else about a face is computed from its area and its angles.

In the code, Heron's formula and the law of cosines are two one-line functions:

```
31 /// Heron's formula (eq. heron): the area of a triangle from its three edge lengths.
32 inline double heronArea(double l0, double l1, double l2) {
33     const double s = 0.5 * (l0 + l1 + l2);
34     return std::sqrt(std::max(0.0, s * (s - l0) * (s - l1) * (s - l2)));
35 }
36
37 /// The law of cosines (eq. law_of_cosines): the angle opposite to the edge of length c
       in a triangle with sides a, b, c.
38 inline double cornerAngle(double a, double b, double c) {
39     return std::acos(std::max(-1.0, std::min(1.0, (a * a + b * b - c * c) / (2.0 * a *
           b))));
40 }
```

In `heronArea`, the variable `s` on line 33 is the half perimeter σ, and line 34 is (29.3) [$A_f = \sqrt{\sigma(\sigma - \ell_{12})(\sigma - \ell_{13})(\sigma - \ell_{23})}$] with one safeguard: for a nearly degenerate triangle, rounding can make the product slightly negative, and `std::max(0.0, ...)` sets it to zero instead of taking the square root of

a negative number. In `cornerAngle(a, b, c)`, `a` and `b` are the two sides that meet at the corner and `c` is the side opposite to it; line 39 is (29.4) [$\cos\alpha_1 = \frac{\ell_{12}^2+\ell_{13}^2-\ell_{23}^2}{2\ell_{12}\ell_{13}}$], with the quotient clamped to $[-1,1]$ for the same reason, since `std::acos` is not defined outside that interval.

As a check against Part I: twice the area is base times height, $2A_f = \ell_{12}\,\ell_{13}\sin\alpha_1$, so $\sin\alpha_1 = 2A_f/(\ell_{12}\ell_{13})$. Dividing (29.4) by it,

$$\cot\alpha_1 = \frac{\cos\alpha_1}{\sin\alpha_1} = \frac{\ell_{12}^2+\ell_{13}^2-\ell_{23}^2}{2\,\ell_{12}\,\ell_{13}} \cdot \frac{\ell_{12}\,\ell_{13}}{2A_f} = \frac{\ell_{12}^2+\ell_{13}^2-\ell_{23}^2}{4A_f},$$

so the cotangent weights of Proposition 12.3 are functions of the lengths, as they must be.

29.4. Laying the triangle out in the plane

Inverse matrices are not the most convenient way to work with a face. It is simpler to have actual coordinates: the gradients become vectors of a plane, and directions in the face become angles that we can measure. Any triangle with the same three lengths has the same Gram matrix (29.2), and so, by Proposition 29.1, the same area and the same inner products of gradients. We may therefore replace the face in space by any triangle of the plane $\mathbb{R}^2$ with the same lengths, an isometric copy, and compute there. In gauge language this is a change of displacement gauge that keeps the metric: the face is described by another map h, with values in a plane instead of space, and the same G_f. If moreover we use the coordinates of that plane themselves as the coordinates of the chart, the map is the identity, the metric in those coordinates is $G = I$, and by (28.5) [$\nabla f\cdot\nabla f' = (\hat{\nabla}\hat{f})^T G^{-1}\hat{\nabla}\hat{f}'$] with $G^{-1} = I$ gradients become plain vectors of the plane. We choose the simplest copy, its *layout* (Figure 23): the first vertex at the origin, the first edge along the x axis, and the third vertex above it,

$$\bar{x}_1 = (0,0), \qquad \bar{x}_2 = (\ell_{12},0), \qquad \bar{x}_3 = \ell_{13}\,(\cos\alpha_1, \sin\alpha_1). \tag{29.5}$$

It has the right lengths. By construction $|\bar{x}_2 - \bar{x}_1| = |(\ell_{12},0)| = \ell_{12}$ and $|\bar{x}_3 - \bar{x}_1| = \ell_{13}\sqrt{\cos^2\alpha_1 + \sin^2\alpha_1} = \ell_{13}$. For the third edge,

$$\begin{aligned}
|\bar{x}_3 - \bar{x}_2|^2 &= (\ell_{13}\cos\alpha_1 - \ell_{12})^2 + (\ell_{13}\sin\alpha_1 - 0)^2 && \text{(components)}\\
&= \ell_{13}^2\cos^2\alpha_1 - 2\ell_{12}\ell_{13}\cos\alpha_1 + \ell_{12}^2 \\
&\quad + \ell_{13}^2\sin^2\alpha_1 && \text{(expand)}\\
&= \ell_{13}^2 - 2\ell_{12}\ell_{13}\cos\alpha_1 + \ell_{12}^2 && (\cos^2+\sin^2 = 1)\\
&= \ell_{13}^2 - 2\ell_{12}\ell_{13}\Big(\frac{\ell_{12}^2+\ell_{13}^2-\ell_{23}^2}{2\ell_{12}\ell_{13}}\Big) + \ell_{12}^2 && \text{(by (29.4))}\\
&= \ell_{13}^2 - (\ell_{12}^2+\ell_{13}^2-\ell_{23}^2) + \ell_{12}^2 && \text{(cancel } 2\ell_{12}\ell_{13})\\
&= \ell_{23}^2 && \text{(cancel terms).}
\end{aligned}$$

It is counterclockwise, because $\sin\alpha_1 > 0$ puts $\bar{x}_3$ above the first edge: walking $\bar{x}_1 \to \bar{x}_2 \to \bar{x}_3$, the inside of the triangle is on the left. That will matter for the direction of the gradients. Its Gram matrix is (29.2), and by Proposition 29.1

any quantity of the method computed on the layout equals the one computed on the face in space.

The gradients on the layout: the quarter turn. On the layout we compute the gradients with the formula (24.2) [$\nabla\phi_a = \frac{(x_c - x_b)\mathsf{T}_f}{2A_f}$] of Part II, with the quarter turn T_f written as a matrix of the plane. Let a, b, c be the three vertices in counterclockwise order. In the plane, the quarter turn counterclockwise is

$$J(x, y) = (-y, x), \qquad J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

We claim

$$\nabla\phi_a = \frac{J(\bar{x}_c - \bar{x}_b)}{2A_f}, \tag{29.6}$$

and we check it from the definition of the gradient: the hat ϕ_a is linear, equal to 1 at $\bar{x}_a$ and 0 at $\bar{x}_b$ and $\bar{x}_c$, so its gradient g is the vector with $g \cdot (\bar{x}_c - \bar{x}_b) = \phi_a(\bar{x}_c) - \phi_a(\bar{x}_b) = 0 - 0 = 0$ and $g \cdot (\bar{x}_a - \bar{x}_b) = \phi_a(\bar{x}_a) - \phi_a(\bar{x}_b) = 1 - 0 = 1$. Two conditions on a vector of the plane determine it, so it suffices to check both for the right-hand side of (29.6). Write $p = \bar{x}_c - \bar{x}_b = (p_1, p_2)$ and $q = \bar{x}_a - \bar{x}_b = (q_1, q_2)$, so $Jp = (-p_2, p_1)$.

- *Along the opposite edge.* $Jp \cdot p = (-p_2)\, p_1 + p_1\, p_2 = 0$: the quarter turn of a vector is orthogonal to it. So $\frac{Jp}{2A_f} \cdot p = 0$, the first condition.
- *Towards a.* $Jp \cdot q = -p_2\, q_1 + p_1\, q_2 = p_1 q_2 - p_2 q_1$, the determinant of the two edges from b, $\det[p \;\; q]$, which is $\pm 2A_f$: plus when turning from p (towards c) to q (towards a) is counterclockwise. Going around the triangle counterclockwise as $a \to b \to c$, at the corner b the edge to c comes first and the edge to a second, counterclockwise; so the sign is plus, $Jp \cdot q = 2A_f$, and $\frac{Jp}{2A_f} \cdot q = 1$, the second condition.

The picture behind the two checks: $\nabla\phi_a$ is orthogonal to the opposite edge, where ϕ_a is constant; it points into the triangle, towards a, because in a counterclockwise triangle the left of every edge is the inside and J turns to the left; and its length is $|p|/(2A_f) = \ell_{bc}/(2A_f)$, one over the height $h_a = 2A_f/\ell_{bc}$ at a. This is (24.2) with the quarter turn T_f of the face written as the matrix J of the layout.

The face frame. The layout also gives the face its frame: the x axis, which is the direction of the first edge, from $\bar{x}_1$ to $\bar{x}_2$. This is the same first frame vector t_f that `computeFaceTangentSpace` of `FEM.cpp` takes, and that `computeGeometry` of `ga_fem.cpp` takes on its line 66, `t = (v[1]->p - v[0]->p).normalized()`. The two methods thus measure directions in a face from the same line, which we use to compare them face by face in Section 33. A direction in the face is now simply an angle measured from the x axis, counterclockwise; we call β_e the angle of the direction of an edge e of the face in this frame (the code's `edgeAngle`).

The rotation gauge of a face vanishes. The layout is an isometric chart, $G = I$, and the face frame is a constant direction of it, the x axis. The rotation gauge (28.7) [$\Omega(a) = \omega(h(a))\,\mathsf{T}$] measures how fast the frame turns relative to parallel transport. In a flat face, parallel transport keeps the direction of

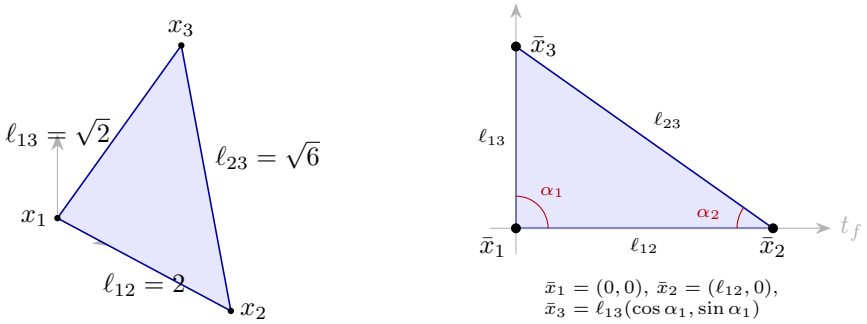


FIGURE 23. Left: the triangle of Example 12.2 in space; Part III reads only its three edge lengths. Right: its layout (29.5), an isometric copy in the plane, with the face frame t_f along the first edge. Every quantity of the method can be computed on the layout.

a vector in the plane of the face, and the frame keeps its direction too, so $\omega = 0$ and $\Omega = 0$. Then the covariant derivative (28.8) [$D_k\psi = \partial_k\psi + \omega_k \mathsf{T}\psi$] becomes, with $\omega_k = 0$,

$$D_k\psi = \partial_k\psi + (0)\,\mathsf{T}\psi = \partial_k\psi,$$

the plain derivative of the two components. This agrees with Section 28.5: a face has no curvature inside, so nothing prevents a flat gauge there. Each face of the mesh is therefore a chart in which the weak form (28.24) is the weak form of two scalar problems; what couples the faces, and what carries the curvature, must be the transitions of Section 31.

The code. `IntrinsicConnection.h` keeps what the method needs of a face, of a vertex and of the whole mesh in three small structures. Each field names the symbol it holds; the vertex fields and the cone angles `outAngle` belong to Section 30, and `rho` to Section 31:

```
42 /// What the method needs of one face, from its lengths (Section 20).
43 struct FaceGeometry {
44     double length[3]; // length[a] = |h[a]|, the edge from v[a] to v[a+1]
45     double angle[3]; // the corner angle at v[a]
46     double area;
47     Vec2 x[3]; // the layout: an isometric copy of the triangle in the plane, v[0] at
           0, v[1] on the x axis
48     Vec2 grad[3]; // gradients of the hat functions in the layout (eq.
           intrinsic_gradient)
49     double edgeAngle[3];// direction of h[a] in the layout, measured from the x axis (
           the face frame t_f)
50     Complex rho[3]; // the connection from the plane of v[a] into the face, e^{I rho_a^
           f} (eq. intrinsic_rho)
51 };
52
53 /// What the method needs of one vertex (Section 21).
54 struct VertexGeometry {
55     double totalAngle; // Theta_i, the sum of the corner angles at the vertex
56     double scale; // s_i = 2 pi / Theta_i for an interior vertex, 1 at the boundary (eq
           . scale)
57     bool boundary;
```

```
};

struct Geometry {
    std::vector<FaceGeometry> faces;
    std::vector<VertexGeometry> vertices;
    std::map<const Edge*, double> outAngle; // the cone angle theta of every outgoing
        half edge (eq. cone_angle)
};
```

The function `layOut` fills the face fields from the three lengths: the layout (29.5) [$\bar{x}_1 = 0,\ \bar{x}_2 = (\ell_{12}, 0),\ \bar{x}_3 = \ell_{13}(\cos\alpha_1, \sin\alpha_1)$], the angles, the area and the gradients (29.6) [$\nabla\phi_a = \frac{J(\bar{x}_c - \bar{x}_b)}{2A_f}$]:

```
/// Section 20: lay a face out in the plane from its lengths alone and compute its area
    , angles and gradients.
inline void layOut(FaceGeometry& g) {
    const double l01 = g.length[0], l12 = g.length[1], l20 = g.length[2];
    g.angle[0] = cornerAngle(l01, l20, l12);
    g.angle[1] = cornerAngle(l01, l12, l20);
    g.angle[2] = M_PI - g.angle[0] - g.angle[1];
    g.area = heronArea(l01, l12, l20);
    g.x[0] = Vec2(0, 0);
    g.x[1] = Vec2(l01, 0);
    g.x[2] = l20 * Vec2(std::cos(g.angle[0]), std::sin(g.angle[0]));
    for (int a = 0; a < 3; ++a) {
        const Vec2 e = g.x[(a + 1) % 3] - g.x[a];
        g.edgeAngle[a] = std::atan2(e.y(), e.x());
        // the opposite edge, from v[a+1] to v[a+2], turned a quarter turn
            counterclockwise, over twice the area
        const Vec2 opposite = g.x[(a + 2) % 3] - g.x[(a + 1) % 3];
        g.grad[a] = Vec2(-opposite.y(), opposite.x()) / (2.0 * g.area);
    }
}
```

Line by line:

- Line 68 names the lengths by their end points, counting from zero: `l01` $= \ell_{12}$, `l12` $= \ell_{23}$, `l20` $= \ell_{13}$.
- Line 69 is α_1 by (29.4): the sides meeting at `v[0]` are `l01` and `l20`, and the opposite side is `l12`. Line 70 is α_2 in the same way: the sides meeting at `v[1]` are `l01` and `l12`, and the opposite side is `l20`. Line 71 takes a shortcut for α_3: the angles of a triangle add up to π, so $\alpha_3 = \pi - \alpha_1 - \alpha_2$.
- Line 72 is Heron's formula (29.3).
- Lines 73 to 75 are the layout (29.5): `x[2]` $= \ell_{13}(\cos\alpha_1, \sin\alpha_1)$.
- In the loop, the corner `a` is the vertex a of (29.6), and $b = a+1$, $c = a+2$ (modulo three) follow it counterclockwise. Line 77 is the edge `h[a]` on the layout, from `x[a]` to `x[a+1]`, and line 78 its angle β_e from the x axis; for `a = 0` it is 0, since the first edge is the x axis.
- Line 80 is the opposite edge $p = \bar{x}_c - \bar{x}_b$, and line 81 applies $J(x, y) = (-y, x)$ and divides by $2A_f$: this is (29.6).

Worked example 29.2. *The triangle of Example 12.2,* $x_1 = 0$, $x_2 = (2, 0, 0)$, $x_3 = (0, 1, 1)$, *has the lengths*

$$\ell_{12} = |(2,0,0)| = 2, \quad \ell_{13} = |(0,1,1)| = \sqrt{2}, \quad \ell_{23} = |(-2,1,1)| = \sqrt{6}.$$

The metric. *By (29.2),* $G_{11} = \ell_{12}^2 = 4$, $G_{22} = \ell_{13}^2 = 2$, *and* $G_{12} = \frac{1}{2}(4 + 2 - 6) = 0$, *so* $G_f = \left(\begin{smallmatrix} 4 & 0 \\ 0 & 2 \end{smallmatrix}\right)$, *exactly the* B^TB *of Example 12.2, now obtained*

without the positions (the program prints $5.3 \cdot 10^{-16}$ *for the zero, a rounding error).*

The area. $\sigma = (2+\sqrt{2}+\sqrt{6})/2 = 2.931852$, *and Heron's formula gives* $A_f = 1.414214 = \sqrt{2}$; *also* $\frac{1}{2}\sqrt{\det G_f} = \frac{1}{2}\sqrt{8} = \sqrt{2}$.

The angles. *By (29.4),* $\cos\alpha_1 = \frac{4+2-6}{2\cdot 2\cdot\sqrt{2}} = 0$, *so* $\alpha_1 = 90°$; *at* x_2, $\cos\alpha_2 = \frac{\ell_{12}^2+\ell_{23}^2-\ell_{13}^2}{2\ell_{12}\ell_{23}} = \frac{4+6-2}{4\sqrt{6}} = \frac{2}{\sqrt{6}}$, *so* $\alpha_2 = 35.26°$; *and* $\alpha_3 = 180° - 90° - 35.26° = 54.74°$.

The layout. $\bar{x}_1 = (0,0)$, $\bar{x}_2 = (2,0)$, $\bar{x}_3 = \sqrt{2}\,(\cos 90°, \sin 90°) = (0,\sqrt{2})$.

The gradients, *by (29.6) with* $2A_f = 2\sqrt{2}$:

$$\nabla\phi_1 = \frac{J(\bar{x}_3-\bar{x}_2)}{2\sqrt{2}} = \frac{J(-2,\sqrt{2})}{2\sqrt{2}} = \frac{(-\sqrt{2},-2)}{2\sqrt{2}} = \Big(-\frac{1}{2},-\frac{1}{\sqrt{2}}\Big),$$

$$\nabla\phi_2 = \frac{J(\bar{x}_1-\bar{x}_3)}{2\sqrt{2}} = \frac{J(0,-\sqrt{2})}{2\sqrt{2}} = \frac{(\sqrt{2},0)}{2\sqrt{2}} = \Big(\frac{1}{2},0\Big),$$

$$\nabla\phi_3 = \frac{J(\bar{x}_2-\bar{x}_1)}{2\sqrt{2}} = \frac{J(2,0)}{2\sqrt{2}} = \frac{(0,2)}{2\sqrt{2}} = \Big(0,\frac{1}{\sqrt{2}}\Big),$$

using $J(x,y) = (-y,x)$ *on each line. Their squared lengths are* $\frac{1}{4}+\frac{1}{2} = \frac{3}{4}$, $\frac{1}{4}$ *and* $\frac{1}{2}$, *those of the gradients* $(-\frac{1}{2},-\frac{1}{2},-\frac{1}{2})$, $(\frac{1}{2},0,0)$, $(0,\frac{1}{2},\frac{1}{2})$ *of Example 12.2, which are the same vectors seen in a different plane; and they agree with Step 5 above. The stiffness* $A_f\,\nabla\phi_a\cdot\nabla\phi_b$ *is the matrix of Example 12.2, entry for entry (the program prints both).*

What we got. Each face is a flat chart with a constant metric G_f, determined by its three lengths, and a flat gauge, $\Omega = 0$. From the metric we have the area A_f by Heron's formula and the gradients $\nabla\phi_a$ on the layout, which is all that the element integrals of Stage 7 will need from the face. What is still missing is where the unknowns live: at the vertices.

30. Stage 5, the Vertices: the Stretched Cone as a Flat Chart

What we want. The unknowns of the finite element method sit at the vertices, one tangent vector per vertex, and Stage 5 must give every vertex a flat chart in which its vector is a spinor coordinate ψ_i, a complex number. The faces already have flat charts; a vertex is where several of them meet, and where, by Section 28.5, the curvature of the mesh must sit. We shall find that no chart of a curved vertex can be an isometry, choose the simplest one that is not, and see what that choice costs. The vertex chart is used only to hold the unknown: the integrals are all done in the face charts.

30.1. The cone around a vertex

Inside a face, the tangent plane is the plane of the face, and the layout describes it intrinsically. Across an edge, from one face to its neighbor, parallel transport is the unfolding of one face onto the plane of the other (Section 22.5): lay the two triangles out in the plane on the two sides of their common edge, and a vector keeps its direction in that common plane. The unfolding depends only on the two triangles, that is, on five edge lengths. So faces and edges pose no problem. The problem is the vertex.

Do the same unfolding with all the faces around a vertex i. Cut the faces along one edge leaving i, the reference edge, and flatten them one after another into the plane: the first face with the reference edge along the positive x axis, the next face on the other side of the next edge, and so on around the vertex, as in Figure 24. The faces form a fan around i, and their angles at i add up to the *total angle*

$$\Theta_i = \sum_{f \ni i} \alpha_{i,f}, \tag{30.1}$$

where $\alpha_{i,f}$ is the angle of the face f at the vertex i. There are three cases.

- If $\Theta_i = 2\pi$, the fan closes exactly: the last face fits against the first. Around i the mesh is flat, even if it is folded in space, and the ant cannot tell the vertex from a point of a plane. Every vertex of a folded sheet of paper is of this kind.
- If $\Theta_i < 2\pi$, there is a gap between the last face and the first: to close the surface the fan must be bent into a cone, and the vertex is its tip. Example: the corner of a cube, three right angles, $\Theta_i = 3 \cdot 90^\circ = 270^\circ$.
- If $\Theta_i > 2\pi$, the last face overlaps the first: there is too much material around i to lie flat, and the vertex is a saddle.

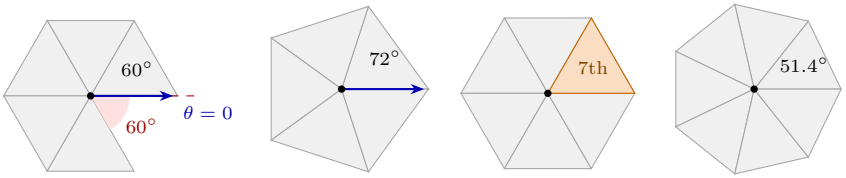
The difference

$$\kappa_i = 2\pi - \Theta_i \tag{30.2}$$

is the *angle defect*: positive at a cone, negative at a saddle, zero at a flat vertex. For the cube corner it is $360^\circ - 270^\circ = 90^\circ$.

The angle defect is a curvature. The ant cannot see its surface bend in space. What it can do is carry an arrow around a closed loop, keeping the arrow parallel at every step, and look at it when it comes back. On a plane the arrow comes back as it left. On a curved surface it comes back turned, and the angle of the turn, the *holonomy* of the loop, is the total Gaussian curvature $\int K\,dA$ of the region enclosed by the loop, by (28.15) [$\psi_{\text{end}} = e^{\mathsf{T} \int K\,dA} \psi_{\text{start}}$].

Let us compute that rotation for a small loop around the vertex i of a mesh. Inside a face the arrow does not change, because the face is flat; across an edge it is unfolded. After flattening the fan, both steps say the same thing: in the plane of the fan, the arrow keeps its direction. Suppose it starts in the first face at the angle u from the reference edge e_0. It goes around through all the faces of the fan, always pointing the same way in the plane. When it reaches the end of the fan, the edge there is the second copy of e_0, which lies at the angle Θ_i from the first copy; relative to that second copy the arrow



cone, cut open
$\Theta_i = 300°$

rescaled by $s_i = \frac{6}{5}$
closes up

saddle, cut open
$\Theta_i = 420°$: overlaps

rescaled by $s_i = \frac{6}{7}$
closes up

FIGURE 24. The faces around a vertex, cut along the reference edge (blue, $\theta = 0$) and flattened. Five equilateral triangles: $\Theta_i = 300°$, a gap of 60°, the angle defect. Rescaling every angle by $s_i = 2\pi/\Theta_i = 6/5$ closes the gap: every 60° corner becomes 72°. Seven equilateral triangles: $\Theta_i = 420°$, the seventh overlaps the first; rescaling by 6/7 turns every corner into 51.4°.

makes the angle $u - \Theta_i$. But the second copy of e_0 is e_0 itself on the surface: the loop is closed. So the arrow is back in the first face at the angle $u - \Theta_i$, which we rewrite by adding and subtracting 2π:

$$\begin{aligned} u - \Theta_i &= u + 2\pi - \Theta_i - 2\pi && (\text{add } 2\pi - 2\pi = 0) \\ &= u + (2\pi - \Theta_i) - 2\pi && (\text{group}) \\ &= u + \kappa_i - 2\pi && (30.2)\ [\,\kappa_i = 2\pi - \Theta_i\,]. \end{aligned}$$

Subtracting a full turn 2π does not change a direction, so the arrow points at $u + \kappa_i$: it has turned by the angle defect κ_i. The loop can be as small as we like, as long as it goes around i, and the turn is always κ_i; a loop that does not go around a vertex stays in flat faces and does not turn at all. So all the curvature of the mesh sits at its vertices, and κ_i is the total curvature $\int K\,dA$ of the neighborhood of i, concentrated at the vertex [11]. In gauge language: the faces are flat gauges glued to one another across the edges by the unfoldings, and the product of these transitions around the vertex is $e^{\mathsf{T}\kappa_i}$, the discrete form of the holonomy (28.15). The field strength is zero in every face and concentrated at the vertex. It is a sum of angles of triangles, which are functions of the edge lengths, so it is intrinsic.

The defects add up to $2\pi\chi$. On a smooth closed surface, the total curvature $\int K\,dA$ is $2\pi\chi$, where χ is the *Euler characteristic*, 2 for any surface shaped like a sphere (the Gauss–Bonnet theorem). On a mesh, $\chi = V - E + F$, the number of vertices minus the number of edges plus the number of faces. We show that $\sum_i \kappa_i = 2\pi(V - E + F)$ for a closed mesh in three steps.

Step 1: the angles. Every corner of every triangle belongs to exactly one vertex. So the sum of all the total angles, $\sum_i \Theta_i$, is the sum of all the corners of all the triangles, which we can add up triangle by triangle instead of vertex by vertex. Each triangle contributes the sum of its three angles, π, so $\sum_i \Theta_i = \pi F$.

Step 2: the edges. Count the pairs (edge, triangle containing it). Each triangle has three edges, so there are $3F$ pairs; on a closed mesh each edge lies in exactly two triangles, so there are $2E$ pairs. Hence $2E = 3F$, that is, $E = \frac{3}{2}F$.

Step 3: the sum. By (30.2) [$\kappa_i = 2\pi - \Theta_i$],

$$\begin{aligned}
\sum_i \kappa_i &= \sum_i (2\pi - \Theta_i) && \text{(substitute)} \\
&= 2\pi V - \sum_i \Theta_i && (V \text{ vertices, each contributing } 2\pi) \\
&= 2\pi V - (\pi F) && \text{(Step 1)} \\
&= 2\pi V - 2\pi\left(\tfrac{3}{2}F - F\right) && (\pi F = 2\pi \cdot \tfrac{1}{2}F \text{ and } \tfrac{1}{2} = \tfrac{3}{2} - 1) \\
&= 2\pi V - 2\pi\left(E - F\right) && (\text{Step 2, } \tfrac{3}{2}F = E) \\
&= 2\pi(V - E + F) = 2\pi\chi && \text{(distribute).}
\end{aligned}$$

For the icosahedron, $V - E + F = 12 - 30 + 20 = 2$; every vertex has five equilateral triangles, $\Theta_i = 5 \cdot 60^\circ = 300^\circ$ and $\kappa_i = 60^\circ$, and the twelve defects add up to $12 \cdot 60^\circ = 720^\circ = 4\pi = 2\pi \cdot 2$. On a mesh with boundary the same count works with $\pi - \Theta_i$ at the boundary vertices.

The program checks this on the cactus. The function `totalCurvature` of `ga_fem.cpp` adds $2\pi - \Theta_i$ over the interior vertices and $\pi - \Theta_i$ over the boundary vertices (line 476), with Θ_i taken from the walk around each vertex that we describe below:

```
474 static double totalCurvature(const intrinsic::Geometry& G) {
475     double sum = 0;
476     for (const auto& v : G.vertices) sum += (v.boundary ? M_PI : 2.0 * M_PI) - v.
            totalAngle;
477     return sum;
478 }
```

The angle defects of the cactus add up to $12.566371 = 4\pi$, the value of a sphere, and the program prints $2\pi(V - E + F) = 12.566371$ next to it.

30.2. Why a vertex has no tangent plane

At a point p of a smooth surface, the directions in which one can leave p form a full circle, a turn of 2π: a small circle of radius r around p, drawn on the surface, has a length closer and closer to $2\pi r$ as $r \to 0$. The tangent vectors at p are the vectors of the tangent plane: they can be added and multiplied by numbers, and the angle between two of them is an angle of the plane.

That is exactly what the finite element method needs at a vertex. The unknown at the vertex i is a tangent vector, and the trial field (22.8) [$\psi_f = \sum_{i \in f} \phi_i \, e^{\mathsf{T}_f \rho_i^f} \psi_i$] combines the vectors of the three vertices of a face linearly, so ψ_i must be a vector of some plane, with lengths and angles.

At a vertex of a mesh, the neighborhood of i is exactly the cone of Figure 24, at every magnification. A small circle of radius r around i crosses the faces one after another, as an arc of angle $\alpha_{i,f}$ and length $\alpha_{i,f}\, r$ in each

face, so its length is

$$\sum_{f \ni i} \alpha_{i,f}\, r = \Big(\sum_{f \ni i} \alpha_{i,f} \Big) r = \Theta_i\, r,$$

not $2\pi r$. So when $\Theta_i \neq 2\pi$ the directions at i form a circle of total angle Θ_i, while the directions of any plane form a circle of total angle 2π. Any way of identifying the directions at i with the directions of a plane must change some angles. The gauge language says the same: a chart around i that were an isometry, $G = I$, with a constant frame, would be a flat gauge, $\Omega = 0$, and by (28.15) a loop around i would have no holonomy; but it has κ_i. So a flat chart at a curved vertex must distort the metric somewhere, and how it distorts is a choice of displacement gauge, a modeling decision. Part I made it with the vertex normal. Here we make it with the angles alone. When $\Theta_i = 2\pi$ the vertex does have a natural plane, the flattened fan, which closes exactly; a good choice should then give that plane.

30.3. Directions at a vertex as angles

The ant can describe a direction at the vertex by a single number: the angle it measures from a fixed edge. Choose a reference edge e_0 leaving the vertex: we take the same edge `vi.edge` whose projection is the vertex frame vector t_i of Part I (in `computeGeometry` of `ga_fem.cpp`, line 52 takes the direction of that edge, `vi.edge->pair->vertex->p - vi.p`, and line 53 projects it). Walk counterclockwise around the vertex, from one outgoing edge to the next, $e_0, e_1, e_2, \ldots$, and assign to each outgoing edge its *cone angle*, the angle the ant measures from e_0:

$$\theta(e_0) = 0, \qquad \theta(e_{m+1}) = \theta(e_m) + \alpha_m, \tag{30.3}$$

where α_m is the angle at i of the face between e_m and e_{m+1}. Unrolling the recursion,

$$\theta(e_1) = 0{+}\alpha_0, \qquad \theta(e_2) = (\alpha_0){+}\alpha_1, \qquad \ldots, \qquad \theta(e_m) = \alpha_0{+}\alpha_1{+}\cdots{+}\alpha_{m-1}.$$

Every direction at the vertex points into some corner and has a cone angle θ in $[0, \Theta_i)$: if it points into the corner between e_m and e_{m+1}, at the angle u from e_m inside that face, its cone angle is $\theta(e_m) + u$. After a full turn, $\theta = \alpha_0 + \cdots + \alpha_{n-1} = \Theta_i$, which is again the direction of e_0. For the five equilateral triangles of Figure 24 the outgoing edges have the cone angles $0°, 60°, 120°, 180°, 240°$, and $300°$ is e_0 again.

To compute (30.3) the program must walk around each vertex in the half-edge structure. Its conventions decide the direction of the walk and the signs of the angles, so we describe them carefully.

Half edges. The structure of `src/HalfEdge` stores each edge of the mesh as two *half edges* of opposite directions, `e` and `e->pair`. A half edge has an origin, `e->vertex`, the vertex it leaves (`Mesh::addEdge` sets `e->vertex` to the first vertex i of the edge from i to j), and a face, `e->face`, the face that lies to its *left*. The faces of the mesh are listed with their vertices in counterclockwise order, seen from the side towards which the face normal points; going from a half edge of a face to `e->next` goes around that face

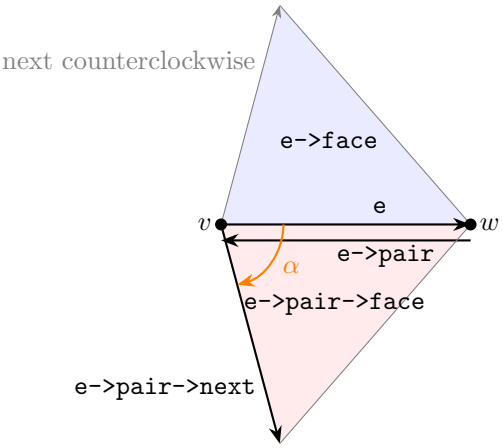


FIGURE 25. The half edges around a vertex v. The face of a half edge lies to its left. `e->pair` runs from w back to v, with the face to the right of `e` on its left; the next half edge of that face, `e->pair->next`, leaves v again, clockwise from `e`. The corner between them, of angle α, belongs to the face to the left of `e->pair->next`, and is its corner at its origin.

counterclockwise, and for a counterclockwise face the inside is always on the left, so all three half edges of a face have it on their left. At the boundary of the mesh, a half edge with nothing on its left has `e->face` equal to null.

Turning around a vertex. Let `e` be a half edge leaving the vertex v, towards a neighbor w (Figure 25). Its pair `e->pair` goes back from w to v. The face to the left of `e->pair` is the face to the *right* of `e`. In that face, the half edge after `e->pair` starts where `e->pair` ends, at v: so `e->pair->next` is again a half edge leaving v, the one on the other side of the face to the right of `e`. It is the next outgoing half edge *clockwise*. This is the step of the iterator `Vertex::EdgeAroundIterator` of `HalfEdge/Vertex.h`, `run = run->pair->next`, so the natural walk around a vertex in this structure is clockwise, and the comment of `IntrinsicConnection.h` (lines 7 and 8) states the same convention.

Which corner lies between two steps. When the walk goes from `e` to `next = e->pair->next`, it crosses one corner at v: that of the face to the right of `e`, which is the face to the left of `next`. Counterclockwise, that corner starts at `next` and ends at `e`. Its angle is the angle of the face of `next` at the origin of `next`. The small helper `angleAtOrigin` returns exactly this: it finds the position `a` of the half edge among the three half edges of its face, so that the half edge goes from `v[a]` to `v[a+1]`, and returns `angle[a]`, the corner angle at `v[a]`, its origin:

```
96      // the corner angle of the face to the left of an outgoing half edge, at its origin
97      auto angleAtOrigin = [&](const Edge* e) {
98          Vertex* v[3];
```

```
99          Edge* h[3];
100         faceCorners(*e->face, v, h);
101         for (int a = 0; a < 3; ++a) if (h[a] == e) return G.faces[e->face->ID].angle[a
                ];
102         return 0.0;
103     };
```

Clockwise walk, negative angles. Because the walk is clockwise, the program builds (30.3) backwards: it starts at e_0 with $\theta = 0$ and *subtracts* the angle of each corner it crosses. In the counterclockwise numbering $e_0, e_1, \dots, e_{n-1}$, the first step goes from e_0 to e_{n-1} across the corner α_{n-1}, and the program gives e_{n-1} the angle $-\alpha_{n-1}$. By (30.3) the counterclockwise angle of e_{n-1} is $\alpha_0 + \cdots + \alpha_{n-2} = \Theta_i - \alpha_{n-1}$. The next step gives e_{n-2} the angle $-\alpha_{n-1} - \alpha_{n-2}$, while (30.3) gives $\Theta_i - \alpha_{n-1} - \alpha_{n-2}$, and so on. Every edge other than e_0 gets

$$\begin{aligned} \theta_{\text{code}}(e_m) &= -(\alpha_m + \cdots + \alpha_{n-1}) \\ &= (\alpha_0 + \cdots + \alpha_{m-1}) - (\alpha_0 + \cdots + \alpha_{n-1}) \\ &= \theta(e_m) - \Theta_i, \qquad m = 1, \dots, n-1, \end{aligned}$$

an angle in $(-\Theta_i, 0)$, and e_0 keeps 0. For the five equilateral triangles the program stores $0°, -240°, -180°, -120°, -60°$ for $e_0, \dots, e_4$, which are $0°, 60°, 120°, 180°, 240°$ minus $300°$. Each corner still spans $[\theta, \theta + \alpha]$ from its counterclockwise first edge; for instance the corner between e_4 and e_0 spans $[-60°, 0°]$. Both descriptions differ by a shift of Θ_i, the angle of a full turn around the vertex: the same direction, as u and $u - \Theta_i$ were in the holonomy computation above. We shall see after the rescaling that the shift becomes exactly 2π, and disappears from every formula of the method.

Boundary vertices. At a boundary vertex the faces do not go all the way around. `Mesh::linkBoundary` sets the reference half edge `vi.edge` of such a vertex to the outgoing boundary half edge, the one with no face on its left: the outside of the mesh lies on its left and all the faces lie clockwise from it. The clockwise walk from it therefore visits every corner, and it stops when it reaches the last outgoing half edge, the one whose `pair` has no face, because there is nothing clockwise beyond it. The cone angles are then measured clockwise from the boundary edge and are negative, in $[-\Theta_i, 0]$. There is no full turn, so there is nothing to shift.

30.4. The stretched cone

The cone angles give every direction at the vertex a number, but on a circle of total angle Θ_i, not 2π. To make a plane we stretch the cone so that it closes. Define the *scale* of the vertex and the *angle* of a direction in the plane of the vertex,

$$s_i = \frac{2\pi}{\Theta_i}, \qquad \varphi = s_i\,\theta. \tag{30.4}$$

As θ runs over $[0, \Theta_i)$, φ runs over $[s_i \cdot 0, s_i\Theta_i) = [0, \frac{2\pi}{\Theta_i}\Theta_i) = [0, 2\pi)$: a full turn. This is our tangent plane of the vertex: a plane whose directions are the angles φ, with $\varphi = 0$ along the reference edge. A tangent vector at i is a

length and an angle φ, that is, the complex number $\psi = |U|\, e^{\mathsf{T}\varphi}$ of Part II, the spinor coordinate of U in the frame of the reference edge.

Picture the stretching as a map of the whole fan onto the plane. A point of the fan at the distance r from the vertex and at the cone angle θ goes to the point of the plane with the polar coordinates r and $s_i\theta$. Each edge leaving the vertex goes to a ray; each corner from θ_e to $\theta_e+\alpha$ goes to the sector from $s_i\theta_e$ to $s_i(\theta_e+\alpha) = s_i\theta_e + s_i\alpha$, of angle $s_i\alpha$; a circle of radius r around the vertex, of length $\Theta_i r$ on the cone, goes to the circle of radius r and length $2\pi r$ in the plane. The stretching multiplies every angle at the vertex by the same factor s_i, so the corners tile the plane without gap or overlap (Figure 24).

The vertex chart. This stretched plane is the flat chart of the vertex, in the sense of Section 28.1, and its frame is the constant direction $\varphi = 0$, the reference edge. The map from the cone to the plane, $(r,\theta) \mapsto (r, s_i\theta)$, is its displacement gauge, read backwards; it keeps distances from the vertex and multiplies the angles at the vertex by s_i, so it is not an isometry when $s_i \neq 1$, as the previous subsection said it could not be. The distortion costs nothing in the integrals, which are all done in the face charts; it shows only in how the vertex chart is connected to the face charts, which is Section 31.

Where the defect went. The angle defect has not disappeared: it has been spread evenly over the corners. A corner of angle α_m becomes $s_i\alpha_m$, so it absorbs the share $s_i\alpha_m - \alpha_m = (s_i-1)\,\alpha_m$ of the defect, in proportion to its angle. The shares add up to the defect:

$$\begin{aligned}
\sum_m (s_i-1)\,\alpha_m &= (s_i-1)\sum_m \alpha_m && \text{(factor out)}\\
&= (s_i-1)\,\Theta_i && (30.1)\\
&= s_i\Theta_i - \Theta_i && \text{(distribute)}\\
&= \Big(\frac{2\pi}{\Theta_i}\Big)\Theta_i - \Theta_i && (30.4)\\
&= 2\pi - \Theta_i = \kappa_i && (30.2).
\end{aligned}$$

On a cone, $s_i > 1$ and the corners are widened to close the gap; at a saddle, $s_i < 1$ and they are narrowed to remove the overlap. On a fine mesh of a smooth surface $\Theta_i \to 2\pi$, $s_i \to 1$, and the stretching vanishes.

The negative angles of the code. They are $\theta_{\text{code}} = \theta - \Theta_i$, so after the rescaling

$$s_i\theta_{\text{code}} = s_i(\theta-\Theta_i) = s_i\theta - s_i\Theta_i = \varphi - 2\pi,$$

exactly one full turn less: the same direction, and the same unit complex number, since $e^{\mathsf{T}(\varphi-2\pi)} = e^{\mathsf{T}\varphi}e^{-\mathsf{T}2\pi} = e^{\mathsf{T}\varphi}(\cos 2\pi - \mathsf{T}\sin 2\pi) = e^{\mathsf{T}\varphi}$. Every formula of the method uses angles only through such unit complex numbers, so the code can keep its negative angles.

Boundary vertices. At a boundary vertex the directions do not form a circle but an arc, from the boundary edge on one side to the boundary edge on the other: there is no loop to close, and we do not rescale, $s_i = 1$. This keeps a flat sheet exactly flat: the corner of a rectangle keeps its angle $\pi/2$.

The code. Here is the walk around every vertex, the total angle and the scale (30.4) [$s_i = 2\pi/\Theta_i$, $\varphi = s_i\theta$], followed, on lines 125 to 138, by the connection of every corner, which we explain in Section 31:

```
104     // Section 21: walk clockwise around each vertex from its reference edge,
            accumulating the cone angle
105     G.vertices.resize(mesh.numVertices());
106     for (Vertex& vi : mesh.getVertices()) {
107         const Edge* e = vi.edge;
108         double theta = 0.0;
109         G.outAngle[e] = theta;
110         bool boundary = false;
111         while (true) {
112             const Edge* pair = e->pair;
113             if (!pair->face) { boundary = true; break; } // past the last face of a
                    boundary vertex
114             const Edge* next = pair->next; // the next outgoing half edge, clockwise
115             theta -= angleAtOrigin(next); // the face between e and next lies to the
                    left of next
116             if (next == vi.edge) break;
117             G.outAngle[next] = theta;
118             e = next;
119         }
120         VertexGeometry& g = G.vertices[vi.ID];
121         g.totalAngle = -theta;
122         g.boundary = boundary || !vi.edge->face;
123         g.scale = g.boundary ? 1.0 : 2.0 * M_PI / g.totalAngle;
124     }
125     // eq. intrinsic_rho: compare the bisector of each corner in the face and in the
            plane of the vertex
126     for (Face& f : mesh.getFaces()) {
127         Vertex* v[3];
128         Edge* h[3];
129         faceCorners(f, v, h);
130         FaceGeometry& g = G.faces[f.ID];
131         for (int a = 0; a < 3; ++a) {
132             const double s = G.vertices[v[a]->ID].scale;
133             const double theta = G.outAngle.at(h[a]); // the corner spans [theta, theta
                    + angle] in the cone
134             const double inFace = g.edgeAngle[a] + 0.5 * g.angle[a];
135             const double inVertex = s * (theta + 0.5 * g.angle[a]);
136             g.rho[a] = std::polar(1.0, inFace - inVertex);
137         }
138     }
```

Line by line, for lines 104 to 124:

- Line 107 starts at the reference half edge, $e = e_0 =$ `vi.edge`, and lines 108 and 109 store $\theta(e_0) = 0$ in the map `outAngle`, which keeps the cone angle of every outgoing half edge.
- Line 112 takes the pair of the current half edge. If it has no face (line 113), the walk has passed the last face of a boundary vertex: there is nothing clockwise beyond, and the loop ends with `boundary` set.
- Line 114 is the clockwise step of Figure 25, `next = e->pair->next`.
- Line 115 subtracts the angle of the corner just crossed, the corner of the face of `next` at its origin: this is (30.3) walked backwards.
- Line 116 ends the loop when the walk is back at e_0, after crossing the last corner. At that point $\theta = -\Theta_i$. Otherwise line 117 stores the cone angle of `next`, and line 118 moves on.
- Line 121 is the total angle (30.1) [$\Theta_i = \sum_{f \ni i} \alpha_{i,f}$], $\Theta_i = -\theta$, the sum of all the corners crossed.

- Line 122 records whether the vertex is on the boundary; the second test, `!vi.edge->face`, says the same as the first for the meshes built by `linkBoundary`, and is only a safeguard.
- Line 123 is the scale (30.4), $s_i = 2\pi/\Theta_i$ for an interior vertex and 1 at the boundary.

30.5. What the vertex normal does instead

The vertex plane of Part II solves the same problem in another way. Each face is rolled into the plane orthogonal to n_i by the rotor $Q_{i\to f}$ of (22.3) [$Q_{i\to f} = \frac{1+n_f n_i}{|1+n_f n_i|}$], run backwards. A rotation preserves angles, so every corner keeps its true angle $\alpha_{i,f}$ in the vertex plane, and the rolled corners together cover Θ_i, not 2π. They cannot tile the plane: they leave gaps between neighbors when $\Theta_i < 2\pi$ and overlap when $\Theta_i > 2\pi$, and the gaps add up to the angle defect. Where the gaps fall depends on how each face is tilted with respect to the vertex normal, an extrinsic fact. The rescaling (30.4) spreads the defect in proportion to the angles, an intrinsic fact. Even at a flat vertex, $\Theta_i = 2\pi$, the rolled corners fit edge to edge only if all the faces are rolled about one line, as along a straight fold; otherwise they leave gaps and overlaps that cancel in total but not one by one, which is what happens on the Miura-ori of Figure 22. On a symmetric vertex the two planes agree; on any other they differ, as the next two examples show.

How to read the tables of the two examples. Each row is one corner at the vertex, in counterclockwise order from the reference edge, and α is its angle. The two groups of columns say where each method puts that corner in its plane of the vertex, as a sector of directions. "From" is the angle, in the plane of the vertex, of the first edge of the corner (counterclockwise), measured from the reference direction; "width" is the angle of the sector. For the intrinsic plane, from $= s_i\,\theta_e$ and width $= s_i\,\alpha$, by (30.4). For the extrinsic plane, "from" is the angle of the rolled first edge measured from t_i, and the width is α itself, since rolling keeps angles; "gap" is the empty angle between the end of this sector and the start of the next. So in the intrinsic columns each row ends where the next begins, while in the extrinsic columns from + width + gap gives the next "from". The program computes the columns with these lines of `workedExamplesIntrinsic` in `ga_fem.cpp`:

```
686         for (Face& f : mesh.getFaces()) {
687             Vertex* v[3];
688             Edge* h[3];
689             intrinsic::faceCorners(f, v, h); // the apex is the first vertex of every
                    face
690             const double alpha = G.faces[f.ID].angle[0];
691             const double cone = std::fmod(G.outAngle.at(h[0]) + va.totalAngle, va.
                    totalAngle);
692             // Part II: roll the face into the plane of the vertex normal and read the
                    direction of its first edge
693             const Vector3d d = ga::rotorBetween(g.faceNormal[f.ID], apex.n).rotate((v
                    [1]->p - v[0]->p).normalized());
694             double rolled = std::atan2(g.vertexT[0].cross(d).dot(apex.n), g.vertexT[0].
                    dot(d));
695             if (rolled < -1e-9) rolled += 2 * M_PI;
696             corners.push_back({ alpha, cone, rolled });
697         }
```

The apex is the first vertex of every face of the pyramid, so its corner is `a = 0`, and `h[0]` is the first edge of the corner. Line 691 turns the code's negative cone angle back into the counterclockwise one of (30.3): adding Θ_i and reducing modulo Θ_i maps $(-\Theta_i, 0]$ onto $[0, \Theta_i)$, the shift of the previous subsection undone. The table prints s_i times it as "from". Lines 693 to 695 are Part II: line 693 rolls the first edge of the face into the plane of the vertex normal, with the smallest rotor from n_f to n_i; line 694 measures its angle from t_i, counterclockwise about n_i, with `atan2` of the sine $(t_i \times d) \cdot n_i$ and the cosine $t_i \cdot d$; and line 695 brings the angle into $[0, 2\pi)$.

Worked example 30.1. *The apex of a pyramid over a regular pentagon of radius* 1*, at the height* $h = 0.6180$ *that makes the five faces equilateral. Each corner at the apex is* $60°$*, so* $\Theta_i = 5 \cdot 60° = 300°$*,* $\kappa_i = 360° - 300° = 60°$ *and* $s_i = 360/300 = 1.2$*; the vertex normal is vertical, by symmetry. The program prints, for the corners in counterclockwise order from the reference edge:*

		intrinsic		*extrinsic (rolled)*		
corner	α	*from*	*width*	*from*	*width*	*gap*
0	$60°$	$0°$	$72°$	$6°$	$60°$	$12°$
1	$60°$	$72°$	$72°$	$78°$	$60°$	$12°$
⋮						
4	$60°$	$288°$	$72°$	$294°$	$60°$	$12°$

The intrinsic plane stretches each corner from $60°$ *to* $1.2 \times 60° = 72°$*, and five of them make* $5 \times 72° = 360°$*. The extrinsic plane keeps each at* $60°$ *and leaves five gaps of* $12°$*, again* $5 \times 12° = 60°$ *in total. Its first sector starts at* $6°$*, not at* $0°$*, because the reference direction* t_i *is the projection of the reference edge onto the plane of the vertex, not its rolled image. The middle directions of the corners, their bisectors, are the same in both: intrinsic* $0° + 72°/2 = 36°$*, extrinsic* $6° + 60°/2 = 36°$*, and then* $108°, 180°, \ldots$ *Since the connection of the next section matches bisectors, the two connections are equal at this vertex; the program finds them equal to rounding (*$2.5 \cdot 10^{-14}$ *degrees).*

Worked example 30.2. *Move the apex of Example 30.1 by* 0.4 *towards one corner of the pentagon. The corners become* $70.49°$*,* $70.49°$*,* $51.37°$*,* $46.81°$*,* $51.37°$*, with* $\Theta_i = 290.53°$*,* $\kappa_i = 69.47°$ *and* $s_i = 360/290.53 = 1.2391$*. The vertex normal is still vertical. The normal of the code averages the cross products* $(x_k - x_i) \times (x_{k+1} - x_i)$ *of the faces, where* x_k *runs over the ring of neighbors (with* x_{k+1} *the next one around). Each of them expands, by the distributive law, as*

$$
\begin{aligned}
&(x_k - x_i) \times (x_{k+1} - x_i) \\
&\quad = x_k \times x_{k+1} - x_k \times x_i - x_i \times x_{k+1} + x_i \times x_i && \textit{(distribute)} \\
&\quad = x_k \times x_{k+1} + x_i \times x_k - x_i \times x_{k+1} + 0 && \textit{(}a \times b = -b \times a,\ a \times a = 0\textit{)} \\
&\quad = x_k \times x_{k+1} - x_i \times (x_{k+1} - x_k) && \textit{(factor } x_i \times\textit{).}
\end{aligned}
$$

Summed around the ring, the differences $x_{k+1} - x_k$ add up to zero (the sum telescopes and the ring is closed), so the terms with x_i cancel: the sum does not depend on the apex at all, and equals twice the vector area of the pentagon, which is vertical. Figure 26 shows where each method puts the corners:

		intrinsic		*extrinsic (rolled)*		
corner	α	*from*	*width*	*from*	*width*	*gap*
0	70.488°	0°	87.341°	13.146°	70.488°	23.662°
1	70.488°	87.341°	87.341°	107.296°	70.488°	15.181°
2	51.373°	174.682°	63.655°	192.964°	51.373°	7.720°
3	46.814°	238.337°	58.007°	252.057°	46.814°	7.720°
4	51.373°	296.345°	63.655°	306.592°	51.373°	15.181°

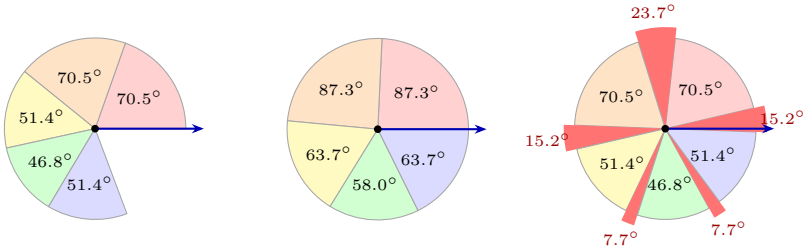
Read the first row: in the intrinsic plane the corner of 70.488° *becomes the sector from* 0° *to* $87.341° = 1.2391 \times 70.488°$, *and the next corner starts exactly there. In the extrinsic plane it keeps its* 70.488°, *from* 13.146° *to* $13.146° + 70.488° = 83.634°$, *and the next corner starts* 23.662° *later, at* $83.634° + 23.662° = 107.296°$.

Both spread the defect of 69.47° *over the five corners, but differently. The intrinsic plane gives each corner* $(s_i - 1)\alpha$*:* 16.85°, 16.85°, 12.28°, 11.19°, 12.28°, *in proportion to the angles. The extrinsic plane leaves gaps of* 23.66°, 15.18°, 7.72°, 7.72°, 15.18°, *which depend on the tilt of the faces: the largest gap opens between the two faces on the side towards which the apex leans, which are tilted most with respect to the vertex normal. The bisectors now differ: from + half the width is* $0° + 87.341°/2 = 43.67°$ *against* $13.146° + 70.488°/2 = 48.39°$ *in the first row, and the five differences are* $4.7°, 11.5°, 12.1°, 8.1°$ *and* 4.1°*; so do the two connections, by up to* 12.1°.

Part III · The Intrinsic Method: Everything from Edge Lengths

31. Stage 5, the Gauge Transitions: from a Vertex Chart into a Face Chart

We now have a flat chart at every vertex, made from the angles, and a flat chart in every face, the layout with the frame along its x axis. The finite element method needs one more thing, the third ingredient of Stage 4: for each corner, the transition (28.12) $[\,\psi_B = e^{-\mathsf{T}\theta_{AB}}\psi_A\,]$ that converts the coordinate of a vector in the chart of the vertex into its coordinate in the chart of the face. In the smooth theory it would be the parallel transport (28.9) $[\,\psi(s) = e^{-\mathsf{T}\int_0^s \omega}\psi(0)\,]$ along the short way from the vertex into the face, combined with the change of frame; on the mesh it must be chosen, and the choice carries the curvature. With it, Stage 5 ends with the finite element space (Section 31.5).



cut open and flattened: $\Theta_i = 290.5°$, a gap of 69.5°

intrinsic: every corner stretched by $s_i = 1.2391$

extrinsic: every corner rolled rigidly, gaps in red

FIGURE 26. The leaning pyramid of Example 30.2, seen from its apex, corners counterclockwise from the reference edge (blue). Left: cut open and flattened, with a gap of 69.5°, the angle defect. Middle: the intrinsic vertex plane, every corner stretched by the same factor. Right: the extrinsic vertex plane of Part II, every corner rolled rigidly into the plane orthogonal to the vertex normal; the defect falls into the gaps between them (red), unevenly.

31.1. What the finite element method needs

The element matrix (24.3) $[\,\mathrm{L}_{ij}^f = A_f(\nabla\phi_i\cdot\nabla\phi_j)\,e^{\mathsf{T}(\rho_j^f-\rho_i^f)}\,]$ uses, for each corner of a face, a unit complex number $e^{\mathsf{T}\rho_i^f}$ with the meaning of (22.4) $[\,e^{\mathsf{T}_f\rho_i^f} = t_f\,(Qt_i\widetilde{Q})\,]$: a vector of angle φ in the plane of vertex i is the vector of angle $\varphi+\rho_i^f$ in the face frame. In words, ρ_i^f is where the reference direction of the vertex, $\varphi = 0$, lands in the face. In spinor coordinates, the transport from the vertex into the face is the multiplication

$$\psi = |\psi|\,e^{\mathsf{T}\varphi} \;\longmapsto\; e^{\mathsf{T}\rho_i^f}\psi = |\psi|\,e^{\mathsf{T}\rho_i^f}e^{\mathsf{T}\varphi} = |\psi|\,e^{\mathsf{T}(\varphi+\rho_i^f)},$$

a *rotation* of the plane by ρ_i^f (the exponents add because T commutes with itself).

It must be a rotation, and not some other map of directions, for this reason. The trial field (22.8) $[\,\psi_f = \sum_{i\in f}\phi_i\,e^{\mathsf{T}_f\rho_i^f}\psi_i\,]$ builds the field in the face from the vectors of the three vertices, each carried into the face by its connection. At the vertex i itself, where $\phi_i = 1$ and the other two hats vanish, the field is $e^{\mathsf{T}\rho_i^f}\psi_i$, and this must be the vector ψ_i of the vertex, with the same length. And the mass and the stiffness integrate inner products $\langle\psi\widetilde{\chi}\rangle$ of such vectors over the face: if two vectors at the vertex make an angle δ, their images in the face must make the same angle δ, or the inner products measured in the face would not be those of the vertex. The maps of the plane that keep all lengths and all angles, and do not reflect, are exactly the rotations, the multiplications by unit complex numbers. So the connection of a corner is a single angle ρ_i^f. In gauge language: the transitions between charts must belong

to the gauge group, the rotations $e^{\mathsf{T}\rho}$, because those are the changes of frame that leave the densities (28.11) of the weak form unchanged (Section 28.4).

31.2. The exact correspondence is not a rotation

What angle? Let us first see what the stretched cone itself says. Look at one corner of the face f at the vertex i. Let e be the first edge of the corner in the counterclockwise order, with cone angle θ_e at the vertex and angle β_e in the face frame; let α be the angle of the corner. A direction inside the corner that makes the angle $u \in [0, \alpha]$ with e has

$$\text{in the face:} \quad \beta = \beta_e + u, \qquad \text{at the vertex:} \quad \varphi = s_i(\theta_e + u),$$

the first because the face is flat and angles in it simply add, the second by (30.4) [$\varphi = s_i\theta$], since the cone angle of the direction is $\theta = \theta_e + u$. The rotation that would carry this vertex direction to this face direction is the difference, which we expand and regroup:

$$\begin{aligned} \beta - \varphi &= (\beta_e + u) - \big(s_i(\theta_e + u)\big) && \text{(substitute)} \\ &= \beta_e + u - s_i\theta_e - s_i u && \text{(distribute } s_i \text{ and the minus sign)} \\ &= (\beta_e - s_i\theta_e) + (1 - s_i)\, u && \text{(group the terms with } u\text{)}. \end{aligned}$$

It changes across the corner, by $(1 - s_i)\,\alpha$ from $u = 0$ to $u = \alpha$, unless $s_i = 1$. The exact correspondence stretches the corner, because the vertex plane is stretched and the face is not; it is not a rotation. For the corner of Figure 27, with $\theta_e = 60°$, $\alpha = 60°$, $s_i = 1.2$ and $\beta_e = 0$, the constant is $\beta_e - s_i\theta_e = 0 - 1.2 \cdot 60° = -72°$ and the slope is $1 - s_i = -0.2$; the difference is $-72°$ along the first edge ($u = 0$), $-72° - 0.2 \cdot 30° = -78°$ in the middle and $-72° - 0.2 \cdot 60° = -84°$ along the second edge ($u = 60°$). We need a single number ρ_i^f for the corner, and no single number is right for every direction.

31.3. The best rotation: match the bisectors

When no single number fits a whole range of values, we choose the number closest to all of them on average, in the sense of least squares: the one that makes the average of the squared mismatch as small as possible. Here the values are the exact rotations $\beta - \varphi$ of the directions of the corner, one for each $u \in [0, \alpha]$, and "on average" means the integral over u, in which each direction of the corner counts equally.

Step 1: name the constants. Write the exact rotation as a linear function of u,

$$m(u) = c + k\,u, \qquad c = \beta_e - s_i\theta_e, \qquad k = 1 - s_i,$$

with c its value along the first edge and k its slope. The quantity to minimize over ρ is

$$E(\rho) = \int_0^\alpha \big(\rho - m(u)\big)^2\, du = \int_0^\alpha \big(\rho - (c + ku)\big)^2\, du.$$

Step 2: expand the integral. Expand the square, $(\rho - m)^2 = \rho^2 - 2\rho\, m + m^2$, and split the integral into three:

$$\begin{aligned} E(\rho) &= \int_0^\alpha \big(\rho^2 - 2\rho\, m(u) + m(u)^2\big)\, du && \text{(expand the square)} \\ &= \int_0^\alpha \rho^2\, du - \int_0^\alpha 2\rho\, m(u)\, du + \int_0^\alpha m(u)^2\, du && \text{(the integral of a sum)} \\ &= \rho^2 \int_0^\alpha du - 2\rho \int_0^\alpha m(u)\, du + \int_0^\alpha m(u)^2\, du && (\rho \text{ does not depend on } u) \\ &= \alpha\, \rho^2 - 2\rho\, b + C, && \Big(\textstyle\int_0^\alpha du = \alpha\Big) \end{aligned}$$

where we named the two numbers that do not depend on ρ:

$$b = \int_0^\alpha m(u)\, du, \qquad C = \int_0^\alpha m(u)^2\, du.$$

So E is a quadratic polynomial in ρ with the positive leading coefficient α: a parabola opening upwards, smallest where its derivative vanishes.

Step 3: the derivative, set to zero. Differentiate term by term with respect to ρ:

$$\frac{dE}{d\rho} = \frac{d}{d\rho}(\alpha\rho^2) - \frac{d}{d\rho}(2b\rho) + \frac{d}{d\rho}C = 2\alpha\, \rho - 2b + 0.$$

Setting it to zero and solving for ρ, one operation per line:

$$\begin{aligned} &2\alpha\, \rho - 2b = 0 \\ &\qquad 2\alpha\, \rho = 2b && \text{(add } 2b\text{)} \\ &\qquad\quad \rho = \frac{b}{\alpha} = \frac{1}{\alpha}\int_0^\alpha m(u)\, du && \text{(divide by } 2\alpha\text{; substitute } b\text{).} \end{aligned}$$

The second derivative is $2\alpha > 0$, so this is the minimum. It is the *average* of the exact rotations over the corner: the constant closest, in least squares, to a function is its mean.

Step 4: the mean of the linear function. Substitute $m(u) = c + ku$ into b:

$$\begin{aligned} b &= \int_0^\alpha (c + ku)\, du && \text{(substitute } m\text{)} \\ &= c\int_0^\alpha du + k\int_0^\alpha u\, du && \text{(split; constants out)} \\ &= c\,\alpha + k\,\frac{\alpha^2}{2} && \Big(\textstyle\int_0^\alpha u\, du = \big[\frac{u^2}{2}\big]_0^\alpha = \frac{\alpha^2}{2}\Big), \end{aligned}$$

and divide by α:

$$\rho = \frac{1}{\alpha}\Big(c\,\alpha + k\,\frac{\alpha^2}{2}\Big) = c + k\,\frac{\alpha}{2} = m\Big(\frac{\alpha}{2}\Big),$$

the value of m at the middle of the interval, $u = \alpha/2$. The mean of a linear function is its value at the midpoint.

Step 5: substitute back. Put $c = \beta_e - s_i\theta_e$ and $k = 1 - s_i$ into $\rho = c + k\frac{\alpha}{2}$:

$$\begin{aligned}
\rho &= \big(\beta_e - s_i\theta_e\big) + \big(1 - s_i\big)\frac{\alpha}{2} && \text{(substitute } c \text{ and } k\text{)}\\
&= \beta_e - s_i\theta_e + \frac{\alpha}{2} - s_i\frac{\alpha}{2} && \text{(distribute } \tfrac{\alpha}{2}\text{)}\\
&= \Big(\beta_e + \frac{\alpha}{2}\Big) - \Big(s_i\theta_e + s_i\frac{\alpha}{2}\Big) && \text{(regroup: terms without } s_i\text{, terms with } s_i\text{)}\\
&= \Big(\beta_e + \frac{\alpha}{2}\Big) - s_i\Big(\theta_e + \frac{\alpha}{2}\Big) && \text{(factor } s_i\text{).}
\end{aligned}$$

This is the connection of the corner:

$$\rho_i^f = \Big(\beta_e + \frac{\alpha}{2}\Big) - s_i\Big(\theta_e + \frac{\alpha}{2}\Big). \tag{31.1}$$

What it means. The direction at the middle of the corner, $u = \alpha/2$, is its *bisector*. The first bracket, $\beta_e + \frac{\alpha}{2}$, is the angle of the bisector in the face frame, and the second, $s_i(\theta_e + \frac{\alpha}{2})$, is s_i times its cone angle, the angle of the bisector in the plane of the vertex. So ρ_i^f is the rotation that carries the bisector of the corner in the vertex plane exactly onto the bisector of the corner in the face: the intrinsic connection *matches the bisectors*. Every other direction of the corner is carried with the error

$$\rho - m(u) = \Big(c + k\frac{\alpha}{2}\Big) - \big(c + ku\big) = -k\Big(u - \frac{\alpha}{2}\Big) = (s_i - 1)\Big(u - \frac{\alpha}{2}\Big),$$

zero at the bisector and largest at the two edges, $u = 0$ and $u = \alpha$, where it is $\mp(s_i - 1)\alpha/2$: half the stretching on each side (Figure 27).

For a flat vertex, $s_i = 1$, (31.1) becomes

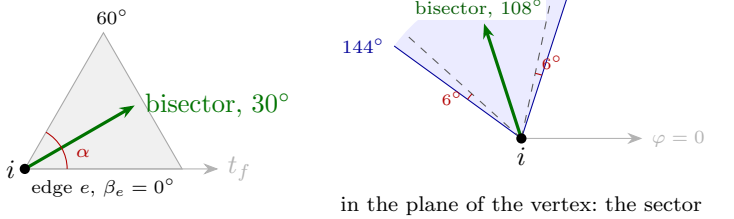
$$\rho_i^f = \Big(\beta_e + \frac{\alpha}{2}\Big) - (1)\Big(\theta_e + \frac{\alpha}{2}\Big) = \beta_e - \theta_e,$$

with no mismatch at all: $k = 1 - 1 = 0$, the exact rotation $m(u) = c$ is the same for every direction, and the least-squares fit is exact. The vertex plane is then the plane of the flattened faces, and transport into each face is the exact unfolding. So on every part of a mesh that is intrinsically flat, however it is folded in space, the intrinsic connection is the exact parallel transport. This is why it reproduces the field of the flat paper on the Miura-ori of Figure 22.

The code. Lines 125 to 138 of the listing of Section 30 evaluate (31.1) for every corner of every face. The loop over the corners is:

```
131         for (int a = 0; a < 3; ++a) {
132             const double s = G.vertices[v[a]->ID].scale;
133             const double theta = G.outAngle.at(h[a]); // the corner spans [theta, theta
                    + angle] in the cone
134             const double inFace = g.edgeAngle[a] + 0.5 * g.angle[a];
135             const double inVertex = s * (theta + 0.5 * g.angle[a]);
136             g.rho[a] = std::polar(1.0, inFace - inVertex);
137         }
```

Line 132 is the scale s_i of the vertex `v[a]`. Line 133 reads the cone angle of `h[a]`, the half edge from `v[a]` to `v[a+1]`. It is the first edge e of the corner in the counterclockwise order: the face lies to the left of `h[a]`, so turning counterclockwise from `h[a]` sweeps through the face to the edge towards



in the face f: directions from 0° to 60°

in the plane of the vertex: the sector $s_i\alpha = 72°$ wide (blue); the face, turned by $-\rho_i^f = 78°$, covers 78° to 138° (dashed)

FIGURE 27. The second corner of the cone of Figure 24 ($\theta_e = 60°$, $\alpha = 60°$, $s_i = 1.2$) with the face frame along its first edge, $\beta_e = 0$. Left: in the face, the directions of the corner run from 0° to 60°. Right: in the plane of the vertex, the corner is the sector from 72° to 144°. The connection (31.1), $\rho_i^f = 30° - 108° = -78°$, carries bisector to bisector; the face's edges land 6° inside the sector on each side.

`v[a+2]`. So this is θ_e, in the code's convention, and the corner spans $[\theta_e, \theta_e+\alpha]$ in the cone, as the comment says. Line 134 is the first bracket of (31.1), the bisector in the face, $\beta_e + \alpha/2$, with $\beta_e =$ `edgeAngle[a]` from `layOut`. Line 135 is the second bracket, the bisector in the plane of the vertex, $s_i(\theta_e+\alpha/2)$. Line 136 stores the unit complex number $e^{\mathsf{T}\rho_i^f}$, since `std::polar(1.0, x)` is $\cos x + \mathsf{T}\sin x$. The code's negative θ_e is Θ_i smaller than that of (30.3), so line 135 is $s_i\Theta_i = 2\pi$ smaller, and the stored complex number is the same.

Worked example 31.1. *In the cone of five equilateral triangles, the corner of Figure 27 has $\theta_e = 60°$ and $\alpha = 60°$; with the face frame along its first edge, $\beta_e = 0$. Then (31.1) [$\rho_i^f = (\beta_e + \frac{\alpha}{2}) - s_i(\theta_e + \frac{\alpha}{2})$] gives*

$$\rho_i^f = \left(0 + \frac{60°}{2}\right) - 1.2\left(60° + \frac{60°}{2}\right) = (30°) - 1.2\,(90°) = 30° - 108° = -78°:$$

the reference direction of the vertex lands at $-78°$ in the face, and the bisector of the vertex sector, at 108°, lands at $108° - 78° = 30°$, the bisector of the face corner. It is the middle value $-78°$ of the exact rotations, which ran from $-72°$ to $-84°$. The program computes the same corner with $\theta_e = 60° - 300° = -240°$: line 135 gives $1.2\,(-240° + 30°) = 1.2\,(-210°) = -252°$ instead of 108°, a full turn less, and $\rho_i^f = 30° - (-252°) = 282°$, which is $-78° + 360°$.

31.4. Going around the vertex

The bisector rule makes a small error in every corner. Do these errors add up when we go around the vertex? We now show that they add up to exactly what is needed to account for the curvature of the vertex.

The transport from a face f into its neighbor g around the vertex i can be done in two ways. The first is to unfold g onto f across their common

edge: this is the exact transport. The second is to go through the plane of the vertex, from f to the vertex with $e^{-\mathsf{T}\rho_i^f}$ and from the vertex into g with $e^{\mathsf{T}\rho_i^g}$; this is what the finite element method does, since the transport between two corners of the same vertex is $e^{\mathsf{T}\rho_i^g}e^{-\mathsf{T}\rho_i^f} = e^{\mathsf{T}(\rho_i^g-\rho_i^f)}$. Unfolding preserves the cone angle: a direction of f at the cone angle θ is carried to the direction of g at the same cone angle, because in the flattened fan both faces lie in one plane. Through the vertex the direction is carried with an extra rotation, which we now compute.

The data. Let g follow f counterclockwise around i, so that the second edge of the corner of f is the first edge of the corner of g. Let θ_f and θ_g be the cone angles of the first edges of the two corners; then, by (30.3) $[\,\theta(e_{m+1}) = \theta(e_m) + \alpha_m\,]$,

$$\theta_g = \theta_f + \alpha_f.$$

Let β_e^f, β_e^g be the angles of those first edges in the two face frames. The connection (31.1) for the two corners reads

$$\rho_i^f = \Big(\beta_e^f + \frac{\alpha_f}{2}\Big) - s_i\Big(\theta_f + \frac{\alpha_f}{2}\Big),$$
$$\rho_i^g = \Big(\beta_e^g + \frac{\alpha_g}{2}\Big) - s_i\Big(\theta_g + \frac{\alpha_g}{2}\Big).$$

The two routes. Take a direction of f at the cone angle θ. In the frame of f its angle is $\beta = \beta_e^f + (\theta - \theta_f)$: the angle of the first edge plus the angle turned from it.

- Through the vertex plane: the transport $e^{-\mathsf{T}\rho_i^f}$ subtracts ρ_i^f, giving $\varphi = \beta - \rho_i^f$ in the vertex chart; then $e^{\mathsf{T}\rho_i^g}$ adds ρ_i^g, giving $\beta' = \varphi + \rho_i^g = \beta - \rho_i^f + \rho_i^g$ in the frame of g.
- By unfolding: the direction at the same cone angle θ in g, of angle $\beta_e^g + (\theta - \theta_g)$ in the frame of g.

The extra rotation. It is the difference of the two, $\beta' - (\beta_e^g + \theta - \theta_g)$. We substitute, one ingredient at a time, each inside parentheses:

$$\begin{aligned}
&\beta' - \big(\beta_e^g + \theta - \theta_g\big)\\
&\quad = \big(\beta - \rho_i^f + \rho_i^g\big) - \big(\beta_e^g + \theta - \theta_g\big) && \text{(substitute } \beta'\text{)}\\
&\quad = \big(\beta_e^f + \theta - \theta_f\big) - \rho_i^f + \rho_i^g - \beta_e^g - \theta + \theta_g && \text{(substitute } \beta\text{)}\\
&\quad = \beta_e^f - \theta_f - \rho_i^f + \rho_i^g - \beta_e^g + \theta_g && (+\theta - \theta = 0).
\end{aligned}$$

Now insert the two connections, each in its own parentheses:

$$
\begin{aligned}
&= \beta_e^f - \theta_f - \Big[\Big(\beta_e^f + \frac{\alpha_f}{2}\Big) - s_i\Big(\theta_f + \frac{\alpha_f}{2}\Big)\Big] \\
&\qquad + \Big[\Big(\beta_e^g + \frac{\alpha_g}{2}\Big) - s_i\Big(\theta_g + \frac{\alpha_g}{2}\Big)\Big] - \beta_e^g + \theta_g \quad \text{(substitute } \rho_i^f, \rho_i^g\text{)} \\
&= \beta_e^f - \theta_f - \beta_e^f - \frac{\alpha_f}{2} + s_i\Big(\theta_f + \frac{\alpha_f}{2}\Big) \\
&\qquad + \beta_e^g + \frac{\alpha_g}{2} - s_i\Big(\theta_g + \frac{\alpha_g}{2}\Big) - \beta_e^g + \theta_g \qquad \text{(open the square brackets)} \\
&= -\theta_f - \frac{\alpha_f}{2} + s_i\Big(\theta_f + \frac{\alpha_f}{2}\Big) \\
&\qquad + \theta_g + \frac{\alpha_g}{2} - s_i\Big(\theta_g + \frac{\alpha_g}{2}\Big) \qquad (\beta_e^f, \beta_e^g \text{ cancel}) \\
&= -\Big(\theta_f + \frac{\alpha_f}{2}\Big) + s_i\Big(\theta_f + \frac{\alpha_f}{2}\Big) \\
&\qquad + \Big(\theta_g + \frac{\alpha_g}{2}\Big) - s_i\Big(\theta_g + \frac{\alpha_g}{2}\Big) \qquad \text{(group as bisectors)} \\
&= -(1 - s_i)\Big(\theta_f + \frac{\alpha_f}{2}\Big) + (1 - s_i)\Big(\theta_g + \frac{\alpha_g}{2}\Big) \qquad \text{(factor in pairs)} \\
&= (1 - s_i)\Big[\Big(\theta_g + \frac{\alpha_g}{2}\Big) - \Big(\theta_f + \frac{\alpha_f}{2}\Big)\Big] \qquad \text{(factor again).}
\end{aligned}
$$

The bracket is the cone angle from the bisector of f to the bisector of g. Substitute $\theta_g = \theta_f + \alpha_f$:

$$
\begin{aligned}
\Big(\theta_g + \frac{\alpha_g}{2}\Big) - \Big(\theta_f + \frac{\alpha_f}{2}\Big) &= \Big((\theta_f + \alpha_f) + \frac{\alpha_g}{2}\Big) - \theta_f - \frac{\alpha_f}{2} \quad \text{(substitute } \theta_g\text{)} \\
&= \alpha_f - \frac{\alpha_f}{2} + \frac{\alpha_g}{2} \qquad (\theta_f \text{ cancels}) \\
&= \frac{\alpha_f}{2} + \frac{\alpha_g}{2} = \tfrac{1}{2}(\alpha_f + \alpha_g),
\end{aligned}
$$

half of each corner. So the extra rotation from f to g is

$$\delta_{fg} = (1 - s_i)\,\tfrac{1}{2}(\alpha_f + \alpha_g),$$

$(1 - s_i)$ times the cone angle crossed from bisector to bisector. The direction of the original vector, the cone angle θ, has dropped out: the extra rotation is the same for every vector, a rotation and nothing worse.

Once around. Now go once around the vertex, through all its n faces in turn, $f_0, f_1, \ldots, f_{n-1}$ and back to f_0. The extra rotations add up to

$$
\begin{aligned}
\sum_{m=0}^{n-1} \delta_{f_m f_{m+1}} &= \sum_{m=0}^{n-1} (1 - s_i)\,\tfrac{1}{2}\big(\alpha_m + \alpha_{m+1}\big) \qquad \text{(indices mod } n\text{)} \\
&= (1 - s_i)\,\tfrac{1}{2}\Big(\sum_m \alpha_m + \sum_m \alpha_{m+1}\Big) \qquad \text{(split the sum)} \\
&= (1 - s_i)\,\tfrac{1}{2}\,(\Theta_i + \Theta_i) \qquad \text{(both are (30.1))} \\
&= (1 - s_i)\,\Theta_i = \Theta_i - s_i\Theta_i \qquad \text{(distribute)} \\
&= \Theta_i - 2\pi = -\kappa_i \qquad (s_i\Theta_i = 2\pi;\ (30.2)).
\end{aligned}
$$

The unfolding, on the other hand, comes back rotated by the holonomy of the vertex, the angle defect $+\kappa_i$, as we computed in Section 30. The route through the vertex is the unfolding plus the extra rotations, so once around it turns by $\kappa_i + (-\kappa_i) = 0$: through the plane of the vertex, a vector comes back unchanged, as it must, since the vertex plane is one plane. Directly: the transports from each face to the next multiply around the vertex to

$$e^{\mathsf{T}(\rho_i^{f_1} - \rho_i^{f_0})}\, e^{\mathsf{T}(\rho_i^{f_2} - \rho_i^{f_1})} \cdots e^{\mathsf{T}(\rho_i^{f_0} - \rho_i^{f_{n-1}})}$$
$$= e^{\mathsf{T}\left((\rho_i^{f_1} - \rho_i^{f_0}) + (\rho_i^{f_2} - \rho_i^{f_1}) + \cdots + (\rho_i^{f_0} - \rho_i^{f_{n-1}})\right)} = e^{\mathsf{T}\,0} = 1,$$

because every $\rho_i^{f_m}$ appears in the exponent once with the sign $+$ and once with the sign $-$. In the cone of five equilateral triangles, each step from one face to the next adds $(1-1.2) \times \frac{1}{2}(60^\circ + 60^\circ) = -0.2 \times 60^\circ = -12^\circ$ to the unfolding, five steps add -60°, and the unfolding contributes $+60^\circ$. The vertex plane does not ignore the curvature; it accounts for it by correcting the transport between consecutive faces, each by a share of the defect proportional to the angle crossed.

Where the curvature sits. In the gauge language of Section 28.5: the frame is constant on each face, the x axis of its layout, and there is one frame at each vertex, the reference edge of its stretched cone. The rotation gauge vanishes inside every face, and so does the field strength. The corner angles ρ_i^f are the *transition functions* between the chart of a vertex and the charts of its faces: the discrete counterpart of (28.12), and of the change of frame (18.10) [$D'_a\psi' = e^{-\mathsf{T}\theta} D_a\psi$]. A field strength cannot be gauged away, so the curvature has been moved into the transitions. Glued to each other across the edges by the unfoldings, the face charts around a vertex multiply to $e^{\mathsf{T}\kappa_i}$, the holonomy (28.15) with all of $\int K\, dA$ concentrated at one point. Glued to the single chart of the vertex, as the finite element method glues them, the composite transitions $e^{\mathsf{T}(\rho_i^g - \rho_i^f)}$ multiply to 1. The two gluings differ by $\delta_{fg} = (1 - s_i)\,\frac{1}{2}(\alpha_f + \alpha_g)$ per step, and these differences add up to $-\kappa_i$: the bisector rule spreads the field strength of the vertex over its corners, in proportion to their angles.

Gauge transformations on the mesh. Two arbitrary choices remain in the discrete gauge fields, and neither may change the result.

- *The frame of a face.* Turning it by γ, for instance by taking another edge as the x axis, replaces every β_e of the face by $\beta_e - \gamma$, and so, by (31.1),

$$\rho_i^{f\prime} = \Big((\beta_e - \gamma) + \frac{\alpha}{2}\Big) - s_i\Big(\theta_e + \frac{\alpha}{2}\Big) = \rho_i^f - \gamma$$

  for every corner of the face: the coordinates in the face become $e^{-\mathsf{T}\gamma}\psi_f$, the discrete $\psi' = e^{-\mathsf{T}\theta}\psi$. The unfolding from a face f to a neighbor g changes from $e^{\mathsf{T}\tau}$ to $e^{-\mathsf{T}\gamma_g} e^{\mathsf{T}\tau} e^{\mathsf{T}\gamma_f}$, and around a vertex these products telescope: the holonomy $e^{\mathsf{T}\kappa_i}$ is gauge invariant, as the field strength is.
- *The reference edge of a vertex.* Taking e_1 instead of e_0 replaces every cone angle θ by $\theta - \alpha_0$ (up to a full turn of the cone), and every angle φ of

the vertex chart by $\varphi - s_i\alpha_0$, so the coordinate becomes $\psi_i' = e^{-\mathsf{T}s_i\alpha_0}\psi_i$. By (31.1),

$$\rho_i^{f\prime} = \Big(\beta_e + \frac{\alpha}{2}\Big) - s_i\Big((\theta_e - \alpha_0) + \frac{\alpha}{2}\Big) = \rho_i^f + s_i\alpha_0,$$

and the vector carried into each face does not change: $e^{\mathsf{T}\rho_i^{f\prime}}\psi_i' = e^{\mathsf{T}(\rho_i^f + s_i\alpha_0)}e^{-\mathsf{T}s_i\alpha_0}\psi_i = e^{\mathsf{T}\rho_i^f}\psi_i$. For the corner of Example 31.1, with e_1 as the reference edge, $\theta_e = 60^\circ - 60^\circ = 0$ and $\rho_i^f = 30^\circ - 1.2\cdot(0 + 30^\circ) = -6^\circ$, which is $-78^\circ + 1.2\cdot 60^\circ = -78^\circ + 72^\circ$.

31.5. Stage 5 completed: the finite element space

What we want. A finite dimensional space V_h of fields, built from the charts and transitions just made, in which Stage 6 will look for the solution; its basis functions; their derivatives; and how close it comes to the conforming requirement $V_h \subset H^1$ of Stage 2.

The basis functions. For each vertex i define, in the chart of each face f that contains i,

$$\Phi_i\big|_f = \phi_i\, e^{\mathsf{T}\rho_i^f}, \qquad \Phi_i = 0 \text{ on the faces that do not contain } i, \tag{31.2}$$

the scalar hat function of the vertex times the transition from the chart of the vertex into the chart of the face. A complex coefficient ψ_i attached to the vertex gives the field $\Phi_i\psi_i$: the vector of the vertex, carried into each of its faces by the transition and faded out by the hat. Writing $\psi_i = u_{i,1} + u_{i,2}\mathsf{T}$,

$$\Phi_i\psi_i = \Phi_i\,(u_{i,1} + u_{i,2}\mathsf{T}) = u_{i,1}\,\Phi_i + u_{i,2}\,(\Phi_i\mathsf{T}),$$

so the space has $2n$ real basis functions, Φ_i and $\Phi_i\mathsf{T}$, which are the vector hat functions (9.2) [$\Psi_{i,k}|_f = \phi_i\, T_f R_i^f \varepsilon_k$] of Part I written in spinor coordinates. A field of the space is, in the chart of the face f, the trial field of Part II, shown again:

$$U\big|_f = t_f\,\psi_f, \qquad \psi_f = \sum_{i\in f} \phi_i\, e^{\mathsf{T}_f\rho_i^f}\,\psi_i. \tag{22.8}$$

Their derivatives. In the chart of a face the gauge is flat, $\omega_k = 0$ (Section 29), so by (28.8) [$D_k\psi = \partial_k\psi + \omega_k\mathsf{T}\psi$] the covariant derivative is the plain derivative. The numbers $e^{\mathsf{T}\rho_i^f}$ and ψ_i are constant on the face, so only the hats vary:

$$D_a\psi_f = a\cdot\hat\nabla\Big(\sum_{i\in f}\phi_i\, e^{\mathsf{T}_f\rho_i^f}\,\psi_i\Big) = \sum_{i\in f}(a\cdot\nabla\phi_i)\, e^{\mathsf{T}_f\rho_i^f}\,\psi_i. \tag{22.9}$$

All the turning of the frames, and all the curvature, is in the transitions $e^{\mathsf{T}\rho_i^f}$, which are applied before differentiating.

Conformity. Stage 2 asked for $V_h \subset H^1$, and for piecewise smooth fields this means continuity across the edges. Take the edge between two faces f and g around a vertex i, with g after f counterclockwise, and let j be the other end of the edge. On the edge the hat of the third vertex of each face vanishes, so the field is

$$\text{in the chart of } f: \quad \phi_i\, e^{\mathsf{T}\rho_i^f}\psi_i + \phi_j\, e^{\mathsf{T}\rho_j^f}\psi_j,$$
$$\text{in the chart of } g: \quad \phi_i\, e^{\mathsf{T}\rho_i^g}\psi_i + \phi_j\, e^{\mathsf{T}\rho_j^g}\psi_j.$$

Continuity means that the first, carried into the chart of g by the exact transition across the edge, the unfolding $e^{\mathsf{T}\tau}$, equals the second, for all ψ_i and ψ_j; comparing the coefficients of $\phi_i\psi_i$ and of $\phi_j\psi_j$,

$$e^{\mathsf{T}\tau}\, e^{\mathsf{T}\rho_i^f} = e^{\mathsf{T}\rho_i^g} \quad \text{and} \quad e^{\mathsf{T}\tau}\, e^{\mathsf{T}\rho_j^f} = e^{\mathsf{T}\rho_j^g}.$$

We computed above how the route through the vertex chart differs from the unfolding: by the extra rotation $\delta_i = \delta_{fg} = (1 - s_i)\,\frac{1}{2}(\alpha_f + \alpha_g)$, that is, $e^{\mathsf{T}\rho_i^g} = e^{\mathsf{T}\delta_i}\, e^{\mathsf{T}\tau}\, e^{\mathsf{T}\rho_i^f}$. So at the end i of the edge, where $\phi_i = 1$, the jump of the field is

$$e^{\mathsf{T}\rho_i^g}\psi_i - e^{\mathsf{T}\tau}e^{\mathsf{T}\rho_i^f}\psi_i = \left(e^{\mathsf{T}\delta_i} - 1\right) e^{\mathsf{T}\tau}e^{\mathsf{T}\rho_i^f}\psi_i,$$

whose length is $|e^{\mathsf{T}\delta_i} - 1|\,|\psi_i|$, since unit complex numbers keep lengths. The first factor is

$$\begin{aligned} |e^{\mathsf{T}\delta} - 1|^2 &= (\cos\delta - 1)^2 + \sin^2\delta && (e^{\mathsf{T}\delta} = \cos\delta + \mathsf{T}\sin\delta) \\ &= \cos^2\delta - 2\cos\delta + 1 + \sin^2\delta && (\text{expand}) \\ &= 2 - 2\cos\delta && (\cos^2 + \sin^2 = 1) \\ &= 4\sin^2\tfrac{\delta}{2} && (1 - \cos\delta = 2\sin^2\tfrac{\delta}{2}), \end{aligned}$$

so the jump has the length

$$\left|e^{\mathsf{T}\delta_i} - 1\right|\,|\psi_i| = 2\left|\sin\tfrac{\delta_i}{2}\right|\,|\psi_i|, \qquad \delta_i = (1 - s_i)\,\tfrac{1}{2}(\alpha_f + \alpha_g), \tag{31.3}$$

and the same at the end j, with the corners of f and g at j. Two cases.

- *Every interior vertex flat*, $s_i = 1$: then $\delta_i = 0$ and no jump at all. On an intrinsically flat mesh, however it is folded, the space is conforming, $V_h \subset H^1$, and Stage 2 applies as it stands: by Céa's lemma the Galerkin solution is the best approximation in the energy norm. On a flat or folded sheet more is true: in the frames of the unfolded sheet every transition is the identity, so the complex system of Stage 6 has real coefficients and is the scalar system times the source value; every computed vector is a real multiple of the parallel field, which is why the method reproduces it to rounding error (Section 33).
- *A curved vertex*, $s_i \neq 1$: a small jump. For a vertex with n equal corners, $\alpha_f = \alpha_g = \Theta_i/n$, so $\frac{1}{2}(\alpha_f + \alpha_g) = \Theta_i/n$ and

  $$|\delta_i| = |1 - s_i|\,\frac{\Theta_i}{n} = \frac{|\Theta_i - s_i\Theta_i|}{n} = \frac{|\Theta_i - 2\pi|}{n} = \frac{|\kappa_i|}{n}.$$

  For the five equilateral triangles this is $60°/5 = 12°$ (the $-12°$ per step found above), and the jump is $2\sin 6° = 0.209$ times $|\psi_i|$. At the most

curved vertices of the coarsest sphere of Section 33 (five corners each, largest defect 17.72°; the corners are nearly but not exactly equal, so this is an estimate) it is about $17.72°/5 \approx 3.5°$, and $0.08°/5 \approx 0.016°$ on the finest. On a mesh of a smooth surface κ_i shrinks like the area of the neighborhood of the vertex, as h^2, and so does the jump.

So on curved meshes the space is not exactly conforming. Such small departures from the theory, called *variational crimes*, are the normal situation for finite elements on surfaces [3, 4]: the theorems of Stage 2 then hold up to a consistency error that vanishes with the mismatch. The extrinsic space of Part II is not conforming either: its two transports of a vertex vector into neighboring faces do not differ by the unfolding (Section 27, the Miura-ori), and there the mismatch depends on the tilt of the faces rather than on the defect.

What we got. Stage 5 is complete: on every face a constant metric and a flat gauge, at every vertex a flat chart with one complex unknown, and at every corner a transition rotation. Stages 6 to 8 now run mechanically.

32. Stages 6 to 8: the Discrete Solution, the Element Integrals and the Assembly

Why nothing else changes. The weak form (28.24) takes from the geometry the metric, the rotation gauge and the transitions, and Stage 5 has supplied all three. The element integrals of a face will use its area A_f, the inner products $\nabla\phi_i \cdot \nabla\phi_j$ and the transitions $e^{\mathsf{T}\rho_i^f}$; the lumped mass uses A_f; the assembly only adds element entries into the global matrix by vertex numbers; the scaling by the mass, the Dirichlet row and the sparse factorization work with the assembled numbers. No step after the element integrals looks at positions, normals or rotors. So if we hand the element integrals the same three kinds of numbers, computed in another way, every later step works as in Part II. We now run the three stages once more with the ingredients of this part.

32.1. Stage 6: the discrete solution and the test functions

What we want. Finitely many unknowns and as many equations. Galerkin's choice: look for the solution in the space V_h of Section 31.5, and ask the weak form only for test fields of the same space.

The discrete solution. With one complex coefficient per vertex,

$$\psi_h = \sum_j \Phi_j \, \psi_j, \tag{32.1}$$

which on each face is the trial field (22.8). The coefficient ψ_j is the value of the field at the vertex j, in the chart of the vertex: at x_j only the hat ϕ_j is nonzero, and it is 1 there.

The test functions. For every vertex i other than the source s, the two real test fields $\chi = \Phi_i$ and $\chi = \Phi_i \mathsf{T}$, that is, $\chi = \Phi_i \chi_i$ with $\chi_i = 1$ or $\chi_i = \mathsf{T}$. They vanish at the source, as the weak form requires.

Substitution. The weak form (28.24) is $a(\psi, \chi) = l(\chi)$, with the forms (28.17), shown again,

$$a(\psi,\chi) = \int_\Omega \langle \psi \widetilde{\chi} \rangle \, dA + \Delta t \int_\Omega \sum_{k,l} G^{kl} \big\langle (D_k \psi)\, \widetilde{(D_l \chi)} \big\rangle \, dA,$$

$$l(\chi) = \int_\Omega \langle \psi^0 \widetilde{\chi} \rangle \, dA.$$

Substitute $\psi = \psi_h$ of (32.1) and $\chi = \Phi_i \chi_i$, each in parentheses:

$$a\Big(\sum_j \Phi_j \psi_j,\ \Phi_i \chi_i\Big) = \int_\Omega \Big\langle \Big(\sum_j \Phi_j \psi_j\Big) \widetilde{(\Phi_i \chi_i)} \Big\rangle \, dA + \Delta t \int_\Omega \sum_{k,l} G^{kl} \Big\langle \Big(D_k \sum_j \Phi_j \psi_j\Big)\, \widetilde{(D_l \Phi_i \chi_i)} \Big\rangle \, dA.$$

We simplify in three steps.

Step 1: the sum over j comes out. D_k is linear, so $D_k \sum_j \Phi_j \psi_j = \sum_j D_k(\Phi_j \psi_j)$; the product with $\widetilde{(\cdot)}$, the scalar part $\langle \cdot \rangle$ and the integral are all linear. Moving the sum out through each of them,

$$\begin{aligned} a\Big(\sum_j \Phi_j \psi_j,\ \Phi_i \chi_i\Big) &= \sum_j \Big[\int_\Omega \big\langle (\Phi_j \psi_j) \widetilde{(\Phi_i \chi_i)} \big\rangle \, dA \\ &\qquad + \Delta t \int_\Omega \sum_{k,l} G^{kl} \big\langle D_k(\Phi_j \psi_j)\, \widetilde{(D_l \Phi_i \chi_i)} \big\rangle \, dA \Big] \\ &= \sum_j a(\Phi_j \psi_j, \Phi_i \chi_i). \end{aligned}$$

Step 2: the constants come out. The numbers ψ_j and χ_i are constant, so $D_k(\Phi_j \psi_j) = (D_k \Phi_j)\, \psi_j$. All the factors are complex numbers (even elements of the plane), which commute; and the reverse of a product is the product of the reverses in the opposite order, $\widetilde{AB} = \widetilde{B}\widetilde{A}$, which for commuting factors is $\widetilde{A}\widetilde{B}$. Hence, one rule per line,

$$\begin{aligned} (\Phi_j \psi_j)\, \widetilde{(\Phi_i \chi_i)} &= \Phi_j \psi_j\, \widetilde{\chi}_i\, \widetilde{\Phi}_i && (\widetilde{AB} = \widetilde{B}\widetilde{A}) \\ &= \Phi_j \widetilde{\Phi}_i\, \psi_j \widetilde{\chi}_i && \text{(commute)}, \\ \big(D_k(\Phi_j \psi_j)\big)\, \big(\widetilde{D_l(\Phi_i \chi_i)}\big) &= (D_k \Phi_j)\, \psi_j\, \widetilde{\chi}_i\, \widetilde{(D_l \Phi_i)} && \text{(constants out)} \\ &= (D_k \Phi_j) \widetilde{(D_l \Phi_i)}\, \psi_j \widetilde{\chi}_i && \text{(commute)}. \end{aligned}$$

The factor $\psi_j\widetilde{\chi_i}$ is constant and comes out of the integrals; the scalar part is taken last:

$$a(\Phi_j\psi_j,\Phi_i\chi_i)=\Big\langle\Big(\int_\Omega \Phi_j\widetilde{\Phi_i}\,dA+\Delta t\int_\Omega\sum_{k,l}G^{kl}(D_k\Phi_j)\widetilde{(D_l\Phi_i)}\,dA\Big)\psi_j\widetilde{\chi_i}\Big\rangle$$
$$=\big\langle(\mathrm{M}_{ij}+\Delta t\,\mathrm{L}_{ij})\,\psi_j\,\widetilde{\chi_i}\big\rangle$$

with the complex numbers

$$\mathrm{M}_{ij}=\int_\Omega \Phi_j\,\widetilde{\Phi_i}\,dA,\qquad \mathrm{L}_{ij}=\int_\Omega\sum_{k,l}G^{kl}\,(D_k\Phi_j)\,\widetilde{(D_l\Phi_i)}\,dA,\tag{32.2}$$

integrals of complex valued functions, taken component by component. On the right, with the source field interpolated, $\psi^0=\sum_j\Phi_j\psi_j^0$, the same two steps give $l(\Phi_i\chi_i)=\big\langle(\sum_j\mathrm{M}_{ij}\psi_j^0)\,\widetilde{\chi_i}\big\rangle$.

Step 3: two real tests make one complex equation. Move everything to the left and call $w=\sum_j(\mathrm{M}_{ij}+\Delta t\,\mathrm{L}_{ij})\psi_j-\sum_j\mathrm{M}_{ij}\psi_j^0=w_1+w_2\mathsf{T}$. The equation is $\langle w\,\widetilde{\chi_i}\rangle=0$. For the two tests:

$$\chi_i=1:\quad\langle w\,\widetilde{1}\rangle=\langle w_1+w_2\mathsf{T}\rangle=w_1,$$
$$\chi_i=\mathsf{T}:\quad\langle w\,\widetilde{\mathsf{T}}\rangle=\langle w\,(-\mathsf{T})\rangle=-\langle w_1\mathsf{T}+w_2\mathsf{T}^2\rangle=-\langle w_1\mathsf{T}-w_2\rangle=w_2,$$

using $\widetilde{\mathsf{T}}=-\mathsf{T}$, $\mathsf{T}^2=-1$, and that T has no scalar part. So the two tests ask $w_1=0$ and $w_2=0$, that is, $w=0$:

$$\sum_j(\mathrm{M}_{ij}+\Delta t\,\mathrm{L}_{ij})\,\psi_j=\sum_j\mathrm{M}_{ij}\,\psi_j^0\qquad\text{for every vertex } i\neq s.$$

This is the complex system of Part II, (25.1) $[\,(\mathrm{M}+\Delta t\,\mathrm{L})\psi=\mathrm{M}\psi^0\,]$, one complex equation per vertex.

Hermitian and positive definite. Reversing the integrand of (32.2) exchanges i and j: $\widetilde{(D_k\Phi_j)}\widetilde{(D_l\Phi_i)}=(D_l\Phi_i)\widetilde{(D_k\Phi_j)}$, and since $G^{kl}=G^{lk}$ the sum over k,l is the integrand of L_{ji}. So $\mathrm{L}_{ji}=\widetilde{\mathrm{L}_{ij}}$, the complex conjugate: the matrix is Hermitian. And for $\psi_h=\sum_j\Phi_j\psi_j$, by Steps 1 and 2 with $\chi=\psi_h$,

$$\sum_{i,j}\big\langle\psi_j\,(\mathrm{M}_{ij}+\Delta t\,\mathrm{L}_{ij})\,\widetilde{\psi_i}\big\rangle=a(\psi_h,\psi_h)>0\qquad\text{for }\psi_h\neq0,$$

by the coercivity of Stage 2: the system is positive definite, and by Stage 3 its solution is the minimizer of J over V_h. The equation of the source is not tested; Stage 8 replaces it by the prescribed value.

Gauge covariance of the discrete system. Changing the reference edge of a vertex i multiplies ψ_i by $e^{-\mathsf{T}\gamma_i}$ and Φ_i by $e^{\mathsf{T}\gamma_i}$ (Section 31, with $\gamma_i=s_i\alpha_0$), so $\Phi_i\psi_i$, the field, is unchanged, and the system is the same system written for the new unknowns. Changing the frame of a face does not change $\Phi_j\widetilde{\Phi_i}$ on it, as the next stage shows.

32.2. Stage 7: the element integrals

What we want. The numbers (32.2), face by face. The basis function Φ_i vanishes outside the faces of i, so the integrals are sums over the faces that contain both i and j, $\mathrm{M}_{ij} = \sum_f \mathrm{M}_{ij}^f$ and $\mathrm{L}_{ij} = \sum_f \mathrm{L}_{ij}^f$, and we compute the integral over one face f, in its chart. There, everything is known from the metric and the transitions: the chart is the reference triangle $\hat{K}$, the metric is G_f with $\sqrt{\det G_f} = 2A_f$, the gauge is flat, $D_k = \partial_k$, and $\Phi_i = \hat{\phi}_i(\xi)\, e^{\mathsf{T}\rho_i^f}$ by (31.2).

The stiffness. Substitute into the stiffness integral of (32.2) the basis functions (31.2) [$\Phi_i|_f = \hat{\phi}_i\, e^{\mathsf{T}\rho_i^f}$], the flat derivative $D_k = \partial_k$ and the area element (28.3) [$dA = \sqrt{\det G}\, d\xi_1 d\xi_2$]:

$$
\begin{aligned}
\mathrm{L}_{ij}^f &= \int_f \sum_{k,l} G^{kl}\, (D_k\Phi_j)\, \widetilde{(D_l\Phi_i)}\, dA \\
&= \int_{\hat{K}} \sum_{k,l} G_f^{kl} \left(\partial_k\big(\hat{\phi}_j\, e^{\mathsf{T}\rho_j^f}\big)\right) \widetilde{\left(\partial_l\big(\hat{\phi}_i\, e^{\mathsf{T}\rho_i^f}\big)\right)} \sqrt{\det G_f}\, d\xi && \text{(substitute)} \\
&= \int_{\hat{K}} \sum_{k,l} G_f^{kl} \left(\partial_k\hat{\phi}_j\, e^{\mathsf{T}\rho_j^f}\right) \widetilde{\left(\partial_l\hat{\phi}_i\, e^{\mathsf{T}\rho_i^f}\right)} \sqrt{\det G_f}\, d\xi && (e^{\mathsf{T}\rho} \text{ constant}) \\
&= \int_{\hat{K}} \sum_{k,l} G_f^{kl} \left(\partial_k\hat{\phi}_j\, e^{\mathsf{T}\rho_j^f}\right)\left(\partial_l\hat{\phi}_i\, e^{-\mathsf{T}\rho_i^f}\right) \sqrt{\det G_f}\, d\xi && \text{(reverse)} \\
&= \int_{\hat{K}} \Big(\sum_{k,l} G_f^{kl}\, \partial_l\hat{\phi}_i\, \partial_k\hat{\phi}_j\Big)\, e^{\mathsf{T}\rho_j^f} e^{-\mathsf{T}\rho_i^f} \sqrt{\det G_f}\, d\xi && \text{(regroup)} \\
&= \Big(\sum_{k,l} G_f^{kl}\, \partial_l\hat{\phi}_i\, \partial_k\hat{\phi}_j\Big)\, e^{\mathsf{T}(\rho_j^f - \rho_i^f)} \sqrt{\det G_f} \int_{\hat{K}} d\xi && \text{(constants out)} \\
&= \big(\hat{\nabla}\hat{\phi}_i^T\, G_f^{-1}\, \hat{\nabla}\hat{\phi}_j\big)\, e^{\mathsf{T}(\rho_j^f - \rho_i^f)} \cdot (2A_f) \cdot \tfrac{1}{2} && \text{(see below)} \\
&= A_f\, (\nabla\phi_i \cdot \nabla\phi_j)\, e^{\mathsf{T}(\rho_j^f - \rho_i^f)} && \text{(Prop. 29.1).}
\end{aligned}
$$

The reverse uses $\widetilde{e^{\mathsf{T}\rho}} = e^{-\mathsf{T}\rho}$, and a real number is its own reverse. The line marked "see below" does three things: the double sum is the matrix product $\hat{\nabla}\hat{\phi}_i^T G_f^{-1} \hat{\nabla}\hat{\phi}_j$; $\sqrt{\det G_f} = 2A_f$; and $\int_{\hat{K}} d\xi = \frac{1}{2}$. The derivatives of the reference hats are constants, the reference gradients (12.2), so the integrand is constant and the integral is the integrand times the flat area of $\hat{K}$. Every factor of the result comes from the metric and the transitions: A_f from Heron's formula (29.3) [$A_f = \sqrt{\sigma(\sigma-\ell_{12})(\sigma-\ell_{13})(\sigma-\ell_{23})}$]; the inner products of the gradients either from G_f^{-1} or, as the code does, as plain inner products of the gradients (29.6) [$\nabla\phi_a = \frac{J(\bar{x}_c - \bar{x}_b)}{2A_f}$] on the layout (29.5), the chart in which $G = I$; and the transitions from (31.1) [$\rho_i^f = (\beta_e + \frac{\alpha}{2}) - s_i(\theta_e + \frac{\alpha}{2})$].

The mass. The same substitution in the mass integral of (32.2):

$$\begin{aligned}
\mathrm{M}_{ij}^f &= \int_f \left(\phi_j\, e^{\mathsf{T}\rho_j^f}\right)\left(\widetilde{\phi_i\, e^{\mathsf{T}\rho_i^f}}\right) dA && \text{(substitute (31.2))}\\
&= \int_f \phi_j\,\phi_i\, e^{\mathsf{T}\rho_j^f} e^{-\mathsf{T}\rho_i^f}\, dA && \text{(reverse; commute)}\\
&= e^{\mathsf{T}(\rho_j^f-\rho_i^f)} \int_f \phi_i\phi_j\, dA && \text{(constant out)}\\
&= \frac{A_f}{12}(1+\delta_{ij})\, e^{\mathsf{T}(\rho_j^f-\rho_i^f)} && \text{(the scalar integral of Part I),}
\end{aligned}$$

the consistent mass (13.2) [$\mathrm{M}_{ab}^f = \frac{A_f}{12}(1+\delta_{ab})(R_a^f)^T R_b^f$] of Part I with the rotation matrices replaced by unit complex numbers. As there, the code lumps it, (13.3) [$\mathrm{M}_{aa}^f = \frac{A_f}{3} I_2$]: the row sum of the scalar factors, $\frac{A_f}{12}\cdot 2 + \frac{A_f}{12} + \frac{A_f}{12} = \frac{A_f}{6} + \frac{A_f}{6} = \frac{A_f}{3}$, goes to the diagonal, where the transition of a vertex to itself is $e^{\mathsf{T}(\rho_i^f-\rho_i^f)} = e^{\mathsf{T}0} = 1$.

The element matrices. Together, computed without leaving the surface:

$$\mathrm{L}_{ij}^f = A_f\,(\nabla\phi_i\cdot\nabla\phi_j)\, e^{\mathsf{T}(\rho_j^f-\rho_i^f)}, \qquad \mathrm{M}_{ii}^f = \frac{A_f}{3}, \tag{32.3}$$

with A_f from Heron's formula, the gradients from the layout, and the connection from the bisectors. It is the element matrix (24.3) of Part II, entry for entry; only the way its three ingredients are computed has changed.

The frame of the face cancels. Turn the frame of the face by some angle γ. We saw in Section 31 that every ρ^f of the face becomes $\rho^f - \gamma$. Substituting into the transport of (32.3),

$$e^{\mathsf{T}\left((\rho_j^f-\gamma)-(\rho_i^f-\gamma)\right)} = e^{\mathsf{T}(\rho_j^f-\gamma-\rho_i^f+\gamma)} = e^{\mathsf{T}(\rho_j^f-\rho_i^f)}:$$

the element matrix does not depend on the frame we chose in the face, the rotation gauge invariance of Section 28.4 on one face.

32.3. Stage 8: assembly and the linear system

What we want. Add the element matrices into the global ones, impose the source condition, solve, and read the result back as vectors.

The assembly. The global matrix is the sum (14.1) [$\mathrm{L} = \sum_f P_f^T \mathrm{L}^f P_f$], where P_f is the $3\times n$ matrix that picks the three vertices of f from the global vector: its row a has a 1 in the column of the vertex v_a of f and zeros elsewhere. Entry by entry, $(P_f^T \mathrm{L}^f P_f)_{ij} = \sum_{a,b} (P_f)_{ai}\, \mathrm{L}_{ab}^f\, (P_f)_{bj}$, and $(P_f)_{ai}(P_f)_{bj}$ is 1 exactly when $v_a = i$ and $v_b = j$. So

$$\mathrm{L}_{ij} = \sum_{f\ni i,j} \mathrm{L}_{ab}^f\Big|_{v_a=i,\ v_b=j} = \sum_{f\ni i,j} A_f\,(\nabla\phi_i\cdot\nabla\phi_j)\, e^{\mathsf{T}(\rho_j^f-\rho_i^f)}, \qquad \mathrm{M}_{ii} = \sum_{f\ni i} \frac{A_f}{3}:$$

each face adds its entry to the position of its pair of vertices. The result is an $n \times n$ Hermitian matrix with one complex entry per pair of neighboring vertices. Then, as in Part II: the scaling by the lumped mass, (15.1) [$(I + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} = \mathrm{u}^0$]; the Dirichlet row of the source, which

replaces the untested equation of s by the prescribed value of ψ_s; the sparse LU factorization and the solution.

The code. The function `intrinsicform::solve` is the assembly of `gaform::solve` of Section 25.2, line by line, with the three numbers of each corner taken from the intrinsic geometry:

```
198 /// Section 23: the assembly of gaform::solve, with the gradients, areas and connection
        of IntrinsicConnection.h.
199 static Eigen::VectorXcd solve(Mesh& mesh, const intrinsic::Geometry& G, double dt, int
        source, Complex value) {
200     const int n = mesh.numVertices();
201     std::vector<double> mass(n, 0.0);
202     std::map<std::pair<int, int>, Complex> A;
203     for (Face& f : mesh.getFaces()) {
204         Vertex* v[3];
205         Edge* h[3];
206         intrinsic::faceCorners(f, v, h);
207         const intrinsic::FaceGeometry& g = G.faces[f.ID];
208         for (int a = 0; a < 3; ++a) {
209             mass[v[a]->ID] += g.area / 3.0;
210             for (int b = 0; b < 3; ++b) {
211                 const Complex transport = a == b ? Complex(1.0) : g.rho[b] * std::conj(g
                        .rho[a]);
212                 A[{ v[a]->ID, v[b]->ID }] += dt * g.area * g.grad[a].dot(g.grad[b]) *
                        transport;
213             }
214         }
215     }
216     std::vector<Eigen::Triplet<Complex>> triplets;
217     for (const auto& [ij, value_ij] : A) {
218         if (ij.first == source) continue;
219         triplets.emplace_back(ij.first, ij.second, value_ij / mass[ij.first] + (ij.
                first == ij.second ? 1.0 : 0.0));
220     }
221     triplets.emplace_back(source, source, 1.0);
222     Eigen::SparseMatrix<Complex> S(n, n);
223     S.setFromTriplets(triplets.begin(), triplets.end());
224     Eigen::VectorXcd rhs = Eigen::VectorXcd::Zero(n);
225     rhs(source) = value;
226     Eigen::SparseLU<Eigen::SparseMatrix<Complex>> lu(S);
227     return lu.solve(rhs);
228 }
```

- Lines 205 to 207 are the only difference from `gaform::solve`: instead of computing the gradients from the positions and the connection from the normals (lines 160 to 165 of `ga_fem.cpp`), they fetch the precomputed `FaceGeometry` of the face, with `g.area` $= A_f$, `g.grad[a]` $= \nabla\phi_a$ on the layout, and `g.rho[a]` $= e^{\mathsf{T}\rho_a^f}$.
- Line 209 is the lumped mass $\mathrm{M}_{ii}^f = A_f/3$ of (32.3), added to the vertex `v[a]`: assembled, the mass of a vertex is one third of the area of its faces.
- Lines 210 to 213 are the element matrix (32.3) and its assembly. The corner `a` is i and the corner `b` is j, so the entry of the map `A` at the key (`v[a]->ID`, `v[b]->ID`) is L_{ij}, with the order of the indices of the text; the `+=` is the sum over faces of the assembly. Line 211 is the transport $e^{\mathsf{T}(\rho_j^f - \rho_i^f)}$, computed as $e^{\mathsf{T}\rho_j^f}\,\overline{e^{\mathsf{T}\rho_i^f}}$, since the conjugate of a unit complex number is its inverse, $\overline{e^{\mathsf{T}\rho}} = e^{-\mathsf{T}\rho}$; on the diagonal it is 1. Line 212 multiplies by $A_f\,\nabla\phi_i \cdot \nabla\phi_j$, and also by the time step Δt, so that `A` holds $\Delta t\,\mathrm{L}$.

- Lines 216 to 221 build the scaled system (15.1): each row of $\Delta t\,\mathrm{L}$ is divided by the mass of its vertex and 1 is added on the diagonal, except for the row of the source, which line 218 skips and line 221 replaces by a row of the identity: the Dirichlet condition.
- Lines 222 to 227 build the sparse matrix, put the prescribed value in the right-hand side at the source and zero elsewhere, and solve with the sparse LU factorization.

The same gauge as Part I.. The reference direction $\varphi = 0$ of a vertex is its reference edge, the same edge whose projection is the frame vector t_i of Part I. So both methods read a spinor coordinate as an angle from the same edge, and the value prescribed at the source, $e^{\mathsf{T}\,2\pi/3}$, means in both "120° from the reference edge" in each method's own vertex plane. The two solutions can be compared number by number, as Section 33 does.

Drawing the result. The positions of the vertices return only at the very end, to draw. To draw the coordinate ψ_i, take the cone angle $\theta = \arg(\psi_i)/s_i$, which undoes the stretching (30.4) [$\varphi = s_i\theta$]; find the corner that contains it; and draw the direction in the plane of that face, turned from the first edge of the corner by $u = \theta - \theta_e$, the angle of the direction inside the corner. This is the function `lift`:

```
145 inline Eigen::Vector3d lift(Mesh& mesh, const Geometry& G, int i, Complex z) {
146     Vertex& vi = mesh.vertexAt(i);
147     const VertexGeometry& gv = G.vertices[i];
148     double c = std::arg(z) / gv.scale; // the cone angle, in (-Theta, Theta) for an
            interior vertex
149     if (!gv.boundary && c > 0) c -= gv.totalAngle; // the corners cover [-Theta, 0]
150     const Edge* best = nullptr;
151     double bestDistance = 1e300;
152     for (Vertex::EdgeAroundIterator it = vi.iterator(); !it.end(); it++) {
153         const Edge* e = it.edge_out();
154         if (!e->face) continue;
155         Vertex* v[3];
156         Edge* h[3];
157         faceCorners(*e->face, v, h);
158         int a = 0;
159         while (h[a] != e) ++a;
160         const double from = G.outAngle.at(e), to = from + G.faces[e->face->ID].angle[a
                ];
161         const double distance = c < from ? from - c : c > to ? c - to : 0.0;
162         if (distance < bestDistance) { bestDistance = distance; best = e; }
163     }
164     const Eigen::Vector3d p0 = best->vertex->p, p1 = best->pair->vertex->p, p2 = best
            ->next->next->vertex->p;
165     const Eigen::Vector3d n = (p1 - p0).cross(p2 - p0).normalized();
166     const Eigen::Vector3d edge = (p1 - p0).normalized();
167     const double turn = c - G.outAngle.at(best);
168     return std::cos(turn) * edge + std::sin(turn) * n.cross(edge);
169 }
```

Line 148 is $\theta = \arg(z)/s_i$. Since `std::arg` returns an angle in $(-\pi, \pi]$, θ lies in $(-\Theta_i/2, \Theta_i/2]$, and line 149 subtracts Θ_i from the positive values to bring θ into $[-\Theta_i, 0]$, where the code's corners are; it is the shift of Section 30 once more, the same direction. Lines 152 to 163 go around the vertex with the clockwise iterator and, for each corner, compute the distance from θ to the interval $[\theta_e, \theta_e + \alpha]$ of the corner, zero if θ is inside; the corner with the smallest distance is kept. At an interior vertex one corner contains θ; at a boundary

vertex the direction may point out of the mesh, and the nearest corner is used. Lines 164 to 166 take the face of that corner in space: its normal n and the unit vector along its first edge. Line 167 is the angle $u = \theta - \theta_e$, and line 168 turns the edge by u counterclockwise about n in the plane of the face, $\cos u$ times the edge plus $\sin u$ times its quarter turn $n\times$ edge.

33. Experiments

`ga_fem` runs the vector heat method with the extrinsic connection of Part II and the intrinsic one of this part, on three folded sheets, five spheres and the cactus, with the same time step $\Delta t = h^2$ (h the mean edge length) and the same solver. (In the program's arrays the extrinsic method is $k = 0$ and the intrinsic one $k = 1$; it also runs two further connections, $k = 2, 3$, which this text does not use.) The folded sheets ask whether a method is exact where it should be; the spheres ask how the error decreases as the mesh is refined; the cactus, which has no exact answer, asks how much the choice of the vertex plane matters on a coarse, strongly curved mesh.

33.1. Where the experiments compare: inside the faces

To measure an error we must compare the computed vector with the exact one in the same plane. A vertex has no tangent plane (Section 30): each method has invented one, and comparing there would favor one of them. A face is a real plane, and both the exact field and each method put a definite vector there. For the corner (i, f), each method carries the vector of its solution at i into the face f with its own connection, $t_f\, e^{\mathsf{T}(\arg u_i + \rho_i^f)}$, by (22.5) [$Q\, t_i \psi\, \widetilde{Q} = t_f\, e^{\mathsf{T} f \rho_i^f}\, \psi_f$]: the value at i of the trial field (22.8), the vector that the finite element method itself uses in that face. We measure its angle to the exact vector of the face, at every corner of every face. Both methods measure angles from the same face frame t_f (Section 29).

The structure `Methods` of `ga_fem.cpp` gives, for each method, the unit vector it puts in the face `f` at its corner `a` for the spinor coordinate `z`:

```
420 static double angleDeg(const Vector3d& a, const Vector3d& b) { return std::atan2(a.
        cross(b).norm(), a.dot(b)) * 180.0 / M_PI; }
421
422 /// The unit vector of face f at the given angle from its frame vector t_f,
        counterclockwise about n_f.
423 static Vector3d faceVector(const Geometry& g, int f, double angle) {
424     return std::cos(angle) * g.faceT[f] + std::sin(angle) * g.faceNormal[f].cross(g.
            faceT[f]);
425 }
426
427 /// What each method puts in face f for the spinor coordinate z of its corner a: the
        vector of angle arg z + rho_a^f
428 /// (eq. ga_transport), which is what the finite element method uses in that face.
        Comparing these vectors face by face
429 /// needs no choice of a plane at the vertex (Section 24).
430 struct Methods {
431     Mesh& mesh;
432     const Geometry& g;
```

```
433     const intrinsic::Geometry& G;
434     Vector3d extrinsic(int f, int a, Complex z) const {
435         Vertex* v[3];
436         Edge* h[3];
437         intrinsic::faceCorners(mesh.faceAt(f), v, h);
438         const int i = v[a]->ID;
439         const Complex rho = ga::connection(g.vertexT[i], mesh.vertexAt(i).n, g.faceT[f
                ], g.faceNormal[f]);
440         return faceVector(g, f, std::arg(z) + std::arg(rho));
441     }
442     Vector3d intrinsic(int f, int a, Complex z) const { return faceVector(g, f, std::
            arg(z) + std::arg(G.faces[f].rho[a])); }
```

Line 420 is the angle between two vectors of space, in degrees, computed with `atan2` of the sine and the cosine, which is accurate also for very small angles. Lines 423 to 425 are the unit vector of the face at the angle β from t_f, $\cos\beta\, t_f + \sin\beta\, n_f \times t_f$. Line 439 is the extrinsic connection (22.4) of the corner, and line 440 the vector at the angle $\arg z + \rho_i^f$. Line 442 is the same for the intrinsic connection, with `rho[a]` from Section 31. Both use the same face frame `g.faceT[f]`, and only the length of z is discarded: the vector heat method keeps only directions.

The exact vectors. They are computed independently of both methods. On a flat or folded sheet, we carry the source vector from face to face across the edges with the rotation about the shared edge, the unfolding of Section 22.5. On an intrinsically flat mesh the result does not depend on the path, because every angle defect vanishes and so does the holonomy of every loop; the function `unfoldFrom` uses the order of a breadth-first search:

```
492 static std::vector<Vector3d> unfoldFrom(Mesh& mesh, const Geometry& g, int f0, const
        Vector3d& u0) {
493     std::vector<Vector3d> u(mesh.numFaces(), Vector3d::Zero());
494     std::vector<bool> seen(mesh.numFaces(), false);
495     std::vector<int> queue{ f0 };
496     u[f0] = u0;
497     seen[f0] = true;
498     for (size_t k = 0; k < queue.size(); ++k) {
499         const int f = queue[k];
500         Edge* e = mesh.faceAt(f).edge;
501         for (int m = 0; m < 3; ++m, e = e->next) {
502             const Face* other = e->pair->face;
503             if (!other || seen[other->ID]) continue;
504             seen[other->ID] = true;
505             u[other->ID] = ga::rotorBetween(g.faceNormal[f], g.faceNormal[other->ID]).
                    rotate(u[f]);
506             queue.push_back(other->ID);
507         }
508     }
509     return u;
510 }
```

Lines 495 to 497 start with the source face `f0` and its vector. Lines 498 to 507 take the faces in the order they were reached and, for each of its three edges, reach the face on the other side if it has not been reached yet (line 503 also skips the boundary, where `e->pair->face` is null). Line 505 is the unfolding: the smallest rotation from the normal of `f` to the normal of its neighbor turns about their common edge, and it carries the vector into the plane of the neighbor. On the sphere, the exact vector is obtained by transport

along the great circle from the source to the vertex, followed by the smallest rotation from the tangent plane of the sphere into the face.

The source vector. Each method interprets the prescribed value in its own plane of the source vertex, so the exact field for each method starts from that method's own source vector, the vector it puts into the first face of the source vertex (the face of its reference edge, found by `firstCorner`).

33.2. Folded sheets: exact bendings

Three sheets, all intrinsically flat, every interior angle sum 2π to 10^{-13} degrees: a flat sheet of 48×16 squares, each cut by a diagonal (833 vertices); the same sheet folded along every fourth column of vertices by 30° (parallel folds); and the Miura-ori of Figure 22, 24×16 parallelograms (425 vertices), with the vertices $X_{jk} = P_j + Q_k$ given by the zigzags $P_j = (ju, 0, (j \bmod 2)\, w)$ and $Q_k = ((k \bmod 2)\, r, kt, 0)$, here with $u = t = 1/12$ and $w = r = 0.6/12$.

Why the Miura-ori is intrinsically flat. At a vertex (take j and k even; the other cases differ only in signs that cancel in the same way), the four edges are

$$a = P_{j+1} - P_j = (u, 0, w), \quad a' = P_{j-1} - P_j = (-u, 0, w),$$
$$b = Q_{k+1} - Q_k = (r, t, 0), \quad b' = Q_{k-1} - Q_k = (r, -t, 0).$$

So $|a| = |a'| = \sqrt{u^2 + w^2}$ and $|b| = |b'| = \sqrt{r^2 + t^2}$, and the inner products of the four consecutive pairs around the vertex are

$$a \cdot b = ur + 0 + 0 = ur, \quad b \cdot a' = -ru, \quad a' \cdot b' = -ur, \quad b' \cdot a = ru.$$

With $x = ur/(|a||b|)$, the four face angles at the vertex have the cosines $x, -x, -x, x$, that is, the angles $\arccos x$, $\arccos(-x)$, $\arccos(-x)$, $\arccos x$. Since $\arccos(-x) = \pi - \arccos x$, their sum is

$$2 \arccos x + 2(\pi - \arccos x) = 2\pi.$$

(Cutting each parallelogram by a diagonal splits one of its angles in two, without changing the sums.) The exact answer on each sheet is the field of the flat paper.

The function `foldExperiment` runs one sheet. After solving with all the methods (line 610) it builds the exact field of each method from its own source vector, and measures every corner of every face:

```
610     const std::vector<Eigen::VectorXcd> u = solveAll(m, g, G, E, h * h, source, value)
            ;
611     // the source vector, as each method puts it in the first face of the source,
            carried exactly to every face
612     int f0, a0;
613     firstCorner(m, source, f0, a0);
614     std::vector<Vector3d> exact[4];
615     for (int k = 0; k < 4; ++k) exact[k] = unfoldFrom(m, g, f0, M.of(k, f0, a0, value)
            );
616     Errors err[4];
617     std::ofstream out;
618     if (!csv.empty()) { out.open(csv); out << "x1,y1,z1,x2,y2,z2,x3,y3,z3,ux,uy,uz,
            error_extrinsic\n"; }
619     for (Face& f : m.getFaces()) {
620         Vertex* v[3];
621         Edge* hh[3];
622         intrinsic::faceCorners(f, v, hh);
```

```
623         double worst = 0;
624         for (int a = 0; a < 3; ++a)
625             for (int k = 0; k < 4; ++k) {
626                 const double d = angleDeg(M.of(k, f.ID, a, u[k](v[a]->ID)), exact[k][f.
                        ID]);
627                 err[k].add(d);
628                 if (k == 0) worst = std::max(worst, d);
629             }
```

Line 613 finds the first face `f0` of the source and its corner `a0`; line 615 puts the prescribed value into `f0` with each method's connection and unfolds it to every face. Lines 619 to 629 visit every corner `(f, a)`: line 626 is the angle between the vector the method puts in the face, from its solution `u[k]` at the vertex `v[a]`, and the exact vector of the face, and line 627 accumulates its mean and maximum.

How to read the table. Each row is a sheet; each entry is an angle error in degrees over all the corners. An entry of order 10^{-13} or smaller is rounding error: the method is exact.

	extrinsic, mean	extrinsic, max	intrinsic, max
flat sheet	$7 \cdot 10^{-15}$	$5 \cdot 10^{-14}$	$7 \cdot 10^{-13}$
parallel folds	$1 \cdot 10^{-14}$	$4 \cdot 10^{-14}$	$3 \cdot 10^{-13}$
Miura-ori	$7.42°$	$21.9°$	$2 \cdot 10^{-13}$

The intrinsic method is exact on all three, as Section 31 predicts: every vertex has $s_i = 1$, so $\rho_i^f = \beta_e - \theta_e$ and every transport into a face is an unfolding. The extrinsic method is exact on the parallel folds too, because there every face is rolled into the vertex plane about the fold line, and rolling about the fold is unfolding. On the Miura-ori the faces around a vertex are rolled about four different axes, the rolled corners do not fit edge to edge, and the transports are wrong by several degrees at every vertex.

33.3. The sphere: convergence

On the geodesic spheres of Example 15.1, refined from 42 to 10242 vertices, we measure at every corner within three quarters of the way to the antipode (at the antipode the shortest path is not unique, and the exact transport is not defined). The function `sphereAll` takes the highest vertex as the source, prescribes $e^{\mathsf{T}\,2\pi/3}$ there, and compares corner by corner:

```
530     int source = 0;
531     for (Vertex& v : mesh.getVertices()) if (v.p.z() > mesh.vertexAt(source).p.z())
            source = v.ID;
532     const Vector3d s = mesh.vertexAt(source).p.normalized();
533     const Complex value = std::polar(1.0, 2.0 * M_PI / 3.0);
534     const std::vector<Eigen::VectorXcd> u = solveAll(mesh, g, G, E, h * h, source,
            value);
535     // the source vector: what each method puts in the first face of the source, rolled
            into the plane of the sphere
536     int f0, a0;
537     firstCorner(mesh, source, f0, a0);
538     Vector3d src[4];
539     for (int k = 0; k < 4; ++k) src[k] = ga::rotorBetween(g.faceNormal[f0], s).rotate(
            M.of(k, f0, a0, value));
540     for (Face& f : mesh.getFaces()) {
541         Vertex* v[3];
542         Edge* hh[3];
543         intrinsic::faceCorners(f, v, hh);
```

```
544         for (int a = 0; a < 3; ++a) {
545             const int i = v[a]->ID;
546             const Vector3d x = v[a]->p.normalized();
547             if (std::acos(std::max(-1.0, std::min(1.0, s.dot(x)))) > 0.75 * M_PI)
                    continue;
548             const ga::Rotor along = ga::rotorBetween(s, x), into = ga::rotorBetween(x,
                    g.faceNormal[f.ID]);
549             for (int k = 0; k < 4; ++k) err[k].add(angleDeg(M.of(k, f.ID, a, u[k](i)),
                    into.rotate(along.rotate(src[k]))));
550         }
551     }
```

Line 539 is the source vector of each method, put into the first face of the source and rolled from that face into the tangent plane of the smooth sphere at the source, whose normal is the position `s` itself. For every corner, line 547 skips the vertices farther than $\frac{3}{4}\pi$ from the source, line 548 builds the two rotations of the exact transport, `along` the great circle from `s` to the vertex `x`, and `into` the face from the tangent plane of the sphere at `x`, and line 549 measures the angle between the method's vector in the face and the exact one.

How to read the table. Each row is one level of refinement: the number of vertices, the mean edge length h, and the largest angle defect. Then the mean and the largest error of each method, in degrees. Read each column downwards for the convergence; compare the two methods on the same row.

			extrinsic		intrinsic	
vertices	h	largest κ_i	mean	max	mean	max
42	0.5823	17.72°	0.277°	1.340°	0.331°	1.502°
162	0.2993	4.71°	0.209°	0.888°	0.210°	0.881°
642	0.1507	1.30°	0.111°	0.473°	0.111°	0.492°
2562	0.0755	0.34°	0.067°	0.307°	0.067°	0.306°
10242	0.0378	0.08°	0.079°	0.506°	0.079°	0.504°

Neither connection is exact on a curved surface, and the two are practically equal from 162 vertices on (Figure 28, left). Each refinement halves h and divides the largest defect by about four, as the area of a vertex's neighborhood (h^2), so $s_i \to 1$ and the two vertex planes converge to the same plane. Only on the coarsest sphere, with defects up to 17.7°, is the extrinsic one slightly better: the averaged normal of a geodesic sphere is very nearly the exact normal of the smooth sphere, information the edge lengths do not contain. The slight increase of the error on the finest sphere is shared by both, so it belongs to the vector heat method with $\Delta t = h^2$, not to the connection.

33.4. The cactus: how large is the choice?

On the cactus of Figure 16 there is no exact answer, but we can measure how much the two connections disagree, corner by corner, with the same source value, which both read as an angle from the same reference edge (Section 32). At every corner the program measures the angle between the vectors the two methods put in the face, sorts these angles, and reads off the median, the 90% value and the largest:

```
1234         for (Face& f : mesh.getFaces()) {
1235             Vertex* v[3];
```

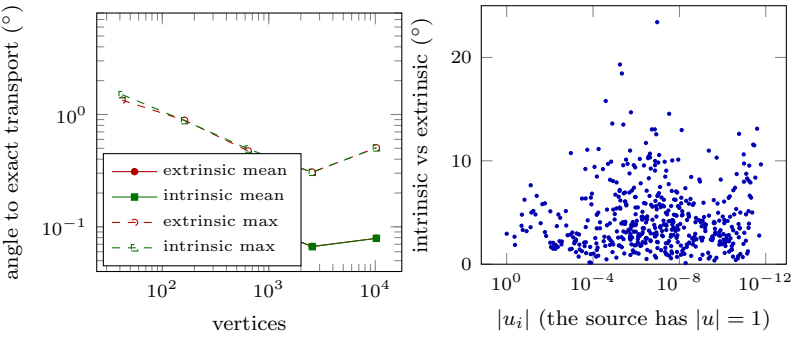


FIGURE 28. Left: the error of the vector heat method on geodesic spheres, against the number of vertices, with the extrinsic connection of Part II (red) and the intrinsic one (green); mean (solid) and maximum (dashed), over all corners within three quarters of the way to the antipode. Right: on the cactus, the largest angle between the two solutions over the corners of each vertex, against the magnitude $|u_i|$ of the intrinsic solution: the two disagree strongly only where the field has almost vanished.

```
1236            Edge* hh[3];
1237            intrinsic::faceCorners(f, v, hh);
1238            for (int a = 0; a < 3; ++a) {
1239                const int i = v[a]->ID, k = pairs[p][0], l = pairs[p][1];
1240                corners.push_back(angleDeg(M.of(k, f.ID, a, u[k](i)), M.of(l, f.ID,
                        a, u[l](i))));
1241                perVertex[p][i] = std::max(perVertex[p][i], corners.back());
1242            }
1243        }
1244        std::sort(corners.begin(), corners.end());
1245        printf("␣␣␣%s␣vs␣%s,␣corner␣by␣corner:␣median␣%.2f,␣90%%␣%.2f,␣max␣%.2f␣
                degrees\n", methodName[pairs[p][0]],
1246              methodName[pairs[p][1]], corners[corners.size() / 2], corners[corners
                    .size() * 9 / 10], corners.back());
```

Line 1240 is the angle between the two vectors of the corner, with $k = 0$ and $l = 1$; line 1241 keeps, for each vertex, the largest such angle over its corners, which Figure 28 plots; and line 1246 prints the entries at the middle and at nine tenths of the sorted list, and the last one.

The numbers. The angle defects add up to 4π, as the discrete Gauss–Bonnet theorem requires for a surface of genus zero; the largest is 63.7°. The angle between the two solutions has median 2.20° (a typical corner), 90% of the corners are below 6.15°, and the largest difference is 23.4°, at a vertex where the field has decayed to $|u_i| = 1.1 \cdot 10^{-7}$. Figure 28, right, shows that the large differences occur only where the field is nearly zero, where a small change of the connection can turn its direction. The few degrees elsewhere measure the size of the modeling choice that a coarse, strongly curved mesh leaves open at its vertices.

34. Part III in the Matrix Language of Part I

What we want. Parts I and III solve the same kind of linear system with different ingredients and in different notation: Part I uses 2×2 rotation matrices and real vectors of $2n$ numbers, Part III uses unit complex numbers and n spinor coordinates. This section rewrites the element matrix of Part III in the notation of Part I. Nothing new is derived. The point is to see that the intrinsic method is Part I's program with three numbers per corner computed differently, and that geometric algebra was a convenience for deriving those numbers, not a requirement for using them. The code does exactly this: `FEM.cpp`, run with the option `--intrinsic`, assembles Part I's real system with the intrinsic ingredients.

34.1. The dictionary

A coordinate is two numbers. Part I stores the coordinates of U_i in the frame T_i as a column $u_i = (u_{i,1}, u_{i,2})^T$; Part III stores the spinor coordinate ψ_i. They are the same two numbers:

$$\psi_i = u_{i,1} + \mathsf{T}\, u_{i,2} \quad \longleftrightarrow \quad u_i = \begin{pmatrix} u_{i,1} \\ u_{i,2} \end{pmatrix}.$$

Multiplying by $e^{\mathsf{T}\rho}$ is multiplying by a rotation matrix. Substitute $e^{\mathsf{T}\rho} = \cos\rho + \mathsf{T}\sin\rho$ and $\psi = u_1 + \mathsf{T}u_2$ into the product, and use $\mathsf{T}^2 = -1$:

$$\begin{aligned} e^{\mathsf{T}\rho}\,\psi &= (\cos\rho + \mathsf{T}\sin\rho)\,(u_1 + \mathsf{T}u_2) && \text{substitute} \\ &= u_1\cos\rho + \mathsf{T}u_2\cos\rho + \mathsf{T}u_1\sin\rho + \mathsf{T}^2 u_2\sin\rho && \text{expand} \\ &= (u_1\cos\rho - u_2\sin\rho) + \mathsf{T}\,(u_1\sin\rho + u_2\cos\rho) && \mathsf{T}^2 = -1;\ \text{group} \end{aligned}$$

The two parts of the result are the two rows of a matrix product:

$$e^{\mathsf{T}\rho}\,\psi \quad \longleftrightarrow \quad R(\rho)\,u = \begin{pmatrix} \cos\rho & -\sin\rho \\ \sin\rho & \cos\rho \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}. \tag{34.1}$$

Conjugating is transposing. The conjugate is $\overline{e^{\mathsf{T}\rho}} = \cos\rho - \mathsf{T}\sin\rho = \cos(-\rho) + \mathsf{T}\sin(-\rho) = e^{-\mathsf{T}\rho}$, so it corresponds to $R(-\rho)$. Substituting $-\rho$ into (34.1), and using $\cos(-\rho) = \cos\rho$ and $\sin(-\rho) = -\sin\rho$:

$$R(-\rho) = \begin{pmatrix} \cos\rho & \sin\rho \\ -\sin\rho & \cos\rho \end{pmatrix} = R(\rho)^T.$$

The transport of the element. The element (32.3) [$\mathrm{L}_{ij}^f = A_f(\nabla\phi_i \cdot \nabla\phi_j)\, e^{\mathsf{T}(\rho_j^f - \rho_i^f)}$] contains the product $e^{\mathsf{T}(\rho_j^f - \rho_i^f)} = \overline{e^{\mathsf{T}\rho_i^f}}\; e^{\mathsf{T}\rho_j^f}$. Replace each factor by its matrix, with the conjugate becoming a transpose:

$$\overline{e^{\mathsf{T}\rho_i^f}}\; e^{\mathsf{T}\rho_j^f} \quad \longleftrightarrow \quad R(\rho_i^f)^T R(\rho_j^f) = R(-\rho_i^f)\, R(\rho_j^f) = R(\rho_j^f - \rho_i^f),$$

the last step because two rotations of the plane compose by adding their angles. Complex numbers commute and matrices in general do not, but rotations of the plane do, so the order of the factors is a matter of convention; we keep the order of Part I, transposed factor first.

34.2. The element

Substituting. Write $R_i^f = R(\rho_i^f)$, the 2×2 matrix of the corner connection of (31.1) [$\rho_i^f = (\beta_e + \frac{\alpha}{2}) - s_i(\theta_e + \frac{\alpha}{2})$]. Replacing the complex transport in (32.3) by its matrix gives the 2×2 block

$$\mathrm{L}_{ij}^f = A_f \, (\nabla\phi_i \cdot \nabla\phi_j) \, (R_i^f)^T R_j^f \in \mathbb{R}^{2\times 2}, \qquad \mathrm{M}_{ii}^f = \frac{A_f}{3} \, I_2. \tag{34.2}$$

Compare it with the element (13.1) [$\mathrm{L}_{ab}^f = A_f (g_a^f \cdot g_b^f)(R_a^f)^T R_b^f$] of Part I: the formula is the same. What differs is only how the three ingredients are computed:

ingredient	Part I (extrinsic)	Part III (intrinsic)
area A_f	$\frac{1}{2}\lvert e_1 \times e_2\rvert$, from the positions	Heron's formula, from the lengths
gradient	$g_a^f = T_f^T \nabla\phi_a$, (12.5), from B in $\mathbb{R}^3$	$\nabla\phi_a = J(\bar{x}_c - \bar{x}_b)/(2A_f)$ on the layout, (29.6)
transport R_a^f	$T_f^T Q_{a\to f} T_a$, (4.7): frames and normals	$R(\rho_a^f)$, (31.1): matched bisectors

The lumped mass is the same in both, one third of the face area on each diagonal block, (13.3) [$\mathrm{M}_{aa}^f = \frac{A_f}{3} I_2$].

The element written out. With $w_{ij} = A_f \, \nabla\phi_i \cdot \nabla\phi_j$, and with $(R_i^f)^T R_i^f = I_2$ on the diagonal, the 6×6 element matrix of one face is

$$\mathrm{L}^f = \begin{pmatrix} w_{11}\, I_2 & w_{12}\,(R_1^f)^T R_2^f & w_{13}\,(R_1^f)^T R_3^f \\ w_{21}\,(R_2^f)^T R_1^f & w_{22}\, I_2 & w_{23}\,(R_2^f)^T R_3^f \\ w_{31}\,(R_3^f)^T R_1^f & w_{32}\,(R_3^f)^T R_2^f & w_{33}\, I_2 \end{pmatrix}.$$

Substituting (34.1) and the transport rule, the block in row 1 and column 2 is, entry by entry,

$$w_{12}\, R(\rho_2^f - \rho_1^f) = w_{12} \begin{pmatrix} \cos(\rho_2^f - \rho_1^f) & -\sin(\rho_2^f - \rho_1^f) \\ \sin(\rho_2^f - \rho_1^f) & \cos(\rho_2^f - \rho_1^f) \end{pmatrix},$$

the real form of the single complex number $w_{12}\, e^{\mathsf{T}(\rho_2^f - \rho_1^f)}$.

The matrix is symmetric. The block in row 2 and column 1 is $w_{21}(R_2^f)^T R_1^f$. Transpose it, with $(XY)^T = Y^T X^T$ and $w_{21} = w_{12}$ (the dot product is symmetric):

$$\big(w_{21}\,(R_2^f)^T R_1^f\big)^T = w_{21}\,(R_1^f)^T \big((R_2^f)^T\big)^T = w_{12}\,(R_1^f)^T R_2^f,$$

which is the block in row 1 and column 2. So L^f, and the assembled $2n \times 2n$ matrix L, are symmetric: this is the real form of the Hermitian property of Part III, $\mathrm{L}_{ji} = \overline{\mathrm{L}_{ij}}$, because conjugation became transposition.

The rest is Part I.. The assembly (14.1) [$\mathrm{L} = \sum_f P_f^T \mathrm{L}^f P_f$], the scaled system (15.1) [$(I + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\,\mathrm{u} = \mathrm{u}^0$], the Dirichlet rows of the source and the sparse LU factorization are those of Part I, unchanged. The only other difference is in reading the result. A coordinate u_i is now an angle on the stretched cone of the vertex, so it is drawn with `lift` (Section 32.3), which divides the angle by s_i before turning the reference edge. Part I rotates the reference edge by the angle itself, about the vertex normal.

34.3. The code

The intrinsic element is the function `AssembleStiffnessElement-Embedded2` of Section 13, with its three ingredients fetched from `IntrinsicConnection.h`:

```
931 /**
932  * Part III in the matrix form of Part I: the element of
         AssembleStiffnessElementEmbedded2 with its three ingredients
933  * taken from the edge lengths alone (IntrinsicConnection.h). The gradients are the 2D
         gradients of the layout
934  * (eq. intrinsic_gradient), the area is Heron's, and the vertex-to-face rotation R_a
         is the 2x2 matrix of the corner
935  * connection e^{T rho_a^f} (eq. intrinsic_rho). The block (a, b) is A_f (grad phi_a .
         grad phi_b) R_a^T R_b, the real
936  * form of the complex entry A_f (grad phi_a . grad phi_b) e^{T (rho_b - rho_a)} (eq.
         intrinsic_element).
937  */
938 Matrix66d AssembleStiffnessElementIntrinsic(const Face& face, const intrinsic::
         Geometry& G) {
939     const intrinsic::FaceGeometry& g = G.faces[face.ID];
940     Eigen::Matrix2d R[3];
941     for (int a = 0; a < 3; ++a) {
942         const double c = g.rho[a].real(), s = g.rho[a].imag();
943         R[a] << c, -s,
944                 s, c;
945     }
946     Matrix66d elementMatrix;
947     elementMatrix.setZero();
948     for (int i = 0; i < 3; ++i) { // for each test function
949         for (int j = 0; j < 3; ++j) { // for each shape function
950             const double w = g.area * g.grad[i].dot(g.grad[j]);
951             if (i == j) elementMatrix.block<2,2>(i * 2, j * 2) = w * Eigen::Matrix2d::
                 Identity();
952             else elementMatrix.block<2,2>(i * 2, j * 2) = w * R[i].transpose() * R[j];
953         }
954     }
955     return elementMatrix;
956 }
```

- Line 939 fetches the geometry of the face computed from the edge lengths: `g.area` is A_f (Heron), `g.grad[a]` is $\nabla\phi_a$ on the layout, and `g.rho[a]` is the unit complex number $e^{\mathsf{T}\rho_a^f}$.
- Lines 941 to 945 turn each $e^{\mathsf{T}\rho_a^f} = \cos\rho_a^f + \mathsf{T}\sin\rho_a^f$ into R_a^f, by (34.1): the real part `c` is $\cos\rho_a^f$, the imaginary part `s` is $\sin\rho_a^f$, and Eigen's comma initializer fills the matrix row by row, $(c, -s)$ and (s, c).
- Line 950 is $w_{ij} = A_f\,\nabla\phi_i\cdot\nabla\phi_j$, a dot product of two plane vectors; Part I needed the projection T_f^T first to get to the plane (lines 377–378), and here the gradients are already in it.

- Lines 951 and 952 are (34.2): $w_{ii} I_2$ on the diagonal, $w_{ij}(R_i^f)^T R_j^f$ off it. Unlike Part I, the loop does not skip $i < j$: it computes every block, which costs a few more products and keeps the code a literal transcription of the matrix written out above.

The assembly is `AssembleMatrix2` of Section 14 with the new element:

```
958 /// AssembleMatrix2 with the intrinsic element: the same loop over faces, the same 2x2
        blocks, the same global matrix.
959 std::shared_ptr<SparseMatrix<double>> AssembleMatrixIntrinsic(Mesh *mesh, double
        delta_t) {
960     g_intrinsicGeometry = intrinsic::compute(*mesh);
961     std::shared_ptr<SparseMatrix<double>> A(new SparseMatrix<double>(mesh->numVertices
            () * 2, mesh->numVertices() * 2));
962     Matrix66d stiffnessMatrix;
963     Eigen::Matrix2d wij;
964     Vertex* v[3];
965     for (Face& face : mesh->getFaces()) {
966         v[0] = face.edge->vertex;
967         v[1] = face.edge->next->vertex;
968         v[2] = face.edge->next->next->vertex;
969         stiffnessMatrix = AssembleStiffnessElementIntrinsic(face, g_intrinsicGeometry);
970         for( int i = 0 ; i < 3 ; ++i ) {
971             for (int j = 0 ; j < 3 ; ++j ) {
972                 wij = delta_t * stiffnessMatrix.block<2,2>(i * 2, j * 2);
973                 for(int row = 0 ; row < 2 ; ++row) {
974                     for(int col = 0 ; col < 2 ; ++col) {
975                         (*A)(v[i]->ID * 2 + row, v[j]->ID * 2 + col) += wij(row, col);
976                     }
977                 }
978             }
979         }
980     }
981     return A;
982 }
```

Line 960 computes the intrinsic geometry of the whole mesh once (Sections 29–31) and stores it, because the drawing needs it too; line 969 is the only other difference from `AssembleMatrix2`, the call of the intrinsic element. Lines 970 to 979 add Δt times each 2×2 block into the global matrix at rows $2i, 2i+1$ and columns $2j, 2j+1$, the index map (14.2) of Part I.

Three lines of `main` and `display` choose between the two versions, without changing the line numbers of the rest of the file: line 554 reads the option `--intrinsic` into `g_intrinsic` (declared, together with the stored geometry, on lines 143–144); line 638 calls `AssembleMatrixIntrinsic` instead of `AssembleMatrix2` when it is set; and line 775 draws each vector with `intrinsic::lift` instead of the rotation about the vertex normal.

A check. Run on the cactus, this real $2n \times 2n$ system and the complex $n \times n$ system of `intrinsicform::solve` (Section 32.3) give the same solution: the largest difference between $u_{i,1} + \mathsf{T} u_{i,2}$ and ψ_i over all vertices is $1.1 \cdot 10^{-16}$ on the cactus of 502 vertices and $1.4 \cdot 10^{-16}$ on the one of 5261, rounding error, for solutions of size up to 1. The two programs are two notations for one method.

TABLE 5. The ingredients of the element matrix in the extrinsic method of Part II and in the intrinsic method of Part III.

Ingredient	Extrinsic (Part II)	Intrinsic (Part III)
input	positions x_i, normals	edge lengths ℓ_{ij}
how it measures	vectors of $\mathbb{R}^3$	metric G_f in a flat chart (displacement gauge)
where curvature shows	turning of the normals	transitions at the vertices (rotation gauge)
face geometry	B, 3×2	metric $G_f = B^T B$ from lengths, (29.2)
area	$\frac{1}{2}\lvert e_1 \times e_2 \rvert$	Heron, (29.3)
gradients	reciprocal frame in space	quarter turn in the layout, (29.6)
face frame	first edge, in space	first edge, the x axis of the layout
vertex plane	orthogonal to the vertex normal n_i	angles rescaled by $s_i = 2\pi/\Theta_i$, (30.4)
where the defect goes	gaps between rolled corners, by the tilt of the faces	into every corner, in proportion to its angle
connection ρ_i^f	$e^{\mathsf{T}\rho} = t_f\, Q t_i \widetilde{Q}$, (22.4)	bisector to bisector, (31.1)
intrinsically flat parts	exact for parallel folds only	exact
undefined when	$n_f = -n_i$	never (non-degenerate triangles)

PART III · THE INTRINSIC METHOD: EVERYTHING FROM EDGE LENGTHS

35. Summary: Two Ways to Give a Vertex a Plane

Table 5 lists what changes between Part II and this part. Every ingredient of a face, the area, the gradients, the stiffness, was intrinsic all along, hidden in the Gram matrix $B^T B$. The one real difference is the tangent plane of a vertex, which a mesh does not have and which each method must invent: Part II invents it from the vertex normal and rolls each face rigidly into it, leaving the angle defect in the gaps; this part invents it from the angles, stretching every corner by the same factor $s_i = 2\pi/\Theta_i$. The connection then follows by matching the bisectors of the corners, the least-squares best rotation for each corner; going around a vertex, the small errors of the corners add up to exactly minus the angle defect, so the curvature is accounted for rather than ignored.

The recipe, in the intrinsic way. The finite element method needs two things from the geometry of a curved surface: a way to measure (lengths, areas, gradients, inner products) and a way to compare vectors at different points (the connection). The intrinsic view gets both from inside the surface: it measures in flat charts with the metric, and it compares vectors with rotations between those charts, the transitions, which carry the curvature as the angle defects. The integrals are always done in flat pieces, so the curvature never enters an integral, only the rotations that connect the pieces. To apply the same machine to another equation on a surface: (i) the strong form in a chart,

with $D_k = \partial_k + \omega_k \mathsf{T}$ (or the plain derivative, for scalar unknowns), and its energy; (ii) the weak form, from Green's identity (28.23) in the flat chart; (iii) on each face, the metric G_f from the three lengths, which gives the area and the gradients; (iv) at each vertex, a flat chart for the unknown, and at each corner the transition into the face, here the bisector rule (31.1); (v) the element integrals on the reference triangle, where the flat gauge makes them integrals of polynomials, as in Stage 7. A scalar equation needs only (i), (ii), (iii) and (v); a vector equation needs (iv) as well.

Applications

Parallel Transport at Work

Applications

36. The Logarithmic Map: a Decal, a Folded Sheet and an Atlas

What we want. We have built a machine that carries one tangent vector, given at a source vertex s, to every point of the surface. A natural thing to do with it is to give every point x *polar coordinates around* s: a distance $r(x)$ and an angle $\theta(x)$. With them we can paste an image on the surface, a *decal*, as if we stuck a sticker at s and smoothed it out along the surface. This section builds those coordinates, called the *logarithmic map*, in the same step-by-step way as the rest of the text, and uses them in two demonstrations: a decal on the cactus, and graph paper on the folded Miura sheet of Figure 22, where the intrinsic connection of Part III and the extrinsic connection of Part II give visibly different pictures. The code is `showcase.cpp`.

The plan has three steps, and only the first is new:

1. the distance r, by a second application of the finite element recipe, now to scalar functions (the *heat method*);
2. the transported source direction Y, which is exactly the vector heat step of Stages 6 to 8;
3. the angle θ, read off at every point by comparing Y with the direction in which r grows.

36.1. The logarithmic map

Fix the source s and a reference unit vector e_s in its tangent plane (for us, the reference direction $\varphi = 0$ of the stretched cone of s, the direction whose spinor coordinate is $\psi = 1$). For a point x, let γ be the shortest curve on the surface from s to x, a *geodesic*, traversed at unit speed, so that its length is the distance $r = r(x)$. It leaves s in some unit direction $\gamma'(0)$, and we call θ the angle from e_s to $\gamma'(0)$, measured counterclockwise in the tangent plane of s. The logarithmic map writes x as the vector of the tangent plane of s with that length and that angle:

$$\log_s(x) = r\, e^{\mathsf{T}\theta}, \qquad (u, v) = (r\cos\theta,\ r\sin\theta). \tag{36.1}$$

with $\log_s(x)$ written as a spinor coordinate in the frame of e_s. The pair (u, v) is what we will use as texture coordinates. (The name comes from the inverse map, the *exponential map*, which walks from s along the direction $e^{\mathsf{T}\theta}$ for the length r and lands at x.) Near s the surface is almost a plane and the map is almost an isometry: the decal is pasted almost without distortion. Further away, curvature bends the geodesics towards each other ($K > 0$) or away from each other ($K < 0$), and the decal stretches or shrinks. Past the *cut locus*, the set of points reached by two different shortest geodesics, the map is not one to one at all; we keep the decal inside it.

36.2. The key fact: the angle can be read at the far end

To compute θ directly we would have to find the geodesic from s to every x, which is exactly what the vector heat method avoids. The following fact lets us read θ at x instead, from two fields we can compute everywhere.

Step 1: parallel transport along a geodesic keeps the angle to the geodesic. Let Y be the parallel transport of e_s along γ, so that $\nabla_{\gamma'} Y = 0$. A geodesic is a curve whose velocity is itself parallel along it, $\nabla_{\gamma'}\gamma' = 0$: it does not turn within the surface. Differentiate the inner product of the two with metric compatibility (4.4) [$X(U \cdot V) = \nabla_X U \cdot V + U \cdot \nabla_X V$], with $X = \gamma'$, $U = Y$ and $V = \gamma'$:

$$\begin{aligned} \frac{d}{d\sigma}\left(Y \cdot \gamma'\right) &= \left(\nabla_{\gamma'} Y\right) \cdot \gamma' + Y \cdot \left(\nabla_{\gamma'}\gamma'\right) && \text{metric compatibility} \\ &= (0) \cdot \gamma' + Y \cdot (0) && Y \text{ parallel, } \gamma \text{ a geodesic} \\ &= 0. \end{aligned}$$

Both vectors keep their length (parallel transport and unit speed), so the angle between them is constant along γ. At $\sigma = 0$ it is the angle from $e_s = Y(0)$ to $\gamma'(0)$, which is θ. So, at the far end x,

$$\theta = \text{the angle from } Y(x) \text{ to } \gamma'(x).$$

Step 2: the direction of the geodesic at x is the gradient of the distance. The distance r grows fastest along the geodesic that arrives at x, at unit rate: moving a small step ϵ along γ' increases the distance by ϵ, and moving across it does not change it to first order (the circles of constant distance cross the geodesics from s at right angles, Gauss's lemma). So $\nabla r(x) = \gamma'(x)$, a unit vector. Substituting into Step 1:

$$\theta(x) = \text{the angle from } Y(x) \text{ to } \nabla r(x).$$

Step 3: the angle as a complex number. At a vertex both vectors are spinor coordinates of the same plane. Write them in polar form, $Y = |Y|\, e^{\mathsf{T}\alpha}$ and $R = \nabla r = |R|\, e^{\mathsf{T}\beta}$. The angle from Y to R is $\beta - \alpha$, and multiplying one by the conjugate of the other subtracts the angles:

$$\begin{aligned} \frac{R}{|R|}\,\overline{\left(\frac{Y}{|Y|}\right)} &= e^{\mathsf{T}\beta}\,\overline{e^{\mathsf{T}\alpha}} && \text{substitute the polar forms} \\ &= e^{\mathsf{T}\beta}\, e^{-\mathsf{T}\alpha} && \overline{e^{\mathsf{T}\alpha}} = e^{-\mathsf{T}\alpha} \\ &= e^{\mathsf{T}(\beta-\alpha)} = e^{\mathsf{T}\theta}. \end{aligned}$$

So

$$e^{\mathsf{T}\theta} = \frac{R\,\overline{Y}}{|R|\,|Y|}, \qquad \log_s(x) = r\, e^{\mathsf{T}\theta}. \tag{36.2}$$

The magnitude of Y does not matter, which is fortunate, because the vector heat method gives the right direction but a magnitude that decays fast with the distance (Section 5).

36.3. The distance by the heat method: the recipe once more, for scalars

We need r and its gradient. The *heat method* computes the distance with two linear solves; we derive it with the same stages as the vector problem, so that it is a second worked example of the recipe.

The idea. Heat released at s spreads, for a short time, mostly along the shortest paths, so the temperature u decreases fastest in the direction away from s: the direction of $-\nabla u$ is the direction of ∇r, although the magnitude of ∇u is not 1. The heat method therefore (a) takes one heat step, (b) keeps only the direction $X = -\nabla u/|\nabla u|$, and (c) finds the function whose gradient is as close as possible to X. Steps (a) and (c) are two scalar PDEs, each solved by the recipe.

(a) The heat step, Stages 1 to 4. The strong form is the scalar version of (5.6) [$U + \Delta t\, \nabla^* \nabla U = 0$ off s, $U(s) = U_s$]:

$$u - \Delta t\, \Delta u = 0 \quad \text{away from } s, \qquad u(s) = 1,$$

with Δ the Laplace–Beltrami operator. The function space is the scalar H^1: functions with square integrable values and gradients, a Hilbert space with $\langle u, v\rangle_1 = \int uv + \int \nabla u \cdot \nabla v$. The energy is $J(u) = \frac{1}{2}\int u^2\, dA + \frac{\Delta t}{2}\int |\nabla u|^2\, dA$. Expanding $J(u + \epsilon v)$ term by term,

$$\begin{aligned} J(u+\epsilon v) &= \frac{1}{2}\int (u+\epsilon v)^2 + \frac{\Delta t}{2}\int |\nabla u + \epsilon \nabla v|^2 \\ &= \frac{1}{2}\int \big(u^2 + 2\epsilon\, uv + \epsilon^2 v^2\big) + \frac{\Delta t}{2}\int \big(|\nabla u|^2 + 2\epsilon\, \nabla u \cdot \nabla v + \epsilon^2 |\nabla v|^2\big) \\ &= J(u) + \epsilon\Big(\int uv + \Delta t \int \nabla u \cdot \nabla v\Big) + \frac{\epsilon^2}{2}\Big(\int v^2 + \Delta t \int |\nabla v|^2\Big), \end{aligned}$$

and the coefficient of ϵ gives the weak form: find u with $u(s) = 1$ such that

$$\int_\Omega u\, v\, dA + \Delta t \int_\Omega \nabla u \cdot \nabla v\, dA = 0 \qquad \text{for every } v \text{ with } v(s) = 0. \tag{36.3}$$

(Integrating the second term by parts, with the scalar Green identity, gives back $\int (u - \Delta t\, \Delta u)\, v = 0$, the strong form.)

(a) The heat step, Stages 5 to 8. No connection is needed: a scalar is a scalar at every vertex. The discrete space is spanned by the hat functions, $u_h = \sum_j u_j \phi_j$, and the test functions are the hats ϕ_i of the vertices $i \neq s$. Substitute both into (36.3):

$$\begin{aligned} 0 &= \int \Big(\sum_j u_j \phi_j\Big)\phi_i\, dA + \Delta t \int \nabla\Big(\sum_j u_j \phi_j\Big) \cdot \nabla \phi_i\, dA && \text{substitute} \\ &= \sum_j u_j \int \phi_j \phi_i\, dA + \Delta t \sum_j u_j \int \nabla \phi_j \cdot \nabla \phi_i\, dA && \text{linearity} \\ &= \sum_j \big(\mathrm{M}_{ij} + \Delta t\, \mathrm{K}_{ij}\big)\, u_j && \text{name the integrals,} \end{aligned}$$

with $\mathrm{K}_{ij} = \int \nabla\phi_j \cdot \nabla\phi_i \, dA$ and the lumped mass $\mathrm{M}_{ii} = \frac{1}{3}\sum_{f \ni i} A_f$ of (13.3) [$\mathrm{M}^f_{aa} = \frac{A_f}{3} I_2$], here for scalars. The stiffness K is assembled from the element matrices $A_f \, \nabla\phi_a \cdot \nabla\phi_b$, the scalar factor of (32.3) [$\mathrm{L}^f_{ij} = A_f (\nabla\phi_i \cdot \nabla\phi_j)\, e^{\mathsf{T}(\rho_j^f - \rho_i^f)}$] without the phase. The row of s is replaced by $u_s = 1$, as in Stage 8. The linear system is

$$\big(\mathrm{M} + \Delta t\, \mathrm{K}\big)\, \mathrm{u} = 0 \quad (\text{rows } i \neq s), \qquad u_s = 1.$$

(b) The direction. On each face u_h is linear, so its gradient is constant there: $\nabla u_h|_f = \sum_{a \in f} u_a \nabla\phi_a$, with the gradients of the layout, (29.6) [$\nabla\phi_a = \frac{J(\bar{x}_c - \bar{x}_b)}{2A_f}$]. The unit field $X_f = -\nabla u_h|_f / |\nabla u_h|_f|$ is constant on each face.

(c) The distance, Stages 1 to 4. We look for the function r whose gradient is closest to X in the least-squares sense:

$$E(r) = \frac{1}{2}\int_\Omega |\nabla r - X|^2 \, dA \quad \longrightarrow \quad \text{minimum.}$$

This is a variational form directly; its strong form is the Poisson equation $\Delta r = \operatorname{div} X$. Expand $E(r + \epsilon v)$:

$$\begin{aligned} E(r+\epsilon v) &= \frac{1}{2}\int |(\nabla r - X) + \epsilon \nabla v|^2 && \nabla \text{ is linear} \\ &= \frac{1}{2}\int \big(|\nabla r - X|^2 + 2\epsilon\,(\nabla r - X)\cdot\nabla v + \epsilon^2 |\nabla v|^2\big) && \text{expand} \\ &= E(r) + \epsilon \int (\nabla r - X)\cdot \nabla v + \frac{\epsilon^2}{2}\int |\nabla v|^2, \end{aligned}$$

so the weak form, the vanishing of the coefficient of ϵ, is

$$\int_\Omega \nabla r \cdot \nabla v \, dA = \int_\Omega X \cdot \nabla v \, dA \qquad \text{for every } v \in H^1. \tag{36.4}$$

A function-space subtlety. On a closed surface, adding a constant c to r does not change ∇r, so E cannot tell r from $r + c$: the bilinear form $a(r, v) = \int \nabla r \cdot \nabla v$ is not coercive on H^1 (it vanishes for $r = v = c$), and Lax–Milgram does not apply as it stands. It applies on the subspace of functions with $r(s) = 0$, or with zero mean, where a is coercive. The equation also needs a compatibility condition: with $v = 1$, the left side of (36.4) is $\int \nabla r \cdot 0 = 0$, so the right side must vanish too, and it does, $\int X \cdot \nabla 1 = 0$. We fix the constant by asking $r(s) = 0$, which is the meaning of a distance.

(c) The distance, Stages 5 to 8. Substitute $r_h = \sum_j r_j \phi_j$ and $v = \phi_i$ into (36.4):

$$\begin{aligned} \int \nabla\Big(\sum_j r_j \phi_j\Big)\cdot\nabla\phi_i \, dA &= \int X \cdot \nabla\phi_i \, dA && \text{substitute} \\ \sum_j r_j \int \nabla\phi_j \cdot \nabla\phi_i \, dA &= \sum_f \int_f X_f \cdot \nabla\phi_i \, dA && \text{linearity; by faces} \\ \sum_j \mathrm{K}_{ij}\, r_j &= \sum_{f \ni i} A_f\, X_f \cdot \nabla\phi_i|_f =: b_i && \text{constants on } f. \end{aligned}$$

In matrix form $\mathrm{K}\,\mathrm{r} = \mathrm{b}$. The discrete version of the subtlety above is visible: K is singular, because its rows add up to zero ($\sum_j \mathrm{K}_{ij} = \int \nabla(\sum_j \phi_j)\cdot\nabla\phi_i = \int \nabla 1 \cdot \nabla\phi_i = 0$), and b is compatible, $\sum_i b_i = \sum_f A_f X_f \cdot \sum_{a\in f} \nabla\phi_a = 0$ because the three gradients of a face add up to zero. Replacing the row of s by $r_s = 0$ removes the singularity and fixes the constant.

In the code. One function assembles both scalar systems: with the mass and a time step for the heat step, without them for the Poisson solve. The row of the pinned vertex is skipped in the loop and replaced by a 1 on the diagonal:

```
90  /// The scalar stiffness K_ab = sum_f A_f grad phi_a . grad phi_b, with the rows of the
        pinned vertex replaced.
91  static Eigen::VectorXd scalarSolve(Mesh& mesh, const intrinsic::Geometry& G, double dt
        , bool withMass, int pinned,
92                                     double pinnedValue, const Eigen::VectorXd& rhs) {
93      const int n = mesh.numVertices();
94      std::vector<Eigen::Triplet<double>> T;
95      for (Face& f : mesh.getFaces()) {
96          Vertex* v[3];
97          Edge* h[3];
98          intrinsic::faceCorners(f, v, h);
99          const intrinsic::FaceGeometry& g = G.faces[f.ID];
100         for (int a = 0; a < 3; ++a) {
101             if (v[a]->ID == pinned) continue;
102             if (withMass) T.emplace_back(v[a]->ID, v[a]->ID, g.area / 3.0);
103             for (int b = 0; b < 3; ++b) T.emplace_back(v[a]->ID, v[b]->ID, dt * g.area
                    * g.grad[a].dot(g.grad[b]));
104         }
105     }
106     T.emplace_back(pinned, pinned, 1.0);
107     Eigen::SparseMatrix<double> A(n, n);
108     A.setFromTriplets(T.begin(), T.end());
109     Eigen::VectorXd b = rhs;
110     b(pinned) = pinnedValue;
111     Eigen::SparseLU<Eigen::SparseMatrix<double>> lu(A);
112     return lu.solve(b);
113 }
```

Line 101 skips the row of the pinned vertex; line 102 is the lumped mass $A_f/3$ of the row; line 103 adds $\Delta t\, A_f\, \nabla\phi_a\cdot\nabla\phi_b$, the scalar element of Stage 7, entry by entry; line 106 is the replaced row; `b(pinned)` on line 110 holds the prescribed value, 1 for the heat step and 0 for the distance. The gradient of a piecewise linear function on a face is the sum $\sum_a u_a \nabla\phi_a$ of (b):

```
115 /// The gradient of a piecewise linear function on face f, in the layout (face) frame.
116 static Vector2d faceGradient(Mesh& mesh, const intrinsic::Geometry& G, const Face& f,
        const Eigen::VectorXd& u) {
117     Vertex* v[3];
118     Edge* h[3];
119     intrinsic::faceCorners(f, v, h);
120     const intrinsic::FaceGeometry& g = G.faces[f.ID];
121     return u(v[0]->ID) * g.grad[0] + u(v[1]->ID) * g.grad[1] + u(v[2]->ID) * g.grad
            [2];
122 }
```

and the heat method is the three steps (a), (b), (c) in order:

```
124 /// The heat method for geodesic distance: one scalar heat step from the source, the
        unit vector field X = -grad u /
125 /// |grad u| on the faces, and the function whose gradient is closest to X, K phi = b
        with b_a = sum_f A_f grad phi_a . X.
126 static Eigen::VectorXd heatDistance(Mesh& mesh, const intrinsic::Geometry& G, double
        dt, int source) {
```

```
127     const int n = mesh.numVertices();
128     const Eigen::VectorXd u = scalarSolve(mesh, G, dt, true, source, 1.0, Eigen::
            VectorXd::Zero(n));
129     Eigen::VectorXd b = Eigen::VectorXd::Zero(n);
130     for (Face& f : mesh.getFaces()) {
131         const Vector2d gu = faceGradient(mesh, G, f, u);
132         const Vector2d X = -gu / std::max(gu.norm(), 1e-300);
133         Vertex* v[3];
134         Edge* h[3];
135         intrinsic::faceCorners(f, v, h);
136         for (int a = 0; a < 3; ++a) b(v[a]->ID) += G.faces[f.ID].area * G.faces[f.ID].
                grad[a].dot(X);
137     }
138     return scalarSolve(mesh, G, 1.0, false, source, 0.0, b);
139 }
```

Line 128 is the heat step (with mass, time step Δt, source value 1); lines 131–132 form X_f; line 136 adds $A_f \nabla\phi_a \cdot X_f$ to b_a, the right side derived above; line 138 is the Poisson solve, without mass, with the factor 1 in place of Δt, and $r_s = 0$.

36.4. The direction: the vector heat step

The transported source direction Y is the solution of Stages 6 to 8, with the value $\psi_s = 1$ (the reference direction e_s) at the source:

```
61  /// One backward Euler step of the vector heat equation, (M + dt L) psi = 0 away from
        the source and psi = 1 at the
62  /// source: the reference direction of the source, carried to every vertex (Stages 6 to
        8 of the text).
63  static Eigen::VectorXcd vectorHeat(Mesh& mesh, const intrinsic::Geometry& G, const
        Corners& rho, double dt, int source) {
64      const int n = mesh.numVertices();
65      std::vector<Eigen::Triplet<Complex>> T;
66      std::vector<double> mass(n, 0.0);
67      for (Face& f : mesh.getFaces()) {
68          Vertex* v[3];
69          Edge* h[3];
70          intrinsic::faceCorners(f, v, h);
71          const intrinsic::FaceGeometry& g = G.faces[f.ID];
72          for (int a = 0; a < 3; ++a) {
73              mass[v[a]->ID] += g.area / 3.0;
74              if (v[a]->ID == source) continue;
75              for (int b = 0; b < 3; ++b) {
76                  const Complex transport = a == b ? Complex(1.0) : rho[f.ID][b] * std::
                        conj(rho[f.ID][a]);
77                  T.emplace_back(v[a]->ID, v[b]->ID, dt * g.area * g.grad[a].dot(g.grad[b
                        ]) * transport);
78              }
79          }
80      }
81      for (int i = 0; i < n; ++i) T.emplace_back(i, i, i == source ? 1.0 : mass[i]);
82      Eigen::SparseMatrix<Complex> A(n, n);
83      A.setFromTriplets(T.begin(), T.end());
84      Eigen::VectorXcd rhs = Eigen::VectorXcd::Zero(n);
85      rhs(source) = 1.0;
86      Eigen::SparseLU<Eigen::SparseMatrix<Complex>> lu(A);
87      return lu.solve(rhs);
88  }
```

It is the system (32.3) $[\,\mathrm{L}^f_{ij} = A_f(\nabla\phi_i \cdot \nabla\phi_j)\, e^{\mathrm{T}(\rho^f_j - \rho^f_i)}\,]$ assembled exactly as in `intrinsicform::solve`, with one difference of form: the rows are $(\mathrm{M} + \Delta t\,\mathrm{L})\,\psi = 0$, not divided by the masses. Dividing a row by a positive number

does not change its solution, so the two systems have the same Y. Line 76 is the phase $e^{\mathsf{T}(\rho_b^f - \rho_a^f)}$, the transport from b to a through the face; line 81 puts the mass on the diagonal, and the 1 of the source row.

36.5. The angle at the vertices

Formula (36.2) $[\, e^{\mathsf{T}\theta} = R\overline{Y}/(|R||Y|)\,]$ needs R and Y in the same plane. A vertex has no tangent plane (Section 30), but each face around it is a plane, and the method already knows how to carry the vertex value into each face: multiply by the corner connection $e^{\mathsf{T}\rho_a^f}$. So in each corner (i, f) we compare, in the face frame of f,

$$R_f = \nabla r_h|_f \quad \text{(constant on } f\text{)}, \qquad y_{i,f} = e^{\mathsf{T}\rho_i^f}\, Y_i,$$

which gives one estimate $e^{\mathsf{T}\theta_{i,f}} = R_f\,\overline{y_{i,f}}/(|R_f||y_{i,f}|)$ per corner. The faces around i give slightly different estimates (the gradient is constant on each face, and the faces are tilted), and we average them as unit complex numbers, each weighted by the angle $\alpha_{i,f}$ of its corner:

$$e^{\mathsf{T}\theta_i} = \frac{\sum_{f\ni i} \alpha_{i,f}\, e^{\mathsf{T}\theta_{i,f}}}{\left|\sum_{f\ni i} \alpha_{i,f}\, e^{\mathsf{T}\theta_{i,f}}\right|}, \qquad \log_s(x_i) = r_i\, e^{\mathsf{T}\theta_i}.$$

Averaging the unit complex numbers, rather than the angles, avoids the jump of the angle at $\pm\pi$. In the code:

```
141 /// The logarithmic map: at every vertex, r e^{T theta}, where theta is the angle from
        the transported source direction
142 /// Y to the direction in which r grows, R. Both are compared inside each face around
        the vertex (Y carried into the
143 /// face with the corner connection), and the corners are averaged with their angles as
         weights.
144 static std::vector<Vector2d> logMap(Mesh& mesh, const intrinsic::Geometry& G, const
      Corners& rho, const Eigen::VectorXcd& Y,
145                                     const Eigen::VectorXd& r, const std::function<Complex(
                                            const Face&, int)>& radial) {
146     const int n = mesh.numVertices();
147     std::vector<Complex> sum(n, 0.0);
148     for (Face& f : mesh.getFaces()) {
149         Vertex* v[3];
150         Edge* h[3];
151         intrinsic::faceCorners(f, v, h);
152         for (int a = 0; a < 3; ++a) {
153             const Complex R = radial(f, a), y = rho[f.ID][a] * Y(v[a]->ID);
154             if (std::abs(R) < 1e-14 || std::abs(y) < 1e-300) continue;
155             sum[v[a]->ID] += G.faces[f.ID].angle[a] * (R / std::abs(R)) * std::conj(y /
                    std::abs(y));
156         }
157     }
158     std::vector<Vector2d> uv(n);
159     for (int i = 0; i < n; ++i) {
160         const double theta = std::abs(sum[i]) > 0 ? std::arg(sum[i]) : 0.0;
161         uv[i] = r(i) * Vector2d(std::cos(theta), std::sin(theta));
162     }
163     return uv;
164 }
```

Line 153 forms R and $y = e^{\mathsf{T}\rho_a^f} Y_i$; line 155 adds $\alpha\,(R/|R|)\,\overline{(y/|y|)}$; lines 160–161 take the angle of the sum and build $r_i(\cos\theta_i, \sin\theta_i)$, the texture coordinates of (36.1) $[\,(u, v) = (r\cos\theta, r\sin\theta)\,]$. The reference direction e_s is the

reference edge of s, an arbitrary choice; for the picture we turn all the coordinates by one angle so that e_s points up the trunk, which changes nothing else:

```
/// The logarithmic map around ‘source‘ with the intrinsic connection, turned so that the source direction points up
/// (along e_z projected into the surface). Returns the texture coordinates, and the distances in r.
static std::vector<Vector2d> uprightLogMap(Mesh& mesh, const intrinsic::Geometry& G, const Corners& rho, double h,
                                           int source, Eigen::VectorXd& r) {
    const Eigen::VectorXcd Y = vectorHeat(mesh, G, rho, h * h, source);
    r = heatDistance(mesh, G, h * h, source);
    auto radial = [&](const Face& f, int) {
        const Vector2d g = faceGradient(mesh, G, f, r);
        return Complex(g.x(), g.y());
    };
    std::vector<Vector2d> uv = logMap(mesh, G, rho, Y, r, radial);
    // turn the decal upright: rotate the texture coordinates so that the source direction (the red arrow) points up
    // the trunk. In the first face of the source, the source direction is the face vector at the angle arg(rho), and
    // "up" is e_z projected into the face.
    {
        const Edge* e = mesh.vertexAt(source).edge;
        const Face& f = *e->face;
        Vertex* v[3];
        Edge* hh[3];
        intrinsic::faceCorners(f, v, hh);
        int a = 0;
        while (hh[a] != e) ++a;
        const Vector3d nf = (v[1]->p - v[0]->p).cross(v[2]->p - v[0]->p).normalized();
        const Vector3d tf = (v[1]->p - v[0]->p).normalized(), bf = nf.cross(tf);
        const Vector3d up = Vector3d(0, 0, 1) - nf.z() * nf;
        const double alpha = std::atan2(up.dot(bf), up.dot(tf)) - std::arg(rho[f.ID][a]);
        const Eigen::Rotation2Dd turn(-alpha);
        for (Vector2d& t : uv) t = turn * t;
    }
    return uv;
}
```

36.6. A decal on the cactus

The pictures of this section are rendered from the coordinates computed above by a small path tracer, `showcase_render.cpp`, which includes `showcase.cpp` unchanged; the pot and the studio light are decoration.

Figure 29 shows the result on a smooth cactus of 5261 vertices: a checkerboard with two arrows, the red one along e_s and the blue one a quarter turn to its left, $e^{\mathsf{T}\pi/2}e_s$. Near the source the squares are square; away from it they bend, following the curvature of the trunk, but the checkerboard stays continuous and keeps its orientation and handedness everywhere inside the decal.

Why a smooth mesh. On the coarse cactus of 502 vertices used in the experiments, a few vertices are sharp cones, with angle defects up to 63.7°. A cone of positive defect κ_v behaves like a small lens: the geodesics from s that pass on either side of it are bent towards each other by the total angle κ_v, and behind the cone they cross. Points behind it are reached by two geodesics, and the exact logarithmic map folds over there: some triangles are mapped with



FIGURE 29. A decal pasted with the logarithmic map, $(u, v) = r(\cos\theta, \sin\theta)$, with r from the heat method and θ from the vector heat method with the intrinsic connection. The red arrow is the source direction, the blue one a quarter turn to its left.

the wrong orientation. We counted 29 such flipped triangles inside a decal of radius 0.17 on the coarse cactus refined by midpoint subdivision (which keeps the cones), against 2 after Loop subdivision (which rounds them off). With the smooth cactus the largest defect is small, and the decal is clean. The renderer leaves untextured any face whose map is stretched more than four times in area, a sign of the cut locus:

```
280 /// Texture coordinates of one face, with the guard: where the map is stretched more
        than 4 times in area (past the
281 /// cut locus of the logarithmic map) the face gets no texture, signalled by
        coordinates far outside any decal.
282 static Vector2d texCoord(const std::vector<Vector2d>& uv, Vertex* const v[3], const
        double l[3], bool guard) {
283     if (guard) {
284         const Vector3d cr = (v[1]->p - v[0]->p).cross(v[2]->p - v[0]->p);
285         const Vector2d du = uv[v[1]->ID] - uv[v[0]->ID], dv = uv[v[2]->ID] - uv[v[0]->
              ID];
286         if (std::abs(du.x() * dv.y() - du.y() * dv.x()) > 4.0 * cr.norm()) return
              Vector2d(1e9, 1e9);
287     }
288     return l[0] * uv[v[0]->ID] + l[1] * uv[v[1]->ID] + l[2] * uv[v[2]->ID];
289 }
```

The whole cactus. Nothing stops us from using one logarithmic map for the whole surface: drop the radius of the decal and paint the checkerboard everywhere (Figure 30, from a source on the front of the trunk). On the front, around the source, the squares are almost square, as in the decal. Away from it they shear and stretch, because the geodesics from s spread apart around the trunk and are bent by the curvature of the arms. On the back, the geodesics that went around the trunk on the left meet those that went around on the right. That line is the cut locus: across it, θ jumps from about $+\pi$ to about $-\pi$, and the map is no longer one to one. The faces that

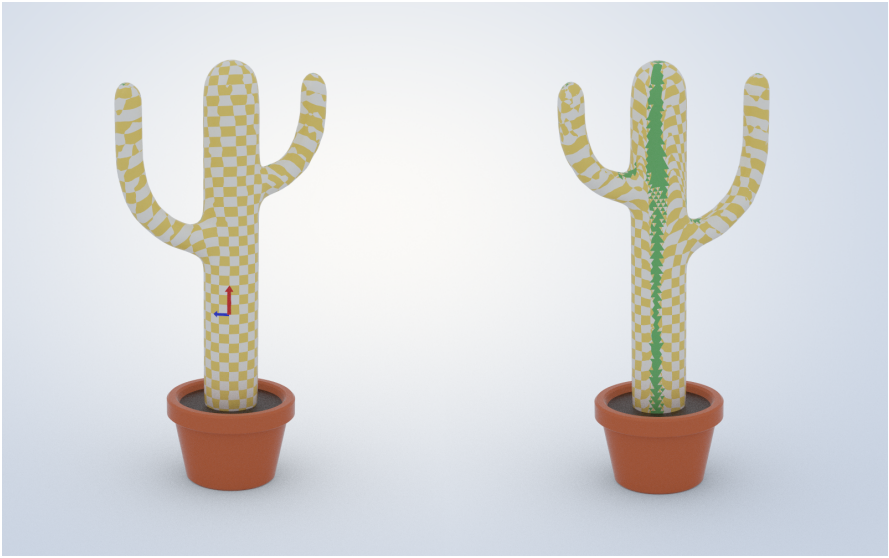


FIGURE 30. One logarithmic map over the whole cactus. Left: the front, with the source and its two arrows. Right: the back; the green seam is the cut locus, where the geodesics that went around the two sides of the trunk meet and the map stops being one to one.

straddle it are stretched past the guard and stay green, so the green seam down the back of the trunk is the cut locus, made visible by the method itself. One chart cannot cover a closed surface; a decal must stay on its own side of the cut locus.

An atlas of charts. The cure is the one of cartography: several charts, each used only near its own source. We take eight sources, one on the front and one on the back of the lower trunk, of the upper trunk and of each arm, compute the eight logarithmic maps, and give each point the chart of its geodesically nearest source. The boundary between the charts of two sources s_1 and s_2 is then the set of points equally far from both, $r_1 = r_2$; on a part shaped like a cylinder, with one source on each side, it runs along the sides of the part, a quarter turn from each source, while the cut locus of each chart is half a turn away, on the other side. So no chart is used up to its cut locus. Each chart is already turned upright (its red arrow along e_z), so two neighbouring checkerboards have nearly the same orientation; only their phases differ, and we remove the mean difference by translating each chart: on the vertices near the boundary with the charts placed before it, where r_k is within three edge lengths of the nearest earlier r_j, we compute the mean of $\log_{s_j}(x_i) - \log_{s_k}(x_i)$ and add it to all the coordinates of chart k. The result is Figure 31: squares of nearly the same size everywhere, and boundaries that are visible only as small shifts of the pattern, mostly at the saddles where the arms meet the

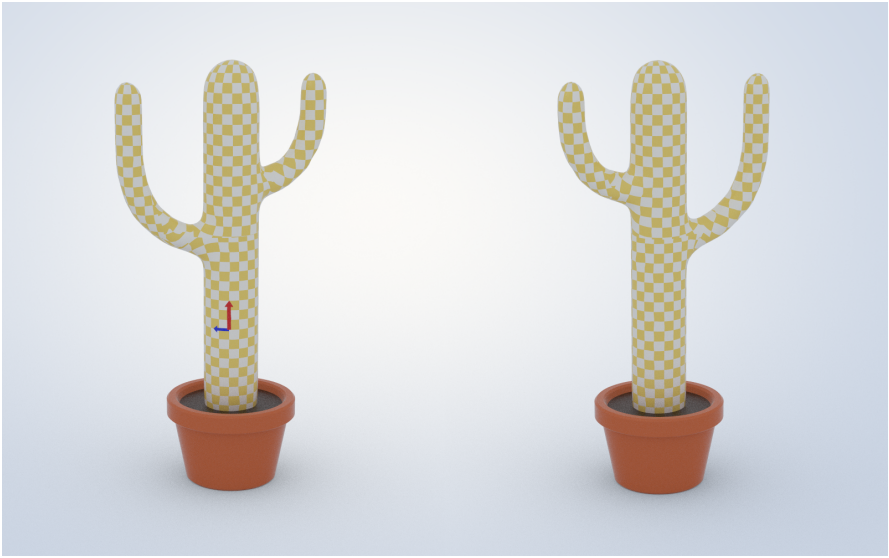


FIGURE 31. The checkerboard of Figure 30 with an atlas of eight logarithmic maps (front and back of the lower trunk, the upper trunk and each arm), each point painted by the chart of its nearest source, and the charts translated to match along their boundaries. The red and blue arrows mark the first chart.

trunk. There the curvature is negative, and no flat chart, and no stitching, can keep every square a square.

36.7. Graph paper on the folded sheet: extrinsic against intrinsic

The Miura-ori of Figure 22 is intrinsically flat, so its exact logarithmic map is simply its unfolding: lay the faces flat in the plane, one after another across their common edges, and read the plane coordinates relative to s. This gives us the exact r and the exact ∇r in every face, so that we can compute the angle (36.2) with the exact R and isolate the only remaining ingredient, the transported direction Y, that is, the connection. The unfolding places the first face by its layout, and each next face by the rigid motion that puts its shared edge on the already placed one:

```
166 /// An intrinsically flat mesh laid out in the plane, face after face across the edges:
        the exact unfolding. Returns
167 /// the plane position of every vertex, and for every face the angle of the rotation
        from its layout into the plane.
168 static std::vector<Vector2d> unfold(Mesh& mesh, const intrinsic::Geometry& G, std::
        vector<double>& faceRotation) {
169     std::vector<Vector2d> p(mesh.numVertices(), Vector2d::Zero());
170     faceRotation.assign(mesh.numFaces(), 0.0);
171     std::vector<bool> done(mesh.numFaces(), false);
172     std::vector<int> queue{ 0 };
173     {
174         Vertex* v[3];
```

```
175         Edge* h[3];
176         intrinsic::faceCorners(mesh.faceAt(0), v, h);
177         for (int a = 0; a < 3; ++a) p[v[a]->ID] = G.faces[0].x[a];
178     }
179     done[0] = true;
180     for (size_t k = 0; k < queue.size(); ++k) {
181         Face& f = mesh.faceAt(queue[k]);
182         Vertex* v[3];
183         Edge* h[3];
184         intrinsic::faceCorners(f, v, h);
185         for (int a = 0; a < 3; ++a) {
186             const Face* g = h[a]->pair->face;
187             if (!g || done[g->ID]) continue;
188             Vertex* w[3];
189             Edge* hg[3];
190             intrinsic::faceCorners(*g, w, hg);
191             int i = 0;
192             while (hg[i] != h[a]->pair) ++i; // hg[i] goes from w[i] = v[a+1] to w[i+1]
                    = v[a]
193             const intrinsic::FaceGeometry& lg = G.faces[g->ID];
194             const Vector2d A = p[w[i]->ID], B = p[w[(i + 1) % 3]->ID];
195             const Vector2d dl = lg.x[(i + 1) % 3] - lg.x[i], dp = B - A;
196             const double gamma = std::atan2(dp.y(), dp.x()) - std::atan2(dl.y(), dl.x()
                    );
197             const Eigen::Rotation2Dd Rg(gamma);
198             p[w[(i + 2) % 3]->ID] = A + Rg * (lg.x[(i + 2) % 3] - lg.x[i]);
199             faceRotation[g->ID] = gamma;
200             done[g->ID] = true;
201             queue.push_back(g->ID);
202         }
203     }
204     return p;
205 }
```

Line 196 finds the rotation angle γ that turns the layout of g so that its copy of the shared edge has the direction of the placed one; line 198 places the third vertex, and line 199 keeps γ. The angle γ is kept for each face, because the exact ∇r in the face frame is the plane direction $p_i - p_s$ turned back by $-\gamma$.

We then compute the logarithmic map twice, with the corner connection of Part II (rolling into the plane of the vertex normal) and with that of Part III (the bisector rule), and render graph paper with it (Figure 32). If the map is exact, the lines of the paper are straight on the unfolded sheet, so on the folded sheet they run straight across every fold. The program measures, at every vertex, the angle between the computed coordinates and the exact unfolded ones:

connection	mean angle error	largest angle error
extrinsic, Part II	$2.2°$	$14.8°$
intrinsic, Part III	$6.6 \cdot 10^{-14}$ degrees	$2.7 \cdot 10^{-13}$ degrees

The intrinsic result is exact for the reason found in Section 31: at a flat vertex, $s_i = 1$, the bisector rule (31.1) $[\,\rho_i^f = (\beta_e + \frac{\alpha}{2}) - s_i(\theta_e + \frac{\alpha}{2})\,]$ becomes $\rho_i^f = \beta_e - \theta_e$, which is the exact unfolding, so every transport the method makes is exact and so is Y. The extrinsic connection rolls the four faces of a Miura vertex into the plane of the vertex normal about four different axes; the rolled corners do not fit edge to edge, every transport is slightly wrong, and the errors add up along the paths from the source.

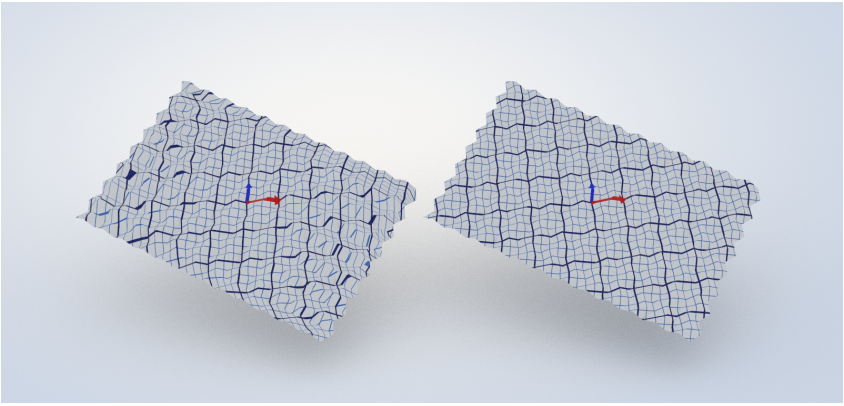


FIGURE 32. Graph paper on the folded Miura sheet, pasted with the logarithmic map from the center. Left: the extrinsic connection of Part II; the lines break at the folds, and the angle is wrong by 2.2° on average and up to 14.8°. Right: the intrinsic connection of Part III; the lines run straight across every fold, exactly as on the flat paper, with angle errors below $3 \cdot 10^{-13}$ degrees.

What the application shows. The finite element machine of this text is reusable: the distance needed one more pass of the recipe, for scalars, with a variational problem (least squares for the gradient) instead of a heat step, and with one function-space subtlety (the constant in the kernel, fixed by $r(s) = 0$). And the difference between the two ways of giving a vertex a plane, which in the experiments of Section 33 was a column of small numbers, becomes something one can see.

References

[1] M. P. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1976.

[2] C. Doran and A. Lasenby. *Geometric Algebra for Physicists*. Cambridge University Press, 2003.

[3] G. Dziuk and C. M. Elliott. Finite element methods for surface PDEs. *Acta Numerica*, 22:289–396, 2013.

[4] P. Hansbo, M. G. Larson, and K. Larsson. Analysis of finite element methods for vector Laplacians on surfaces. *IMA Journal of Numerical Analysis*, 40(3):1652–1701, 2020.

[5] D. Hestenes and G. Sobczyk. *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. D. Reidel, 1984.

[6] F. Knöppel, K. Crane, U. Pinkall, and P. Schröder. Globally optimal direction fields. *ACM Transactions on Graphics*, 32(4):59:1–59:10, 2013.

[7] H. P. Langtangen. Introduction to finite element methods. 2014.

[8] B. Liu, Y. Tong, F. De Goes, and M. Desbrun. Discrete connection and covariant derivative for vector field analysis and design. *ACM Transactions on Graphics*, 35(3):23:1–23:17, 2016.

[9] M. C. Lopez Belon. Project on github. Available at https://github.com/mauriciocele/basic-fem-connection-laplacian-degoes, 2024.

[10] F.-J. Sayas. A gentle introduction to the finite element method. 2015.

[11] N. Sharp, Y. Soliman, and K. Crane. The vector heat method. *ACM Transactions on Graphics*, 38(3):24:1–24:19, 2019.

Mauricio Cele Lopez Belon
Madrid, Spain
e-mail: `mclopez@outlook.com`